

THOMSON



INGENIERÍA DE

SOFTWARE

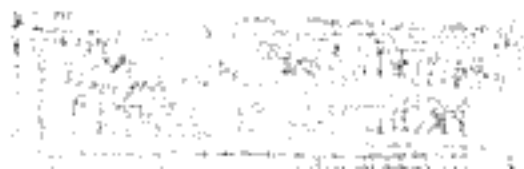
ORIENTADA

A OBJETOS

CON UML. JAVA

E INTERNET

Alfredo Weitzenfeld



Ingeniería de Software Orientada a Objetos con UML, Java e Internet

This One



AAUO-Z5B-9PJS

**MATERIAL PROMOCIONAL
PROHIBIDA SU VENTA**

United material



Ingeniería de Software Orientada a Objetos con UML, Java e Internet

Alfredo Weitzenfeld

THOMSON



Australia • Brasil • Canadá • España • Estados Unidos • México • Reino Unido • Singapur

THOMSON



Ingeniería de Software Orientada a Objetos con UML, Java e Internet
Alfredo Weitzenfeld

Director general:
Miguel A. Toledo Castellanos

Director editorial y de producción:
José Tomás Pérez Bonilla

Editor de desarrollo:
Rocío Cabañas Chávez

Gerente de producción:
René Garay Argueta

Editor de producción:
Alejandro A. Gómez Ruiz

Supervisora de manufactura:
Claudia Calderón Valderrama

Diseño de portada:
Maré, Concepto Gráfico

COPYRIGHT © 2005 por International Thomson Editores, S.A. de C.V., una división de Thomson Learning, Inc. Thomson Learning™ es una marca registrada usada bajo permiso.

Impreso en México
Printed in Mexico
1 2 3 4 07 06 05

Para mayor información contáctenos en:
Séneca núm. 53
Col. Polanco
México, D.F., 11560

Puede visitar nuestro sitio en
<http://www.thomsonlearning.com.mx>

DERECHOS RESERVADOS. Queda prohibida la reproducción o transmisión total o parcial del texto de la presente obra bajo cualesquiera formas, electrónica o mecánica, incluyendo fotocopiado, almacenamiento en algún sistema de recuperación de información, o grabado sin el consentimiento previo y por escrito del editor.

Datos para catalogación bibliográfica:
Weitzenfeld, Alfredo
Ingeniería de Software Orientada a Objetos con UML, Java e Internet

ISBN 970-686-190-4
I. Introducción. 1. Costo y complejidad del software. 2. Tecnología orientada a objetos. 3. Proceso de software. II. Modelado y programación orientada a objetos. 4. Modelado con UML. 5. Programación orientada a objetos con Java. III. Desarrollo de software orientado a objetos. 6. Modelo de requisitos. 7. Modelo de análisis. 8. Modelo de diseño. 9. Modelo de implementación. 10. Modelo de pruebas. IV. Programación y desarrollo de software para Internet. 11. Programación con HTML, Servlets y JSP. 12. Desarrollo de software para Internet.

División Iberoamericana

México y América Central
Thomson Learning
Séneca núm. 53
Col. Polanco
México, D.F., 11560
Tel. 52 (55) 1500 6000
Fax 52 (55) 5281 2858
editor@thomsonlearning.com.mx

El Caribe
Thomson Learning
59B Aldebaran St.
00920, Altamira
San Juan, Puerto Rico
Tel. (787) 641 1112
Fax (787) 641 1119

Cono Sur
Buenos Aires, Argentina
thomson@thomsonlearning.com.ar

América del Sur
Thomson Learning
Calle 39 núm. 24-09
La Soledad
Bogotá, Colombia
Tel. (571) 340 9470
Fax (571) 340 9475
cliente@thomsonlearning.com.co

España
Paraninfo Thomson Learning
Calle Magallanes núm. 25
28015 Madrid
España
Tel. 34 (0) 91 446 3350
Fax 34 (0) 91 445 8218
clientes@paraninfo.es

impreso en publimex, s.a.
calz. san lorenzo 279-32
col. estrella iztapalapa

SEMBLANZA DEL AUTOR



Alfredo Weitzenfeld es doctor en ciencia de la computación y maestro en ingeniería en computación, ambos por la Universidad del Sur de California (USC), Los Ángeles, California, es ingeniero electrónico por el Instituto Tecnológico de Israel, Technion, Haifa, Israel.

El doctor Weitzenfeld tiene más de 20 años de experiencia en las áreas de ingeniería de software, inteligencia artificial, redes neuronales y robótica. Ha trabajado profesionalmente en México, Estados Unidos, Israel y Colombia. Ha sido profesor del Departamento de Ciencia de la Computación de la USC y actualmente es profesor del Departamento de Ingeniería en Computación del ITAM, en México. Es director del laboratorio de Robótica y del laboratorio de Modelado y Simulación Neuronal (CANNES) en el ITAM. También dirige el equipo Eagle Knights del mismo instituto, cuyos robots participan en diversas competencias internacionales de fútbol, RoboCup, en varias categorías.

El doctor Weitzenfeld es miembro del Sistema Nacional de Investigadores (SNI) en México. Ha participado en diversos proyectos de desarrollo tecnológico e investigación financiados por el CONACyT (Consejo Nacional de Ciencia y Tecnología) en México y NSF (National Science Foundation) en Estados Unidos, así como el programa UC MEXUS entre California y México.

El doctor Weitzenfeld es el autor principal del libro *The Neural Simulation Language NSL: A System for Brain Modeling* (coautores M. Arbib y A. Alexander) publicado en 2002 por MIT Press. Ha impartido diversos cursos en el área de la computación a nivel licenciatura, maestría y extensión universitaria. Ha publicado más de 40 artículos en diferentes revistas y ha participado en innumerables conferencias internacionales.

PREFACIO

Este libro es el producto de casi una década de investigación y docencia a nivel licenciatura, maestría y cursos especiales, en el área de la tecnología orientada a objetos. Aunque en la actualidad existe un número cada vez mayor de libros de texto, en especial en inglés, relacionados con la ingeniería de software orientada a objetos, un objetivo importante de esta obra es su enfoque, que cubre de manera integral tanto los aspectos teóricos como los prácticos. En particular se busca que el lector aprenda cómo desarrollar sistemas de calidad mediante la creación de una arquitectura de software y el seguimiento de una metodología bien definida. Por tal razón, una parte importante del libro se basa en un sistema comprensivo de software, en lugar de pequeños ejemplos aislados. Otra motivación para la escritura de este libro es reducir la brecha existente entre el número de textos en inglés y en español. Considero que este libro contribuye en este aspecto, pues lamentablemente la mayoría de los libros en áreas tecnológicas son, cuando existen, traducciones de originales en inglés.

Los lectores hacia quienes va dirigido este libro son tanto estudiantes universitarios como profesionistas en el área del desarrollo de software orientado a objetos. Como texto, esta obra tiene como objetivo cubrir las necesidades de un curso de licenciatura o maestría, con programas semestrales o trimestrales, de manera completa o parcial, pues es una introducción a la ingeniería de software con tecnología orientada a objetos. El título particular de este libro, "Ingeniería de software orientada a objetos con UML, Java e Internet", busca resaltar ciertos aspectos que en la actualidad tienen un significado muy especial. El Lenguaje de Modelado Unificado (UML por las siglas de Unified Modeling Language) es el estándar más importante a nivel mundial para el modelado de sistemas orientados a objetos. El lenguaje de programación de Java es uno de los más utilizados en el desarrollo de aplicaciones orientadas a objetos. Por último, Internet es en la actualidad uno de los ambientes más importantes para la creación de software por su fácil acceso desde cualquier lugar del planeta.

El libro cubre los temas más relevantes para un curso de ingeniería de software basada en la tecnología orientada a objetos. Se divide en cuatro secciones principales:

Parte	Capítulos	Tema	Descripción
I	1, 2, 3	Introducción	Se presentan las razones de la necesidad de la ingeniería de software y la tecnología orientada a objetos. Se describen los modelos de procesos de desarrollo de software más importantes, incluyendo metodologías y herramientas.

Parte	Capítulos	Tema	Descripción
II	4, 5	Programación orientada a objetos	Se describe el modelado en UML y el lenguaje de programación en Java. Se proporcionan ejemplos de desarrollo de pantallas en Java dirigidos a la creación de los prototipos a utilizarse en la parte III.
III	6, 7, 8, 9, 10	Desarrollo de software orientado a objetos	Se analizan las actividades más importantes del desarrollo de software para sistemas orientados a objetos, con especial énfasis en los modelos de requisitos, análisis, diseño, implementación y pruebas. Se describe la metodología de <i>casos de uso</i> , la base del <i>proceso unificado</i> , complementada por el <i>diseño por responsabilidades</i> (RDD, siglas de Responsibility Driven Design). Se presenta como ejemplo práctico un sistema de reservaciones de vuelos.
IV	11, 12	Programación y desarrollo de software para Internet	Se describe la programación para Internet basada en tecnología Java, en particular Servlets y JSP. Se extiende el sistema de reservaciones de vuelos a una arquitectura cliente-servidor para Internet.

Como libro de texto para un curso, recomiendo, según el tiempo disponible y el nivel de conocimiento previo de los alumnos, seguir alguno de los siguientes esquemas de horario:

Curso	Duración	Nivel previo	Horario
Ingeniería/desarrollo de software orientado a objetos básico (Semestral)	48 horas	Programación tradicional	I. Introducción (6 horas). II. Programación orientada a objetos (18 horas). III. Desarrollo de software orientado a objetos (24 horas).
Ingeniería/desarrollo de software orientado a objetos básico (Trimestral)	33 horas	Programación tradicional	I. Introducción (3 horas). II. Programación orientada a objetos (12 horas). III. Desarrollo de software orientado a objetos (18 horas).
Ingeniería/desarrollo de software orientado a objetos extendido (Semestral)	48 horas	Programación orientada a objetos	I. Introducción (6 horas). II. Programación orientada a objetos (6 horas). III. Desarrollo de software orientado a objetos (24 horas). IV. Programación y desarrollo de software para Internet (12 horas).
Ingeniería/desarrollo de software orientado a objetos extendido (Trimestral)	33 horas	Programación orientada a objetos	I. Introducción (3 horas). II. Programación orientada a objetos (3 horas). III. Desarrollo de software orientado a objetos (18 horas). IV. Programación y desarrollo de software para Internet (9 horas).

La diferencia principal entre el curso básico y el extendido son los requerimientos de conocimiento previo. Con un conocimiento de programación tradicional o estructurada recomiendo el curso básico, que permite al estudiante lograr un nivel inicial de programación orientada a objetos durante el curso. Con un co-

nocimiento de programación orientada a objetos es más conveniente el curso extendido, que permite al estudiante lograr un nivel más avanzado de programación orientada a objetos y a la vez aprender sobre la programación en Internet. Por otro lado, en el caso del curso extendido, se reduce el tiempo del curso básico dedicado a la programación orientada a objetos para incluir en su lugar la sección de desarrollo de software para Internet. Estos cursos pueden también variar en el orden preciso de capítulos a seguir y la profundidad que se dedique a los diferentes materiales. El lector encontrará de gran ayuda el sitio del libro, <http://www.cannes.itam.mx/Alfredo/Espaniol/Publicaciones/IngSW.htm>, por el material de apoyo, incluyendo diagramas, código y base de datos correspondientes al sistema de reservaciones de vuelo, el cual puede ser obtenido sin costo.

Alfredo Weitzenfeld
Ciudad de México

AGRADECIMIENTOS

Externo mi agradecimiento a todos los involucrados directa o indirectamente en la elaboración de este libro, en especial a los estudiantes que han desarrollado innumerables proyectos basados en las enseñanzas de estos últimos años y que aquí se presentan. Agradezco el apoyo del Instituto Tecnológico Autónomo de México¹ y en particular del Departamento Académico de Computación por haberme permitido ser el primero en impartir este curso y haber continuado con él durante estos últimos años, siendo una parte integral de los programas de licenciatura y maestría.

Expreso mi reconocimiento a Francisco Álvarez por colaborar en la elaboración del material referente a la calidad del software y modelo de madurez del capítulo 3.

Por último, agradezco de manera especial a mi familia por el apoyo brindado y ser fuente de inspiración. A mi madre, Sara, y a mi padre, Henryk, quien dedicó tiempo y esfuerzo invaluable en la revisión de este material en sus diferentes etapas de elaboración; a mi esposa, Tica, y a mis hijos, Jonathan, Gabriela y Ariel.

¹ Asociación Mexicana de Cultura S.A.

CONTENIDO BREVE

Parte I: Introducción 1

<u>Capítulo 1. Costo y complejidad del software</u>	<u>3</u>
<u>Capítulo 2. Tecnología orientada a objetos</u>	<u>21</u>
<u>Capítulo 3. Proceso de software</u>	<u>35</u>

Parte II: Modelado y programación orientada a objetos 67

<u>Capítulo 4. Modelado con UML</u>	<u>69</u>
<u>Capítulo 5. Programación orientada a objetos con Java</u>	<u>129</u>

Parte III: Desarrollo de software orientado a objetos 193

<u>Capítulo 6. Modelo de requisitos</u>	<u>195</u>
<u>Capítulo 7. Modelo de análisis</u>	<u>253</u>
<u>Capítulo 8. Modelo de diseño</u>	<u>333</u>
<u>Capítulo 9. Modelo de implementación</u>	<u>523</u>
<u>Capítulo 10. Modelo de pruebas</u>	<u>577</u>

Parte IV: Programación y desarrollo de software para Internet 599

<u>Capítulo 11. Programación con HTML, Servlets y JSP</u>	<u>601</u>
<u>Capítulo 12. Desarrollo de software para Internet</u>	<u>629</u>

<u>Glosario</u>	<u>669</u>
-----------------	------------

<u>Bibliografía</u>	<u>671</u>
---------------------	------------

<u>Índice</u>	<u>675</u>
---------------	------------

CONTENIDO

[Semblanza del autor](#) v

[Prefacio](#) vii

[Agradecimientos](#) xi

[Parte I: Introducción](#) 1

[Capítulo 1. Costo y complejidad del software](#) 3

[1.1 Costos ocultos y consecuencias por fallas del software](#) 3

[1.1.1 Fallas en sistemas de software](#) 4

[1.1.2 Sobrecostos, retrasos y cancelaciones en los sistemas de software](#) 11

[1.2 Complejidad del software](#) 13

[1.2.1 Confiabilidad del software](#) 14

[1.2.2 Software suficientemente bueno](#) 15

[1.2.3 La bala de plata](#) 16

[1.2.4 Ciclo de vida del software](#) 18

[Resumen](#) 19

[Referencias](#) 19

[Capítulo 2. Tecnología orientada a objetos](#) 21

[2.1 Conceptos básicos](#) 21

[2.1.1 Conceptos de la programación tradicional](#) 21

[2.1.2 Conceptos de la programación orientada a objetos](#) 22

[2.1.3 Revisión del problema del año 2000 \(Y2K\)](#) 23

[2.2 Programación y lenguajes orientados a objetos](#) 25

[2.2.1 Aspectos que mejoran la calidad de los sistemas](#) 25

[2.2.2 Características esenciales de los lenguajes orientados a objetos](#) 27

[2.3 Lenguajes de programación](#) 29

[Resumen](#) 32

[Referencias](#) 33

[Capítulo 3. Proceso de software](#) 35

[3.1 Modelo de proceso](#) 35

[3.1.1 Arquitectura](#) 36

3.1.2	Actividad	38
3.1.3	Métodos y metodologías	42
3.1.4	Estrategias	46
3.1.5	Herramientas	49
3.2	Modelos Clásicos	50
3.2.1	Cascada	50
3.2.2	Incremental	51
3.2.3	Evolucionario	52
3.2.4	Espiral	53
3.3	Modelos recientes	54
3.3.1	Ganar-ganar	54
3.3.2	Programación extrema (XP)	55
3.3.3	Proceso unificado (UP)	56
3.4	Calidad de software y modelos de madurez del proceso	56
3.4.1	Modelo de madurez de capacidades (CMM)	57
3.4.2	Organización Internacional para la Estandarización (ISO)	59
3.4.3	Modelo de madurez de ingeniería de desempeño (PEMM)	60
3.4.4	TickIt	61
3.4.5	Mejora del proceso de software y determinación de la capacidad (ISO-15504 / SPICE)	61
3.4.6	Proceso de software personal (PSP)	63
3.4.7	Proceso de software en equipos (TSP)	64
	Resumen	64
	Referencias	65

Parte II: Modelado y programación orientada a objetos 67

Capítulo 4. Modelado con UML 69

4.1	Objetos	69
4.1.1	Diagramas de objetos	71
4.1.2	Identidad	71
4.2	Clases	72
4.2.1	Diagrama de clases	73
4.2.2	Instanciación	73
4.3	Atributos	74
4.3.1	Identificadores	77
4.3.2	Atributos derivados	78
4.3.3	Restricciones de atributos	78
4.4	Operaciones	79
4.5	Ligas y asociación	83
4.5.1	Implementación	85
4.5.2	Grado de la asociación	85
4.5.3	Asociaciones reflexivas	87
4.5.4	Multiplicidad	88
4.5.5	Rol	91
4.5.6	Restricciones de ligas y asociaciones	93
4.5.7	Asociaciones derivadas	94
4.5.8	Accesos	96
4.5.9	Atributos de liga y asociación	96
4.5.10	Operaciones de asociación	99
4.5.11	Asociaciones como clases	100

4.6	Ensamblados: agregación y composición	101
4.6.1	<i>Tipos de ensamblados</i>	104
4.6.2	<i>Propagación de operaciones</i>	105
4.7	Generalización y herencia	106
4.7.1	<i>Extensión de clase</i>	112
4.7.2	<i>Restricción de clase</i>	112
4.7.3	<i>Sobreescritura de operaciones</i>	113
4.7.4	<i>Discriminador</i>	115
4.7.5	<i>Clases abstractas</i>	116
4.7.6	<i>Herencia múltiple</i>	119
4.8	Módulos	121
	Resumen	127
	Referencias	127
Capítulo 5. Programación orientada a objetos con Java		129
5.1	Introducción a Java	129
5.1.1	<i>Características</i>	129
5.1.2	<i>Procesamiento</i>	131
5.1.3	<i>Bibliotecas</i>	133
5.2	Programación básica	134
5.2.1	<i>Aspectos generales</i>	134
5.2.2	<i>Estructuras básicas</i>	137
5.3	Programación avanzada	145
5.4	UML y Java	148
5.4.1	<i>Objetos y clases</i>	148
5.4.2	<i>Ligas, asociaciones y composición</i>	159
5.4.3	<i>Generalización y herencia</i>	167
5.4.4	<i>Estructuras estáticas</i>	178
5.4.5	<i>Metaclases</i>	180
5.5	Aplicaciones y applets	180
5.5.1	<i>Aplicaciones</i>	181
5.5.2	<i>Applets</i>	181
5.6	Interfaces gráficas del usuario	182
	Resumen	191
	Referencias	191
Parte III: Desarrollo de software orientado a objetos		193
Capítulo 6. Modelo de requisitos		195
6.1	Descripción del problema	197
6.2	Modelo de casos de uso	199
6.2.1	<i>Actores</i>	200
6.2.2	<i>Casos de uso</i>	202
6.2.3	<i>Documentación</i>	208
6.3	Modelo de interfaces	209
6.4	Actores y casos de uso para el sistema de reservaciones de vuelos	210
6.4.1	<i>Actores</i>	210
6.4.2	<i>Casos de uso</i>	211
6.5	Modelo del dominio del problema	235
6.5.1	<i>Identificación de clases</i>	235

6.5.2	<i>Identificación de asociaciones</i>	242
6.5.3	<i>Identificación de atributos</i>	246
6.5.4	<i>Diccionario de clases</i>	247
6.5.5	<i>Identificación de Módulos</i>	249
Resumen		251
Referencias		251
Capítulo 7.	Modelo de análisis	253
7.1	Arquitectura de clases	254
7.1.1	<i>Clases con estereotipos</i>	255
7.1.2	<i>Clases para casos de uso</i>	257
7.2	Identificación de clases según estereotipos	258
7.2.1	<i>Borde</i>	258
7.2.2	<i>Entidad</i>	264
7.2.3	<i>Control</i>	268
7.3	Clases según casos de uso	271
7.3.1	<i>Validar Usuario</i>	271
7.3.2	<i>Ofrecer Servicios</i>	271
7.3.3	<i>Registrar Usuario</i>	271
7.3.4	<i>Registrar Tarjeta</i>	272
7.3.5	<i>Consultar Información</i>	272
7.3.6	<i>Hacer Reservación</i>	274
7.3.7	<i>Pagar Reservación</i>	275
7.4	Diagramas de secuencias	275
7.4.1	<i>Registrar Usuario</i>	277
7.4.2	<i>Registrar Tarjeta</i>	284
7.4.3	<i>Consultar Información</i>	292
7.4.4	<i>Hacer Reservación</i>	298
7.4.5	<i>Pagar Reservación</i>	304
7.5	Casos de uso para el sistema de reservación de vuelos	310
7.5.1	<i>Validar Usuario</i>	310
7.5.2	<i>Ofrecer Servicios</i>	311
7.5.3	<i>Registrar Usuario</i>	312
7.5.4	<i>Registrar Tarjeta</i>	315
7.5.5	<i>Consultar Información</i>	317
7.5.6	<i>Hacer Reservación</i>	322
7.5.7	<i>Pagar Reservación</i>	325
7.6	Diccionario de clases según Módulos	327
7.6.1	<i>InterfaceUsuario</i>	328
7.6.2	<i>Principal</i>	328
7.6.3	<i>Registro</i>	328
7.6.4	<i>Servicios</i>	329
Resumen		332
Referencias		332
Capítulo 8.	Modelo de diseño	333
8.1	Estrategias de diseño	336
8.1.1	<i>Arquitectura</i>	336
8.1.2	<i>Robustez</i>	337

8.1.3	<i>Reuso</i>	338
8.1.4	<i>Extensibilidad</i>	338
8.2	Diseño de objetos	338
8.2.1	<i>Tarjetas de clases</i>	339
8.2.2	<i>Responsabilidades</i>	341
8.2.3	<i>Colaboraciones</i>	402
8.2.4	<i>Jerarquías</i>	412
8.2.5	<i>Contratos</i>	432
8.2.6	<i>Subsistemas</i>	457
8.2.7	<i>Protocolos</i>	470
8.2.8	<i>Atributos</i>	482
8.2.9	<i>Algoritmos</i>	490
8.3	Diseño de Sistema	499
8.3.1	<i>Lenguajes de programación</i>	499
8.3.2	<i>Interfaces gráficas</i>	500
8.3.3	<i>Bases de datos</i>	503
8.3.4	<i>Archivos</i>	505
8.4	Revisión del diseño	510
8.5	Diagramas de secuencias del diseño	515
8.5.1	<i>Registrar Usuario: Crear Registro Usuario</i>	516
8.5.2	<i>Registrar Usuario: Actualizar Registro Usuario</i>	517
8.5.3	<i>Registrar Usuario: Eliminar Registro Usuario</i>	518
8.5.4	<i>Registrar Tarjeta: Crear Registro Tarjeta</i>	519
8.5.5	<i>Registrar Tarjeta: Actualizar Registro Tarjeta</i>	520
8.5.6	<i>Registrar Tarjeta: Eliminar Registro Tarjeta</i>	521

[Resumen](#) 522

[Referencias](#) 522

[Capítulo 9. Modelo de implementación](#) 523

9.1	Programación en Java	523
9.1.1	<i>InterfaceUsuario</i>	523
9.1.2	<i>Principal</i>	529
9.1.3	<i>Domnio</i>	536
9.1.4	<i>Registro</i>	539
9.1.5	<i>Servicios</i>	569
9.2	Diagrama de clases	570
9.2.1	<i>InterfaceUsuario</i>	571
9.2.2	<i>Principal</i>	571
9.2.3	<i>Domnio</i>	572
9.2.4	<i>Registro</i>	573
9.2.5	<i>Servicios</i>	575

[Resumen](#) 576

[Referencias](#) 576

[Capítulo 10. Modelo de pruebas](#) 577

10.1	Definición de conceptos	577
10.2	Tipos de pruebas	578
10.2.1	<i>Técnicas de pruebas</i>	578
10.2.2	<i>Nivel de pruebas</i>	579
10.3	Proceso de pruebas	582
10.3.1	<i>Estrategia de prueba</i>	582

10.3.2	<i>Planeación de la prueba</i>	583
10.3.3	<i>Construcción de la prueba</i>	583
10.3.4	<i>Ejecución de la prueba</i>	584
10.4	Pruebas del Sistema de Reservas de Vuelos	584
10.4.1	<i>Registrar Usuario</i>	584
Resumen		598
Referencias		598

Parte IV: Programación y desarrollo de software para Internet 599

Capítulo 11. Programación con HTML, Servlets y JSP 601

11.1 Arquitectura cliente-servidor 601

11.1.1 Cliente 602

11.1.2 Servidor 602

11.2 HTML 602

11.2.1 Etiquetas 603

11.2.2 Texto 603

11.2.3 Ligas 604

11.2.4 Formas 606

11.3 Servlets 608

11.3.1 Definición 609

11.3.2 Formas 611

11.4 JSP 614

11.4.1 Scripts 615

11.4.2 JavaBeans 621

11.5 Integración servlets y JSP 626

Resumen 628

Referencias 628

Capítulo 12. Desarrollo de software para Internet 629

12.1 Modelo de diseño 629

12.1.1 Diseño de sistema 629

12.1.2 Diseño de objetos 631

12.1.3 Diagramas de secuencias 642

12.2 Modelo de implementación 643

12.2.1 Clases comunes 643

12.2.2 Clases stand-alone 648

12.2.3 Clases cliente-servidor 650

12.2.4 Diagrama de clases 656

12.3 Modelo de pruebas 656

12.3.1 Registrar Usuario 656

12.3.2 Registrar Tarjeta 663

Resumen 667

Referencias 667

Glosario 869

Bibliografía 671

Índice 675

Introducción

En esta era tecnológica, muchos aspectos de nuestras vidas están regidos por las computadoras y el software que las controla. Las consecuencias generadas por el uso del software, tanto a favor como en contra, son cruciales. Cuando todo funciona bien, no nos percatamos de esto, pero cuando fallan, lo percibimos, y de qué manera. El resultado puede ser nefasto, lo cual se ha corroborado en múltiples ocasiones.

En esta primera parte del libro se presentan casos relacionados con el software que guiarán el resto del libro: (1) costo y complejidad del software, (2) tecnología orientada a objetos y (3) proceso de software.

Costo y complejidad del software

Comenzamos con una interrogante muy sencilla: ¿Cuál es el costo del software? Aunque esta pregunta es general, podemos analizar tres tipos de costos relacionados con él:

- 1) El **costo directo** para adquirir el software, el cual incluye el software *empacado*, se puede adquirir en un negocio de computación o por Internet; y el software *a la medida*, que requiere un desarrollo especializado y adaptado a las necesidades particulares de una empresa.
- 2) El **costo indirecto** para utilizar el software incluye aspectos como la capacitación, instalación, soporte técnico, así como otros costos que por lo general se pueden conocer de antemano.
- 3) El **costo oculto** ocasionado principalmente por las fallas del software. A diferencia de los costos directos e indirectos, los cuales son previsibles, los costos ocultos por definición son difíciles de prever. Vale la pena destacar que el tema de costos ocultos afecta principalmente a los sistemas conocidos como de *misión crítica* (aquellos sistemas críticos para la operación correcta de una empresa).

No es el objetivo de este capítulo profundizar de manera general en el tema de los costos del software, sino más bien ilustrar mediante ejemplos hasta dónde pueden llegar a ser significativos los *costos ocultos y consecuencias por las fallas de estos sistemas*, seguido de una discusión sobre la *complejidad del software* y el origen de estos graves problemas.

1.1 Costos ocultos y consecuencias por fallas del software

Dado que la razón principal de los costos ocultos son las fallas en los sistemas de software, analizaremos cuáles podrían llegar a ser las consecuencias del funcionamiento incorrecto del software. Para ello los dividiremos en dos rubros:

- (i) Consecuencias inmediatas y efectos directos, y
- (ii) consecuencias a mediano y largo plazo, y efectos indirectos.

1) Consecuencias inmediatas y efectos directos. Son los perjuicios ocasionados mientras dura la caída de los sistemas. En el caso de sistemas de *misión crítica*, de los cuales depende la operación exitosa de una empresa, por ejemplo un sistema financiero de un banco, una falla puede significar ingresos que se dejan de percibir por transacciones perdidas y egresos que continúan a pesar de la interrupción en la operación. Estos costos son relativamente predecibles dado que dependen directamente del tiempo que dure la interrupción en la operación.

2) Consecuencias a mediano y largo plazo y efectos indirectos. Son los perjuicios posteriores a la caída de los sistemas. Las consecuencias varían, desde la restauración de datos, servicios de emergencia, propaganda negativa y pérdida de clientes, hasta posibles accidentes y juicios en contra. Estos costos adicionales pueden volver insignificantes los costos directos e indirectos del software; por lo que es difícil predecir el costo real del software a mediano y largo plazo.

Por otro lado, dejar de utilizar software no es una alternativa aceptable, ya que los efectos negativos pueden ser aún mayores. Éste es el caso de los sistemas de *misión crítica*, por ejemplo, una empresa que opera aviones y el software controla el funcionamiento correcto de éstos.

En los últimos años han ocurrido fallas de software con consecuencias nefastas. Existen muchos casos donde errores en el software, o su mala utilización, han causado pérdidas económicas multimillonarias, e incluso vidas humanas.

Peter G. Neumann, moderador del foro de la Association for Computer Machinery (ACM) sobre riesgos al público en el uso de computadoras y sistemas, mantiene una lista de desastres ocasionados por fallas del software o su mal uso.¹ Estos temas son tratados por diversos autores, como Robert Glass² y Nancy Leveson³ junto con otros grupos, como los Profesionales de la Computación para la Responsabilidad Social (CPSR, por sus siglas en inglés),⁴ quienes reportan las implicaciones sociales debidas a fallas en las computadoras. Lamentablemente se aprende más de los errores que de los sistemas que funcionan de manera correcta.

A continuación se presentan algunos ejemplos con el objeto de motivar y concientizar al lector de que el software no es algo que pueda o deba tomarse a la ligera. Más adelante en este capítulo analizaremos qué podemos hacer como ingenieros de software para desarrollar sistemas de mayor calidad que reduzcan los *costos ocultos*.

1.1.1 Fallas en sistemas de software

Aquí se presentan algunos ejemplos de desastres ocasionados de manera directa o indirecta por fallas en el software. Estos ejemplos se describen en orden cronológico.

Fracaso del Mariner 1 (1962).⁵ La primera misión del programa Mariner (cuyo costo total, desde la misión Mariner 1 hasta la Mariner 10, fue de 554 millones de dólares) fracasó por un carácter incorrecto ('-') en la especificación del programa de control para el cohete de propulsión Atlas, lo cual causó finalmente

que se saliera de curso. Tanto el cohete como el vehículo espacial tuvieron que ser destruidos poco después del lanzamiento. Se cree que un error de computadora también fue la causa del fracaso del Mariner 8 en 1971.

Sobregiro del Bank of New York (1985).⁶ En noviembre de 1985, el Bank of New York (BoNY) tuvo accidentalmente un sobregiro de 32 000 millones de dólares (una buena suma si consideramos que esto fue hace 15 años!). Esto fue ocasionado por un contador de 16 bits (la mayoría de los contadores eran de 32 bits) que se activó provocando un desbordamiento (*overflow*) del contador que nunca fue verificado. El banco no pudo procesar nuevas transferencias, por lo que la Reserva Federal de Nueva York automáticamente hizo un traspaso de 24 000 millones de dólares al BoNY para cubrir sus gastos por un día. El banco tuvo que pagar 5 millones de dólares de intereses diarios mientras se arreglaba el software.

Accidente de un F-18 (1986).⁷ En abril de 1986 un avión de combate F-18 se estrelló por culpa de un giro descontrolado (*unrecoverable spin*), atribuido a una expresión "*if-then*", para la cual no había una instrucción "*else*", por considerarse innecesaria, lo que originó una excepción fuera de control del programa. Por suerte el piloto pudo salir del avión a tiempo.

Muertes por el Therac-25 (1985-1987).⁸ El acelerador lineal médico Therac-25, producido por Atomic Energy of Canada Limited (AECL), fue diseñado para tratamientos de radiación de dos tipos: (i) tratamiento de rayo directo de bajo poder y (ii) tratamiento de rayo indirecto reflejado de alto poder. Entre 1985 y 1987 este sistema ocasionó la muerte de varios pacientes en diferentes hospitales de Estados Unidos y Canadá debido a radiaciones de alto poder aplicadas sin control. A partir de ciertas secuencias de comandos del operador de la máquina, los controles de la computadora lo llevaban a un estado interno erróneo muy peligroso, generando una sobredosis masiva de radiación. Después de una amplia publicidad de estos accidentes, se descubrió que la Federal Drug Agency (FDA) no especificaba requisitos, ni hacía revisiones sobre prácticas de desarrollo o control de calidad de software en dispositivos médicos. La FDA informó en septiembre de 1987 que comenzaría a exigir controles de software integrados a ciertas clases de dispositivos médicos. Lamentablemente, en muchos de estos sistemas el operador también juega un papel crítico, como el caso de Panamá en el 2001, donde datos insertados incorrectamente por los operadores durante tratamientos de cáncer mediante radiación ocasionaron la muerte de cinco pacientes en este país.⁹ Aunque la falla no fue directamente atribuida al software, se supone que el sistema de radiación contaba con protección para prevenir la radiación de tejido sano.

Avión derribado por el USS Vincennes (1988).¹⁰ En julio de 1988, la fragata USS Vincennes asignada al golfo Pérsico, registró en su radar la presencia de un avión no identificado que se acercaba rápidamente al barco. Al no lograr una comunicación directa que permitiera confirmar la identidad del avión, se disparó un misil derribando lo que resultó ser un avión comercial iraní de tipo Airbus, matando a las 290 personas que estaban a bordo. El USS Vincennes registró de forma incorrecta que se trataba un avión de combate F-14 descendiendo sobre el barco de manera hostil. Aunque el comandante del navío dio la orden de disparo, se consideró como causa del accidente al sistema de radar AEGIS, el cual mostraba únicamente un punto junto a un dato textual represen-

tando al avión, en lugar del eco real del radar sobre el avión. Posteriormente, se creyó que en algún momento el avión iraní estuvo en la proximidad de un F-14, quizá durante el despegue del aeropuerto, confundiendo al sistema AEGIS y asociando de manera incorrecta la información transmitida por los *transponders* aire-tierra del F-14 a la aerolínea. Al despegar quedaron asociados los datos del F-14 sobre la pantalla. Una representación inconveniente y quizá confusa de la información de altitud del avión confundió aún más a los oficiales del barco, los cuales supusieron que el F-14 estaba descendiendo, aunque en realidad estaba ganando altura.

Falla del software de AT&T (1990).¹¹ El 15 de enero de 1990, American Telegraph and Telephone (AT&T), compañía que controla las redes del mayor sistema de comunicación en el mundo, tuvo una falla masiva en su sistema de comunicaciones, durando alrededor de nueve horas e interrumpiendo millones de llamadas de larga distancia internacional. Un error en el software de manejo de excepciones de un tipo particular de sistema de enlace o ruteo telefónico ocasionó una cadena de fallas en cascada en los enlaces. Se reportó que el problema se originó en uno de los programas de ruteo escritos en el lenguaje C.

Falla de software en la Estación Nuclear Bruce, Canadá (1990).¹² El 31 de enero de 1990 un error de software en la estación nuclear de Bruce en Canadá ocasionó la liberación de miles de litros de agua radiactiva, los cuales escaparon en forma de vapor. Los encargados de seguridad en la industria nuclear describieron la situación como "muy inusual y también muy significativa". Por suerte la situación se controló rápidamente causando únicamente la pérdida de dinero y tiempo, manteniendo la estación fuera de operación por varias semanas.

Aberración esférica en el telescopio espacial Hubble (1990).¹³ El 25 de abril de 1990 se puso en órbita el famoso telescopio espacial Hubble desde la nave espacial Discovery. Al poco tiempo, la NASA descubrió que el componente más crítico del telescopio de 4000 millones de dólares, su espejo principal, tenía una *aberración esférica* que imposibilitaba producir imágenes nítidas. Una investigación de la NASA reveló que el espejo se había construido más plano de lo estipulado en el diseño original, con un defecto de 2 micrones ($1 \text{ micrón} = 10^{-6} \text{ metros}$), un error bastante grande según los estándares de precisión de la óptica moderna. Aunque se identificaron problemas adicionales, como en sus paneles solares, sus giroscopios y contactos eléctricos, el problema principal del telescopio era su lente, el cual nunca fue probado en tierra antes de ser enviado al espacio. Se hicieron simulaciones en computadora como alternativa de menor costo para validar el rendimiento del espejo. Por desgracia, se utilizaron datos incorrectos de entrada en la simulación. Para corregir el error en el espacio, se agregó óptica correctiva a un costo mucho mayor y sin lograr que el espejo funcionara tan bien como se planeó. Los astrónomos tendrán que limitarse a las restricciones actuales del Hubble, con el cual sólo se pueden percibir objetos con tamaños 20 veces mayores, aproximadamente.

Falla del software de los misiles Patriot (1991).¹⁴ En las primeras etapas de la guerra del golfo Pérsico en 1991, el sistema de defensa antimisiles Patriot fue descrito como muy exitoso. Sin embargo, permitió que un misil iraquí Scud destruyera parte de las barracas militares en Daharan, Arabia Saudita, causando 29 muertos y 97 heridos, la peor baja estadounidense durante la guerra. Aparente-

mente, el sistema de radar nunca vio al misil Scud, por lo cual no se lanzó el misil Patriot. Según oficiales del ejército: "una combinación imprevista de *docenas* de variables —incluyendo la velocidad, altura y trayectoria del Scud— causaron la falla del sistema del radar... [este caso fue] una anomalía que nunca apareció durante las horas de pruebas". El error se atribuye a una acumulación de inexactitudes en la operación del tiempo interno de la computadora del sistema. Aunque éste obedecía a las especificaciones, debía apagarse y prenderse con la suficiente frecuencia para que el error acumulado nunca fuera peligroso. Después de 8 horas de uso se detectó el problema del error acumulado en el reloj. La corrección sólo se logró al día siguiente de la catástrofe. El sistema Patriot había sido diseñado para trabajar en ambientes más limitados y menos hostiles que el que había en Arabia Saudita. Lamentablemente, el resto de la operación del sistema Patriot tampoco fue muy exitoso, en análisis posteriores la estimación de su efectividad disminuyó de 95 a 13%.

Administradora de capital de riesgo quiebra por datos incorrectos en un modelo de cómputo (1994).¹⁵ En 1994 la compañía Askin Capital Management, un imperio de fondos de cobertura de 600 millones de dólares, quebró por culpa de valuaciones imprecisas insertadas a un modelo utilizado para negociar garantías basadas en hipotecas.

Error en el procesador Pentium de Intel (1994).¹⁶ En 1994, un error de punto flotante en el procesador Pentium le costó a Intel 475 millones de dólares. El error no fue reconocido públicamente durante meses por Intel, declarando que el procesador era "suficientemente bueno" y que sería muy difícil que ocurriera el error. Actualmente, Intel tiene otros problemas similares con sus procesadores, como la unidad MTH (Memory Translator Hub) usada para transferir señales de la memoria a otra unidad de la computadora (Intel 820) y la generación del Pentium III de 1 GHz, la cual fue retirada del mercado. ¡Al menos la compañía ya ha aprendido a reconocer sus errores!

Error en un sistema de autenticación de tarjeta de crédito (1995).¹⁷ Según un artículo del 4 de noviembre de 1995 en el periódico *Guardian* del Reino Unido, los dos sistemas más grandes en ese país para la autorización de crédito (Barclay's PDQ y NatWest's Streamline) fallaron el sábado 28 de octubre de 1995 imposibilitando que los comercios verificaran las tarjetas de crédito de sus clientes. En el caso de Barclay, más de 40% de las transacciones fallaron por un *error en el sistema de software*. Para NatWest, el problema fue ocasionado por una gran cola de llamadas, que obstruyó la comunicación por razones desconocidas, y que retrasó la autenticación de tarjetas. Aunque ambos sistemas estaban preparados para afrontar este tipo de contingencias, que permitían a los comercios llamar por teléfono para autenticar solicitudes, el volumen de ventas ocasionó que las líneas se saturaran rápidamente.

Explosión del cohete Ariane 5 (1996).¹⁸ El 6 de junio de 1996 se culpó a una computadora por la explosión del primer vuelo, el 501, del cohete Ariane 5 con un costo de 500 millones de dólares. El cohete, que al parecer no estaba asegurado, llevaba cuatro satélites, cuya explosión ocasionó pérdidas totales de 1800 millones de dólares. El Ariane 5 estaba funcionando perfectamente hasta los 40 segundos iniciales, cuando de repente empezó a salirse de su trayectoria y sólo fracciones de segundo después, fue destruido por control remoto mediante una señal enviada por un controlador del Ariane desde Tierra. Según la

European Spacial Agency (ESA), administradora del programa, la desviación en la trayectoria fue ocasionada por la computadora que controlaba los dos poderosos impulsores del cohete. Se especuló que la computadora creyó que el cohete se estaba saliendo de su curso y de esta manera trataba de corregir la trayectoria de vuelo. De acuerdo con el reporte final, la causa de la falla del sistema ocurrió durante la conversión de un número flotante de 64 bits a un número entero de 16 bits. Al convertir un número con punto flotante daba como resultado un valor mayor del que podía ser representado por un número entero de 16 bits (con signo), ocasionando un *error de operando*. Las instrucciones de conversión de datos (código en Ada) no estaban protegidas para evitar el error de operando, aunque otras conversiones en variables similares en el mismo lugar sí lo estaban. El origen del problema radicó en que el Ariane 5 podía llevar un mayor número de satélites que el Ariane 4, incrementando así su peso. Sin embargo, el Ariane 5 utilizaba una gran cantidad de software diseñado para el Ariane 4.

Error del sistema de cobranza lleva a una compañía a la quiebra (1996).¹⁹

En la edición de abril de 1996 de *TVRO Dealer* (publicación sobre televisión por satélite), se describió cómo el intento por cambiar un nuevo sistema de software de cobranza, de un servicio de programación de una gran compañía de televisión por satélite, causó la quiebra de la compañía el 28 de marzo anterior.

Error del sistema de cobranza en MCI (1996).²⁰ En la edición del 29 de marzo de 1996 del *Washington Post*, MCI reportó que devolverían aproximadamente 40 millones de dólares a sus clientes por un error de cobranza causado por un sistema de cómputo. El error de cobranza fue descubierto por un reportero investigador de una estación local de televisión en Richmond, VA, quien encontró que fueron facturados por 4 minutos cuando en realidad la llamada fue de tan sólo 2.5 minutos, dando lugar a una profunda investigación.

Mayor falla de una computadora en la historia de los bancos en Estados Unidos (1996).²¹ El 18 de mayo de 1996 la revista *US & World Report*, y al siguiente día el diario *The Boston Globe*, informaron que aproximadamente 800 clientes del First National Bank of Chicago se sorprendieron al ver que sus saldos eran de 924 millones de dólares más de lo que tenían la semana anterior. La causa fue el tradicional "cambio en el programa de la computadora". De acuerdo con la Asociación de Banqueros Americanos, los 763 900 millones fueron la cantidad más grande producida por un error de computadora en la historia bancaria de Estados Unidos, más de seis veces el total de fondos del banco. El problema fue atribuido oficialmente a un "error de la computadora".

Falla de la computadora del Centro de Control de Tráfico Aéreo de Nueva York (1996).²² El 20 de mayo de 1996 falló la computadora del Centro de Control de Tráfico Aéreo de Nueva York (ARTCC) que controla el tráfico aéreo sobre los estados de Nueva York, Connecticut, Nueva Jersey, Pennsylvania y parte del océano Atlántico. La computadora, con siete años de operación, perdió capacidad de servicio efectivo ("falló") dos veces la tarde del lunes 20 de mayo; la primera durante 23 minutos y la segunda alrededor de una hora, una hora más tarde. Parece que cuatro días antes se había instalado un nuevo software en el sistema. Se regresó al sistema anterior, con procedimientos de control de tráfico aéreo menos eficientes, ocasionando mayor saturación de tráfico y retrasos

en los despegues de alrededor de una hora en los aeropuertos principales en el área, además de un incremento en la carga de trabajo de los controladores y menor seguridad, incluyendo la desactivación de la "alerta automática de conflictos".

Mala planificación del nuevo sistema de una administradora de servicios de salud (1997).²³ Según reportó el *Wall Street Journal* el 11 de diciembre de 1997, Oxford Health Plans Inc., administradora de servicios de salud en Estados Unidos, compañía de gran crecimiento en los últimos tiempos, anunció que registraría una pérdida de 120 millones de dólares o más durante ese trimestre, además de otra adicional de 782 millones de dólares, su primera pérdida desde que salió a la bolsa en 1991. La razón principal fue la larga lista de problemas ocasionados por un sistema informático que se puso en línea en 1996; desde el diseño del sistema y su instalación hasta cómo fue administrado por los ejecutivos del grupo Oxford; los cuales ocasionaron que Oxford no pudiera enviar facturas mensuales a miles de clientes, además de incapacitarla para monitorear los pagos a cientos de médicos y hospitales. En menos de un año, los pagos no cobrados de sus clientes se triplicaron a más de 400 millones de dólares, mientras que el monto que Oxford debía a los proveedores de servicios médicos aumentó en más de 50%, alcanzando una suma superior a los 650 millones de dólares.

Pérdida de un banco por datos incorrectos de un modelo (1997).²⁴ En 1997 el banco UBS de Suiza perdió 412 millones de dólares por pérdidas en derivados, en parte causadas por precios incorrectos insertados en un modelo de derivados de acciones.

Error en equipo de Cisco (1998).²⁵ En abril de 1998 un error en un equipo de ruteo ("switch") de Cisco en uso por AT&T se propagó por cientos de equipos de ruteo en su red de alta velocidad, dejando fuera de servicio miles de cajeros automáticos y lectores de tarjetas de crédito.

Software inapropiado llevó a un distribuidor de medicina a la quiebra (1998).²⁶ El 27 de agosto de 1998 la revista *Der Spiegel*, en Alemania, informó de una demanda de 500 millones de dólares a SAP por parte del distribuidor de medicinas FoxMeyer Corp. Ésta última acusó a SAP de venderle software inapropiado para sus necesidades, lo cual tuvo como resultado la quiebra de FoxMeyer. Analistas alemanes comentaron que no consideran que un "software sea apropiado para llevar a la ruina a una compañía".

Error en sistema de control de cohete ruso (1998).²⁷ En septiembre de 1998 la computadora del cohete Ucraniano Zenit 2 apagó por error el motor cinco minutos después del despegue. El cohete se estrelló destruyendo 12 satélites comerciales propiedad de GlobalStar Telecom con un costo superior a 185 millones de dólares.

Error en sistema de subastas de eBay (1999).²⁸ En junio de 1999 un error en el software dejó fuera de servicio por 22 horas al sistema de subastas de eBay.

Error de un controlador de discos de Toshiba (1999).²⁹ En noviembre de 1999 Toshiba llegó a un arreglo fuera de la Corte que le costaría más de 2000

millones de dólares, para cubrir los errores ocasionados por la pérdida de información debida a fallas en los controladores de discos floppy de sus computadoras portátiles a partir de 1980. Aunque los controladores fueron diseñados originalmente por NEC, Toshiba producía sus propios componentes y nunca incluyó la modificación hecha por NEC en 1987, lo cual habría evitado el problema. Lo más interesante del caso es que realmente nunca se reportó falla alguna. Queda por ver qué consecuencias traerá este caso al resto de los fabricantes de computadoras para quienes este precedente los tiene sumamente preocupados.

Actualización de software mal planificado paralizó Nasdaq (1999).³⁰ El 17 de noviembre de 1999 los corredores de la bolsa de valores de Nasdaq no pudieron comprar ni vender acciones durante 17 minutos cruciales, después de que empleados de Nasdaq intentaran actualizar, sobre la marcha, un sistema de software durante la última media hora de la sesión. Algo funcionó mal y los inversionistas tuvieron que dejar de operar.

Error del milenio (2000).³¹ El "error del milenio" o "Y2K" (del inglés, Year 2 K, donde K = kilo = mil), se remonta a la década de 1960, cuando los programadores adoptaron la convención de representar el año con dos dígitos, en lugar de cuatro; a estos dígitos *alambraba* al inicio el 19 para generar la fecha completa. Por ejemplo el año 1999 se representaba mediante dos dígitos "99", para luego agregar el "19" y obtener la fecha verdadera. Sin embargo, al llegar al año 2000 esto ocasionaría fallas en los sistemas dado que la especificación "00" correspondía a 1900 en lugar de 2000. Para complicar más las cosas, a menudo los dígitos "99" o "00" son valores reservados en los sistemas de cómputo ("números mágicos"), que significaban "nunca borrar esto" o "ésta es una cuenta de demostración". El problema tampoco se limitó a la generación directa de la fecha, si no que además, los algoritmos para reconocer años bisiestos se volvieron también incorrectos. No está clara la razón original por la que se escogió la representación de la fecha con base en dos dígitos. Pudo deberse al alto costo de la memoria de las computadoras en esa época, o que nadie se imaginó que aquellos sistemas duraran tanto, o quizá simplemente no se reconoció el problema.

Actualmente se acepta que es difícil estimar cuál fue el costo total del Y2K. Según la compañía Technology Management Reports (TMR) de San Diego, los costos podrían superar el billón de dólares.³² Esto incluía la reescritura de programas existentes, adquisición e instalación de sistemas que los reemplazara y productividad perdida por la interrupción y fallas de los sistemas a partir del 2000. Otras compañías estimaron costos similares, como el grupo Gartner, que calculó un costo de 600 000 millones de dólares a nivel mundial.³³ Para tratar este problema Gartner informó que un buen número de compañías asignaron presupuestos especiales, como Chase Manhattan, que declaró que gastaría 250 millones de dólares y American Airlines y Hughes Electronics 100 millones de dólares cada una. La Oficina de Administración del Presupuesto de la Casa Blanca calculó sus costos de reparación de sus sistemas en 2800 millones de dólares. Esto incluía 4500 computadoras para defensa nacional, tráfico aéreo, pagos de impuestos y seguridad social. El Grupo Gartner estimó que los costos para el Departamento de Defensa de Estados Unidos podrían ser superiores a los 30 000 millones de dólares. Dada la gran preparación que finalmente hubo, incluyendo la renovación de una gran cantidad de sistemas de cómputo, los pro-

blemas reportados fueron menores. Por ejemplo, una agencia de viajes alemana reportó que las reservaciones hechas por su sistema, que se debieron autorizar para enero del 2001, fueron acreditadas para enero del 2000. Este problema parece haber afectado a otras 200 agencias de viajes.³⁴

1.1.2 Sobrecostos, retrasos y cancelaciones en los sistemas de software

Lamentablemente los costos ocultos no se restringen únicamente a fallas en el software, también pueden ocurrir durante su desarrollo. Según un reporte dado por el Standish Consulting Group en 1994,³⁵ en el que se evaluaron 175 000 proyectos con un costo total de 250 000 millones de dólares, 31.1% se cancelaron, 52.7% tuvieron sobrecostos y retrasos, y sólo 16.2% se hicieron a tiempo, a bajo costo y de acuerdo con los requisitos estipulados. De acuerdo con varias encuestas hechas a diferentes compañías, las tres razones más importantes para el éxito de un proyecto son:

- (i) Participación del usuario.
- (ii) Apoyo de la administración ejecutiva.
- (iii) Clara especificación de requerimientos.

En esta sección se muestran, en orden cronológico, algunos ejemplos de cancelaciones, sobrecostos y retrasos de sistemas.

Sobrecosto y retraso en sistema de Allstate Insurance (1982).³⁶ En 1982, Allstate Insurance comenzó a construir un sistema para automatizar su negocio por 8 millones de dólares. El esfuerzo de cinco años continuó hasta al menos 1993 con un costo final de 100 millones de dólares.

Sobrecosto, retraso y cancelación en el sistema de la London Stock Exchange (1983-1988).³⁷ El proyecto Taurus de la Bolsa de Valores de Londres estaba originalmente cotizado en 6 millones de libras. Varios años más tarde y con un sobrecosto en el presupuesto de más de 100 veces (13,200%), el proyecto fue cancelado, costando a la ciudad de Londres 800 millones de libras al momento de ser abandonado.

Sobrecosto y retraso en el sistema del bombardero B-1 (1985).³⁸ El bombardero B-1 en servicio desde 1985 necesitó 1000 millones de dólares adicionales para mejorar su software de defensa aérea, no resultó totalmente efectivo, pues no logró los objetivos originales.

Sobrecosto, retraso y cancelación en el sistema de registro de licencias de manejo del DMV (1987).³⁹ En 1987, el Departamento de Vehículos de Motores (DMV) de California, USA, emprendió un proyecto de revitalización de sus procesos de aplicación de registro y licencias de manejo.

Para 1993, después de 45 millones de dólares gastados, el proyecto fue cancelado. Según el reporte hecho por el DMV, la razón principal para el nuevo desarrollo de estas aplicaciones fue la adopción de nueva tecnología.

Sobrecosto, retraso y cancelación en el sistema del Bank of America (1988).⁴⁰ En 1988, Bank of America gastó 23 millones de dólares en un plan

inicial de cinco años para desarrollar MasterNet, sistema computarizado para contabilidad y reportes de fideicomisos. Luego de abandonar el sistema anterior, se gastó 60 millones de dólares adicionales para que el nuevo sistema funcionara. El sistema fue finalmente cancelado. Las cuentas de los clientes perdidos pudieron haber representado miles de millones de dólares.

Sobrecosto y retraso en un sistema de control de rastreo por satélite (1989).⁴¹ La modernización del software del sistema de Control de Rastreo por Satélite ("Satellite Tracking Control Facility") tomó siete años más de lo previsto, costó 300 millones de dólares adicionales y proporcionó menos capacidad de la requerida.

Sobrecosto y retraso en el sistema Airborne Self-Protection Jammer (1989).⁴² El sistema Airborne Self-Protection Jammer (ASJP), sistema electrónico de defensa aérea instalado en alrededor de 2000 aviones de combate y ataque de la marina estadounidense, costó 1000 millones de dólares adicionales, tomó cuatro años más y sólo fue *efectivo y apropiado marginalmente*.

Sobrecosto en sistema del avión de carga C-17 (1989).⁴³ El avión de carga C-17, construido por Douglas Aircraft, costó 500 millones de dólares más de lo previsto debido a problemas del software aeronáutico. Un reporte de la agencia GAO (General Accounting Office) del gobierno americano notó que existieron 19 computadoras a bordo, 80 microprocesadores y seis lenguajes diferentes de programación.

Cancelación del sistema de reservaciones CONFIRM (1994).⁴⁴ En 1994, American Airlines llegó a un acuerdo fuera de corte con Budget Rent-A-Car, Marriott Corp. y Hoteles Hilton luego que el proyecto del sistema de reservaciones de hoteles y renta de automóviles CONFIRM por 165 millones de dólares *se hundió en un caos*.

Cancelación del sistema de facturación para PG&E (1998).⁴⁵ Entre 1996-1998, Pacific Gas & Electric Co. (PG&E) gastó millones de dólares en un sistema desarrollado por IBM para tramitar facturas de sus clientes junto con otras funciones. El sistema no pudo hacer frente a las nuevas necesidades de la industria luego de la desregulación del mercado, originando la cancelación del sistema.

Los ejemplos anteriores son sólo una pequeña muestra, aunque representativa, de los problemas que pueden ser causados por fallas en el software, tanto durante su utilización como durante su desarrollo. ¿Cómo podemos justificar tantos problemas con el software? Una pequeña anécdota refleja esta situación.

Se dice que hace unos años se juntaron un médico, un ingeniero civil y un ingeniero en computación para discutir cuál era la profesión más antigua del universo. El médico explicó: "Dios creó a Eva de la costilla de Adán; obviamente se requirió cirugía, por lo cual la medicina tiene que haber sido la profesión más antigua del universo." A esto contestó el ingeniero civil: "Antes de Adán y Eva, Dios creó el universo, del caos puso orden en el cielo y en la Tierra; definitivamente ésta es la obra más espectacular de la ingeniería civil y también la más antigua." Finalmente, exclamó el ingeniero en computación: "pero, ¿quién creen que creó el *caos* en primer lugar?"

Esta anécdota refleja de manera bastante acertada la problemática del software. Un famoso proverbio chino dice que es tarea fácil hacer las cosas complejas, pero es tarea compleja hacer las cosas fáciles. Una frase más actual y relacionada con el software dice que para hacer las cosas mal es suficiente una persona, pero para hacerlas verdaderamente desastrosas se requiere una computadora. En las siguientes secciones analizaremos con mayor profundidad los problemas del software.

1.2 Complejidad del software

La problemática del software está directamente relacionada con el tamaño de los sistemas de éste. Cuanto más grandes son los sistemas, mayor es su complejidad o el caos que pueden ocasionar. Se puede hablar de dos factores principales que causan esta complejidad: la *complejidad del problema* y la *complejidad de la solución*.

- La **complejidad del problema** tiene que ver con la funcionalidad que el sistema debe brindar. Cuanto mayor sea el número de requerimientos o funcionalidad ofrecida por una aplicación, mayor será el tamaño del sistema, creando sistemas más difíciles de comprender y desarrollar. La complejidad radica en los aspectos intrínsecos del problema, o sea, sus requerimientos funcionales. Para reducir la complejidad habría que reducir la funcionalidad que el sistema deba tener.
- La **complejidad de la solución** tiene que ver con el diseño del sistema, el cual debe satisfacer la funcionalidad del problema. Cuando la complejidad del problema es bastante grande y difícil de reducir, es muy importante reducir la otra fuente de complejidad: el de la solución, o sea, el software. Éste será el objetivo a lograr en este libro, reducir la complejidad del sistema de software.

De manera adicional, se consideran dos factores relacionados con la complejidad de un sistema, uno *estático* y otro *dinámico*.

- El **factor estático** corresponde a la funcionalidad que un sistema de software debe ofrecer al ser inicialmente desarrollado.
- El **factor dinámico** corresponde a la funcionalidad que varía con el tiempo, en otras palabras, con los posibles cambios en el sistema. Estos cambios pueden ser considerables y, en muchos casos, son la causa de los retrasos y cancelaciones de los proyectos. Es muy común que los requisitos de un sistema se modifiquen, incluso antes de completarlos. Según la **ley de Lehman**,⁴⁶ *todo programa que se use se modificará*, y cuando un programa se modifica su complejidad aumenta, siempre y cuando uno no trabaje activamente contra esto. Esto es similar al problema de la entropía, una medida del desorden de un sistema, el cual tuvo origen en la termodinámica. Según la segunda ley de la termodinámica, la entropía de un sistema cerrado no puede ser reducida, sólo puede aumentar o, posiblemente, mantenerse sin cambios. Algo similar ocurre con los sistemas de software, incrementando su complejidad al ser éstos modificados o extendidos. El incremento en la complejidad al momento de modificar o extender un sistema de software significa mayor número de fallas junto con retrasos y

sobrecostos. La alternativa a esto es reducir la entropía mediante la reingeniería y el mantenimiento continuo del sistema. En el caso extremo, se puede llegar a tal desorden, que se vuelve económicamente injustificable continuar manteniendo el sistema.

Muchos de los ejemplos anteriores son muestras de sistemas que son cancelados al no satisfacer las necesidades actuales de una empresa y significar excesivos costos durante su mantenimiento y adaptación a las nuevas necesidades del negocio. Sin embargo, según veremos en el capítulo 3, no todo es negativo. Un aspecto importante para evitar estos problemas y lograr sistemas que puedan ser mantenidos de manera efectiva, es seguir un buen proceso de desarrollo de software utilizando tecnologías y herramientas adecuadas.

1.2.1 Confiabilidad del software

La **confiabilidad** (*reliability*) de un sistema de software describe qué tan correcto y a prueba de fallas sea un sistema. La confiabilidad depende de la cantidad de errores que un sistema posea, cuanto menor sea ésta, mayor será su confiabilidad. Otro concepto altamente relacionado con la confiabilidad es la **robustez** (*robustness*) del software, la cual describe qué tan bien el sistema responde ante circunstancias anormales, como por ejemplo, datos de entrada incorrectos. Aunque es casi imposible lograr software 100% confiable y robusto, o sea, software perfecto, es posible reducir el número de fallas o defectos que contenga. Tomemos el caso de los sistemas de control de tráfico aéreo de Estados Unidos, teniendo como requisito que jamás dejen de funcionar por más de tres segundos al año y que la probabilidad de defectos catastróficos en los sistemas de las aerolíneas civiles sea menor a 10^{-9} (1/1 000 000 000) por hora. El problema es cómo encontrar las fallas y comprobar que cumplan con la probabilidad establecida, dado que ocurrirán muy raras veces. Por ejemplo, para encontrar un error con probabilidad 10^{-9} se tendría que ejecutar un programa de software varios múltiplos de 10^9 . De igual modo, si se requiere detectar un error al año, habría que correr el programa aproximadamente 100 000 años (10^9 horas/8760 horas por año, que equivale a cerca de 10^5 años).

No todas las fallas se deben a problemas similares. Se considera que un tercio de todos los errores (*bugs*) son defectos conocidos como de "5000 años" MTBF—(Mean Time Between Failures), en otras palabras, se produce en promedio un error cada 5000 años. Aparte de la dificultad para encontrar los defectos, está el esfuerzo de eliminarlos, MTTR (Mean Time To Repair), y el desafío adicional de no agregar más. En general, se puede estimar el número de defectos *latentes* en un sistema típico de software de acuerdo con su tamaño, el cual puede ser medido en líneas de código o en **puntos de función**,⁴⁷ donde éstos son una medida de la funcionalidad ofrecida por un sistema. En el caso de puntos de función, un cálculo aproximado es elevar el número de puntos de función a la 1.2 potencia.⁴⁸ Por ejemplo, se reportó que Windows 95 contenía alrededor de 8 millones de líneas de código de acuerdo con un cálculo conservador, o bien, 80 000 puntos de función, según un cálculo equivalente aproximado de 100 líneas de código por punto de función. Esto sugiere que Windows 95 produjo inicialmente alrededor de 765 000 errores latentes, es decir que tendrían que corregirse durante las etapas de pruebas. Digamos que durante las pruebas se eliminaran 99% de los errores, aún quedarían 7650 para ser encontrados después de lanzar el producto al mercado. Si consideramos que, según lo repor-

tado, Windows 2000 contiene 30 millones de líneas de código, equivalente a unos 300 000 puntos de función, según nuestra equivalencia, el número de errores latentes sería alrededor de 3 737 000. ¡Estos números aterran a cualquiera! Nuevamente, si se eliminara 99% de los errores aún quedarían 37 370 para ser corregidos. Si recordamos, Microsoft se tomó más tiempo de lo previsto en la etapa de prueba, ocasionando demoras en la salida al mercado de Windows 2000.

1.2.2 Software suficientemente bueno

En general, no existe una sola medida que nos diga qué tan bueno es un sistema de software. Un sistema se puede considerar correcto o exitoso cuando satisface y posiblemente excede los deseos de los usuarios al momento de utilizarse. También se considera exitoso si el sistema se termina a tiempo, de manera económica y permitiendo modificaciones y extensiones posteriores.

De manera general se pueden caracterizar aspectos externos e internos para el éxito de un sistema:

- ▶ Como **factores externos**, los usuarios esperan resultados rápidos, facilidad de aprendizaje, confiabilidad, robustez, etcétera.
- ▶ Como **factores internos** los administradores esperan que el sistema sea fácil de modificar, extender, comprender, verificar y migrar a diferentes ambientes de cómputo, etcétera.

De todos estos aspectos, lo que más cuantitativamente se puede medir es la cantidad de errores o defectos que un sistema contenga o al menos que se observen. Dado que en la práctica no es posible garantizar software perfecto, con cero defectos, la pregunta que debiéramos hacernos es: ¿cuándo es un sistema *suficientemente bueno*? Esto implica otra pregunta, ¿hasta cuándo vale la pena invertir esfuerzo en eliminar defectos?

Para entender mejor las implicaciones de estas preguntas, se deben considerar los aspectos que más afectan el negocio del software: **riqueza funcional** (*feature richness*), **calidad** y **tiempo/costo** (*schedule/cost*) como se muestra en la figura 1.1. Por lo general no es posible fortalecer cada uno de estos elementos sin afectar a los demás.

En el caso de **software suficientemente bueno** típicamente se sacrifica la calidad del sistema, para así reducir el tiempo y costo, maximizando la riqueza funcional. La incorporación frecuente de nueva funcionalidad a un producto es finalmente la base del negocio de una compañía de software, ya que por lo general nadie compraría una nueva versión de software que simplemente elimina errores anteriores. Para ilustrar esto podemos tomar como ejemplo la competencia entre Netscape y Microsoft para el desarrollo de nuevos visualizadores (*browsers*) para el web a finales de 1990, cuando se buscaba sacar al mercado cada vez más rápidamente la siguiente versión del visualizador, llegando a tener ciclos de desarrollos de sólo unos pocos meses. Esto obviamente afectó la calidad de los visualizadores de web, significando muchos defectos, descubiertos únicamente por los propios usuarios encargados de llevar a cabo las pruebas del software.

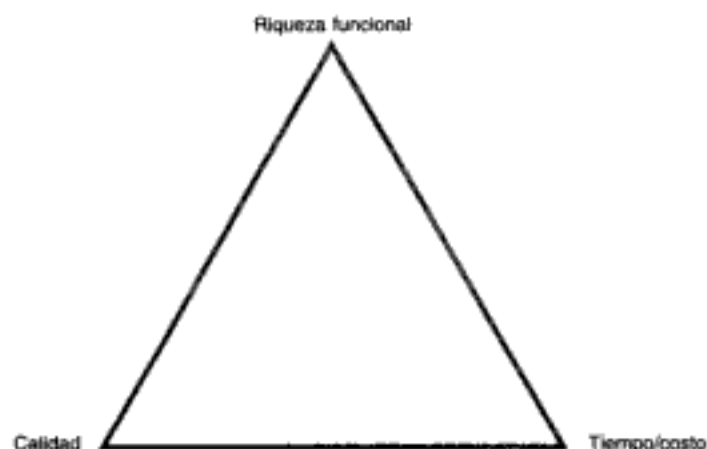


Figura 1.1 Diagrama de calidad versus riqueza funcional versus tiempo/costo del software. Cada esquina del triángulo corresponde al fortalecimiento del elemento correspondiente, la cual, en el caso del tiempo/costo significa entrega de software en menor tiempo y costo.

En 1997, errores de seguridad, tanto en el buscador de Netscape como en el de Microsoft, hicieron que las compañías tuvieran que quitarlos temporalmente del mercado para poder revisarlos. Situaciones similares son comunes en la actualidad. Lo peor del caso es que ante la opción de escoger entre un software de buena calidad, con pocos defectos o una nueva versión, pero que pudiera tener muchos defectos, la gente por lo general prefiere la nueva. En cierta manera nosotros mismos impulsamos el deterioro en la calidad del software comercial. La famosa frase "más rápido, más barato, mejor", realmente significa "suficientemente rápido, suficientemente barato, suficientemente bueno". ¿Qué tan grave es esto? Bastante grave si consideramos que en la "letra chica" de los paquetes de software que adquirimos se menciona que el fabricante del software no se hace responsable por ningún daño ocasionado por el uso del mismo y, más aún, no existe en la actualidad ninguna ley, incluso en Estados Unidos, que permita demandar a una empresa por software defectuoso.

1.2.3 La bala de plata

En 1975, Fred Brooks, el padre del Sistema 360 de IBM, escribió su famoso libro *The Mythical Man-Month* (vuelto a publicar en 1995 en su vigésimo aniversario),⁴⁹ un clásico muy vigente donde se resalta la problemática de los sistemas de software. Este libro transmite la experiencia de Brooks como director del proyecto del sistema operativo OS/360 de IBM escrito en la década de los sesenta. En su apogeo, este proyecto llegó a contar con alrededor de 1000 personas, incluyendo programadores, documentadores, operadores, secretarías, administradores y demás. Entre 1963 y 1966 se calculó que se utilizaron para el diseño, construcción y documentación del sistema 5000 años-persona, lo cual, utilizando un cálculo de 100 líneas de código por persona al mes sería equivalente a $5000 \times 100 \times 12 = 6$ millones de líneas de código, un sistema de tamaño nada despreciable, incluso para los estándares actuales. Vale la pena destacar que el sistema OS 360 es el precursor de MVS/370 y MVS/390 utilizados hasta la fecha por muchos de los *mainframes* de IBM. Relacionado a estos conceptos está la famosa ley de Brooks, la cual resalta que cuanto más gente se

agregue a un proyecto de software que esté retrasado, más se demorará el proyecto hasta el punto de nunca acabarse. ¿Suenan familiares? Esto se muestra en la gráfica de la figura 1.2.

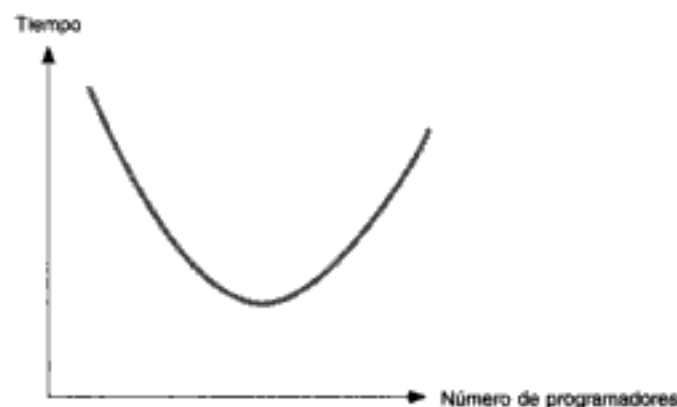


Figura 1.2 Ley de Brooks: cuanto más se aumente el número de trabajadores en un proyecto retrasado, mayor será el tiempo de desarrollo.

La razón para el retraso incremental se basa en que las necesidades de personal, se calculan inicialmente, según una simple medida de líneas de código producidas por una persona al mes (el estándar actual es entre 100 y 1000 líneas de código por persona al mes), lo cual significaría que si un proyecto requiere 10 millones de líneas, pudiera hacerse un simple cálculo que divida entre los meses y personas necesarias para lograr esa cantidad. Aplicando este mismo cálculo, si llega a haber un retraso, se pudiera agregar más personas para completar el sistema en menor tiempo. Sin embargo, en la realidad esto no funciona. La razón principal es que a las nuevas personas hay que entrenarlas y explicarles el proyecto. Significa que se quita temporalmente personal ya involucrado en el proyecto para trabajar en conjunto con las nuevas contrataciones, causando aún más demoras en el proyecto.

Aparte de esta famosa ley, un legado importante de Fred Brooks fue el célebre, aunque controversial artículo "No Silver Bullet",⁵⁰ en el cual se discuten los desafíos más importantes de la ingeniería de software. En este artículo Brooks discute la diferencia entre la *esencia* correspondiente a los aspectos inherentes y abstractos relacionados a la naturaleza del software y los *accidentes* correspondientes a las dificultades existentes para producir software. De manera adicional, Brooks deja un desafío a las futuras generaciones: "según miramos al horizonte de una década, no vemos ninguna bala de plata. No existe un solo desarrollo, en la tecnología o técnica de administración, que por sí mismo prometa incluso una mejora de un orden de magnitud en productividad, confiabilidad, simplicidad, dentro de una década". En otras palabras, Brooks sugiere que no hay nada en el horizonte que permita mejorar la calidad del software de manera drástica, dados los avances tecnológicos de ese momento y los que están en puerta. Esta aseveración ha sido muy discutida, sin embargo, es extremadamente relevante para la ingeniería de software: ¿cuándo lograremos fabricar software de buena calidad y cuándo podremos eliminar los defectos del software garantizando su corrección?

1.2.4 Ciclo de vida del software

Para apreciar un poco más el problema del software, es necesario considerar su *ciclo de vida*, correspondiente a las diversas etapas por las cuales debe pasar un sistema, comenzando con la formulación de un problema, seguido por la especificación de requisitos, análisis, diseño, implementación o codificación, integración y pruebas del software. Después viene una fase operacional durante la cual se mantiene y extiende el sistema.

Todo desarrollo de software incluye aspectos *esenciales*, como la planeación, correspondiente a las etapas de requisitos, análisis y diseño, junto con aspectos *secundarios* o *accidentales*, como codificación y pruebas. Según Brooks, existe una regla empírica (*thumb rule*) que dice que para el desarrollo de un proyecto de software se debe asignar $1/3$ del tiempo a la planeación, $1/6$ a codificación, $1/4$ a pruebas de componentes y $1/4$ a pruebas del sistema, como se muestra en la figura 1.3. En otras palabras, la mitad del esfuerzo ($2/4$) son dedicados a pruebas, lo cual significa en total $2/3$ a lo accidental, mientras que sólo $1/3$ a lo esencial. Esto es preocupante ya que dedicamos menor tiempo a lo esencial que a lo accidental.

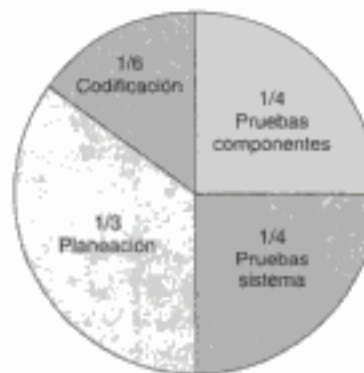


Figura 1.3 Estimación general del tiempo dedicado al desarrollo de un proyecto de software.

De manera adicional, la mayor parte de los avances en la productividad del software se han dado históricamente gracias a herramientas, ambientes y lenguajes de programación que reducen el esfuerzo en el desarrollo de las tareas secundarias o accidentales. La premisa de Brooks de mejorar por un orden de magnitud los aspectos esenciales del desarrollo de software significaría perfeccionar por 10 el esfuerzo dedicado a lo esencial, o sea, de $1/10$ a $10/10$. Esto se lograría dedicando 10 veces más de tiempo a lo esencial que a lo accidental. ¿Entonces, cómo hacer para mejorar tan radicalmente la productividad del software? Aunque no todos coinciden en la gravedad de la situación, la gran mayoría de los expertos en el área de ingeniería de software están de acuerdo en que se requiere de un proceso de desarrollo de software eficiente y sistemático. Por ejemplo, Ed Yourdon, autor de los libros *Decline and Fall of the American Programmer*⁵¹ y *Rise and Resurrection of the American Programmer*⁵² discute que aunque no hay un solo desarrollo que sea la "bala de plata", se pueden considerar varios aspectos que juntos pueden dar ese incremento en el orden de magnitud. Estos aspectos incluyen seguir buenos procesos y metodo-

logías, utilizar tecnología de objetos y herramientas avanzadas, incluir reuso y métricas de software, y considerar de manera especial el aspecto humano (*peopleware*).

En los capítulos 2 y 3 se discuten los temas de tecnología orientada a objetos y proceso para el desarrollo de software, aspectos relevantes para mejorar la calidad de los sistemas de software.

RESUMEN

En este capítulo se presentó una introducción a la problemática del software, y se explicaron los diferentes tipos de costos asociados con el software: directo, indirecto y oculto. Asimismo, se expusieron varios ejemplos de sistemas cuyas fallas han ocasionado pérdidas multimillonarias e incluso vidas humanas. Se presentaron ejemplos de sobrecostos y cancelaciones de proyectos de gran tamaño. Se explicaron las razones de los costos ocultos, relacionados directamente con la complejidad de los sistemas de software. Se analizó el estado actual del mercado de software en relación con lo que se conoce como software suficientemente bueno. Se especuló sobre si algún día tendremos la "bala de plata" que permita lograr software de alta calidad, algo que se espera alcanzar mediante tecnologías y procesos de software avanzados.

REFERENCIAS

1. <http://catless.ncl.ac.uk/Risks/>
2. "Software Runaways", Prentice Hall, 1998.
3. "SafeWare: System Safety and Computers", Addison-Wesley, 1995.
4. <http://www.cpsr.org/>
5. <http://catless.ncl.ac.uk/Risks/5.65.html>, The Risks Digest 5(65), nov 25, 1987.
6. <http://catless.ncl.ac.uk/Risks/1.25.html>, The Risks Digest 1(25), nov 1, 1985.
7. Neumann, P., Risks to the public on computer systems. ACM SIGSOFT Software Engineers Notes, 11(2), abr 1986.
8. Levesan, N., Turner C., An investigation of the thesac-25 accidents, IEEE COMPUTER , VOL 26, No. 7, jul 1993, pp. 18-41.
9. <http://catless.ncl.ac.uk/Risks/21.49.html>, The Risks Digest 21(49), jun 18, 2001.
10. <http://catless.ncl.ac.uk/Risks/7.20.html>, The Risks Digest 7(20), jul 11, 1988.
11. <http://catless.ncl.ac.uk/Risks/9.64.html>, The Risks Digest 9(64), feb 1, 1990.
12. <http://catless.ncl.ac.uk/Risks/9.64.html>, The Risks Digest 9(64), feb 1, 1990.
13. Chaesson E., Early results from the Hubble telescope, Scientific American, jun 1992, pp. 44-51.
14. <http://www.fas.org/spp/starwars/gao/im92026.htm>, *Performance of the Patriot Missile in the Gulf War*, Activities of the House Committee on Governmental Operations, 102nd Congress, Reporte 102-1086, abr 7, 1992, pp. 179-188.
15. BusinessWeek, sept 21, 1998.
16. <http://www.byte.com/art/9503/sec13/art1.htm>
17. <http://catless.ncl.ac.uk/Risks/17.46.html>, The Risks Digest 17(46), nov 20, 1995.
18. http://www.esa.int/export/esaCP/Pr_33_1996_p_EN.html
19. <http://catless.ncl.ac.uk/Risks/17.95.html>, The Risks Digest 17(95), abr 1, 1996.
20. <http://catless.ncl.ac.uk/Risks/18.01.html>, The Risks Digest 18(1), abr 5, 1996.
21. <http://catless.ncl.ac.uk/Risks/18.14.html>, The Risks Digest 18(14), may 22, 1996.
22. <http://catless.ncl.ac.uk/Risks/18.16.html>, The Risks Digest 18(16), jun 1, 1996.
23. http://securities.stanford.edu/news-archive/2002/20021220_Headline05_Staff.htm.
24. BusinessWeek, sep 21, 1998.
25. BusinessWeek, dic 6, 1999.
26. <http://catless.ncl.ac.uk/Risks/19.94.html>, The Risks Digest 19(94), sep 4, 1998.
27. BusinessWeek, dic 6, 1999.
28. BusinessWeek, dic 6, 1999.

29. <http://overlawyered.com/archives/99nov1.html#991103a>
30. <http://catless.ncl.ac.uk/risks/20.65.html>, The Risks Digest 20(65), nov 21, 1999.
31. <http://www.y2k.gov/>
32. <http://www.pa.press.net/tech/year280797.html>, PA NewsCentre, ago 8, 1997.
33. <http://www.herald.com/business/docs/040358.htm>, The Miami Herald, jul 28, 1997.
34. <http://catless.ncl.ac.uk/Risks/21.25.html>, The Risks Digest 21(25), feb 21, 2001.
35. http://www.standishgroup.com/sample_research/chaos_1994_1.php, The CHAOS Report, Standish Group, 1994.
36. <http://catless.ncl.ac.uk/Risks/7.68.html>, The Risks Digest 7(68), oct 31, 1988.
37. http://www.scit.wlv.ac.uk/ucm_1995/cbe/cases/case_02/NINE.htm
38. <http://catless.ncl.ac.uk/Risks/9.50.html>, The Risks Digest 9(50), dic 3, 1989.
39. http://www.standishgroup.com/sample_research/chaos_1994_3.php, The CHAOS Report, Standish Group, 1994.
40. <http://catless.ncl.ac.uk/Risks/6.16.html>, The Risks Digest 7(68), dic 24, 1991.
41. <http://catless.ncl.ac.uk/Risks/9.50.html>, The Risks Digest 9(50), dic 3, 1989.
42. <http://catless.ncl.ac.uk/Risks/9.50.html>, The Risks Digest 9(50), dic 3, 1989.
43. <http://catless.ncl.ac.uk/Risks/9.50.html>, The Risks Digest 9(50), dic 3, 1989.
44. http://www.standishgroup.com/sample_research/chaos_1994_3.php, The CHAOS Report, Standish Group, 1994.
45. The Wall Street Journal, abr 30, 1998.
46. Lehman, M.M. & Belady, L. Program Evolution: Processes of Software Change, London: Academic Press, 1985.
47. <http://www.ifpug.org/>
48. Capers Jones, "Software estimating rules of thumb", Computer, v.29 n.3, p.116-118, mar 1996.
49. Frederick Brooks, "The Mythical Man-Month", Addison-Wesley, 1995.
50. Frederick Brooks, "No Silver Bullet", Memorias de la Décima Conferencia en Computación Mundial IFIP, editada por H.J. Kugler, 1986, pp 1069-76.
51. Edward Yourdon, "Decline & Fall of the American Programmer", Prentice Hall, 1993.
52. Edward Yourdon, "Rise & Resurrection of the American Programmer", Prentice Hall, 1996.

Tecnología orientada a objetos

Un aspecto fundamental de este libro es la tecnología orientada a objetos. Para apreciar la importancia de esta tecnología discutiremos sus mitos, realidades y los aspectos esenciales que la distinguen de la tecnología tradicional. De manera adicional, se describirán la motivación y conceptos básicos de esta tecnología, junto con una reseña de los lenguajes orientados a objetos más importantes.

2.1 Conceptos básicos

La orientación a objetos es un buen ejemplo de cómo los pequeños detalles tecnológicos pueden resultar tan significativos. Estos detalles se aprecian mejor si se contrastan con la programación tradicional, como veremos a continuación.

2.1.1 Conceptos de la programación tradicional

En la tecnología tradicional o programación tradicional, también conocida como **programación estructurada**, un programa o aplicación consta de múltiples datos y funciones “globales”, como lo muestra la figura 2.1. El término “global”

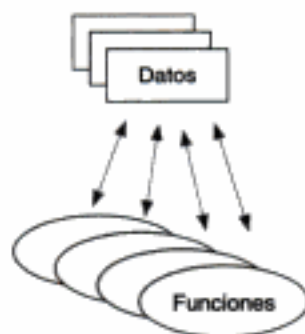


Figura 2.1 Programación tradicional o estructurada: datos y funciones globales.

describe el hecho que todos los datos o funciones son “visibles” en todo el programa y, por lo tanto, pueden ser llamados desde cualquier ubicación en la aplicación.

Esta forma de programación estructurada tiene sus orígenes en las primeras computadoras modernas basadas en la arquitectura *Von Neumann*,¹ donde las instrucciones de un programa se guardaban en memoria creando el concepto de **programa almacenado**. Las instrucciones de un programa, definidas dentro de funciones o procedimientos, eran ejecutadas por el procesador de manera secuencial afectando los datos del programa, los cuales eran almacenados en otras secciones de la memoria. Esta arquitectura es similar a la que se utiliza actualmente en la mayoría de las computadoras personales. Debido a esta separación de funciones y datos en la memoria, se ha desarrollado un gran número de lenguajes de programación que explotan este concepto. Sin embargo, este tipo de programación tiene dos problemas principales:

- a) El primero es obligar a un programador a que organice su programa de acuerdo con la arquitectura de la computadora, en otras palabras, que piense como la máquina. Debería ser lo opuesto.
- b) El segundo es que al estar separados de las funciones, los datos se vuelven globalmente visibles para poder ser llamados. Dada esta situación, cualquier cambio en la estructura de alguno de los datos pudiera llegar a requerir la modificación de todas las funciones del programa en correspondencia con los cambios en los datos.

Un buen ejemplo de los inconvenientes de la programación estructurada es el problema del año 2000, donde un dato tan pequeño como la fecha, al ser modificado de dos a cuatro dígitos, potencialmente pudo ocasionar costos de miles de millones de dólares. El problema radicaba en que los programas afectados, típicamente aplicaciones bancarias y nóminas, utilizaban la fecha de manera continua y desde todas sus funciones. Por lo tanto, fue necesario hacer modificaciones extensas a lo largo de todo el programa, lo cual significó largos períodos de mantenimiento.

2.1.2 Conceptos de la programación orientada a objetos

A diferencia de la programación tradicional, la programación orientada a objetos define una estructura de más alto nivel llamada **objeto**, que ofrece dos ventajas sobre la programación tradicional:

- a) La primera es permitir al programador que organice su programa de acuerdo con abstracciones de más alto nivel, siendo éstas más cercanas a la manera de pensar de la gente. En otras palabras, los objetos son las unidades de representación de las aplicaciones, por ejemplo, cuentas de banco, reservaciones de vuelos, etcétera.
- b) La segunda es que los datos globales desaparecen, siendo éstos junto con las funciones parte interna de los objetos. Por lo tanto, cualquier cambio en la estructura de alguno de los datos sólo debiera afectar las funciones definidas en ese mismo objeto y no en los demás.

En general, un programa orientado a objetos se define exclusivamente en términos de objetos y sus relaciones, como se muestra en la figura 2.2.

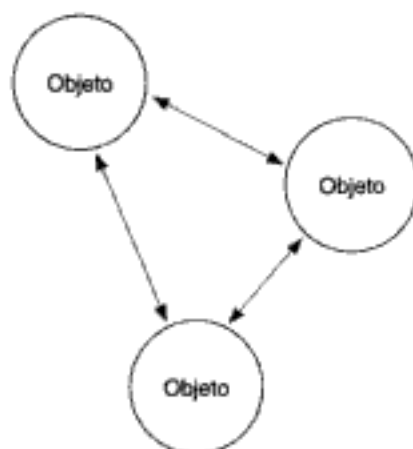


Figura 2.2 Programación orientada a objetos: objetos globales.

Los datos y funciones se guardan dentro de objetos, como se muestra en la figura 2.3. Los datos están ubicados en el centro del objeto (un concepto puramente ilustrativo), lo cual hace que un cambio en su estructura sólo afecte las funciones del mismo objeto, pero no al resto de la aplicación. En el capítulo 4, analizaremos todo lo relacionado con los objetos, sus datos y funciones.

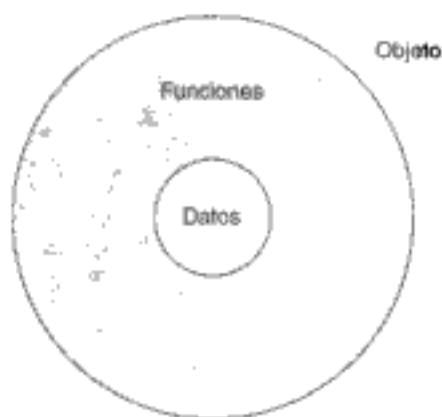


Figura 2.3 Programación orientada a objetos: objetos globales que contienen datos y funciones locales.

2.1.3 Revisión del problema del año 2000 (Y2K)

Posiblemente se hubieran evitado muchas dificultades si las aplicaciones afectadas por el problema del año 2000 se hubiesen hecho mediante la programación orientada a objetos. Siguiendo una buena programación orientada a objetos, la fecha se hubiera representado mediante un objeto **Fecha** como se mues-

tra en la figura 2.4. Los objetos del resto de la aplicación se relacionan únicamente con el objeto Fecha y no con su estructura interna que contiene el día, mes y año.

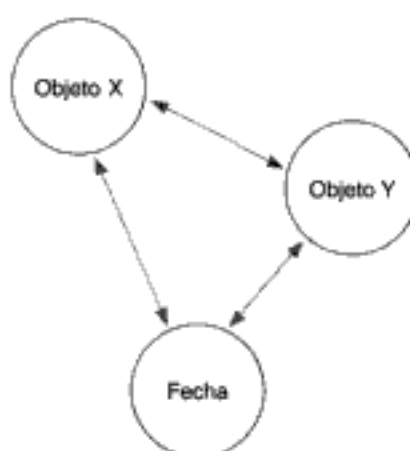


Figura 2.4 El objeto Fecha el cual internamente contiene datos que guarda la información correspondiente al día, mes y año.

Al llegar el año 2000 y reconocer la deficiencia en los dos dígitos correspondientes al año, en la estructura interna del objeto Fecha, se habrían cambiado éstos de dos a cuatro dígitos. Esto sólo habría afectado las funciones propias del objeto de encargadas de modificar los datos correspondientes de la fecha, como se muestra en la figura 2.5.



Figura 2.5 Extensión de la estructura del año del dato Fecha, de 2 a 4 dígitos.

El resto de la aplicación no sufriría modificaciones y el diagrama de la figura 2.4 hubiese sido idéntico. Como consecuencia, ¡el mundo se hubiera ahorrado miles de millones de dólares! Es un pequeño cambio en un programa, aunque de grandes repercusiones. Por supuesto que aún con la tecnología orientada a objetos, un mal diseño no hubiese solucionado el problema. Sin embargo, ¡el esfuerzo para hacer malos diseños con la programación orientada a objetos, es mucho mayor que con la tradicional!

2.2 Programación y lenguajes orientados a objetos

La programación orientada a objetos va más allá de los aspectos discutidos hasta el momento. En esta sección se describen aspectos adicionales de la programación orientada a objetos:

- a) Los aspectos que mejoran la calidad de los sistemas.
- b) Las características esenciales para que un lenguaje se considere orientado a objetos.

2.2.1 Aspectos que mejoran la calidad de los sistemas

Existen aspectos adicionales que hacen que la programación orientada a objetos mejore la calidad de los sistemas. Estos aspectos son principalmente la *abstracción*, *modularidad*, *extensibilidad* y *reutilización*.

ABSTRACCIÓN

Uno de los mecanismos más importantes para el manejo de la complejidad del software es la **abstracción**, la cual consiste en elevar el nivel de las *representaciones* necesarias para un sistema de software, de manera que se reduzcan los detalles. Por ejemplo, la programación estructurada representa un sistema en términos de datos y funciones, mientras que la programación orientada a objetos representa un sistema mediante objetos omitiendo, en un alto nivel, los detalles correspondientes a datos y funciones. Cuanto más alto sea el nivel de la representación, menor será el número de elementos necesarios para representar un sistema completo y más fácil será el manejo de la complejidad. Visto de otra manera, aunque sería posible representar un programa en código binario, ninguna persona es capaz de comprender una aplicación partiendo de 0 y 1. Esto requeriría de programas mucho más extensos, a diferencia de los sistemas de software contruidos con lenguajes de programación de más alto nivel, los cuales reducen el número total de líneas de código. Con la programación orientada a objetos se definen dos niveles de abstracción: (i) el nivel más alto, el de los objetos, se utiliza para describir la aplicación; (ii) el nivel más bajo, el de los datos y las funciones, se usa para describir sus detalles. Este nivel inferior corresponde al único nivel de la programación tradicional. Además de simplificar la representación, el objeto como estructura básica sirve para separar el "qué" del "cómo", o sea, de los detalles. En la programación tradicional el "qué" y el "cómo" se resuelven simultáneamente.

MODULARIDAD

Otro aspecto esencial de una aplicación es la **modularidad**, la cual permite dividir un sistema en componentes separados. Al contar con abstracciones de más alto nivel, la modularidad de un sistema se logra con base en componentes, también de más alto nivel. Esto reduce el número final de componentes en un sistema y, a su vez, facilita su operación y mantenimiento. Con la orientación a objetos, la modularidad del sistema se basa en objetos, un nivel más alto que

los datos y funciones tradicionales. El número final de módulos u objetos es menor que el número de datos y funciones. Esto reduce la complejidad de la aplicación, ya que el programador piensa en menos componentes a la vez, descartando detalles innecesarios.

EXTENSIBILIDAD

Otro aspecto fundamental de un sistema es su **extensibilidad**, la cual corresponde a la facilidad en modificar un sistema durante el transcurso de su vida. Como se mencionó en el capítulo 1, es común que un sistema que se use se modifique. La necesidad de extensibilidad nos lleva a cuestionarnos, ¿cuándo se vuelve más costoso mantener un sistema de software que crear uno nuevo? Los sistemas compuestos por múltiples módulos facilitan esta extensibilidad dado que los cambios en el sistema se pueden, generalmente, reducir a cambios en módulos particulares y no en todo el sistema a la vez. Con la programación orientada a objetos, los cambios se dan a dos niveles: (i) modificación externa a un objeto y (ii) modificación interna de los objetos. Los cambios internos de los objetos afectan principalmente al propio objeto, mientras que los cambios externos de los objetos repercuten de mayor manera en el resto del sistema. Dadas las abstracciones de nivel más alto y la reducción en el número de entidades que representan en un sistema, se lograrán sistemas que serán más fácilmente modificados.

REUTILIZACIÓN

La **reutilización** o reúso de componentes es otro de los mecanismos importantes para administrar la complejidad del software. La reutilización reduce el tiempo de diseño, codificación y costo del sistema al amortizar el esfuerzo sobre varios desarrollos. Mediante la reutilización se aprovechan componentes o bibliotecas ya desarrolladas, logrando una mayor estandarización y simplificación en las aplicaciones. Tradicionalmente, los componentes o bibliotecas han existido por mucho tiempo, principalmente en aplicaciones numéricas y estadísticas mediante procedimientos y funciones reutilizables. Aunque es posible la reutilización por medio de lenguajes tradicionales, los lenguajes orientados a objetos aumentan sustancialmente las posibilidades de reutilización gracias a la mayor modularidad de los sistemas. En general, el mayor problema de la reutilización radica en construir componentes genéricos, sencillos, con interfaces bien definidas y que puedan utilizarse en varias áreas de aplicación. El esfuerzo para construir componentes genéricos es, por lo general, mucho mayor que para construir componentes específicos para una aplicación particular. Con la orientación a objetos, el objeto es la unidad de reúso más pequeña, lo que permite aprovechar definiciones similares de objetos dentro de la misma aplicación, o incluso entre distintas aplicaciones. Al agrupar objetos similares, se puede lograr la reutilización de componentes de más alto nivel. También se pueden aprovechar objetos con estructuras de datos y funciones similares, definiendo una sola vez los aspectos comunes y especializándolos en objetos adicionales. Todo esto es parte del mecanismo de **herencia** en la orientación a objetos. A un nivel más amplio existen **marcos de aplicación** (*frameworks*), donde una aplicación genérica en un dominio particular se especializa según las necesidades de diferentes empresas, algo que ha sido muy exitoso en el ámbito de la planificación de recursos empresariales (*ERP – Enterprise Resource Planning*). Al definir una

aplicación en términos suficientemente abstractos o generales, se puede, en teoría, especializar su comportamiento sin hacer ningún cambio en la estructura básica de los componentes y de la propia aplicación, lo que permite extender de manera radical su utilidad. Sería el elixir de la ingeniería de software crear nuevas aplicaciones sin escribir una sola línea de código, solamente integrando componentes ya existentes, como en la construcción de casas o puentes prefabricados. Dado que para lograr grandes niveles de reuso primero es necesario lograr niveles intermedios de reuso, existe en la actualidad un esfuerzo muy importante mediante **patrones de diseño** (*design patterns*), los cuales están encaminados a solucionar aspectos particulares de las arquitecturas de software.²

2.2.2 Características esenciales de los lenguajes orientados a objetos

En la sección anterior mencionamos los aspectos más importantes de la programación orientada a objetos que permiten mejorar la calidad del software. En esta sección describimos los conceptos básicos que hacen que un lenguaje sea considerado efectivamente orientado a objetos, la base del término *orientación a objetos*. Para ello, deben existir cuatro aspectos esenciales: *encapsulamiento*, *clasificación*, *generalización* y *polimorfismo*. En esta sección únicamente introducimos los conceptos, los cuales se describirán más extensamente y con ejemplos en el capítulo 4.

ENCAPSULACIÓN

La **encapsulación** o el **encapsulamiento** es el mecanismo básico de la programación orientada a objetos para ocultar los detalles internos del objeto de los demás objetos. El encapsulamiento permite distinguir entre la **interface** del objeto, o sea, los aspectos del objeto conocidos externamente, y su **implementación**, o sea, sus aspectos conocidos sólo internamente. La interface de un objeto corresponde a las interfaces de sus funciones, mientras que la implementación corresponde a los datos y código, o implementación de sus funciones, como se muestra en la figura 2.6. Esta separación es muy importante. Si nos re-



Figura 2.6 Un objeto da a conocer a los demás objetos sólo las interfaces de sus funciones, mientras que su implementación corresponde a la implementación de sus funciones y los propios datos.

ferimos al diagrama de la figura 2.2, el conocimiento de un objeto por otros objetos se da exclusivamente con base en su interface. Todo el detalle, al estar encapsulado, es desconocido por el resto de la aplicación, limitando el impacto de cualquier cambio interno del objeto. Obviamente, cualquier cambio en la interface del objeto podrá afectar potencialmente al resto de la aplicación. Sin embargo, el porcentaje de código dedicado a la interface es, en general, mucho menor que el total de líneas de código utilizadas para datos e implementación de las funciones. De tal manera se reduce la complejidad del sistema, protegiendo los objetos contra posibles errores y logrando de mejor manera extensiones futuras de éstos.

CLASIFICACIÓN

En la programación orientada a objetos, la **clasificación** permite organizar objetos de acuerdo con estructuras comunes. Los objetos con datos y funciones similares se clasifican como si fueran de una misma *clase*. Éstos podrán tener datos con valores distintos.

GENERALIZACIÓN

Así como los objetos pueden organizarse como miembros de una misma clase, podemos también organizar a las propias clases de los objetos de acuerdo con sus datos y funciones comunes. Estas organizaciones se conocen como de **generalización** o **especialización**. Mediante la generalización, las clases con estructura y comportamiento similar se pueden reutilizar en la definición de nuevas clases, siendo éstas más especializadas que las anteriores. Las clases anteriores más especializadas son conocidas como **subclases**, mientras que las más generales son **superclases**. El mecanismo para describir jerarquías de generalización de clases se conoce como **herencia** —término común en la orientación a objetos—, se dice que una subclase hereda de una superclase. La herencia puede ser **sencilla** (una subclase es heredera directa de una sola superclase), o **múltiple** (una subclase es heredera directa de múltiples superclases).

POLIMORFISMO

El *polimorfismo* es posiblemente el concepto más poderoso de la programación orientada a objetos, aunque también el más complicado. Mediante el **polimorfismo** se definen múltiples funciones con nombres e interfaces similares sólo que en distintas clases. Las funciones son implementadas de manera diferente en las clases. El polimorfismo es útil para extender la funcionalidad del sistema al definir nuevas clases aún desconocidas al momento de especificarlo. La idea general es definir un estándar de interfaces que deben seguir todas las clases, las existentes y las nuevas. Si hacemos una analogía con los sistemas de software que ejecutan en una misma computadora, veríamos que todo el tiempo aparecen nuevas aplicaciones que pueden ejecutarse, sin tener que modificar el software de la propia computadora. Para que todo funcione correctamente, el diseñador del nuevo software debe mantener un estándar en la interface de sus funciones que sea conocido y aceptado, aunque la implementación de las funciones sea obviamente distinta. Este concepto será descrito con mayor detalle y ejemplificado en el capítulo 4.

2.3 Lenguajes de programación

Al igual que los lenguajes de programación tradicionales varían en sus estructuras y flujos de control, de manera similar también cambian los lenguajes de programación orientados a objetos. A pesar que estos diferentes lenguajes están diseñados para un mismo tipo de programación, existen aspectos que hacen que ciertos lenguajes ofrezcan mejor apoyo que otros durante el desarrollo de un sistema de software. Por ejemplo, Java, el lenguaje que utilizaremos para los ejemplos en este libro, se caracteriza por cinco motivos principales: su uso de forma gratuita, su portabilidad, su simpleza de programación, el gran número de librerías o bibliotecas que han sido definidas de manera estándar, y el gran número de extensiones que incluye, como es el caso del Web. Estos aspectos serán descritos con mayor detalle en el capítulo 5. Sin embargo, no sería completa una descripción de la programación orientada a objetos sin mencionar algunos de los lenguajes de programación más importantes que han sido desarrollados. La tabla 2.1 muestra diversos lenguajes de programación, tradicionales y orientados a objetos, en orden cronológico.

Es importante señalar que los primeros lenguajes de programación orientados a objetos fueron creados hace varias décadas, antes que muchos de los lenguajes de programación tradicionales. También vale la pena resaltar que a partir de la década de 1980, la mayoría de los nuevos lenguajes son orientados a objetos.

Tabla 2.1 La tabla describe los lenguajes más importantes de la historia de la computación y hace énfasis en aquellos orientados a objetos.

Año	Lenguaje	Descripción	¿Es un lenguaje de programación orientado a objetos?
1957	FORTRAN	Fortran (FORMula TRANslator) fue el primer lenguaje de alto nivel y aún es el que más se utiliza para cálculos numéricos. Lo diseñó originalmente John Backus entre 1954 y 1957. Las versiones más recientes son FORTRAN 95 y FORTRAN 2000. ³	No
1959	LISP	Lisp (LIST Processing) fue diseñado por McCarthy entre 1956 y 1961. Existen diferentes extensiones, la más conocida es CommonLisp. Este lenguaje se utiliza principalmente en el área de inteligencia artificial. En un esfuerzo por hacer de LISP un lenguaje más moderno, se extendió en 1988 con orientación a objetos dando lugar a CLOS (Common LISP Object System), desarrollada en 1988 por David Moon (Symbolics), Daniel Bobrow (Xerox), Gregor Kiczales (Xerox) y Richard Gabriel (Lucid), entre otros. ⁴	No

Continúa

Tabla 2.1 *Continuación*

1959	COBOL	Cobol (COmputer Business Oriented Language) fue diseñado originalmente por Grace Hopper de la Armada americana a partir de 1952. Desde 1959, el diseño de COBOL fue continuado por un grupo de profesionales conocidos como CODASYL (COmference on DATA SYstems Languages). Este lenguaje fue creado principalmente para aplicaciones financieras, siendo el mayor culpable del problema del año 2000. Las versiones más recientes son COBOL-97 y COBOL 2002, que también contienen extensiones de orientación a objetos. ⁵	No
1960	ALGOL	ALGOritmic Language fue creado por J. Backus y P. Naur entre 1958 y 1960. Se le considera el primer lenguaje de propósito general para aplicaciones industriales y científicas. La última versión fue Algol68.	No
1962	SIMULA	El primer sistema con objetos fue B1000 en 1961, seguido por Sketchpad en 1962, el cual contenía clones o copias de objetos e instancias de objetos. Sin embargo, Simula I fue el primer lenguaje completamente orientado a objetos, estructurado mediante objetos y clases. Fue creado por Ole Dahl y Kristen Nygaard del Centro de Computación de Noruega (NCC, Oslo) en 1962. El lenguaje fue implementado por primera vez en 1964, diseñado como una extensión a Algol 60 haciendo posible simular eventos discretos. En 1967, se introdujo el lenguaje más general Simula 67 con un número mayor de tipos de datos, además del apoyo a objetos. Simula se estandarizó en 1977. ⁶	Sí
1962	PL/I	PL/I (Programming Language I) fue un lenguaje bastante complejo diseñado en IBM a partir de 1962 para su famoso System/360. PL/I (PL, numeral romano I) quería ser el lenguaje principal de desarrollo para los sistemas y aplicaciones de gran tamaño. Fue utilizado principalmente en la década de 1980 aunque aún existe. ⁷	No
1962	APL	APL (A Programming Language) fue diseñado por Ken Iverson a partir de 1962 y utilizado por IBM. El objetivo principal de este lenguaje era el apoyo a la programación matemática. La versión actual es APL 2000. ⁸	No
1964	BASIC	Este lenguaje fue inventado por los profesores John G. Kemeny y Thomas E. Kurtz de la Universidad de Dartmouth, Estados Unidos. El primer programa escrito en BASIC fue procesado el 1 de mayo de 1964. Entre sus dialectos más modernos se incluye VisualBasic, diseñado por Microsoft en 1991. ⁹	No
1968	Pascal	Este lenguaje fue inventado por Niklaus Wirth del Instituto Tecnológico Federal de Zurich entre 1968 y 1971. Pascal evolucionó el diseño de Algol, por lo que muchos años fue el lenguaje más utilizado como introducción a la programación en diversas universidades. Entre sus dialectos más modernos se incluye TurboPascal y ObjectPascal, como parte del ambiente de desarrollo Delphi de Borland. ¹⁰	No

Continúa

Tabla 2.1 Continuación

1972	Smalltalk	Creado por Alan Kay en Xerox Park, también diseñador de la primera computadora personal basada en programación orientada a objetos, FLEX de 1967 a 1968. La primera versión fue conocida como Smalltalk 72, cuyas raíces fueron Simula 67, siguiendo Smalltalk 76, versión totalmente orientada a objetos. En 1980, Smalltalk 80 fue la primera versión comercial del lenguaje, incluyendo un ambiente de programación orientado a objetos uniforme. El lenguaje Smalltalk ha influido sobre otros muchos lenguajes como C++ y Java. Smalltalk es un lenguaje aún muy utilizado. ¹¹	Sí
1972	Prolog	Prolog (PROgramming in LOGic) fue el progenitor de la programación lógica. Lo diseñó Robert A Kowalski de la Universidad de Edinburgo, Reino Unido, junto con Alain Colmerauer de la Universidad de Aix-Marseille, Francia. ¹²	No
1972	C	El lenguaje C fue diseñado por Ritchie y Thompson entre 1969 y 1973. Este lenguaje fue diseñado en forma paralela a los primeros desarrollos del sistema operativo Unix ¹³ . Una segunda etapa del desarrollo fue hecha por Kernighan y Ritchie entre 1977 y 1979, como adaptación a la extensión de Unix a múltiples plataformas. Es uno de los lenguajes más utilizados en la actualidad. ¹⁴	No
1977	CLU	CLU (CLUster) es un lenguaje diseñado por Barbara Liskov del MIT entre 1974 y 1977. El lenguaje utiliza conceptos básicos de la orientación a objetos aunque no es propiamente considerado como tal. ¹⁵	No
1980	Modula	La versión original de este lenguaje se llamó Modula-2 y fue desarrollada por Niklaus Wirth a mediados de la década de 1970 como descendiente directo de Pascal. Este lenguaje incluía concurrencia y ciertos aspectos de la orientación a objetos. La última versión conocida como Modula-3 fue diseñada por Luca Cardelli. Dada su simpleza se desconoce por qué no se ha tenido mayor éxito en su utilización. ¹⁶	Sí
1983	Ada	El lenguaje fue diseñado principalmente por Jean Ichibah del Departamento de la Defensa de Estados Unidos en 1977, para apoyar la programación a gran escala y promover la robustez del software. Se denominó como Ada en honor de Lady Ada Lovelace (1815-1852), considerada la primer programadora en la historia, amiga y confidente de Charles Babbage, considerado a su vez como el padre de la computación por su trabajo teórico hace un siglo y medio. Aunque la versión original de este lenguaje no era orientada a objetos, la última versión, Ada 95, sí lo es. Las versiones anteriores, incluyendo Ada 83, no se consideran orientadas a objetos. ¹⁷	Sí
1983	Objective-C	Este lenguaje fue diseñado por Brad Cox como una extensión a C pero con orientación a objetos. Contenía muchos aspectos inspirados en Smalltalk 80. Se hizo popular debido a su utilización en la computadora NeXT, incluyendo una interface bajo el ambiente NeXTSTEP, conocido luego como OpenStep, y actualmente adquirido por Apple como base para su nuevo sistema operativo MacOS X. ¹⁸	Sí

Continúa

Tabla 2.1 Continuación

1983	Beta	Beta fue desarrollado por Ole Lehrmann Madsen en la Universidad de Aarhus en Dinamarca. Beta es un lenguaje orientado a objetos inspirado por Simula con sintaxis similar a Pascal y a C. ¹⁹	Sí
1984	ML	ML (Meta Language) representa una familia de lenguajes funcionales propuesta originalmente en 1983 y diseñada entre 1984 y 1988 por Milner y Tofte. La versión actual, Standard ML 97, es una revisión modesta y simplificada del lenguaje. ²⁰	No
1985	C++	C++ fue diseñado por Bjarne Stoustrup de AT&T Bell Labs, entre 1982 y 1985. C++ agrega mecanismos de orientación a objetos al lenguaje de C, lo que lo hace un lenguaje híbrido. ²¹	Sí
1986	Eiffel	Se llama así en honor a la famosa torre de París. Lo diseñó Bertrand Meyer como un lenguaje orientado a objetos con una sintaxis similar a C. El diseño del lenguaje basado en Eiffel apoya un enfoque de ingeniería de software conocido como diseño por contrato. Aunque es un lenguaje muy sencillo y poderoso, nunca logró la aceptación lograda por C++ y Java. ²²	Sí
1986	Self	El lenguaje Self fue diseñado por David Ungar y Randall Smith en Sun Microsystems. Self es un lenguaje cuya sintaxis es similar a Smalltalk aunque sin incluir la noción de clases en el lenguaje. En lugar de instanciar clases, los objetos se obtienen de otros objetos (sus prototipos) por medio de copiado y refinado. ²³	Sí
1990	Haskell	Fue desarrollado por un comité (Hughes, Wadler, Peterson y otros) que lo bautizó así en honor a Haskell Brooks Curry, cuyo trabajo en lógica matemática sirve como fundamento para los lenguajes funcionales. Haskell fue altamente influido por Lisp, aunque fue extendido con algunos mecanismos de la orientación a objetos para ser más moderno. ²⁴	Sí
1992	Dylan	Dylan (DYnamic LAnguage) es un lenguaje orientado a objetos, originalmente desarrollado por Apple. Se parece mucho a CLOS y Scheme, aunque tiene influencia de Smalltalk y Self. ²⁵	Sí
1995	Java	Java, diseñado por Gosling de Sun Microsystems entre 1994 y 1995, es el lenguaje orientado a objetos más utilizado en la actualidad. Es sencillo y portátil, bastante similar a C++, aunque tomando ideas de Modula-3, Smalltalk y Objective-C. ²⁶	Sí
2000	C#	Este lenguaje, conocido como Sharp, fue diseñado por Microsoft. Es una extensión a C con orientación a objetos, inspirado en C++ y en particular en Java. El lenguaje evita muchos de los problemas de diseño de C++. ²⁷	Sí

RESUMEN

En este capítulo se ofreció una introducción a la tecnología orientada a objetos, comparándola con los conceptos de programación más tradicionales. Se expusieron las ventajas de este tipo de programación y se describieron las características principales de los lenguajes de programación que apoyan esta tecnología, los cuales se presentan desde una perspectiva histórica con la cual

se deja en claro que la orientación a objetos tiene sus orígenes en los primeros días de la programación moderna.

REFERENCIAS

1. von Neumann, John. 30 de junio de 1945. Primera versión de un reporte sobre la EDVAC. Contrato no. W-670-ORD-492, Moore School of Electrical Engineering, Univ. of Penn., Philadelphia. Reimpreso (en parte) en Randell, Brian; 1982. *Origins of Digital Computers: Selected Papers*, Springer-Verlag, Berlin Heidelberg, pp. 383-392.
2. Gamma, E., Helm, R., Johnson, R. y Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1995.
3. <http://www.ibiblio.org/pub/languages/fortran/>
4. <http://www8.informatik.uni-erlangen.de/html/lisp-enter.html>
5. http://www.legacyj.com/cobol/cobol_history.html
6. <http://java.sun.com/people/jag/SimulaHistory.html>
7. <http://www-3.ibm.com/software/awdtools/pli/plihome.html>
8. http://www.aplbooks.com/html/a_bit_of_apl_history.HTM
9. <http://www.kbasic.org/1/history.php3>
10. <http://www.oberon.ch/docu/history.html>
11. <http://www.smalltalk.org/>
12. <http://www.codebox.8m.com/prolog.htm>
13. <http://www.bell-labs.com/history/unix/>
14. <http://cm.bell-labs.com/cm/cs/who/dmr/chist.html>
15. <http://www.pmg.lcs.mit.edu/CLU.html>
16. <http://www.research.compaq.com/SRC/modula-3/html/home.html>
17. <http://www.adahome.com/>
18. <http://developer.apple.com/techpubs/macosx/Cocoa/ObjectiveC/index.html>
19. <http://www.daimi.au.dk/~beta/>
20. <http://www.smlnj.org/sml97.html>
21. <http://www.research.att.com/~bs/C++.html>
22. <http://burks.bton.ac.uk/burks/language/eiffel/index.htm>
23. <http://research.sun.com/self/>
24. <http://www.haskell.org/>
25. <http://www.dylanpro.com/DylanExchange.html>
26. <http://java.sun.com>
27. <http://www.hitmill.com/programming/dotNET/csharp.html>

Proceso de software

Un **proceso** define *quién hace qué, cuándo y cómo* para alcanzar cierto objetivo. En general, el éxito de las empresas u organizaciones depende en gran medida de la definición y seguimiento adecuados de sus procesos. En el caso de una empresa que se dedica al desarrollo de software, se requieren procesos especializados que abarquen desde la creación hasta la administración de un sistema de software. Como hemos visto en el capítulo 1, los sistemas de software pueden llegar ser extremadamente complejos. Para administrar la complejidad de tales sistemas, es necesario contar con modelos de procesos y tecnologías de software apropiadas. En este capítulo se describirá en qué consiste el proceso de software.

3.1 Modelo de proceso

Un **modelo de proceso** de software define cómo solucionar la problemática del desarrollo de sistemas de software. Para desarrollar el software se requiere resolver ciertas fases de su proceso, las cuales se conocen en su conjunto como el **ciclo de vida**¹ del desarrollo de software. Un modelo de proceso debe considerar una variedad de aspectos, como el *conjunto de personas, estructuras organizacionales, reglas, políticas, actividades, componentes de software, metodologías y herramientas utilizadas*.

Una creencia común, aunque equivocada, es la existencia de un sólo modelo de proceso aplicable a cualquier proyecto, pues el modelo de proceso depende del tipo particular de proyecto, por ejemplo:

- **Primer proyecto de su tipo.** Se crea la mayoría del software desde cero. Por ser la primera vez que se crea este tipo de proyecto, se requiere más tiempo para especificarlo y analizarlo. En un primer proyecto, la incertidumbre crea riesgos adicionales.
- **Segundo proyecto de su tipo.** Se busca agregar una nueva funcionalidad a un proyecto conocido. En general, el desarrollador tiende a excederse agregando demasiada funcionalidad en comparación con el proyecto anterior (esto se conoce como *featuritis*). El sistema se vuelve mucho más grande que el original causando retrasos en el sistema, como actualmente ocurre con muchos proyectos.

- **Variación de un proyecto.** Se extiende un sistema ya existente, lo cual involucra introducir componentes de software reutilizables como un marco de trabajo (*framework*), crear nuevos componentes o simplemente extender la aplicación existente mediante nueva funcionalidad. Dependiendo de la estrategia a utilizar, el modelo de proceso debe variar. Por lo general, el riesgo en este tipo de proyectos es mucho menor que en los primeros proyectos de su tipo.
- **Proyecto de reescritura de legado (*legacy*).** Se busca transformar o hacer una "reingeniería" de un sistema ya existente, desarrollado bajo tecnologías anteriores, a un sistema desarrollado bajo nuevas tecnologías, tales como las orientadas a objetos. Esta ha sido la situación más común para resolver el problema del año 2000. En lugar de remendar sistemas, se aprovechó para reescribirlos. El proyecto de reescritura de legado tiene varias características en común con otros tipos de proyectos, entre las que están la variación de un proyecto, por ya existir un proyecto anterior con funcionalidad similar, y un primer proyecto de su tipo, ya que se debe crear una nueva arquitectura sin contar con software reutilizable del proyecto anterior. Además, existen los riesgos relacionados con un primer proyecto, ya que se requiere el uso de tecnología nueva.
- **Proyecto de creación de software reutilizable.** Se busca crear uno o más componentes de software reutilizables. Este tipo de proyecto es muy similar a otros proyectos de desarrollo de software, donde es necesario incluir los requisitos y desarrollar el diseño completo del componente. Sin embargo, es diferente de otro tipo de sistemas, donde se deben considerar las necesidades de múltiples proyectos, de manera que se asegure que el diseño sea lo suficientemente general para ser útil en otras situaciones desconocidas. Esto requiere de mayores esfuerzos, razón por la cual, la mayoría del software existente no es reutilizable.
- **Proyecto de mejora de sistema o mantenimiento.** Se busca modificar los componentes básicos de un sistema para apoyar una nueva funcionalidad. Estos proyectos a menudo son relativamente pequeños, y afectan sólo parte del sistema. Se debe comprender bien qué componentes se tienen que mejorar y cómo repercuten estos cambios al resto del sistema.

Dada la variedad de tipos de proyectos, es necesario considerar los diferentes componentes de un modelo de procesos. Estos componentes son principalmente: la *arquitectura*, la *actividad*, los *métodos* y las *metodologías*, la *estrategia* y las *herramientas* para la administración de software, las cuales se describen a continuación.

3.1.1 Arquitectura

Una **arquitectura** de software define la estructura general de un sistema y varía de acuerdo con el tipo de sistema a desarrollarse. Así, puede estar basada en elementos sencillos o componentes prefabricados de mayor tamaño, y se especifica de acuerdo con los diferentes tipos de sistemas. A continuación se dan algunos ejemplos de familias de sistemas:

- **Transformación en lote (*batch*).** Son sistemas de transformación sobre un conjunto de entradas de valor constante, para generar un conjunto de salidas. Un ejemplo de un sistema de este tipo es un compilador.
- **Transformación continua.** Son sistemas de transformación sobre un conjunto de entradas de valor variable, que genera un conjunto de salidas que difieren en el tiempo. Ejemplos de éstos son los sistemas de control de señales.
- **Sistemas interactivos.** Son sistemas regidos por interacciones externas, por lo general, con un usuario. Estos sistemas son controlados por *manejadores de eventos*, encargados de procesar acontecimientos generados por el usuario, como un *click* del ratón o la presión de una tecla.
- **Simulación dinámica.** Son sistemas que simulan entidades del mundo real y evolucionan con el tiempo. Ejemplos de estos sistemas son simuladores de sistemas financieros, redes neuronales, etcétera.
- **Sistemas de tiempo real.** Son sistemas regidos por restricciones estrictas en el tiempo, que requieren garantías en el tiempo de respuesta. Ejemplos de estos sistemas son los controladores de procesos industriales y dispositivos de comunicación.
- **Administración de transacción.** Son sistemas para interactuar con bases de datos y que incluyen, por lo general, acceso *concurrente* y *distribuido* de múltiples usuarios. Ejemplos de estos sistemas son los de reservaciones de vuelos y los de control de inventario.

Además de depender del tipo de sistema a desarrollar, la selección de una arquitectura afecta aspectos como la *extensibilidad* del sistema (vea el capítulo 2). Por lo tanto, la arquitectura debe ser escogida de manera que minimice los efectos de cambios futuros en el sistema. Para ello, existen ciertas heurísticas que muestran la tendencia a cambios en varios elementos de un sistema, como se muestra en la tabla 3.1. Las **interfaces** representan los elementos gráficos, la **funcionalidad** son las reglas del negocio, los **datos** y **funciones** son los elementos internos de los **objetos**, correspondientes a las estructuras básicas de la orientación a objetos, mientras que la información representa el dominio del problema en una aplicación.

Tabla 3.1 Probabilidad de cambios futuros en el software, de acuerdo con el tipo de elemento de diseño.²

Elemento	Probabilidad de cambio
Interfaces	Alto
Funcionalidad	Alto
Datos	Medio
Funciones	Medio
Objetos	Bajo
Información	Bajo

Esta tabla resalta dos aspectos del desarrollo de software: (i) la arquitectura del sistema debe distinguir entre elementos con mayor y menor probabilidad de cambios y (ii) el desarrollo de software debe considerar un modelo de proceso, donde aquellos elementos de mayor probabilidad de cambio no “arrastren” a los más estables. Este tema lo discutiremos con mayor detalle en la sección “métodos y metodologías”.

3.1.2 Actividad

Una **actividad** es una unidad o paso básico de un proceso. En el proceso de software las actividades definen los pasos necesarios para lograr las metas y los objetivos; por ejemplo, especificar los requisitos del sistema. Las actividades deben: ser fáciles de definir y seguir; simplificar la comprensión del sistema; y ofrecer flexibilidad, precisión y extensibilidad. Las actividades básicas del proceso de desarrollo de software, conocidas como el *ciclo de vida* del software, son las siguientes:

- (i) *requisitos* para especificar los aspectos funcionales del sistema, que describen cómo interactuaría un usuario con la aplicación.
- (ii) *análisis* para dar al sistema una estructura o *arquitectura* robusta y extensible independiente del ambiente de implementación final.
- (iii) *diseño* para adoptar y refinar la arquitectura del sistema y adaptarla al ambiente de implementación específico.
- (iv) *implementación* para codificar el sistema.
- (v) *integración* para combinar componentes del sistema.
- (vi) *pruebas* para validar y verificar el sistema.
- (vii) *documentación* para describir los diversos aspectos del sistema.
- (viii) *mantenimiento* para extender la funcionalidad del sistema.

La tabla 3.2 muestra las actividades más importantes del ciclo de vida del proceso de software.

Tabla 3.2 Actividades del desarrollo de software

Actividad	Descripción
Requisitos	Se especifican las necesidades del sistema a desarrollar. La especificación de requisitos sirve como base para la negociación entre los desarrolladores y clientes del sistema, y también para planear y controlar el proceso de desarrollo.
Análisis	Se busca comprender los requisitos del sistema con el propósito de estructurar la arquitectura del sistema. Responde a la pregunta “qué” del sistema.
Diseño	Se transforma la arquitectura obtenida durante el análisis en una arquitectura especializada, donde se considera el ambiente de implantación particular del sistema. Obedece al “cómo” del sistema.
Implementación	Se expresa la arquitectura del sistema en una forma aceptable para la computadora, o sea, el código.
Integración	Se combinan los componentes creados de manera independiente para formar el sistema completo.

Continúa

Tabla 3.2 Continuación

Pruebas	Se verifica y valida el sistema a nivel de componentes individuales y su integración. Este es uno de los aspectos más críticos del desarrollo y debe desarrollarse de manera concurrente al resto de las actividades. Se busca descubrir cualquier defecto en los requisitos, análisis, diseño, implementación e integración. Las pruebas se hacen a varios niveles, desde funciones sencillas hasta el sistema completo.
Documentación	Se describen los aspectos sobresalientes de los requisitos, análisis, diseño, implementación, integración y pruebas. Esto servirá para usuarios externos e internos, aquellos encargados de mantener el sistema y extenderlo.
Mantenimiento	Se corrigen errores no encontrados durante el desarrollo y las pruebas originales del sistema. Se extiende el sistema si surgen nuevas necesidades.

La transición entre las distintas actividades debe ser natural, y que haya continuidad o *rastreabilidad* (*traceability*) de una actividad a la siguiente o a la anterior. A continuación describimos con mayor detalle cada actividad.

REQUISITOS

El **modelo de requisitos** tiene como meta definir y delimitar la funcionalidad del sistema de software. Sirve como base de negociación entre el desarrollador del sistema y el cliente, y debe reflejar los deseos de éste. Es esencial que los clientes que no tengan conocimientos en computación comprendan el modelo de requisitos para facilitar la interacción con ellos.

El modelo de requisitos gobierna el desarrollo de los demás modelos, donde éstos se deben basar en aquél. También sirve como base para elaborar las instrucciones de operación y los manuales, los cuales se tienen que redactar desde el punto de vista del usuario.

El desafío en la especificación de requisitos comienza cuando el experto debe comunicar los conceptos, lo que, en general, no es posible hacer adecuadamente de manera sencilla. Por ello, se provee de múltiples explicaciones, orales o escritas, que el desarrollador integra en una forma coherente. La especificación de requisitos es particularmente difícil cuando la información es incompleta, los expertos no pueden articular lo que saben, o no están seguros o, incluso, son incoherentes en su información.

ANÁLISIS

Después de desarrollar el modelo de requisitos y de que los usuarios del sistema o clientes lo aprueben, se puede continuar con el **modelo de análisis**, que tiene como objetivo construir una arquitectura capaz de resolver el problema bajo condiciones ideales. Esto significa desarrollar una estructura lógica del sistema, la cual debe ser estable y extensible. El análisis se enfoca en qué debe hacer el sistema, en lugar de cómo se supone que lo hará. El alcance del modelo de análisis está directamente relacionado con la naturaleza del problema. En el caso de un análisis orientado a objetos, se desea identificar los objetos y describir cómo interactúan entre sí.

DISEÑO

El propósito del **modelo de diseño** es extender la arquitectura de análisis. La razón para no hacer esta extensión durante el modelo de análisis se debe a que la propia aplicación controla la arquitectura del sistema y no las circunstancias existentes durante su implementación. En otras palabras, el modelo de análisis debe ser visto como un modelo conceptual y lógico del sistema, mientras que el modelo de diseño debe definir todo lo necesario para alcanzar el código final. Dado que los ambientes de implementación tienden a cambiar, es necesario guardar y congelar el modelo de análisis para un mantenimiento futuro, incluso después de terminar el diseño. El modelo de diseño se concentra en dos aspectos principales: *diseño de objetos* y *diseño de sistemas*, como se describe a continuación:

- **Diseño de objetos.** El modelo de análisis no es lo suficientemente formal, por lo cual, para llegar al código final, se debe refinar las estructuras de la arquitectura de análisis. Se debe especificar el detalle de cada clase, en otras palabras, las operaciones y atributos. Este aspecto se conoce como el *diseño de estructuras* o, de manera general, como el *diseño de objetos* en el caso de arquitecturas orientadas a objetos. El diseño de objetos incluye la selección de algoritmos y estructura de datos para satisfacer los objetivos de rendimiento y espacio. El modelo de análisis y el diseño de objetos tienen bastante en común, incluyendo conceptos, técnicas y notaciones similares. Como consecuencia, las mismas herramientas de desarrollo se utilizan para llevar a cabo ambas actividades. A menudo estas similitudes hacen difícil saber qué actividad se está llevando a cabo. Uno de los beneficios de la tecnología orientada a objetos es representar la solución como una consecuencia directa de la representación del problema, por lo cual la distinción entre análisis y diseño de objetos no es realmente crítica, a diferencia de otros enfoques más tradicionales.
- **Diseño de sistema.** Durante el análisis se considera un mundo ideal para el sistema. En la realidad este mundo ideal debe adaptarse al ambiente donde se implementará el sistema. Entre otros aspectos, se debe considerar los requisitos de rendimiento, uso de memoria, protocolos de comunicación, tiempo real, concurrencia, propiedades del lenguaje de programación, el sistema de manejo de base de datos, etc. Este aspecto se conoce como *diseño de sistema*, que define las decisiones estratégicas sobre cómo organizar la funcionalidad del sistema en torno al ambiente de implementación, incluyendo tanto hardware como software. Para adaptar el modelo de análisis al ambiente de implementación, es necesario identificar las restricciones técnicas bajo las cuales se tiene que construir el sistema. Además, el diseño del sistema debe apoyar aspectos como una terminación inesperada durante la ejecución del sistema por agotamiento de recursos o por fallas externas del hardware.

IMPLEMENTACIÓN

El **modelo de implementación** toma el resultado del modelo de diseño para generar el código final del sistema. Esta traducción debe ser relativamente sencilla y directa, ya que todas las decisiones importantes han sido hechas en las etapas previas. La especialización al lenguaje de programación, o base de datos,

describe cómo traducir los términos usados en el diseño a términos y propiedades del lenguaje de implementación. Muchas herramientas, como veremos más adelante, apoyan el proceso de generación automática de código a través de un proceso de *ingeniería hacia delante* (*forward engineering*), como en el caso de herramientas CASE basadas en UML. Sin embargo, la generación de código es parcial y requiere que el desarrollador la complete de manera manual. En el modelo de implementación, el concepto de *rastreabilidad* (*traceability*) también es importante, dado que al leer el código fuente se debe poder relacionarlo con los modelos de diseño y análisis.

- **Lenguajes de programación.** Aunque existen muchos tipos de lenguajes de programación, el uso de un lenguaje orientado a objetos facilita la implementación de un diseño orientado a objetos. La elección del lenguaje influye en el diseño, pero el diseño no debe depender de los detalles del lenguaje, de tal manera que si se cambia de lenguaje de programación no debe ser necesario el rediseño del sistema. Por razones de rastreabilidad, es deseable siempre tener una buena y fácil correspondencia entre objetos del modelo de diseño y objetos del lenguaje de programación.
- **Bases de datos.** En la actualidad, las bases de datos son parte integral de los sistemas de software. Aunque existen bases de datos orientadas a objetos, aún siguen siendo más populares las bases de datos *relacionales*. El modelo de implementación debe contemplar el diseño de base de datos en la generación del sistema final.

INTEGRACIÓN

El **modelo de integración** es un aspecto importante del desarrollo de software. En todo diseño es deseable mantener una buena modularidad en el sistema, de manera que el desarrollo actual, junto con las futuras extensiones, puedan hacerse con base en componentes independientes y no en la totalidad del sistema. Cuando esto ocurre, es necesario integrar los diversos componentes para obtener como resultado el sistema final.

PRUEBAS

El **modelo de pruebas** es el responsable de revisar la calidad del sistema. Este modelo consiste en la *validación* del sistema o *prueba de especificación* y la *verificación* o *prueba de resultado*. De manera adicional, el modelo de pruebas combina *pruebas de unidad* y *pruebas de integración*.

- **Validación.** Se prueba si la funcionalidad del sistema corresponde a la especificación del cliente. Se cuestiona si se está construyendo el sistema "correcto", en contraste con la verificación, donde se cuestiona si se está haciendo el sistema "correctamente". Durante el diseño e implementación, se revisa el sistema de acuerdo con las especificaciones de los modelos de análisis y requisitos.
- **Verificación.** Se prueba si se está construyendo el sistema "correctamente". La verificación debe comenzar lo antes posible, desde el nivel más bajo,

con la revisión de los componentes individuales, prosiguiendo con la integración de éstos hasta verificar el sistema completo. La especificación de verificación del sistema debe ser una extensión del modelo de requisitos, e integrarse en la arquitectura del sistema.

DOCUMENTACIÓN

La **documentación** se debe hacer durante la elaboración del sistema y no como una etapa final del mismo. Existen diferentes tipos de documentos que se deben generar como apoyo al sistema, cada uno tiene diferentes objetivos y está dirigido a distintos tipos de personas, desde los usuarios no técnicos hasta los desarrolladores más técnicos. Los siguientes son algunos documentos o manuales más importantes:

- ▶ **Manual del usuario.** Permite a un usuario comprender cómo utilizar el sistema.
- ▶ **Manual del programador.** Contiene información para que un desarrollador entienda los aspectos más relevantes de diseño.
- ▶ **Manual del operador.** Posibilita al operador del sistema comprender qué pasos debe seguir para que el sistema funcione bajo cierta configuración y con base en un ambiente de implementación particular.
- ▶ **Manual del administrador.** Permite que el encargado de administrar el sistema comprenda sus aspectos más generales, como son los modelos de requisitos y análisis.

Realmente no hay límite en cuanto al número y el detalle de la documentación, así como tampoco hay límite respecto a cuánto se puede extender y optimizar un sistema. El objetivo es mantener un nivel de documentación que sea útil y extensible.

MANTENIMIENTO

El **mantenimiento de un sistema** es la continuación del ciclo de vida, luego de haber completado, una primera versión de éste. Aunque parte del objetivo involucra resolver problemas, durante el mantenimiento se deben considerar las extensiones del sistema de acuerdo con las nuevas necesidades. De cierta manera, el mantenimiento significa seguir un nuevo ciclo de actividades de desarrollo, pero a partir de un sistema ya existente.

3.1.3 Métodos y metodologías

Los **métodos** definen las reglas para las transformaciones internas de las actividades, mientras que las **metodologías** definen el conjunto de métodos. Un método es un procedimiento que define tareas o acciones a realizar, donde cada tarea incluye condiciones de entrada y de salida que se deben satisfacer antes y después de completarse. Las diferentes metodologías varían en el alcance del apoyo:

- ▶ **Dominio de aplicabilidad.** Los métodos deben apoyar conceptos básicos que se consideren significativos para resolver el problema. Se deben utilizar los métodos en distintos dominios de aplicación, y emplearlos en sistemas basados en diferentes arquitecturas, incluyendo secuencial, concurrente, distribuido, e incluso en tiempo real.
- ▶ **Ciclo de vida.** Los métodos deben ajustarse al ciclo de vida del proceso, apoyando las distintas actividades, incluyendo la documentación. Deben explicar las suposiciones, las metas y los objetivos que originaron un resultado particular. Los métodos no deben contradecir el orden establecido para las actividades del modelo de proceso, sino proveer guías para llevarlas a cabo. El mantenimiento de un sistema también debe ser apoyado por los métodos.
- ▶ **Información recopilada.** Los métodos deben proveer técnicas para recopilar información de acuerdo con el proceso de desarrollo. Por ejemplo, si el proceso se basa en tecnologías orientadas a objetos, los métodos deben apoyar la identificación de objetos en el sistema. Por otro lado, si el proyecto tiene como objetivo crear componentes reutilizables, los métodos deben incluir técnicas para la obtención y verificación de estos componentes.
- ▶ **Extensibilidad.** Los métodos deben apoyar su propia extensibilidad, identificando qué aspectos del método pueden ser modificados por el desarrollador para adaptarlo a sus necesidades particulares; por ejemplo, formato de la documentación.
- ▶ **Modelos generados.** Los métodos deben permitir la generación de modelos a partir de la información recopilada por el método. Por ejemplo, si en cierto desarrollo se requiere un modelo de seguridad o uno de rendimiento, entonces los métodos deben considerar estos requisitos o poder extenderse para obtener la información deseada. Se debe evaluar esta capacidad, además del esfuerzo necesario para obtener tales resultados.
- ▶ **Manejo de consistencia.** Los métodos deben apoyar la integridad de los modelos generados, verificando y evitando errores de consistencia, además de incluir técnicas para detectar problemas. Esto significa que las herramientas que sólo apoyan la diagramación son muy limitadas como apoyo a métodos, ya que carecen de manejo de consistencia. Los métodos deben permitir desarrollo independiente, algo esencial para sistemas de gran tamaño con múltiples analistas y diseñadores.
- ▶ **Integración.** Los métodos deben ofrecer entradas y salidas bien definidas que permitan la integración de varios métodos, incluso pertenecientes a distintas metodologías. A veces es deseable aplicar diferentes metodologías a diversas actividades de desarrollo. Esto ocurre cuando ciertas metodologías son más apropiadas para ciertos aspectos del desarrollo, como análisis o diseño.
- ▶ **Escalabilidad.** Los métodos y herramientas correspondientes deben ser apropiados para el tamaño del problema a resolver. Un método necesita escalar hacia arriba o hacia abajo según las necesidades del proyecto.

- **Notaciones.** Es muy importante contar con una notación estandarizada para representar los modelos desarrollados, los cuales deben incluir elementos gráficos, de texto o alguna combinación de ambos. Una notación no es simplemente buena o mala, si no más o menos efectiva en comunicar los resultados. Una buena notación debe tener la suficiente expresividad para modelar conceptos a nivel del detalle deseado. Algunas notaciones tienen un vocabulario más extenso que otras; por lo cual, permiten mostrar más detalles. También se desea una notación que permita representar modelos con varios niveles de abstracción. Una buena notación también facilita su comprensión y aprendizaje, aunque tiene un subconjunto mínimo como apoyo a los principiantes. Las notaciones más pobres expresan grandes cambios semánticos con pequeños cambios en los símbolos. Las notaciones deben comunicar información de manera que minimicen el factor sorpresa. Como no siempre se tiene apoyo de herramientas para dibujar una notación, se buscan las que sean fáciles de dibujar.
- **Confianza.** Se debe tener confianza en los métodos y las herramientas correspondientes; para lo cual, se debe contemplar que éstos se mantendrán en el mercado y, que cuenten con capacitación y apoyo técnico.

Existe una gran variedad de métodos y metodologías en apoyo al proceso de software. A continuación, se describen las metodologías *estructuradas* o *tradicionales* y las *orientadas a objetos*.

ESTRUCTURADAS

Las **metodologías tradicionales** o **estructuradas** se enfocan principalmente en la descomposición *funcional* de un sistema. El objetivo es lograr una definición completa del sistema en términos de funciones, estableciendo los datos de entrada y salida correspondientes. Se conocen a estas metodologías como *análisis y diseño estructurado* (*SA/SD, Structured Analysis and Structured Design*).³⁻⁷ Durante las actividades de desarrollo se utilizan diferentes herramientas de modelado:

- **Diagramas de flujo de datos.** Sirven para modelar la transformación de datos entre funciones del sistema. Un diagrama de flujo de datos se compone de *procesos*, *flujo de datos*, *actores* (entidades externas) y *almacenamiento de datos*. Durante el análisis, los procesos del diagrama de flujo de dato (DFD) se descomponen hasta convertirse, durante el diseño, en funciones de programación, creando una *carta (chart) estructurada* del sistema.
- **Diagramas de transición de estados.** Sirven para modelar el comportamiento en el tiempo. Los diagramas de transición describen el efecto de eventos externos en los procesos y funciones.
- **Diagramas de entidad-relación.** Sirven para modelar un almacenamiento de datos.

ORIENTADAS A OBJETOS

Las metodologías **orientadas a objetos** se enfocan principalmente en el modelado de un sistema en términos de objetos. A diferencia de las metodologías

estructuras, se identifican inicialmente los objetos del sistema para luego especificar su comportamiento. Existe un gran número de metodologías orientadas a objetos, como se muestra en la tabla 3.3. Durante las actividades de desarrollo se utilizan diferentes herramientas de modelado:

- **Diagramas de clases.** Sirven para describir los componentes esenciales de la arquitectura de un sistema. A diferencia de los diagramas de flujo de datos, los diagramas de clases muestran relaciones de asociación entre clases y no flujo de datos entre ellas.
- **Diagramas de casos de uso.** Especifican un sistema en término de su funcionalidad. A diferencia de las metodologías estructuradas, los diagramas de casos de uso no son descompuestos en funciones de programación.
- **Diagramas de transición de estado.** Describen los cambios de estado en los objetos, siendo equivalentes sus similares en las metodologías estructuradas.
- **Diagramas de secuencia.** Sirven para describir los aspectos dinámicos del sistema, mostrando el flujo de eventos entre objetos en el tiempo.
- **Diagramas de colaboración.** Se utilizan para describir la comunicación entre objetos de un sistema.
- **Diagramas de subsistemas.** Se usan para describir agrupaciones de clases en un sistema.

Tabla 3.3 Métodos de desarrollo de software orientados a objetos.

Siglas	Método
RDD	Responsibility-Driven Design ⁸
CRC	Tarjetas Clase-Responsabilidad-Colaboración ⁹
OOAD	Object-Oriented Analysis and Design ¹⁰⁻¹¹
OOD	Object-Oriented Design ¹²
OMT	Object Modeling Technique ¹³
OOSE	Object Oriented Software Engineering ¹⁴
OOK/MOSES	Object-Oriented Knowledge ¹⁵
OOSA	Object-Oriented System Analysis ¹⁶
OOSA	Object-Oriented Systems Analysis ¹⁷
OBA	Object Behavior Analysis ¹⁸
OORA	Object-Oriented Requirements Analysis ¹⁹
Synthesis	Synthesis Method ²⁰
OOSD	Object-Oriented System Development ²¹
OOAD/ROSE	Object-Oriented Analysis & Design ²²
FUSION	Object-Oriented Development ²³
UP	Unified Software Development Process ²⁴

A menudo se confunden algunos conceptos descritos hasta el momento, como *actividad*, *método* y *notación*. La figura 3.1 ilustra esta relación. El lado izquierdo de la figura muestra diferentes actividades, en la parte media se muestran ejemplos de métodos para llevar a cabo las actividades correspondientes, mientras que el lado derecho contiene ejemplos de notaciones para representar el resultado de estos métodos.

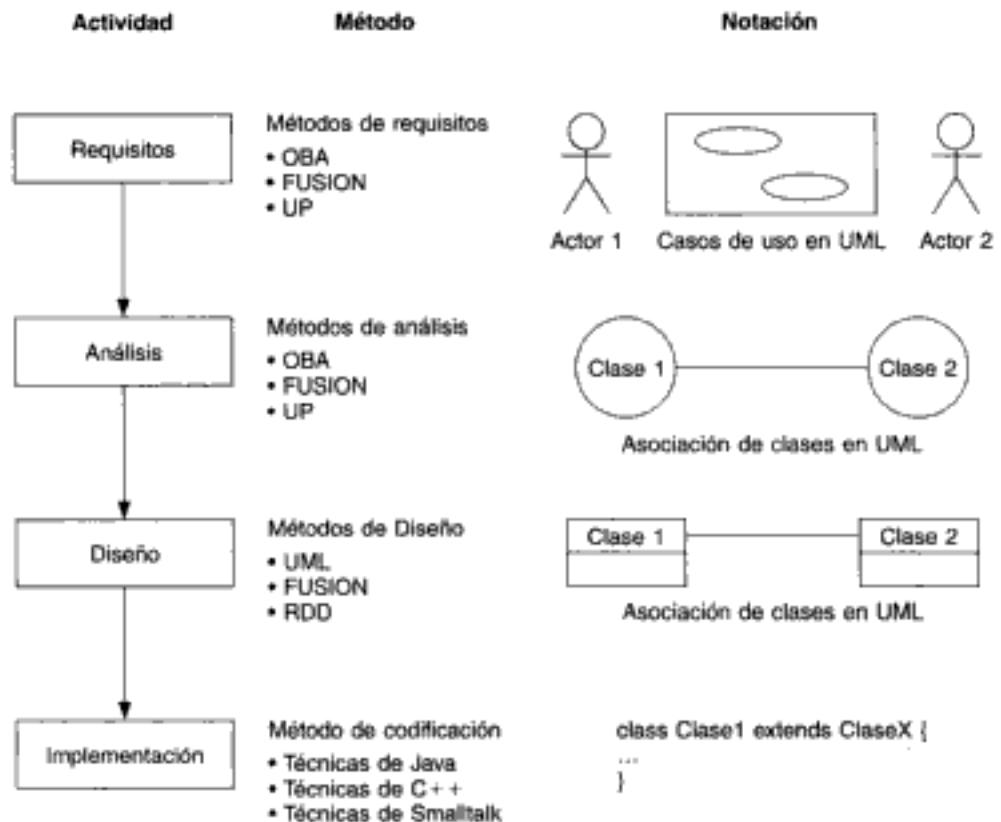


Figura 3.1 Contraste de las actividades, métodos y notaciones de desarrollo de software.

3.1.4 Estrategias

Una **estrategia** se define como un plan para lograr un objetivo. Las estrategias afectan aspectos como la arquitectura del sistema, el orden en que se llevarán a cabo las actividades del proceso y las metodologías a utilizarse. Dada la variedad de posibilidades, es necesario tomar ciertas decisiones iniciales correspondientes al tipo de proyecto a desarrollarse. Estas decisiones son parte de una estrategia de desarrollo, la cual incluye la selección de una tecnología y lenguaje de programación particular; por ejemplo, tecnología orientada a objetos y el lenguaje Java, respectivamente. Otras estrategias aceptadas en la actualidad son los *prototipos* y la *reutilización*, los cuales se describirán a continuación.

Prototipos

Un **prototipo** es una versión preliminar, intencionalmente incompleta o reducida de un sistema. El uso de prototipos es una estrategia que puede aplicarse en casi todas las actividades del proceso de software. El propósito de los prototipos es obtener rápidamente la información necesaria para ayudar en la toma de decisiones. Los siguientes son algunos tipos de prototipo:

- ▶ **Prototipos de requisitos.** Un prototipo de requisitos permite que los usuarios perciban la funcionalidad del producto final a través del diseño de interfaces o pantallas del sistema. El objetivo es ayudar a aclarar los requisitos y solicitar nuevas ideas.
- ▶ **Prototipos de análisis.** Un prototipo de análisis hace posible generar rápidamente una arquitectura general que considere las características principales del sistema de acuerdo a la especificación de requisitos.
- ▶ **Prototipos de diseño.** Estos prototipos permiten explorar y comprender la arquitectura particular del sistema, para poder evaluar aspectos como cuellos de botella (rendimiento y uso de memoria) o inconsistencias en el diseño.
- ▶ **Prototipos verticales.** Un prototipo vertical ayuda a comprender parte de un problema y desarrollar su solución completa. Ésto se hace generalmente cuando los conceptos básicos no están bien comprendidos; por ejemplo, el seguimiento de cierta metodología.
- ▶ **Prototipos de factibilidad.** Un prototipo de factibilidad demuestra si es posible lograr ciertos objetivos del proyecto; por ejemplo, aplicar una arquitectura particular, conectarse a una base de datos bajo ciertas restricciones de rendimiento, aprender a programar en un lenguaje en un tiempo determinado o predecir los costos de desarrollo de un proyecto.

Un prototipo no es un producto de calidad que deba mantenerse a largo plazo. Por el contrario, los prototipos son creados y probados rápidamente, para luego ser desechados. Sin embargo, es común que por presiones de tiempo, se trate de enviar un prototipo al mercado como si éste fuera el producto final. En general, siempre existirá un conflicto entre un desarrollo rápido y un producto de calidad. Las siguientes dos listas muestran algunas consideraciones para el éxito o fracaso con prototipos.²⁵

Los prototipos tienen éxito cuando:

- Se tiene claro el propósito del prototipo y se usa de manera adecuada.
- Se comprende la tecnología a utilizarse y su relación con el proceso de prototipos.
- Se integra un grupo de técnicos apropiado para hacer el prototipo: líder de proyecto, documentador, elaborador de prototipos de requisitos y análisis, y elaborador de prototipos de diseño.
- Se evalúa al grupo y las entregas finales.

- Se involucra a tiempo en el proceso a los usuarios finales.
- Se está dispuesto a repetir el proceso de prototipos para comprender mejor la arquitectura básica.
- Se establecen criterios de evaluación apropiados al comienzo de cada etapa de prototipos y basarse firmemente en estos criterios para su terminación.
- Se construyen prototipos basados en una biblioteca de código reusable, controlada por el bibliotecario asignado.

Los prototipos fallan cuando:

- No se entiende qué es un prototipo y cómo debe usarse.
- No se comprende el proceso lo suficientemente bien como para organizar al grupo correctamente.
- No se sabe hasta cuándo dejar de evolucionar el prototipo y comenzar de cero, extendiendo demasiado el proceso.
- No se sabe hasta cuándo continuar para tratar de lograr los criterios de evaluación deseados, terminando prematuramente el proceso.
- No se utilizan ambientes o herramientas de apoyo adecuadas para el desarrollo particular.
- Se cree que un prototipo razonable es un producto aceptable.
- Los prototipos nunca terminan.

REUTILIZACIÓN

La **reutilización** es la explotación de componentes desarrollados anteriormente dentro de un mismo proyecto o entre proyectos. En un mismo proyecto, la reutilización se aprovecha mediante estructuras comunes de *bajo* nivel, como procedimientos, clases o herencia. Este tipo de reutilización produce generalmente programas más compactos. Entre proyectos, la reutilización se aprovecha mediante estructuras comunes de *alto* nivel, como paquetes gráficos y bibliotecas de análisis numérico. La reutilización entre proyectos requiere planeación y representa una inversión tanto para producir componentes reutilizables como para consumirlos. La decisión para reutilizar componentes se basa en una evaluación de costos, entre crear una nueva solución o adaptar una existente.

- **Consumo de componentes reutilizables.** Consumir componentes reutilizables requiere identificar si ya existe una solución disponible, parcial o completa. Las ventajas de la reutilización incluyen soluciones consistentes entre aplicaciones y mejoras en la calidad del componente al haber sido probado anteriormente en múltiples aplicaciones. Para tener éxito con la reutilización, la solución debe adaptarse a las necesidades del sistema. Es común que los desarrolladores sólo reutilicen soluciones si confían en su nivel de calidad. Las oportunidades de reutilización pueden ocurrir durante todo el ciclo de vida de un proyecto.
- **Producción de componentes reutilizables.** Producir componentes reutilizables significa tener una perspectiva de múltiples proyectos. Una organización debe considerar los costos adicionales de producir un componente reutilizable, planificando la reducción de costos que ocasiona utilizar los componentes en otros proyectos. Generalmente, el esfuerzo para pro-

ducir componentes reutilizables es bastante mayor al esfuerzo de desarrollo de componentes *normales*.

A continuación se muestran algunas razones a favor de la reutilización.²⁶

La reutilización es valiosa cuando:

- Se desea apoyar lo común.
- Se desea promover la consistencia mediante estándares.
- Se desea incrementar la habilidad para discutir problemas entre diversos grupos.
- Se desea reducir costos.
- Se desea facilitar el inicio de un desarrollo.
- Se desea acelerar el tiempo de entrega.
- Se desea mejorar la calidad del producto.

La reutilización no es valiosa cuando:

- Se tienen que ajustar los componentes para satisfacer los requisitos de rendimiento.
- Se tiene que ajustar el tiempo de aprendizaje para consumir y producir componentes reutilizables a los tiempos del proyecto.
- Se tiene que ajustar haciendo demasiados cambios.
- Se provee exceso de funcionalidad.
- No se entienden las interfaces de programación de los componentes.
- Los componentes son propiedad de un proyecto específico.
- Los componentes incluyen demasiada funcionalidad no requerida.
- No disminuye la cantidad de código a ser probada.

3.1.5 Herramientas

Las **herramientas** son aplicaciones que apoyan la administración del proceso de software. El conjunto de estas herramientas se conoce como **ingeniería de software asistida por computadora** (CASE, *Computer-Aided Software Engineering*), cuyo objetivo es asistir al desarrollador durante las diferentes actividades del ciclo de vida del proceso de software. Las herramientas varían en su apoyo a los procesos integrando componentes como editores de texto, generadores de modelos gráficos (*diagramas*) generadores de código, compiladores, depuradores, verificadores, validadores, medidores (*monitores*), administradores de configuración y administradores del proyecto. Las herramientas CASE son indispensables en la administración del proceso de software. La selección de estas herramientas deben considerar el apoyo a las metodologías utilizadas:

- Proveer apoyo explícito para cada paso del método.
- Administrar toda la información que el método requiere obtener o especificar.
- Permitir manejar grandes cantidades de información y ser escalable.
- Incluir un mecanismo que permita probar que la información recolectada es consistente.
- Apoyar la organización de los diagramas de manera automática.

- Permitir usuarios simultáneos en uno o más proyectos.
- Poder generar una implementación inicial junto con la documentación.
- Apoyar la ingeniería en reversa para asegurar que los cambios directos en la implementación sean consistentes con los modelos administrados.

3.2 Modelos clásicos

Los modelos de proceso dependen de las opiniones o creencias de las personas involucradas en un proyecto.²⁷ Por ejemplo, algunas de estas creencias son:

- Es necesario comprender el problema antes de desarrollar una solución.
- El proceso para resolver un problema debe dar un resultado predecible, sin importar qué individuo hace el trabajo.
- Es indispensable planear y calcular el proceso con gran precisión.
- Para que un proceso tenga éxito, es importante evaluar y administrar el riesgo.
- La entrega de etapas intermedias bien definidas aumentan la confianza que se tiene en el resultado final.

A continuación se describen los modelos de procesos "clásicos", discutiendo las creencias en las cuales se basan.

3.2.1 Cascada

El **modelo de cascada** original se desarrolló entre las décadas de los sesenta y setenta,²⁸ y se define como una secuencia de actividades, donde la estrategia principal es seguir el progreso del desarrollo de software hacia puntos de revisión bien definidos (*milestones* o *checkpoints*) mediante entregas calendarizadas (*schedule*). La figura 3.2 muestra un diagrama del modelo de cascada que des-

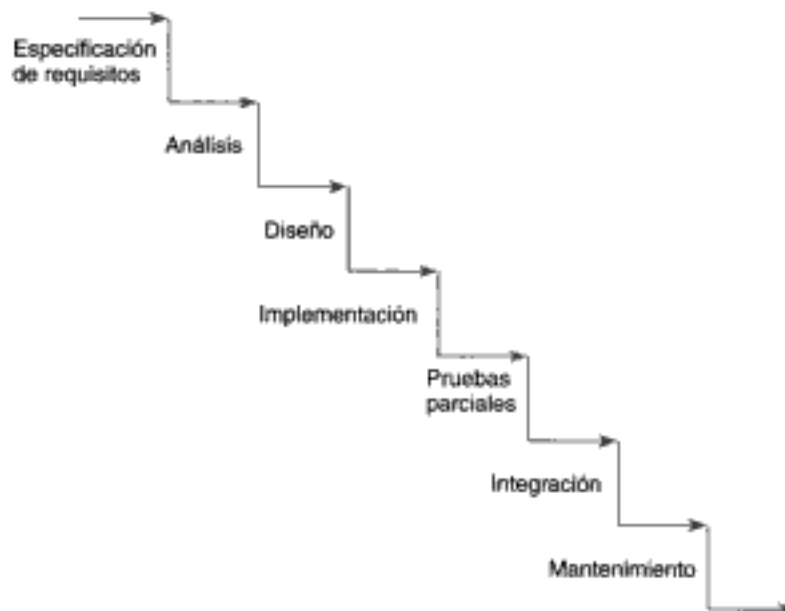


Figura 3.2 Secuencia de actividades para el modelo de cascada.

cribe el orden de las actividades del desarrollo de software. No se muestra una etapa explícita de documentación dado que ésta se lleva a cabo en el transcurso de todo el desarrollo. El modelo original planteaba que cada actividad debía completarse antes de poder continuar con la siguiente actividad. Sin embargo, en una revisión posterior se extendió el modelo permitiendo el regreso a actividades anteriores.²⁹

Las siguientes son algunas creencias del modelo de cascada:

- Las metas se logran mejor cuando se tienen puntos de revisión bien preestablecidos y documentados, dividiendo el desarrollo en actividades secuenciales bien definidas.
- Los documentos técnicos son comprensibles para usuarios y administradores no técnicos.
- Cada detalle de los requisitos se conoce de antemano antes de desarrollar el software, y los detalles son estables durante el desarrollo.
- Las pruebas y evaluaciones se realizan eficientemente al final del desarrollo.

El modelo de cascada fue inicialmente bien recibido, dado que las actividades de las etapas eran razonables y lógicas. Lamentablemente, no explicaba cómo modificar un resultado, en especial, considerando lo difícil que es definir todos los requisitos de un sistema inicialmente y, que se mantengan estables y sin cambios durante el desarrollo. Además, el modelo toma demasiado tiempo en ver resultados, lo que retrasa la detección de errores hasta el final. El modelo también hace difícil rastrear, en otras palabras, ver la dependencia entre los requisitos iniciales y el código final. Esta rigidez trajo dudas respecto a su utilidad, lo que provocó que no se utilizara de acuerdo con su definición original, llevando a los desarrolladores a utilizar variantes del modelo básico, que incluían el uso de prototipos y reutilización de software.³⁰

3.2.2 Incremental

El **modelo incremental** es un desarrollo inicial de la arquitectura completa del sistema, seguido de incrementos y versiones parciales del mismo.³¹ Cada incremento tiene su propio ciclo de vida. Cada incremento agrega funcionalidad adicional o mejorada sobre el sistema. Conforme se completa cada etapa, se verifica e integra la versión con las demás versiones ya completadas del sistema. Durante cada incremento, el sistema se evalúa con respecto al desarrollo de versiones futuras. Las actividades se dividen en procesos y subprocesos, dando lugar al término *software factory*. Para que la secuencia de desarrollo sea exitosa, es esencial definir etapas que no requieran cambiar los resultados anteriores al agregar nuevas. Por lo tanto, es importante comprender al inicio los requisitos completos del sistema, algo que normalmente es muy difícil de lograr. El desarrollo incremental evita la teoría del "Big Bang" para el desarrollo de software, donde una gran explosión de desarrollo se transforma repentinamente en el sistema final.

Las siguientes son algunas creencias del modelo incremental:

- La administración de proyectos es más fácil de lograr en incrementos más pequeños.
- Es más fácil comprender y probar incrementos de funcionalidad más pequeños.
- La funcionalidad inicial se desarrolla más temprano, logrando resultados de inversión en menor tiempo.
- Hay más probabilidad de satisfacer el cambio en los requisitos de usuario mediante incrementos del software en el tiempo, que si fueran planeados todos a la vez en un mismo periodo.

3.2.3 Evolucionario

El **modelo evolucionario** es una extensión al modelo incremental, donde los incrementos se hacen de manera secuencial en lugar de en paralelo.³² Desde el punto de vista del cliente, el sistema evoluciona según se van entregando los incrementos. Desde el punto de vista del desarrollador, los requerimientos que son claros al principio del proyecto dictarán el incremento inicial, mientras que los incrementos para cada uno de los siguientes ciclos de desarrollo serán clarificados a través de la experiencia de los incrementos anteriores. Este modelo considera que el desarrollo de sistemas es un proceso de cambios progresivos mediante *deltas de especificación de requerimientos*. En la figura 3.3 se ilustra este concepto.

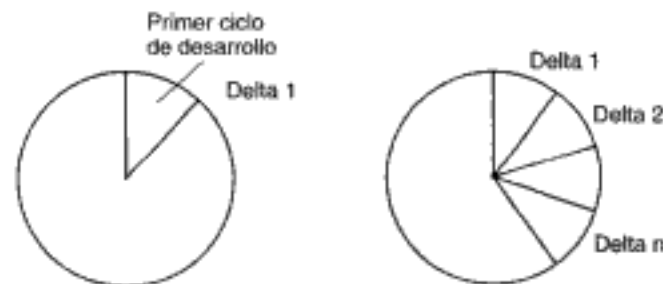


Figura 3.3 Secuencia de versiones en el modelo evolucionario.

El modelo evolucionario es también conocido como *desarrollo rápido de aplicaciones* (RAD, *Rapid Application Development*), que se basa tradicionalmente en el uso de prototipos. Un prototipo de software se considera como un medio para especificar los requisitos y un enlace de comunicación entre el usuario final y el diseñador, lo que ayuda a reducir el riesgo de carecer de requerimientos iniciales completos y estables.

Las siguientes, son algunas creencias del modelo evolucionario:

- Se entrega temprano parte del sistema, aunque no estén completos todos los requerimientos.
- Se permite entregar parte del sistema como herramienta para la generación de requerimientos faltantes.
- Se obtienen beneficios para el sistema mediante entregas iniciales, mientras las entregas posteriores están en desarrollo.

3.2.4 Espiral

El **modelo de espiral**,⁵³ desarrollado durante la década de los ochenta, es una extensión del modelo de cascada. A diferencia del modelo de cascada, que es dirigido por documentos, el modelo de espiral se basa en una estrategia para reducir el riesgo del proyecto en áreas de incertidumbre, como requerimientos iniciales incompletos e inestables. El modelo enfatiza ciclos de trabajo, cada uno de los cuales estudia el riesgo antes de proceder al siguiente ciclo. Cada ciclo comienza con la identificación de los objetivos, soluciones alternativas, restricciones asociadas con cada alternativa y, finalmente, se procede a su evaluación. Cuando se identifica incertidumbre, se utilizan diversas técnicas para reducir el riesgo de las distintas alternativas. Cada ciclo del modelo de espiral termina con una revisión que discute los logros actuales y los planes para el siguiente ciclo. La figura 3.4 muestra un diagrama conceptual del modelo de espiral.

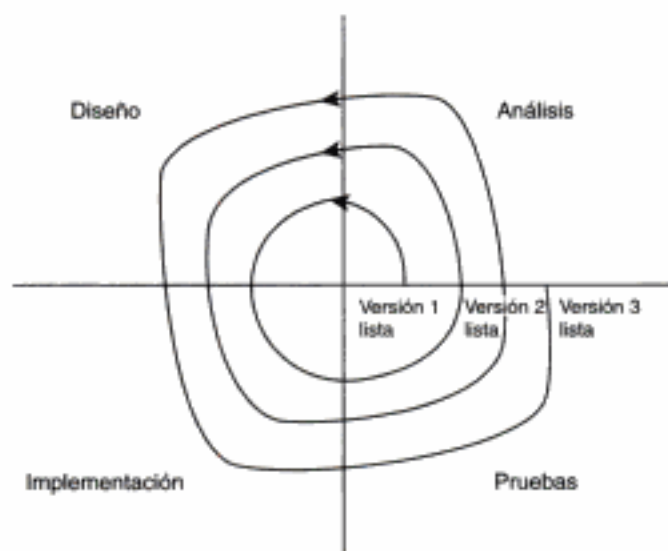


Figura 3.4 Secuencia de actividades para el modelo de espiral.

Al igual que el modelo evolucionario, el modelo de espiral incorpora una estrategia de uso de prototipos como parte del manejo del riesgo.

Las creencias del modelo de espiral son:

- Una actividad comienza cuando se entienden los objetivos y riesgos involucrados.
- Basado en la evaluación de soluciones alternas, se usan las herramientas que mejor reduzcan los riesgos.
- Todo el personal relacionado debe involucrarse en una revisión que determine cada actividad, planeando y comprometiéndose con las siguientes actividades.
- El desarrollo se incrementa en cada etapa, permitiendo prototipos sucesivos del producto.

Con algunas variantes, este es el modelo de proceso más importante en la actualidad.

3.3 Modelos recientes

En esta sección se describen algunos modelos de procesos más recientes.

3.3.1 Ganar-ganar

El modelo **ganar-ganar** (*win-win*)³⁴ extiende el modelo de espiral, haciendo énfasis en la identificación de las condiciones de *ganancia* para todas las partes, creando un plan para alcanzar las condiciones ganadoras y los riesgos correspondientes. Se establecen las reglas para definir el proceso de desarrollo del proyecto, tomando en cuenta todas las partes implicadas. El modelo no necesita mucho tiempo de gestión, lo que permite utilizarlo tanto en proyectos pequeños, como mayores.

Se consideran cuatro los ciclos, compuestos cada uno de cuatro actividades que se detallan a continuación:

1. Elaborar los objetivos, restricciones y alternativas del proceso y producto del sistema y subsistema.
2. Evaluar las alternativas con respecto a los objetivos y restricciones. Identificar y resolver las fuentes principales de riesgo en el proceso y el producto.
3. Elaborar la definición del producto y proceso.
4. Planear el siguiente ciclo y actualizar el plan de su ciclo de vida, incluyendo la partición del sistema en subsistemas para ser considerados en ciclos paralelos. Esto puede incluir un plan para terminar el proyecto si es muy riesgoso o no es factible. Asegurar el compromiso de la administración para continuar según lo planeado.

Una vez revisadas las actividades, los ciclos definen líneas específicas a seguir:

Ciclo 0. Grupos de aplicación. Se determina la viabilidad de un grupo apropiado de aplicaciones.

Ciclo 1. Objetivos del ciclo de vida de la aplicación. Se desarrollan los objetivos del ciclo de vida, incluyendo prototipos, planes y especificaciones de aplicaciones individuales, y se verifica la existencia de al menos una arquitectura viable para cada aplicación.

Ciclo 2. Arquitectura del ciclo de vida de la aplicación. Se establece una arquitectura del ciclo de vida detallado, se verifica la viabilidad y determina que no existen riesgos mayores en satisfacer los planes y especificaciones.

Ciclo 3. Capacidad de operación inicial. Alcanzar una capacidad operacional inicial para cada etapa crítica del proyecto en el ciclo de vida del software.

Las creencias del modelo son las siguientes:

- Crear software basado en componentes para incrementar la calidad en los sistemas de mayor tamaño.
- Escribir software reutilizable para hacer eficiente el proceso de desarrollo.
- Medir la calidad del sistema como aspecto clave del desarrollo del producto.
- * Lograr mayor calidad en el proceso de ensamblaje a partir de componentes menores.
- Usar tecnologías basadas en objetos como aspecto básico para lograr la calidad.
- Producir sistemas rápidamente, sencillos, confiables y de calidad, empleando procesos bien definidos.
- Utilizar el modelo de espiral como base del proceso.
- Flexibilizar el proceso de creación del software para lograr los objetivos generales de eficiencia.
- Involucrar al cliente mediante el manejo de prototipos.
- Analizar los riesgos en el proceso del desarrollo del software para asegurar la calidad final del sistema.

3.3.2 Programación extrema (XP)

La **programación extrema** (XP, *eXtreme Programming*)³⁵ es un modelo de proceso de software que toma los principios y prácticas aceptadas, y las lleva a niveles extremos. Tiene como objetivo reducir el riesgo en el ciclo de vida del software mediante grupos de desarrollo pequeños. Considera que la mejor manera de tratar la falta de requisitos estables en un sistema, es mediante la agilidad de un grupo pequeño de desarrollo. Aunque XP define varias prácticas a seguir, quizás la más representativa del proceso de XP es la *programación en pares* (*pair programming*), donde todo desarrollo requiere de dos programadores que trabajan juntos. El modelo considera varios aspectos problemáticos del desarrollo de software, como son los retrasos, proyectos cancelados, cambios en el negocio y la rotación del personal. Para ello se definen cuatro variables de control en el desarrollo de software: costo, tiempo, calidad y alcance. Mientras que las fuerzas externas asignan los valores para tres de estas variables, el equipo de desarrollo escoge el valor de la cuarta. Si los interesados en un sistema perciben las cuatro variables, pueden concientemente escoger cuáles de éstas controlar, y si no les gustan los valores resultantes de la cuarta variable, pueden cambiar o controlar un conjunto diferente de tres de las cuatro variables. Ajustando el alcance del proyecto basado en las otras tres variables se puede incrementar la probabilidad de éxito.

Las creencias del modelo son las siguientes:

- Los cambios en un sistema son frecuentes.
- Se deben manejar los cambios de manera incremental.
- Se debe apoyar los cambios.
- Se debe lograr una rápida retroalimentación.
- Se debe lograr un trabajo de calidad.
- Se debe buscar la simpleza.
- Dado un conjunto apropiado de prácticas y tecnología, la curva de costo puede aplanarse.

Vale la pena resaltar que el término *programación extrema* es algo confuso, ya que XP es mucho más que un modelo de programación.

3.3.3 Proceso unificado (UP)

El **proceso unificado** (UP, *Unified Process*)³⁶ es una extensión al proceso *objectory* (*object factory*)³⁷ que tiene sus orígenes en la década de 1980. Estos modelos de proceso se basan principalmente en la especificación de requerimientos de un sistema mediante *casos de uso*. El proceso unificado tiene como aspecto esencial del desarrollo de software una visión que parte de la arquitectura del sistema, siguiendo un proceso iterativo e incremental. El proceso integra diferentes aspectos, como son los ciclos, fases, flujos de trabajo, mitigación de riesgo, control de calidad, administración de proyecto y control de configuración. De manera adicional, el proceso unificado considera las cuatro "P" del desarrollo de software: personas, proyecto, producto y proceso. El proceso unificado se basa en las siguientes creencias:

- Para construir un sistema exitoso se debe conocer qué quieren y necesitan los usuarios potenciales.
- Al igual que la arquitectura en la construcción, permite diseñar edificios desde múltiples puntos de vista, estructura, electricidad, etc. Las arquitecturas de los sistemas de software deben permitir visualizar un sistema desde múltiples perspectivas.
- El desarrollo de un producto de software comercial puede significar un gran esfuerzo durante meses, e incluso años. Es práctico dividir el trabajo en etapas, donde cada iteración resulta en un incremento del proyecto.

Como se verá a partir del capítulo 6, la ingeniería de software presentada en este libro está basada en gran medida en el proceso unificado.

3.4 Calidad de software y modelos de madurez del proceso

La **calidad de software** tiene diferentes significados para distintos grupos. Para el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE, siglas de *Institute of Electrical and Electronic Engineers*),³⁸ es el grado en que un sistema, componente o proceso cumple con los requerimientos especificados y las necesidades del cliente o usuario. En la definición de la norma ISO-9000 de la Organización Internacional para la Estandarización (ISO, siglas de *International Organization for Standardization*),³⁹ la calidad de software es el grado (pobre, bueno o excelente) en que un conjunto de características inherentes del software cumplen con los requisitos del sistema.

La calidad del software está directamente relacionada con su proceso de desarrollo. Se considera que un proceso bien conocido y ampliamente utilizado, sustentado en medición y predicción de eventos, permite controlar en buena medida la producción de software y, en consecuencia, producir software de calidad.⁴⁰

Los factores que más afectan la obtención de un producto de calidad son los siguientes:

- El **cliente** o **usuario** es el participante primordial en el proceso de desarrollo del producto y responsable de definir los requisitos del producto final.
- El **desarrollador** es responsable del proceso de producción y de asegurar la calidad del producto.
- El **proceso** seguido para el desarrollo del producto final.
- El **producto** correspondiente al sistema a ser desarrollado.

Estos factores tienen una estrecha correlación, que afecta tanto a la ingeniería del producto como a la organización, la cual debe establecer estándares para los procesos de desarrollo y evaluación, empleando medidas bien establecidas que permitan mejoras continuas de los productos. La evaluación de los procesos evita especificaciones incompletas o anómalas, la aplicación incorrecta de metodologías, etc.⁴¹ Con el objetivo de evaluar la calidad de los procesos de software, se define el concepto de **modelo de madurez del proceso** de producción de software, que son modelos que apoyan no sólo la mejora continua de los procesos de desarrollo de software, sino que también la estandarización de la producción en toda la organización. Cabe resaltar que no se deben aplicar modelos de madurez, bajo el supuesto de mejorar su calidad, sin antes establecer y definir completamente los procesos de desarrollo. Dada la importancia de los procesos, existen diversas certificaciones basadas en los modelos de madurez de proceso que evalúan que las empresas "digan lo que hacen", "hagan lo que dicen" y "demuestren que lo que dicen así lo hacen".⁴² La razón adicional para estas certificaciones es que los modelos a menudo requieren cambios en la cultura de la organización y una fuerte inversión en recursos, como son los financieros, tecnológicos y humanos.

En la tabla 3.4 se muestra una cronología de algunos estándares y modelos más importantes para la calidad de software.

En las siguientes secciones se describen algunos modelos de madurez más importantes.

3.4.1 Modelo de madurez de capacidades (CMM)

El modelo más importante en la actualidad para la evaluación de la madurez de los procesos de desarrollo es el **modelo de madurez de capacidades** (CMM, *Capability Maturity Model*) del *Software Engineering Institute* (SEI, Instituto de Ingeniería de Software).⁴³ CMM tiene como objetivo evaluar los procesos en sus niveles de madurez e identificar los niveles que una organización debe formar para establecer una cultura de excelencia en la ingeniería de software. Los modelos de CMM se generan gracias a la experiencia colectiva de los proyectos más exitosos de software. Dentro de la familia de modelos CMM, se define el modelo para el área de software, SW-CMM.

En particular, CMM es un marco de trabajo que especifica guías para organizaciones de software que quieren incrementar su capacidad de procesos, considerando los siguientes puntos:

- Identificar fortalezas y debilidades en la organización.
- Ponderar los riesgos de seleccionar entre diferentes contratos y monitorear los mismos.

Tabla 3.4 Cronología de estándares y modelos de madurez para la calidad de software.

Año	Descripción
2000	ISO 9000:2000 SEI CMMI V1.02
1999	PEMM V1.0
1998	ISO 15504 (SPICE) (lanzamiento al público como Reportes Técnicos "tipo 2") TickIT V4.0
1997	ISO 9000-3 (nuevo lanzamiento) SEI para las revisiones de SW-CMM apoyando a CMM Integración (CMMI)
1996	IEEE/EIA 12207 (corresponde a ISO 12207) TSP
1995	ISO 12207 (lanzamiento inicial) ISO 15504 (SPICE) (lanzamiento inicial) PSP
1994	ISO 9001 (nuevo lanzamiento)
1993	SEI SW-CMM V1.1
1992	TickIT V2.0
1991	ImproveIT V1.0 (origen de TickIT) ISO 9000-3 (lanzamiento inicial) SEI SW-CMM V1.0 (lanzamiento inicial)
1987	ISO 9001 (lanzamiento inicial)

- Entender las actividades necesarias para planear e implementar los procesos de software.
- Ayudar a definir e implementar procesos de software en la organización a través de una guía.

El modelo CMM se evalúa según el *área de proceso clave* (KPA, *key process area*), donde cada organización debe incorporar procesos adecuados en cada una de las áreas establecidas.

Los procesos se evalúan mediante distintos niveles de madurez, como se muestra en la figura 3.5.

En la tabla 3.5 se describen con mayor detalle los niveles de madurez.

Según información del SEI, hasta abril del 2003 se contaba con cerca de 100 empresas certificadas en el nivel cinco en todo el mundo.⁴⁴

Existe una extensión del modelo básico de madurez, la *integración del modelo de madurez de capacidades* (CMMI, *Capability Maturity Model Integration*), el cual tiene como objetivo integrar los diferentes dominios donde se ha aplicado CMM, más allá de sólo el desarrollo de software. Esto es algo similar al modelo de calidad de ISO-9000 que se aplica en múltiples ámbitos como se explica a continuación.

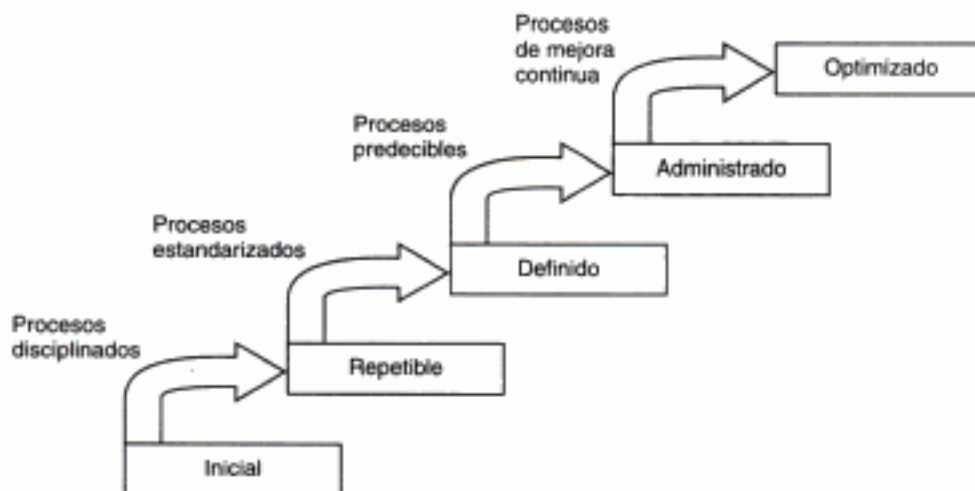


Figura 3.5 Los cinco niveles de madurez del proceso de software.

Tabla 3.5 Los cinco niveles del modelo de CMM.

	Nivel	Características	Transición al siguiente nivel
1	Inicial	<i>Ad hoc</i> , poca formalización, herramientas aplicadas de manera informal al proceso.	Iniciar una administración rigurosa del proyecto y asegurar la calidad.
2	Repetible	Se cuenta con un proceso estable y un nivel repetible de control estadístico.	Establecer un grupo y una arquitectura de proceso de desarrollo de software. Introducir métodos y tecnologías de ingeniería de software.
3	Definido	Se cuenta con una base para un progreso mayor y continuo.	Establecer un conjunto básico de administraciones del proceso para identificar la calidad y costo de los parámetros, y una base de datos del proceso. Juntar y mantener los datos del proceso. Calcular la calidad relativa de cada producto e informar a la administración.
4	Administrado	Mejoras sustanciales en la calidad junto con medidas comprensivas del proceso.	Apoyar la recopilación automática de datos del proceso. Usar los datos para analizar y modificar el proceso.
5	Optimizado	Mejoras con base en mayor calidad y cantidad.	Continuar mejorando y optimizando el proceso.

3.4.2 Organización Internacional para la Estandarización (ISO)

Existen diversas normas de la Organización Internacional para la Estandarización (ISO)⁴⁵ que son aplicables al desarrollo de software, como se muestra a continuación.

ISO-9000⁶⁶. Es una familia de normas internacionales relacionadas con la administración de la calidad en productos y servicios, la cual especifica *qué* requisitos de calidad se deben cumplir pero no *cómo*. Dentro de la familia ISO-9000 existen tres estándares: ISO-9001, ISO-9002 e ISO-9003, que se deben considerar dependiendo del alcance de las actividades de la empresa. En el caso del desarrollo de software, ISO-9001 es la más apropiada, ya que incluye actividades de diseño y desarrollo. En cierta manera, ISO-9000 es comparable a CMMI por ser una familia de estándares, mientras que ISO-9001 es similar a CMM. De tal manera, ISO-9001 es un estándar aplicable a la administración de la calidad en el desarrollo de software. A diferencia de CMM, el modelo ISO-9001 no incluye múltiples niveles de calidad. La certificación es equivalente al nivel tres de la escala de SEI.

ISO-12207⁶⁷. Es una familia de normas internacionales relacionadas con la administración de la calidad en los procesos del ciclo de vida del software. Esta norma está dirigida a lograr acuerdos o contratos entre los desarrolladores y clientes donde se requiere el desarrollo, mantenimiento u operación de un sistema de software. Es una norma de alto nivel que deja sin especificar los detalles de cómo llevar a cabo las actividades y tareas de los procesos. Se describen cinco procesos primarios: adquisición, suministro, desarrollo, mantenimiento y operación. Los cinco procesos se dividen en actividades y las actividades en tareas, con lo cual se agregan requisitos para su ejecución. Así, se especifican ocho procesos: de apoyo, documentación, administración de configuración, aseguramiento de calidad, verificación, validación, auditoría y resolución de problemas. Además, se consideran cuatro procesos organizacionales: administración, infraestructura, mejora y entrenamiento. Esta norma busca que las organizaciones adapten estos procesos según el alcance de los proyectos particulares, eliminando actividades que no se aplican.

ISO-15504. (Ver sección 3.4.5.)

3.4.3 Modelo de madurez de ingeniería de desempeño (PEMM)

El **modelo de madurez de ingeniería de desempeño** (PEMM, *Performance Engineering Maturity Model*)⁶⁸ sirve para evaluar los niveles de integración, aplicación, ejecución y diseño, de la ingeniería de desempeño como modelo de mejora de proceso. El modelo sirve tanto para evaluar una organización como para mejorar los propios procesos de desarrollo. Sirve también para definir el criterio con el que se debe escoger a un proveedor de software para los productos críticos o semicríticos de la empresa.

Al igual que CMM, PEMM cuenta con cinco niveles, los cuales determinan la mejora del comportamiento de ejecución y el decremento del riesgo de ejecución, como se muestra en la figura 3.6.

La evaluación de una compañía se hace midiendo los aspectos relacionados con la organización, la definición de procesos de ingeniería, el proyecto de la dirección y la tecnología. Esto se logra mediante encuestas expertas del método meta-pregunta-métrico (GQM, *Goal Question Metric*), identificando y midiendo los objetivos con preguntas y respuestas cuantificables.



Figura 3.6 Modelo general PEMM.

3.4.4 TickIt

TickIt,⁴⁹ desarrollado por el departamento de comercio e industria del Reino Unido, es primordialmente una guía con estrategias para lograr la certificación en la producción de software según los estándares ISO-9000. Los objetivos principales de TickIt son, además de desarrollar un sistema de certificación aceptable en el mercado, estimular a los desarrolladores de software a implementar sistemas de calidad, dando la dirección y guías necesarias para tal efecto. El objetivo de la certificación es demostrar que existen, y son verificables, las prácticas necesarias para asegurar la calidad durante el desarrollo de software. La guía de auditoría provee la liga para evaluar la conformación del sistema auditado con respecto al modelo TickIt, de manera que pueda ser expresada en función de los criterios de la ISO 9001.

Esta guía se compone de (i) un capítulo de conceptos de calidad, (ii) la norma ISO 9000-3, (iii) una serie de guías para proveedores y compradores, (iv) una guía para la auditoría del sistema de calidad, (v) el proceso de certificación y (vi) guías complementarias.

3.4.5 Mejora del proceso de software y determinación de la capacidad (ISO-15504 / SPICE)

El modelo de **mejora del proceso de software y determinación de capacidad** (SPICE, *Software Process Improvement and Capability Determination*),⁵⁰ es un modelo de evaluación o determinación de la capacidad de los procesos, conocido también como la norma ISO-15504.⁵¹ Es una familia de normas internacionales que tiene como objetivo el desarrollo de sistemas de calidad en el software. Combina los enfoques de CMM con los de ISO-9000, incorporando al marco de referencia de ISO 9000 con la evaluación de capacidad y madurez de proceso de CMM. Su objetivo es lograr ganancias significativas en productividad y calidad, además de ayudar a los compradores de productos de software a obtener un mayor retorno para su inversión y reducir el riesgo asociado con los grandes proyectos de software. Este modelo busca mejorar la calidad del

producto mediante una evaluación comprobada, consistente y confiable del estado de los procesos de software de una organización y usar los resultados de estas evaluaciones como parte de programas coherentes de mejora. La norma establece un denominador común para una evaluación uniforme de los procesos de desarrollo; ya que la tecnología evoluciona, ISO-15504 hace énfasis en la calidad, actualización y vigencia del producto.

ISO-15504 está conformado por nueve documentos que permiten instrumentar paso a paso la norma con su correspondiente evaluación, como se muestra en la figura 3.7:

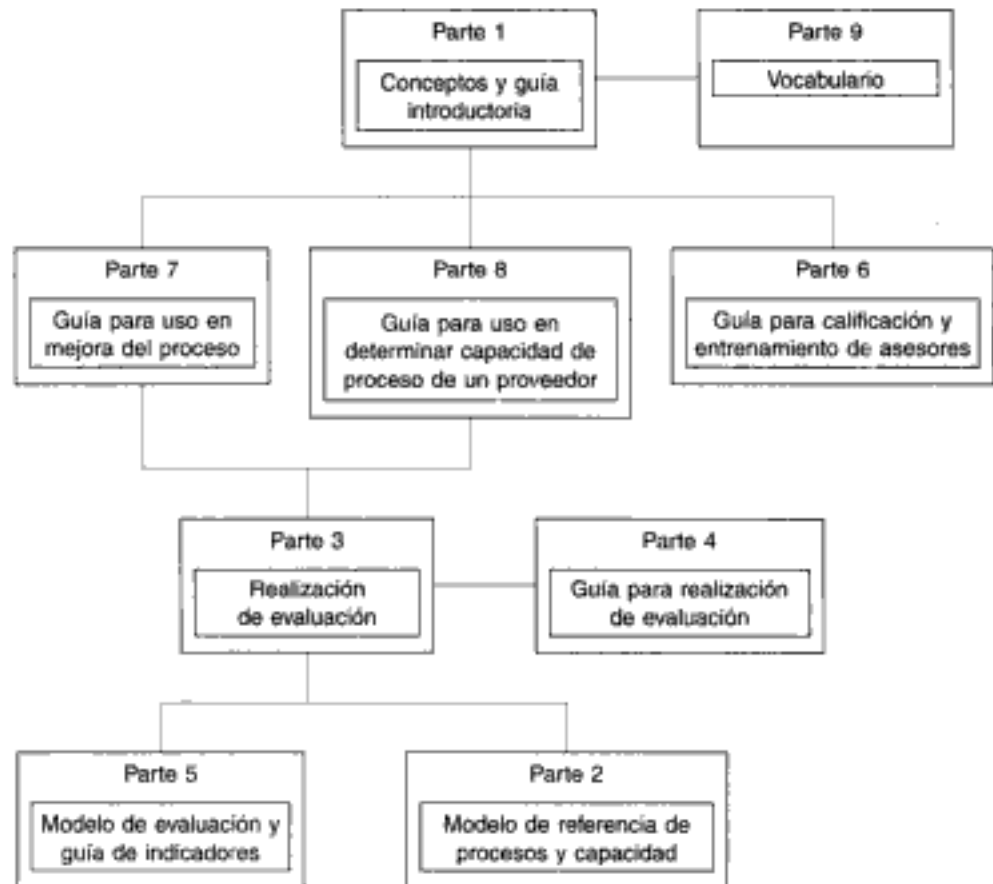


Figura 3.7 Componentes del modelo ISO-15504.

- * Parte 1 – conceptos y guía introductoria.
- * Parte 2 – modelo de referencia de procesos y capacidad.
- * Parte 3 – realización de evaluación.
- * Parte 4 – guía para realización de evaluación.
- * Parte 5 – modelo de evaluación y guía de indicadores.
- * Parte 6 – guía para calificación y entrenamiento de asesores.
- * Parte 7 – guía para uso en mejora del proceso.
- * Parte 8 – guía de uso para determinar la capacidad del proceso de un proveedor.
- * Parte 9 – vocabulario.

La arquitectura de evaluación de SPICE define las siguientes prácticas y procesos.

- Prácticas. Al igual que CMM, las prácticas de SPICE son tratadas de acuerdo con niveles de capacidad: nivel 0 - *incompleto*, nivel 1 - *definido*, nivel 2 - *administrado*, nivel 3 - *establecido*, nivel 4 - *predecible* y nivel 5 - *optimizado*.
- Procesos. Los procesos consideran cinco áreas de actividad: *cliente-proveedor*, *ingeniería*, *soporte*, *administración* y *organización*.

Al igual que CMM, ISO-15504 integra una serie de niveles de madurez por los que deberán pasar sus procesos.

3.4.6 Proceso de software personal (PSP)

El proceso de software personal (PSP, *Personal Software Process*)⁵² es un modelo para la mejora del proceso de desarrollo de software, está basado en la creencia de que la calidad del software depende del trabajo de cada uno de los ingenieros; por lo cual, el proceso debe ayudar a controlar, manejar y mejorar el trabajo de éstos. El objetivo de PSP es mejorar la planeación del trabajo, conocer con precisión el desempeño, medir la calidad de los productos y mejorar las técnicas para su desarrollo. La instrumentación de esta tecnología consiste en lo que se conoce como la *evolución del PSP*. Se siguen ciertos pasos comenzando con las líneas base *PSP0* y *PSP0.1*, el proceso personal de planeación *PSP1* y *PSP1.1*, el manejo personal de calidad *PSP2* y *PSP2.1*, y por último, el proceso personal cíclico *PSP3*, como se muestra en la figura 3.8.

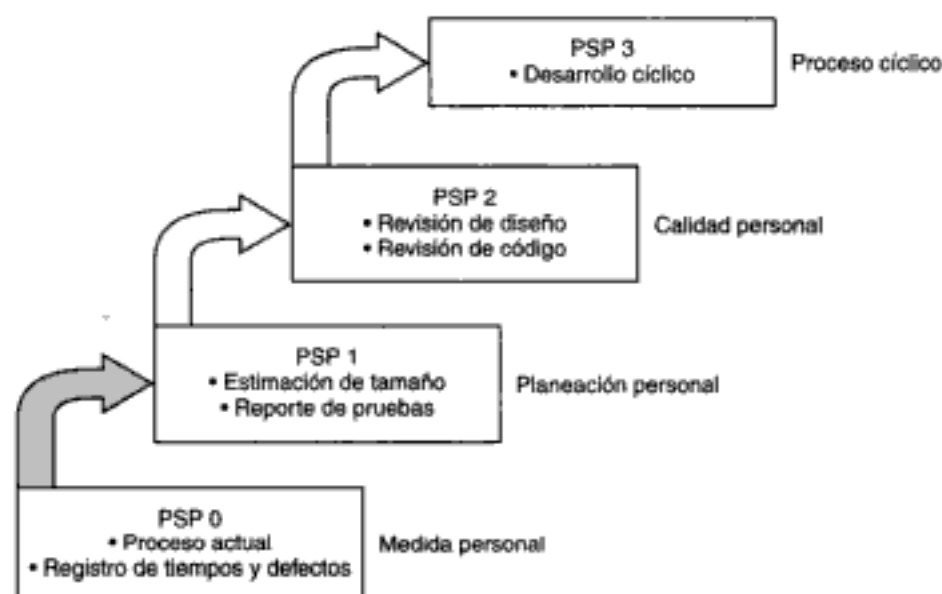


Figura 3.8 Los Niveles de PSP.

- *PSP0* (i) define el proceso de trabajo personal identificando y ordenando las principales actividades; (ii) introduce la recolección de datos para medir la productividad y calidad a través del registro de tiempos y defectos; (iii)

establece las bases para las mejoras en planificación de trabajo por tiempos y evaluación de resultados; y (iv) documenta el proceso usando formas específicas. *PSP0.1* (i) registra el tamaño del producto utilizando puntos funcionales y la estandarización de la codificación y (ii) registra los problemas y propuestas de mejora.

- *PSP1* (i) mejora la planeación introduciendo la estimación del tamaño del producto y (ii) introduce los reportes de pruebas. *PSP1.1* (i) introduce las estimaciones de recursos y (ii) introduce la calendarización.
- *PSP2* (i) introduce las actividades de detección temprana de defectos mediante revisiones de diseño, código y uso de listas de verificación. *PSP2.1* (i) introduce formas para el diseño detallado, facilitando la revisión del diseño.
- *PSP3* (i) introduce el proceso cíclico para crear programas de mayor tamaño, (ii) introduce el registro de seguimiento de asuntos y (iii) lleva el resumen de planeación y registro de tiempo, tamaño y defectos por ciclo.

PSP se considera la solución para pasar rápidamente entre niveles de CMM al lograr un mejor entendimiento de nuestras capacidades y habilidades, y contempla un mejor control sobre el trabajo. Sin embargo, PSP tiene el problema de que se emplea a nivel individual.

3.4.7 Proceso de software en equipos (TSP)

El proceso de software en equipos, (TSP, *Team Software Process*)⁵³ extiende el modelo PSP e integra los aspectos del desarrollo de software realizados por equipos de trabajo, definiendo aspectos como la asignación y control de tareas para los diversos miembros del equipo. TSP define un marco de trabajo en equipos con los objetivos de:

1. Desarrollar productos en varios ciclos.
2. Proporcionar métricas para equipos.
3. Evaluar roles y equipos.
4. Ofrecer guías para la solución de problemas en equipos.

Para lograr estos objetivos, se especifican condiciones para los equipos de trabajo basados en roles de personas, asignación de tareas y control de su ejecución.

RESUMEN

En este capítulo se presentó una introducción al proceso de software mediante la descripción y ejemplos de los conceptos de modelo de proceso, arquitectura, actividad, método y metodología, estrategia, prototipo, reutilización y herramientas. Se describen algunos modelos de proceso "clásicos" y "nuevos" más importantes. Se expone el tema de calidad de software y modelos de madurez de proceso y se estudian algunos de ellos.

REFERENCIAS

1. Scacchi, W., 2001, Process Models in Software Engineering, en J. Marciniak (Ed.), Encyclopedia of Software Engineering, 2nd Ed., Wiley.
2. Coad, P., Yourdon, E., 1991, Object-oriented analysis, Yourdon Press.
3. Yourdon, E., Constantine, L., 1978, Structured Design, Prentice-Hall/Yourdon Press.
4. DeMarco, T., 1979, Structured Analysis and System Specification, Prentice-Hall.
5. Page-Jones, M., 1980, Practical Guide to Structured System Design, Prentice-Hall.
6. Ward, P., Mellor, S., 1985, Structured Development for Real-Time Systems, Prentice-Hall.
7. Yourdon, E., 1989, Modern Structured Analysis, Prentice-Hall.
8. Wirfs-Brock, R., Wilkerson, B., Wiener, L., 1990, Designing Object-Oriented Software, Prentice-Hall.
9. Beck, K., Cunningham, W., 1989, A Laboratory for Teaching Object-Oriented Thinking, Proceedings OOPSLA '89, pp. 1-6, New Orleans, Louisiana, oct. ACM. Published as ACM SIGPLAN Notices, volumen 24, número 10.
10. Coad, P., Yourdon, E., 1991, Object-Oriented Analysis, 2nd Ed., Yourdon Press/Prentice-Hall.
11. Martin, J., Odell, J.J., 1992, Object-Oriented Analysis and Design, Prentice-Hall.
12. Booch, G., 1991, Object-Oriented Design with Applications, Benjamin Cummings.
13. Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., Lorensen, W., 1991, Object-Oriented Modeling and Design, Prentice-Hall.
14. Jacobson, C., Jonsson, P., Overgaard, G., 1992, Object-Oriented Software Engineering, Addison-Wesley.
15. Henderson-Sellers, B., Edwards, J.M., 1994, Book Two of Object Oriented Knowledge: The Working Object, Prentice-Hall, Sydney, Australia.
16. Shlaer, S., Mellor, S.J., 1992, Object Lifecycles, Modeling the World in States, Yourdon Press.
17. Embley, D.W., Kirtz, B.D., Woodfield, S.N., 1992, Object-Oriented Systems Analysis: A Model Driven Approach, Yourdon Press/Prentice-Hall.
18. Rubin, K.S., Goldberg, A., 1992, Object Behavior Analysis, Communications of the ACM, 35(9):48-62.
19. Firesmith, D.G., 1993, Object-Oriented Requirements Analysis and Design: A Software Engineering Approach, Wiley.
20. Page-Jones, M., Weiss, S., 1989, Synthesis/Analysis Object Oriented Method, DCI Object Oriented Systems Symposium, June.
21. de Champeaux, D., Lea, D., Faure, P., 1993, Object-Oriented Systems Development, Addison Wesley, MA.
22. Booch, G., 1994, Object-Oriented Analysis and Design with Application, Benjamin/Cummings.
23. Coleman, D., Arnold, P., Bodoff, S., Dollin, C., Gilchrist, H., Hayes, F., Jeremaes, P., 1994, Object Oriented Development: The FUSION Method, Prentice-Hall, NJ.
24. Jacobson, C., Booch, G., Rumbaugh, J., 1999, The Unified Software Development Process, Addison-Wesley.
25. Goldberg, A., Rubin, K., 1995, Succeeding with Objects: Decision Framework for Project Management, 1st Ed., Addison-Wesley.
26. Goldberg, A., Rubin, K., 1995, Succeeding with Objects: Decision Framework for Project Management, 1st Ed., Addison-Wesley.
27. Goldberg, A. y Rubin, K., 1995, Succeeding with Objects: Decision Framework for Project Management, 1st Ed., Addison-Wesley.
28. Royce, W., 1970, Managing the Development of Large Software Systems: Concepts and Techniques, Proceedings WESTCON, IEEE Computer Society Press, San Francisco, CA.
29. Boehm, B., 1981, Software Engineering Economics, Englewood Cliffs, NJ: Prentice-Hall.
30. Yourdon, E., 1992, Decline and Fall of the American Programmer, Prentice-Hall.
31. Boehm, B., 1981, Software Engineering Economics, Englewood Cliffs, NJ: Prentice-Hall.
32. Boehm, B., 1987, A Spiral Model of Software Development and Enhancement, Software Engineering Project Management IEEE.
33. Boehm, B., 1987, A Spiral Model of Software Development and Enhancement, Software Engineering Project Management IEEE.
34. Boehm, B., Egyed, A., Port, D., Shah, A., Kwan, J., Madachy, R., A Stakeholder Win-Win Approach to Software Engineering Education, Annals of Software Engineering, ene. 1999.
35. Beck, K., 2000, Extreme Programming Explained: Embrace Change, Addison-Wesley.
36. Jacobson, I., Booch, G., Rumbaugh, J., 1999, The Unified Software Development Process, Addison-Wesley.

37. Jacobson, C., Christensen, M., Jonsson, P., Overgaard, G., 1992, Object-Oriented Software Engineering: A Use-Case Driven Approach, Addison-Wesley.
38. ANSI/IEEE Std. 729-1983, IEEE Standard Glossary of Software Engineering Terminology, IEEE Inc., New York, 1983.
39. ISO 9000-3:1997, <http://www.iso.org>
40. DeMarco, T., 1982, Controlling Software Projects: Management, Measurement and Estimation, Prentice-Hall.
41. Jones, C., 1994, Assessment and Control of Software Risks, Prentice-Hall.
42. Humphrey, W., 1995, A Discipline for Software Engineering, Addison-Wesley.
43. <http://www.sei.cmu.edu/cmm/>
44. <http://www.sei.cmu.edu/serna/profile.html>, <http://www.sei.cmu.edu/serna/pdf/SW-CMM/2003apr.pdf>
45. <http://www.iso.org>
46. <http://www.iso.org/iso/en/iso9000-14000/iso9000/iso9000index.html>
47. <http://www.12207.com/>
48. Schmietendorf A., Scholz A., 1999, The Performance Engineering Maturity Model, in Metrics News volumen 4 número 2.
49. <http://www.tickit.org/>
50. Dorling, A., 1993, SPICE: Software Process Improvement and Capability Determination, Software Quality Journal 2, 209-224.
51. <http://www.sei.cmu.edu/iso-15504/>
52. Humphrey, W.S., 1997, Introduction to the Personal Software Process. Addison-Wesley, Reading, MA.
53. Humphrey, W.S., 2000, Introduction to the Team Software Process. Addison-Wesley, Reading, MA.

Modelado y programación orientada a objetos

En esta segunda parte se describirá la programación orientada a objetos desde dos perspectivas distintas. La primera analiza el *modelado* (capítulo 4), como una descripción teórica de los conceptos básicos de la orientación a objetos, utilizando la notación UML (*Unified Modeling Language*). La segunda analiza la *programación* (capítulo 5), como la descripción práctica basada en el modelado orientado a objetos, mediante el uso del lenguaje Java.

Modelado con UML

El **modelado**, o **modelo de objetos**, describe los conceptos principales de la orientación a objetos: las estructuras estáticas y sus relaciones. Las principales estructuras estáticas son los **objetos** y **clases**, los cuales están compuestos de **atributos** y **operaciones**, mientras que las principales relaciones entre objetos y clases corresponden a las **ligas** y **asociaciones**, respectivamente. En este capítulo, éstos y otros temas se describirán en términos de los objetos, clases, atributos, operaciones, asociaciones, composición, herencia y módulos. Se utilizará UML (Unified Modeling Language) como notación para el modelado.¹

4.1 Objetos

Los **objetos** son las entidades básicas del modelo de objeto. La palabra objeto proviene del latín *objectus*, donde *ob* significa *hacia*, y *jacere* es *arrojar*, o sea, que teóricamente un *objeto* es cualquier cosa que se pueda arrojar.

Ejemplo: Una *pelota* o un *libro* se pueden arrojar, por tanto, son objetos. Por otro lado, un *avión* o un *elefante* también se consideran objetos, aunque sean bastante pesados para ser arrojados.

Los objetos son más que simples cosas que se puedan arrojar, son conceptos que a su vez se dividen en abstractos o concretos.

Ejemplo: Una *mesa* es un objeto concreto, en tanto que un *viaje* es un objeto abstracto.

Por lo general, los objetos corresponden a sustantivos, pero no a gerundios.

Ejemplo: *Mesa* y *viaje* son ambos sustantivos y, por tanto, objetos. *Trabajando* y *estudiando* son gerundios por lo que no se consideran objetos.

Cualquier cosa que incorpore una estructura y un comportamiento o acción se le puede considerar un objeto.

Ejemplo: Una *pelota* es sólida y redonda, y se le puede arrojar o atrapar. Un *libro* es rectangular y sólido, y se le puede abrir, cerrar y leer.

Un objeto debe tener una identidad coherente para que se le pueda asignar un nombre lógico y conciso.

Ejemplo: Se consideran *manzanas* todas las frutas con un sabor, textura y forma similares.

La existencia de un objeto depende del contexto del problema. Lo que puede ser un objeto apropiado en una aplicación, puede no serlo en otra, y al revés. Por lo general, existen muchos objetos en una aplicación, y parte del desafío es encontrarlos.

Ejemplo: La *temperatura* se puede considerar un objeto abstracto, con propiedades como el valor de la temperatura y el tipo de la escala en que se mide (Celsius o Fahrenheit). Por otro lado, si hablamos de un *termómetro*, la temperatura será una propiedad del termómetro.

Los objetos se definen según el contexto de la aplicación.

Ejemplo: Para una compañía, una persona llamada *Juan Pérez* se considera un objeto, mientras que para un laboratorio el *bígado* de Juan Pérez es un objeto. Una *universidad* se considera un objeto, mientras que dentro de ésta los objetos serían las *aulas*, los *estudiantes* y los *profesores*.

Los objetos son entidades que existen de forma independiente. Es necesario distinguir entre los objetos, los cuales contienen características o propiedades, y las características mismas.

Ejemplo: El *color* y la *forma* de una manzana no se consideran propiamente objetos, sino propiedades del objeto *manzana*. El *nombre* de una persona se considera una propiedad de la persona.

Un grupo de cosas puede ser un objeto si existe como una entidad independiente.

Ejemplo: Un *automóvil* se considera un objeto, el cual está formado por varias partes, como el *motor* y la *carrocería*.

Los objetos deben tener nombres en singular y no en plural.

Ejemplo: Un *automóvil* es un objeto, varios *automóviles* son simplemente muchos objetos y no un solo objeto.

Parte de una cosa puede considerarse un objeto.

Ejemplo: La *rueda*, que es parte del automóvil, se puede considerar un objeto. Por otro lado, el lado izquierdo del automóvil sería un mal objeto.

Los objetos deben tener nombres lógicos y concisos para evitar la construcción de objetos que no tengan una identidad coherente.

Ejemplo: Datos o información no son nombres concisos de objetos. Por otro lado, un estudiante es un objeto, ya que contiene propiedades como el número

ro de matrícula y nombre del estudiante, además tiene un comportamiento como ir a clases, presentar exámenes y graduarse. Pero decir que todos los estudiantes cuyos apellidos comiencen con "A" son objetos es una falsedad, ya que el nombre del objeto no es conciso.

El objeto integra una estructura de datos (*atributos*) y un comportamiento (*operaciones*).

4.1.1 Diagramas de objetos

Los objetos se describen gráficamente por medio de un **diagrama de objetos** o **diagrama de instancias**.

La notación general para un objeto es una caja rectangular, que contiene el nombre del objeto subrayado, el cual sirve para identificar al objeto, como se muestra en la figura 4.1.

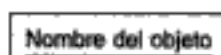


Figura 4.1 Notación para un objeto.

Ejemplo: Los objetos Juan Pérez y Universidad Tecnológica se muestran en la figura 4.2.



Figura 4.2 Notación para los objetos Juan Pérez y Universidad Tecnológica.

4.1.2 Identidad

Los objetos se distinguen por su propia existencia, su **identidad**, aunque internamente los valores para todos sus datos sean iguales, debido a que todos los objetos se consideran diferentes.

Ejemplo: Si tenemos una biblioteca llena de libros, cada libro, incluyendo múltiples copias, se consideran e identifican como objetos diferentes. Dos manzanas aunque sean exactamente del mismo color y forma, son diferentes objetos.

Los objetos tienen un nombre que puede no ser único.

Ejemplo: Pueden existir múltiples copias de un solo libro, lo cual requiere identificadores especiales para distinguir entre diferentes objetos con propiedades similares, como el código del libro en una biblioteca.

Los objetos necesitan un **identificador interno único** cuando se instrumentan o implementan en un sistema de computación con el fin de permitir el acceso y distinguir entre los objetos. Estos identificadores no deben incluirse como una propiedad del objeto, ya que sólo son importantes en el momento de la implementación.

Ejemplo: Las diferentes personas se distinguen internamente dentro de una computadora mediante los identificadores *Persona1*, *Persona2*, *Persona3*, etc. Por otro lado, el número del seguro social de la persona es un identificador externo válido, ya que existe fuera de la implementación en una computadora.

4.2 Clases

Una **clase** describe un grupo de objetos con estructura y comportamiento común. (Clase y tipo no son necesariamente equivalentes, **tipo** se define por las manipulaciones que se le puede dar a un objeto dentro de un lenguaje y **clase** involucra una estructura, que puede corresponder a una implementación particular de un tipo. En el capítulo 5 se hablará más de esto.)

Las estructuras o propiedades de la clase se conocen como **atributos** y el **comportamiento** como **operaciones**. Una clase define uno o más objetos que pertenecen a la clase y que tienen características comunes. En general, el nombre de una clase debe iniciar con una letra mayúscula.

Ejemplo: *Juan Pérez* y *María López* se consideran miembros de la clase *Persona*, donde todas las personas tienen una edad y un nombre. La *Universidad Tecnológica* y la *Universidad Internacional* pertenecen a la clase *Universidad*, donde tienen una dirección y un grado máximo. *Chrysler* y *Microsoft* pertenecen a la clase *Compañía*, donde todas tienen una dirección, un número de empleados y una ganancia al año.

Una clase se considera un "molde" a partir del cual se crean múltiples objetos.

Ejemplo: La clase es como un molde de cerámica, del cual se pueden crear múltiples cerámicas, todas con exactamente las mismas características. Para modificar las cerámicas primero hay que construir un nuevo molde.

Al definir múltiples objetos en clases se logra una abstracción del problema. A partir de los casos específicos se generalizan definiciones comunes, como nombres de la clase, atributos y operaciones.

Ejemplo: Los objetos *impresora láser*, *impresora de burbuja* e *impresora de punto* son todos objetos que pertenecen a la clase *impresora*.

Una clase como se ha definido en esta sección se conoce también como **clase básica**.

4.2.1 Diagrama de clases

Las clases se describen por medio del **diagrama de clases**. La notación para una clase es una caja rectangular, que contiene el nombre de la clase, como se muestra en la figura 4.3.

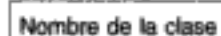


Figura 4.3 Notación para una clase.

Ejemplo: Las clases *Persona* y *Universidad* se muestran en la figura 4.4.



Figura 4.4 Notación para las clases *Persona* y *Universidad*.

La notación general para el objeto se extiende mediante el nombre de la clase subrayado seguido del nombre del objeto, como se muestra en la figura 4.5.



Figura 4.5 Notación para un objeto que incluye el nombre de la clase.

Ejemplo: Los objetos *Juan Pérez* y *Universidad Tecnológica* se muestran en la figura 4.6, incluyendo el nombre de sus respectivas clases *Persona* y *Universidad*.



Figura 4.6 Notación para los objetos *Juan Pérez* y *Universidad Tecnológica*, que incluyen el nombre de la clase.

Por lo general, se utilizan más los diagramas de clases que los diagramas de objetos, ya que los diagramas de clases son más generales y corresponden a varios diagramas de objetos.

4.2.2 Instanciación

El proceso de crear objetos pertenecientes a una clase se conoce como **instanciación**, donde los objetos son las *instancias* de la clase. El objeto, es la instancia de la clase a la que pertenece. Se utiliza una flecha punteada para mostrar los objetos como instancias de las clases, como se muestra en la figura 4.7.



Figura 4.7 Notación para la instanciación de objetos.

Ejemplo: *Juan Pérez* y *María López* son instancias de la clase *Persona*, como se muestra en la figura 4.8.

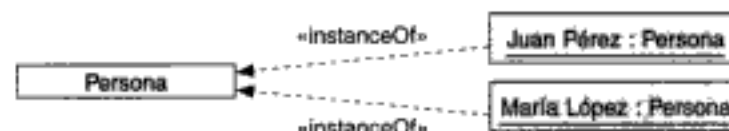


Figura 4.8 Notación para la instanciación de objetos de la clase *Persona*.

Se puede instanciar un número indefinido de objetos de cierta clase.

4.3 Atributos

Los **atributos** definen la estructura de una clase y de sus correspondientes objetos. El atributo define el valor de un dato para todos los objetos pertenecientes a una clase.

Ejemplo: *Nombre*, *edad* y *peso* son atributos de la clase *Persona*. *Color*, *precio* y *modelo* son atributos de la clase *Automóvil*.

Los atributos corresponden a sustantivos, y sus valores pueden ser sustantivos o adjetivos.

Ejemplo: *Nombre*, *edad*, y *color* son sustantivos *Juan*, *24* son sustantivos y *verde* es un adjetivo.

Se debe definir un valor para cada atributo de una clase. Los valores pueden ser iguales o distintos en los diferentes objetos. No se puede dar un valor en un objeto si no existe un atributo correspondiente en la clase.

Ejemplo: El valor del atributo *edad* puede ser "24" para los objetos *Juan Pérez* y *María López*, y "15" para *Ramón Martínez*.

Dentro de una clase, los nombres de los atributos deben ser únicos (aunque puede aparecer el mismo nombre de atributo en diferentes clases).

Ejemplo: Las clases *Persona* y *Compañía* pueden tener ambas un atributo *dirección*; en cambio no pueden existir dos atributos llamados *dirección* dentro de la clase *Persona*.

Los atributos no tienen ninguna identidad, a diferencia de los objetos.

Ejemplo: Los atributos *nombre* y *edad* de la clase *Persona* tienen valores simples. El valor para *nombre* puede ser "Juan" o "María", mientras que el valor para *edad* puede ser "17" o "25". (Note que pueden existir dos objetos distintos con exactamente el mismo *nombre* y *edad*, los cuales identificarían a dos personas distintas.)

Un atributo, como se ha definido en esta sección, se conoce también como *atributo básico*. Los atributos se listan en el diagrama de clases a continuación del nombre de la clase, en una segunda sección, como se muestra en la figura 4.9.

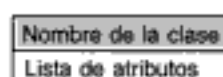


Figura 4.9 Diagrama de clases que contiene atributos.

Ejemplo: En la figura 4.10 se muestran los atributos *nombre* y *edad*, para la clase *Persona*.

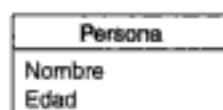


Figura 4.10 Diagrama de clases para la clase *Persona*, que contiene los atributos: *Nombre* y *Edad*.

La notación para el diagrama de objetos incluye los valores de los atributos que se ubican en el centro de la caja, con letra normal, a continuación del nombre de la clase. Existen dos notaciones alternas:

La **notación extendida** se muestra en la figura 4.11.

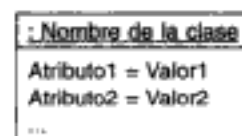


Figura 4.11 Notación extendida del diagrama de objetos o instancias para objetos que contienen valores de atributos.

Ejemplo: En la figura 4.12 se muestran dos objetos de tipo *Persona*, que utilizan la notación extendida. Los valores de los dos objetos para el atributo *Nombre* son: *María López* y *Juan Pérez*, y para el atributo *Edad*: 24 y 21, respectivamente. (Los valores para los atributos pueden ser o no iguales.)

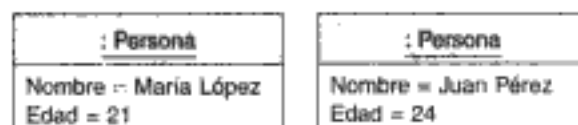


Figura 4.12 Diagrama de objetos con objetos de tipo *Persona*, que contienen valores de atributos, usando la notación extendida.

La **notación compacta** se muestra en la figura 4.13.

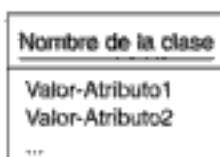


Figura 4.13 Notación compacta del diagrama de objetos, para objetos que contienen valores de atributos.

Ejemplo: En la figura 4.14 se muestran dos objetos de tipo *Persona*, que utilizan la notación compacta. Los valores de los dos objetos para el atributo *Nombre* son: *María López* y *Juan Pérez*, y para el atributo *Edad*: *21* y *24*, respectivamente. En esta notación es muy importante mantener el mismo orden en que se han definido los atributos en el diagrama de clases.



Figura 4.14 Diagrama de objetos con dos instancias de tipo *Persona*, que contienen valores de atributos, usando la notación compacta.

Se puede asociar con cada atributo un *tipo* de dato, por ejemplo, entero, cadena, etcétera, para restringir sus posibles valores. El tipo se añade, separado por dos puntos, al diagrama de clases inmediatamente después del nombre del atributo. También se puede definir un valor de **omisión** en cada atributo, o sea, un valor que se asigna en caso de no haberse especificado uno, el cual se añade separado por un signo de "igual" a continuación del tipo de dato. La notación se muestra en la figura 4.15. (El diagrama de clases no varía con respecto a esta información adicional.)

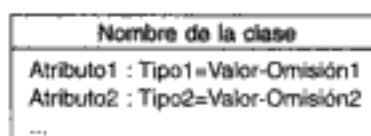


Figura 4.15 Notación extendida para diagrama de clases conteniendo atributos.

Ejemplo: En la figura 4.16 se muestran los dos atributos de la clase *Persona*: *nombre* y *edad*, donde *nombre* está definido como una *cadena* de caracteres, con valor de omisión "", mientras que *edad* está definido como un *entero*, con valor de omisión "0".

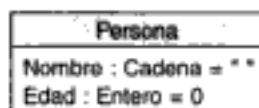


Figura 4.16 Diagrama de clases para la clase *Persona*, que contiene atributos con definición de tipo de datos y valores de omisión.

La notación compacta sería análoga a la anterior. (El detalle que se muestra en cualquier diagrama se puede variar.)

Ejemplo: Los diagramas de clases de la figura 4.17 son todos correctos, y varían según el detalle deseado en la descripción.

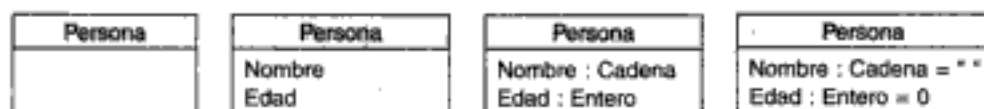


Figura 4.17 Diagrama de clases para la clase *Persona*, que contiene diferente nivel de detalle.

4.3.1 Identificadores

En el momento de incluir atributos en la descripción de una clase se deben distinguir los atributos —los cuales reflejan las características de los objetos en la realidad—, y los **identificadores** —que se utilizan exclusivamente por razones de implementación—. Estos identificadores internos del sistema no deben ser incluidos como atributos.

Ejemplo: *Número del Seguro Social* o *número de la licencia de conducir* son identificadores válidos del mundo real, en cambio un identificador para distinguir entre objetos de tipo *Persona* no se debe incluir en el diagrama. En la figura 4.18 se muestra la forma incorrecta de incluir un identificador ("identificador: ID") en la clase del objeto, seguido por la forma correcta (omitido).

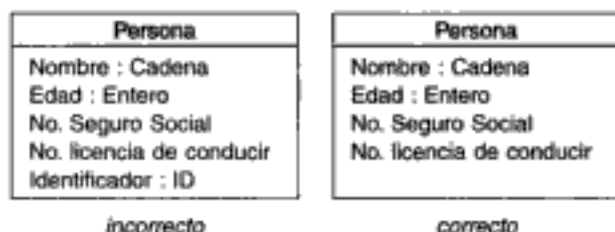


Figura 4.18 Diagrama de clases que muestra de forma incorrecta, la inclusión de un atributo identificador, seguido por la forma correcta.

4.3.2 Atributos derivados

Los **atributos básicos** son atributos independientes dentro del objeto. En contraste, los **atributos derivados** son atributos que dependen de otros atributos. Los atributos derivados dependen de otros atributos del objeto, los cuales pueden ser *básicos* o *derivados*. La notación es una diagonal como prefijo del atributo, como se muestra en la figura 4.19.

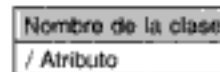


Figura 4.19 Notación para atributos derivados.

Ejemplo: El *área* de un *rectángulo* se puede calcular conociendo su *Ancho* y *Largo*, por lo cual no se define como un atributo básico de la caja, sino como uno derivado, como se muestra en la figura 4.20.

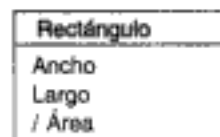


Figura 4.20 Diagrama con área como un atributo derivado de los atributos básicos *Ancho* y *Largo*.

4.3.3 Restricciones de atributos

Los valores de los atributos de una clase pueden restringirse. La notación para una restricción (en inglés *constraint*) es incluir, por debajo de la clase y entre corchetes, la restricción para los valores del atributo (véase fig. 4.21).

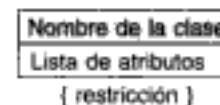


Figura 4.21 Notación para la restricción en un diagrama de clases.

Ejemplo: Un *Rectángulo* puede restringir que su *Ancho* y *Largo* sean siempre iguales, lo que es equivalente a un *Cuadrado*. Asimismo, el *Área* del *Rectángulo* está definida como el *Ancho* por el *Largo*. Ambas restricciones se muestran en la figura 4.22.

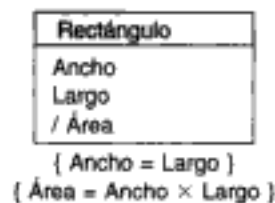


Figura 4.22 Diagrama que muestra dos restricciones para un *Rectángulo*: la restricción de que el *Largo* sea igual al *Ancho* para el caso de cuadrados, y la restricción del *Área* es igual al *Ancho* por el *largo*.

4.4 Operaciones

Las **operaciones** son funciones o transformaciones que se aplican a todos los objetos de una clase particular. La operación puede ser una acción ejecutada por el objeto o sobre el objeto.

Ejemplo: *Arrojar, atrapar, inflar y patear* son operaciones para la clase *pelota*. *Abrir, cerrar, ocultar y dibujar* son operaciones para la clase *ventana*.

Las operaciones deben ser únicas dentro de una misma clase, aunque no necesariamente para diferentes clases.

Ejemplo: Las clases *Pelota* y *Libro* pueden, ambas, tener operaciones de *comprar*, pero no pueden tener cada una dos operaciones *comprar*.

No se debe utilizar el mismo nombre en operaciones que tengan un significado totalmente diferente.

Ejemplo: No se debe utilizar el mismo nombre *invertir* para la operación de invertir una figura y para la operación de invertir una matriz, ya que son operaciones totalmente diferentes. Invertir una figura es rotarla 180 grados, mientras que invertir una matriz M es encontrar su inverso N , para que $M \times N = 1$. Se deben usar nombres diferentes, como *invertir-figura* e *invertir-matriz*.

Las operaciones pueden tener **argumentos**, es decir, una lista de *parámetros*, cada uno con un tipo, y pueden también devolver resultados, cada uno con un tipo. Las operaciones se incorporan en la tercera sección de la clase, como se muestra en la figura 4.23.

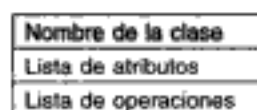


Figura 4.23 Notación para diagrama de clases conteniendo atributos y operaciones.

Ejemplo: En la figura 4.24 se muestran tres clases: *Persona*, *Universidad* y *Rectángulo*, que contienen atributos y operaciones. *Trabajar* y *Votar* son operaciones en *Persona*; *Enseñar* y *Graduar* son operaciones en *Universidad*; mientras que *Dibujar* y *Borrar* son operaciones en *Rectángulo*.

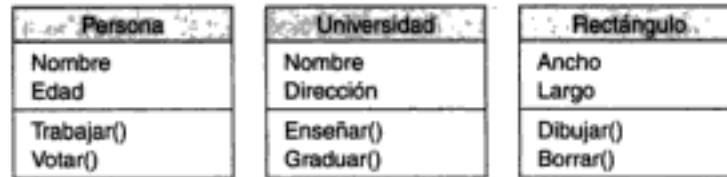


Figura 4.24 Diagrama de clases que contiene atributos y operaciones para las clases *Persona*, *Universidad* y *Rectángulo*.

En la figura 4.25 se muestra la notación extendida para una clase conteniendo atributos y operaciones, donde las operaciones pueden incluir una lista de tipos de argumentos, además de una lista de tipos de resultados.

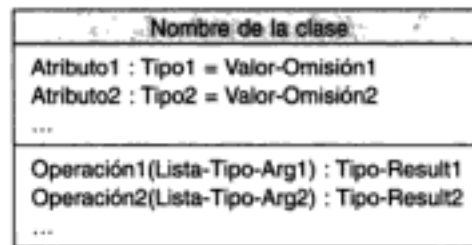


Figura 4.25 Notación extendida para una clase que contiene atributos y operaciones.

Ejemplo: En la figura 4.26 se muestran dos clases: *Figura* y *Archivo*, las cuales tienen atributos y operaciones. *Mover* y *Rotar* son operaciones de *Figura*, que tienen los argumentos *V* de tipo *Vector* y *Ángulo*, respectivamente. Ambas operaciones devuelven un resultado de tipo booleano, el cual devuelve un valor de *Cierto* o *Falso* (*true* o *false*). *Imprimir* es una operación de *Archivo*, que contiene un argumento *D* de tipo *dispositivo*, que puede ser el nombre de una impresora y el número *N* de copias a imprimir. El resultado puede ser un valor booleano.

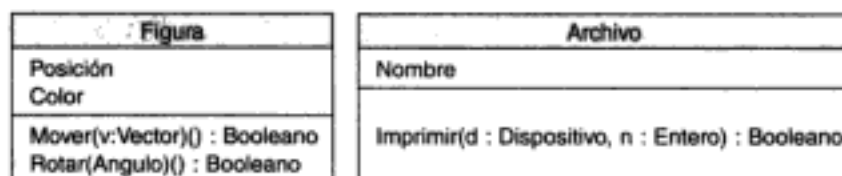


Figura 4.26 Diagrama de clases *Figura* y *Archivo*, que contienen atributos y operaciones con notación extendida.

Note que los objetos no incluyen ninguna información sobre sus operaciones, a diferencia de las clases, ya que las operaciones son idénticas para todos los objetos de una misma clase, a diferencia de los atributos que varían entre objetos en relación con su valor.

A continuación, se describen otros conceptos relacionados con operaciones: *consultas, accesos, métodos, polimorfismo, parametrización y firmas*.

Consultas. A las operaciones que no tienen efectos secundarios, las cuales no cambian los valores de los atributos en el objeto, se les llama **consultas** (*query*). Una consulta es una operación que no modifica al objeto. Las consultas, en general, devuelven valores de atributos, básicos o derivados, y pueden o no tener argumentos.

Ejemplo: Para una clase *Rectángulo* que contiene los atributos *Ancho* y *Largo* pueden existir operaciones de consulta para leer los valores del *Ancho* y *Largo* sin afectarlos.

Una consulta se puede definir para la lectura de atributos derivados, ya que éstos no cambian los valores de los atributos del objeto.

Ejemplo: En la clase *Rectángulo*, el atributo *Área* puede ser leído por medio de una consulta implementada como la multiplicación de los valores de los atributos *Largo* y *Ancho*.

La notación para las operaciones de consulta es la misma que para las operaciones, sólo difieren en su comportamiento. Por regla general, estas consultas no se incluyen de forma explícita en el modelo de objetos, sino que se agregan durante la etapa de diseño, ya que son operaciones bastante básicas.

Accesos. Es posible definir **operaciones de acceso** para leer o escribir los atributos de un objeto. Si el acceso está hecho para leer solamente, sin afectar los valores de los atributos, entonces se considera también una operación de consulta. Se utiliza la notación de punto para indicar, dentro de la operación de acceso, el acceso a un atributo: *objeto.atributo*.

Ejemplo: Para ingresar el *nombre* de una *persona* se utiliza: *persona.nombre*.

La notación para las operaciones de acceso es la misma que para las operaciones, la diferencia es sólo en su comportamiento. Por regla general, estas operaciones de acceso no se incluyen de forma explícita en el modelo de objetos, sino que se agregan durante la etapa de diseño, ya que son operaciones bastante básicas.

Métodos. El término **método** se utiliza para distinguir la operación como un concepto de alto nivel de la propia implementación de la operación, algo correspondiente a un concepto de más bajo nivel.

Ejemplo: La operación *Imprimir* de la clase *Archivo* se implementa mediante un método *Imprimir*, que contiene un argumento llamado dispositivo, que a su vez tiene el código para la implementación de la operación *Imprimir*.

En ciertos lenguajes de programación orientados a objetos se permite incluir más de un método para implementar una misma operación, en otras palabras, se pueden incluir múltiples descripciones de bajo nivel o implementaciones para un mismo concepto a alto nivel. En estos casos los métodos varían según el número y tipo de argumentos, debiendo ser todos los métodos consistentes entre sí.

Ejemplo: La operación *Imprimir* de la clase *Archivo* se puede implementar con diferentes métodos. Un método *Imprimir* contiene el argumento *dispositivo*, el cual manda el archivo a la impresora correspondiente. Otro método *Imprimir*, con exactamente el mismo nombre, pero sin argumentos, mandaría el archivo a la pantalla, en lugar de a la impresora.

Polimorfismo. El **polimorfismo** se define como una misma operación que toma diferentes formas. Una operación se considera polimórfica si ésta se implementa en *diferentes clases* de forma distinta.

Ejemplo: Para las clases *Archivo-ASCII* y *Archivo-PostScript* pueden existir varios métodos para implementar la operación *Imprimir*. Estos métodos corresponden a la misma operación *Imprimir*, pero se implementan de diferentes formas. El *Archivo-ASCII* imprime el texto en *ASCII*, mientras que el *Archivo-PostScript* requiere un interpretador *PostScript*.

Una operación también se considera polimórfica si se implementa en una *misma clase* por diferentes métodos, con otro número y tipo de argumentos.

Ejemplo: La operación *Imprimir* de la clase *Archivo* implementada con diferentes métodos *Imprimir*, que contiene distinto número de argumentos, se considera polimórfica. Con polimorfismo, el transmisor de un mensaje no necesita saber la clase del objeto receptor. Usando terminología más técnica, el vínculo (*binding*) entre el mensaje recibido y la operación apropiada se hace mediante: (i) un *vínculo estático* durante la compilación del lenguaje o (ii) un *vínculo dinámico* durante la ejecución, el cual también se conoce como *vínculo virtual*.

Parametrización. La **parametrización** de una operación está definida por el número y tipo de argumentos de cada método.

Ejemplo: La parametrización de la operación *Imprimir* de la clase *Archivo* está definida por su argumento de tipo *dispositivo*.

Firmas. La **firma** de una operación se define por el tipo y número de argumentos y el tipo de resultados que devuelve.

Ejemplo: La firma de la operación *Imprimir* en la clase *Archivo* está definida por su argumento de tipo *dispositivo*.

En operaciones polimórficas a través de diferentes clases, las operaciones deben mantener la misma firma.

Ejemplo: La operación *Imprimir* para las clases *Archivo-ASCII* y *Archivo-PostScript* deben tener la misma firma a través de sus respectivos métodos.

4.5 Ligas y asociación

La relación entre objetos se conoce como **liga**. Una **asociación** describe la relación entre clases de objetos y posibles ligas, donde una liga es una *instancia* de una asociación, al igual que un objeto es una instancia de una clase. **Tipos de asociaciones entre clases:**

- Una **asociación de conocimiento** (*acquaintance associations*) es una asociación estática entre instancias y significa que una instancia conoce de la existencia de otra instancia. Denota conocimiento entre clases durante largos periodos.
- Una **asociación de comunicación** es una asociación dinámica que modela la comunicación entre dos objetos, y sirve para intercambiar información entre objetos, denota relación entre clases cuando existe una comunicación entre ellos. A través de estas asociaciones, un objeto envía y recibe eventos.

Ejemplo: Los objetos Juan Pérez y Universidad Tecnológica están relacionados por la liga *estudia-en* que describe que "Juan Pérez estudia en la Universidad Tecnológica".

Ejemplo: Las clases *Estudiante* y *Universidad* están relacionadas por la asociación *estudia-en* que describe que un "estudiante estudia en la universidad".

El nombre de una liga debe ser igual al nombre de la correspondiente asociación.

Ejemplo: Juan Pérez es una instancia de la clase *Estudiante* y Universidad Tecnológica es una instancia de la clase *Universidad*. Por tanto, la liga *estudia-en* entre Juan Pérez y Universidad Tecnológica lleva el mismo nombre que la asociación *estudia-en* entre Estudiante y Universidad.

La asociación, al igual que la liga, es por naturaleza bidireccional. Por lo general, el nombre de la liga o asociación implica una dirección, pero puede ser invertida para mostrar la dirección opuesta. Cualquiera de las dos direcciones es igualmente correcta, aunque por lo general, se acostumbra leer de izquierda a derecha.

Ejemplo: El opuesto de "estudiante estudia en la universidad" sería "universidad da estudios a estudiante". Las dos direcciones se refieren a la misma asociación.

La notación que describe una liga es una línea que conecta a los dos objetos, y que contiene el nombre de la liga en letras cursivas, como se muestra en la figura 4.27.

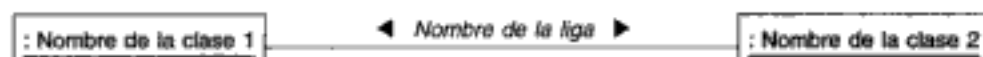


Figura 4.27 Notación para el diagrama de objetos que contiene una liga.

En la figura 4.28 se muestra la liga *estudia-en* entre objetos de tipo *Estudiante* y *Universidad*.



Figura 4.28 Diagrama de objetos que contiene la liga *estudia-en* entre los objetos Juan Pérez, de tipo *Estudiante* y *Universidad Tecnológica*, de tipo *Universidad*.

Ejemplo: En la figura 4.29 se muestra la liga *trabaja-para* entre los objetos Raúl González, de tipo *Persona* y *Compañía Tecnológica S.A.*, de tipo *Compañía*.



Figura 4.29 Diagrama de objetos que contiene la liga *trabaja-para* entre los objetos Raúl González, de tipo *Persona* y *Compañía Tecnológica S.A.*, de tipo *Compañía*.

La notación que describe una asociación es una línea, la cual conecta las dos clases, y contiene el nombre de la asociación en letras cursivas, como se muestra en la figura 4.30.

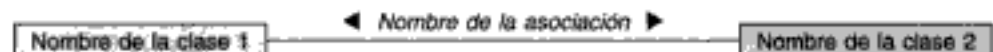


Figura 4.30 Notación para diagrama de clases que contiene una asociación.

Ejemplo: En la figura 4.31 se muestra la asociación *estudia-en*, entre *Estudiante* y *Universidad*.



Figura 4.31 Diagrama de clases conteniendo la asociación *estudia-en*, entre *Estudiante* y *Universidad*.

Ejemplo: En la figura 4.32 se muestra la asociación *trabaja-para* entre *Persona* y *Compañía*.

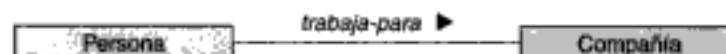


Figura 4.32 Diagrama de clases conteniendo la asociación *trabaja-para* entre *Persona* y *Compañía*.

4.5.1 Implementación

Vale la pena profundizar en la implementación de ligas y asociaciones, ya que es un aspecto que la gran mayoría de los lenguajes orientados a objetos no apoyan de forma directa. El mecanismo principal que se utiliza en la mayoría de los lenguajes es la **referencia** o **apuntador** que permite a una entidad referirse a otra de manera primitiva. Por tal motivo, la referencia o apuntador se guardaría como un atributo adicional de la clase, algo que veremos con mayor detalle en los capítulos 8 y 9, donde discutiremos aspectos de diseño e implementación, respectivamente. Sin embargo, esta solución casi obligatoria, oculta el hecho de que la asociación no es parte de ninguna clase, sino depende de la combinación de varias clases. Como las asociaciones son bidireccionales, sería necesario usar un par de atributos, lo cual ocultaría más aún el hecho de que las dos direcciones de la asociación son dependientes. Por tales razones, es un error conceptual modelar asociaciones por medio de referencia o apuntadores, pero —como se mencionó antes—, no tenemos alternativa. (Durante el diseño se puede decidir implementar la asociación por medio de uno o más referencias o apuntadores.)

Ejemplo: Sería incorrecto modelar la asociación *estudia-en* por medio de un *apuntador* de *Estudiante* hacia *Universidad*, y otro de *Universidad* hacia *Estudiante*, como se muestra en la figura 4.33.



Figura 4.33 Diagrama de clases que describe de forma incorrecta la asociación *estudia-en* por medio de una referencia entre *Estudiante* y *Universidad*.

Ejemplo: Sería también incorrecto modelar la liga *estudia-en* por medio de una referencia apuntando de *Juan Pérez* hacia *Universidad Tecnológica*, y otro de *Universidad Tecnológica* hacia *Juan Pérez*, como se muestra en la figura 4.34. ("&" se usa como notación para referencias.)



Figura 4.34 Diagrama de objetos que describe de forma incorrecta la liga *estudia-en* por medio de una referencia entre *Juan Pérez* y *Universidad Tecnológica*.

4.5.2 Grado de la asociación

El **grado de una asociación** se determina por el número de clases conectadas por la misma asociación. Las asociaciones pueden ser binarias, ternarias o de

mayor grado. Las asociaciones se consideran binarias si relacionan sólo dos clases.

Ejemplo: La asociación entre Persona y Universidad es una asociación binaria.

Las asociaciones pueden ser de mayor grado si relacionan a la vez más de dos clases. Aparte de relaciones binarias, lo más común son relaciones *ternarias* (entre tres clases); las relaciones de más alto nivel son menos comunes. Mientras el grado de una relación aumenta, su comprensión se dificulta, por tanto, se debe considerar partir las relaciones en varias relaciones binarias.

Ejemplo: Puede existir una relación ternaria entre Estudiante, Profesor y Universidad donde: "un estudiante estudia con un profesor en una universidad".

El grado de las ligas corresponde al de las asociaciones.

Ejemplo: Para una asociación binaria entre las clases Estudiante y Universidad, la liga correspondiente también es binaria ya que relaciona exactamente dos objetos, un objeto de tipo Estudiante y otro de tipo Universidad, como Juan Pérez y Universidad Tecnológica.

La notación para una relación ternaria entre objetos se muestra en la figura 4.35. La relación no se etiqueta.

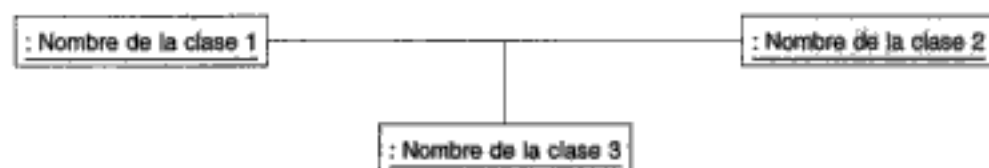


Figura 4.35 Notación para diagrama de instancias describiendo una asociación ternaria.

Ejemplo: En la figura 4.36 se muestran posibles relaciones entre objetos que corresponden a esta relación ternaria. El *Estudiante Juan Pérez* estudia con el *Profesor Roberto González* en la *Universidad Tecnológica*.



Figura 4.36 Diagrama de instancias que describe una relación ternaria entre objetos de tipo *Estudiante*, *Profesor* y *Universidad*.

La notación para una relación ternaria entre clases se muestra en la figura 4.37. La relación no se etiqueta.



Figura 4.37 Notación para diagrama de clases que describe una asociación ternaria.

Ejemplo: En la figura 4.38 se muestra una asociación ternaria entre las clases *Estudiante*, *Profesor* y *Universidad*, que describen a diferentes estudiantes que estudian con distintos profesores en diversos institutos.

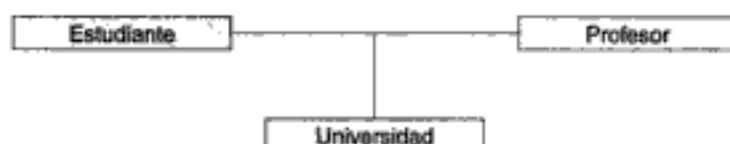


Figura 4.38 Diagrama de clases describiendo una asociación ternaria entre *Estudiante*, *Profesor* y *Universidad*.

4.5.3 Asociaciones reflexivas

Las **asociaciones** pueden ser *reflexivas*, y relacionan distintos objetos de una misma clase.

Ejemplo: Para una clase *Persona* puede existir una asociación *pariente* que describe que dos objetos de tipo *Persona*, como *Juan Pérez* y *Laura Pérez* son *parientes*.

El grado de una asociación reflexiva puede ser binario, ternario o de mayor grado, dependiendo del número de objetos involucrados.

Ejemplo: Para la clase *Persona* puede existir una asociación ternaria entre tres individuos donde uno es el *abuelo*, el otro es el *hijo del abuelo* y el tercero es el *nieto del abuelo*.

Las asociaciones reflexivas relacionan distintos objetos de una misma clase.

Ejemplo: *Juan Pérez es pariente-de Laura Pérez*, donde ambos son objetos de tipo *Persona*, como se muestra en la figura 4.39.



Figura 4.39 Diagrama de instancias que describe una asociación reflexiva de objetos de la clase *Persona*.

Ejemplo: La asociación reflexiva *pariente-de* para la clase *Persona* se muestra en la figura 4.40.



Figura 4.40 Diagrama de clases que describe una asociación reflexiva para la clase *Persona*.

4.5.4 Multiplicidad

La **multiplicidad** (cardinalidad) de una asociación especifica cuántas instancias de una clase se pueden relacionar a *una sola instancia* de otra clase.

Ejemplo: En el caso de *Estudiante* y *Universidad*, la multiplicidad está dada por el número de estudiantes que puedan estudiar en una sola universidad. En otras palabras, muchos objetos de tipo *Estudiante* se conectan a un solo objeto de tipo *Universidad*.

Es necesario decidir la multiplicidad de cada clase en una asociación, o sea dos multiplicidades por cada relación binaria, una para cada extremo de la relación.

Ejemplo: En la relación *estudia-en*, es necesario definir la multiplicidad para el *Estudiante* y la *Universidad*.

La multiplicidad restringe una asociación limitando el número de objetos que pueden relacionarse a un objeto en particular.

Ejemplo: En la asociación *estudia-en*, se restringe el número de estudiantes que pueden estudiar en una universidad.

La multiplicidad depende del contexto de la aplicación. Existen distintos tipos de multiplicidades para las asociaciones, de los cuales los más relevantes son:

- “Uno-uno”: donde dos objetos se relacionan de forma exclusiva, uno con el otro.

Ejemplo: Cada *Universidad* tiene un *Rector* y cada *Rector* rige una *Universidad*.

- “Uno-muchos”: donde uno de los objetos puede estar ligado a muchos otros objetos.

Ejemplo: Muchos *Estudiantes* pueden estudiar en una *Universidad*, y una sola *Universidad* da estudios a cada *Estudiante*.

- “Muchos-muchos”: donde cada objeto de cada clase puede estar ligado a muchos otros objetos.

Ejemplo: Muchos *Estudiantes* pueden estudiar en varias *Universidades*.

La multiplicidad se incluye en el diagrama de clases únicamente. La multiplicidad para relaciones de mayor grado es más compleja, por lo que esta notación se vuelve un poco ambigua para relaciones de mayor orden, ya que no sabría cómo leerse la relación.

Se incorpora la siguiente notación en cada extremo de una asociación binaria. La notación para relaciones "uno-uno", donde dos objetos sólo pueden tener una liga entre ellos, es la notación básica de asociación hasta ahora dada, como se muestra en la figura 4.41.



Figura 4.41 Diagrama de clases que describe una multiplicidad de "uno-uno".

La notación para relaciones "uno-muchos", donde uno de los objetos puede estar ligado a muchos otros objetos está dada por un "asterisco", que representa el lado de "muchos", el cual corresponde a cero o más ligas, como se muestra en la figura 4.42.



Figura 4.42 Diagrama de clases que describe una multiplicidad de "uno-muchos".

Ejemplo: En el caso de una *Universidad* que puede atender a muchos *Estudiantes*, el diagrama se muestra en la figura 4.43, donde la relación de "muchos" se incorpora del lado de *Estudiantes*. (Lo contrario significaría que un estudiante puede atender a muchas universidades.)



Figura 4.43 Diagrama de clases que describe una multiplicidad de "uno-muchos" entre *Estudiantes* y *Universidad*.

La notación para relaciones "muchos-muchos", donde los dos objetos pueden estar ligados a muchos otros objetos, está dada por dos "asteriscos", correspondientes cada una a una multiplicidad de "muchos", como se muestra en la figura 4.44.



Figura 4.44 Diagrama de clases que describe una multiplicidad de "muchos-muchos".

Ejemplo: En el caso de muchas *Universidades* que pueden atender a muchos *Estudiantes*, el diagrama se muestra en la figura 4.45.



Figura 4.45 Diagrama de clases que describe una multiplicidad de "muchos-muchos" entre *Estudiantes* y *Universidades*.

La notación para representar una relación opcional, donde la multiplicidad es "uno" o "cero", describe una relación opcional 0 o 1. Esto significa que dos objetos pueden o no estar conectados y, si lo están, corresponden a una multiplicidad de 1. La notación se muestra en la figura 4.46.



Figura 4.46 Diagrama de clases que describe una multiplicidad "opcional".

Ejemplo: El caso de muchos *Estudiantes* que pueden o no atender a una sola *Universidad* se muestra en la figura 4.47. (Esto es a diferencia del ejemplo anterior donde los estudiantes *tienen* que atender a una universidad.)



Figura 4.47 Diagrama de clases que describe una multiplicidad de "opcional-muchos" entre *Estudiantes* y *Universidad*.

La relación de muchos se puede restringir con un número o secuencia de números.

Ejemplo: En la figura 4.48 se muestra una relación con multiplicidad de cero o más personas que trabajan para una compañía.



Figura 4.48 Diagrama de clases que incluyen multiplicidad en la asociación.

Ejemplo: En la figura 4.49 se muestra una relación donde exactamente dos personas trabajan en una compañía.



Figura 4.49 Diagrama de clases que incluyen multiplicidad de "2" en la asociación.

Ejemplo: En la figura 4.50 se muestra una relación donde por lo menos diez personas trabajan para una compañía.



Figura 4.50 Diagrama de clases que incluye multiplicidad de "10" o más en la asociación.

Ejemplo: La figura 4.51 muestra una relación donde una o dos personas trabajan para una compañía.



Figura 4.51 Diagrama de clases que incluye multiplicidad de "1" o "2" en la asociación.

Ejemplo: La figura 4.52 muestra una relación de tipo opcional, donde cero o una persona trabajan para una compañía.



Figura 4.52 Diagrama de clases que incluye multiplicidad de tipo opcional, donde "0" o "1" personas se relacionan con la compañía.

4.5.5 Rol

El **rol** describe el papel que juega cada extremo de una asociación. Una asociación binaria tiene dos roles, uno en cada extremo, los cuales pueden tener un nombre diferente cada uno. Una relación de n clases tendría n roles. El nombre del rol provee una forma de atravesar la asociación de un objeto en un extremo sin mencionar explícitamente el nombre de la asociación.

Ejemplo: Una *Persona* asume el rol de *Empleado* con respecto a la *Compañía*, y la *Compañía* asume el rol de *Empleador* con respecto a la *Persona*.

Cuando hay solamente una asociación conectando dos clases, a menudo el nombre de la clase sirve como nombre de rol, y no es necesario agregar un nombre de rol de forma explícita.

Ejemplo: *Estudiante* y *Universidad* son tan descriptivos que no es necesario agregar un nombre de rol. Si la clase fuera *Persona*, el nombre de rol *Estudiante* sería más descriptivo.

Los nombres de los roles no deben duplicar los atributos de la clase a la cual describen. (Esto se hace por razones de implementación.)

Ejemplo: El rol *Empleado* no debe ser un atributo de *Persona*.

Se pueden incorporar los nombres de los roles y de la asociación a la misma vez, o uno de los dos solamente, lo cual es más que suficiente para describir la relación. La figura 4.53 muestra la notación para un rol.



Figura 4.53 Notación para diagrama de clases que contiene nombres de los roles.

Ejemplo: *Persona* tiene el rol de *Empleado* con respecto a la *Compañía*, la cual tiene el rol de *Empleador* con respecto a *Persona*, como se muestra en la figura 4.54.



Figura 4.54 Diagrama de clases para una asociación con los nombres de los roles, nombre de la asociación y multiplicidad.

Los nombres del rol son necesarios para asociaciones reflexivas (asociaciones entre objetos de la misma clase), ya que sólo saber el nombre de la asociación no es suficiente para distinguir el papel que en ella juegan los diferentes objetos.

Ejemplo: Si una *Persona* puede ser jefe o empleado en una *Compañía*, entonces la única forma de distinguir el papel que la *Persona* juega es por medio de un nombre de rol, como se muestra en la figura 4.55.

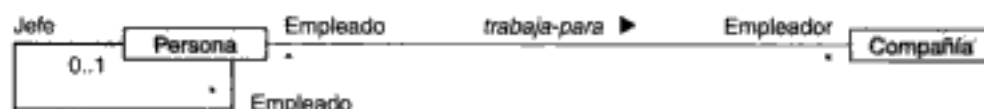


Figura 4.55 Diagrama de clases para una asociación reflexiva con roles.

Es importante utilizar nombres de rol para distinguir entre dos asociaciones distintas, las cuales relacionan un mismo par de clases. Los roles deben ser diferentes según la asociación para el mismo extremo de la relación.

Ejemplo: Una *Persona*, además de ser empleado de una *Compañía*, también puede ser dueño de ella. Por tanto, existen dos asociaciones entre *Persona* y *Compañía*, la primera es *trabaja-para* y la segunda es *dueño-de*. Ambas relaciones se pueden describir mediante roles, como lo ilustra la figura 4.56.



Figura 4.56 Diagrama de clases para asociaciones entre dos clases distinguidas por los nombres de rol.

4.5.6 Restricciones de ligas y asociaciones

Las **restricciones** especifican una relación particular entre las diferentes asociaciones o ligas. Como su nombre lo indica restringen los valores que estas entidades pueden asumir. Las restricciones sencillas se pueden añadir al modelo de objeto, mientras que las más complejas deben ser especificadas en el modelo funcional. Por lo general, las restricciones se describen de forma declarativa, aunque luego deban convertirse a un procedimiento para ser implementadas. Las restricciones pueden ser expresadas en lenguaje natural o mediante ecuaciones, se limitan por corchetes y se ubican cerca de la entidad restringida, como se muestra en la figura 4.57.



Figura 4.57 Notación para un diagrama de clases mostrando una restricción de asociación. La restricción debe ubicarse cerca de la entidad que afecte, y no necesariamente en el centro como se muestra en la figura.

Subconjunto. Las posibles ligas de una asociación pueden ser un subconjunto de las posibles ligas de otra asociación. La multiplicidad de la asociación del subconjunto debe ser igual o menor que la multiplicidad de la asociación del superconjunto. Se utiliza una flecha para relacionar el subconjunto con la entidad de la cual depende, como se muestra en la figura 4.58.



Figura 4.58 Diagrama de clases de una restricción de subconjunto entre dos asociaciones.

Ejemplo: El *Presidente* de un *País* debe ser un *habitante* del *País*. La asociación *presidente-de* es un subconjunto de la asociación *habitante-de*, como se muestra en la figura 4.59.

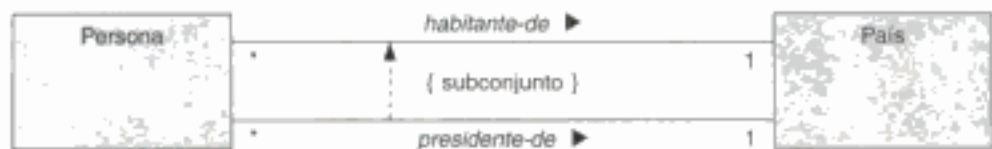


Figura 4.59 Diagrama de clases que muestra a *presidente-de* como un subconjunto de *habitante-de*.

Orden. La multiplicidad “muchos” indica que un conjunto de objetos puede estar relacionado con un mismo objeto. Estas relaciones pudieran estar ordenadas, como se muestra con la notación en la figura 4.60.



Figura 4.60 Notación para un diagrama de clases que presenta una restricción de orden.

Ejemplo: Una ventana en una estación de trabajo está superpuesta a otras ventanas. Las ventanas están ordenadas para que se desplieguen en el orden correcto, de modo que la de más arriba se despliegue por último. Para indicar esta situación se incluye una restricción especial de orden, como se muestra en la figura 4.61.

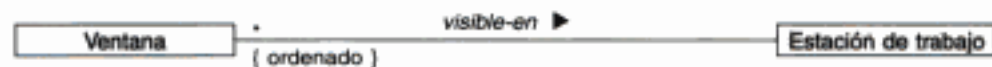


Figura 4.61 Diagrama de clases de una restricción de orden para las ventanas en una estación de trabajo.

4.5.7 Asociaciones derivadas

Las **asociaciones derivadas** se consideran asociaciones *dependientes* o *redundantes*, y están determinadas directa o indirectamente a través de otras asociaciones, que se agregan para facilitar la búsqueda de información.

La notación es una diagonal que atraviesa la asociación, como se muestra en la figura 4.62.

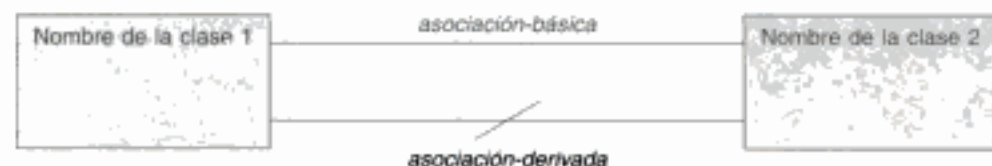


Figura 4.62 Notación para un diagrama de clases que muestra asociaciones derivadas.

Ejemplo: Para la clase *Persona*, puede existir una asociación *padre-de* que define que una persona es el padre y la otra el hijo. Se puede crear una asociación derivada *abuelo-de*, que depende de que existan dos ligas *padre-de*, las cuales definan a uno de los objetos como abuelo y el otro como nieto. La relación *abuelo-de* es una asociación derivada, como se muestra en la figura 4.63. También se podrían definir las relaciones derivadas tío, suegra, primo, las cuales se deducen de otras relaciones básicas como *padre-de*, *esposo* y *hermana*.



Figura 4.63 Diagrama que presenta la clase *Persona*, que contiene *abuelo-de* como una asociación derivada de *padre-de*.

Ejemplo: Un *Profesor* enseña una sola *Materia* a muchos *Estudiantes*, como muestra la figura 4.64. La relación *Estudiante* estudia *Materia* es una asociación derivada, ya que conociendo al *Profesor*, se puede deducir qué *Materia* se les enseña a los *Estudiantes*.

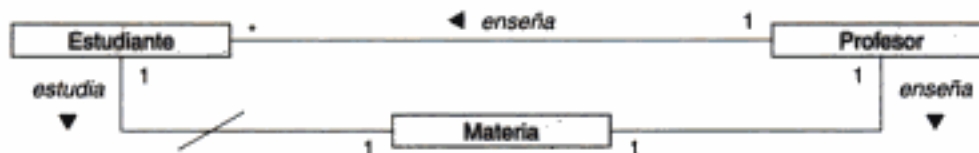


Figura 4.64 Diagrama de clases con asociaciones derivadas redundantes.

No todas las asociaciones que forman múltiples conexiones entre clases indican redundancia. A veces la existencia de una asociación se deriva de dos o más asociaciones primitivas, pero la multiplicidad no. Se debe mantener la asociación adicional, ya que puede ser importante para una restricción adicional de multiplicidad.

Ejemplo: Los *Profesores* enseñan muchas *Materias* a muchos *Estudiantes*, como muestra la figura 4.65. La relación *Estudiante* estudia *Materia* no es en este caso una asociación derivada, ya que, conociendo al *Profesor*, no se puede deducir la *Materia*, pues cada profesor da muchas materias y, por tanto, es necesario agregar la relación adicional para saber qué *Materia* se le enseña a cada *Estudiante*.

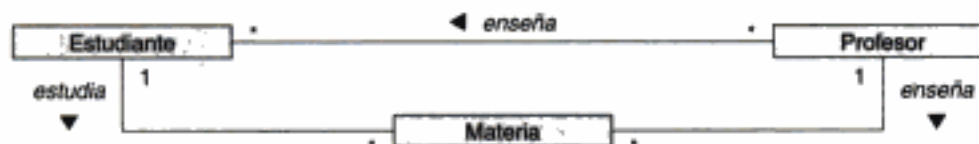


Figura 4.65 Diagrama de clases con asociaciones no redundantes.

4.5.8 Accesos

Las **operaciones de acceso** además de leer o escribir atributos, pueden servir para acceder a las ligas relacionadas con un objeto. Se utiliza la notación de punto para indicar el acceso a una liga: *objeto1.objeto2*.

Ejemplo: Se puede *acceder* a la *cuenta*, la cual está ligada a una *persona*, por medio de la instrucción *persona.cuenta*, que se incluiría dentro de la operación *accesar-cuenta*, como se muestra en la figura 4.66.



Figura 4.66 Diagrama que muestra a una *Persona* relacionada con una *Cuenta*.

Se puede tener acceso a los objetos por medio de “pseudoatributos” de los roles de las asociaciones.

Ejemplo: Se puede tener acceso a la *Universidad* a la cual asiste una *Persona* por medio de su rol de *Estudiante*, utilizando la instrucción *estudiante.universidad*, la cual se incluiría dentro de la operación *accesar-universidad*, como se muestra en la figura 4.67.



Figura 4.67 Diagrama de una *Universidad* relacionada con varias *Personas*, según su rol de *Estudiantes*.

4.5.9 Atributos de liga y asociación

Al igual que un atributo de clase es propiedad de la clase, un **atributo de asociación (o atributo de liga)** es propiedad de una asociación. La notación es similar a la usada para los atributos de clases, excepto que se añade a la asociación y no se incorpora un nombre de clase, como se muestra en la figura 4.68.



Figura 4.68 Notación para diagrama de clases con asociaciones que contienen una lista de atributos de asociación.

Ejemplo: Para una asociación entre *Persona* y *Compañía*, se pueden definir los atributos *salario* y *puesto* como atributos de la asociación *trabaja-para*, como se muestra en la figura 4.69.



Figura 4.69 Diagrama de clases para *Persona* y *Compañía* con los atributos de asociación *salario* y *puesto*.

La notación en el diagrama de objetos es similar, como se muestra en la figura 4.70.

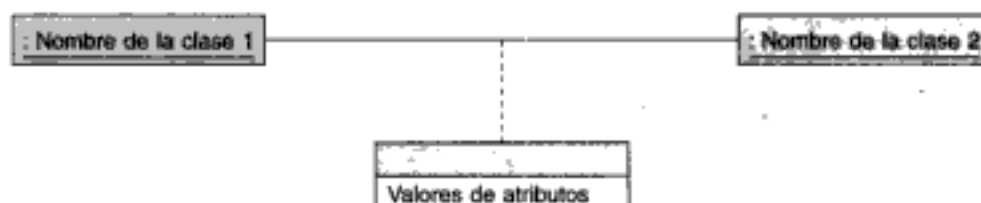


Figura 4.70 Notación para diagrama de objetos con ligas y valores de atributos en las ligas.

Ejemplo: Para una liga entre *Raúl González* y *Compañía Tecnológica S.A.*, se da un valor de \$100 000 al salario y un valor de gerente al puesto, como se muestra en la figura 4.71.

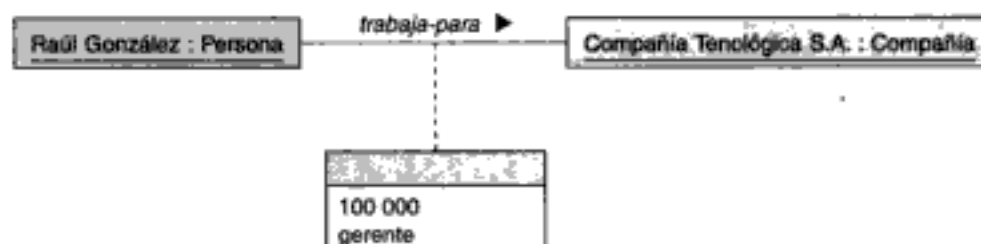


Figura 4.71 Diagrama de objetos para *Raúl González* y *Compañía Tecnológica S.A.* con valor de \$100 000 para el atributo de asociación *Salario*, y *Gerente* para el atributo de liga *Puesto*.

MODELO ALTERNATIVO

A continuación se presentan diferentes alternativas al modelo como atributo de asociación. Se puede modelar tal atributo como atributo de una de las clases

involucradas en la relación, según la multiplicidad de la relación. Aunque las alternativas son posibles, es conceptualmente más correcto describir los atributos, que dependen de ambas clases, como atributo de asociación.

Si la multiplicidad de la asociación es de "uno-uno", el atributo de asociación se podría modelar como un atributo en cualquiera de las dos clases.

Ejemplo: En la figura 4.72 se incluyen *Salario* y *Puesto* como parte de *Persona* (podría también ser parte de *Compañía*).



Figura 4.72 Diagrama de clases para *Persona* y *Compañía*, que incluye multiplicidad "uno-uno" y los atributos de asociación *salario* y *puesto*, incluidos directamente en la clase *Persona* (o de forma alterna en la clase *Compañía*).

Si la multiplicidad de la asociación es de "uno-muchos", el atributo de asociación se podría modelar como un atributo en el lado de la clase "muchos".

Ejemplo: En la figura 4.73 se incluyen *salario* y *puesto* como parte de *Persona*, ya que *Persona* está del lado "muchos" de la asociación. Del otro lado sería incorrecto, ya que significaría que todas las *Personas* estarían ganando el mismo salario y tendrían el mismo puesto.



Figura 4.73 Diagrama de clases para *Persona* y *Compañía* que contiene multiplicidad "uno-muchos" y el atributo de asociación *salario* y *puesto* incluidos directamente en la clase *Persona*.

Si la multiplicidad de la asociación es de "muchos-muchos", no es posible modelarlo como un atributo de clase.

Ejemplo: En la figura 4.74 se muestra de forma incorrecta la incorporación de *salario* y *puesto* a *Persona* (o de forma alterna a *Compañía*), ya que la relación es de "muchos-muchos", y esto significaría que una *Persona* tendría el mismo *salario* y el mismo *puesto* en todas las *Compañías* en las cuales trabajara. Esto no es necesariamente correcto. La representación correcta será mostrada más adelante.



Figura 4.74 Diagrama de clases incorrecto para *Persona* y *Compañía* que tiene los atributos de asociación *salario* y *puesto* incluidos directamente en la clase *Persona*.

ASOCIACIONES TERNARIAS

Los atributos de asociación también pueden existir para las asociaciones ternarias.

Ejemplo: Un *Estudiante* toma una *Materia* en una *Universidad* donde se le da una *Calificación*, la cual depende de las tres clases, como se muestra en la figura 4.75.

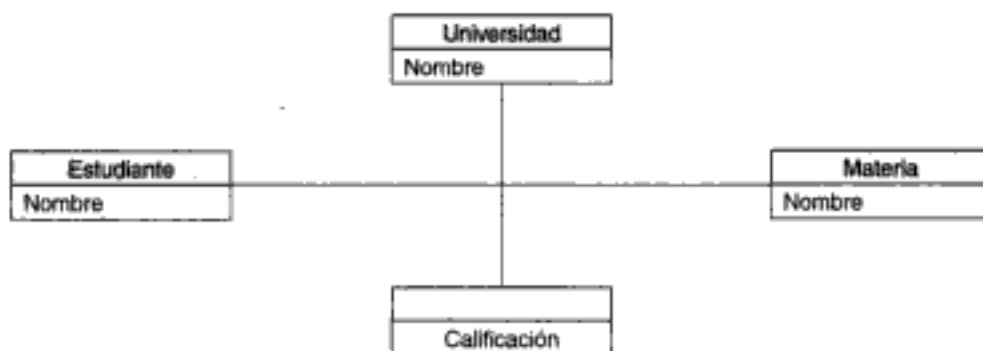


Figura 4.75 Diagrama de clases mostrando una asociación ternaria entre las clases *Estudiante*, *Materia* y *Universidad* incluyendo el atributo *Calificación*.

Ejemplo: En la figura 4.76 se muestra un ejemplo de modelo de objetos para la relación ternaria anterior.

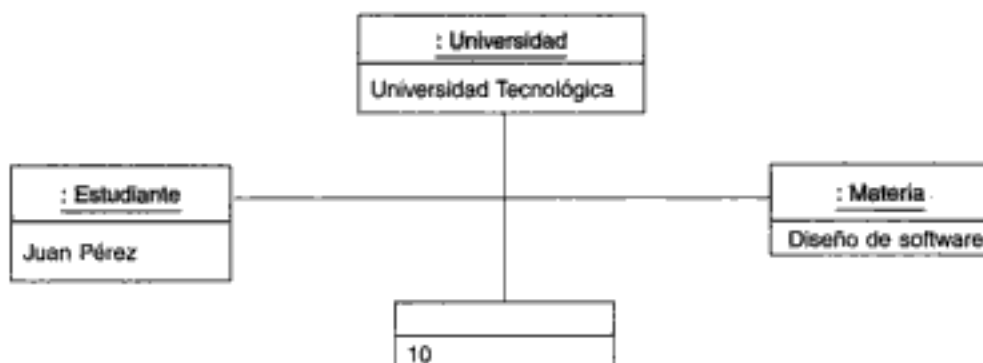


Figura 4.76 Diagrama de instancias para *Estudiante*, *Materia* y *Universidad*, que incluyen un valor para el atributo *Calificación* para la asociación ternaria entre estas clases.

4.5.10 Operaciones de asociación

Se pueden modelar como operaciones de asociación aquellas operaciones que dependan de las clases involucradas en la relación, de forma análoga a los atributos de asociación.

La notación se muestra en la figura 4.77.



Figura 4.77 Notación para el diagrama de clases con atributos y operaciones de asociación.

Ejemplo: La operación *Calcular-Ganancia* depende del *Salario* de la *Persona* con respecto a la *Compañía*, por lo cual se modela como operación de asociación, como se ilustra en la figura 4.78.

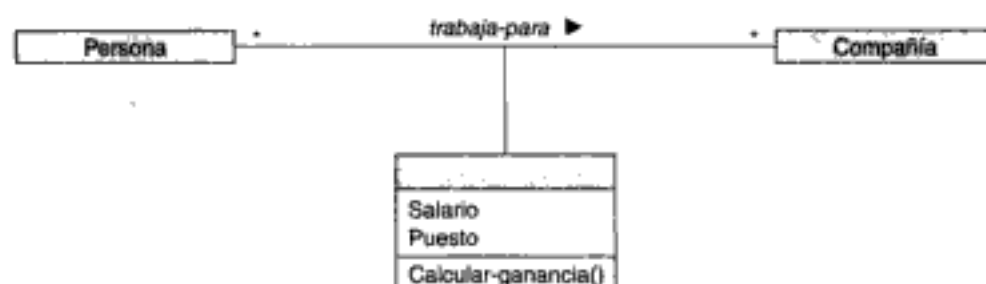


Figura 4.78 Diagrama de clases con los atributos y operaciones de asociación: *Salario* y *Puesto*, y *Calcular-Ganancia*, respectivamente.

4.5.11 Asociaciones como clases

Se puede modelar una asociación como si fuera una clase, en particular si se desea asociar la propia asociación con otras clases. Los atributos y operaciones de la asociación pasan a ser miembros de la clase correspondiente a la asociación.

La notación se muestra en la figura 4.79.

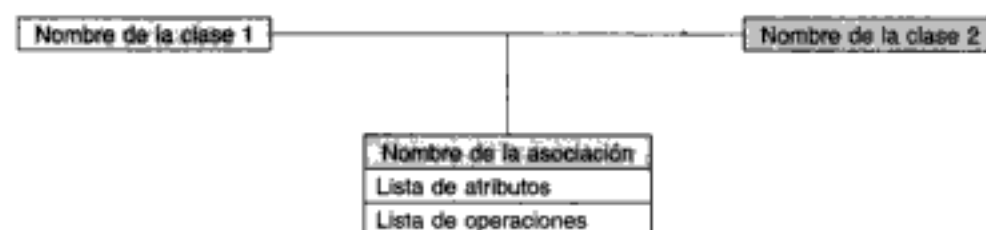


Figura 4.79 Notación para diagrama de clases modelando la asociación como una clase.

Ejemplo: La relación *trabaja-para* entre *Persona* y *Compañía*, se puede convertir en una clase, si pensamos en relaciones con otra clase como *seguro-de-trabajo* para cada *Persona* que esté trabajando en una *Compañía*, como se muestra en la figura 4.80.

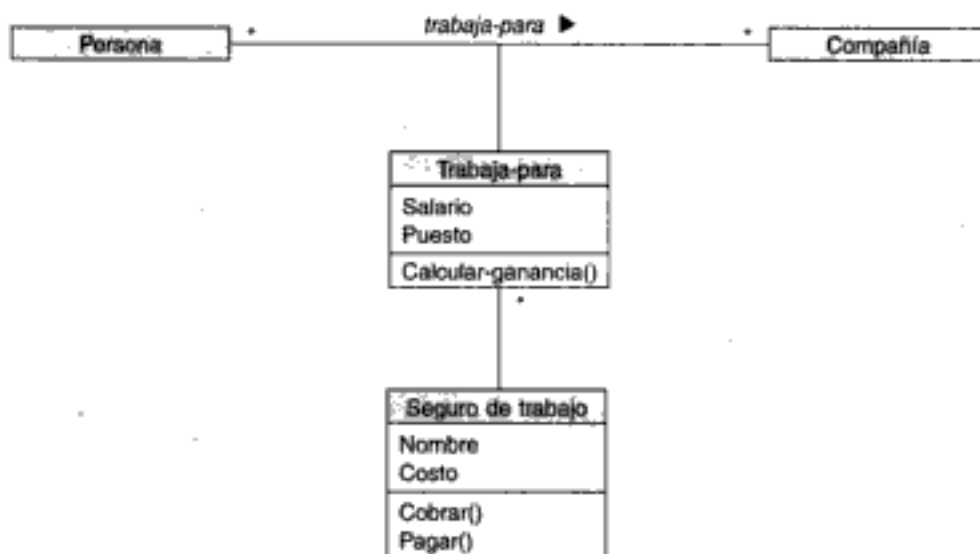


Figura 4.80 Diagrama de clases para una asociación, *trabaja-para*, modelada como una clase.

4.6 Ensamblados: agregación y composición

Los **ensamblados**, en particular la **agregación** y la **composición**, son formas especiales de asociación entre un todo y sus partes, en donde el **ensamblado** está compuesto por sus componentes. El ensamblado es el **objeto** central, y la estructura completa se describe como una **jerarquía de contenido**. Un ensamblado puede componerse de varias partes, donde cada relación *parte-todo* se considera una relación separada. En el caso de la **agregación**, las partes del ensamblado pueden aparecer en múltiples ensamblados. En el caso de la **composición**, las partes del ensamblado no pueden ser compartidas entre ensamblados.

Ejemplo: Una *Red de computadoras* se puede considerar un ensamblado, donde las *Computadoras* son sus componentes. Éste también es un ejemplo de agregación, ya que las *Computadoras* pueden ser partes de múltiples *Redes de computadoras* a la vez. Además, las *Computadoras* pueden existir independientemente de la existencia de la *Red de computadoras*.

Ejemplo: Un *Automóvil* también se puede considerar un ensamblado, donde el *Motor* y la *Carrocería* son sus componentes. Éste es un ejemplo de composición, ya que el *Motor* y la *Carrocería* son partes del *Automóvil*, y a diferencia de la agregación, no pueden ser compartidos entre varios *Automóviles* a la vez. Además, no tiene mucho sentido que el *Motor* y la *Carrocería* existan de manera independiente al *Automóvil*, por lo cual la composición refleja de manera importante el concepto de *propiedad*.

El ensamblado tiene propiedades de **transición**: Si A es parte de B y B es parte de C; entonces A es parte de C.

Ejemplo: Si el *Motor* es parte del *Automóvil*, entonces sus propiedades, como su posición y velocidad, están dadas por la posición y velocidad del *Automóvil*.

El ensamblado es **antisimétrico**: Si A es parte de B, entonces B no es parte de A. Estas propiedades se propagan entre el ensamblado y sus componentes.

Ejemplo: Si el *Motor* es parte del *Automóvil*, entonces el *Automóvil* no es parte del *Motor*.

Se considera un ensamblado y no una asociación regular:

- ▶ Si se puede usar la frase "parte-de" o "consiste-de" o "tiene".
- ▶ Si algunas operaciones en el todo pueden *propagarse* a sus partes.
- ▶ Si algunos atributos en el todo se pueden propagar a sus partes.

El ensamblado es común en los objetos interface. En un sistema de ventanas, por ejemplo, una ventana puede consistir en botones, menús y barras (*scroll-bars*), cada uno modelado por su propio objeto interface. El resultado es una estructura de interface en forma de árbol. La decisión de usar ensamblados es un poco arbitraria, y en la práctica no causa grandes problemas la distinción imprecisa entre agregación, composición y asociación, aunque es bueno ser consistente.

La notación para un ensamblado, en particular para un agregado, es un diamante adherido al lado del objeto correspondiente al ensamblado total, conectado por una línea a sus componentes, como se muestra en la figura 4.81.



Figura 4.81 Notación en un diagrama de clases para una agregación.

La notación para la composición es similar a una agregación y al ensamblado en general, aunque a diferencia de la agregación, el diamante se rellena de color negro, como se muestra en la figura 4.82.



Figura 4.82 Notación en un diagrama de clases para una composición.

Existe una notación adicional para la composición descrita como un anidamiento gráfico, como se muestra en la figura 4.83.



Figura 4.83 Notación en un diagrama de clases para una composición.

Ejemplo: El *Automóvil* con sus componentes: *Motor* y *Carrocería*, se muestran en el diagrama de la figura 4.84.



Figura 4.84 Diagrama de clases para la composición de un *Automóvil* que contiene un componente *Motor* y otro *Carrocería*.

Ejemplo: Una *Computadora personal* (PC) está compuesta por uno o varios *Monitores*, un *Sistema*, un *Teclado* y, opcionalmente, un *Ratón*. El sistema tiene un *Chasis*, un *Procesador central* (CPU), varias *Tarjetas de memoria* (RAM) y opcionalmente un *Ventilador*. El diagrama se muestra en la figura 4.85.

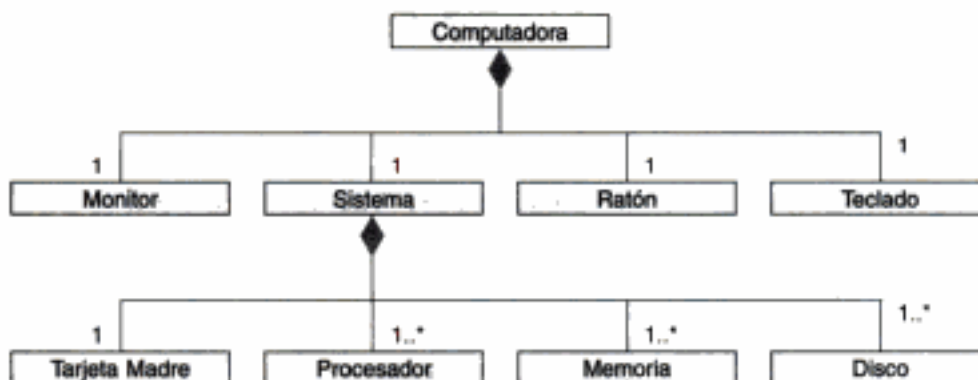


Figura 4.85 Diagrama de clases para la composición de una *Computadora personal* (PC) con diferentes componentes.

Ejemplo: Una *Universidad* está compuesta por sus *Divisiones*, que están a su vez compuestas de sus *Departamentos*. Una *Universidad* es indirectamente una composición de sus *Departamentos*; pero la *Universidad* no es una composición de sus *Estudiantes*, sino más bien una agregación, ya que sus *Estudiantes* son objetos independientes, incluso pudiendo pertenecer a múltiples *Universidades*, como se muestra en la figura 4.86. (Este es un ejemplo de ensamblado variable, como se describe en la siguiente sección.)



Figura 4.86 Diagrama de clases para la composición de una *Universidad* que contiene componentes de *División*, que a su vez contiene componentes de *Departamento*. La relación entre *Universidad* y *Estudiante* es de agregación.

4.6.1 Tipos de ensamblados

Los tipos de ensamblados pueden ser:

Fijos. Los que tienen una estructura fija donde el número de componentes está predefinido.

Ejemplo: Un *Automóvil* tiene un *Motor*, una *Carrocería* y cuatro *Ruedas*, como se muestra en la figura 4.87.

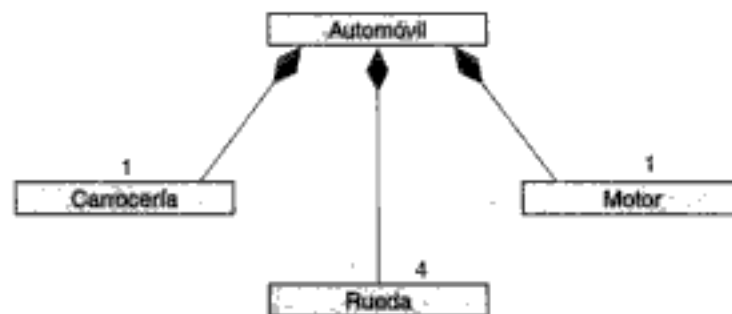


Figura 4.87 Diagrama de clases para un ensamblado fijo de un *Automóvil* que contiene varios componentes: un *Motor*, una *Carrocería* y cuatro *Ruedas*.

Variables. Tienen un número finito de niveles, pero el número de componentes varía.

Ejemplo: Un *País* contiene varias *Ciudades*, que contienen a su vez varias *Casas*. No se sabe cuántas ciudades existen, ni tampoco cuántas casas, aunque sí se sabe el nivel del ensamblado, como se muestra en la figura 4.88.



Figura 4.88 Diagrama de clases para un ensamblado variable de un *País* que contiene varias *Ciudades*, que a su vez contienen varias *Casas*.

Recursivos. Los ensamblados **recursivos** contienen de forma directa o indirecta una instancia del mismo tipo de agregado, donde el número de niveles de ensamblado es potencialmente ilimitado.

Ejemplo: Un *Directorio* en un sistema operativo está definido de forma recursiva, pudiendo contener otros directorios, que a su vez pueden incluir otros directorios, y así sucesivamente de forma indefinida, como se muestra en la figura 4.89.



Figura 4.89 Diagrama de clases para un ensamblado recursivo de un *Directorio*, que puede contener otros *Directorios* de forma recursiva.

4.6.2 Propagación de operaciones

Uno de los objetivos del ensamblado es que las operaciones aplicadas a éste puedan **propagarse** de forma automática a sus objetos componentes. La operación se propaga en una sola dirección, y puede ser aplicada a las partes del ensamblado de forma independiente.

La notación para la propagación de operaciones se muestra en la figura 4.90. La propagación se indica con una flecha, en el sentido de la propagación, junto al nombre de la operación que etiqueta la asociación.



Figura 4.90 Notación de un diagrama de clases para la propagación de operaciones en un ensamblado.

Ejemplo: Cuando el *Automóvil* se mueve, la operación de moverse se propaga por el *Automóvil*, y también se mueven todas sus partes, como el *Motor*, la *Carrocería* y las *Ruedas*. La operación no se propaga en sentido contrario, ya que, por ejemplo, la *Carrocería* no puede moverse sin que lo haga el *Automóvil* completo, como se muestra en la figura 4.91. También se pudieran propagar otras operaciones, como *pintar*, *vender*, *comprar*, etcétera.

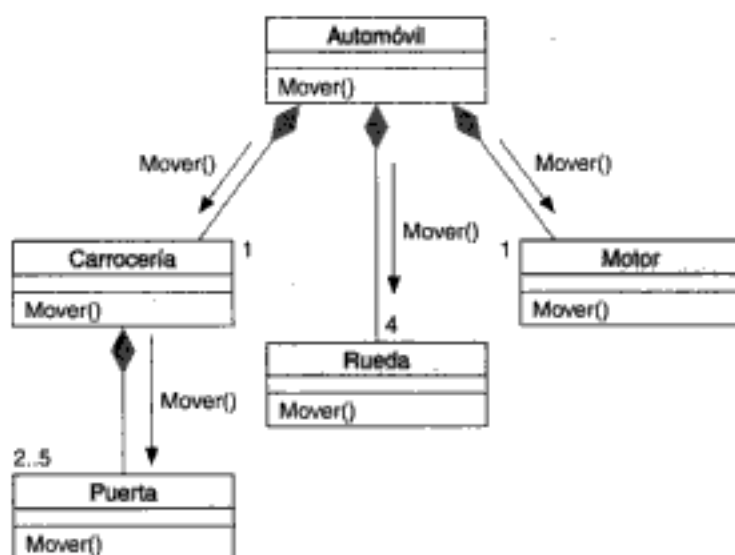


Figura 4.91 Diagrama de clases para la propagación de la operación mover del ensamblado *Automóvil* a *Carrocería*, y luego a *Puerta*, la cual es parte a su vez de la *Carrocería*. La operación se propaga a todos los componentes.

4.7 Generalización y herencia

Las clases con atributos y operaciones comunes se pueden organizar de forma jerárquica mediante la **herencia**, ésta es una abstracción importante para compartir similitudes entre clases, donde todos los atributos y operaciones comunes a varias clases se pueden compartir por medio de la **superclase**, una clase más general. Las clases más refinadas se conocen como **subclases**.

Ejemplo: Las *Impresoras láser*, de *Burbuja* y de *Matriz*, son todas subclases de la superclase *Impresora*. Los atributos generales de una *Impresora* son el *Modelo*, *Velocidad* y *Resolución*, mientras que sus operaciones son *Imprimir* y *Alimentar*.

La herencia es una relación "es-una" entre las clases más refinadas y generales.

Ejemplo: *Impresora láser* es una *Impresora*.

La **herencia** es útil para el modelo conceptual, al igual que para la implementación; como modelo conceptual da buena estructuración a las clases. Como modelo de implementación es un buen vehículo para no replicar innecesariamente el código. La **generalización** define una relación entre una clase más *generalizada*, y una o más versiones refinadas de ella.

Ejemplo: La clase *Impresora* es una generalización de las clases *Impresoras láser*, de *Burbuja* y de *Matriz*.

El opuesto de la generalización es la **especialización**, que define una relación entre una clase más general y una o más versiones *especializadas* de ella.

Ejemplo: *Impresoras láser*, de *Burbuja* y de *Matriz* son todas especializaciones de la clase más general *Impresoras*.

La superclase *generaliza* a sus subclases y éstas *especializan* a la superclase. El proceso de especialización es el inverso de generalización. Una instancia de una subclase, o sea un objeto, es también una instancia de su superclase.

Ejemplo: Cuando se crea un objeto de tipo *Impresora láser*, este objeto incluye toda la información descrita en la subclase *Impresora láser*, al igual que en la superclase *Impresora*; por tanto, se considera que el objeto es una instancia de ambas.

La herencia es transitiva a través de un número arbitrario de niveles. Los **ancestros** de una clase son las superclases de una clase en cualquier nivel superior de la jerarquía, y los **descendientes** de una clase son las subclases de una clase en cualquier nivel inferior de la jerarquía.

Ejemplo: Si además de *Impresora de burbuja*, se define una clase más especializada como *Impresora de burbuja portátil*, entonces *Impresora* e *Impresora de burbuja* son ancestros de la clase *Impresora de burbuja portátil*, mientras que *Impresora de burbuja* e *Impresora de burbuja portátil* son descendientes de *Impresora*.

Las siguientes características se aplican a clases en una jerarquía de herencia:

- ▶ Los valores de una instancia incluyen valores para cada atributo de cada clase ancestral.
- ▶ Cualquier operación de cualquier clase ancestral se puede aplicar a una instancia.
- ▶ Cada subclase no sólo hereda todas las características de sus ancestros, sino también añade sus propios atributos y operaciones.

(Los nombres de atributos y operaciones deben ser únicos en la jerarquía de herencia.)

Ejemplo: Una *Impresora de burbuja portátil* incorpora todas las características, primero de una *Impresora* y luego de una *Impresora de burbuja*, que contiene valores para todos los atributos ancestrales y puede aplicar todas las operaciones de sus ancestros (siempre y cuando éstas no sean privadas a la superclase).

La generalización se puede extender a múltiples niveles de jerarquías, donde una clase hereda de su superclase, que a su vez hereda de otra superclase, hacia arriba en la jerarquía. En otras palabras, las relaciones entre subclases y superclases son relativas. La herencia define una jerarquía de clases donde existen *ancestros* y *descendientes*, que pueden ser directos o no.

Para representar herencia y generalización, se utiliza un triángulo conectando a la superclase con sus subclases. La superclase está del lado superior del vértice del triángulo, mientras que las subclases están en la parte inferior de la base del triángulo, como se muestra en la figura 4.92.

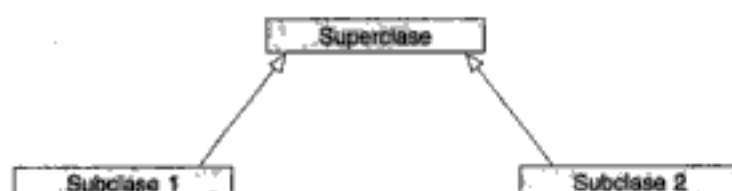


Figura 4.92 Notación en diagrama de clases para generalización y herencia.

Ejemplo: Un *Barco* tiene un *Nombre*, *Fabricante* y *Costo*. Tipos especiales de *Barco*, como *Velero*, tienen además de estas características básicas, un *Número de velas*, mientras que otro tipo especial de *Barco*, como *Barco de motor*, tiene un *Motor*. *Barco* es la clase básica (la superclase), mientras que *velero* y *barco de motor* son las clases refinadas (las subclases). Se deben definir las características básicas de los barcos una sola vez, y luego añadir detalles para veleros, barcos de motor, etc. En la figura 4.93 se muestra el diagrama de clases, que describe la relación de herencia.



Figura 4.93 Diagrama de clases que describe herencia de la superclase *Barco* a las subclases *Velero* y *Yate*. Se incluyen atributos para las diferentes clases.

Ejemplo: Una jerarquía con una superclase *Mueble* y varias subclases *Mesa* y *Asiento*, puede ser extendida con nuevas subclases, como *Mesa circular*, *Mesa rectangular*, mientras que un *Asiento* puede extenderse con las subclases *Silla*, *Sillón* y *Taburete*, como se muestra en la figura 4.94. Cada clase tiene

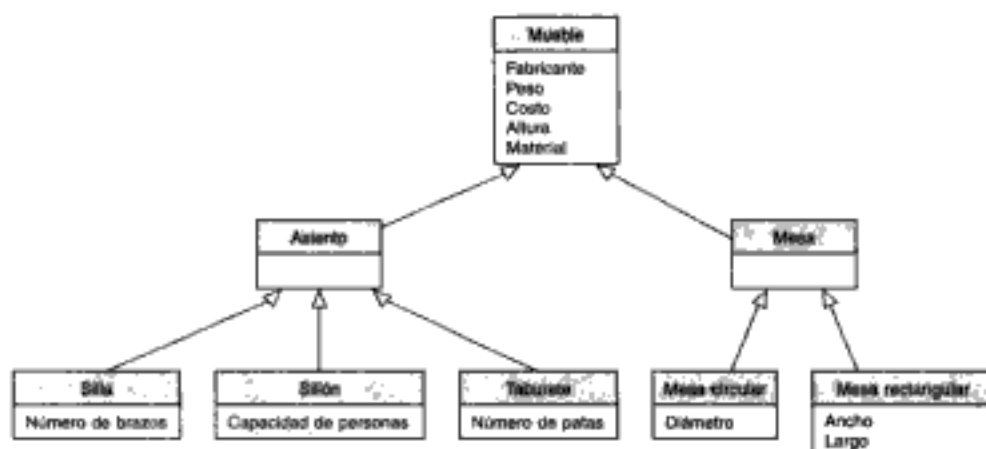


Figura 4.94 Diagrama de clases que describe diferentes tipos de *Mueble*, *Asiento* y *Mesa*, con sus respectivas subclases.

sus propios atributos, los cuales se van especializando a medida que las clases son cada vez más especializadas. Note que no necesariamente todas las clases tienen que incluir atributos.

Ejemplo: En la figura 4.95 se muestran instancias de *Sillón* y *Mesa circular* con valores para los distintos atributos.

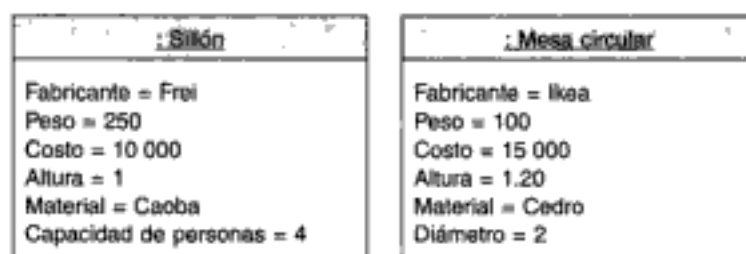


Figura 4.95 Diagrama de instancias para un *Sillón* y una *Mesa circular*.

La **agregación** o la **composición** no son lo mismo que la generalización, el ensamblado relaciona clases correspondientes a muchos objetos; mientras que la generalización relaciona clases, que finalmente corresponden a un solo objeto. La **agregación** o **composición** es una forma de estructurar la descripción de un solo objeto; mientras que con la generalización, un solo objeto es una instancia de la combinación de su superclase y subclases. El ensamblado es una relación *parte-de*, en cambio generalización es una relación *tipo-de* o *es-una*.

Ejemplo: Un *Automóvil* está compuesto de un *Motor*, una *Carrocería* y cuatro *Ruedas*. El *Automóvil* puede ser clasificado como *Automóvil deportivo* y *Automóvil sedán*. Cada subclase puede tener sus propias partes, como *Rin de magnesio* o *Parrilla para maletas*. *Rin de magnesio* es subclase de *Rin*, el cual es un componente de *Rueda*, al igual que *Llanta*. El diagrama se muestra en la figura 4.96.

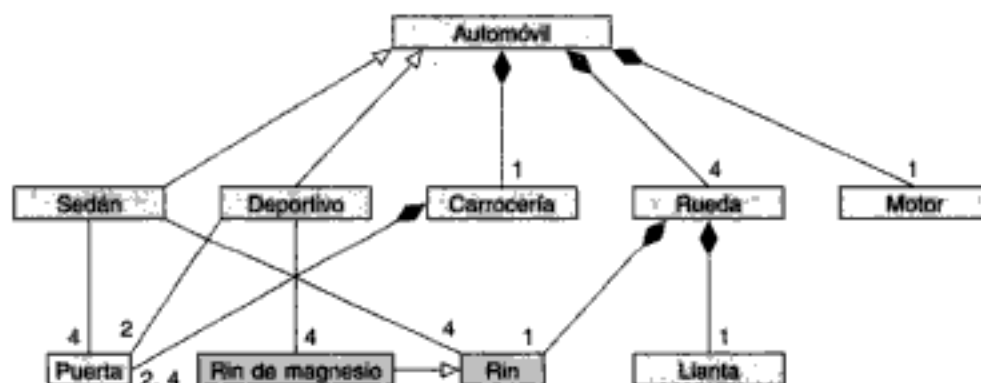


Figura 4.96 Diagrama de clases para un ensamblado de un *Automóvil* que contiene varios componentes: un *Motor*, una *Carrocería* y cuatro *Ruedas*. Pueden haber diferentes tipos de *Automóvil*: *Deportivo* o *Sedán*, que contienen *Rines de magnesio* o una *Parrilla para maletas*, respectivamente. *Rin de magnesio* es subclase de *Rin*, el cual es componente de *Rueda* al igual que *Llanta*.

Ejemplo: Una clase *Ventana* tiene atributos para los vértices de la ventana y operaciones para *Desplegar*, *Ocultar*, *Mover* y *Modificar* la ventana. *Canvas*, *Panel* y *Ventana* de texto son tipos diferentes de ventanas. Un *Canvas* se utiliza para diferentes despliegues gráficos, incluyendo atributos como el tamaño del elemento gráfico y operaciones para *Añadir* y *Borrar* tales elementos; y se relaciona con varios *Elementos* (formas) que son *Líneas* o *Formas cerradas*, como *Elipses* o *Polígonos*. Un *Polígono* consiste en una lista ordenada de *Puntos*. Un *Panel* contiene diferentes *Artículos de panel*, los cuales pueden ser de tipo *Botón*, *Selección* o *Texto*. Todos los *Artículos de panel* están relacionados con *Eventos* del ratón, y el artículo de tipo *Texto* se asocia además con un *Evento* del teclado. Cuando un *Artículo de panel* se escoge, un *Evento* se genera. Una *Selección* se relaciona con diferentes *Selecciones posibles*, aunque sólo una puede escogerse a la vez. El diagrama se ilustra en la figura 4.97.

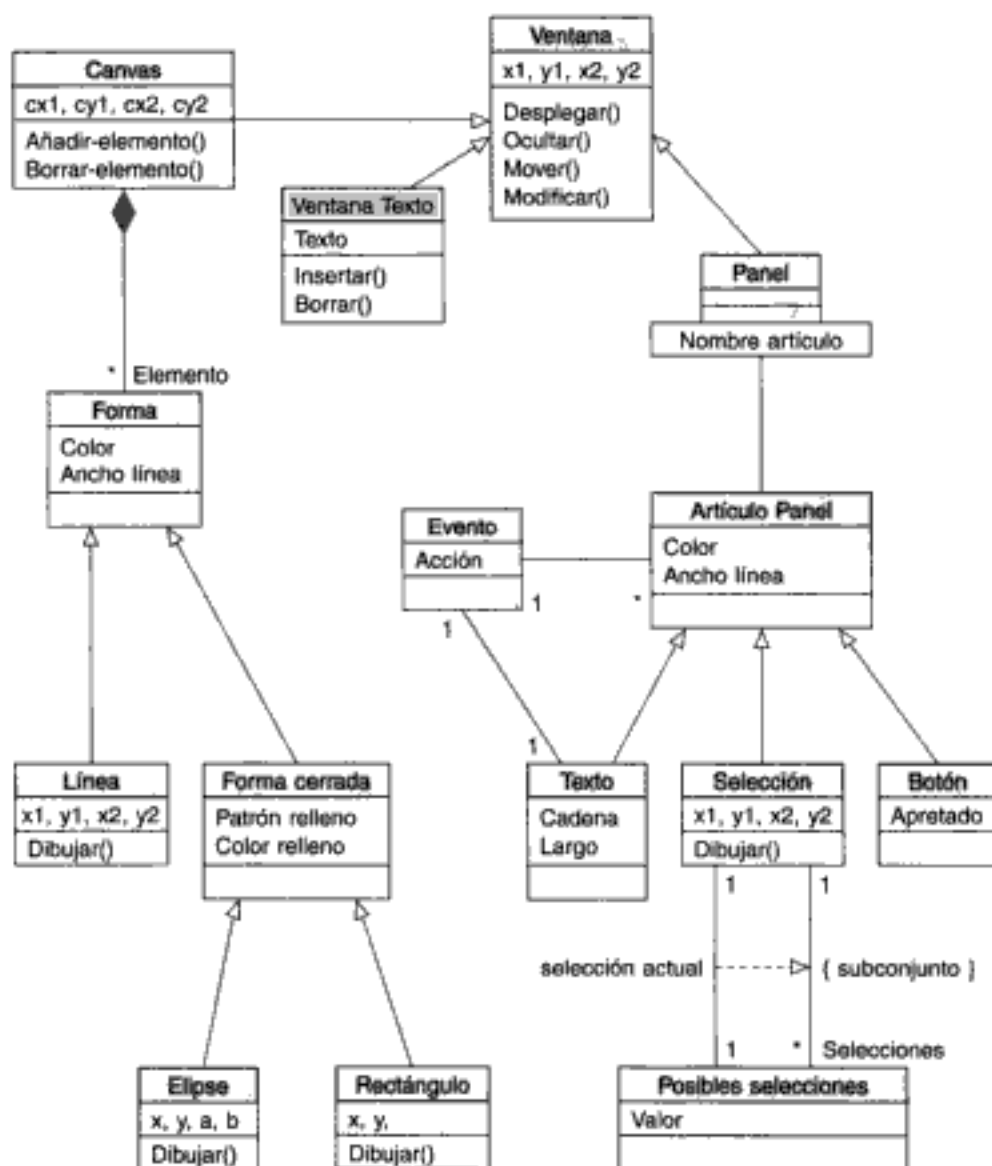


Figura 4.97 Diagrama de clases para un sistema de ventanas.

Ejemplo: La clase *Persona* tiene *Nombre*, *Dirección* y *Número del Seguro Social*. Una persona puede trabajar en algún proyecto y ganar un salario. Una *Compañía* tiene un *Nombre*, *Dirección*, *Número telefónico* y *Producto primario*. Una *Compañía* contrata y despide *Personas*. *Persona* y *Compañía* tienen una relación "muchos-muchos". El título del trabajo depende de la persona y de la compañía. Hay dos tipos de *Personas*: *Trabajadores* y *Administradores*. Cada *Trabajador* está involucrado en varios *Proyectos*; cada *Administrador* es responsable de varios proyectos. En un proyecto pueden trabajar varios trabajadores y un solo administrador. Cada proyecto tiene *Nombre*, *Presupuesto* y una *Prioridad interna* para asegurar recursos. Además, una *Compañía* está compuesta de múltiples *Departamentos*; cada departamento dentro de una compañía se identifica de forma única por su *Nombre*. Un departamento usualmente tiene un administrador. La mayoría de los administradores manejan un departamento; y algunos administradores no están asignados a ningún departamento. Cada departamento manufactura varios productos; mientras que cada producto está hecho por un solo departamento. Un *Producto* tiene *Nombre*, *Costo* y *Peso*. El diagrama se muestra en la figura 4.98.

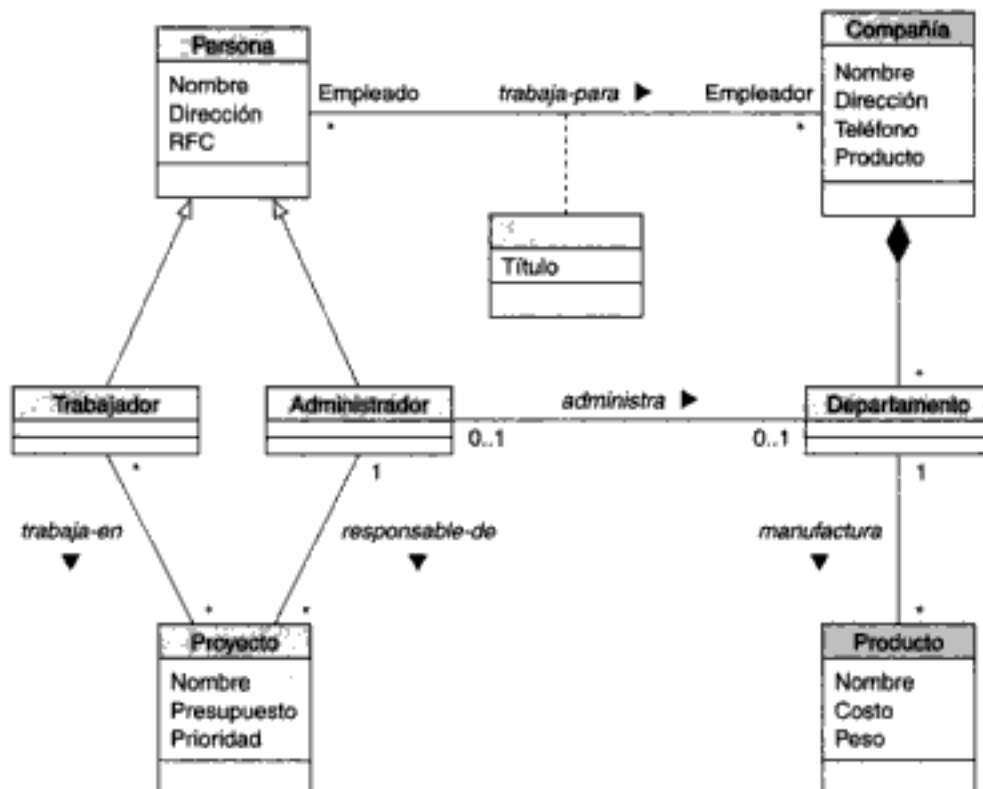


Figura 4.98 Diagrama de clases para *Personas* que trabajan en *Compañías*.

La generalización puede hacerse por diferentes razones:

- **Extensión:** clases definidas por operaciones con estructuras de información.
- **Restricción:** clases como implementaciones de tipos, especializaciones, subconjunto de todas las instancias de un ancestro.

4.7.1 Extensión de clase

La **extensión de clase** expresa que una subclase puede añadir nuevos atributos u operaciones a la superclase.

Ejemplo: La clase *Rectángulo* añade los nuevos atributos: *Ancho* y *Largo*, como lo muestra la figura 4.99.

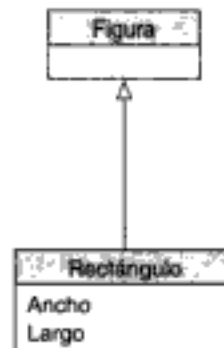


Figura 4.99 Diagrama de clases para una herencia que describe una superclase *Figura* y una subclase *Rectángulo*.

4.7.2 Restricción de clase

La **restricción** de clase indica cómo una subclase puede restringir los atributos heredados de una superclase. Una clase descendiente no puede suprimir los atributos u operaciones de sus ancestros.

Ejemplo: La clase *Cuadrado* restringe los atributos de *Rectángulo*, ya que *Ancho* debe ser igual a *Largo*, como se muestra en la figura 4.100.

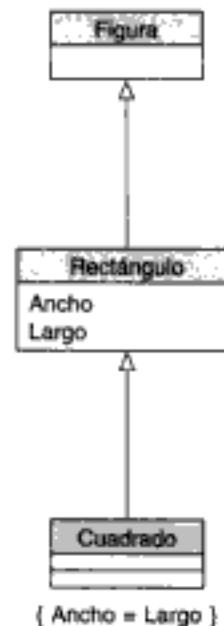


Figura 4.100 Diagrama de clases para una herencia de una superclase *Figura*, una subclase *Rectángulo* y una nueva subclase *Cuadrado*.

Los cambios arbitrarios a los valores de los atributos pueden violar las restricciones de una clase creada por medio de restricciones. La restricción define la condición de la membresía en una clase, donde todos los objetos cuyos valores satisfacen la regla pertenecen a la clase.

Ejemplo: La clase *Cuadrado* debe suprimir la *Escala* desigual, ya que una escala arbitraria en las dimensiones de los ejes puede resultar en un *Rectángulo* y no en un *Cuadrado*. (La clase *Rectángulo* está cerrada bajo la operación de *Escala*, pero no la clase *Cuadrado*.)

4.7.3 Sobreescritura de operaciones

Una subclase puede sobrecribir atributos y operaciones de una superclase, al definir nuevos atributos y operaciones con el mismo nombre. Se pueden sobrecribir atributos, por ejemplo, para redefinir sus valores por omisión, y se pueden sobrecribir operaciones para mejorar el rendimiento de un algoritmo. (No se puede sobrecribir las *firmas* de los atributos o de las operaciones.)

Ejemplo: La operación *Desplegar* se define en la superclase *Figura* y se re-define (sobrecribe) en varias de las subclases, como *Punto* y *Línea*.

En general, la sobreescritura va contra el principio de herencia, ya que se dejan de heredar ciertos aspectos del ancestro. Sin embargo, la sobreescritura es parte fundamental de la orientación a objetos, en particular del polimorfismo.

Se pueden sobrecribir operaciones por varias razones: *extensión*, *restricción*, *optimización* e *implementación*.

Extensión. En la **sobreescritura por extensión**, una nueva operación es igual a una heredada, excepto que agrega algún comportamiento nuevo afectando algunos atributos de la subclase.

Ejemplo: La clase *Ventana* tiene una operación *Dibujar* para *Borde* y *Contenido*, mientras que la clase *Ventana-etiquetada* tiene una operación *Dibujar*, donde el método *Dibujar-ventana-etiquetada* llama al método *Dibujar* de *Ventana*, agregando un código para dibujar la *Etiqueta*, como se ilustra en la figura 4.101.



Figura 4.101 Diagrama de clases para una *Ventana* y una *Ventana-etiquetada*.

Restricción. En la **sobreescritura por restricción**, una nueva operación restringe el protocolo de la operación (*firma*), como el tipo de argumentos.

Ejemplo: Una superclase *Conjunto* puede tener una operación *Sumar (Objeto)*. La subclase *Conjunto-entero* tiene una operación más restringida *Sumar (entero)*. El diagrama se muestra en la figura 4.102.



Figura 4.102 Diagrama de clases para un *Conjunto* y un *Conjunto-entero*, que restringe la operación *Sumar*.

Optimización. En la **sobreescritura por optimización**, la implementación de una clase puede aprovechar las restricciones impuestas para mejorar el código de la operación. El protocolo externo debe ser el mismo para la nueva operación, aunque internamente sea totalmente diferente.

Ejemplo: La superclase *Conjunto-entero* puede tener una operación *Máximo* para encontrar el *Entero* más grande. El método podría ser implementado usando una búsqueda secuencial. La subclase *Conjunto-entero-ordenado* puede proveer una implementación más eficiente del método *Máximo*, considerando que los números ya están ordenados, como se muestran en la figura 4.103, optimizando la operación *Máximo*.

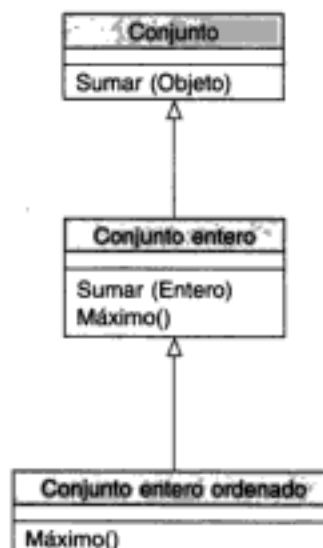


Figura 4.103 Diagrama de clases para un *Conjunto*, un *Conjunto-entero* y un *Conjunto-entero-ordenado*, que optimiza la operación *Máximo*.

Implementación. Nos es conveniente desarrollar nuevas clases sobre escribiendo los métodos de las superclases simplemente por conveniencia de implementación, **sobreescritura por implementación**, sobre las cuales no existe ninguna relación conceptual. Se puede introducir herencia adicional en el sistema durante la etapa de diseño.

Ejemplo: Las clases *Persona* y *Compañía* tienen ambas los atributos *Nombre* y *Dirección*, y se podría crear una superclase que contenga los atributos comunes. Hacer esto durante la etapa del modelo de objetos es incorrecto, ya que las clases *Persona* y *Compañía* son semánticamente diferentes. Crear una superclase, *Dato común*, que contenga ambos atributos, como se muestra en la figura 4.104, sería más bien una decisión hecha durante la etapa del diseño.



Figura 4.104 Diagrama de clases para una *Persona* y una *Compañía*, que comparten atributos comunes guardados en la superclase *Dato común*.

4.7.4 Discriminador

Un **discriminador** indica el atributo de la superclase cuyo valor distingue a las subclases. El discriminador indica cuál propiedad común de las subclases se generaliza en la superclase. (Sólo una propiedad debe ser discriminada a la vez.) La etiqueta "tipo" representa el discriminador más común, por lo que a veces no se incluye. Si el discriminador es obvio no es necesario incluirlo. La notación, que es opcional, se muestra en la figura 4.105.

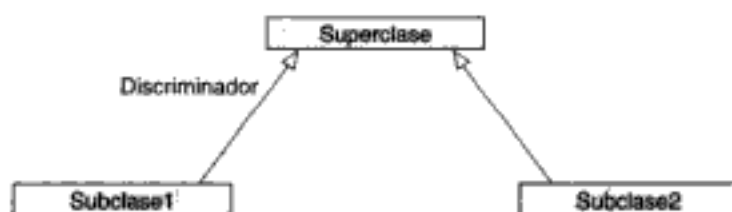


Figura 4.105 Notación para el diagrama de clases con generalización que contiene discriminadores.

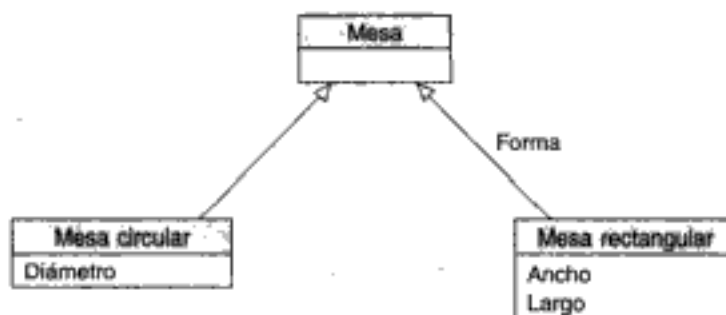
Ejemplo: Un *Asiento* se puede discriminar según su *Funcionalidad*, en *Silla*, *Sillón* o *Taburete*, como se muestra en la figura 4.106.

Figura 4.106
Diagrama de clases para diferentes tipos de Asientos, según su funcionalidad.



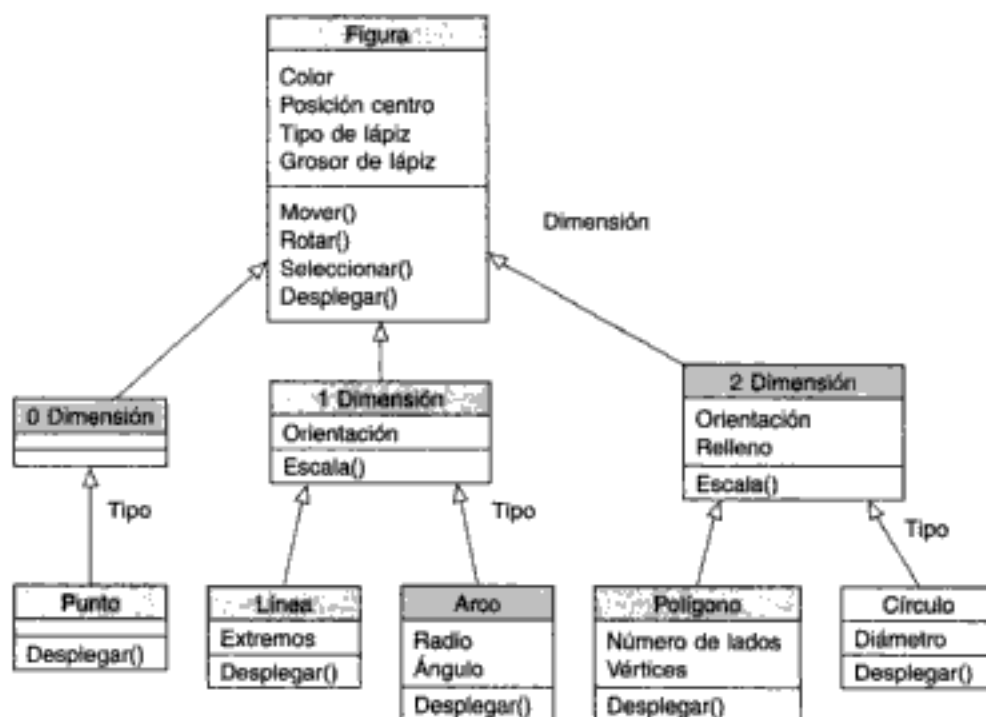
Ejemplo: Una mesa se puede discriminar según su forma, en mesa circular o mesa rectangular, como se muestra en la figura 4.107.

Figura 4.107
Diagrama de clases para diferentes tipos de Mesa según su Forma.



Ejemplo: En la figura 4.108 se muestra un diagrama de clases para figuras gráficas geométricas. Las figuras se discriminan según sus Dimensiones: 0, 1 y 2. La operación *Desplegar* se ha sobrescrito en todas las subclases de último nivel.

Figura 4.108
Diagrama de clases para diferentes tipos de Figuras, según sus Dimensiones.



4.7.5 Clases abstractas

Hasta ahora todas las clases descritas anteriormente se conocían como **clases concretas** o simplemente **clases** donde cualquiera de estas clases tenía la cualidad de ser instanciable. En contraste, una **clase abstracta** es una clase que no tiene directamente instancias, pero cuyas clases descendientes sí las tienen. Las clases abstractas organizan características comunes a varias clases; pueden aparecer naturalmente en la aplicación, o pueden ser introducidas artificialmente para promover la reutilización de código y compartir atributos y métodos.

Ejemplo: Para la clase *Mueble*, la clase *Asiento* es abstracta, ya que para crear un asiento se debe instanciar una de sus tres subclases: *Silla*, *Sillón* o *Taburete*.

Cuando una superclase se divide en subclases por un discriminador, y existe una subclase para cada valor posible del discriminador, entonces la superclase se considera una *clase abstracta*, ésta puede definir métodos para ser usados por la subclase, o definir el protocolo de la operación, o sea el tipo y número de argumentos: el resultado y la intención semántica, sin dar el correspondiente método (operación abstracta). Cada clase *concreta* debe proveer su propia implementación y no debe tener operaciones abstractas.

Ejemplo: Ocupaciones como *Ingeniero*, *Arquitecto* y *Doctor*, son clases concretas. Una superclase *Trabajador*, que guarda aspectos comunes a todas las ocupaciones, sería una clase abstracta, ya que se desea instanciar únicamente trabajadores con una ocupación particular, como se muestra en el diagrama de la figura 4.109.

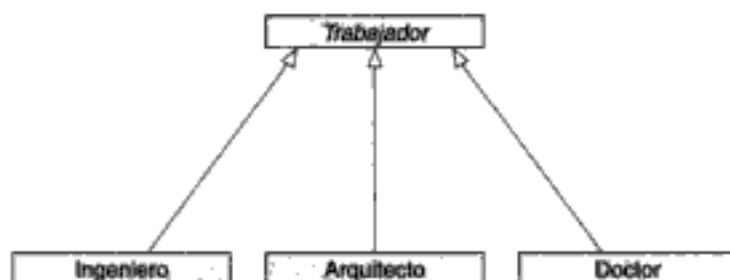


Figura 4.109 Diagrama de clases para una clase abstracta *Trabajador* y diferentes clases concretas *Ingeniero*, *Arquitecto* y *Doctor*. (Note que el nombre de la clase *Trabajador* está en cursivas, porque corresponde a una clase abstracta.)

Ejemplo: En el diagrama de la figura 4.110, se muestra una variante del problema anterior, donde *Trabajador* es una clase concreta, ya que para instanciar a un trabajador general se debe hacer una instancia de la superclase *Trabajador*.

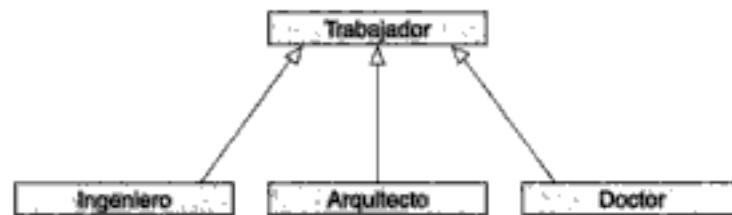


Figura 4.110 Diagrama de clases donde *Trabajador* es una clase concreta, ya que además de instanciar el objeto de las clases concretas *Ingeniero*, *Arquitecto* y *Doctor*, también se pueden instanciar trabajadores no especificados en el diagrama. (Observe que el nombre de la clase *Trabajador* está en letras normales, correspondiente a una clase concreta.)

Ejemplo: La clase *Empleado* es una clase abstracta, ya que debe especificarse si los empleados son *Honorarios* o *Nómina*, como se muestra en la figura 4.111. La operación *Computar pago* es una operación abstracta en la clase abstracta *Empleado*, que requiere su implementación en la subclase *Empleado*. Note que el nombre de la clase abstracta está en letras cursivas.



Figura 4.111 Diagrama de clases donde *Empleado* es una clase abstracta, mientras que *Empleado honorarios* y *Empleado nómina* son clases concretas. La operación *Computar pago* debe ser definida como una operación abstracta en la clase abstracta *Empleado*.

La jerarquía general de clases, para clases abstractas y concretas, se muestra en la figura 4.112. El diagrama muestra la distinción entre clases concretas y abstractas. Comenzando desde arriba y yendo hacia abajo en la jerarquía, podemos ver que la clase "clase" puede especializarse en una "clase concreta" o en una "clase abstracta". La "clase concreta", que es una clase instanciable, incluye una asociación "tiene-subclase" con "clase" donde la multiplicidad es de "0" a "n" ("**"). De tal manera, el árbol *puede* continuar indefinidamente mediante una nueva "clase concreta" o una "clase abstracta". Sin embargo, este árbol puede ser interrumpido gracias al "0". La "clase abstracta" que por definición no es instanciable, tiene una asociación "tiene-subclase" con "clase" donde la multiplicidad es de "1" a "n" ("1..*"). De tal manera, el árbol *debe* continuar indefinidamente mediante una nueva "clase concreta" o una "clase abstracta". Este árbol no puede ser interrumpido por la ausencia de "0". En otras palabras, la jerarquía de herencia sólo puede ser interrumpida mediante una "clase concreta", ya que la "clase abstracta" obliga a continuar nuevamente la jerarquía. De manera adicional, la multiplicidad de "*" del lado de "clase concreta" y "clase abstracta" muestra la opción de tener "0" o más superclases en la jerarquía.

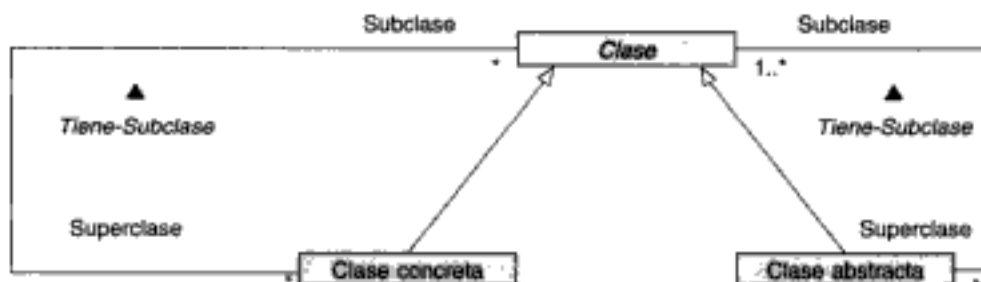


Figura 4.112 Jerarquía general de clases, que describe la relación entre clases abstractas y concretas.

4.7.6 Herencia múltiple

La herencia múltiple permite a una clase tener más de una superclase y heredar aspectos de todos sus ancestros.

Ejemplo: Una jerarquía de clases que contiene una superclase *Vehículo*, con dos subclases *Vehículo de agua* y *Vehículo de tierra*, y una subclase común a ambas llamada *Vehículo anfibio*. El diagrama se muestra en la figura 4.113.



Figura 4.113 Diagrama de clases de herencia múltiple para *Vehículo*, que tiene la clase *Vehículo anfibio*, la cual hereda a la vez de *Vehículo de agua* y *Vehículo de tierra*.

En herencia sencilla no existen conflictos entre superclases. Cuando se hereda la misma operación de múltiples ancestros, como opción se debe sobrescribir la operación en la nueva clase para evitar conflictos. También se debe verificar que no se herede más de una vez aspectos de una clase (superclase común a varias jerarquías).

La ventaja de incorporar herencia múltiple es que permite integrar información de dos o más superclases a la vez, dando más poder en la especificación de clases y más oportunidad de reutilización, siendo por lo general más natural para el modelo de objetos que la herencia sencilla.

Por otro lado, la desventaja de incorporar herencia múltiple es que es más complicada que herencia sencilla, ya que se pierde la simplicidad conceptual y de implementación. El mayor problema resultante es que se podría estar heredando varias veces una misma característica de diferentes clases ancestrales. En general, se pueden definir reglas para resolver ambigüedades y evitar conflictos entre las características heredadas por varios recorridos en la jerarquía de herencia.

Ejemplo: Un *Vehículo anfibio* estaría heredando características comunes a todos los *Vehículos*, como *Velocidad*, de forma repetida, ya que heredaría tales características a través de *Vehículo de agua* y *Vehículo de tierra*.

Las **clases disjuntas** son clases que semánticamente describen características diferentes, de distinta jerarquía de herencia (discriminadores diversos).

Las **clases no disjuntas** son clases que tienen aspectos comunes (mismo discriminador), por lo cual una clase que herede características de ambas estaría compartiendo tales propiedades. Se puede incorporar herencia múltiple de una misma generalización, si las clases dentro de la generalización son *no disjuntas*. (No se debe heredar de dos clases perteneciendo a una misma generalización, si las clases dentro de la generalización son *disjuntas*.)

Ejemplo: La clase *Vehículo anfibio* está heredando de una sola jerarquía de generalización, *Tipo de vehículo*, por lo cual *Vehículo de tierra* y *Vehículo de agua* deben ser no disjuntos para que la herencia múltiple sea correcta, o sea, que exista la posibilidad de tener un vehículo que funcione en agua y tierra a la vez. Por otro lado, una clase *Empleado honorario asalariado* estaría heredando a la vez de las clases *Empleado honorario* y *Empleado asalariado*, siendo esto incorrecto, ya que éstas son clases disjuntas dentro de una misma jerarquía de generalización, o sea que es incorrecto que un empleado se le considere que gana por honorarios y también es asalariado. La decisión de si dos clases en una misma generalización son disjuntas o no disjuntas depende de su interpretación.

Se puede incorporar herencia múltiple de dos generalizaciones distintas, donde las clases son *disjuntas*. Cada generalización debe cubrir una sola propiedad, por lo cual, una nueva clase creada por medio de la herencia múltiple estaría siendo refinada sobre dimensiones de generalización diferentes e independientes.

Ejemplo: De la generalización de *Empleado* se pueden crear dos jerarquías diferentes de herencia, una jerarquía se define según el *Tipo de pago* del *Empleado*, dando lugar a *Empleado honorario* y *Empleado asalariado*. Otra jerarquía se define según el *Tipo de prestación*, dando lugar a *Empleado con prestación* y *Empleado sin prestación*. Se puede definir una nueva clase *Empleado honorario con prestación*, el cual hereda características de dos clases disjuntas, *Empleado honorario* y *Empleado con prestación*, de dos jerarquías de generalización independientes, y la herencia múltiple es correcta. En la figura 4.114, se muestra la clase *Empleado honorario con prestación* que hereda de dos generalizaciones diferentes, *Tipo de pago* y *Tipo de prestación*. Por tanto, *Empleado honorario* y *Empleado con prestación* definen clases disjuntas de dos jerarquías de generalización independientes, y la herencia múltiple de *Empleado honorario con prestación* es correcta.



Figura 4.114 Diagrama de clases de herencia múltiple para *Empleado honorario con prestación*, que hereda a la vez de *Empleado honorario* y *Empleado con prestación*.

4.8 Módulos

Un **módulo** o **paquete** (*package*) es una construcción lógica para agrupar clases, asociaciones y generalizaciones. El módulo captura diferentes perspectivas de un sistema. Los bordes entre los distintos módulos pueden ser bastante arbitrarios. Un modelo de objetos consiste de uno o más módulos. Los nombres de clases y asociaciones deben ser únicos en cada módulo, y se debe mantener consistencia entre los nombres de varios módulos. La misma clase puede aparecer en diferentes módulos, aunque debe ser definida solamente en uno de los módulos y referida en los otros. Debe haber menos conexiones entre módulos, que asociaciones dentro de los módulos. En sistemas grandes la jerarquía de módulos puede ser de múltiples niveles.

Cada módulo debe proveer una visión de alto nivel de las clases más importantes del sistema, mostrando las clases y sus asociaciones sin atributos u operaciones. Cada una de estas clases se asigna a su propio módulo, mostrando su descomposición en clases por generalización y agregación. En la figura 4.115, se muestra la notación para un módulo o paquete en UML. Note que el módulo no tiene ninguna propiedad, a diferencia de la clase. Sirve únicamente como elemento organizacional de las clases.



Figura 4.115 Notación para un módulo en UML.

Por ejemplo, arquitecturas en forma de "estrella" son bastante comunes, donde la estructura de alto nivel está en un módulo, y otros módulos expanden las superclases con generalización y añaden asociaciones a clases de bajo nivel.

Para integrar éstos y los conceptos anteriores que se han descrito a lo largo del capítulo, presentamos en esta sección un ejemplo más completo de un *Automóvil*, cuyo diagrama se ilustra en la figura 4.116.



Figura 4.116 Diagrama para un módulo de un *Automóvil*.

Los módulos en los cuales está subdividido el módulo principal del *Automóvil* son los siguientes: *Carrocería*, *Sistema de frenos*, *Sistema de alimentación*, *Sistema eléctrico* y *Sistema mecánico*, como se muestra en la figura 4.117.

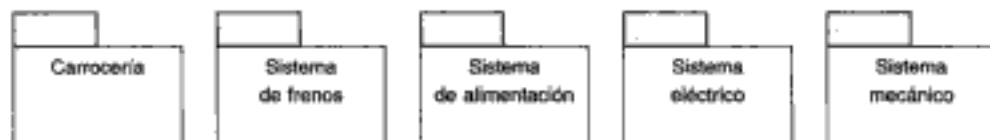


Figura 4.117 Módulos que componen el módulo principal del *Automóvil*: *Carrocería*, *Sistema de frenos*, *Sistema de alimentación*, *Sistema eléctrico* y *Sistema mecánico*.

Estos módulos son descritos con mayor detalle a continuación y cada uno por separado. En el capítulo 6, veremos cómo se puede aplicar el concepto del módulo para organizar sistemas de mayor complejidad. Vale la pena señalar que esta descripción de un *Automóvil* corresponde al "dominio" de una aplicación, en el caso del automóvil. En el diagrama de la figura 4.118 se muestran los módulos descritos como clases. Esta descripción puede coexistir con la de los módulos anteriores si así se desea, donde cada una de estas clases se asigna al módulo con su mismo nombre.



Figura 4.118 Clases para el módulo de un *Automóvil*: *Carrocería*, *Sistema de frenos*, *Sistema de alimentación*, *Sistema eléctrico* y *Sistema mecánico*.

El módulo de la *Carrocería* incluye el *Chasis*, la *Cajuela*, el *Cofre*, las *Puertas* y las *Ventanas*, las cuales pueden ser partes de las *Puertas*, como se muestra en la figura 4.119.

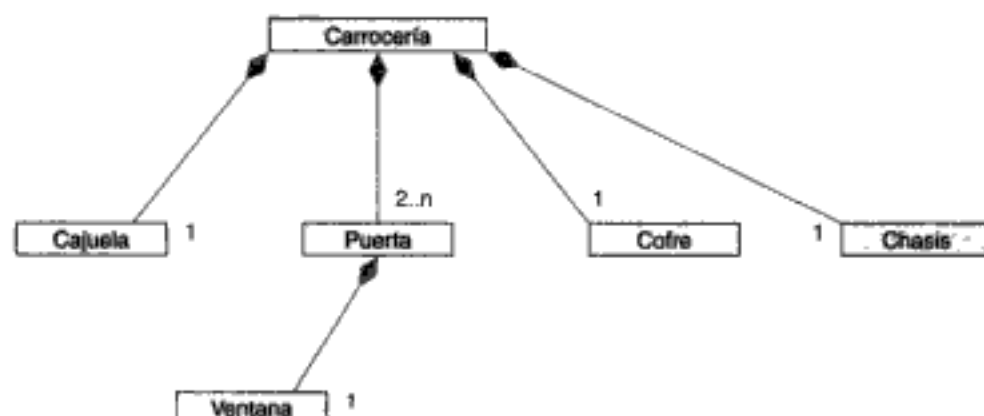


Figura 4.119 Módulo de la *Carrocería*, el cual está compuesto por el *Chasis*, la *Cajuela*, el *Cofre*, las *Puertas* y las *Ventanas*.

El módulo del *Sistema de freno* incluye el *Freno de mano*, el *Pedal de freno*, los *Frenos de tambor* y de *Disco* y las *Balatas*, como se muestra en la figura 4.120.



Figura 4.120 Módulo del *Sistema de freno*, que incluye el *Freno de mano*, el *Pedal de freno*, los *Frenos de tambor* y de *Disco* y las *Balatas*.

El módulo de alimentación incluye el *tanque de gasolina*, la *bomba de gasolina*, el *filtro de aire* y de *gasolina* y el *carburador* o la *inyección*, como lo ilustra la figura 4.121.



Figura 4.121 Módulo de Alimentación, que incluye el Tanque de gasolina, la Bomba de gasolina, el Mezclador (que contiene el Filtro de aire y de Gasolina), y que se especializa en el Carburador o la Inyección.

El módulo eléctrico incluye el *distribuidor*, la *bobina*, la *batería*, el *interruptor de arranque*, el *regulador de voltaje*, el *alternador* y las *bujías*, como lo muestra la figura 4.122.

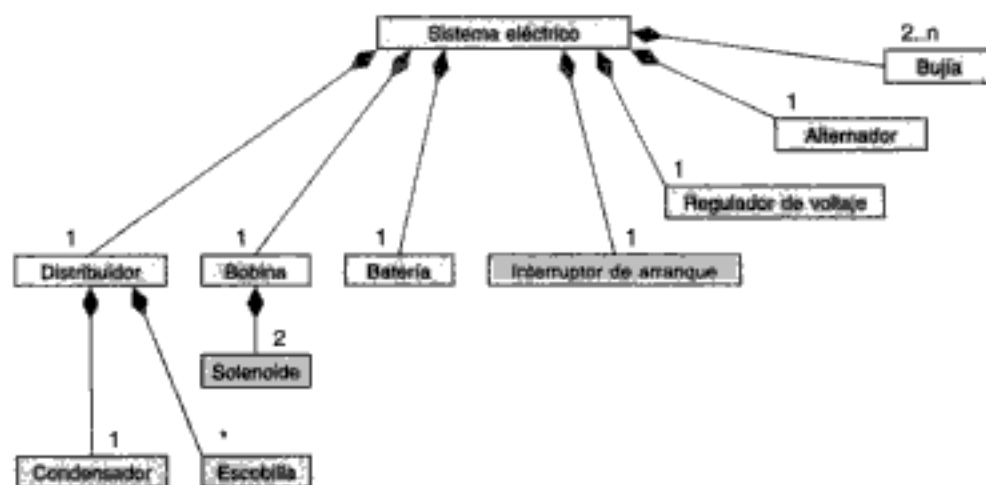


Figura 4.122 Módulo Eléctrico, que incluye el Distribuidor, la Bobina compuesta por dos Solenoides, la Batería, el Interruptor de arranque, el Regulador de voltaje, el Alternador y las Bujías.

El módulo *Sistema mecánico* incluye los módulos de *Sistema de transmisión*, *Sistema de lubricación*, *Sistema motriz*, *Sistema de enfriamiento*, *Sistema de suspensión* y *Sistema de dirección*, como se ilustra en la figura 4.123.

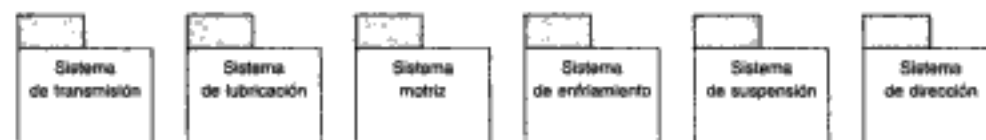


Figura 4.123 Módulo Mecánico que incluye los módulos de *Sistema de transmisión*, *Sistema de lubricación*, *Sistema motriz*, *Sistema de enfriamiento*, *Sistema de suspensión* y *Sistema de dirección*.

Estos módulos de *Sistema mecánico* también se pueden describir como clases: *Sistema de transmisión*, *Sistema de lubricación*, *Sistema motriz*, *Sistema de enfriamiento*, *Sistema de suspensión* y *Sistema de dirección*, como se muestra en la figura 4.124.



Figura 4.124 El módulo *Sistema mecánico*, que incluye las clases de *Sistema de transmisión*, *Sistema de lubricación*, *Sistema motriz*, *Sistema de enfriamiento*, *Sistema de suspensión* y *Sistema de dirección*.

El módulo de *Transmisión* incluye la *Palanca de velocidades*, la *Caja de velocidades*, el *Overdrive*, la *Transmisión manual* o *Automática* y el *Embrague*, como se muestra en la figura 4.125.

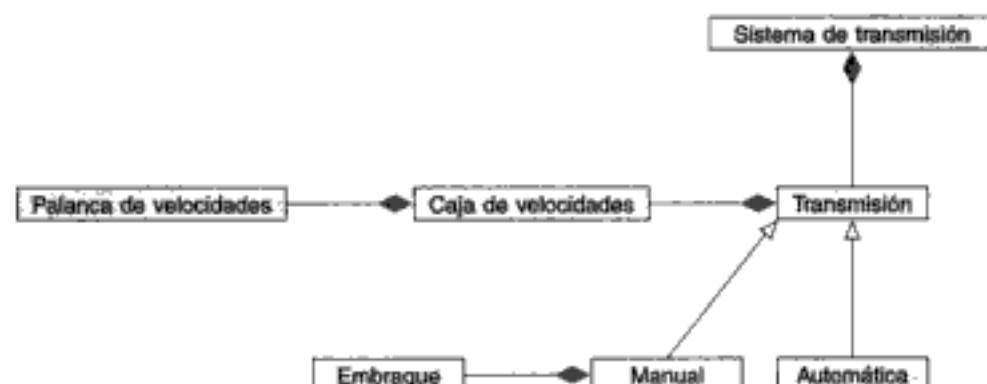


Figura 4.125 Módulo de *Transmisión*, que incluye la *Palanca de velocidades*, la *Caja de velocidades*, el *Overdrive*, la *Transmisión manual* o *Automática* y el *Embrague*.

El módulo de *Lubricación* incluye la *Bomba de aceite* y el *Cárter*, como se observa en la figura 4.126.

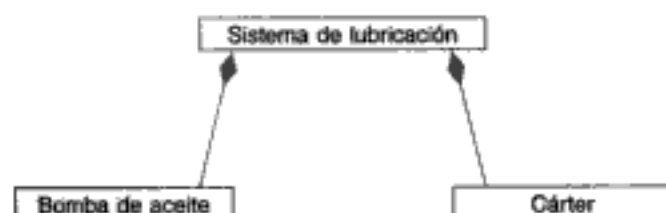


Figura 4.126 Módulo de *Lubricación*, que incluye la *Bomba de aceite* y el *Cárter*.

El módulo *Motriz* incluye el *Cigüeñal*, el *Árbol de levas*, los *Cilindros* y los *Pistones*, como lo muestra la figura 4.127.



Figura 4.127 Módulo *Motriz*, que incluye el *Cigüeñal*, el *Árbol de levas*, los *Cilindros* y los *Pistones*.

El módulo de *Enfriamiento* incluye el *Radiador*, el *Termostato*, el *Ventilador* y la *Bomba de agua*, como se muestra en la figura 4.128.

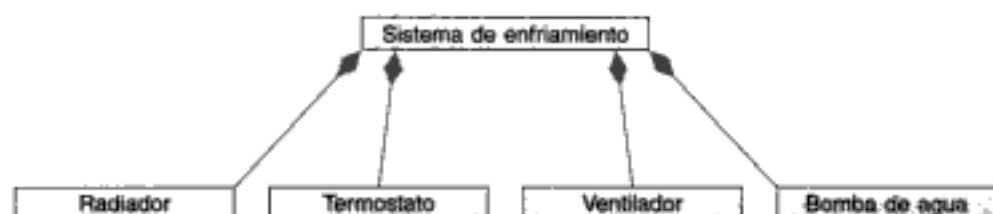


Figura 4.128 Módulo de *Enfriamiento*, que incluye el *Radiador*, el *Termostato*, el *Ventilador* y la *Bomba de agua*.

El módulo de *Suspensión* incluye los *Amortiguadores*, la *Barra estabilizadora* y los *Resortes*, como lo ilustra la figura 4.129.



Figura 4.129 Módulo de *Suspensión*, que incluye los *Amortiguadores*, la *Barra estabilizadora* y los *Resortes*.

El módulo de *Dirección* incluye la *Caja de dirección*, el *Eje* y el *Volante*, como se muestra en la figura 4.130.



Figura 4.130 Módulo de *Dirección*, que incluye la *Caja de dirección*, el *Eje* y el *Volante*.

Ya que el diagrama de un sistema complejo puede ser muy grande, los módulos se deben diagramar en una o más páginas. Por lo general, no se debe incluir más de un módulo por página, siendo el uso de páginas una conveniencia de notación y no un aspecto de lógica. Las asociaciones y generalizaciones deben aparecer en una sola página, mientras que ciertas clases, pueden aparecer en múltiples páginas para servir de vínculo entre ellas. Se busca minimizar el número de estas clases puente.

RESUMEN

En este capítulo se introduce al lector en el tema del modelado orientado a objetos con UML. Se describe la notación para objetos, clases, atributos, operaciones, asociaciones y agregados, así como composición, generalización y herencia y módulos. A lo largo del capítulo se proporcionan múltiples ejemplos.

REFERENCIAS

1. "http://www.uml.org. Booch, G., Rumbaugh, J., Jacobson, L., 1998. The Unified Modeling Language User Guide, Addison-Wesley."

Programación orientada a objetos con Java

En este capítulo haremos una breve introducción al lenguaje de Java¹, mostrando la relación entre el modelado en UML² y la programación en Java, y se describirán algunos aspectos relevantes para el desarrollo de los capítulos posteriores del libro.

5.1 Introducción a Java

Para apreciar el gran movimiento que hay detrás de Java, es necesario comprender que es mucho más que un lenguaje, más bien es un sistema de gran alcance. En cierta manera, desató un fenómeno parecido al de Smalltalk hace 20 años, gracias al sistema que tenía alrededor de su lenguaje. Por tanto, en esta sección analizaremos las características principales del lenguaje de Java, para después seguir con otros aspectos significativos.

5.1.1 Características

El lenguaje de Java tiene características que lo han hecho un lenguaje esencial para la programación de sistemas de cómputo, que consta de los siguientes puntos:

- **Orientado a objetos.** Ante todo Java es un lenguaje orientado a objetos, lo cual lo pone en la misma categoría que lenguajes como C++ y Smalltalk. Como parte de esta característica, se cuenta con un **ligado dinámico** (*dynamic linkage*) de clases en tiempo de ejecución, herencia y polimorfismo; además de aspectos de metanivel similares a los de Smalltalk.
- **Portátil.** Un aspecto que ha hecho de Java un lenguaje muy utilizado es su portabilidad. A diferencia de lenguajes como C y C++, que varían en su detalle dependiendo de la máquina en que se ejecuten, Java es exactamen-

te igual en cualquier plataforma. Por ejemplo, a diferencia de C y C++, el tamaño de los tipos de datos en Java es fijo, independiente de la máquina. La importancia de este aspecto es que si se compila el programa en una plataforma particular, el sistema correrá en cualquier máquina, reduciendo mucho el costo de desarrollo (tiempo y dinero). Para ello, está el concepto de la máquina virtual de Java (*Java Virtual Machine*, JVM), que debe existir en cada plataforma donde se ejecute un programa de Java.

- ▶ **Abierto.** Este aspecto de portabilidad ocurre gracias a su diseño abierto, que permite a cualquier compañía, e incluso desarrollador, tomar el código fuente, para adaptarlo a una nueva plataforma donde aún no se ha probado. Ninguno de los demás lenguajes ofrecen esta característica. Otra razón de la gran popularidad de Java.
- ▶ **Gratis.** Muy de la mano con el aspecto "abierto" está que el lenguaje se ofrece gratis, aunque bajo licencia, a cualquier usuario. Esto reduce el costo de la aplicación y fortalece la decisión de utilizarlo en distintas plataformas, donde no se incurre en el costo de pagar gran número de licencias, como es obligatorio en la mayoría de los demás productos.
- ▶ **Integrado a la web.** Éste es uno de los aspectos que ha impulsado la gran difusión de Java, en una época donde la Internet ha sido de crucial importancia. Java es el único lenguaje, con excepción de algunos lenguajes *scripts*, que viene integrado con los navegadores (*browsers*) más utilizados en la Web.
- ▶ **Simple.** Otro aspecto es su similitud con C y C++, en relación con las expresiones básicas del lenguaje. Esto ha permitido a los programadores aprender Java de manera más rápida, a diferencia de lenguajes como Smalltalk que requieren un cambio en la manera de pensar de los programadores ya acostumbrados a C y C++. Sin embargo, Java se considera más *puro* que C++, ya que no contiene más que clases, lo que simplifica el programa y al propio compilador. Java disminuye la complejidad de C++, como es la aritmética de apuntadores, que a su vez agrega complejidad a la administración de memoria. Se elimina la complejidad adicional de tipos como estructuras y el uso de asociaciones de tipo, a través de *typedefs*, junto con el preprocesador de C++ con palabras reservadas como `#define`, `#include` y `#ifdef`. Otro aspecto que se elimina es la sobreescritura de operadores. También se eliminan aspectos de manejo complicado como la herencia múltiple.
- ▶ **Robusto.** En contraste con C++ y, en especial, con C, Java está *fuertemente tipificado*, lo que ayuda a encontrar con mayor facilidad los errores de programación durante la etapa de compilación. Java también incluye manejo de excepciones y recolección de basura, con objeto de lograr programas más robustos.
- ▶ **Seguro.** Debido a la eliminación de los apuntadores de C y C++, Java logra un modelo de manejo de memoria mucho más seguro, que además se apoya en el modelo de verificación de código en tiempo de ejecución, como veremos más adelante en la descripción del modelo completo de Java.
- ▶ **Eficiencia.** En la actualidad, Java está considerado como un lenguaje eficiente. Aunque nunca llegue a la eficiencia de C, en este aspecto se le compara con C++. Esta eficiencia se basa en que cuenta con un compilador para generar el código en contraste con aquellos lenguajes completamente interpretados, donde el rendimiento es menor. Ahora Java cuenta con un compilador incremental (*Just-in-Time Compiler*, JIT), que ayuda a lograr estos objetivos.

- **Bibliotecas.** Otro aspecto que ha hecho de Java un lenguaje muy aceptado es la riqueza de sus bibliotecas o *paquetes* (*package*). Esto está en contraste radical con C y C++, donde las bibliotecas realmente no existen. En cambio Java contiene un gran número de bibliotecas que facilitan la creación de programas, además de asegurar una estandarización entre aplicaciones. Existen bibliotecas para el manejo de estructuras de datos avanzadas, manejo de multimedia, manejo de redes como TCP/IP, procedimientos remotos y concurrencia mediante múltiples *hilos de procesamiento* (*multiple threads*), estos últimos también conocidos como *procesos finos o livianos*. En la actualidad, aprender el lenguaje de Java como tal es sólo 10% del esfuerzo, 90% restante se enfoca a aprender a utilizar sus bibliotecas. Obviamente se estudian sólo aquellas que se desea conocer. Por ejemplo, una biblioteca importante es la del sistema de ventanas que puede correr bajo cualquier plataforma. Existe el Abstract Window Toolkit (AWT) desde la primera versión de Java, y se cuenta en la actualidad con las bibliotecas Java Foundation Classes (JFC), también conocidas como *SWING*. Además de éstas existen bibliotecas de manejo de gráficas en dos y tres dimensiones. Incluso existen versiones para correr en plataformas móviles, como asistentes personales.
- **Tecnología.** Existe un gran número de productos y tecnología desarrollados alrededor de Java. Aparte de este lenguaje se cuenta con productos tales como Enterprise JavaBeans (EJB), Java Server Pages (JSP), Java Servlets y Java Data Base Connectors (JDBC). Además, existen productos relacionados con estándares tales como Common Object Request Broker Architecture (CORBA) y eXtended Markup Language (XML). En la actualidad hay tres ediciones principales Java: Java2 Enterprise Edition (J2EE), Java2 Standard Edition (J2SE) y Java2 Micro Edition (J2ME).

5.1.2 Procesamiento

La figura 5.1 ilustra el procesamiento de un programa escrito en Java. Del lado izquierdo se muestran los pasos para la compilación de un programa en Java, mientras que del derecho están los pasos para su ejecución.

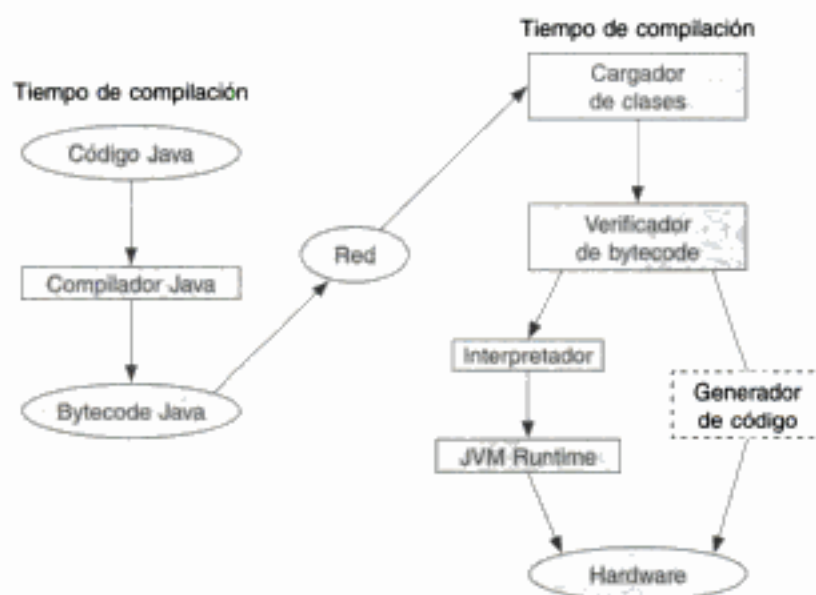


Figura 5.1 Procesamiento de un programa escrito en Java.

COMPILACIÓN

Se escribe un programa en código Java utilizando el sufijo ".java", el cual se compila mediante cualquiera de los compiladores de Java en alguna de las distintas plataformas. En general, debe haber un archivo ".java" por cada clase que exista en el programa, donde el archivo tendrá el mismo nombre que la clase contenida. El compilador genera el código final, conocido como **bytecode**, a ser interpretado por la máquina virtual de Java. El programa generado tiene como extensión el sufijo ".class". Se origina un archivo ".class" por cada clase que se tenga en la aplicación.

Por ejemplo, si se tiene una clase llamada "ej", el nombre del archivo debe ser "ej.java". El archivo se compilaría mediante algún ambiente de desarrollo o utilizando el comando `javac` que viene incluido en los kit de desarrollo de Java como Java Development Kit (JDK) o Standard Development Kit (SDK). Por ejemplo, para compilar el archivo anterior se ejecutaría

```
javac ej.java
```

Esta compilación resultaría en el archivo "ej.class".

EJECUCIÓN

Durante la ejecución se obtiene el *bytecode*, guardado en los archivos ".class", que puede estar ya en la plataforma actual o haber sido enviado por la red, como en el caso de un *browser*. El *bytecode* se carga en la máquina virtual por el cargador de clases. A continuación este código es procesado por el verificador de *bytecode* y, dependiendo del hardware con que se cuenta, puede ser interpretado y ejecutado por el procesador virtual de la máquina o traducido al código de un procesador de Java mediante el generador de código.

- Existen dos maneras de ejecutar (y estructurar) un programa dependiendo de su ambiente de ejecución. En el caso de una aplicación "normal" (*standalone*), se ejecuta mediante el siguiente interpretador de Java, llamado simplemente `java`:

```
java ej2
```

- En el caso de una aplicación que se ejecuta desde un *navegador web* (*web browser*), llamado **applet**, el contenido de los archivos `.class` que están almacenados en el servidor, se transmiten a través de la red y se ejecutan en la máquina cliente (que puede ser la misma máquina que el servidor). Dado que un *browser* sólo comprende archivo `.html`, el *applet* debe ser relacionado con un archivo llamado, por ejemplo `ej.html`. Este archivo debe contener la siguiente línea:

```
<applet code=ej.class width=200 height=200></applet>
```

Ya que pueden haber múltiples archivos `.class`, sólo el principal es el que se incluye en la línea anterior. Otra forma adicional de ejecutar el *applet* es mediante el comando `appletviewer`, de la siguiente forma:

```
appletviewer ej.html
```

A lo largo del capítulo iremos describiendo con mayor detalle el desarrollo de programas en Java junto con ejemplos.

5.1.3 Bibliotecas

Java lleva a un nuevo nivel el concepto de **bibliotecas** o **paquetes**, éstos proveen una amplia funcionalidad para crear nuevas aplicaciones de Java. Además de servir como bibliotecas, definen una *Application Program Interface* (API) [interface de aplicación de programa], que permite al desarrollador extender las clases de estos paquetes para adaptarlos a las necesidades básicas de un programa. Java organiza estos paquetes en componentes jerárquicos a partir de dos directorios principales. El primero es `java`, que es parte esencial de lo que actualmente se conoce como el API 1 de Java. Los paquetes de este API se muestran en la tabla 5.1.

Tabla 5.1 Paquetes básicos de Java.

Paquete	Contenido
<code>Java.applet</code>	Clases para implementar <i>applets</i> , correspondientes a aplicaciones que corren en los browsers.
<code>Java.awt</code>	Clases para gráficas, componentes Graphic User Interface (GUI) y administradores de control de ventanas, además de clases más especializadas como para procesamiento de imágenes Abstract Window Toolkit (AWT).
<code>java.beans</code>	Clases e interfaces para construir <i>JavaBeans</i> , correspondientes a GUI independientes de plataformas.
<code>java.io</code>	Clases para control de entradas y salidas, tales como archivos y streams.
<code>java.lang</code>	Clases que componen el núcleo del lenguaje.
<code>java.math</code>	Clases para aritmética avanzada, incluyendo manejo de precisión numérica arbitraria.
<code>java.net</code>	Clases relacionadas con el manejo de redes, tales como datagramas y <i>sockets</i> .
<code>java.rmi</code>	Clases para el manejo de métodos remotos.
<code>java.security</code>	Clases para aspectos de seguridad, tales como criptografía.
<code>java.sql</code>	Clases para acceso a base de datos con el lenguaje Standard Query Language (SQL). ¹
<code>java.text</code>	Clases para internacionalización del idioma, independiente del lenguaje particular.
<code>java.util</code>	Clases adicionales, tales como estructuras de datos avanzadas y compresión de datos.

En la actualidad se cuenta con el API 2 de Java, mejor conocido como Java2, el cual incluye además del paquete `java`, el paquete `javax`, donde se encuentran componentes más avanzados, como se muestra en la tabla 5.2.

En Java, cada clase debe ser parte de un **paquete** (*package*), y puede ser referida por su nombre completo “calificado”, el cual consiste en la jerarquía del paquete y el nombre de la clase, todos separados por puntos. Los propios nombres de los paquetes generalmente están compuestos de múltiples componen-

Tabla 5.2 Paquetes extendidos de Java.

Paquete	Contenido
<code>javax.accessibility</code>	Clases que definen contratos entre componentes de interfaces de usuario y una tecnología asistente que provee acceso a esos componentes.
<code>javax.activation</code>	Clases que definen activación de los componentes de <i>JavaBeans</i> .
<code>javax.ejb</code>	Clases para el manejo de <i>Enterprise Java Beans (EJB)</i> .
<code>javax.jms</code>	Clases para el manejo de <i>Java Message Server (JMS)</i> .
<code>javax.mail</code>	Clases para el manejo de correo.
<code>javax.naming</code>	Clases para el acceso de los servicios de nombres.
<code>javax.rmi</code>	Clases para la invocación de métodos remotos incluyendo CORBA.
<code>javax.servlet</code>	Clases para el manejo de <i>servi</i> ets y <i>Java Server Pages (JSP)</i> .
<code>javax.sql</code>	Clases para el acceso a base de datos con <i>SQL</i> .
<code>javax.swing</code>	Clases que proveen un conjunto de componentes para GUI que trabajan en cualquier plataforma.
<code>javax.transaction</code>	Clases para el manejo de transacciones entre componentes.

tes separados por puntos. Por ejemplo, la clase `PixelGrabber` que se encuentra en el paquete `java.awt.image` se ingresaría mediante:

```
java.awt.image.PixelGrabber
```

Vale la pena notar que los paquetes se guardan en distintos directorios, donde el "." realmente corresponde a "/" ("\" en la PC), donde se traduce, por ejemplo `java.awt.image` a `java/awt/image`. Por tanto, la clase `PixelGrabber` estaría guardada dentro del directorio anterior.

Además de los paquetes mencionados en las tablas 5.1 y 5.2, existe un número muy extenso de productos adicionales desarrollados por Sun y otras compañías, como los paquetes para gráficas en dos y tres dimensiones que son también parte de Java, y los paquetes para acceso a bases de datos de Oracle y Sybase.

5.2 Programación básica

En las siguientes secciones se describen algunos de los conceptos básicos de la programación en Java.

5.2.1 Aspectos generales

COMENTARIOS

El primer aspecto que debe conocerse en cualquier lenguaje es cómo distinguir entre código y comentarios. En Java existen tres tipos distintos para la especificación de comentarios, como se muestra en la tabla 5.3.

Tabla 5.3 Notación para comentarios.

Notación	Contenido
// línea comentada	Las dos diagonales indican el comienzo de un comentario que tiene efecto hasta el final de la línea.
/* párrafo comentado */	La diagonal seguida por un asterisco indica el inicio de un párrafo comentado. Para finalizar el comentario debe añadirse un asterisco seguido de otra diagonal. Estos comentarios no pueden anidarse uno dentro del otro.
/** párrafo comentado documentable */	La diagonal seguida por dos asteriscos indica el inicio de un párrafo comentado. Para terminar el comentario debe añadirse un asterisco seguido de otra diagonal. A diferencia del comentario anterior, éste es un comentario documentable que puede ser extraído mediante el comando javadoc, para producir un documento sencillo a partir del código fuente de Java. Estos comentarios no pueden anidarse uno dentro del otro.

CARACTERES

La especificación de **caracteres** en Java es distinta de la mayoría de los demás lenguajes. Java utiliza 16 bits en lugar de los más comunes 8 bits correspondientes al código ASCII para especificar caracteres. Este código de 16 bits es conocido en Java como *Unicode*, el cual mantiene compatibilidad con ASCII.

Existen actualmente 34 000 caracteres definidos en Unicode, pero no todos pueden desplegarse en cualquier plataforma, por lo cual se utilizan secuencias especiales de *escape* con el siguiente formato: "\uxxxx", donde xxxx representa una secuencia de uno a cuatro dígitos hexadecimales. Por ejemplo, el carácter nulo es "\u0000". Además de este formato de especificación, también se apoyan las secuencias especiales de escape, como en C, que son "\n" y "\t", y de manera más general "\xxx", donde xxx representa un dígito octal.

PALABRAS RESERVADAS

Todo lenguaje de programación cuenta con un número de **palabras reservadas** que tienen un significado especial y predefinido. En Java son 48 las palabras reservadas, las cuales el usuario no puede utilizar para sus propios usos, tales como *if*, *else*, etcétera.

IDENTIFICADORES

Dentro de las palabras que el usuario puede definir se encuentran los **identificadores**, que sirven para relacionarse con estructuras del programa. Por ejemplo, para definir una clase o introducir un objeto, se requiere definir un identificador. Los identificadores que guardan o se refieren a valores de algún tipo son también conocidos como *variables*. Los nombres de los identificadores son cualquier palabra, con excepción de las reservadas por Java, que inician con alguna letra del alfabeto o que comienzan con los siguientes símbolos: \$ o _. Estos identificadores pueden ser de cualquier tamaño. Por ejemplo, identificadores aceptables son: *ID*, *nombre*, *_temp*, *\$dolar*, entre otros.

ORACIONES

Toda **oración** en Java, correspondiente a una línea de ejecución, tiene como carácter final el ";". Una notación relaciona el *bloque* de llaves que utiliza "{" y "}" para especificar el inicio y final de un grupo de oraciones.

PAQUETES

Como mencionamos antes, Java organiza sus bibliotecas alrededor del concepto de **paquetes** (*packages*). Dado el amplio número de paquetes que existen en Java, es necesario tener un manejo modular de manera que las clases dentro de un paquete no tengan conflictos con clases en otros paquetes, inclusive con las que pudieran tener el mismo nombre. Por tal motivo, Java utiliza la expresión **package**, que debe aparecer como primera instrucción en un archivo de código fuente en Java. Por ejemplo, un paquete con el nombre *ej* se incluiría como primera instrucción de todos los archivos que contengan clases de este paquete:

```
package ej.
```

De tal manera se especifica de qué paquete es componente la clase (y el archivo correspondiente). Las clases que son parte de un paquete particular tienen acceso a todas las demás clases del mismo paquete, lo que veremos con mayor detalle más adelante. Cuando un archivo es parte de un paquete, la clase compilada debe ubicarse de manera apropiada dentro de la jerarquía de directorios para ser ingresada de manera correcta, como se mencionó anteriormente.

IMPORTACIÓN

Una instrucción relacionada con el concepto de paquetes es **import**, la cual permite utilizar clases definidas en otros paquetes bajo nombres abreviados. Aunque siempre es posible utilizar otras clases por medio de sus nombres calificados completos, **import**, que no lee la clase ni la incluye, permite ahorrar escritura haciendo al código más legible. En otras palabras, sin la expresión de **import**, se puede utilizar la clase `PixelGrabber` siempre y cuando al emplearla se le llame por su nombre calificado completo `java.awt.image.PixelGrabber`.

Existen tres formatos para **import**:

- **import package;** posibilita que el paquete especificado sea conocido por el nombre de su último componente. Por ejemplo, la siguiente expresión

```
import java.awt.image;
```

hace que la clase `java.awt.image.PixelGrabber` se le llame de la siguiente manera

```
image.PixelGrabber
```

- **import package.class;** facilita que la clase especificada en el paquete sea conocida por su nombre de clase directamente. Por lo tanto, la expresión

```
import java.awt.image.PixelGrabber;
```

permite que la clase `java.awt.image.PixelGrabber` se le llame de la siguiente manera

`PixelGrabber`

- `import package.*;` hace que todas las clases en un paquete sean accesibles por medio del nombre de su clase directamente. Por ejemplo, la expresión

```
import java.awt.image.*;
```

permite que la clase `java.awt.image.PixelGrabber` y cualquiera otra dentro de ese mismo paquete, se le llame mediante su nombre directo, como

`PixelGrabber`

Se pueden incluir cualquier número de expresiones `import`, aunque todas deben aparecer después de la expresión inicial de `package` y antes de cualquier definición de clases o código en general. Si dos paquetes importados mediante esta forma contienen clases con el mismo nombre, es un error usar sus nombres ambiguos en vez de usar el nombre calificado completo.

5.2.2 Estructuras básicas

TIPOS PRIMITIVOS

En Java existen un número de tipos primitivos o predefinidos como parte del lenguaje. Estos tipos se muestran en la tabla 5.4.

Tabla 5.4 Tipos primitivos predefinidos en Java.

Tipo	Descripción	Valor omisión
byte	Es una estructura de 8 bits.	0
char	Es una estructura con base en unicode de 16 bits (sin signo).	\u0000
short	Es una estructura numérica de tipo entero de 16 bits.	0
int	Es una estructura numérica de tipo entero de 32 bits.	0
long	Es una estructura numérica de tipo entero de 64 bits.	0
float	Es una estructura numérica de tipo real de 32 bits.	0.0
double	Es una estructura numérica de tipo real de 64 bits.	0.0
boolean	Es una estructura de 1 bits, con valores true o false.	false

Los **tipos primitivos** son estructuras que guardan un valor primitivo y que se manipulan a través de variables, declaradas de manera correspondiente, como veremos más adelante. Los valores asignados por omisión a variables de estos tipos se muestran en la última columna. Los nombres correspondientes a tipos primitivos comienzan con una letra minúscula. Existe un "sin-tipo", llamado `void`.

Los primeros cinco tipos en la tabla `byte`, `char`, `short`, `int` y `long`, son conocidos como tipos integrales.

TIPOS NO-PRIMITIVOS

Los **tipos no-primitivos** corresponden a clases que son instanciadas en objetos. Los objetos se accesan mediante referencias almacenadas en variables. A diferencia de las variables de tipos primitivos que guardan un "valor", las variables correspondientes a tipos no-primitivos guardan una "referencia" al objeto instanciado. En Java no existe un número predeterminado de tipos no-primitivos, aunque sí hay algunos predefinidos a través de las bibliotecas del lenguaje. Existe una clase muy particular que vale la pena mencionar, que es `Object`, y tiene como particularidad servir como la *superclase* de todas las demás clases del sistema, clases definidas en una biblioteca de Java o bien directamente por el programador. Por ejemplo, una de estas clases ya definidas en Java es `String`, la cual es esencial en el manejo de cadenas.

VARIABLES

Para utilizar uno de estos tipos de datos primitivos, primero es necesario definir alguna variable que guarde el valor del tipo correspondiente. Las **variables** se representan por medio de un nombre seleccionado dentro de los posibles identificadores. Por ejemplo, dos nombres de variables válidos serían `x` y `y`. Después de definir el nombre de la variable se prosigue a *declararla* de acuerdo con algún tipo de dato. Como nota importante, todas las variables existen dentro de las clases y no pueden estar "sueltas" en un archivo como ocurre en C++.

DECLARACIONES

Una **declaración** consiste en relacionar una variable con el tipo de dato que va a guardar. Por ejemplo si consideramos las dos variables `x` y `y`, donde se desea que cada una guarde un valor entero, tendríamos que hacer la siguiente declaración:

```
int x,y;
```

Estas variables guardan inicialmente el valor de 0, hasta que se les haga una asignación con un nuevo valor, lo cual veremos más adelante.

Por ejemplo, una variable *valor* de tipo `boolean` se declararía de la siguiente forma:

```
boolean valor;
```

Las variables que hacen referencias a objetos de alguna clase se declaran de la misma forma. Si `obj` representa una variable de tipo `ClassX`, su declaración será la siguiente:

```
ClassX obj;
```

A diferencia de las variables de tipos primitivos que guardan valores, las variables de clases guardan únicamente referencias a los objetos instanciados de ellas,

debido a que los objetos son estructuras más complejas que los tipos primitivos, por lo cual éstos se guardan en la memoria y las variables se refieren a esta ubicación. De tal manera, una variable de clase guarda por omisión una referencia vacía o nula que corresponde a `null` en Java. Vale la pena resaltar que esta referencia nula no equivale al 0 como ocurre en C. En particular, `null` es una palabra reservada que significa ausencia de referencia.

CONSTANTES

Como parte de las declaraciones de variables, Java permite agregar distintos tipos de modificadores, que afectan diversos aspectos de las variables. Existen por ejemplo modificadores para la visibilidad, correspondiente al manejo del encapsulamiento, que serán mostrados más adelante cuando se describan objetos y clases. Sin embargo, vale la pena mencionar un modificador particular que hace que una variable se vuelva una **constante**. Esto se hace a través del modificador `final` como se muestra a continuación. Por ejemplo, la siguiente declaración haría que la variable `c` no pueda cambiar de valor.

```
final int c = 5;
```

Dado que la variable no puede cambiar de valor, es necesario inicializarla con algún valor en el momento de su declaración. Sería un error tratar de cambiar luego este valor.

ARREGLOS

El **arreglo** es una estructura presente en la gran mayoría de los lenguajes, por lo cual tampoco falta en Java. Aunque se utiliza una notación similar a los demás lenguajes, su manejo es un poco diferente. Esto último se debe a que los arreglos se manipulan por referencia, al igual que los objetos. La declaración básica de un arreglo es de la siguiente forma:

```
int numeros[];
```

Esta declaración especifica que la variable `numeros` hará referencia a un arreglo de números enteros, que luego serán accedidos mediante la siguiente notación, donde `i` representa el elemento del arreglo:

```
numeros[i];
```

Para que esto funcione correctamente, es necesario inicializar la variable `numeros` para que se refiera a algún arreglo, ya que hasta ahora sólo hemos hecho una declaración. Existen dos formatos diferentes para hacer esta inicialización. El primer formato es similar al de C:

```
int numeros[] = {1,2,4,8,16,32,64,128}
```

Esta declaración crea un arreglo de ocho elementos inicializando sus elementos a los valores especificados. La variable `numeros` guarda la referencia a dicho arreglo.

La segunda manera de inicializar un arreglo es mediante la palabra `new`, que como veremos más adelante, se utiliza para instanciar cualquier tipo de obje-

tos. La inicialización de un arreglo mediante la palabra `new` es más común y especifica el tamaño del arreglo, como se muestra a continuación:

```
int numeros[] = new int[50];
```

Por ejemplo, esta declaración incluye la inicialización de `numeros` como referencia a un arreglo de 50 enteros, donde cada uno se inicializa con el valor de 0.

Un aspecto importante con los arreglos es que se puede conocer su largo utilizando el modificador `length`, como se muestra a continuación:

```
numeros.length
```

Esto permite obtener el largo definido para el arreglo.

Los arreglos no tienen que ser de una sola dimensión, ya que Java apoya múltiples dimensiones. Estos arreglos se implementan como "arreglos de arreglos". Por ejemplo, un arreglo de dos dimensiones de enteros se declara e inicializa de la siguiente manera:

```
int numeros2[][] = new int[10][20];
```

Esta declaración genera un arreglo con 200 elementos inicializados todos a 0. Como mencionamos antes, el manejo de Java implica que existen 10 arreglos, cada uno se refiere a un arreglo de una dimensión de 20 elementos.

Cuando se asigna un arreglo multidimensional, no se tiene que especificar el número de elementos contenido en cada dimensión. Por ejemplo, la siguiente es una declaración válida:

```
int numero3[][][] = new int[10][][];
```

Esta declaración asigna un arreglo que contiene 10 arreglos de una dimensión, donde cada uno se refiere a un arreglo de tipo `int[][]`. La regla básica en Java es que las primeras n dimensiones, con $n \geq 1$, deben especificar el número de elementos.

Observe que en el caso de funciones (que veremos más adelante en la sección de clases), la declaración de un arreglo como argumento de la función es similar al manejo descrito aquí. Sin embargo, existen dos formatos aceptados:

► Se puede utilizar la notación similar a C:

```
void func(char buf[]) {  
    char s[] = new char [50]; ... }
```

► Pero también se puede utilizar la siguiente notación:

```
void func(char[] buf) {  
    char[] s = new char [50]; ... }
```

En el caso de arreglos de objetos, su manejo es bastante similar al de los tipos primitivos. La consideración principal es que crear un arreglo de objetos no crea

todos los objetos que se guardan en él. Esto se hace principalmente para ofrecer mayor flexibilidad en la creación del arreglo, como permitir al programador hacer las llamadas deseadas a los constructores. Por tanto, si se quiere crear, por ejemplo, un arreglo de 20 objetos de tipo *Persona*:

```
Persona p[] = new Persona[20];
for (int i = 0; i < i.length; i++)
    p = new Persona();
```

Note que la primera declaración e instanciación del arreglo genera el arreglo de 20 elementos de tipo *Persona*, aunque cada elemento está vacío. Dentro del ciclo "for" se instancia cada elemento, en este caso con los valores de omisión de la clase. Algunos detalles utilizados en este ejemplo se expondrán más adelante en el capítulo.

CADENAS

Como en la mayoría de los lenguajes, se pueden definir **cadenas** como arreglos de caracteres. En otras palabras se puede definir una cadena de la siguiente manera:

```
char s[] = "Prueba";
```

La problemática con esta manera de definir cadenas es que su manipulación se vuelve complicada, por ejemplo comparar o concatenar cadenas. Para lograr un mejor manejo, Java define la cadena como una clase con todos los beneficios que esto significa. Aunque el tema de clases y objetos será tratado más adelante, vale la pena mencionar algunos aspectos del manejo de las cadenas en Java.

Los objetos tipo cadenas son instancias de la clase `java.lang.String`. Por ejemplo, una cadena puede instanciar cadenas de la siguiente forma:

```
String s = "Prueba";
```

Un aspecto que se resalta en Java, en relación con el manejo de cadenas, es el operador `+`, que corresponde a una concatenación. Por ejemplo, la siguiente expresión es válida y resultaría en las dos cadenas concatenadas:

```
String s1 = "Hola";
String s2 = "Mundo";
String s3 = s1 + s2;
```

Esta operación tiene efectos profundos para el compilador. Por ejemplo, la siguiente función de impresión en C es muy problemática para el compilador, ya que comprende un número de argumentos variables (al igual que muchas otras funciones similares):

```
printf("%s%s", s1, s2);
```

Esto se resuelve en Java mediante el operador de concatenación:

```
print(s1 + s2);
```

(En realidad, la función `print` requiere del prefijo `System.out` para ser llamada correctamente.) Éste es sólo un ejemplo de las facilidades del manejo de cadenas en Java (al igual que de muchos otros aspectos). La única restricción importante en los objetos de tipo `String` es que son inmutables, no pueden cambiarse una vez asignados. Por ejemplo, para manejar cadenas modificables se deben instanciar objetos de la clase `StringBuffer`, a partir de un objeto `String`, algo que va más allá del alcance introductorio de este capítulo.

Existen ciertas funciones que se pueden aplicar a las cadenas para manipularlas, como son: `length()`, `charAt()`, `equals()`, `compareTo()`, `indexOf()`, `lastIndexOf()`, `substring()`. Estas funciones hacen de Java un lenguaje muy poderoso.

ASIGNACIÓN

Las variables de tipos primitivos se utilizan para guardar valores de sus respectivos tipos. Por tanto, la expresión más importante que hay es la de **asignación** de valores. Por ejemplo, para asignarle un valor a una variable `x` de tipo entero se haría lo siguiente:

```
x = 10;
```

Obviamente tuvo que haberse hecho antes la declaración correspondiente. Incluso puede hacerse la asignación al momento de la declaración:

```
int x = 10;
```

La asignación es similar para todos los demás tipos primitivos.

OPERADORES

Java apoya todos los operadores estándares de C con la misma precedencia. La tabla 5.5 muestra todos los operadores que se pueden aplicar a tipos primitivos (incluyendo aritméticos, integrales y booleanos) en orden de precedencia.

Por ejemplo, la siguiente es una expresión de multiplicación,

```
int x;  
x = 23*54;
```

Aunque no es el objetivo mostrar ejemplos de todos los posibles operadores, vale la pena resaltar algunas diferencias de Java con C y C++:

- ▶ Java no apoya los operadores de apuntadores `*`, `&` o `sizeof`.
- ▶ Java no considera a `.` (acceso a campo) y `[]` (acceso a arreglo) como operadores.
- ▶ Java no apoya la sobrecarga de operadores.

Acerca de los operadores en Java, es necesario comentar que todos los tipos son valores con signo, el operador `>>` se define como un corrimiento a la derecha con extensión de signo, mientras que el operador `>>>` trata el valor de corrimiento lógico como un valor sin signo, y lo corre a la derecha con extensión de cero.

Tabla 5.5 Operadores para tipos primitivos predefinidos en Java.

Operador	Tipo de operando(s)	Descripción de la operación
++, --	Aritmético	Incremento, decremento (unario) (pre o post)
+, -	Aritmético	Más, menos (unario)
~	Integral	Complemento de bit (unario)
!	Booleano	Complemento lógico (unario)
(tipo)	Cualquiera	"Cast"
*, /, %	Aritmético	Multiplicación, división, resta
+, -	Aritmético	Suma, resta
+	Cadena	Concatenación de cadenas
<<	Integral	Desplazamiento hacia la izquierda
>>	Integral	Desplazamiento hacia la derecha con signo
>>>	Integral	Desplazamiento hacia la derecha con extensión de cero
<, <=	Aritmético	Menor que, menor o igual que
>, >=	Aritmético	Mayor que, mayor o igual que
==	Primitivo	Igual (tienen valores idénticos)
!=	Primitivo	No igual (tienen valores diferentes)
&	Integral	AND para bits
&	Booleano	AND booleano
^	Integral	XOR para bits
^	Booleano	XOR booleano
	Integral	OR para bits
	Booleano	OR booleano
&&	Booleano	AND condicional
	Booleano	OR condicional
? :	Booleano, cualquiera	Operador condicional (ternario)
=	Variable, cualquiera	Asignación
*, /=, %=, +=, -=, <<=, >>=, >>>=, &=, ^=, =	Variable, cualquiera	Asignación con operación

CONTROL

Además de los operadores, existen varias expresiones de **control** que utilizan los lenguajes para controlar el flujo de la lógica del programa. Estas expresiones son bastante estandarizadas en los lenguajes modernos, aunque varían en ciertos detalles. La tabla 5.6 muestra las expresiones de control.

Tabla 5.6 Expresiones de control en Java.

Expresión	Descripción de la expresión
<code>if (condición-1) bloque-1</code> <code>else if (condición-i) bloque-i</code> <code>else bloque-n</code>	Si la condición-1 es verdadera, se ejecuta el bloque-1; de lo contrario se prosigue con la condición-i de manera similar para ejecutar el bloque-i. Puede haber un número infinito de estas condiciones. Si ninguna condición es verdadera se ejecuta el bloque-n.
<code>while (condición) bloque</code> <code>do bloque while (condición)</code>	Mientras la condición sea verdadera, se ejecuta el bloque. A diferencia de la expresión anterior, primero se ejecuta el bloque y luego se revisa la condición para la siguiente ejecución.
<code>switch (variable)</code> <code>case valor-i: bloque-i</code> <code>default: bloque-n</code>	Se verifica el valor de la variable (tipo integral). Se comparan los valores especificados para los diversos casos, valor-i, hasta encontrar uno igual. En ese momento se ejecuta bloque-i. Si no se encuentra ninguno igual, se ejecuta bloque-n.
<code>for (expresión-1;</code> <code>condición-2; expresión-3) bloque</code>	Se ejecuta expresión-1 al inicio de esta expresión. Si condición-2 es verdadera, se ejecuta el bloque. A continuación se ejecuta expresión-3 y se prueba condición-2. Si ésta es verdadera, nuevamente se ejecuta el bloque. Se sigue ejecutando el bloque, precedido de la ejecución de expresión-3, mientras condición-2 sea verdadera.
<code>break etiqueta;</code>	Esta expresión permite interrumpir el flujo de una estructura de control con la opción de saltar fuera de la sección etiquetada por etiqueta; o en su ausencia, saltar fuera de la estructura de control actual.
<code>continue etiqueta;</code>	Esta expresión permite interrumpir el ciclo actual de una estructura de control con la opción de saltar a la última línea de la sección etiquetada por etiqueta; o en su ausencia, saltar a la última línea de la estructura de control actual.
<code>etiqueta: expresión;</code>	Etiqueta asignada a una expresión, utilizada en conjunción con break y continue.
<code>return expresión;</code>	Esta expresión devuelve el valor generado por expresión.

Si los bloques en la tabla involucran más de una oración, deben incluir llaves para especificar el inicio y fin del bloque. Todas las condiciones deben ser expresiones que devuelvan un valor booleano. Nuevamente, hacemos énfasis en que un valor numérico no puede utilizarse como uno booleano en Java, ni siquiera haciendo un cast. Los valores `false` y `0` no son equivalentes.

En el caso de la expresión de `for`, en Java no se permite el uso de la coma para separar múltiples expresiones dentro de la condición, aunque sí es permitido dentro de las dos secciones, `expresión-1` y `expresión-3`. El resto de las expresiones, con excepción de la *etiqueta*, son similares en su uso a las del lenguaje de programación de C.

5.3 Programación avanzada

En las siguientes secciones se describen conceptos más avanzados de la programación en Java.

ARCHIVOS

En Java es relativamente sencillo tener acceso a archivos, como lo ilustra el siguiente código en Java:

```
File file = new File(dir,archivo);
BufferedReader is = new BufferedReader(new FileReader(file));
String s = is.readLine();
...
is.close();
```

Se instancia un archivo `file` de tipo `File`, especificando su ubicación en el sistema, `dir`, y su nombre, `archivo`. A continuación se instancia un objeto de tipo `FileReader`, el cual se conecta al archivo `file` y, luego, en la misma línea, se instancia el objeto `is` de tipo `BufferedReader` —que permite conectarse a través de un búfer de lectura—. La llamada `is.readLine()` hace una lectura de una línea completa y la guarda en una variable `s` de tipo `String`. Luego de terminar de leer la información deseada del archivo, éste se cierra mediante la llamada `is.close()`.

La escritura es análoga a la lectura, como se muestra a continuación:

```
BufferedWriter os = new BufferedWriter(new FileWriter(file));
os.write(s);
...
os.close();
```

Se instancia un archivo `file`, como se mostró anteriormente. A continuación se instancia un objeto de tipo `FileWriter`, el cual se conecta al archivo `file` y, luego, en la misma línea, se instancia el objeto `os` de tipo `BufferedWriter`, que permite conectarse a través de un búfer de escritura. La llamada `os.write(s)` hace una escritura de una cadena referida por la variable `s` de tipo `String`. Después de terminar de escribir la información deseada en el archivo, éste se cierra mediante la llamada `os.close()`.

BASES DE DATOS

Accesar una base de datos también es relativamente sencillo en Java. Inicialmente se verifica que exista el paquete o biblioteca de Java que permite administrar la conexión a las bases de datos, como se presenta a continuación:

```
Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver");
```

La propia conexión a la base de datos se hace a través de una llamada similar a la siguiente:

```
Connection con = DriverManager.getConnection("jdbc:odbc nombre", login,
password);
```

Esto genera una conexión a una base de datos llamada *nombre*, que puede accerse de forma opcional a través de un nombre de usuario *login* y una contraseña *password*. La conexión queda abierta hasta que se cierre mediante la siguiente llamada:

```
con.close();
```

Luego, se debe instanciar una variable de tipo *Statement*, la cual se utiliza como contenedor de la llamada en SQL:

```
Statement stmt = con.createStatement();
```

Por ejemplo, si se quisiera hacer una consulta a una tabla con un nombre de usuario *log*, primero se genera una cadena *query* para guardar la llamada de SQL, y luego se ejecuta mediante la llamada *stmt.executeQuery(query)*, como se muestra a continuación:

```
String query = "SELECT * FROM tabla WHERE (login = 'log')";
```

```
ResultSet rs = stmt.executeQuery(query);
```

Esta llamada regresa un resultado *rs* de tipo *ResultSet*, del cual se obtiene la estructura de la tabla, o sea la descripción del **metadato**; para luego poder leer de manera correcta los datos, como se ilustra a continuación:

```
ResultSetMetaData rsmd = rs.getMetaData();
int numCols = rsmd.getColumnCount();
while (rs.next()) {
    for (int i = 1; i <= numCols; i++)
        String str = rs.getString(i);
    ...
}
```

En el caso de una cadena, la propia lectura se hace mediante la llamada *rs.getString(i)*, donde *i* identifica la columna de la tabla a partir del valor 1. Mientras *rs.next()* no sea nulo, existen resultados adicionales para seguir leyendo.

A diferencia de la consulta, las inserciones o actualizaciones de las tablas se hacen utilizando la llamada *stmt.executeUpdate(update)*, como se muestra a continuación:

```
String update = "INSERT INTO tabla ...";
int n = stmt.executeUpdate(update);
```

Note que la llamada *stmt.executeUpdate(update)* regresa ahora un entero correspondiente al número de récords, que fueron insertados o modificados exitosamente. El formato de las inserciones y actualizaciones corresponden a las especificaciones de SQL y no serán tratadas aquí en más detalle.

EXCEPCIONES

Un aspecto de suma importancia y que es integral en lenguajes como Java, es el manejo de *excepciones*. Cuando un error ocurre en un programa, el sistema lanza una **excepción** que el programa atrapa. El manejo de excepciones se requiere para cualquier código que pueda resultar en estados inconsistentes, en particular situaciones donde se trate de acceder entidades externas al programa, como son los archivos y las bases de datos.

En Java, este manejo se logra mediante tres palabras reservadas:

- ▶ **try**. Define un bloque de código donde pudieran ocurrir excepciones.
- ▶ **catch**. Define una sección para el manejo de las excepciones (a través de la clase `Throwable` o alguna de sus subclases). Este bloque es opcional.
- ▶ **finally**. Código a ejecutarse como finalización del bloque, ocurran o no excepciones. Este bloque es opcional.

Por ejemplo, veamos el siguiente caso para la lectura de un archivo:

```
try {  
    BufferedReader is = new BufferedReader(new FileReader(file));  
    leerArchivo(is);  
    is.close();  
}  
catch(IOException e) {  
    System.out.print("Error Lectura Registro: " + e);  
    System.exit(1);  
}  
finally {  
    System.out.print("Lectura Archivo: " + file);  
}
```

El bloque `try` abre la conexión al archivo de lectura `file`, llama al método de lectura, `leerArchivo` y cierra posteriormente la conexión al archivo. El bloque `catch` define el manejo de la excepción, `IOException`, la cual se pasa como argumento único dentro del bloque. En este ejemplo, se imprime el tipo de excepción y se sale del programa. El bloque `finally` siempre se ejecuta al finalizar los dos bloques anteriores (a menos que se salga de la aplicación). En el ejemplo se imprime un mensaje sobre el archivo que se está leyendo.

Java requiere que las posibles excepciones sean manejadas mediante bloque `throws`. Por ejemplo, el acceso a archivos puede ocasionar una excepción de lectura o escritura si el archivo no existe o se encuentra vacío. El método `leerArchivo` se muestra a continuación:

```
public void leerRegistros(BufferedReader is) throws IOException {  
    String s = is.readLine();  
    ...  
}
```

Observe que la excepción `IOException`, que se incluye con el `throws`, también debe aparecer como argumento en el bloque `catch` correspondiente, si no ocurriría un error de compilación.

CÓDIGO NATIVO

Java permite al desarrollador de software incluir código escrito en otros lenguajes, en particular C, como parte de programa en Java. Para ello, se aplica el modificador `native` a las declaraciones de los métodos que son implementados de manera **nativa** en C o algún otro lenguaje, pero no en Java. Al igual que un método abstracto, se agrega un punto y coma al final de una declaración nativa.

SINCRONIZACIÓN

Al igual que otros lenguajes, Java permite procesar múltiples objetos y métodos de manera concurrente dentro de una aplicación mediante el uso de *múltiples hilos* (*threads*) de control. La concurrencia en el procesamiento requiere de *sincronización* de los bloques afectados. Para ello, Java incluye el modificador `synchronized`, el cual especifica una *sección crítica* de procesamiento que no puede interrumpirse durante su ejecución.

SERIALIZACIÓN

Como apoyo al procesamiento distribuido de aplicaciones, Java permite enviar objetos entre diversos procesos. Este envío se facilita mediante el uso de la interface `serializable`, la cual debe ser incluida en la implementación de la clase afectada.

FINALIZADOR

Existe en Java un método de finalización de clase llamado `finalize`. El objetivo del finalizador, de manera complementaria al constructor, es cerrar, por ejemplo, archivos o sockets que se encuentren abiertos. Un finalizador se vería de la siguiente forma:

```
protected void finalize() throws IOException {  
    if (fd != null) close();  
}
```

Java llama a un finalizador una sola vez por objeto. El método finalizador se llama por lo general antes de la recolección de basura; sin embargo, Java no garantiza el orden de estas llamadas, y cualquier recurso aún no cerrado o recolectado sería liberado al terminar el programa.

5.4 UML y Java

5.4.1 Objetos y clases

Todo programa de Java debe incluir clases, por lo que es necesario considerar diversos aspectos de éstas, como se describió en el capítulo 4. Utilizamos la notación UML para representar una clase, como se ilustra en la figura 5.2.



Figura 5.2 Notación de UML para una clase.

En Java, el código correspondiente a una clase se muestra a continuación:

```
class NombreClase {  
}
```

Observe que el nombre de la clase siempre comienza con mayúscula. Si el nombre es compuesto, como en este caso, la siguiente palabra debe iniciarse también con mayúsculas para facilitar su lectura. No deben haber espacios dentro del nombre de la clase.

La anterior es probablemente la definición más sencilla que puede asignarse a una clase. La primera palabra `class`, sirve de prefijo para indicar el inicio de una clase.

Por ejemplo, consideremos la clase `Persona` como se muestra en la figura 5.3.



Figura 5.3 Notación de UML para una clase llamada `Persona`.

La clase `Persona` se define de la siguiente manera:

```
class Persona {  
}
```

En las siguientes secciones mostraremos de manera incremental el resto de las definiciones relacionadas con la clase.

ATRIBUTOS

El siguiente paso en la definición de una clase es indicar sus atributos, éstos se muestran nuevamente en la figura 5.4.

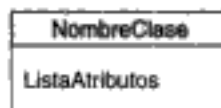


Figura 5.4 Notación de UML para una clase con atributos.

En Java, el código correspondiente a una clase con atributos se muestra a continuación:

```
class NombreClase {  
    // atributos  
    tipoAtributo1 nombreAtributo1;  
    ...  
    tipoAtributoi nombreAtributoi;  
    ...  
    tipoAtributoN nombreAtributoN;  
}
```

La lista de atributos corresponde a declaraciones de tipos primitivos, compuestos de un tipo, `tipoAtributo`, seguido de un nombre, `nombreAtributo` (los “...” son únicamente para resaltar que es una lista de atributos, y la línea “// atributos” representa un comentario únicamente). Observe que los atributos comienzan siempre con una letra minúscula, aunque las palabras siguientes en el caso de nombres compuestos, pueden iniciar con mayúsculas. Como con los nombres de clases, no deben existir espacios dentro del nombre y, en particular, no deben haber nombres repetidos.

Por ejemplo, consideremos la clase **Persona** con varios atributos, como se muestra en la figura 5.5.

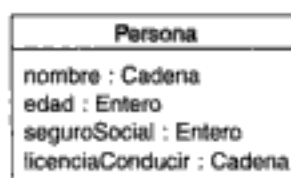


Figura 5.5 Notación de UML para una clase llamada **Persona**, que contiene atributos.

La clase **Persona** y sus atributos se definen de la siguiente manera.

```
class Persona {
// atributos
    String nombre;
    int edad;
    int seguroSocial;
    String licenciaConducir;
}
```

El orden de los atributos no tiene ninguna importancia dentro de la clase. Note que los tipos de los atributos no necesariamente tienen que ser tipos primitivos, como es el caso de `String`.

OPERACIONES

El siguiente paso en la definición de una clase es indicar sus operaciones, éstas, junto con los atributos, se ilustran en la figura 5.6.

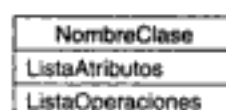


Figura 5.6 Notación de UML para una clase con atributos y operaciones.

En Java, el código correspondiente a una clase con atributos y operaciones se muestra a continuación:

```

class NombreClase {
// atributos
    tipoAtributo1 nombreAtributo1;
    ...
    tipoAtributoi nombreAtributoi;
    ...
    tipoAtributoN nombreAtributoN;
// operaciones
    tipoRetorno1 nombreMétodo1 ( listaParámetrosMétodo1 )
    { cuerpoMétodo1 }
    ...
    tipoRetornoj nombreMétodoj ( listaParámetrosMétodoj )
    { cuerpoMétodoj }
    ...
    tipoRetornoM nombreMétodoM ( listaParámetrosMétodoM )
    { cuerpoMétodoM }
}

```

Aunque conceptualmente se habla de operaciones en los lenguajes de programación, es más preciso hablar de métodos. La relación entre estos dos términos radica en que múltiples métodos pueden corresponder a una misma operación. La lista anterior de métodos esta compuesta por el tipo de valor de retorno, `tipoRetornoj`; el nombre del método, `nombreMétodoj`; los parámetros que recibe el método, `listaParámetrosj`; y, finalmente, el cuerpo del método, `nombre-Cuerpoj`. (Nuevamente, los “...” son únicamente para resaltar que es una lista de métodos.) Observe que los nombres de los métodos comienzan siempre con una letra minúscula, aunque las siguientes palabras, en el caso de nombres compuestos, pueden comenzar con mayúsculas. Como con los nombres de clases y atributos, no deben existir espacios dentro del nombre. En particular, `lista-Parámetros`, tiene el siguiente formato:

```

tipoRetorno nombreMétodo ( tipo1 par1, tipo2 par2,...,tipoN parN )
{ cuerpoMétodo }

```

Por otro lado, `cuerpoMétodo`, es una lista de expresiones similares a las descritas en la sección correspondiente, además de llamadas a otros métodos.

A diferencia de los atributos, pueden haber nombres repetidos para los métodos; a esto se le conoce como *sobrecarga* de métodos.

Por ejemplo, consideremos la clase `Persona` con varios métodos, además de los atributos anteriores, ver figura 5.7.

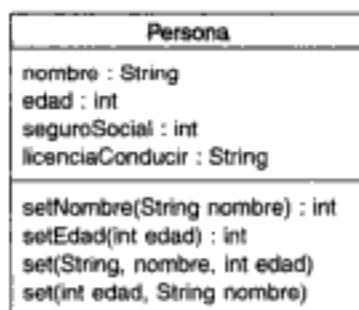


Figura 5.7 Notación de UML para una clase `Persona` que contiene atributos y métodos.

La clase `Persona`, con sus atributos y métodos, se define de la siguiente manera.

```
class Persona {
    String nombre;
    int edad;
    int seguroSocial;
    String licenciaConducir;

    int setNombre(String nom) {
        nombre = nom; return 1; }
    int setEdad(int ed) {
        edad = ed; return 1; }
    void set(String nom, int ed) {
        setNombre(nom); setEdad(ed); }
    void set(int ed, String nom) {
        setNombre(nom); setEdad(ed); }
}
```

El orden de los métodos no tiene ninguna importancia dentro de la clase. Es importante que el nombre de un parámetro sea diferente al de un atributo para evitar posibles conflictos. Si los dos nombres fueran iguales, la variable que se utilizará se resuelve según su alcance (*scope*). (Se emplea la variable cuya definición sea la más cercana a su lugar de utilización, en este caso el parámetro del método tendría precedencia.) Otro aspecto a resaltar es el uso del `return`, en el caso de métodos que devuelven algún tipo que no sea `void`. Además, los últimos dos métodos tienen nombre similar, por lo cual realmente corresponden a una misma operación que es "asignar el valor al nombre y edad", sin importar el orden de los parámetros. Este uso de la sobreescritura de métodos es muy común. Por último, vale la pena resaltar el manejo de parámetros. Todos los parámetros relacionados con tipos primitivos se envían *por valor*. Esto significa que si el valor del parámetro se cambia dentro del método, no afectaría de ninguna manera su valor original. Por ejemplo, consideremos la siguiente versión de los métodos anteriores:

```
int setEdad(int ed) {
    edad = ed; ed = 0; return 1; }
void set(String nom, int ed) {
    setEdad(ed); setEdad(ed); }
```

En el primer método se asigna el valor `ed` a `edad` y, luego, se asigna `0` a `ed`. En el segundo método, se llama dos veces al método `setEdad`. Dado que `ed` se envió *por valor*, el `0` nunca fue devuelto al llamado original, y el segundo `setEdad` vuelve a asignar la `edad` correcta.

Sin embargo, éste no es el caso con los objetos, ya que son enviados "por referencia". En otras palabras, aunque las variables no sean globales, los objetos a los que se refieren éstas sí lo son, tal como un objeto de tipo `String`. Consideremos ahora la siguiente modificación a los métodos originales:

```
int setNombre(String nom) {
    nombre = nom; nom = null; return 1; }
void set(String nom, int ed) {
    setNombre(nom); setNombre(nom); }
```

En el primer método, `setNombre`, se asigna la referencia que guarda la variable `nom` a `nombre`. A continuación se asigna el valor `null` a `nom`, o sea una referen-

cia nula. En el segundo método, `set`, existen dos llamadas al primer método, `setNombre`. La primera llamada asigna el valor original del parámetro `nom`, mientras que la segunda llamada asigna una referencia nula, ocasionada por la expresión `nom = null` de la primera llamada a `setNombre`.

ENCAPSULAMIENTO

En Java, como en la mayoría de los lenguajes orientados a objetos, es importante considerar el encapsulamiento de los atributos y métodos definidos en la clase. Aunque todos los campos de una clase son accesibles dentro de ella misma.

Para ello, Java define tres modificadores básicos del manejo del encapsulamiento, que pueden aplicarse a los campos o miembros (atributos y métodos) de una clase, y a la propia clase completa: `public`, `private` y `protected`, como se muestra a continuación:

- ▶ **public.** Se agrega a los campos de la clase que pueden ser accesados fuera de ésta. En general, deben ser públicos los métodos de la clase, aunque no necesariamente todos sus métodos.
- ▶ **private.** Se agrega a los campos de la clase que son accesados únicamente desde dentro de la misma, o sea, dentro de sus propios métodos. En general, deben ser privados los atributos de la clase, y posiblemente algunos métodos de uso interno.
- ▶ **protected.** Se agrega a los campos de la clase que son accesados sólo desde dentro de ésta o de una subclase que hereda de la actual, o sea, dentro de sus propios métodos o métodos de alguna de sus subclases. En general, los atributos de la clase deben protegerse y, posiblemente, también algunos métodos de uso interno.

La distinción entre estos modificadores de encapsulamiento puede volverse un poco confusa, dado que además de afectar el encapsulamiento de los campos entre clases, también lo hace en el acceso, dependiendo si las clases son, o no, campos del mismo paquete. Por tanto, Java define dos maneras generales de aplicar estos modificadores, como se muestra a continuación:

- ▶ **Modificador de encapsulamiento para campo de una clase.** Se aplica únicamente a un atributo o método de una clase, y puede consistir de cualquiera de los tres modificadores: `public`, `private` y `protected`. Este modificador se añade al inicio de una declaración, sea atributo o método, como se muestra a continuación:

```
class Persona {  
    private String nombre;  
    protected int edad;  
    public int seguroSocial;  
    public String licenciaConducir;  
  
    private int setNombre(String nom) {  
        nombre = nom; return 1; }  
    protected int setEdad(int ed) {  
        edad = ed; return 1; }  
    public void set(String nom, int ed) {  
        setNombre(nom); setEdad(ed); }  
}
```

```

        public void set(int ed, String nom) {
            setNombre(nom); setEdad(ed); }
    }

```

- **Modificador de encapsulamiento para una clase.** Se aplica a toda la clase como tal y puede consistir sólo del modificador **public**. Afecta la visibilidad de la clase entre paquetes. Este modificador se añade al inicio de la especificación de la clase, como se muestra a continuación:

```

public class Persona {
    private String nombre;
    protected int edad;
    public int seguroSocial;
    public String licenciaConducir;
    private int setNombre(String nom) {
        nombre = nom; return 1; }
    protected int setEdad(int ed) {
        edad = ed; return 1; }
    public void set(String nom, int ed) {
        setNombre(nom); setEdad(ed); }
    public void set(int ed, String nom) {
        setNombre(nom); setEdad(ed); }
}

```

En general, una vez que la clase es pública en otro paquete, entra en rigor la visibilidad de sus campos. La tabla 5.7 muestra los diferentes efectos de estos modificadores, que dependen de cuatro formas de acceder campos de una clase: dentro de la misma clase; dentro de una clase en el mismo paquete; dentro de una subclase en otro paquete; o dentro de una clase en otro paquete, pero que no es una subclase de la actual. Se consideran cuatro niveles de encapsulamiento: **public**, **protected** y **private**, para campos de clase, y **paquete**, que corresponde a la visibilidad existente si se omite el modificador de sus campos. Esto último indica que no es obligatorio utilizar los modificadores de encapsulamiento. Sin embargo, su efecto no corresponde a ninguno de los tres casos, como a menudo ocurre con otros lenguajes. Esto se debe principalmente a la existencia de paquetes.

Tabla 5.7 Modificadores de encapsulamiento y su efecto sobre las diversas estructuras.

Es accesible por:	Encapsulamiento			
	public	protected	package	private
Misma clase	Sí	Sí	Sí	Sí
Clase en el mismo paquete	Sí	Sí	Sí	No
Subclase en un paquete diferente	Sí	Sí	No	No
No subclase en un paquete diferente	Sí	No	No	No

La explicación de la tabla 5.7 es la siguiente:

- Todos los campos o miembros de una clase son siempre accesibles dentro de una misma clase, sin importar el modificador de sus campos.
- Todos los campos de una clase son siempre accesibles por cualquier otra clase, incluyendo subclases, dentro del mismo paquete, siempre y cuando el campo no sea **private**.

- Una subclase en un paquete distinto al de la superclase sólo puede tener acceso a campos `public` o `protected`. Observe que la clase debe ser `public` para ser vista en otro paquete.
- Una clase, que no sea una subclase sólo puede acceder campos `public` de una clase en otro paquete. Nuevamente la clase debe ser `public` para poderse ver en otro paquete.

CONSTRUCTORES

Para completar los aspectos fundamentales de una clase, se deben considerar sus **constructores**, que son métodos especiales que pueden ser llamados únicamente durante la instanciación de un nuevo objeto (esto lo estudiaremos en la siguiente sección). El constructor lleva el mismo nombre de la clase y puede incluir parámetros. A diferencia del resto de los métodos, un constructor no especifica ningún tipo de retorno, ya que de por sí, el objeto recién creado es lo que se devuelve. Su formato es como se muestra a continuación:

```
class NombreClase {
// atributos
    listaAtributos
// constructor
    NombreClase ( listaParámetrosConstructor1 )
    { cuerpoConstructor1 }
    ...
    NombreClase ( listaParámetrosConstructor1 )
    { cuerpoConstructor1 }
    ...
    NombreClase ( listaParámetrosConstructorN )
    { cuerpoConstructorN }
// operaciones
    listaMétodos
}
```

Pueden existir múltiples constructores, pero todos deben tener el mismo nombre, que es idéntico al de la clase. Éste es otro ejemplo de sobrecarga, en este caso el de constructor de la clase. Como en los métodos, no pueden existir dos constructores con una lista de parámetros iguales, y como los métodos, la lista de parámetros puede estar vacía, el constructor no es obligatorio en Java, ya que por omisión se generaría uno con lista de parámetros y un cuerpo vacíos. A continuación se muestra un ejemplo del uso de los constructores:

```
class Persona {
    private String nombre;
    private int edad;
    private int seguroSocial;
    private String licenciaConducir;

    public Persona(String nom, int ed, int seg, String lic) {
        set(nom, ed); seguroSocial = seg; licenciaConducir = lic; }

    public int setNombre(String nom) {
        nombre = nom; return 1; }
    public int setEdad(int ed) {
        edad = ed; return 1; }
    public void set(String nom, int ed) {
        setNombre(nom); setEdad(ed); }
    public void set(int ed, String nom) {
        setNombre(nom); setEdad(ed); }
}
```

Observe que cambiamos los modificadores de encapsulamiento de los atributos y métodos para volverlos privados y públicos, respectivamente. Además, note que los constructores también aceptan los modificadores de encapsulamiento de manera similar a los métodos. Un constructor **private** nunca puede llamarse (un ejemplo de una clase que nunca podrá ser instanciada) y un constructor **protected** sólo puede ser instanciado por una subclase. En el ejemplo anterior, el constructor acepta valores de inicialización para todos los atributos de la clase.

A diferencia de los lenguajes como C++, Java no requiere un destructor, aunque sí existe la función especial *finalize*, que permite el manejo avanzado de recolección de basura.

INSTANCIACIÓN

Una vez definidos los aspectos esenciales de la clase, el siguiente paso es instanciar objetos. Para crear un nuevo objeto, se utiliza el operador **new**, seguido por la clase a la que pertenece el objeto. Además de la clase, puede haber una lista de argumentos opcionales entre paréntesis, que se asignan y deben corresponder a la lista de parámetros de algún constructor de la clase. Si la clase no tiene un constructor, la lista deberá estar vacía. Debe quedar muy claro que este operador permite instanciar un nuevo objeto, pero si no existe una variable que guarde la referencia al nuevo objeto, el objeto se perderá. Por lo tanto, antes de proceder con la instanciación, debe declararse una variable que guarde la referencia al nuevo objeto, como se muestra a continuación:

```
Persona p1 = new Persona("Juan",35,1234567,"x254f");
```

Esta instanciación asigna los valores especificados a cada uno de los atributos.

En caso de no haber especificado ningún constructor, Java permite instanciar un nuevo objeto utilizando la llamada a un constructor vacío generado implícitamente, como se muestra a continuación:

```
Persona p2 = new Persona();
```

Esta instanciación crea un objeto tipo **Persona**, donde los atributos toman los valores asignados por omisión por Java. Esta generación implícita de Java no ocurre si ya se ha definido al menos otro constructor para esa clase. En este último caso, de no haber un constructor que tome argumentos, esta llamada hubiese ocasionado un error.

Como nota adicional, si se utilizara una sola variable para guardar la referencia a ambos objetos, la referencia del primero se perdería, ya que la variable siempre guarda el último valor asignado. Más aún, no es esencial que una variable explícitamente guarde la referencia a un objeto. Siempre que esa referencia esté guardada y sea accesible, por ejemplo dentro de alguna lista o como parámetro de un método, el objeto no será eliminado por el *recolector de basura*. En cambio, si no existe ninguna forma de acceder al objeto, éste será automáticamente borrado.

ACCESO A CAMPOS

Una vez instanciado un objeto, lo más importante es acceder los campos de la clase, los cuales ya han sido definidos previamente.

ATRIBUTOS

Los atributos (tipos primitivos) son manipulados como se explicó en la sección correspondiente. Estas manipulaciones se hacen mediante expresiones tales como asignación o flujos de control. Lo que se debe considerar en los objetos es el acceso mediante la variable que guarda la referencia al objeto seguido por un "." y el nombre del atributo. Por ejemplo, para asignar el número del `seguroSocial` del objeto `p1` al valor `8888888`, donde el objeto es de tipo `Persona` se hace lo siguiente:

```
p1.seguroSocial = 8888888;
```

Observe que esta asignación se hace si el atributo no es privado, ya que de lo contrario, el encapsulamiento no lo permitiría. Cuando esto sucede, el acceso se logra a través de un método no privado del objeto, como veremos a continuación.

MÉTODOS

Los métodos se llaman desde la variable que guarda la referencia al objeto, seguido por un "." y el nombre del método. Por ejemplo, para copiar el número de la `edad` del objeto `p1` al valor `25`, donde el objeto es de tipo `Persona` se realiza lo siguiente:

```
p1.setEdad(25);
```

Note que esta llamada al método para la asignación puede hacerse si el método no es privado, ya que de lo contrario, el encapsulamiento no lo permitiría.

REFERENCIA PROPIA

Existe una palabra reservada, `this`, que es muy importante para referirse al objeto actual, y que se utiliza de dos maneras distintas: como referencia a algún campo del objeto o como llamada a un constructor. Veamos los dos casos:

- Como ejemplo de referencia al campo de un objeto, el atributo `edad` se puede acceder dentro del método `setEdad`, mediante el siguiente código:

```
protected int setEdad(int edad) {  
    this.edad = edad;  
    return 1;  
}
```

El `this` no cambia en absoluto la lógica original del método, aunque evita una posible confusión del programador en el caso de un argumento del método con nombre similar al de un atributo de la clase. Java resuelve esta situación con base en el *alcance* (*scope*) de la variable, utilizando aquella que sea definida más *cerca* de la expresión donde se usa la variable.

Sin embargo, el uso más importante del `this` es como variable a ser devuelta por un método. Por ejemplo, modifiquemos el método anterior, en particular el tipo devuelto, de la siguiente manera:

```
protected Persona setEdad(int ed) {  
    edad = ed;  
    return this;  
}
```

En este ejemplo, el `this` desempeña un papel primordial, pues permite que el método devuelva la referencia del propio objeto para ser usado por otras variables. Obviamente, la referencia debe ser de tipo `Persona` para que el código sea correcto.

- Como ejemplo de llamada a un constructor de un objeto, consideremos el siguiente constructor adicional para la clase `Persona`:

```
public Persona() {  
    this(null, 0, 0, null);  
}
```

El segundo constructor no tiene argumentos y se valió del primero para redirigir la llamada utilizando `this()`, en este caso con parámetros de omisión. En este ejemplo particular no es necesario hacer la última llamada para asignar valores similares a los que asigna Java por omisión a las variables, sin embargo, es una buena práctica. Por otro lado, sería necesario incluir esta llamada si quisiéramos asignar valores de omisión diferentes a los que asigna Java. La llamada a `this()` debe utilizarse dentro de un constructor y aparecer como su primera línea.

EXPRESIONES

Las expresiones que se aplican a los objetos son más bien limitadas, en el sentido de que las manipulaciones principales en el control de un programa son sobre los tipos primitivos. Los objetos como tales se usan más bien como encapsuladores de estos tipos primitivos, y no tanto como la base de expresiones. Sin embargo, existen algunos operadores que vale la pena mencionar y que se muestran en la tabla 5.8.

Tabla 5.8 Operadores que pueden ser aplicados a objetos.

Operador	Descripción de la operación
(tipo)	"cast"
instanceof	comparación de tipo
==	igual (se refieren al mismo objeto)
!=	no igual (se refieren a distintos objetos)

Compare esta lista reducida de operadores en relación con la lista extensa de operadores aplicables a tipos primitivos. De esta lista vale la pena resaltar los siguientes aspectos:

- Los operadores `==` y `!=` comparan referencias. Para comparar los propios valores a donde apuntan las referencias, se debe utilizar el método

```
equals()
```


- El operador `instanceof` devuelve `true` si el objeto a la izquierda es una instancia de la clase especificada a la derecha, de lo contrario devuelve `false`. Tiene la misma precedencia que los operadores de comparación.

Además, los objetos como tales se utilizan comúnmente en expresiones que involucren funciones, donde las referencias a los objetos son sus argumentos.

5.4.2 Ligas, asociaciones y composición

Hasta aquí se ha mostrado cómo se definen las clases y se crean los objetos. Para generar una aplicación completa es necesario relacionar clases, o sea objetos, entre sí, lo cual se hace mediante los conceptos de **ligas** y **asociaciones** entre objetos y clases, respectivamente. En la mayoría de los lenguajes orientados a objetos no existe ninguno de estos dos conceptos; por tanto, deben implementarse a través de algún mecanismo existente en el lenguaje. Es común que las asociaciones se describan mediante la especificación de referencias a otras clases, donde las referencias se guardan como atributos de la clase. En general, las asociaciones de grados mayores a dos se implementan mediante asociaciones binarias, por lo que analizaremos éstas últimas. Consideremos la relación entre las dos clases mostradas en el diagrama de la figura 5.8.



Figura 5.8 Asociación entre clases.

Una asociación binaria se implementa mediante un atributo correspondiente a cada clase de la asociación, como se muestra a continuación:

```
class Clase1 {  
    Clase2 ref;  
}  
class Clase2 {  
    Clase1 ref;  
}
```

El mayor problema que existe con la implementación de las asociaciones es mantener la consistencia entre las referencias. En otras palabras, si se elimina la asociación, o la liga correspondiente, es necesario que ambas referencias sean también eliminadas. Dado que los atributos de referencia no deben ser accedidos públicamente, se deben incluir métodos para eliminarlas, o en general, para actualizarlas. Éste es un ejemplo de la complejidad dada por el manejo adicional que requiere un programa, a falta de un mecanismo que implemente asociaciones y ligas de manera natural.

ROL

En el código de Java anterior no hay ningún indicio del concepto de la asociación, más que las dos referencias mencionadas. Por tanto, aunque la asociación tuviera un nombre, éste no se asignaría a ninguna de las dos clases, ya que el

concepto como tal de la asociación se ha perdido. Sin embargo, el nombre de rol es más fácil de asignar. Consideremos el diagrama de la figura 5.9.



Figura 5.9 Asociación entre clases con nombres de rol.

Los nombres de roles se utilizan para nombrar referencias entre clases, como se ve a continuación:

```
class Clase1 {
    Clase2 rol2;
}
class Clase2 {
    Clase1 rol1;
}
```

Observe que se utilizan los nombres de roles opuestos a la ubicación del atributo de referencia.

ACCESO

Analicemos ahora qué ocurre si integramos el concepto del acceso, o navegación, para las asociaciones apoyado por UML. El diagrama de la figura 5.10 muestra una asociación con navegación bidireccional, que es equivalente al código anterior. Vea que se agregó el nombre de la asociación, aunque esto no afecta el código.



Figura 5.10 Asociación entre clases con nombres de rol y navegación bidireccional.

Simplifiquemos un poco la asociación mediante la navegación de una sola dirección, como se muestra en la figura 5.11.



Figura 5.11 Asociación entre clases con nombres de rol y navegación en una sola dirección.

En este último diagrama la navegación es de la clase 2 a la clase 1, pero no viceversa. Por tanto, la relación puede implementarse mediante el siguiente código simplificado:

```
class Clase1 {
}
class Clase2 {
    Clase1 rol1;
}
```

MULTIPLICIDAD

En los ejemplos anteriores, la multiplicidad de la relación fue de *uno* a *uno*, en otras palabras, *un* objeto de una clase se relaciona con un objeto de otra clase. La multiplicidad múltiple, *uno* a *muchos* o *muchos* a *muchos*, implica cierta complicación ya que requiere de estructuras adicionales para administrar las referencias, en lugar de una sola referencia directa. Consideremos el diagrama de la figura 5.12.



Figura 5.12 Asociación entre clases con nombres de rol y multiplicidad *uno* a *muchos*, donde el asterisco, “*”, representa el lado de *muchos* en la relación.

El código para el lado “muchos” requiere de un conjunto de objetos, o un arreglo de referencias, como se muestra a continuación:

```
class Clase1 {
    Clase2 rol2[];
}
class Clase2 {
    Clase1 rol1;
}
```

En el caso de utilizar un arreglo para la administración de las múltiples referencias, el número máximo de posibles ligas debe conocerse con anterioridad. Una opción más eficaz es utilizar estructuras de datos más avanzadas, como un objeto contenedor *Vector*, o algún otro ofrecido por Java o escrito por el programador, que evite predefinir el número máximo de relaciones que pueden haber para una clase particular.

El caso de multiplicidad *muchos* a *muchos* es una extensión del caso anterior. Consideremos el diagrama de la figura 5.13.



Figura 5.13 Asociación entre clases con nombres de rol y multiplicidad *muchos* a *muchos*.

El código para ambos lados de la asociación requiere un conjunto de objetos, o un arreglo de referencias, como se muestra a continuación:

```
class Clase1 {
    Clase2 rol2[];
}
class Clase2 {
    Clase1 rol1[];
}
```

ASOCIACIÓN REFLEXIVA

Una **asociación reflexiva** se implementa como caso especial de una asociación entre clases distintas. Consideremos el diagrama de la figura 5.14.



Figura 5.14 Asociación reflexiva con nombres de rol y multiplicidad.

El código para ambos lados de la asociación se muestra a continuación:

```
class Clase {
    Clase rol1;
    Clase rol2[];
}
```

Observe que las referencias son a la misma clase. Por otro lado, la multiplicidad 0..1 se implementa como la de *uno*.

ASOCIACIÓN COMO CLASE

Como se describió en el capítulo 4, es posible modelar las asociaciones como clases; lo cual en cierta manera simplifica la implementación de las asociaciones, en especial el manejo de la consistencia entre las referencias de clases. Aunque estos objetos no son difíciles de implementar, siempre ayuda que la biblioteca de clases contenga componentes que ayuden a su implementación. El enfoque más sencillo es implementar un objeto de asociación como un objeto *diccionario*, una estructura que a partir de una dirección obtiene la complementaria de la asociación. El diccionario debe actualizarse al mismo tiempo que la asociación. Consideremos el diagrama de la figura 5.15.

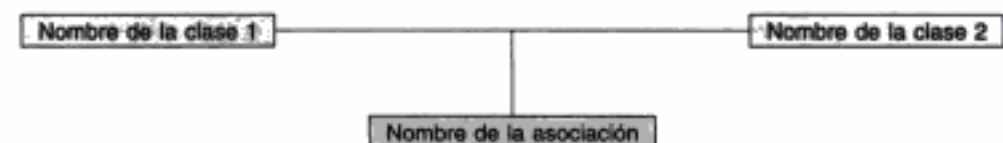


Figura 5.15 Asociación como clase.

El código para ambos lados de la asociación se muestra a continuación:

```
class Clase1 {
    Asociacion ref;
}
class Clase2 {
    Asociacion ref;
}
class Asociacion {
    Clase1 ref1[];
    Clase2 ref2[];
}
```

Note que las clases `Clase1` y `Clase2` tienen referencias a `Asociacion` (diccionario), mientras que la clase `Asociacion` es responsable de la consistencia de las múltiples relaciones entre `Clase1` y `Clase2`. Cualquier modificación en la multiplicidad de la asociación sólo afecta a la clase `Asociacion`. Además, la clase `Asociacion` puede guardar atributos y operaciones propias de la relación.

COMPOSICIÓN

La composición es básicamente una extensión del concepto de asociación. Dado que ésta no tiene ningún mecanismo que la soporte en Java, la composición tampoco. Consideremos el diagrama de la figura 5.16.

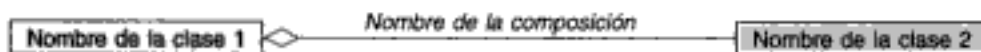


Figura 5.16 Composición entre clases.

El código para la composición se muestra a continuación:

```
class Clase1 {
    Clase2 ref;
}
class Clase2 {
    Clase1 ref;
}
```

Como se aprecia, no hay diferencia de implementación con la asociación, y todas las consideraciones descritas en la sección de ligas y asociaciones se aplican.

CLASES CONTENEDORAS

Las **clases contenedoras** son un buen ejemplo de clases que contienen a otras y que se basan en asociaciones y composición. Dos ejemplos importantes de estas clases son la **lista** o **lista ligada** (*linked list*) y la **pila** (*stack*).

LISTA

El siguiente diagrama muestra un diseño genérico de una *lista* que puede contener cualquier tipo de objeto, como se muestra en la figura 5.17.

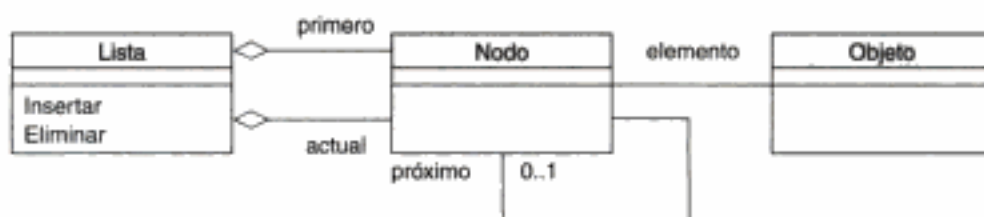


Figura 5.17 Diagrama de una lista ligada.

Se utilizan tres clases:

- **Lista.** Agrupa las operaciones de *insertar* y *eliminar* los objetos contenidos, a través de un número indefinido de nodos. Se guarda la referencia al *primero* y al *actual* (corresponde al *ultimo* elemento). Se utiliza una relación de composición para resaltar que la lista contiene nodos.
- **Nodo.** Son los contenedores para cada uno de los objetos. Cada nodo tiene una referencia al *próximo* o *siguiente* nodo (que puede ser nula), además de una referencia al propio objeto. Es muy importante esta clase ya que ofrece la funcionalidad de liga entre nodos.
- **Objeto.** Corresponde a la propia información que guarda la lista. Es importante separarla de los nodos, para no requerir ninguna funcionalidad que no sea exclusiva del objeto, a diferencia del nodo que guarda funcionalidad propia de la lista.

Aunque se utilizaran menos clases (también pudieran ser más), este diseño es muy compacto, lo que evita cualquier mezcla de la información con la lista, un requisito importante de una clase *contenedora*.

La clase **Objeto** se representa mediante la clase **Object**, la superclase de todas las clases en Java, algo que facilita el diseño y el manejo de las clases contenedoras. La clase **Nodo** se muestra a continuación,

```

class Nodo
{
    private Nodo proximo;
    private Object elemento;

    public Nodo(Object elem) {
        setElemento(elem);
    }

    public void setElemento(Object elem) {
        elemento = elem;
    }
    public Object getElemento() {
        return elemento;
    }
    public void setProximo(Nodo prox) {
        proximo = prox;
    }
    public Nodo getProximo() {
        return proximo;
    }
}
  
```

En el código anterior, todos los atributos son privados, por lo cual se agregan los métodos *get* y *set* para consultar y modificar sus valores. Se tiene un solo constructor para inicializar el nodo con el elemento respectivo. Como debe ocu-

rir con cualquier clase contenedora, la lógica del modelo debe guardarse en el agregado, o sea la clase `Lista`:

```
public class Lista {
    private Nodo primero;
    private Nodo actual;

    private Nodo getPrimero() {
        return primero;
    }
    private Nodo getActual() {
        return actual;
    }
    private Object getElemento() {
        if (actual != null)
            return actual.getElemento();
        else
            return null;
    }
    public void insertar(Object elem) {
        Nodo tmp = new Nodo(elem);
        if (actual != null) {
            tmp.setProximo(actual.getProximo());
            actual.setProximo(tmp);
        }
        if (primero == null)
            primero = tmp;
        actual = tmp;
    }
    public Object eliminar() {
        Nodo tmp = null;
        Object elem = null;
        if (primero != null) {
            tmp = primero.getProximo();
            elem = primero.getElemento();
            primero = tmp;
        }
        return elem;
    }
}
```

Nuevamente, observe la correspondencia con el diagrama. En este punto habrá que resaltar ciertos aspectos del código. No hubo necesidad de agregar un constructor, ya que los atributos se inicializan por omisión a un valor nulo. Los tres métodos `get` son privados, mientras que los únicos métodos públicos para la manipulación de la lista son `insertar` y `eliminar`, lo que es importante para manejar bien el encapsulamiento de la lista. Más aún, sólo la clase `Lista` es pública, mientras que `Nodo` es privada si se accesa desde otro paquete. En el ejemplo, se insertan elementos al final de la lista y se eliminan los del inicio. Un comentario adicional es que esta lista sólo inserta y elimina elementos. Su funcionalidad puede ser fácilmente extendida mediante operaciones, por ejemplo, para revisar o imprimir los elementos de la lista.

PILA

El siguiente diagrama muestra un diseño genérico para una *pila* que puede contener cualquier tipo de objeto, como lo ilustra la figura 5.18.

Note la similitud con el diagrama de la figura 5.17 para la *lista*. Se utilizan nuevamente tres clases:

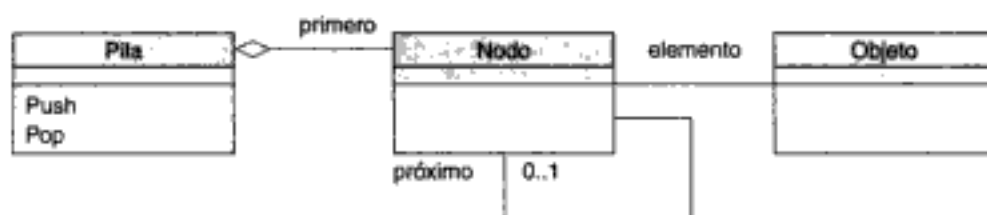


Figura 5.18 Diagrama para una pila.

- **Pila.** Agrupa las operaciones de **push** (meter) y **pop** (sacar) de los objetos contenidos a través de un número indefinido de nodos. Se guarda la referencia al *primero* únicamente y se utiliza una relación de composición para resaltar que la pila contiene nodos.
- **Nodo.** Son los contenedores para cada uno de los objetos. Cada nodo tiene una referencia al *próximo* o *siguiente* nodo (que puede ser nula), además de una referencia al propio objeto. Es similar al nodo de la lista.
- **Objeto.** Corresponde a la propia información que la pila guarda.

En el ejemplo de la pila, reutilizaremos el mismo diseño de la clase **Nodo** empleado para la lista. La clase **Objeto** es nuevamente implementada por la clase **Object**. La **Pila** se muestra a continuación:

```

public class Pila
{
    private Nodo primero;

    public void push(Object elem) {
        Nodo tmp = new Nodo(elem);
        if (primero != null)
            tmp.setProximo(primero);
        primero = tmp;
    }
    public Object pop() {
        Nodo tmp;
        Object elem;
        if (primero != null)
        {
            elem = primero.getElemento();
            tmp = primero;
            primero = tmp.getProximo();
            return elem;
        }
        return null;
    }
}

```

Nuevamente, existe correspondencia con el diagrama. Algunos aspectos del código que se deben resaltar son: a) No hubo necesidad de agregar un constructor ya que los atributos son inicializados por omisión a un valor nulo. b) El código de la clase **Pila** es más sencillo que el de la clase **Lista**. c) Se omitieron los métodos **get**, mientras que los únicos métodos públicos para la manipulación de la lista son **push** y **pop**; esto es importante para un buen manejo del encapsulamiento de la pila. d) De manera similar al ejemplo de la lista, sólo la

clase `Pila` es pública, mientras que `Nodo` es privada si se accesa desde otro paquete.

5.4.3 Generalización y herencia

La herencia es un aspecto fundamental de Java y de los lenguajes orientados a objetos. Tomemos el diagrama de herencia (sencillo) que se muestra en la figura 5.19.

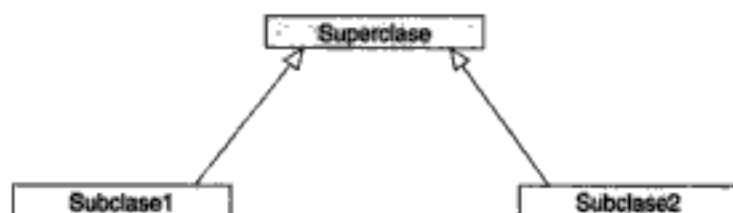


Figura 5.19 Herencia de clases.

En la figura se muestra una superclase de la cual heredan dos subclases. La herencia se codifica utilizando la palabra `extends` como se muestra a continuación:

```
class Superclase {  
}  
class Subclase1 extends Superclase {  
}  
class Subclase2 extends Superclase {  
}
```

Un comentario general sobre el esquema de herencia en Java es que de no especificarse una superclase, Java genera implícitamente una herencia a la clase `Object`. De tal manera, `Object` es la superclase, directa o indirecta, de todo el resto de las clases en una aplicación. Así, la clase `Object` es la única que no tiene una superclase.

Consideremos el siguiente ejemplo de uso de herencia, como se muestra en el diagrama de la figura 5.20.

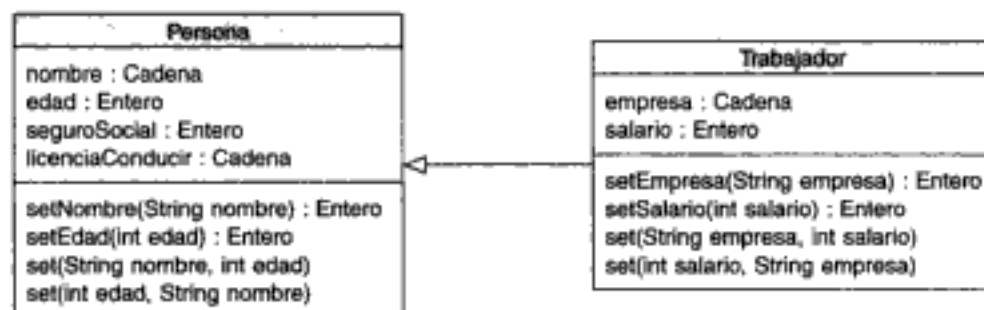


Figura 5.20 Herencia de persona a trabajador.

El código para la herencia entre `Persona` y `Trabajador` se muestra a continuación. El código de la clase `Persona` se modifica ligeramente para que sus atributos sean `protected` en lugar de `private`, de tal manera que la clase `Trabajador` pueda utilizarlos después:

```
class Persona {
    protected String nombre;
    protected int edad;
    protected int seguroSocial;
    protected String licenciaConducir;

    public Persona(String nom, int ed, int seg, String lic) {
        set(nom, ed); seguroSocial = seg; licenciaConducir = lic; }
    public Persona() {
        Persona(null, 0, 0, null); }

    public int setNombre(String nom) {
        nombre = nom; return 1; }
    public int setEdad(int ed) {
        edad = ed; return 1; }
    public void set(String nom, int ed) {
        setNombre(nom); setEdad(ed); }
    public void set(int ed, String nom) {
        setNombre(nom); setEdad(ed); }
}
```

El código para la clase `Trabajador` se muestra a continuación:

```
class Trabajador extends Persona {
    private String empresa;
    private int salario;

    public Trabajador(String emp, int sal) {
        empresa = emp; salario = sal; }
    public Trabajador() {
        this(null,0); }

    public int setEmpresa(String emp) {
        empresa = emp; return 1; }
    public int setSalario(int sal) {
        salario = sal; return 1; }
    public void set(String emp, int sal) {
        setEmpresa(emp); setSalario(sal); }
    public void set(int sal, String emp) {
        setEmpresa(emp); setSalario(sal); }
}
```

Observe la similitud entre ambas clases, aunque en este caso `Trabajador` hereda de `Persona`. La instanciación de un objeto de tipo `Trabajador` es similar a la que se hizo antes para `Persona`, aunque obviamente se cambió el nombre de la clase. A continuación hacemos dos instancias como ejemplo:

```
Trabajador t1 = new Trabajador ();
Trabajador t2 = new Trabajador ("IBM",35000);
```

Un pequeño detalle que remediamos en la siguiente sección, es que no se está asignando ningún valor a los atributos de `Persona` cuando se instancia un nuevo objeto. En otras palabras, se le asigna valores a los atributos de `Trabajador`, pero no a los de su superclase `Persona`.

Existe un modificador especial, `final`, que si se agrega como prefijo en la primera línea de la definición de una clase, hace que otras clases no puedan heredarla.

REFERENCIA A LA SUPERCLASE

En Java, existe una palabra reservada llamada `super`, que es algo similar en su uso al `this` descrito anteriormente. La palabra `super` se utiliza de dos maneras distintas: como referencia a algún campo de la superclase del objeto o como llamada a un constructor de la superclase. Veamos ambos casos:

- Como ejemplo de **referencia a un campo de la superclase**, consideremos que se define un segundo atributo `edad` dentro de la clase `Trabajador`:

```
class Trabajador extends Persona {  
    ...  
    private int edad;  
    ...  
}
```

Para acceder al atributo de la superclase, se agrega un nuevo método `setSuperEdad` dentro de la clase `Trabajador` de la siguiente forma:

```
private int setSuperEdad(int edad) {  
    super.edad = edad; return 1; }
```

Así, el método `setEdad` de la clase `Persona` modificaría el atributo `edad` de la clase `Trabajador`. Éste es un ejemplo del gran cuidado que se debe tener cuando se usa la herencia. Asimismo, es necesario hacer notar que es ilegal especificar `super.super.edad`.

- Como ejemplo de **llamada a un constructor de la superclase**, consideremos el siguiente constructor adicional para la clase `Trabajador`, que incluye parámetros para inicializar valores en los atributos heredados de `Persona`:

```
public Trabajador (String emp, int sal,  
    String nom, int ed, int seg, String lic) {  
    super(nom, ed, seg, lic);  
    set(emp, sal);  
}
```

Un nuevo objeto de tipo `Trabajador` que usa el constructor anterior sería:

```
Trabajador t3 = new Trabajador ("IBM",35000,"Juan",35,1234567,  
    "x254f");
```

Este segundo constructor agrega argumentos a los atributos de ambas clases y se aprovecha del constructor de la superclase para redirigir la llamada utilizando `super( )`. De manera análoga a `this( )`, existe la restricción de que `super( )` sólo se utiliza dentro de un constructor, y debe aparecer como su primera línea. Dada la restricción de ambas llamadas, no es posible combinarlas dentro de un mismo constructor. Por omisión, Java siempre llama al constructor vacío de la

superclase, por lo cual éste debe existir de manera explícita si hay otros constructores en la superclase, o en el caso de no haber constructores, Java genera uno de manera implícita para la superclase. La única excepción que hace Java de no llamar a `super()` de manera implícita, es cuando ya existe una llamada a `this()` en el constructor.

SOBREESCRITURA Y POLIMORFISMO

En los ejemplos de las secciones anteriores se han mostrado algunos casos de sobreescritura de atributos y métodos. A continuación describimos estos casos con mayor detalle.

Atributos

La sobreescritura de atributos (*shadowed*) corresponde a dos atributos, uno definido en la superclase y otro en la subclase, ambos con el mismo nombre. Esto es útil si desea usar en la subclase la misma variable definida con un tipo diferente y también cuando éstas son inicializadas con valores distintos. En los ejemplos anteriores se dio el caso de la sobreescritura del atributo `edad` en la clase `Trabajador`, con respecto a la ya definida en la clase `Persona`. En este caso no hay distinción de tipo ni de valor de inicialización entre ambas. Para distinguir estos atributos es necesario utilizar `this.edad` o `super.edad`, con el fin de referirse a la `edad` definida en `Trabajador` o `Persona`, respectivamente. Esto se aplica sólo si el objeto donde se encuentran las llamadas fue instanciado como `Trabajador` (realmente no importa si las llamadas están dentro de un método que pertenece a la superclase o a la subclase). Por ejemplo, el siguiente código para un objeto de tipo `Trabajador` corresponde al caso `this.edad` donde se accesa la `edad` del `Trabajador`, a pesar de que el método `setEdad` está definido dentro de la clase `Persona`:

```
private int setEdad(int ed) {  
    edad = ed; return 1; }
```

Si el objeto fue instanciado de la clase `Persona`, la situación de sobreescritura ya no existe.

Otra forma de distinguir entre ambos atributos es mediante un *cast* utilizando `this: ((Trabajador)this).edad` o `((Persona)this).edad`, donde el atributo se refiere a la clase correspondiente al *cast*.

Métodos

La *sobreescritura* es la base del *polimorfismo* en los lenguajes orientados a objetos. La **sobreescritura** se basa en definir métodos con la misma firma exacta en la superclase, al igual que la subclase (análoga al uso de *virtual* en C++). En los ejemplos de las clases `Persona` y `Trabajador` se sobreescribió el método `set`, como se puede ver a continuación para la clase `Persona`:

```
class Persona {  
    ...  
    public void set(String nom, int ed) {  
        setNombre(nom); setEdad(ed); }
```

```

    public void set(int ed, String nom) {
        setNombre(nom); setEdad(ed); }
}

```

La clase `Trabajador` sobrescribe el método `set` de la siguiente manera:

```

class Trabajador extends Persona {
    ...
    public void set(String emp, int sal) {
        setEmpresa(emp); setSalario(sal); }
    public void set(int sal, String emp) {
        setEmpresa(emp); setSalario(sal); }
}

```

La sobrescritura de los métodos anteriores se puede apreciar mejor con el siguiente ejemplo:

```

Trabajador t4 = new Trabajador ();
t4.set("Perez",50);

```

Dada la sobrescritura, se llama al método `set` de la clase `Trabajador`. Para apreciar mejor el poder de la sobrescritura y del polimorfismo, consideremos el siguiente ejemplo que consta de las clases que se muestran en el diagrama de la figura 5.21.

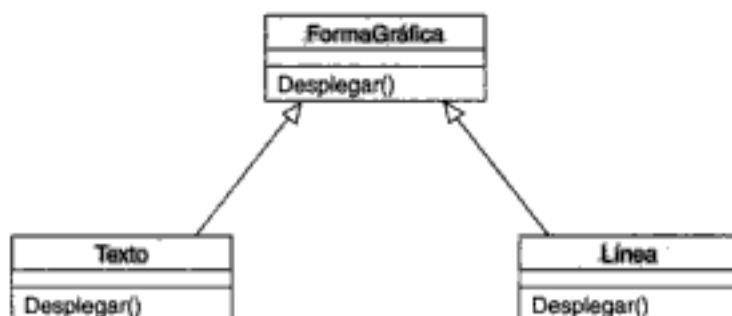


Figura 5.21 Ejemplo de polimorfismo.

A continuación definimos los aspectos esenciales de estas clases.

```

public class FormaGrafica {
    ...
    public void desplegar(int x, int y) {
    }
    ...
}
public class Texto extends FormaGrafica {
    ...
    public void desplegar(int x, int y) {
        desplegarTexto(); }
    ...
}
public class Línea extends FormaGrafica {
    ...
}

```

```

        public void desplegar(int x, int y) {
            desplegarLinea(); }
        ...
    }

```

Sin entrar en detalles, y omitiendo otros, las tres clases definen el método `desplegar` con la misma firma; aunque la superclase la tiene vacía, mientras que las dos subclases, `Texto` y `Linea`, solicitan `desplegarTexto` y `desplegarLinea`, respectivamente. Ahora, aprovechemos la lista que definimos anteriormente y escribamos el siguiente método `desplegarVentana` perteneciente a alguna otra clase:

```

    public void desplegarVentana (Lista l) {
        ...
        FormaGrafica fg;
        while ((fg = (FormaGrafica) l.siguiente()) != null) {
            fg.desplegar();
        }
    }

```

El método `desplegarVentana` tiene como argumento una `Lista l`, que para nuestro ejemplo suponemos que está llena, con objetos de tipo `Texto` o `Linea`. Como simple ejercicio, obtendremos cada uno de estos objetos de la lista (dentro del `while`) y si éste no es nulo, lo desplegaremos. Lo interesante de este ejercicio es que la variable `fg` fue declarada como `FormaGrafica`, la superclase con el método `desplegar` a ser sobrescrito. Aunque en ningún momento se menciona el tipo de las dos subclases, `Texto` y `Linea`, el despliegue es correcto, ya que Java reconoce dinámicamente el tipo verdadero (no el declarado por `fg`) del objeto en la lista y hace el llamado de acuerdo con la sobrescritura correspondiente. Esto significa que si en un futuro definimos nuevas subclases de `FormaGrafica` y sobrescribimos de manera adecuada el método `desplegar`, el método `desplegarVentana` no tendrá que ser modificado para el manejo adecuado de la nueva clase. Esto se considera máxima extensibilidad, el código nuevo no afecta en absoluto al código viejo. Cuando se logra manejar y aprovechar de forma adecuada el polimorfismo, se considera que uno domina la programación orientada a objetos.

Como ejercicio mental, consideremos qué ocurriría si el lenguaje no apoyara el polimorfismo, o sea, la sobrescritura de métodos. Dentro del método `desplegarVentana` tendríamos que revisar el tipo verdadero de cada objeto al que se refiere `fg`, posiblemente mediante múltiples expresiones `if-else` que permitan conocer su tipo y hacer la llamada adecuada de manera explícita. Esto sería muy tedioso y requeriría de modificaciones constantes para adecuarse a nuevas subclases.

Vale la pena destacar, como se vio en los ejercicios de las secciones anteriores, que se puede invocar un método sobrescrito por medio de la referencia `super`, seguido por el nombre del método.

Como comentario final a esta sección, los métodos que tengan el modificador `final` en su definición de la superclase, no pueden ser sobrescritos. Más aún, todos los métodos de una clase con el prefijo `final` también se consideran `final`. Además de no poderse sobrescribir, estos métodos `final` son más eficientes,

ya que no participan en la sobreescritura —que es un proceso dinámico de búsqueda en Java, como en la mayoría de los demás lenguajes orientados a objetos.

CLASES ABSTRACTAS

Las **clases abstractas** son un aspecto básico de la generalización, dado que definen clases que requieren subclases para poder utilizarse. De manera básica, se define una clase como abstracta mediante el modificador **abstract**. Una clase así definida puede ser instanciada, y requiere de una subclase para ser utilizada. Una clase abstracta se define de la siguiente manera:

```
abstract class NombreClase
```

Por ejemplo, podríamos modificar la definición de la clase *FormaGrafica* para volverla una clase abstracta que no pudiera instanciarse:

```
public abstract class FormaGrafica {  
    ...  
    public void desplegar(int x, int y) {  
    }  
    ...  
}
```

Fuera de la restricción de no poder ser instanciada directamente, una clase abstracta puede contener atributos y métodos como cualquier otra clase normal (concreta).

MÉTODOS ABSTRACTOS

La utilización del modificador **abstract**, como se mostró en la sección anterior, define una clase abstracta. Además, se pueden definir métodos abstractos, utilizando el modificador **abstract**, como se muestra a continuación:

```
public abstract class FormaGrafica {  
    ...  
    public abstract void desplegar(int x, int y);  
    ...  
}
```

Si el método **desplegar** de la clase *FormaGrafica* de los ejemplos anteriores fuera definido de esta manera, cualquier clase que herede de *FormaGrafica* debería, por fuerza, sobrecribir el método **desplegar**. En efecto, cualquier clase con un método abstracto, automáticamente se vuelve una clase abstracta, la cual no puede ser instanciada. Es obligatorio que la clase se defina como abstracta, si incluye algún método abstracto. El opuesto no es obligatorio. También note que al volverse abstracto el método, se elimina su implementación (que antes estaba vacía).

Como se aprecia en el ejemplo anterior, un método abstracto no tiene cuerpo, sólo una firma. Todas las subclases que hereden de esta clase tienen que sobrecribir los métodos abstractos definidos en la superclase, si no la subclase se consideraría también abstracta. (Esto es similar a una función en C++ igualada a 0 en su definición, por ejemplo, `void func( )=0.`)

INTERFACES

Como una alternativa a la definición de clases y métodos abstractos, Java ofrece otra estructura: la *interface*. Las interfaces son similares a clases abstractas, excepto que se utiliza la palabra *interface* en lugar de *abstract* y *class*. Una *interface* se define de la siguiente manera:

```
public interface NombreInterface {  
    listaMetodos  
}
```

Una *interface* sólo permite definir métodos, pero no atributos. Estos métodos son implícitamente abstractos y no pueden contener una implementación dentro de la *interface*. (Al igual que una clase, una *interface* puede incluso estar completamente vacía.) La única otra estructura que puede definirse dentro de la *interface* es una *constante estática* (*static final*), que se tratará en la siguiente sección. Consideremos la modificación de la clase *FormaGrafica* para volverse una *interface*:

```
public interface FormaGrafica {  
    ...  
    public void desplegar(int x, int y);  
    ...  
}
```

Como se ve, ya no se utiliza el modificador *abstract* dentro de la declaración del método *desplegar*. ¿Cómo se debería modificar a las clases *Texto* y *Linea* para utilizar *FormaGrafica*, si ésta se vuelve una *interface*? En lugar de utilizar la palabra *extends*, ahora se debe usar la palabra *implements*. Por tanto, la nueva definición de *Texto* y *Linea* sería la siguiente:

```
public class Texto implements FormaGrafica {  
    ...  
    public void desplegar(int x, int y) {  
        desplegarTexto();  
    }  
    ...  
}  
public class Linea implements FormaGrafica {  
    ...  
    public void desplegar(int x, int y) {  
        desplegarLinea();  
    }  
    ...  
}
```

A diferencia de que se permite un solo *extends* para la herencia de clases en Java (o sea herencia sencilla), Java permite utilizar múltiples *implements* dentro de una clase. Por ejemplo, consideremos la siguiente *interface*:

```
public interface FormaEscalable {  
    ...  
    public void escalar(double s);  
    ...  
}
```

Esta *interface* define el método *escalar* que permite a un objeto gráfico cambiar su tamaño. Las clases *Texto* y *Linea* se modificarían de la siguiente forma:

```

public class Texto implements FormaGrafica, FormaEscalable {
    ...
    public void desplegar(int x, int y) {
        desplegarTexto(); }
    public void escalar(double s) { ... }
    ...
}
public class Linea implements FormaGrafica, FormaEscalable {
    ...
    public void desplegar(int x, int y) {
        desplegarLinea(); }
    public void escalar(double s) { ... }
    ...
}

```

De esta forma, una clase puede implementar cualquier número de interfaces. También es posible que una clase herede de su superclase mediante el **extends**, y a la vez implemente a su interface a través de **implements**, donde el número de interfaces implementadas no tiene límite. Por ejemplo, volvamos a la definición original de la clase *FormaGrafica*:

```

public class FormaGrafica {
    ...
    public void desplegar(int x, int y);
    ...
}

```

Las clases *Texto* y *Linea* se podrían modificar de la siguiente forma:

```

public class Texto extends FormaGrafica implements FormaEscalable {
    ...
    public void desplegar(int x, int y) {
        desplegarTexto(); }
    public void escalar(double s) { ... }
    ...
}
public class Linea extends FormaGrafica implements FormaEscalable {
    ...
    public void desplegar(int x, int y) {
        desplegarLinea(); }
    public void escalar(double s) { ... }
    ...
}

```

Las clases *Texto* y *Linea* se consideran una instancia de ambos tipos *FormaGrafica* y *FormaEscalable*.

De manera análoga a como las clases se extienden por jerarquías a través de subclases, las interfaces pueden extenderse en subinterfaces. Una subinterface hereda los métodos abstractos y constantes estáticas de la superinterface, y puede definir nuevos métodos abstractos y constantes estáticas. Una interface es capaz de extender más de una interface a la vez. Por ejemplo, consideremos la siguiente interface que permite rotar objetos gráficos:

```

public interface FormaRotable {
    ...
    public void rotar(double r);
    ...
}

```

Ahora definamos una nueva interface `FormaTransformable`, que extiende a `FormaEscalable` y `FormaRotable`:

```
public interface FormaTransformable extends FormaEscalable, FormaRotable
{
}
```

Las clases `Texto` y `Línea` se podrían modificar de la siguiente forma:

```
public class Texto extends FormaGrafica implements FormaTransformable {
    ...
    public void desplegar(int x, int y) {
        desplegarTexto();
    }
    public void escalar(double s) { ... }
    public void rotar(double r) { ... }
    ...
}
public class Línea extends FormaGrafica
implements FormaTransformable {
    ...
    public void desplegar(int x, int y) {
        desplegarLínea();
    }
    public void escalar(double s) { ... }
    public void rotar(double r) { ... }
    ...
}
```

Este manejo de jerarquías de interfaces permite consolidar múltiples interfaces en una, para ser luego implementadas a través de una sola interface.

HERENCIA MÚLTIPLE

El tema de la **herencia múltiple** es uno de los aspectos más complejos en los lenguajes de programación orientados a objetos, debido a las dificultades de implementación de los compiladores de estos lenguajes. Los distintos lenguajes toman diferentes enfoques con respecto a la herencia múltiple. Como se discutió inicialmente en el capítulo 4, existe la problemática de heredar atributos y métodos similares de distintas superclases, lo que ocasiona el problema de resolver cuáles se van a utilizar. Por ejemplo, consideremos el diagrama de la figura 5.22, donde las `Transformable`, `Escalable` y `Rotable` se vuelven clases en lugar de interfaces, por lo cual aceptan atributos e implementación de métodos.

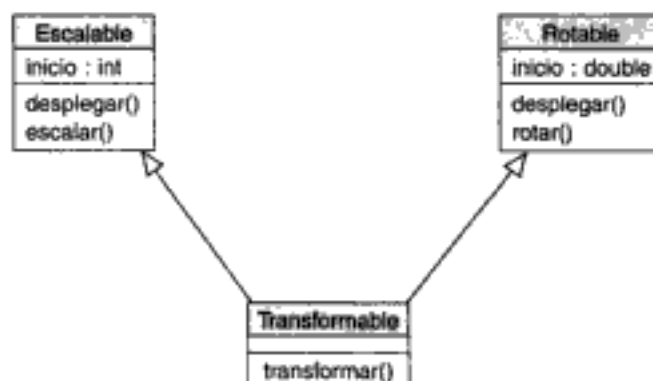


Figura 5.22 Ejemplo de herencia múltiple.

Ahora definamos el método `transformar` para la clase `Transformable`:

```
public void transformar() {  
    inicio = 4.5;  
    desplegar();  
}
```

¿A cuál atributo se refiere `inicio`: al de la clase `Escalable`, que es un `int`; o al de `Rotable`, que es un `double`? ¿A cuál `desplegar` se llama, al definido en la clase `Escalable` o al definido en la clase `Rotable`? Ésta es la base de la complejidad que ocasiona la herencia múltiple y que requiere de mecanismos adicionales bastante complejos para ser resueltos por un lenguaje de programación. Por tanto, los distintos lenguajes toman diversos enfoques; por ejemplo, C++ apoya la herencia múltiple, aunque con ciertas dificultades para el usuario (tales como conflictos con el manejo de apuntadores y referencias especiales para resolver la herencia de atributos y métodos), mientras que Smalltalk no apoya la herencia múltiple de forma directa. Por otro lado, Java toma un enfoque muy original de herencia múltiple "restringida". Java, como hemos visto, permite la herencia sencilla de clases, pero con implementación de múltiples interfaces. Si nos olvidamos de la nomenclatura especial por un momento, o sea, `interface` e `implements`, estas estructuras son simplemente clases sin atributos ni implementación de métodos; lo que ofrecen es una solución a la herencia múltiple, pero sin los conflictos de herencia de múltiples atributos e implementación de métodos de múltiples superclases. En otras palabras, Java elimina la complejidad de herencia múltiple y ofrece un mecanismo similar.

En general, como muchos lenguajes de programación orientados a objetos, tales como Java, no apoyan la herencia múltiple, es necesario en estos casos implementar la herencia múltiple a través de herencia sencilla y posiblemente agregación (*delegación*). Los siguientes tres casos describen el enfoque general:

- **Implementación de la herencia múltiple usando agregación.** Una superclase con múltiples generalizaciones individuales se redefine como un agregado, en el cual cada uno de sus componentes reemplaza una de las ramas de la generalización. Se reemplazan las posibles instancias de la herencia múltiple por un grupo de instancias que componen el agregado. La herencia de las operaciones a través del agregado no es automática, deben delegarse a los componentes apropiados. Si una subclase tiene varias superclases, todas de igual importancia, es mejor usar delegación y preservar la simetría.
- **Implementación de la herencia múltiple heredando de la clase más importante y delegando el resto.** Se toma una como subclase de la superclase más importante y se combina con un agregado correspondiendo a las generalizaciones restantes. Si se tiene una superclase principal, se implementa la herencia múltiple a través de herencia sencilla y agregación. Si el número de combinaciones es pequeño, se puede usar la generalización anidada (siguiente caso). Si el número de combinaciones, o el tamaño del código, es grande, se debe evitar este tipo de implementación.
- **Implementación de herencia múltiple usando generalización anidada.** Se crean varios niveles de generalización, se termina la jerarquía con subclases para todas las posibles combinaciones de clases unidas. En este caso no se utiliza agregación. Se preserva la herencia, pero se duplican las declaraciones, rompiendo con el espíritu de la orientación a objetos. Prime-

ro se factoriza según el criterio de herencia más importante, y luego el resto. Si una superclase tiene más características que las otras superclases, o si es el cuello de botella en el rendimiento, se debe preservar la herencia en relación con esa clase.

5.4.4 Estructuras estáticas

Existe en Java el concepto de estructuras estáticas de clases. A diferencia de los *atributos (atributos de instancia o atributos de objeto)* y *métodos (métodos de instancia o métodos de objeto)* descritos anteriormente, los cuales requieren de un objeto instanciado de la clase que los define para ser utilizados; los **atributos estáticos (atributos de clase)** y **métodos estáticos (métodos de clases)** no requieren de la existencia de los objetos, y pueden utilizarse directamente a partir de las clases que los definen. Observe que un objeto siempre puede acceder a sus campos de clase (estáticos), mientras que éstos no pueden acceder a los campos del objeto. Los campos estáticos pueden recibir todos los modificadores aplicables a los no estáticos, incluso se aplican las mismas operaciones; para ello se utiliza la palabra `static` que convierte un atributo o un método en estático, como veremos a continuación. Es importante aclarar que ni los atributos ni los métodos estáticos pueden sobrescribirse.

ATRIBUTOS

Los atributos estáticos o de clase se distinguen de los atributos de objeto en que se tiene una sola copia para todos los objetos de una clase. Por ejemplo, consideremos el siguiente atributo estático:

```
class Persona {  
    public static String nacionalidad;  
    ...  
}
```

Definamos el siguiente método (fuera de la clase `Persona`), que muestra el manejo de los atributos estáticos:

```
public void print () {  
    Persona.nacionalidad = "mexicano";  
    ...  
}
```

Como se puede ver, el acceso de la variable `nacionalidad` es por medio de la clase y no a través de un objeto. Es importante resaltar que todos los objetos instanciados de la clase `Persona` tienen acceso a una sola copia de `nacionalidad`, por lo cual cualquier cambio a su valor afectaría a todos estos objetos. Los atributos de clase se inicializan cuando la clase se carga por primera vez, a diferencia de las variables de instancia que se inicializan sólo cuando se instancian nuevos objetos.

Como se mencionó antes, los atributos estáticos aceptan todos los modificadores que los atributos normales. Así, se pueden definir atributos estáticos constantes utilizando el `static final`. Este tipo de constantes, que como todos los atributos, se declaran dentro de la definición de clase y son equivalentes al `#define` en C y C++. El compilador de Java utiliza el valor asignado a la cons-

tante para calcular inicialmente otras constantes de “tiempo de compilación”, lo cual no se puede hacer con constantes no estáticas. Además, el `static final` también se utiliza en el caso de compilación condicional. Por ejemplo:

```
public static final boolean DEBUG = false;
```

puede utilizarse en secciones `if` para que se compilen o no.

MÉTODOS

Los métodos estáticos o de clase se declaran también con `static` y se invocan con el nombre de la clase, de manera similar a los atributos de clase. Estos métodos no pueden pasar el `this` como referencia, ya que existen sin que se hayan instanciado objetos. Por ejemplo, la siguiente es una declaración de un método estático:

```
class Persona {  
    public static String nacionalidad;  
    public static String getNacionalidad() {  
        return nacionalidad;  
    }  
    ...  
}
```

Note que los métodos estáticos tienen acceso a los atributos estáticos dentro de una misma clase. Modifiquemos el método `print`, descrito en la sección anterior, que muestra el manejo de los métodos estáticos:

```
public void print () {  
    Persona.getNacionalidad();  
    ...  
}
```

Nuevamente, el acceso es mediante el nombre de la clase. Muchas bibliotecas de Java aprovechan los métodos estáticos para definir funciones que no requieren instanciación de objetos para utilizarse. Por ejemplo, todos los métodos de la clase `System` son métodos de clase, tales como `System.out.print()`; al igual que los métodos de la clase `Math`, que funcionan como una biblioteca de funciones más que como instancias de objetos.

Existe un método estático muy importante, `main`, que indica el inicio de la aplicación, como explicaremos en la sección de aplicaciones y applets.

INICIALIZADOR

Para mantener el máximo posible de similitud con el manejo de clases “normales”, existe un inicializador análogo al constructor, que permite inicializar los aspectos estáticos de la clase (no de las instancias). No tiene argumentos ya que automáticamente se carga cuando la clase se carga. Un inicializador estático de clase tiene el siguiente formato:

```
static {  
    ...  
}
```


A diferencia de los constructores, no tiene nombre ni se pasa argumentos. Java permite múltiples bloques estáticos como el anterior, los cuales se llaman todos al cargar la clase. Una de las aplicaciones es cargar métodos nativos de la máquina virtual, típicamente en C.

5.4.5 Metaclases

Existe en Java el concepto de **metaclases**, o sea clases de clases. Si un *objeto* es la instancia de una clase, entonces la propia *clase* es la instancia de una *metaclass*. Este concepto se muestra en el diagrama de la figura 5.23.



Figura 5.23 Concepto de metaclass.

El concepto de metaclass es de gran utilidad, en particular porque permite tratar a una clase como si fuera un objeto. Por ejemplo, la figura 5.24 muestra las relaciones anteriores utilizando la metaclass `Class`, y relaciones con cualquier clase y objeto.



Figura 5.24 Concepto de metaclass en Java.

Por ejemplo, veamos el siguiente código:

```
Class miclase = Class.forName(nombre_clase);  
Object miobjeto = miclase.newInstance();
```

En la primera línea se traduce el `nombre_clase` (definido como `String`) a una clase `miclase`. Observe que esta variable se refiere a una clase manipulada como objeto. En la segunda línea se instancia `miobjeto` a partir de `miclase`. Note que este proceso se puede aplicar a *cualquier* clase en Java, dado que la instancia-ción se realiza de manera totalmente anónima, todo gracias a la manipulación de la clase como si fuese un objeto. De hecho, éste también es un ejemplo de polimorfismo (a través del método `newInstance`).

5.5 Aplicaciones y *applets*

Existen dos maneras de estructurar un programa en Java: por aplicaciones o por *applets*. Ambos siguen el mismo proceso de desarrollo, incluyendo la gran mayoría de las facilidades que Java ofrece. La diferencia es que las aplicaciones se ejecutan como cualquier programa "normal", mientras que los *applets* están específicamente diseñados para correr en el Web a través de un *browser*.

5.5.1 Aplicaciones

Las aplicaciones requieren de un método especial para iniciar el programa: el método `main`. La aplicación más sencilla es la famosa "Hola Mundo", la cual se programa de la siguiente manera como aplicación en Java:

```
class ej {  
    public static void main(String args[]) {  
        System.out.println("Hola Mundo!");  
    }  
}
```

Al ejecutar el programa escribiría "Hola Mundo".

En una aplicación que contiene múltiples clases, el método `main` puede agregarse a cualquiera de las clases. Dado que el método `main` está definido como *estático*, éste no tiene acceso a las estructuras internas de la clase. A continuación se muestra un ejemplo del uso del `main`,

```
public class Persona {  
    public static void main(String args[]) {  
        for (int i = 0; i < args.length; i++)  
            System.out.print(args[i] + " ");  
        System.out.print("\n");  
        System.exit(0);  
    }  
}
```

Observe el argumento `args` de `main`, que recuerda cierta similitud con `argc` y `argv` en el lenguaje C, sólo que integrándolos en uno solo.

5.5.2 Applets

Tomamos ahora el programa anterior de "Hola Mundo", que se programaría de la siguiente manera como un applet en Java:

```
public class ej extends Applet {  
    public void paint(Graphics g){  
        g.drawString("Hola Mundo!",25,25);  
    }  
}
```

En lugar del método `main`, un applet requiere una clase que herede de `Applet` y sobrescriba el método `paint`, para desplegar textos o gráficas en la pantalla. Todo applet requiere de una página html para su ejecución, por ejemplo `ej.html`, como se muestra a continuación:

```
<applet code=ej.class width=200 height=200></applet>
```

El archivo html puede ejecutarse en un browser o mediante la aplicación `appletviewer` de la siguiente forma:

```
appletviewer ej.html
```

A diferencia del método `main`, el paso de argumentos iniciales es a través de los parámetros del archivo html.

5.6 Interfaces gráficas del usuario

Programar en Java sin utilizar **interfaces gráficas del usuario** (GUI, por sus siglas en inglés) es no aprovechar uno de los aspectos más importantes que ofrece la programación y, en particular, Java.

A continuación, se describe el manejo de ventanas, textos, botones y paneles en Java a través de la biblioteca básica *Abstract Window Toolkit* (AWT). El ejemplo a desarrollarse en esta sección es muy importante, ya que servirá de base para el desarrollo de software generado en los capítulos posteriores en el libro.

Todo sistema de ventanas requiere alguna ventana donde desplegar la información. Existen dos filosofías separadas, aunque relacionadas en Java, que afectarán el diseño como ya se mencionó antes: aplicaciones y applets. Las aplicaciones requieren de un *Frame* (*marco*) donde desplegar y permitir la interacción con el usuario; mientras que un *applet* puede hacer esto en forma directa en la pantalla del *brouser* o mediante nuevos marcos, de manera similar a las aplicaciones. Empecemos por lo más básico y mostremos el uso de los marcos a través de la clase *InterfaceUsuario*:

```
class InterfaceUsuario extends Frame
```

Esta clase heredará todas las características de un marco para manejarse como una ventana. Antes de proseguir, es necesario comprender que toda aplicación con ventanas requiere un manejo de eventos de entradas y salidas. Esto significa que la aplicación, antes de hacer algo, debe especificar cómo controlará eventos generados por el teclado y, en especial, el ratón. Java define dos interfaces muy importantes en AWT, que son *WindowListener* y *ActionListener*. *WindowListener* define los métodos relacionados con el manejo de eventos para una ventana, como abrir y cerrar. Y *ActionListener* define los métodos para manejar eventos dentro de la ventana, como apretar un botón. De tal manera, es necesario que la clase *InterfaceUsuario* implemente estas dos interfaces si se ha planeado manejar estos tipos de eventos. Por tanto, extendemos la definición de la clase *InterfaceUsuario* de la siguiente manera:

```
class InterfaceUsuario extends Frame implements WindowListener,  
    ActionListener
```

Dado que las interfaces deben tener sus métodos sobrescritos, primero hagamos esto con los siguientes métodos de *WindowListener*:

```
public void windowClosed(WindowEvent event) {}  
public void windowDeiconified(WindowEvent event) {}  
public void windowIconified(WindowEvent event) {}  
public void windowActivated(WindowEvent event) {}  
public void windowDeactivated(WindowEvent event) {}  
public void windowOpened(WindowEvent event) {}  
public void windowClosing(WindowEvent event) {  
    System.exit(0); }
```

Éstos son todos los métodos definidos por la interface `WindowListener`:

- ▶ `windowClosed` para el manejo de eventos a partir de una ventana cerrada.
- ▶ `windowDeiconified` para el manejo de eventos a partir de una ventana no iconificada.
- ▶ `windowIconified` para el manejo de eventos a partir de una ventana iconificada.
- ▶ `windowActivated` para el manejo de eventos a partir de una ventana activada.
- ▶ `windowDeactivated` para el manejo de eventos a partir de una ventana desactivada.
- ▶ `windowOpened` para el manejo de eventos a partir de una ventana abierta.
- ▶ `windowClosing` para el manejo de eventos en el momento que se cierra una ventana.

Estos métodos deben sobrescribirse aunque queden vacíos. El único que realmente se ha sobrescrito es `windowClosing` para permitir salir de la aplicación cuando este evento ocurra. (Existen alternativas para sobrescribir únicamente los métodos deseados utilizando *adaptadores* en lugar de *interfaces*, pero este tema no se abordará en este libro.)

La sobrescritura de la interface `ActionListener` es mucho más importante para las ventanas, ya que a través del método `actionPerformed` se debe especificar qué hacer cuando, por ejemplo, se presiona algún botón dentro de la ventana. El siguiente método muestra la sobrescritura de `actionPerformed`, al imprimir el evento ocurrido:

```
public void actionPerformed(ActionEvent event) {  
    System.out.println("Action: "+event.getActionCommand());  
}
```

Ésta es la lógica básica del manejo de eventos de un marco. Para completar la funcionalidad necesitamos inicializar la clase `InterfaceUsuario` y definir alguna pantalla a desplegar. Para ello definimos el siguiente constructor:

```
public InterfaceUsuario() {  
    setSize(800,600);  
    setBackground(Color.lightGray);  
    addWindowListener(this);  
    pantalla = new PantallaPrincipal(this);  
    desplegarPantalla(pantalla);  
}
```

Describamos las llamadas dentro de este constructor, la mayoría de las cuales son opcionales:

- ▶ `setSize` define el tamaño del marco.
- ▶ `setBackground` asigna un color de fondo de manera opcional.
- ▶ `addWindowListener` registra el marco (`this`) con el administrador de ventanas de Java para que podamos manejar los eventos. Este método es necesario.
- ▶ `PantallaPrincipal` instancia la pantalla a ser desplegada, algo que veremos a continuación. Observe que la variable `pantalla` se define como un atributo.

buto de la clase `InterfaceUsuario`, "`private Pantalla pantalla;`", esto también se explicará junto con la clase `Pantalla`.

► `desplegarPantalla` despliega la pantalla recién instanciada.

Antes de mostrar los detalles de `PantallaPrincipal`, se estudiará cómo se despliega una pantalla mediante el método `show`, definido por la clase `Frame`:

```
protected void desplegarPantalla(Pantalla p) {
    show();
}
```

Es de notar que aún no se ha utilizado el argumento de tipo `Pantalla`. Esto se remediará cuando se aclare la lógica que se aplicará mediante la explicación de la clase `PantallaPrincipal`. Antes de hacer eso, mostraremos el método `main` para instanciar la clase `InterfaceUsuario`:

```
public static void main(String[] args) {
    System.out.println("Starting System...");
    InterfaceUsuario iu = new InterfaceUsuario();
}
```

Si se quisiera instanciar el marco bajo control de un applet, lo cual también es aceptable, se definiría la siguiente clase `InterfaceUsuarioApplet`, que hereda de la clase `Applet`, y se sobrescribiría el método `init`, en lugar del método `main`:

```
public class InterfaceUsuarioApplet extends Applet {
    public void init() {
        showStatus("Starting System...");
        InterfaceUsuario iu = new InterfaceUsuario();
    }
}
```

También hay que señalar que definiremos la gran mayoría de los métodos como `protected`, dado que se accesan dentro de un mismo paquete. Sólo los métodos sobrescritos de las interfaces deben definirse como públicos. Asimismo, se definirán los constructores como públicos, aunque algunos pudieran también establecerse como `protected` si son los objetos instanciados dentro del mismo paquete. Como ejemplo vamos a crear un `PantallaPrincipal` que genere el despliegue que se muestra en la figura 5.25.

Para simplificar el diseño de una de estas pantallas, se dividen en secciones lógicas, llamadas **paneles**, que visualmente no afectan la presentación de la pantalla, pero son una buena guía para el diseñador. En nuestro caso dividiremos la pantalla en paneles horizontales que contengan los diversos elementos de la pantalla, o sea, textos y botones.

Dado que en general se despliegan múltiples pantallas, definamos una superclase `Pantalla`, como algunos lectores ya se habrán imaginado, de la siguiente manera:

```
class Pantalla {
    protected InterfaceUsuario interfaceUsuario;
    protected Panel panel;
```



Figura 5.25 Ejemplo de marco desplegando `PantallaPrincipal`.

```
protected Button boton;
protected Vector paneles, botones;
}
```

Inicialmente definimos un atributo de tipo `InterfaceUsuario` para guardar la referencia a la clase encargada de administrar el despliegue. Los atributos de tipo `Panel` y `Button` son referencias que se utilizarán cuando se estén manipulando paneles o botones de manera temporal. El aspecto más importante de esta clase es que se definirán dos arreglos de tamaño dinámico, mediante la clase `Vector`, con el fin de guardar la lista de los paneles y botones que se instancien en las diversas pantallas, ya que éstos requieren un manejo especial, y los arreglos facilitarán su manipulación como veremos más adelante.

Se define el constructor para la clase `Pantalla` de manera que reciba una referencia a guardarse de la clase `InterfaceUsuario`.

```
public Pantalla (InterfaceUsuario ui) {
    interfaceUsuario ui;
    inicializarPantalla();
    crearPantalla();
}
```

Además, este constructor inicializará la pantalla mediante la llamada `crearPantalla`, que está sobreescribida por las pantallas particulares. En la superclase, el método se puede definir como abstracto y protegido.

```
protected abstract void crearPantalla ();
```

Este método es el corazón de la lógica de despliegue. Pero antes se reinician los vectores para los paneles y botones que están definidos por el método `inicializarPantalla`.

```
protected void inicializarPantalla() {  
    paneles = new Vector();  
    botones = new Vector();  
}
```

Definamos ahora la clase `PantallaPrincipal` como subclase de `Pantalla`:

```
class PantallaPrincipal extends Pantalla
```

El constructor de `PantallaPrincipal` llamará al constructor de la superclase mediante la llamada `super`, a la cual pasará el parámetro `ui` de tipo `InterfaceUsuario`.

```
public PantallaPrincipal (InterfaceUsuario ui) {  
    super(ui);  
}
```

El diseño de las pantallas se hará utilizando paneles, o sea, secciones de la pantalla. Esto se guardará dentro del método `crearPantalla`, en este caso de la clase `PantallaPrincipal`, que se vio en la figura 5.25. Esta clase no tiene que ser pública, ya que se llama dentro del constructor de la superclase. Dado que existe una sobrescritura del método, éste debe definirse como protegido.

```
protected void crearPantalla ()
```

El primer panel contiene el título de la pantalla, como se muestra a continuación:

```
panel = new Panel();  
panel.setLayout(new GridLayout(2,1));  
panel.add(new Label("SISTEMA DE RESERVACIONES DE VUELO",  
    Label.CENTER));  
panel.add(new Label("Pantalla Principal (P-1)",  
    Label.CENTER));  
paneles.addElement(panel);
```

Luego de instanciar el objeto de tipo `Panel`, se le asigna a un administrador para realizar la organización interna del panel, por ejemplo `GridLayout`, que define a una cuadrícula de 2×1 . Luego añadimos los elementos del panel, en este caso un título (`Label`) correspondiente a un texto que no es modificable, el cual se centra dentro de la cuadrícula. Una vez completado el panel, lo agregamos a la lista de `paneles`.

El panel que sigue contiene cuatro filas y una sola columna, donde se insertan cuatro líneas de texto como se ve a continuación:

```
panel = new Panel();  
panel.setLayout(new GridLayout(4,1));  
panel.add(new Label("Servicios Ofrecidos:", Label.CENTER));  
panel.add(new Label("* Consulta de Vuelos, Tarifas y Horarios",  
    Label.CENTER));  
panel.add(new Label("* Reserva de Vuelos", Label.CENTER));
```



```
panel.add(new Label("• Compra de Boletos", Label.CENTER));
paneles.addElement(panel);
```

De nuevo se agrega el panel a la lista de paneles. El panel que viene es similar al primero y agrega una etiqueta, como se ve a continuación:

```
panel = new Panel();
panel.setLayout(new GridLayout(1,1));
panel.add(new Label("Para registrarse por primera vez oprima:",
    Label.CENTER));
paneles.addElement(panel);
```

En el siguiente panel se hace algo diferente. Se instancia un botón (Button), al cual le ponemos como etiqueta "Registrarse por Primera Vez", como se ve a continuación:

```
panel = new Panel();
panel.setLayout(new GridLayout(1,1));
boton = new Button ("Registrarse por Primera Vez");
botones.addElement(boton);
panel.add(boton);
paneles.addElement(panel);
```

Además de agregar el panel a la lista de paneles, se añade el botón a la lista de botones. El próximo panel es similar al primero y agrega una etiqueta, como se ve a continuación:

```
panel = new Panel();
panel.setLayout(new GridLayout(1,1));
panel.add(new Label("Para acceder todos los servicios de vuelo
    (consulta, reserva, compra) o modificar su registro,
    oprima", Label.CENTER));
paneles.addElement(panel);
```

En este panel se instancia un campo de texto (TextField), además se le agrega una etiqueta Login:, como se ve a continuación:

```
panel = new Panel();
panel.setLayout(new GridLayout(1,1));
panel.add(new Label("Login:", Label.LEFT));
panel.add(new TextField(20));
paneles.addElement(panel);
```

Para el penúltimo panel instanciamos un campo de texto adicional (TextField), al que, además, se le agrega una etiqueta Password:, como se describe a continuación:

```
panel = new Panel();
panel.setLayout(new GridLayout(1,1));
panel.add(new Label("Password:"));
panel.add(new TextField(20));
paneles.addElement(panel);
```

En el último panel instanciamos dos botones adicionales, OK y Salir:

```
panel = new Panel();
panel.setLayout(new GridLayout(1,1));
```

```

        boton = new Button("OK");
        botones.addElement(boton);
        panel.add(boton);
        boton = new Button("Salir");
        botones.addElement(boton);
        panel.add(boton);
        paneles.addElement(panel);

```

Ahora, ¿cómo desplegamos todo esto y manejamos de manera adecuada los eventos relacionados? Regresamos al método `desplegarPantalla`, inicialmente definido para la clase `InterfaceUsuario`, y se le agregan algunas líneas adicionales antes del `show`:

```

protected void desplegarPantalla(Pantalla p) {
    if (pantalla != null)
        pantalla.borrarPantalla();
    if (p != null)
        pantalla = p;
    if (pantalla != null)
        pantalla.desplegarPantalla();
    show();
}

```

Dado que estamos en proceso de desplegar una nueva pantalla, lo primero que debemos hacer es borrar la anterior, tanto paneles como registro de botones. Observe que se guarda en `p` la nueva ventana, mientras que se borra la pantalla anterior. Eso se hace a través del método `borrarPantalla` dentro de la clase `Pantalla`, lo que describiremos en un momento. Utilizamos siempre un `if` para asegurar que no existan valores nulos. Luego de borrar la pantalla actual, cambiamos el valor del atributo `pantalla` al de la nueva pantalla, la cual será desplegada mediante el método `desplegarPantalla`. El método `borrarPantalla` se muestra a continuación, y puede definirse como `protected` para ser llamado por cualquier subclase dentro o fuera del paquete:

```

protected void borrarPantalla() {
    interfaceUsuario.removeAll();
    int bs = botones.size();
    for (int i = 0; i < bs; i++)
        if ((boton = (Button)botones.elementAt(i)) != null)
            boton.removeActionListener(interfaceUsuario);
}

```

El método `removeAll` borra todo lo que hay en la pantalla, mientras que `removeActionListener` borra el registro para manejo de eventos de los botones de la pantalla anterior.

El despliegue de la pantalla se hace mediante el método `desplegarPantalla`, perteneciente a la clase `Pantalla`, como se ve a continuación:

```

protected void desplegarPantalla() {
    System.out.println("Desplegando: " + this);
    int ps = paneles.size();
    interfaceUsuario.setLayout(new GridLayout(ps,1));
    for (int i = 0; i < ps; i++)
        interfaceUsuario.add((Panel)paneles.elementAt(i));
    int bs = botones.size();
    for (int i = 0; i < bs; i++)

```

```

        if ((boton = (Button)botones.elementAt(i)) != null)
            boton.addActionListener(interfaceUsuario);
    }

```

Se obtiene el número de paneles, *ps*, de la lista de paneles, para lo cual se le solicita a la *interfaceUsuario* que organice la pantalla en una cuadrícula mediante *GridLayout*, que conste en un número de filas *ps* y una sola columna. De tal manera, se agrega a la *interfaceUsuario* cada uno de los paneles, mediante la llamada *interfaceUsuario.add*. Con esto es suficiente para desplegar los paneles junto con todos los campos definidos en cada uno anteriormente, en el momento se ejecuta la llamada *show*. Sin embargo, falta registrar los botones con el manejador de eventos. Para hacerlo obtenemos el número de botones *bs*. Luego, extraemos cada uno de la lista y lo registramos en el sistema mediante la llamada *boton.addActionListener*. De esta manera, ya podemos desplegar la *PantallaPrincipal* con todos sus elementos, incluyendo botones registrados, para luego saber cuál de ellos fue oprimido.

Después de desplegar la pantalla anterior viene el siguiente paso: el usuario puede llenar los campos de texto, que no generan ningún evento, y desplegar alguno de los tres botones. ¿Qué hace el sistema en el momento que se oprime alguno de estos botones? Recordemos el método *actionPerformed* de la clase *InterfaceUsuario*, definida precisamente para ello. Le agregamos una nueva línea que será la encargada de manejar el evento llamando a *manejarEventos* a partir de cada pantalla desplegada:

```

public void actionPerformed(ActionEvent event) {
    System.out.println("Action: "+event.getActionCommand());
    pantalla.manejarEvento(event.getActionCommand());
}

```

Para que esto funcione, primero debemos definir el método *manejarEvento* dentro de la clase *Pantalla* (recuerden la explicación de *polimorfismo*) y lo hacemos de manera abstracta:

```
protected abstract Pantalla manejarEvento(String str);
```

Luego sobrescribimos el método *manejarEvento* dentro de la clase *PantallaPrincipal*, de manera que según se oprima un botón, la pantalla instanciará la siguiente a desplegarse. Esto se hace comparando el nombre del botón presionado contra las distintas posibilidades que se destacan en los siguientes *if*:

```

protected Pantalla manejarEvento(String str) {
    if (str.equals("Registrarse por Primera Vez")) {
        if (pantallaCrearRegUsuario == null)
            pantallaCrearRegUsuario = new PantallaCrearRegUsuario(this);
        return pantallaCrearRegUsuario;
    }
    else if (str.equals("OK")) {
        if (pantallaServicio == null)
            pantallaServicio = new PantallaServicio(this);
        return pantallaServicio;
    }
    else if (str.equals("Salir")) {
        System.exit(0);
    }
    else

```

```

        System.out.println("Error en PantallaPrincipal: "+str);
        return this;
    }

```

Por ejemplo, si el usuario presiona el botón **Registrarse por Primera Vez** (note que la comparación de cadenas se hace mediante el método `equals`), entonces se instancia una pantalla de tipo `PantallaCrearRegUsuario`, que explicaremos en un momento. Si es un **OK**, se solicita la pantalla `PantallaServicio`; y si es **Salir**, simplemente se sale del programa. Antes de definir nuestra siguiente pantalla, veamos qué ocurre con la lógica principal. La llamada `manejarEvento` fue hecha desde `actionPerformed` en la clase `InterfaceUsuario`. Extendamos este método con una nueva versión de la siguiente manera:

```

public void actionPerformed(ActionEvent event) {
    System.out.println("Action: "+event.getActionCommand());
    Pantalla p = pantalla.manejarEvento(event.getActionCommand());
    desplegarPantalla(p);
}

```

La siguiente llamada es `desplegarPantalla` y el ciclo se completa. En otras palabras volvimos al inicio de nuestra lógica. La pantalla `PantallaCrearRegUsuario` se muestra en la figura 5.26.

Figura 5.26 Ejemplo de marco desplegando `PantallaCrearRegUsuario`.

La figura 5.27 ilustra las clases principales, junto con sus atributos y métodos, para el ejemplo de las pantallas mostrado aquí. Es importante señalar que en

el diagrama se muestra la notación de implementar para las interfaces como una herencia "punteada" a partir de clases abstractas.

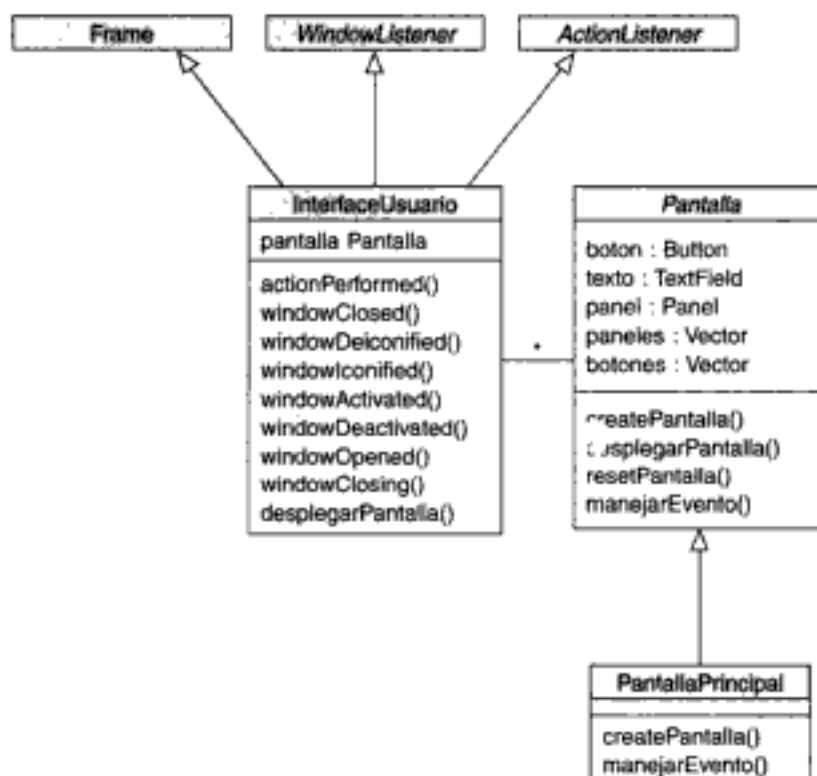


Figura 5.27 Diagrama de clases para el ejemplo de las pantallas.

Como veremos en el siguiente capítulo, estas pantallas y lógica nos servirá para crear el prototipo inicial del sistema de reservaciones de vuelos que utilizaremos a lo largo del libro.

RESUMEN

De manera introductoria, en este capítulo se presenta la programación en Java, de la cual se describen sus características principales, así como sus esquemas de procesamiento, compilación y ejecución, y sus bibliotecas. Se muestra cómo programar en Java los conceptos de modelado representados en UML descritos en el capítulo 4. Se dan ejemplos de despliegues gráficos y acceso a base de datos y archivos, aspectos que servirán más adelante durante el desarrollo de un sistema de software como caso práctico. Se describen aspectos avanzados de la programación en Java como es el caso de las metaclasses. Además, se proporcionan múltiples ejemplos a todo lo largo de él.

REFERENCIAS

1. <http://java.sun.com>
2. <http://www.uml.org>, Booch G., Rumbaugh, J., Jacobson, I., 1998, The Unified Modeling Language User Guide, Addison-Wesley.

Desarrollo de software orientado a objetos

En esta tercera parte del libro se describirán las actividades más importantes relacionadas con el desarrollo de software: *requisitos* (capítulo 6), *análisis* (capítulo 7), *diseño* (capítulo 8), *implementación* (capítulo 9) y *pruebas* (capítulo 10).

Hidden page

Modelo de requisitos

El **modelo de requisitos** tiene como objetivo delimitar el sistema y capturar la funcionalidad que ofrecerá desde la perspectiva del usuario. Este modelo puede trabajar como un contrato entre el desarrollador y el cliente, o usuario del sistema, por lo que deberá proyectar lo que el cliente desea según la percepción del desarrollador. Por ello, es esencial que los clientes lo comprendan.

El modelo de requisitos es el primero en desarrollarse, y es la base para formar todos los demás modelos en el desarrollo de software. En general, cualquier cambio en la funcionalidad del sistema es más fácil de hacer, y con menores consecuencias a este nivel que posteriormente. El modelo de requisitos que se desarrollará se basa en la metodología *Objectory* (Jacobson *et al.* 1992), que tiene su fundamento en el modelo de *casos de uso*. Actualmente esta metodología es parte del *Proceso unificado racional* (RUP) [Rational Unified Process].¹ El modelo de casos de uso y el propio modelo de requisitos son la base para los demás modelos. A continuación se repasarán los conceptos detallados en el capítulo 3, que nos servirán en esta sección:

- ▶ **Requisitos:** El modelo de casos de uso sirve para expresar el modelo de requisitos, el cual se desarrolla en cooperación con otros modelos como se verá más adelante.
- ▶ **Análisis:** La funcionalidad especificada por el modelo de casos de uso se estructura en el modelo de análisis, que es estable con respecto a cambios, lo que lo hace un modelo lógico independiente del ambiente de implementación.
- ▶ **Diseño:** La funcionalidad de los casos de uso, ya estructurada por el análisis, la realiza el modelo de diseño, adaptándose al ambiente de implementación real y refinándose aún más.
- ▶ **Implementación:** Los casos de uso se instrumentan mediante el código fuente en el modelo de implementación.
- ▶ **Pruebas:** Los casos de uso se comprueban a través de las pruebas de componentes y de integración.
- ▶ **Documentación:** El modelo de casos de uso se debe registrar a lo largo de las diversas actividades, dando lugar a distintos documentos como los manuales de usuario, de administración, etcétera.

El diagrama de la figura 6.1 ilustra los distintos modelos, los detalles y la notación que se describirán más adelante.

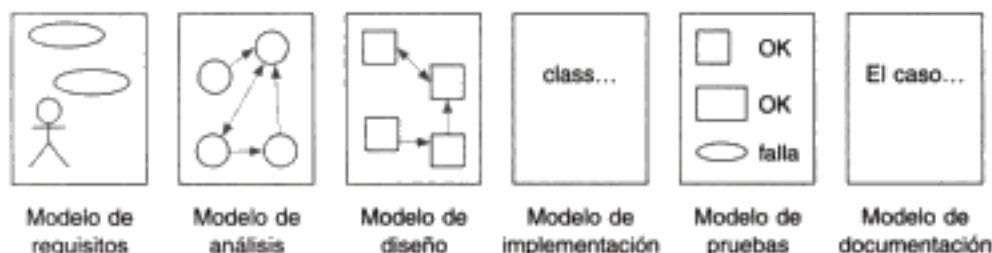


Figura 6.1 Dependencia de los distintos modelos del proceso de software del modelo de casos de uso.

El propósito del modelo de requisitos es comprender en su totalidad el problema y sus implicaciones. Los demás modelos, análisis, diseño, implementación y pruebas dependen directa o indirectamente del modelo de requisitos. Asimismo, este modelo sirve de base para el desarrollo de las instrucciones operacionales y los manuales, ya que todo lo que el sistema deba hacer se describe aquí desde la perspectiva del usuario. El modelo de requisitos no es un proceso mecánico, el analista debe interactuar constantemente con el cliente para completar la información faltante, y así resolver ambigüedades e inconsistencias. También debe separar los requisitos verdaderos de las decisiones relacionadas con el diseño e implementación. Se deben indicar cuáles aspectos son obligatorios y cuáles opcionales para evitar que se limite la flexibilidad de la implementación. Durante el diseño, se debe extender el modelo de requisitos con las especificaciones de rendimiento y los protocolos de interacción para los sistemas externos, al igual que las provisiones sobre modularidad y futuras extensiones. Incluso, en ciertas ocasiones se pueden incluir aspectos de diseño, como el uso de lenguajes de programación particulares.

En la metodología de *Objectory*, el modelo de requisitos consta de tres modelos principales, visualmente representado por un diagrama de tres dimensiones como se muestra en la figura 6.2.

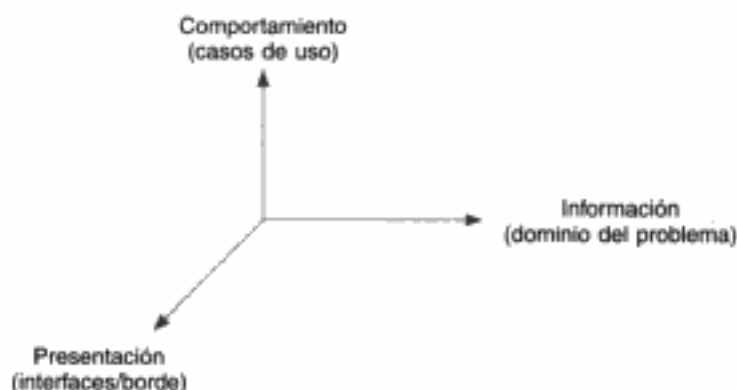


Figura 6.2 Los tres ejes de modelado del modelo de requisitos.

- El modelo de *comportamiento*, basado directamente del modelo de *casos de uso*, especifica la funcionalidad que ofrece el sistema desde el punto de vista del usuario. Este modelo utiliza dos conceptos claves: *actores* para re-

presentar los distintos papeles que los usuarios desempeñan con el sistema y *casos de uso* para significar qué pueden hacer los actores con respecto al sistema.

- El modelo de *presentación* o modelo de *interfaces* (o *borde*) especifica cómo interactúa el sistema con actores externos al ejecutar los casos de uso. En particular, en los sistemas de información que tienen mucho contacto con el usuario, especifica cómo se verán visualmente las interfaces gráficas, y qué funcionalidad ofrecerá cada una.
- El modelo de *información* o modelo del *dominio del problema* especifica los aspectos estructurales de la aplicación en términos de objetos. Este modelo permite identificar rápidamente cuáles objetos podrán ser guardados en una base de datos, si ése fuera el caso. Además, es utilizado para guardar información temporal en la aplicación, y eventualmente servirá como parámetro de llamadas entre funciones en el sistema.

Es importante resaltar que esta separación en tres ejes de modelado independientes sirve para una mayor estabilidad en el desarrollo del sistema, lo que permite minimizar los efectos de cada uno sobre los otros dos.

Para ilustrar el modelo de requisitos y el desarrollo de los modelos posteriores, se utilizará el ejemplo del "Sistema de reservaciones de vuelo" como se mencionó antes. Para ello, mostraremos inicialmente una *descripción del problema*. A partir de esta descripción inicial se describirán los tres modelos básicos del modelo de requisitos.

6.1 Descripción del problema

La **descripción del problema** es un resumen preliminar de necesidades que sirve como punto de partida para comprender los requisitos del sistema. Aquí se trata de simular una descripción preparada por un cliente, la cual debe evolucionar por medio del modelo de requisitos, con objeto de lograr la especificación final del sistema a desarrollarse. La descripción del problema debe ser una especificación de necesidades y no una propuesta de solución. La descripción inicial puede ser incompleta e informal, pues al realizarse sin un análisis completo, no hay ninguna razón para esperar que sea correcta.

Como ejemplo se desarrollará un *Sistema de reservaciones de vuelos*, el cual permitirá al usuario hacer consultas y reservaciones de vuelos; además de comprar los boletos aéreos de forma remota, sin la necesidad de recurrir a un agente de viajes. En la actualidad, existen múltiples sistemas de reservaciones de vuelos que utilizan las agencias de viajes para dar servicio a los clientes, entre éstos los cuatro más importantes son: *Sabre*², *Galileo*³, *Worldspan*⁴ y *Amadeus*⁵. Estos sistemas son conocidos como sistemas de distribución global. También existen sistemas de reservaciones de vuelo por Internet, como *Travelocity*⁶ y *Expedia*⁷, entre otras, los cuales se basan en su mayoría, de manera directa o indirecta, en los sistemas de distribución global anteriores.

La descripción del problema para nuestro sistema de reservaciones de vuelos es la siguiente:

El sistema de reservaciones de vuelos, permite al usuario hacer consultas y reservaciones de vuelos, además de poder comprar los boletos aéreos de forma remota, sin la necesidad de recurrir a un agente de viajes. Se desea que el sistema de reservaciones sea accesible a través de Internet (World Wide Web).

El sistema presenta en su pantalla principal un mensaje de bienvenida describiendo los servicios ofrecidos junto con la opción para registrarse por primera vez, o si ya se está registrado, poder utilizar el sistema de reservaciones de vuelos. Este acceso se da por medio de la inserción de un *login* previamente especificado y un *password* ya escogido y que debe validarse.

Una vez registrado el usuario, y después de haberse validado el registro y contraseña del usuario, se pueden seleccionar las siguientes actividades:

- Consulta de vuelos*
- Reservación de vuelos*
- Pago de boletos*

La consulta de vuelos se puede hacer de tres maneras diferentes:

- Horarios de vuelos*
- Tarifas de vuelos*
- Estado del vuelo*

La consulta según horarios muestra los horarios de las diferentes aerolíneas que dan servicio entre dos ciudades.

La consulta según tarifas muestra los diferentes vuelos entre dos ciudades que dan prioridad a su costo.

La información de vuelo se utiliza principalmente para consultar el estado de algún vuelo, incluyendo información de disponibilidad de asientos y, en el caso de un vuelo para el mismo día, si está a tiempo.

Se pueden incluir preferencias en las búsquedas, como fecha y horario deseado, categoría de asiento, aerolínea y si se desea, sólo vuelos directos.

La reservación de vuelo permite al cliente hacer una reservación para un vuelo particular, especificando la fecha y horario, bajo una tarifa establecida. Es posible reservar un itinerario compuesto de múltiples vuelos, para uno o más pasajeros, además de poder reservar asientos.

El pago permite al cliente, dada una reservación de vuelo previa y una tarjeta de crédito válida, adquirir los boletos aéreos.

(continúa)

(continuación)

Los boletos serán enviados al cliente posteriormente, o estarán listos para ser recogidos en el mostrador del aeropuerto antes de la salida del primer vuelo.

Es necesario estar previamente registrado con un número de tarjeta de crédito válida para poder hacer compras de boletos, o de lo contrario proveerla en el momento de la compra.

Además de los servicios de vuelo, el usuario podrá, en cualquier momento, acceder, modificar o cancelar su propio registro, todo esto después de haber sido validado en el sistema.

Es conveniente que el lector note lo informal y limitado de esta descripción, que se refinará a lo largo del capítulo. Dado que el modelo de casos de uso es el principal de todo el sistema, comenzaremos con él.

6.2 Modelo de casos de uso

El **modelo de casos de uso** describe un sistema en términos de sus distintas formas de utilización, cada una de las cuales se conoce como un **caso de uso**. Cada caso de uso o flujo se compone de una secuencia de eventos iniciada por el usuario. Dado que los casos de uso describen el sistema a desarrollarse, los cambios en los requisitos significarán cambios en los casos de uso. Por ejemplo, un caso de uso para manejar un automóvil sería la secuencia de eventos desde que el conductor entra en el coche y enciende el motor hasta llegar a su destino final. Por tanto, para comprender los casos de uso de un sistema primero es necesario saber quiénes son sus usuarios; por ejemplo, conducir un automóvil es distinto de arreglarlo, pues los usuarios también son diferentes: el dueño del automóvil y el mecánico, respectivamente. Para ello, se define el concepto de *actor*, que es el tipo de usuario que está involucrado en la utilización de un sistema, y que además es una entidad externa al propio sistema. Juntos, el actor y el caso de uso, representan los dos elementos básicos de este modelo, lo cual se muestra de manera gráfica en la figura 6.3 de acuerdo con la notación UML.



Figura 6.3 El actor y el caso de uso son las entidades básicas del modelo de casos de uso.

Los casos de uso son ideas simples y prácticas que no requieren muchas habilidades tecnológicas para ser utilizadas (a diferencia de las demás actividades del desarrollo). Por el contrario, si se volvieran muy complejas se perdería su

utilidad. Dado que el modelo de requisitos es la primera actividad del desarrollo del sistema, permite hacer muchos cambios en su especificación sin afectar al resto del sistema. Cuando se identifican y describen los casos de uso, habrá ciertas imprecisiones que se irán resolviendo de manera gradual. De esta manera, se pueden desarrollar de forma independiente los distintos casos de uso para después integrarlos y formar el modelo de requisitos completo. Esta habilidad de tomar parte de la funcionalidad permite un desarrollo más flexible, incluso concurrente.

6.2.1 Actores

Los **actores** son entidades distintas a los usuarios, en el sentido de que éstos son las personas reales que utilizarán el sistema, mientras que los actores representan cierta función que una persona real realiza. En la terminología orientada a objetos, se considera al actor una *clase* de usuario, mientras que los usuarios se consideran como *objetos* o instancias de esa clase. Incluso, una misma persona puede aparecer como diversas instancias de diferentes actores.

Los actores modelan cualquier entidad externa que necesite intercambiar información con el sistema. No están restringidos a ser personas físicas, por lo que pueden representar otros sistemas externos al actual. Lo esencial es que los actores representen entidades externas al sistema. Además, cada uno de estos actores podrá ejecutar una o más tareas del sistema.

Antes de identificar los casos de uso, se identifican los actores del sistema, esto es para que éstos sean la herramienta principal que permita encontrar los casos de uso. Cada actor ejecuta un número específico de casos de uso en el sistema. Una vez definidos todos los actores y casos de uso en el sistema, se establece la funcionalidad completa de éste.

Encontrar actores implica trabajo y raramente se encuentran todos los actores de una vez. Por ejemplo, un sistema de computación puede tener diferentes tipos de usuarios: programadores, operadores, administradores o usuarios generales. Cada uno de estos tipos de usuario corresponde a un actor diferente y, como mencionamos anteriormente, una misma persona puede desempeñar la función de programador u operador.

Para especificar los actores de un sistema, se dibuja un diagrama correspondiente a la **delimitación del sistema**, la cual representa al sistema como una "caja negra", y a los diferentes actores, como entidades externas a ésta, como se muestra en la figura 6.4.



Figura 6.4 Delimitación de un sistema según los actores.

En general, no se describen los actores con demasiado detalle por ser externos al sistema, además de que sus acciones no son deterministas; en otras palabras, un actor —a diferencia del propio sistema— en cada momento puede decidir entre múltiples opciones. Por otro lado, el sistema y los casos de uso correspondientes deben ser deterministas, de lo contrario, el sistema hará lo que crea conveniente, lo cual no es aceptable. Sin embargo, para reconocer los casos de uso, es necesario identificar primero a los actores del sistema, comenzando por aquellos que son la razón principal del sistema, conocidos como **actores primarios**. Estos actores típicamente rigen la secuencia lógica de ejecución del sistema. Además de los actores primarios, existen actores supervisando y manteniendo el sistema, a los que se les llama **actores secundarios** y existen primordialmente como complemento a los actores primarios, siendo esta distinción importante para dedicarle el esfuerzo principal a las necesidades de los actores primarios. Al contrario de éstos, que típicamente pertenecen a personas físicas, los actores secundarios corresponden, por lo general a máquinas o sistemas externos (estos últimos son más difíciles de identificar). Los actores secundarios tienden a responder a secuencias lógicas del sistema y no tanto a inicializarlas de manera propia. En particular, existe siempre la duda, por ejemplo, de si el sistema operativo o una base de datos serían actores. La decisión depende de la función que desempeñen con respecto al sistema en desarrollo, si desempeñan una función activa entonces deben modelarse como actores.

Retomando la descripción del Sistema de Reservaciones de Vuelos, se puede identificar al menos un actor, el *Usuario*; quien está encargado de hacer las consultas y reservaciones en el sistema. Si se analiza un poco más, se puede identificar que las bases de datos de los sistemas externos de reservaciones tienen una función muy activa con respecto al sistema en desarrollo. A este actor lo llamaremos la *Base de Datos de Reservaciones*, el cual mantiene la información sobre los vuelos y reservaciones. Más aún, podemos identificar un actor adicional, representando una segunda base de datos, que se involucre en la información de los usuarios más que de las reservaciones. A este actor lo llamaremos la *Base de Datos de Registros*, encargado de mantener la información de los usuarios sobre la utilización del sistema. El diagrama de delimitación del sistema con los actores correspondientes se muestra en la figura 6.5.



Figura 6.5 Delimitación del sistema de reservaciones de vuelo.

Volviendo a la distinción entre actor y persona, una misma persona puede desempeñar la función del actor *Usuario* cuando hace reservaciones, y además puede trabajar para el sistema de reservaciones, por ejemplo como *Operador*, que correspondería a otro actor no mostrado en nuestro ejemplo.

El actor *Usuario* se considera un actor primario, ya que el sistema se construye pensando en sus usuarios, mientras que *Base de Datos de Reservaciones* y *Base de Datos de Registros* son ambos actores secundarios, ya que si no existieran usuarios no habría necesidad del sistema.

Cuando diferentes actores realizan roles similares, pueden heredar de un actor abstracto común, como lo muestra el actor abstracto *Base de Datos* en el ejemplo de la figura 6.6. El resto de los actores se conoce como **actores concretos**, y utilizan terminología similar a la de herencia, como se vio en el capítulo 4.

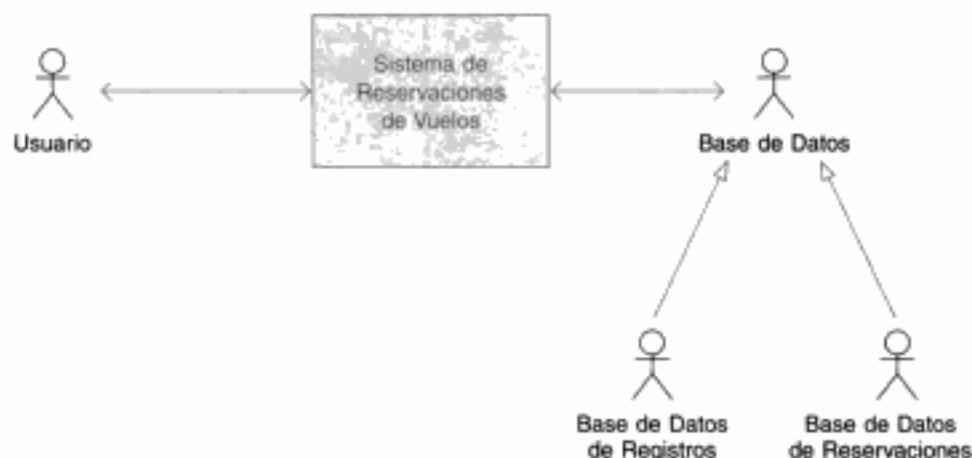


Figura 6.6 Delimitación del sistema de reservaciones de vuelo con ejemplo de herencia entre actores.

La ventaja de modelar actores abstractos es que expresan similitudes entre casos de uso. Si el mismo o parte del mismo caso de uso se puede ejecutar por varios actores diferentes, el caso de uso necesita ser especificado sólo con respecto a un actor en lugar de varios. Por otro lado, los actores abstractos también pueden utilizarse para especificar privilegios comunes a múltiples actores en un sistema.

6.2.2 Casos de uso

Después de haber definido a los actores del sistema, se establece la funcionalidad propia del sistema por medio de los **casos de uso**. Al usarse terminología orientada a objetos, cada caso de uso define una clase o forma particular de usar el sistema, mientras que cada ejecución del caso de uso se puede ver como una instancia del mismo, o sea, un objeto, con estado y comportamiento. Cada caso de uso constituye un flujo completo de eventos, que especifican la interacción que toma lugar entre el actor y el sistema. El actor primario se encarga de dar inicio a esta interacción, mientras que los casos de uso son instanciados como respuesta al evento anterior. Una instancia de un actor puede ejecutar varias de estas secuencias, que constan de diferentes acciones que a su vez deben llevarse a cabo. La instancia del caso de uso existe mientras éste se siga ejecutando. La ejecución del caso de uso termina cuando el actor genera un evento que requiere un caso de uso nuevo. Las diferentes instancias de los casos de uso se conocen como **escenarios**. Como varios casos de uso pue-

den comenzar de una misma forma, no es siempre posible decidir qué caso de uso se ha instanciado, hasta que éste se haya completado.

La descripción de los casos de uso se basa en diagramas similares a los de transición de estados. Se puede ver a cada caso de uso como si representara un estado en el sistema, donde un estímulo enviado entre un actor y el sistema ocasiona una transición entre estados.

En la figura 6.7 se muestra un ejemplo de casos de uso, donde un programador escribe y depura un programa, mientras que otro usuario lo ejecuta.

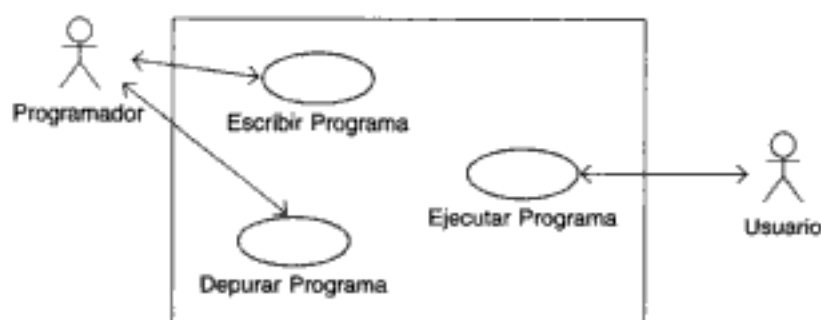


Figura 6.7 Ejemplo de casos de uso que muestran la relación con los actores.

Para identificar los casos de uso, se puede leer la descripción del problema y discutirlo con aquellos que actuarán como actores, empleando preguntas como:

- ▶ ¿Cuáles son las tareas principales de cada actor?
- ▶ ¿Tendrá el actor que consultar y modificar información del sistema?
- ▶ ¿Deberá el actor informar al sistema sobre cambios externos?
- ▶ ¿Desea el actor ser informado sobre cambios inesperados?

Por ejemplo, en un sistema de conmutación telefónica, un actor podría ser un suscriptor, y un caso de uso típico sería hacer una llamada local. El caso de uso comienza cuando el suscriptor levanta el teléfono. Otro caso de uso es ordenar una llamada al servicio de despertador. Ambos casos de uso comienzan cuando el suscriptor levanta el teléfono, pero cuando esto ocurre, no sabemos cuál caso desea ejecutar. Por tanto, los casos de uso pueden comenzar de forma similar y no podemos saber cuál se está instanciando hasta que se termine. En otras palabras, el actor no requiere de que un caso de uso se ejecute, sólo que inicie una secuencia de eventos que finalmente resulte en la terminación de algún caso de uso.

En el *sistema de reservaciones de vuelos* utilizamos los actores ya identificados como punto de partida. Dado que el *Usuario* es el actor primario se comienza con él. El sistema debe ser capaz de dar ciertos servicios al usuario, como consultas y reservaciones. De aquí podemos definir nuestros casos de uso principales, *Consultar Información* y *Hacer una Reservación*. Observe que los nombres de los casos de uso deben corresponder a acciones y no tanto a sustantivos. La figura 6.8 ilustra el sistema con los dos casos de uso, además de un caso de uso adicional, *Mantener el Sistema*, correspondiente a un actor se-

cundario *Operador*. Sin embargo, para lograr una mejor especificación de requisitos y evitar complejidad adicional, no se verá ninguna funcionalidad de mantenimiento en nuestro desarrollo: un ejemplo adicional de cómo concentrarse en ciertos aspectos de los requisitos durante diversas etapas.

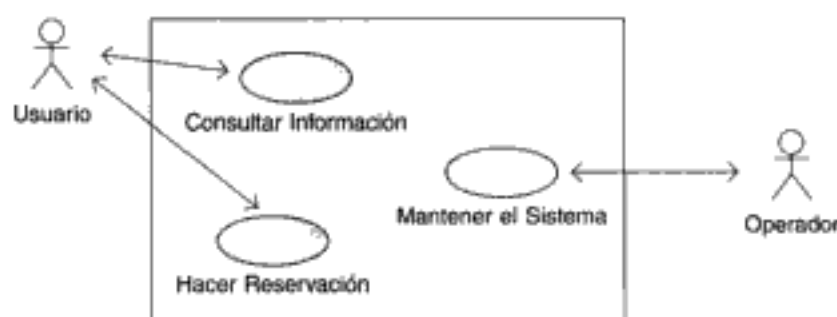


Figura 6.8 Ejemplo de casos de uso para un sistema de reservaciones de vuelo.

En la descripción del problema se menciona que, para utilizar el sistema, el usuario debe estar registrado, por lo cual agregamos un caso de uso *Registrar usuario*. Por otro lado, se debe incluir la *Base de Datos de Reservaciones* y la *Base de Datos de Registros* ya que son actores secundarios necesarios. Estos tres casos de uso se muestran en la figura 6.9. Nótese las flechas unidireccionales dirigiéndose del actor primario hacia el caso de uso y del caso de uso hacia el actor secundario.

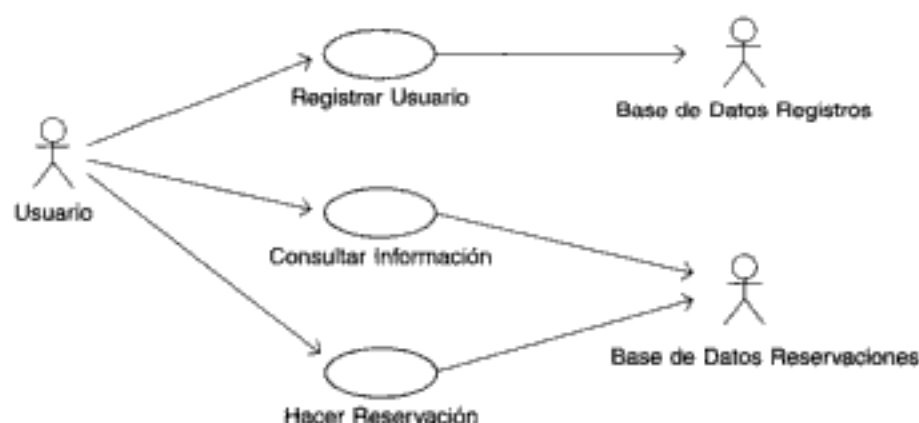


Figura 6.9 Principales casos de uso para el sistema de reservaciones de vuelo. Se eliminó en el diagrama la delimitación del sistema.

Aunque la idea del modelo de casos de uso es que los diagramas sean lo más sencillo posible, a menudo es necesario contar con mecanismos que permitan extender los diagramas de manera más elaborada. El modelo de casos de uso permite la subdivisión de partes del flujo completo de un caso de uso en casos de uso separados. Las razones son aprovechar distintos *subflujos* que se repiten en múltiples casos de uso, de manera análoga al concepto de herencia, y resaltar los flujos menos comunes. Aunque, la complejidad de los casos de uso es un factor importante para tal decisión, en general, no siempre es obvio que subflujos deberían definirse como casos de uso separados. Existen dos enfoques distintos para expresar extensiones:

- ▶ Si las diferencias entre los casos de uso son pequeñas, éstos se pueden describir como variantes de un mismo caso de uso, dando lugar al concepto de *subflujos* o ramificaciones de flujos dentro de un mismo caso de uso. Por ejemplo, en el caso de uso *Registrar Usuario*, la creación por primera vez del registro y su actualización se pueden considerar dos secuencias de eventos lógicamente similares, por lo cual se tratan como distintos subflujos en un mismo caso de uso. En general, primero se describe el *flujo principal* o *básico*, el cual es el flujo más importante dentro del caso de uso y generalmente el punto de entrada al caso de uso. Posteriormente se describen los *subflujos alternos* que pudieran incluir flujos para el manejo de variantes en la lógica, en cuyo caso se conocen como *subflujos alternos*. Normalmente un caso de uso tiene sólo un curso básico, pero varios subflujos y cursos alternos.
- ▶ Si las diferencias entre los casos de uso son grandes, se deben describir como casos de uso separados. Cuando esto ocurre, se utilizan principalmente las relaciones de *extensión*, *inclusión* y *generalización*, como se describe a continuación.

La identificación de casos de uso es normalmente un proceso iterativo, donde se hacen múltiples revisiones hasta llegar a una solución final estable. Cuando esto ocurre, cada caso de uso debe describirse con más detalle.

EXTENSIÓN

Un concepto importante que se utiliza para estructurar y relacionar casos de uso es la **extensión**, la cual especifica cómo un caso de uso puede *insertarse* en otro para extender la funcionalidad del anterior. El caso de uso donde se insertará la nueva funcionalidad debe ser un flujo completo, por lo cual éste es independiente del caso de uso a insertarse. De esta manera, el caso de uso inicial no requiere consideraciones adicionales al caso de uso a ser insertado, únicamente se especifica su punto de inserción.

Tomando como ejemplo el *Sistema de Reservaciones de Vuelos*, se muestra en la figura 6.10 la notación para extensión, utilizando la etiqueta "extiende" (*extend*), donde el caso de uso de *Hacer Reservación* se extiende mediante el caso de uso *Pagar Reservación*.



Figura 6.10 Casos de uso *Hacer Reservación* con extensión de *Pagar Reservación*.

En general, la extensión se utiliza para modelar secuencias de eventos opcionales de casos de uso, que al manejarse de manera independiente pueden agregarse o eliminarse del sistema de manera modular. Se puede considerar a la

asociación de extensión como una interrupción en el caso de uso original que ocurre donde el nuevo caso de uso se va a insertar. Para cada caso de uso que vaya a insertarse en otro caso de uso, se especifica la posición en el caso de uso original, donde la extensión se insertará. El caso de uso original se ejecuta de forma normal hasta el punto donde el caso de uso nuevo se inserta. En este punto, se continúa con la ejecución del nuevo curso. Después que la extensión se ha terminado, el curso original continúa como si nada hubiera ocurrido. Por regla general, se describe primero los casos de uso básicos, totalmente independientes de cualquier extensión de funcionalidad, y luego los de extensión.

INCLUSIÓN

Una relación adicional entre casos de uso es la *inclusión*. A diferencia de una extensión, la **inclusión** se define como una sección de un caso de uso que es parte obligatoria del caso de uso básico. El caso de uso donde se insertará la funcionalidad depende del caso de uso a ser insertado. Esta relación se etiqueta con "incluye" (*include*). Por ejemplo, en el *Sistema de Reservaciones de Vuelos*, el caso de uso *Consultar Información* incluye el caso de uso *Validar Usuario* como se muestra en la figura 6.11.

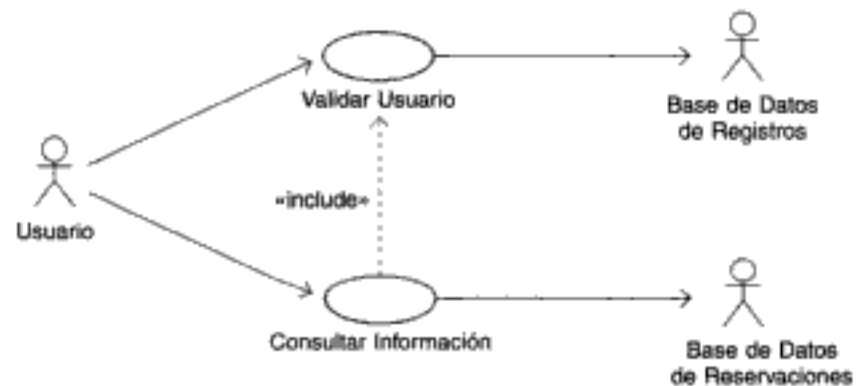


Figura 6.11 Casos de uso *Consultar Información* con inclusión de *Validar Usuario*.

GENERALIZACIÓN

Una relación adicional entre casos de uso es la **generalización**, la cual apoya la reutilización de los casos de uso. Mediante la relación de generalización es necesario describir las partes similares una sola vez, en lugar de repetirlas para todos los casos de uso con comportamiento común. Los casos de uso extraídos se conocen como **casos de uso abstractos**, ya que no serán instanciados independientemente, y servirán sólo para describir partes que son comunes a otros casos de uso. Los casos de uso que realmente serán instanciados se llaman **casos de uso concretos**. Las descripciones de los casos de uso abstractos se incluyen en las descripciones de los casos de uso concretos. Una técnica para extraer casos de uso abstractos es identificar actores abstractos. Esta generalización es una "herencia de secuencias" (en lugar de operaciones, en el caso de herencia de objetos). Por ejemplo, en el *Sistema de Reservaciones de Vuelos*, el caso de uso *Pagar Reservación* pudiera generalizarse en dos casos de

uso, *Pagar con Tarjeta* y *Pagar con Transferencia*, como se muestra en la figura 6.12. Uno de estos dos "sub" casos de uso debería instanciarse, ya que el "súper" caso de uso, *Pagar Reservación*, sería abstracto por no conocerse la forma de pago.

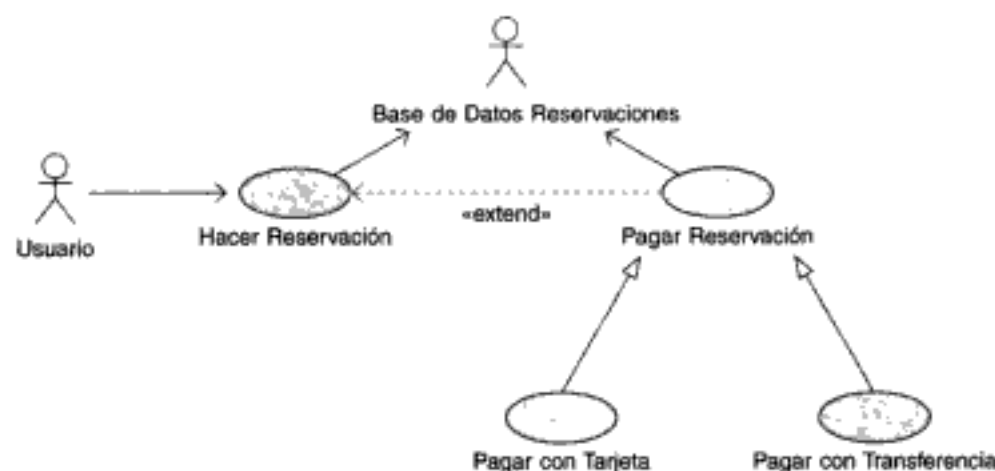


Figura 6.12 Casos de uso *Pagar Reservación* con generalización de pagos: *Pagar con Tarjeta* y *Pagar con Transferencia*.

Las relaciones de *extensión* e *inclusión* entre casos de uso se implementan ambas mediante asociaciones de clases, a diferencia de la *generalización*, la cual se implementa mediante herencia de clases. En la mayoría de los casos, la selección es bastante obvia; sin embargo, el criterio principal es ver qué tanto se acoplan las funcionalidades de los casos de uso. Si el caso de uso a ser extendido es útil por sí mismo, la relación debe ser descrita utilizando extensión. Si los casos de uso son fuertemente acoplados y la inserción debe tomar lugar para obtener un curso completo, la relación debe ser descrita mediante la inclusión. Por otro lado, la generalización se emplea cuando dos o más casos de uso comparten funcionalidad común, la cual es extendida por cada uno al estilo de la generalización entre clases. También hay una diferencia en cómo se identifican estas relaciones. Las extensiones se identifican en un caso de uso existente, que el usuario desea extender con secuencias adicionales, mientras que las inclusiones se encuentran de la extracción de secuencias comunes ya existente entre varios casos de usos. Las generalizaciones se encuentran al tratar de especializar de diferente manera un caso de uso existente.

Finalmente, se desean diagramas sencillos que no sean "telarañas", pero que muestren de manera esquemática las posibles secuencias de interacciones entre los actores y el sistema.

En la figura 6.13, se ilustra el diagrama completo de casos de uso para el *Sistema de Reservas de Vuelos*. Los casos de uso adicionales en este diagrama son la extensión de *Registrar Tarjeta* y *Pagar Reservación*. Este último caso de uso es interesante, porque extiende *Hacer Reservación* e incluye *Registrar Tarjeta*, ambos requisitos con el fin de comprar un boleto con el sistema. Además de la inclusión anterior, también se incluyen los casos de uso de *Validar*

usuario y Ofrecer Servicios en los casos de uso básicos: *Registrar Usuario*, *Consultar Información* y *Hacer Reservación*.

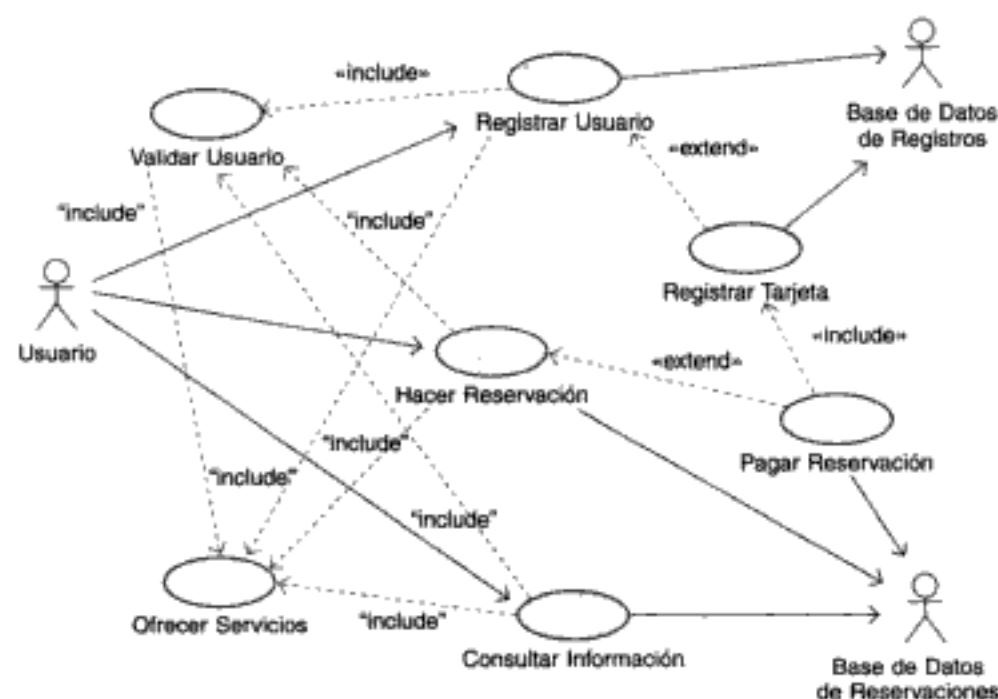


Figura 6.13 En el diagrama se muestran los casos de uso completos para el sistema de reservas de vuelo, que consiste en tres actores y siete casos de uso.

6.2.3 Documentación

Parte fundamental del modelo de casos de uso es la descripción textual detallada de cada uno de los actores y casos de uso identificados. Estos documentos son sumamente críticos ya que a partir de ellos se desarrollará el sistema completo. El formato de **documentación** para cada actor es el siguiente:

Actor	Nombre del actor.
Casos de uso	Nombre de los casos de uso en los cuales participa.
Tipo	<i>Primario o secundario.</i>
Descripción	Breve descripción del actor.

Las descripciones de los casos de uso representan todas las posibles interacciones de los actores con el sistema en los eventos enviados o recibidos por éstos. En esta etapa no se incluyen eventos internos al propio sistema, ya que esto se estudiará durante el análisis, y únicamente agregaría una complejidad innecesaria. El formato de **documentación** para cada caso de uso es el siguiente:

Caso de uso	Nombre del caso de uso.
Actores	Actores primarios y secundarios que interaccionan con el caso de uso.
Tipo	Tipo de flujo: <i>básico, inclusión, extensión, generalización</i> o algún otro.
Propósito	Razón de ser del caso de uso.
Resumen	Resumen del caso de uso.
Precondiciones	Condiciones que deben satisfacerse para ejecutar el caso de uso.
Flujo principal	El flujo de eventos más importante del caso de uso, donde dependiendo de las acciones de los actores, se continuará con alguno de los subflujos.
Subflujos	Los flujos secundarios del caso de uso, numerados como (S-1), (S-2), etcétera.
Excepciones	Excepciones que pueden ocurrir durante el caso de uso, numerados como (E-1), (E-2), etcétera.

Dado que el modelo de casos de uso está motivado y enfocado principalmente hacia los sistemas de información, donde los usuarios juegan un papel primordial, es importante relacionarse con las interfaces a ser diseñadas en el sistema. Éstas sirven para apoyar de mejor manera la descripción de los casos de uso, además de servir de base en prototipos iniciales.

Un comentario importante sobre la especificación de estos documentos, es que se sigue un proceso iterativo para definir cada uno de ellos, el cual se podrá modificar o refinar después. Obviamente, cuanto más tarde ocurran estos cambios, más costoso será implementarlos. Note que en las siguientes descripciones, ya se hace referencia a pantallas de interacción con el usuario, las cuales serán mostradas en un diseño preliminar en la siguiente sección.

Los actores y casos de uso del sistema de reservaciones de vuelo son descritos en la sección 6.4.

6.3 Modelo de interfaces

El **modelo de interfaces** describe la presentación de información entre los actores y el sistema. Se especifica en detalle cómo se verán las interfaces de usuario al ejecutar cada uno de los casos de uso. Si se trata de la *interface humano-computadora (HCI, Human Computer Interface)*, se pueden usar esquemas de cómo vería el usuario las pantallas cuando se ejecuta cada caso de uso. También se puede generar una simulación más sofisticada usando un *sistema manejador de interfaces de usuario (UIMS, User Interface Management System)*. Normalmente, un prototipo funcional de requisitos que muestra las interfaces de usuario es una estrategia importante. Esto ayuda al usuario a visualizar los casos de uso según se mostrarán en el sistema a construirse. Tal enfoque elimina muchas posibilidades de malos entendidos. Cuando se diseñan las interfaces de usuario, es esencial tener a los usuarios involucrados, y que las interfaces refle-

jen la visión lógica del sistema, debiendo haber consistencia entre la imagen conceptual del usuario y el comportamiento real del sistema. Si las interfaces son protocolos de hardware, se pueden referir a los diferentes estándares, como protocolos de comunicación. Estas descripciones de interfaces son, por tanto, partes esenciales de las descripciones de los casos de uso y las deben acompañar. En estas etapas iniciales del desarrollo, el diseño de las pantallas no es tan importante como el manejo de información que se ofrece, el cual, debe corresponder a las necesidades de cada caso de uso, algo que se mostrará a continuación en el *Sistema de Reservaciones de Vuelos*.

6.4 Actores y casos de uso para el sistema de reservaciones de vuelos

Tomando como ejemplo el sistema de reservaciones de vuelo, mostraremos la documentación de los actores y casos de uso junto con el diseño de las interfaces que se usarán como prototipo del sistema. Estos diseños pueden hacerse en papel o aprovechar una herramienta que simplifique la tarea del diseño de pantallas. El objetivo primordial es llegar a un acuerdo rápido sobre la funcionalidad de la aplicación, más que lograr un diseño gráfico sofisticado.

6.4.1 Actores

Se describen en total tres **actores** en el sistema de reservaciones de vuelos. El *Usuario* interactúa con todos los casos de uso, aunque todas las asociaciones no fueron diagramadas de manera explícita en la figura 6.13.

Actor	<i>Usuario.</i>
Casos de uso	<i>Validar Usuario, Registrar Usuario, Registrar Tarjeta, Consultar Información, Hacer Reservación, Pagar Reservación, Ofrecer Servicios.</i>
Tipo	Primario.
Descripción	Es el actor principal y representa a cualquier persona que desee utilizar el sistema de reservaciones.

La *Base de Datos de Registros* interactúa con los casos de uso relacionados exclusivamente con registro.

Actor	<i>Base de Datos de Registros.</i>
Casos de uso	<i>Validar Usuario, Registrar Usuario, Registrar Tarjeta.</i>
Tipo	Secundario.
Descripción	Es un actor secundario y representa a la base de datos donde se guarda toda la información relacionada con los usuarios, pero independiente de las reservaciones.

La *Base de Datos de Reservas* interactúa con los casos de uso relacionados exclusivamente con reservas.

Actor	<i>Base de Datos de Reservas.</i>
Casos de uso	<i>Consultar Información, Hacer Reservación, Pagar Reservación.</i>
Tipo	Secundario.
Descripción	Es un actor secundario y representa la base de datos donde se guarda toda la información relacionada con las reservas, pero independiente de los propios usuarios del sistema.

6.4.2 Casos de uso

Como apoyo en la descripción de los **casos de uso**, mostraremos las diversas pantallas diseñadas, comenzando con la pantalla principal. Dado que el requisito de uso del sistema es que todo usuario deba haberse registrado, la pantalla principal debe dar dos opciones: *a)* registrarse por primera vez y *b)* validar un registro existente como se muestra en la figura 6.14.



Figura 6.14. Pantalla principal del sistema (P-1).

Observe que el caso de uso *Registrar Usuario*, como lo describiremos en nuestro ejemplo, agrupa la lógica relacionada con un usuario ya registrado junto con otro que se registra por primera vez. Dado que la opción para registrarse por

primera vez debe aparecer en la pantalla principal (*P-1*), la opción en la pantalla de servicios (*P-2*) es utilizada por un usuario ya registrado. Y aunque se podrían definir dos casos de uso separados para la lógica de registro, por ejemplo, *Crear Registro Usuario* y *Obtener Registro usuario*, consideramos que éstos son más bien subflujos de un mismo caso de uso. Es conveniente hacer notar que existen muchas maneras de describir un sistema similar, donde por lo general una forma no es necesariamente más correcta que la otra, siendo más bien una cuestión de estandarización o estilo del analista. En nuestro caso, buscamos soluciones compactas que reduzcan el número total de casos de uso, siempre y cuando la lógica esté muy relacionada. Se incluyen además varios subflujos para llamar secciones del flujo desde distintos lugares, ya que no se puede comenzar un flujo o subflujo en la mitad del proceso. A continuación describimos los distintos casos de uso con las pantallas correspondientes. Se excluyen pantallas menores como aquellas con mensajes de error o confirmación del éxito de la operación. Comenzamos describiendo los flujos *Validar Usuario* y *Ofrecer Servicios*, que se incluyen en los diversos casos de uso. Posteriormente describimos cada uno de los casos básicos y de extensión.

VALIDAR USUARIO

El caso de uso *Validar Usuario* está vinculado con la pantalla principal (*P-1*) y se llama a partir de los casos de uso *Registrar Usuario*, *Consultar Información* y *Hacer Reservación* como se mostró en el diagrama de la figura 6.13. Dado que este caso de uso se inserta en los anteriormente mencionados, depende de éstos incluirlo y llamarlo apropiadamente.

Caso de uso	<i>Validar Usuario.</i>
Actores	<i>Usuario, Base de Datos de Registros.</i>
Tipo	Inclusión.
Propósito	Validar a un usuario ya registrado para el uso del sistema de reservaciones de vuelo.
Resumen	Este caso de uso, se inicia por el <i>usuario</i> . Valida al usuario mediante un <i>login</i> y <i>password</i> a verificarse con su respectivo registro de usuario, para que pueda utilizar el sistema de reservaciones.
Precondiciones	Se requiere haber ejecutado anteriormente el caso de uso <i>Registrar Usuario</i> subflujo <i>Crear Registro Usuario</i> .
Flujo principal	Se presenta al usuario la pantalla principal (<i>P-1</i>). El usuario puede seleccionar entre las siguientes opciones: "Registrarse por primera vez", "OK" y "Salir". Si la actividad seleccionada es "Registrarse por primera vez", se ejecuta el caso de uso <i>Registrar Usuario</i> , subflujo <i>Crear Registro Usuario</i> (<i>S-1</i>). Si la actividad seleccionada es "OK", se valida el registro de usuario mediante un <i>login</i> y un <i>password</i> insertados por el usuario en la pantalla principal (<i>P-1</i>). Una vez validado el usuario (<i>E-1</i>), se continúa con el caso de uso <i>Ofrecer Servicios</i> . Si la actividad seleccionada es "Salir" se saldrá del sistema.

(continúa)

(continuación)

Subflujos	Ninguno.
Excepciones	<i>E-1 no hubo validación: el login/password no se validó correctamente. Se solicita al usuario volver a registrarse. Después de tres intentos se saldrá del sistema.</i>

OFRECER SERVICIOS

Si observamos el diagrama de casos de uso de la figura 6.13, vemos que existen tres casos de uso básicos que el usuario puede instanciar: *Registrar Usuario*, *Hacer Reservación* y *Consultar Información*. El caso de uso *Ofrecer Servicio* se incluye en estos tres casos de uso básicos para delegarles de manera adecuada, según las opciones seleccionadas por el usuario. También se incluye en el caso de uso *Validar Usuario*, luego de la validación de un usuario.

Caso de uso	Ofrecer Servicios.
Actores	Usuario.
Tipo	Inclusión.
Propósito	Ofrecer los diversos servicios a un usuario ya registrado para que use el sistema de reservaciones de vuelo.
Resumen	El usuario inicia este caso de uso. Tiene capacidad de utilizar las diversas opciones del sistema de reservaciones.
Precondiciones	Se requiere haber validado correctamente al usuario.
Flujo principal	Se presenta al usuario la pantalla servicios (P-2). El usuario puede seleccionar entre las siguientes actividades: "Consultar información", "Hacer reservación", "Obtener registro" y "Salir". Si la actividad seleccionada es "Consultar información", se continúa con el caso de uso <i>Consultar Información</i>, subflujo <i>Consultar</i> (S-1). Si la actividad seleccionada es "Hacer reservación", se continúa con el caso de uso <i>Hacer reservación</i>, subflujo <i>Solicitar Clave Reservación</i> (S-1). Si la actividad seleccionada es "Obtener Registro", se continúa con el caso de uso <i>Registrar Usuario</i>, subflujo <i>Obtener Registro Usuario</i> (S-2). Si la actividad seleccionada es "Salir" se saldrá del sistema.
Subflujos	Ninguno.
Excepciones	Ninguno.

Por tal motivo, la siguiente pantalla del sistema debe permitir al usuario seleccionar las opciones correspondientes, como se muestra en la figura 6.15.

REGISTRAR USUARIO

El caso de uso *Registrar Usuario* está vinculado con el registro inicial del usuario y la modificación de la información de registro. Además, ya deben incluir-



Figura 6.15 Pantalla de Menú de servicios (P-2).

se los puntos de inclusión y extensión para los casos de uso *Validar Usuario*, *Ofrecer Servicios* y *Registrar Tarjeta*, respectivamente.

Se describe a continuación las secciones iniciales del caso de uso, con las secciones restantes, subflujos y excepciones, más adelante.

Casos de uso	<i>Registrar Usuario.</i>
Actores	<i>Usuario, Base de Datos Registros.</i>
Tipo	Básico.
Propósito	Permitir a un usuario registrarse en el sistema de reservaciones de vuelo para usarlo.
Resumen	El usuario inicia este caso de uso. Ofrece funcionalidad para crear, modificar y eliminar el registro de usuario con el sistema de reservaciones.
Precondiciones	Todos los subflujos, con excepción de <i>Crear Registro Usuario (S-1)</i> , requieren ejecutar inicialmente el caso de uso <i>Validar usuario</i> .
Flujo principal	Se ejecuta el caso de uso <i>Validar Usuario</i> . Dependiendo de las opciones seleccionadas por el usuario, se continuará con los diversos subflujos de este caso de uso.

El diseño de la pantalla de registrarse por primera vez se muestra en la figura 6.16.

SISTEMA DE RESERVACIONES DE VUELO
Pantalla Crear Registro Usuario (P-3)

Nombre: Apellido:

Calle: Colonia:

Ciudad: País: Código Postal:

Tel. Casa: Tel. Of.: Fax:

Login: E-Mail:

Password: Repetir Password:

Figura 6.16 Pantalla de Crear registro de usuario por primera vez (P-3).

El subflujo *Crear Registro Usuario (S-1)*, descrito a continuación, se instancia al presionar el botón correspondiente de la pantalla principal (P-1).

Subflujos	S-1 Crear Registro Usuario. Se presenta al usuario la pantalla "crear registro usuario" (P-3), que contiene información de registro que debe llenar el usuario, lo cual incluye nombre, apellido, calle, colonia, ciudad, país, código postal, teléfonos de la casa y oficina, número de fax, <i>login</i>, email, <i>password</i> y una entrada adicional de repetir <i>password</i> para asegurarse de que se escribió correctamente. El sistema usará el <i>login</i> y el <i>password</i> para validar al usuario. El usuario puede seleccionar entre las siguientes actividades: "Registrar" y "Salir". Si el usuario selecciona "Registrar", el sistema genera un nuevo registro de usuario (E-1, E-2, E-3, E-4). Se continúa con el subflujo <i>Administrar Registro Usuario (S-3)</i>. Si la actividad seleccionada es "Salir" se saldrá del sistema (si aún no se ha presionado "Registrar", la información se perderá).
------------------	---

En la figura 6.17 se ilustra el diseño de la pantalla para administrar un registro de usuario existente. Note que la información que ofrece es exactamente igual

a la anterior. La única diferencia son las opciones que se ofrecen: eliminar y actualizar en lugar de registrar.

Figura 6.17 Pantalla de Obtener registro de usuario (P-4).

El resto de los subflujos se describen a continuación. El subflujo *Obtener Registro Usuario* (S-2) se instancia al presionar el botón correspondiente de la pantalla de servicios (P-2). El subflujo *Administrar Registro Usuario* (S-3) se instancia una vez que se presente la información correspondiente en la pantalla de registro (P-4). El subflujo *Actualizar Registro Usuario* (S-4) se instancia al presionar el botón correspondiente de la pantalla de registro (P-4). El subflujo *Eliminar Registro Usuario* (S-5) se instancia al presionar el botón correspondiente de la pantalla de registro (P-4).

Subflujos (continuación)	S-2 Obtener Registro Usuario. El sistema obtiene el registro de usuario de la base de datos de registro. Se continúa con el subflujo <i>Administrar Registro Usuario</i> (S-3). S-3 Administrar Registro Usuario. Se presenta al usuario la pantalla "Obtener registro usuario" (P-4) con la información de registro de usuario. Éste podrá seleccionar entre las siguientes actividades: "Eliminar", "Actualizar", "Registrar tarjeta", "Servicios" y "Salir".
------------------------------------	---

(continúa)

Subflujos (continuación)	Si el usuario presiona "Actualizar", se ejecuta el subflujo <i>Actualizar Registro Usuario (S-4)</i> .
	Si el usuario selecciona "Eliminar", se ejecuta el subflujo <i>Eliminar Registro Usuario (S-5)</i> .
	Si el usuario presiona "Registrar Tarjeta", se continúa con el caso de uso <i>Registrar Tarjeta</i> .
	Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i> .
	Si la actividad seleccionada es "Salir", se saldrá del sistema (si aún no se ha presionado "Actualizar", la nueva información se perderá).
	S-4 Actualizar Registro Usuario. Se actualiza el registro de usuario con la información modificada (E-1, E-3, E-4). Se continúa con el subflujo <i>Administrar Registro Usuario (S-3)</i> .
	S-5 Eliminar Registro Usuario. Se elimina el registro de usuario y se continúa con el subflujo <i>Crear Registro Usuario (S-1)</i> .

Las excepciones del caso de uso son las siguientes:

Excepciones	E-1 Información incompleta: falta llenar información en el registro de usuario. Se vuelve a solicitar al usuario que complete el registro. E-2 Registro ya existe: si ya existe un registro bajo ese <i>login</i>, se solicitará al usuario que lo cambie o que termine el caso de uso. E-3 Login incorrecto: el <i>login</i> no es válido. Se le solicita al usuario que corrija el registro. E-4 Password incorrecto: el <i>password</i> escogido es muy sencillo o no se validó correctamente. Se solicita al usuario que corrija el registro.
--------------------	---

REGISTRAR TARJETA

El caso de uso *Registrar Tarjeta* es una extensión del caso de uso *Registrar Usuario*. De manera similar a *Registrar Usuario*, el caso de uso *Registrar Tarjeta* está compuesto por un registro inicial y por la modificación de la información ya registrada.

Se describe a continuación las secciones iniciales del caso de uso, con las secciones restantes, subflujos y excepciones, más adelante. Note que el flujo principal se llama al presionar "Registrar tarjeta" en la pantalla de obtener registro de usuario (P-4).

Caso de uso	<i>Registrar Tarjeta.</i>
Actores	<i>Usuario, Base de Datos Registros.</i>
Tipo	Extensión.
Propósito	Permitir a un usuario registrar una tarjeta de crédito en el sistema de reservaciones de vuelo para pagar boletos.
Resumen	Este caso de uso lo inicia el <i>Usuario</i> . Ofrece funcionalidad para crear, modificar y eliminar el registro de tarjeta usuario con objeto de pagar las reservaciones directamente en el sistema de reservaciones.
Precondiciones	El usuario ya se debió registrar mediante la activación del caso de uso <i>Registrar Usuario</i> .
Flujo principal	Se continúa con el subflujo <i>Obtener Registro Tarjeta (S-2)</i> . Si no existe un registro de tarjeta válido se continúa con el subflujo <i>Crear Registro Tarjeta (S-1)</i> . De lo contrario, si ya existe uno, se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i> .

El diseño de la pantalla para registrar la tarjeta por primera vez se muestra en la figura 6.18.

SISTEMA DE RESERVACIONES DE VUELO

Pantalla de Crear Registro Tarjeta (P-5)

Nombre:

Número de tarjeta (requerido para pagos):

Tipo: Fecha de vencimiento:

Figura 6.18 Pantalla de Crear registro de tarjeta por primera vez (P-5).

El subflujo *Registrar Tarjeta (S-1)*, descrito a continuación, se instancia al presionar el botón correspondiente de la pantalla “servicios” (P-5).

Subflujo	<i>S-1 Crear Registro Tarjeta.</i> Se presenta la pantalla "Crear Registro Tarjeta" (P-5). La pantalla incluye el nombre como aparece en la tarjeta, número de tarjeta, el tipo de tarjeta y la fecha de vencimiento. El usuario puede seleccionar entre las actividades "Registrar", "Servicios", "Salir". Si el usuario presiona "Registrar", el sistema verifica la información (E-1), y se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i>. Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", se saldrá del sistema (si aún no se ha presionado "Registrar", la nueva información se perderá).
-----------------	--

En la figura 6.19 se ve el diseño de la pantalla para la administración de un registro de tarjeta existente. Observe que la información que ofrece es exactamente igual a la anterior. La única diferencia son las opciones que se ofrecen: eliminar y actualizar en lugar de registrar.

Figura 6.19 Pantalla de Obtener Registro Tarjeta (P-6).

El resto de los subflujos se describen a continuación. Los subflujos *Obtener Registro Tarjeta (S-2)* y *Administrar Registro Tarjeta (S-3)* se utilizan para leer y manipular el registro de tarjeta, respectivamente. El subflujo *Actualizar Registro Tarjeta (S-4)* se instancia presionando el botón correspondiente de la pantalla.

lla de registro (P-6). El subflujo *Eliminar Registro Tarjeta* (S-5) se instancia presionando el botón correspondiente de la pantalla de registro (P-6).

Subflujos (continuación)	S-2 Obtener Registro Tarjeta. El sistema obtiene el registro de tarjeta de la base de datos de registro. Se regresa al flujo anterior.
	S-3 Administrar Registro Tarjeta. Se presenta la pantalla "Obtener Registro Tarjeta" (P-6), que incluye el nombre como aparece en la tarjeta, número de tarjeta, el tipo de tarjeta y la fecha de vencimiento. El usuario podrá seleccionar entre las actividades "Eliminar", "Actualizar", "Servicios" y "Salir". Si el usuario presiona "Actualizar" se ejecuta el subflujo <i>Actualizar Registro Tarjeta</i> (S-4). Si el usuario presiona "Eliminar", se ejecuta el subflujo <i>Eliminar Registro Tarjeta</i> (S-5). Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i> . Si la actividad seleccionada es "Salir" se saldrá del sistema.
	S-4 Actualizar Registro Tarjeta. Se actualiza el registro de tarjeta con la información modificada (E-1). Se continúa con el subflujo <i>Administrar Registro Tarjeta</i> (S-2).
	S-5 Eliminar Registro Tarjeta. Se elimina el registro de tarjeta y se continúa con el subflujo <i>Crear Registro Tarjeta</i> (S-1).

Las excepciones del caso de uso son las siguientes.

Excepciones	E-1 Información incompleta: falta llenar información indispensable para completar el registro de tarjeta. Se le vuelve a pedir al usuario que complete el registro de tarjeta.
--------------------	---

CONSULTAR INFORMACIÓN

El caso de uso *Consultar información* se instancia a partir de la pantalla de servicios (P-2), una vez que se haya validado el usuario y seleccionado el botón correspondiente. Este caso de uso es el más complejo de todos los del sistema, ya que incluye tres tipos de consultas distintas: consulta de horarios de vuelo, consulta de tarifas y consulta de estado de un vuelo.

El caso de uso se describe a continuación, y excluye los subflujos y restricciones que se describen más adelante.

Caso de uso	Consultar información.
Actores	Usuario, Base de datos de reservas.
Tipo	Básico.
Propósito	Permitir a un usuario consultar información con el sistema de reservaciones de vuelo.
Resumen	El usuario inicia este caso de uso. Ofrece funcionalidad para consultar información de horarios, tarifas y estado de vuelos con el sistema de reservaciones.
Precondiciones	Se requiere haber ejecutado antes el caso de uso Validar usuario.
Flujo principal	Se ejecuta el caso de uso Validar usuario. Dependiendo de las opciones seleccionadas por el usuario, se continuará con los diversos subflujos de este caso de uso.

Antes de proseguir con la descripción del caso de uso, mostramos la pantalla que resume esta decisión, como se muestra en la figura 6.20.



Figura 6.20 Pantalla de Selección de tipo de Consultas (P-7).

Dado que el usuario puede iniciar una consulta de varias páginas distintas, como se verá posteriormente, se incluye un primer subflujo para iniciar estas consultas llamado *Consultar (S-1)*, como se observa a continuación:

Subflujos	<i>S-1 Consultar.</i> Se despliega la pantalla "Consultas" (P-7). El usuario puede seleccionar entre las siguientes actividades: "Horarios", "Tarifas", "Estado", "Servicios" y "Salir". Si el usuario presiona "Horarios", se activa el subflujo <i>Consultar Horarios (S-2)</i>. Si el usuario presiona "Tarifas", se activa el subflujo <i>Consultar Tarifas (S-4)</i>. Si el usuario presiona "Estado", se activa el subflujo <i>Consultar Estado (S-6)</i>. Si el usuario presiona "Servicios", se pasa al caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", sale del sistema.
------------------	--

En la Figura 6.21 se muestra el diseño de la pantalla para la consulta de horarios de vuelos, correspondiente al subflujo *Consultar Horarios (S-2)*.

SISTEMA DE RESERVACIONES DE VUELO

Pantalla Consultar Horarios de Vuelos (P-8)

Origen (ciudad o código): Destino (ciudad o código):

PREFERENCIAS

Aerolínea: Horario:

☐ Solo directo

Figura 6.21 Pantalla de Consulta de horarios de vuelos (P-8).

En la Figura 6.22 se muestra el diseño de la pantalla para el resultado de la consulta de horarios de vuelos, correspondiente al subflujo *Devolver Horarios (S-3)*.

SISTEMA DE RESERVACIONES DE VUELO
Pantalla Resultado de Horarios (P-9)

Fecha de consulta:

Seleccionar	Aerolínea	Vuelo	Días	Horario	Restricciones
☐					
☐					
☐					
☐					
☐					
☐					
☐					
☐					
☐					
☐					

Figura 6.22 Pantalla de Resultados de consulta de horarios de vuelos (P-9).

Los subflujos *Consultar Horarios (S-2)* y *Devolver Horarios (S-3)* se muestran a continuación:

Subflujos (continuación)	S-2 Consultar Horarios. Se presenta al usuario la pantalla "Consulta horarios" (P-8). Esta pantalla debe ser llenada con información de ciudad de origen y destino, y preferencias opcionales de: aerolínea, horario y una opción de vuelo directo. El usuario puede seleccionar las siguientes actividades: "Consultar", "Servicios" y "Salir". Si el usuario presiona "Consultar", el sistema recibe la información (E-1, E-2), se continúa con el subflujo <i>Devolver Horarios (S-3)</i>. Si el usuario presiona "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir" se sale del sistema. S-3 Devolver Horarios. Se presenta la pantalla "Resultado horarios" (P-9) que contiene información sobre los diferentes vuelos encontrados. La información incluye la aerolínea, el vuelo, los días, el horario y las restricciones, tales como fecha de inicio o terminación del vuelo. Al principio de cada fila se encuentra una opción de selección para obtener información adicional sobre el vuelo.
------------------------------------	---

(continúa)

Subflujos (continuación)	El usuario puede seleccionar entre las siguientes opciones: "+", "-", "Nueva consulta", "Servicios" y "Salir". Si el usuario presiona "+", se muestran resultados adicionales de horarios. Se continúa al inicio de este subflujo. Si el usuario presiona "-", se muestran resultados anteriores de horarios. Se continúa al inicio de este subflujo. Si el usuario presiona "Nueva consulta", se continúa con el subflujo <i>Consultar Horarios (S-2)</i>. Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", sale del sistema.
------------------------------------	---

En la figura 6.23 se muestra el diseño de la pantalla para la consulta de tarifas de vuelos, correspondiente al subflujo *Consultar Tarifas (S-4)*.

Figura 6.23 Pantalla de Consulta de tarifas de vuelos (*P-10*).

En la figura 6.24 se ilustra el diseño de la pantalla para el resultado de la consulta de tarifas de vuelos, correspondiente al subflujo *Devolver Tarifas (S-5)*.

SISTEMA DE RESERVACIONES DE VUELO
Pantalla Resultado Tarifas (P-11)

Fecha de consulta:

Seleccionar	Aerolínea	Vuelo	Días	Horario	Clase	Tarifa O/W	Tarifa R/P	Restricciones
☐								
☐								
☐								
☐								
☐								
☐								
☐								
☐								
☐								
☐								
☐								

Figura 6.24 Pantalla de Resultado de consulta de tarifas de vuelos (P-11).

Los subflujos *Consultar Tarifas* (S-4) y *Devolver Tarifas* (S-5) se muestran a continuación:

Subflujos (continuación)	S-4 Consultar Tarifas. Se presenta al usuario la pantalla "Consultar tarifas" (P-10), la cual se llena con información de la ciudad de origen y el destino, y las preferencias opcionales: fecha de salida, fecha de regreso, aerolínea, clase y opciones de organizar la información por menor tarifa, una opción de vuelo directo y si la tarifa se basa en viaje redondo. El usuario puede seleccionar entre las siguientes actividades: "Consultar", "Servicios" y "Salir". Si el usuario presiona "Consultar", el sistema recibe la información (E-1, E-2), se continúa con el subflujo <i>Devolver Tarifas</i> (S-5). Si el usuario presiona "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", sale del sistema. S-5 Devolver Tarifas. Se presenta la pantalla "Resultado tarifas" (P-11), que contiene información sobre los diferentes vuelos encontrados. La información incluye la aerolínea, el vuelo, la fecha, el horario, la tarifa en viaje sencillo o viaje redondo y restricciones correspondientes. Al principio de cada fila se encuentra una opción para seleccionar en caso de hacer consultas o reservas sobre los vuelos obtenidos.
------------------------------------	--

(continúa)

Subflujos (continuación)	El usuario puede seleccionar entre las siguientes opciones: "+", "-", "Nueva Consulta", "Servicios" y "Salir". Si el usuario presiona "+", se muestran resultados adicionales de horarios. Se continúa al inicio de este subflujo. Si el usuario presiona "-", se muestran resultados anteriores de horarios. Se continúa al inicio de este subflujo. Si el usuario presiona "Nueva Consulta", se continúa con el subflujo <i>Consultar Tarifas (S-4)</i>. Si el usuario presiona "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", sale del sistema.
------------------------------------	---

La figura 6.25 ilustra el diseño de la pantalla para la consulta de estado de vuelo, correspondiente al subflujo *Consultar Estado (S-6)*.

Figura 6.25 Pantalla de Consulta de estado de vuelo (P-12).

En la figura 6.26 se muestra el diseño de la pantalla para el resultado de la consulta de estado de vuelo, correspondiente al subflujo *Devolver Estado (S-7)*.

Figura 6.26 Pantalla de Resultado de consulta de estado de vuelo (P-13).

Los subflujos de *Consultar estado* (S-6) y *Devolver Estado* (S-7) se muestran a continuación:

Subflujos (continuación)	S-6 Consultar Estado. Se presenta al usuario la pantalla "Consultar estado" (P-12). Esta pantalla debe ser llenada con información de la ciudad de origen y el destino, la aerolínea, el número de vuelo y la opción de vuelo de día consultado. El usuario puede seleccionar alguna de las siguientes actividades: "Consultar", "Servicios" y "Salir". Si el usuario presiona "Consultar", el sistema recibe la información (E-1, E-2) y se continúa con el subflujo <i>Devolver Estado</i> (S-7). Si el usuario presiona "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", sale del sistema. S-7 Devolver Estado. Se presenta la pantalla "Resultado estado" (P-13), que contiene información sobre el vuelo, como su estado, por ejemplo, confirmado, retrasado, cancelado.
------------------------------------	--

(continúa)

Subflujos (continuación)	El usuario puede seleccionar entre las siguientes opciones: "Nueva consulta", "Servicios" y "Salir". Si el usuario presiona "Nueva consulta", se continúa con el subflujo <i>Consultar Estado</i> (S-6). Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", sale del sistema.
------------------------------------	--

Las excepciones del caso de uso son las siguientes:

Excepciones	<i>E-1 Información incompleta:</i> falta llenar información indispensable, la ciudad de origen o de destino. Se le vuelve a pedir al usuario la información. <i>E-2 Información inválida:</i> una de las entradas de la solicitud es incorrecta.
--------------------	--

HACER RESERVACIÓN

El caso de uso *Hacer Reservación* se instancia a partir de la pantalla de servicios (P-2), una vez se haya validado el usuario y seleccionado el botón correspondiente.

El caso de uso se describe a continuación, excluyendo los subflujos y excepciones que se describen más adelante.

Caso de uso	<i>Hacer Reservación.</i>
Actores	<i>Usuario, Base de Datos de Reservaciones.</i>
Tipo	Básico.
Propósito	Permitir a un usuario hacer reservaciones con el sistema de reservaciones de vuelo.
Resumen	El usuario inicia este caso de uso. Ofrece funcionalidad para crear, obtener, modificar y eliminar reservaciones de vuelos con el sistema de reservaciones.
Precondiciones	Se debe haber ejecutado anteriormente el caso de uso <i>Validar Usuario</i> .
Flujo principal	Se ejecuta el caso de uso <i>Validar Usuario</i> . Dependiendo de las opciones seleccionadas por el usuario, se continuará con los diversos subflujos de este caso de uso.

En la figura 6.27 se muestra el diseño de la pantalla para crear u obtener una reservación, si se cuenta con una clave.

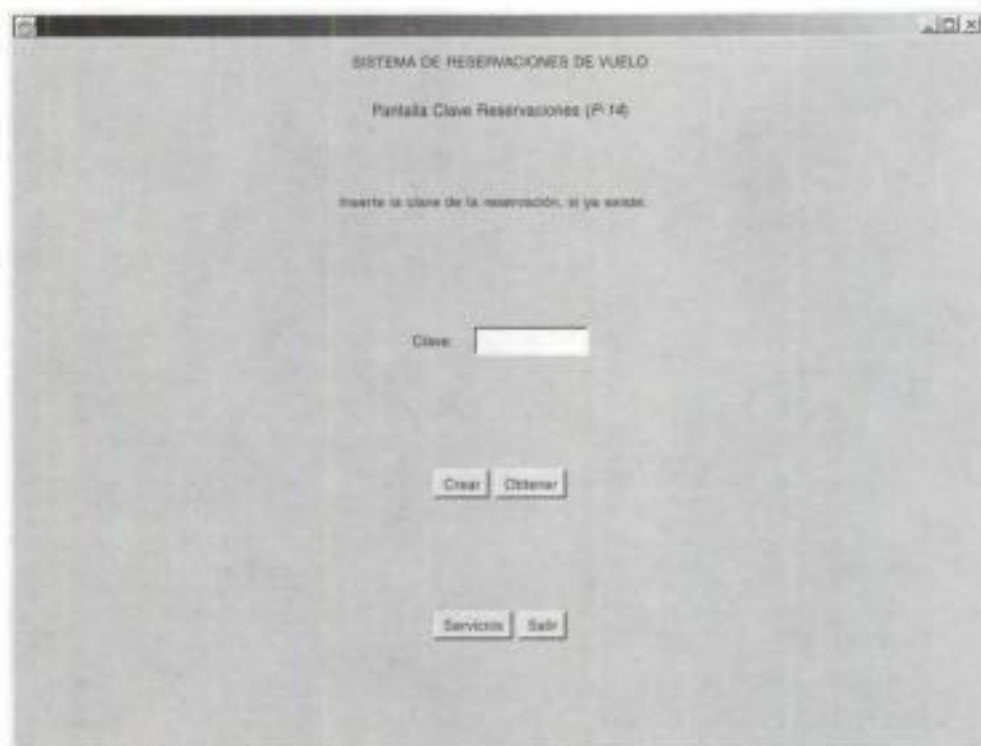


Figura 6.27 Pantalla de inserción de Clave de reservaciones (P-14).

El subflujo *Solicitar Clave Reservación (S-1)* se muestra a continuación:

Subflujos	<i>S-1 Solicitar Clave Reservación.</i> Se presenta al usuario la pantalla "Clave Reserva" (P-14). El usuario puede seleccionar entre las siguientes actividades: "Crear", "Obtener", "Servicios" y "Salir". Si el usuario presiona "Crear" se ejecutará el subflujo <i>Crear Reservación (S-2)</i>. Si el usuario presiona "Obtener" (E-1), se ejecuta el subflujo <i>Obtener Reservación (S-3)</i>. Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", se saldrá del sistema.
------------------	---

En la figura 6.28 se muestra el diseño de la pantalla para la solicitud de reservaciones de vuelos, utilizado por los distintos subflujos.

SISTEMA DE RESERVACIONES DE VUELO
Pantalla Crear Reservas de vuelos (P-15)

Apellido: Nombre:

Número de viajero frecuente:

Aerolínea: Vuelo:

Salida de: Llega a:

☐ Seleccionar asiento Fecha: Clase:

Asiento: ☐ Ventana: ☐ Pasillo: ☒ Comida: ☐ Veg.: ☐ Carne: ☒

Agregar + -

Reservar Servicios Salir

Figura 6.28 Pantalla Crear reservas de vuelos (P-15).

La figura 6.29 ilustra el diseño de la pantalla para el *récord* o registro de reservas de vuelos utilizado por los distintos subflujos.

SISTEMA DE RESERVACIONES DE VUELO
Pantalla Récord Reservas de vuelos (P-16)

Apellido: Nombre:

Clave: Número de viajero frecuente:

Aerolínea: Vuelo: Línea:

Salida de: Llega a: Estado:

Fecha: Hora salida: Hora llegada:

Asiento: No Comida: Comida:

Clase: Tarifa: Estado de pago:

Eliminar Actualizar + -

Nueva reserva Pagar Reembolsar Servicios Salir

Figura 6.29 Pantalla Récord de reservación de vuelos (P-16).

Los demás subflujos se muestran a continuación.

Subflujos (continuación)	S-2 Crear Reservación. Se presenta la pantalla "Crear reservaciones" (P-15), la cual debe llenarse con el apellido y nombre del pasajero, un número de viajero frecuente opcional, la aerolínea, el número de vuelo, la ciudad de origen y de destino, la fecha, la clase, la asignación de asiento y si desea ventana o pasillo, y opcionalmente comida vegetariana o carne. El usuario puede seleccionar entre las siguientes actividades: "Agregar", "+", "-", "Reservar", "Servicios" y "Salir". Si el usuario selecciona "Agregar", el sistema agrega una nueva pantalla "Crear reservaciones" (P-15) para ser llenada por el usuario. Si el usuario selecciona "+", el sistema avanza a la siguiente pantalla de reservación. Si el usuario selecciona "-", el sistema retrocede a la pantalla anterior de reservación. Si el usuario selecciona "Reservar", el sistema acepta la solicitud (E-2, E-3), enviándola a la base de datos del sistema de reservaciones (E-5); se continúa con el subflujo <i>Administrar Reservación</i> (S-4). Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", se saldrá del sistema.
	S-3 Obtener Reservación. El sistema obtiene el récord de reservación de la base de datos de registro. Se continúa con el subflujo <i>Administrar Reservación</i> (S-4).
	S-4 Administrar Reservación. Se presenta la pantalla récord reservación (P-16) con la opción a modificar la información (E-1). El usuario puede seleccionar entre las siguientes opciones: "Eliminar", "Actualizar", "+", "-", "Nueva reservación", "Pagar", "Reembolsar", "Servicios" y "Salir". Si el usuario presiona "Actualizar", se ejecuta el subflujo <i>Actualizar Reservación</i> (S-5). Si el usuario presiona "Eliminar", se ejecuta el subflujo <i>Eliminar Reservación</i> (S-6). Si el usuario selecciona "+", el sistema avanza a la siguiente pantalla de reservación. Si el usuario selecciona "-", el sistema retrocede a la pantalla anterior de reservación. Si el usuario selecciona "Nueva reservación", se continúa con el subflujo <i>Crear Reservación</i> (S-2). Si el usuario selecciona "Pagar", el sistema continúa con el caso de uso <i>Pagar Reservación</i>. Si el usuario selecciona "Reembolsar", el sistema continúa con el caso de uso <i>Pagar Reservación</i>. Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", se saldrá del sistema.

(continúa)

Subflujos (continuación)	S-5 Actualizar Reservación. Se actualiza el récord de reservaciones (E-2, E-3, E-4). Se continúa con el subflujo <i>Administrar Reservación</i> (S-3).
	S-6 Eliminar reservación. Se elimina el récord de reservación (E-5). Se continúa con el subflujo <i>Crear Reservación</i> (S-2).

Las excepciones del caso de uso son las siguientes.

Excepciones	E-1 Récord inválido: no existe el récord especificado. E-2 Información incompleta: falta llenar información indispensable, la ciudad de origen o destino. Se le vuelve a pedir al usuario la información. E-3 Información inválida: una de las entradas de la solicitud es incorrecta. E-4 Reservación sin éxito: no se logró obtener una reservación. E-5 Eliminación reservación sin éxito: no se logró eliminar la reservación.
--------------------	--

PAGAR RESERVACIÓN

El caso de uso *Pagar Reservación* se instancia a partir de la pantalla *Récord reservaciones* (P-16), una vez que se ha seleccionado el botón correspondiente.

El caso de uso se describe a continuación, excluyendo los subflujos y excepciones que se describen más adelante.

Caso de uso	<i>Pagar Reservación.</i>
Actores	<i>Usuario, Base de Datos de Reservaciones.</i>
Tipo	Extensión.
Propósito	Permitir a un usuario pagar reservaciones con el sistema de reservaciones de vuelo.
Resumen	El usuario inicia este caso de uso. Ofrece funcionalidad para pagar y reembolsar pagos de reservaciones de vuelos con el sistema de reservaciones mediante tarjetas de crédito registradas en el sistema.
Precondiciones	Se requiere haber ejecutado anteriormente el caso de uso <i>Validar Usuario</i> y tener ya hecha una reservación mediante el caso de uso <i>Hacer Reservación</i> .
Flujo principal	Se obtiene el registro de tarjeta ejecutando el caso de uso <i>Registrar Tarjeta</i> , subflujo <i>Obtener Registro Tarjeta</i> (S-2). Si la solicitud original fue pagar, se continúa con el subflujo <i>Pagar Reservación</i> (S-1), si fue reembolsar se continúa con el subflujo <i>Reembolsar Pago</i> (S-2).

La figura 6.30 ilustra el diseño de la pantalla para el pago de reservaciones de vuelos, utilizando el subflujo *Pagar Reservación (S-1)*.

SISTEMA DE RESERVACIONES DE VUELO

Pantalla Pagar Registro Tarjeta (P-17)

Nombre:

Número de tarjeta (requerido para pagos):

Tipo: Fecha vencimiento:

Tarifa:

Figura 6.30 Pantalla de Pago de reservaciones de vuelos (P-17).

El subflujo *Pagar Reservación (S-1)* se muestra a continuación:

Subflujos	S-1 Pagar Reservación. Se presenta al usuario la pantalla "Pagar registro tarjeta" (P-17), la cual incluye información sobre el nombre tal como aparece en la tarjeta, el número de tarjeta, el tipo de tarjeta, la fecha de vencimiento y la cantidad a pagar (E-1). El usuario podrá seleccionar entre las siguientes actividades: "Pagar", "Servicios" y "Salir". Si la actividad seleccionada es "Pagar", el sistema utiliza los datos de la tarjeta registrada por el usuario y le envía una pantalla de confirmación (E-2). Se continúa con el caso de uso <i>Hacer Reservación</i>, subflujo <i>Solicitar Clave Reservación (S-1)</i>. Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", se sale del sistema.
------------------	--

En la figura 6.31 se muestra el diseño de la pantalla para el pago de reservaciones de vuelos, utilizando el subflujo *Reembolsar Pago (S-2)*.

Figura 6.31 Pantalla de Reembolso de reservaciones de vuelos (*P-18*).

El subflujo *Reembolsar Pago (S-2)* se muestra a continuación:

Subflujos (continuación)	S-2 Reembolsar Pago. Se presenta al usuario la pantalla "Reembolsar registro tarjeta" (<i>P-18</i>), la cual incluye información del nombre tal como aparece en la tarjeta, el número de tarjeta, el tipo de tarjeta, la fecha de vencimiento y la cantidad a reembolsar (<i>P-1</i>). El usuario podrá seleccionar entre las siguientes actividades: "Reembolsar", "Servicios" y "Salir". Si la actividad seleccionada es "Reembolsar", el sistema utiliza los datos de la tarjeta registrada por el usuario y envía una pantalla de confirmación al usuario (<i>E-3</i>, <i>E-4</i>). Se continúa con el caso de uso <i>Hacer Reservación</i>, Subflujo <i>Solicitar Clave Reservación (S-1)</i>. Si la actividad seleccionada es "Servicios", se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", se sale del sistema.
------------------------------------	--

Las excepciones del caso de uso son las siguientes:

Excepciones	<i>E-1 Récord inválido:</i> no existe el récord especificado. El usuario deberá insertar los datos de la tarjeta. <i>E-2 Pago inválido:</i> el pago no tuvo éxito o la información de pago está incompleta. <i>E-3 Pago inexistente:</i> la reservación aún no ha sido pagada. <i>E-4 Reembolso inválido:</i> no se pudo hacer el reembolso del pago.
--------------------	---

6.5 Modelo del dominio del problema

El **modelo del dominio del problema** define un modelo de clases común para todos los involucrados en el modelo de requisitos, tanto analistas como clientes. Este modelo de clases consiste en los objetos del dominio del problema, o sea objetos que tienen una correspondencia directa en el área de la aplicación. Como los usuarios y clientes deben reconocer todos los conceptos, se puede desarrollar una terminología común al razonar sobre los casos de uso y, por lo tanto, disminuir la probabilidad de malos entendidos entre el analista y el usuario. Al discutirlo, se evolucionará el modelo del dominio del problema. Una técnica utilizada cuando se trabaja con tal modelo, es darle al cliente un papel y un lápiz, y pedirle que dibuje su visión del sistema.

Históricamente, el modelo del dominio del problema se utilizaba como el modelo fundamental para la especificación de requisitos en muchas de las metodologías de ingeniería de software orientada a objetos de primera generación (ver capítulo 3). Sin embargo, dadas sus limitaciones que impedían obtener los requisitos funcionales de un sistema, el modelo del dominio del problema dejó de ser la base única para el desarrollo completo del sistema y pasó a ser un elemento adicional en la especificación de éste, como en el modelo de casos de uso. El propósito principal del dominio del problema en el modelo de requisitos de nuestra metodología es formar una base común de entendimiento del desarrollo y no definir el sistema completo. Por tanto, se pueden aprovechar algunas heurísticas de los métodos anteriores para identificar los objetos en el dominio del problema, con lo que se logrará un **glosario o diccionario de clases** que sirva como común denominador a todos los componentes del sistema, incluyendo a las personas involucradas a lo largo del desarrollo. A diferencia de los métodos anteriores, el modelo del dominio del problema no debe ser demasiado extenso, ya que se realizarán varios grados de refinamiento después. Y aunque es suficiente describir el dominio del problema en términos de objetos o clases, es posible refinar todavía más mediante la inclusión de asociaciones, atributos, herencia y operaciones, siempre y cuando esto ayude a comprender mejor el problema, y que no se vuelva un esfuerzo demasiado grande durante esta etapa. Se debe tener cuidado con hacer demasiado trabajo al inicio, ya que esto puede dificultar su modificación posterior durante el modelo de análisis. La experiencia muestra que muchos, si no todos, de los objetos del dominio podrán aparecer durante éste. Sin embargo, pueden haber cambios durante éste, incluyendo la eliminación de clases identificadas en esta etapa, o incluso, la incorporación de clases adicionales.

En esta sección describiremos cómo identificar las clases del dominio del problema junto con aspectos adicionales, como asociaciones y atributos. Lo que definitivamente no se hará aquí, y que era parte esencial de las metodologías anteriores, es identificar la herencia y las operaciones en esta etapa. La herencia y, en especial, las operaciones de un sistema son los aspectos de mayor complejidad, algo que nosotros elaboraremos de manera muy cuidadosa durante el diseño del sistema.

6.5.1 Identificación de clases

La **identificación de clases** del dominio del problema, se obtiene principalmente de algún documento textual que describa el sistema. Aunque se pudie-

ra tomar como punto de partida los documentos desarrollados para el modelo de casos de uso, a menudo la descripción original del problema es suficiente. Se comienza este proceso a partir de la identificación de las clases *candidatas*, explícitas o implícitas, a las que se refiera la descripción del problema. Para ello, se extraen todos los sustantivos de la descripción del problema o de algún otro documento similar, de acuerdo con las siguientes consideraciones:

- ▶ Los sustantivos en la descripción del problema son los posibles candidatos a clases de objetos. Por ejemplo en "un sistema de reservaciones que vende boletos para funciones a varios teatros", las clases candidatas serían, *sistema de reservaciones*, *boletos*, *función* y *teatro*.
- ▶ Se deben identificar las entidades físicas al igual que las conceptuales.
- ▶ No se debe tratar de diferenciar entre las clases y los atributos.
- ▶ Dado que no todas las clases se describen de manera explícita, siendo algunas implícitas en la aplicación, será necesario añadir clases que puedan ser identificadas por nuestro conocimiento del área.
- ▶ Se deben revisar los pronombres en la descripción del problema, para asegurar que no se haya perdido ningún sustantivo descrito de forma implícita.
- ▶ Para facilitar la identificación de clases, se subrayan todos los sustantivos de la descripción del problema.

En el caso del sistema de reservaciones de vuelos, partimos de la descripción del problema y subrayamos todos los sustantivos, con sus complementos adjetivos, como se ve a continuación:

El Sistema de reservaciones de vuelos es un sistema que permite al usuario hacer consultas y reservaciones de vuelos, además de poder comprar los boletos aéreos en forma remota, sin la necesidad de recurrir a un agente de viajes. Se desea que el sistema de reservaciones sea accesible a través de Internet (World Wide Web).

El sistema presenta en su pantalla principal un mensaje de bienvenida que describe los servicios ofrecidos junto con la opción para registrarse por primera vez, o si ya se está registrado, poder utilizar el sistema de reservaciones de vuelos.

Este acceso se da por medio de la inserción de un login previamente especificado y un password previamente escogido y que debe validarse.

Una vez registrado el usuario, y después de haberse validado el registro y la contraseña del usuario, se pueden seleccionar de las siguientes actividades:

Consulta de vuelos
Reservación de vuelos
Compra de boletos

(continúa)

(continuación)

La consulta de vuelos se puede hacer de tres maneras diferentes:

Horarios de vuelos

Tarifas de vuelos

Estado de vuelo

La consulta según horario muestra los horarios de las diferentes aerolíneas que dan servicio entre dos ciudades.

La consulta según tarifas muestra los diferentes vuelos entre dos ciudades dando prioridad a su costo.

La información de vuelo se utiliza principalmente para consultar el estado de algún vuelo, incluyendo información de disponibilidad de asientos y, en el caso de un vuelo para el mismo día, si el vuelo está a tiempo.

Se puede incluir preferencias en las búsquedas, como fecha y horario deseado, categoría de asiento, aerolínea y si se desea sólo vuelos directos.

La reservación de vuelo permite al cliente hacer una reservación para un vuelo particular, especificando la fecha y horario, bajo una tarifa establecida. Es posible reservar un itinerario compuesto de múltiples vuelos, para uno o más pasajeros, además de poder reservar asientos.

El pago permite al cliente, dada una reservación de vuelo previa y una tarjeta de crédito válida, adquirir los boletos aéreos.

Los boletos serán posteriormente enviados al cliente, o estarán listos para ser recogidos en el mostrador del aeropuerto antes de la salida del primer vuelo.

Es necesario estar previamente registrados con un número de tarjeta de crédito válida para poder hacer compras de boletos, o de lo contrario, proveerla en el momento de la compra.

Además de los servicios de vuelo, el usuario podrá en cualquier momento acceder, modificar o cancelar su propio registro, todo esto después de haber sido el usuario validado en el sistema.

A partir de estos sustantivos se prepara una lista inicial de clases candidatas, como se muestra en la tabla 6.1. Se deben excluir clases repetidas, manteniendo todos los nombres en singular.

SELECCIÓN DE CLASES

A partir de las clases candidatas, se deben seleccionar las clases relevantes, tomando en cuenta las siguientes consideraciones:

Tabla 6.1 Clases candidatas para el sistema de reservaciones de vuelo identificadas en la descripción del problema.

Clases candidatas		
Sistema de reservaciones de vuelo	Login	Información
Sistema	Email	Asiento
Usuario	Password	Día
Consulta	Registro	Hora
Reservación	Actividad	Preferencia
Vuelo	Consulta de vuelos	Búsqueda
Boleto aéreo	Reservación de vuelos	Fecha
Agente de viajes	Compra de boletos	Categoría de asiento
Sistema de reservaciones	Horario de vuelos	Vuelo directo
Internet	Tarifa de vuelos	Cliente
World Wide Web	Información de vuelo	Itinerario
Pantalla principal	Horario	Pasajero
Mensaje de bienvenida	Aerolínea	Pago
Servicios	Ciudad	Tarjeta de crédito
Opción	Tarifa	Boleto
Reservaciones de vuelos	Costo	Mostrador del aeropuerto
Acceso	Estado	Número de tarjeta de crédito
	Disponibilidad	

- Todas las clases deben tener sentido en el área de la aplicación, la relevancia del problema debe ser el único criterio para la selección.
- Como regla general, se deben escoger los nombres de las clases con cuidado, que no sean ambiguos y que mejor describan el problema. Éste es uno de los procesos más difíciles. Los nombres deben establecerse con un formato consistente (e.g., nombres en singular).
- Durante esta etapa, no hay que preocuparse por la asociación, agregación o herencia. Primero hay que tener las clases específicas correctas para no suprimir detalles en el intento de ajustarse a estructuras preconcebidas.
- Ante la duda, se deben conservar las clases, ya que posteriormente siempre habrá oportunidad de eliminarlas.
- Eliminar clases redundantes, si expresan la misma información. La clase más descriptiva debe ser guardada.
- Eliminar clases irrelevantes, que tienen poco o nada que ver con el problema. Esto requiere de juicio, porque en un contexto, una clase puede ser importante, mientras que en otro, la clase podría no serlo.
- Se deben clarificar las clases imprecisas. Algunas pueden tener bordes mal definidos o demasiado generales.

- ▶ Es necesario eliminar las clases que debieran ser atributos más que clases, cuando los nombres corresponden a propiedades más que a entidades independientes.
- ▶ Eliminar las clases que debieran ser roles más que clases, cuando los nombres corresponden a la función que tienen las clases más que a las entidades independientes.
- ▶ Suprimir las clases que debieran ser operaciones más que clases, si las entidades representan operaciones que se aplican a los objetos y no a entidades manipuladas por sí mismas.
- ▶ Hay que eliminar del análisis las clases que corresponden a construcciones de implementación, si están alejadas del mundo real. No se necesita una clase para representar una entidad física. Por ejemplo: *subrutinas*, *listas* y *arreglos*, son clases típicas de implementación; *Internet* y *World Wide Web* son el medio de implementación del sistema.
- ▶ Se deben eliminar clases que correspondan a aspectos de interfaces de usuario y no de la aplicación.
- ▶ Se deben eliminar clases que correspondan a todo un sistema completo.
- ▶ Suprimir clases que correspondan a actores del sistema.
- ▶ Se deben agregar clases implícitas que no aparezcan en la descripción del problema.

Se tomarán algunas clases candidatas del sistema de reservaciones de vuelo identificadas anteriormente y se seleccionarán las que mejor se apliquen a nuestro problema, como se ve a continuación:

- ▶ Clases redundantes: *Cliente* y *Usuario*, ambos términos pueden ser equivalentes, pero en el caso del sistema de reservaciones, consideramos que *Usuario* es más descriptivo por ser la persona que utiliza el sistema.
- ▶ Clases irrelevantes: *Mostrador del Aeropuerto*, *Agente de Viajes* y *Boleto Aéreo*. Si el sistema generara o se refiriera a un boleto aéreo, esta clase se mantendría.
- ▶ Clases imprecisas: *Sistema*, *Servicios*, *Actividad*, *Preferencia*, *Búsqueda*, *Información*, *Estado*, *Disponibilidad*, *Opción*, *Acceso* e *Itinerario* son clases imprecisas. Durante la introducción de herencia puede que sea necesario una clase para compartir aspectos comunes a ambas clases.
- ▶ Nombres de clases: *Aeropuerto* en lugar de *Ciudad*.
- ▶ Clases que son atributos: *Número de tarjeta de crédito* es un atributo de *Tarjeta de crédito*, *Categoría de asiento* (*Asiento*), *Información de vuelo* (*Vuelo*) y *Horario de vuelo* (*Vuelo*).
- ▶ Clases que son operaciones: *Consulta*, *Pago* y *Reservaciones*.
- ▶ Clases de interfaces de usuario: *Mensaje de Bienvenida* y *Pantalla Principal*.
- ▶ Clases del sistema completo: *Sistema de Reservaciones*.
- ▶ Clases de actores: *Cliente*.

La tabla 6.2 muestra las modificaciones a las clases candidatas originales.

Tabla 6.2 Clases candidatas seleccionadas para el análisis de requisitos de reservas de vuelo.

Clases candidatas	Modificación
Sistema de reservas de vuelo	Eliminada (sistema completo)
Sistema	Eliminada (imprecisa)
Usuario	Eliminada (actor)
Consulta	Eliminada (operación)
Reserva	Eliminada (operación)
Vuelo	
Boleto aéreo	Eliminada (irrelevante)
Agente de viajes	Eliminada (irrelevante)
Sistema de reservas	Eliminada (sistema completo)
Internet	Eliminada (implementación)
<i>World Wide Web</i>	Eliminada (implementación)
Hoja principal	Eliminada (interface)
Mensaje de bienvenida	Eliminada (interface)
Servicios	Eliminada (imprecisa)
Opción	Eliminada (imprecisa)
Reservaciones de vuelos	Renombrada: Reservación
Acceso	Eliminada (imprecisa)
Login	Eliminada (atributo)
Email	Eliminada (atributo)
<i>Password</i>	Eliminada (atributo)
Registro	Renombrada: RegistroUsuario
Actividad	Eliminada (imprecisa)
Consulta de vuelos	Eliminada (operación)
Reservación de vuelos	Eliminada (operación)
Pago de boletos	Eliminada (operación)
Horario de vuelos	Eliminada (duplicada con horario)
Tarifa de vuelos	Eliminada (duplicada con tarifa)
Información de vuelo	Eliminada (atributo)
Horario	
Aerolínea	
Ciudad	Renombrada: Aeropuerto

(continúa)

Tabla 6.2 Continuación

Tarifa	
Costo	Eliminada (redundante)
Estado	Eliminada (imprecisa)
Disponibilidad	Eliminada (imprecisa)
Información	Eliminada (imprecisa)
Asiento	
Día	Eliminada (atributo)
Hora	Eliminada (atributo)
Preferencia	Eliminada (imprecisa)
Búsqueda	Eliminada (operación)
Fecha	Eliminada (atributo)
Categoría de asiento	Eliminada (atributo)
Vuelo directo	Eliminada (atributo)
Cliente	Eliminada (redundante y actor)
Itinerario	Eliminada (imprecisa)
Pasajero	
Compra	Eliminada (operación)
Tarjeta de crédito	Renombrada: RegistroTarjeta
Boleto	Eliminada (irrelevante)
Mostrador del aeropuerto	Eliminada (irrelevante)
Número de tarjeta de crédito	Eliminada (atributo)

Las clases identificadas se muestran en la tabla 6.3. Note que se agregan dos nuevas clases: *Avión* y *ViajeroFrecuente*, que no aparecían en la descripción del problema. Esto se hizo para lograr un dominio más completo. En general, distintos analistas identificarán listas similares de clases, aunque siempre con alguna variación.

Tabla 6.3 Clases identificadas para el sistema de reservaciones de vuelo.

Clases Identificadas	
Vuelo	Tarifa
Reservación	Asiento
RegistroUsuario	Pasajero
Horario	RegistroTarjeta
Aerolínea	Avión
Aeropuerto	ViajeroFrecuente

DIAGRAMA DE CLASES

Después de identificar y seleccionar las clases, se debe construir el **diagrama de clases** para el dominio del problema. Este diagrama se muestra en la figura 6.32 y puede ayudar a identificar clases adicionales, además servirá de base para encontrar las atributos y asociaciones entre ellas.

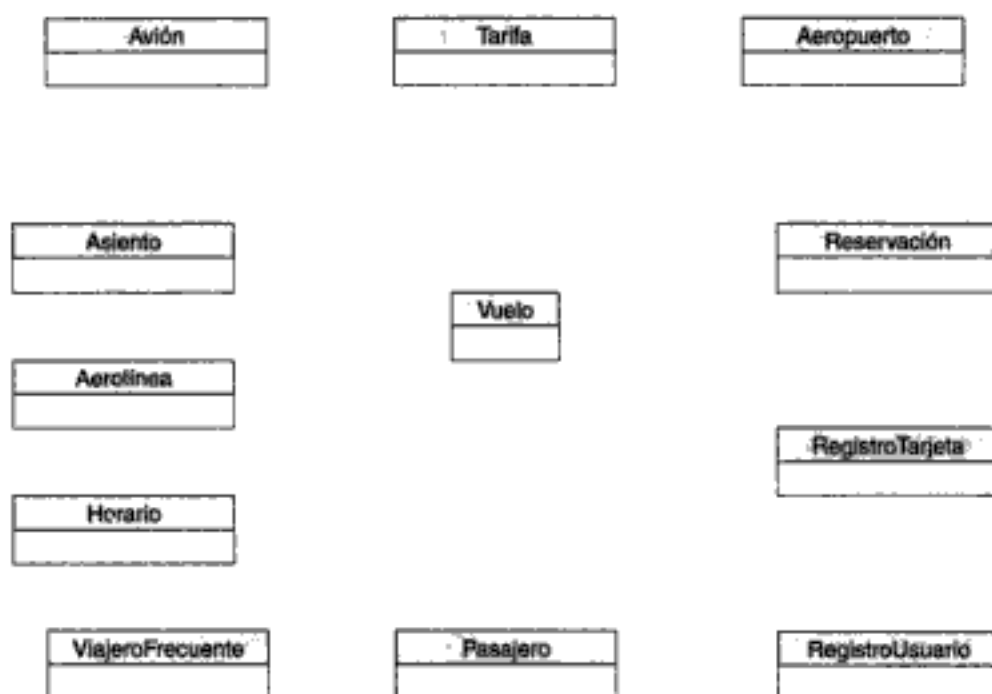


Figura 6.32 Diagrama de clases identificadas para el sistema de reservas de vuelo.

Aunque se puede parar aquí el proceso de desarrollo del dominio del problema, continuaremos con la identificación de asociaciones y atributos para el sistema de reservas de vuelo.

6.5.2 Identificación de asociaciones

El proceso de **identificación de asociaciones** es bastante similar al de identificación de clases, sólo que en lugar de sustantivos buscamos frases que relacionen a los sustantivos de clases ya identificadas. En general, este proceso ya no es necesario como parte del modelo de casos de uso, dado que las asociaciones y operaciones del sistema serán identificadas durante el modelo de diseño. Sin embargo, aquí se dará un pequeño ejemplo de la identificación de asociaciones con base en nuestro conocimiento del dominio del problema. (En cierta manera, se escogió como ejemplo el desarrollo de un sistema de reservas de vuelos, por ser un dominio con el cual la mayoría de los lectores pueden identificarse.)

DIAGRAMA DE CLASES CON ASOCIACIONES

En lugar de partir de la descripción del problema o los documentos de casos de uso, simplemente identificamos nuestras propias frases correspondientes al dominio del problema del sistema de reservaciones, como se observa en la tabla 6.4. Este proceso de identificación es sencillo cuando el problema es muy limitado y el dominio es fácil de analizar. De lo contrario, se requiere un proceso de identificación mucho más extenso, como veremos en la etapa de diseño.

Tabla 6.4 Asociaciones identificadas para relacionar clases en el dominio del problema.

Asociaciones identificadas

- Un vuelo requiere reservaciones.
- Un vuelo se dirige a un aeropuerto.
- Un vuelo contiene tarifas.
- Un vuelo se efectúa en un avión.
- Un vuelo tiene asientos.
- Un vuelo pertenece a una aerolínea.
- Un vuelo tiene un horario.
- Un pasajero puede acumular millas como viajero frecuente.
- Un pasajero efectúa reservaciones.
- Una reservación requiere de un registro de tarjeta de crédito.
- Un registro de tarjeta pertenece a un registro de usuario.

Después de haber identificado las asociaciones, se debe construir una versión del diagrama de clases que incluya estas asociaciones, como se muestra en la figura 6.33.

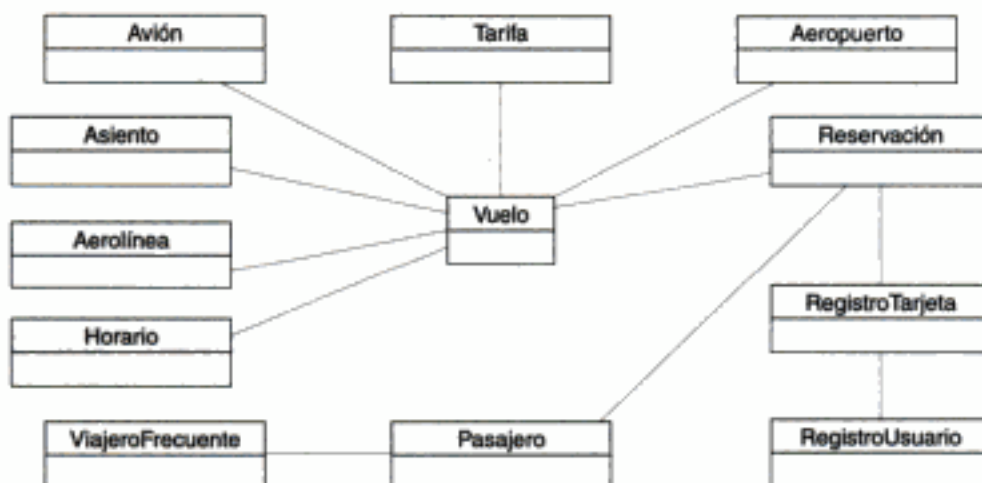


Figura 6.33 Diagrama de clases con asociaciones entre clases identificadas. Se omiten los nombres de las asociaciones.

DIAGRAMA DE CLASES CON ROLES

Después de haber hecho el diagrama de clases con asociaciones, se puede construir una versión del diagrama de clases incluyendo roles. Como se explicó en el capítulo 4, el nombre del rol describe la función que desempeñará una clase en la asociación desde el punto de vista de la otra clase. Si hay una sola asociación entre un par de clases, y el nombre de la clase describe adecuadamente su rol, se pueden omitir los nombres del mismo.

Para tal propósito, modificamos levemente las asociaciones identificadas anteriormente, como se muestran en la tabla 6.5.

Tabla 6.5 Asociaciones identificadas en el dominio del problema

Asociaciones identificadas con roles

Un vuelo contiene reservaciones.
Un vuelo llega a un aeropuerto de destino.
Un vuelo tiene un aeropuerto de origen.
Un vuelo puede hacer escalas en otros aeropuertos.
Un vuelo contiene tarifas de ida y vuelta.
Un vuelo tiene tarifas solamente de ida.
Un vuelo se efectúa en un avión.
Un vuelo tiene asientos.
Un vuelo pertenece a una aerolínea.
Un vuelo tiene un horario de llegada.
Un vuelo tiene un horario de salida.
Un pasajero puede acumular millas como viajero frecuente.
Un pasajero efectúa reservaciones.
Una reservación requiere de un registro de tarjeta de crédito.
Un registro de tarjeta pertenece a un registro de usuario.
Un vuelo puede tener múltiples conexiones.

Se extiende el diagrama de clases con asociaciones para incluir roles, como se muestra en la figura 6.34.

DIAGRAMA DE CLASES CON MULTIPLICIDAD

Después de haber hecho el diagrama de clases con asociaciones y roles, se puede construir una versión del diagrama de clases incluyendo multiplicidad, la cual se determina para cada asociación.

Para tal propósito, modificamos levemente las asociaciones identificadas anteriormente, como se muestran en la tabla 6.6. Note que la multiplicidad, al igual que las relaciones entre las clases, son bastante subjetivas, y pueden variar entre analistas. Dentro de esa subjetividad, todas deben transmitir un dominio similar del problema.

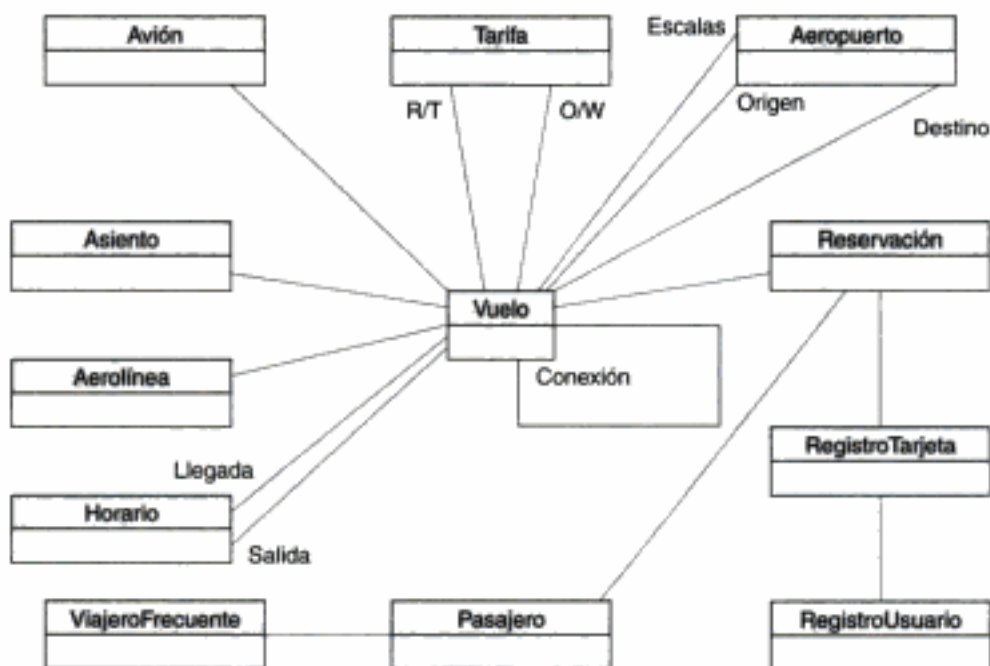


Figura 6.34 Diagrama de clases con asociaciones y roles entre clases identificadas. Se omiten los nombres de las asociaciones. (O/W significa viaje sencillo, vuelo de ida o *one way*, mientras que R/T significa viaje redondo o *round trip*.)

Tabla 6.6 Asociaciones identificadas con roles y multiplicidad para relacionar clases en el dominio del problema.

Asociaciones identificadas con roles y multiplicidad

- Un vuelo contiene múltiples reservaciones.
- Una reservación se relaciona con múltiples vuelos.
- Un vuelo tiene un aeropuerto de destino.
- Un vuelo tiene un aeropuerto de origen.
- Un vuelo puede hacer escalas en múltiples aeropuertos.
- Un vuelo contiene múltiples tarifas de ida y vuelta.
- Un vuelo contiene múltiples tarifas solamente de ida.
- Un vuelo se efectúa en múltiples aviones (dependiendo del día).
- Múltiples vuelos se efectúan en un mismo avión (en diferente momento).
- Un vuelo tiene múltiples asientos.
- Un vuelo pertenece a una aerolínea.
- Un vuelo tiene múltiples horarios de llegada (correspondiendo a diferentes destinos).
- Un vuelo tiene múltiples horarios de salida (correspondiendo a diferentes escalas).
- Un pasajero puede acumular millas en múltiples cuentas de viajero frecuente.
- Un pasajero efectúa múltiples reservaciones.
- Múltiples pasajeros pueden pertenecer a una misma reservación.
- Múltiples reservaciones pueden requerir de un mismo registro de tarjeta de crédito.
- Un registro de tarjeta pertenece a un registro de usuario.
- Un vuelo puede tener múltiples conexiones.

Se extiende el diagrama de clases con asociaciones y roles, para incluir multiplicidad, como se muestra en la figura 6.35.

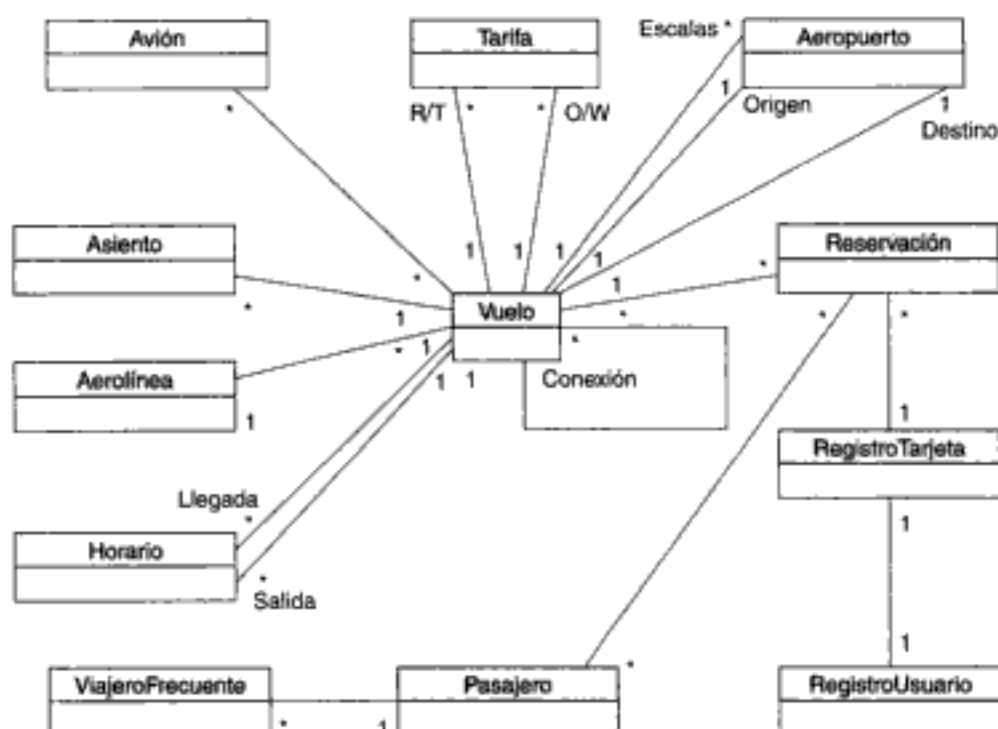


Figura 6.35 Diagrama de clases con asociaciones, roles y multiplicidad entre clases identificadas. Se omiten los nombres de las asociaciones.

6.5.3 Identificación de atributos

De manera similar a las asociaciones, se pueden determinar los atributos del dominio del problema. Este proceso tiene una complejidad similar al de identificación de las asociaciones. Sin embargo, identificar atributos puede resultar más difícil si se hace a través de un proceso de búsqueda a partir de la descripción del problema, e incluso del documento de casos de uso. En lugar de esto, simplemente se identifican nuestros propios atributos para las distintas clases identificadas en el dominio del problema del sistema de reservaciones, como se muestran en la tabla 6.7.

Nuevamente, este proceso de identificación es sencillo cuando el problema es limitado y el dominio es fácil de analizar. De lo contrario, se requiere un proceso de identificación mucho más extenso como veremos en la etapa de diseño. No es necesario listar todos los atributos, solamente los atributos más relevantes, omitiendo atributos menores.

Tabla 6.7 Atributos identificados para las clases del sistema de reservaciones de vuelo.

Clases	Atributos
<i>Vuelo</i>	<i>Número.</i>
<i>Reservación</i>	<i>Clave.</i>
<i>RegistroUsuario</i>	<i>Nombre, Dirección, Colonia, Ciudad, País, Código Postal, Teléfono casa, Teléfono oficina, Fax, Email, Login, Password.</i>
<i>Horario</i>	<i>Día, Hora.</i>
<i>Aerolínea</i>	<i>Nombre.</i>
<i>Aeropuerto</i>	<i>Nombre, Ciudad, País.</i>
<i>Tarifa</i>	<i>Clase, Precio, Impuestos.</i>
<i>Asiento</i>	<i>Fila, Letra.</i>
<i>Pasajero</i>	<i>Nombre.</i>
<i>RegistroTarjeta</i>	<i>Nombre, Número, Expedidor, Vencimiento.</i>
<i>Avión</i>	<i>Fabricante, Modelo.</i>
<i>ViajeroFrecuente</i>	<i>Número, Aerolínea.</i>

DIAGRAMA DE CLASES CON ATRIBUTOS

Se extiende el diagrama de clases con asociaciones, roles y multiplicidad, para incluir los atributos principales, como se muestra en la figura 6.36.

6.5.4 Diccionario de clases

El **diccionario de clases** o **diccionario de datos**, describe textualmente las clases identificadas durante el modelo del dominio del problema. Este diccionario sirve como un glosario de términos y se muestra a continuación.

- ▶ **Vuelo:** se denomina por medio de un número. El vuelo tiene como origen un aeropuerto en una ciudad y tiene como destino un aeropuerto de otra ciudad. Un vuelo puede tener múltiples escalas, y diversos vuelos se relacionan por medio de conexiones. El vuelo pertenece a una aerolínea y puede operar varios días a la semana, con un horario de salida y otro de llegada.
- ▶ **Reservación:** para poder tomar un vuelo, es necesario contar con una reservación previa, la cual debe pagarse antes de una fecha límite, incluso el mismo día del vuelo. Una reservación puede hacerse para múltiples vuelos y distintos pasajeros. La reservación cuenta con una clave que corresponde a un récord de reservación particular.
- ▶ **RegistroUsuario:** para poder utilizar el sistema de reservaciones, el usuario debe estar registrado en él. El registro contiene información acerca del usuario como nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa, teléfono de oficina, fax, email, *login* y *password*.

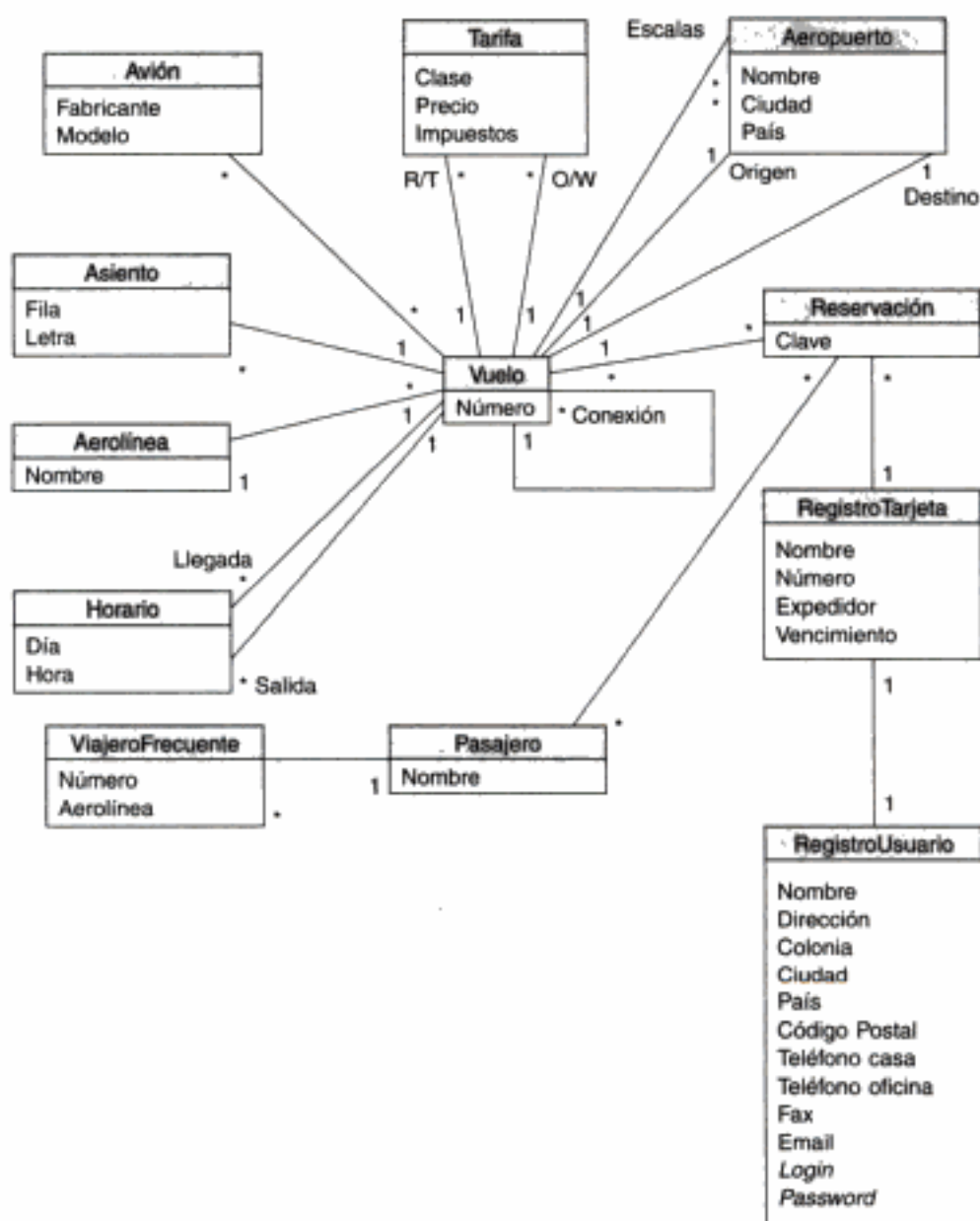


Figura 6.36 Diagrama de clases con asociaciones, roles, multiplicidad y atributos para clases identificadas. Se omiten los nombres de las asociaciones y sólo se muestran los atributos más relevantes.

- **Horario:** el horario de un vuelo se determina por su hora de salida y de llegada durante los días que opera.
- **Aerolínea:** la aerolínea provee varios vuelos entre diferentes ciudades bajo diversos horarios. La aerolínea se identifica por un nombre.
- **Aeropuerto:** el aeropuerto sirve como origen, destino y escalas de un vuelo. El aeropuerto se encuentra en una ciudad de un país determinado.
- **Tarifa:** un mismo vuelo puede contar con diferentes tarifas, que varían según la clase de boleto, si son de viaje sencillo o viaje redondo, y dependiendo de las restricciones y ofertas existentes.

- ▶ **Asiento:** una reservación de vuelo puede incluir la asignación de asiento, especificada mediante una fila y un número. El número de asientos disponibles en un vuelo dependen del tipo de avión que opere ese día.
- ▶ **Pasajero:** para hacer una reservación, se requiere dar el nombre del pasajero. Varios pasajeros pueden aparecer bajo una sola reservación.
- ▶ **RegistroTarjeta:** para pagar con una tarjeta de crédito, se debe tener un registro de tarjeta. El registro contiene información acerca de la tarjeta, incluyendo nombre, número, expedidor y vencimiento. La tarjeta está ligada a un registro de usuario.
- ▶ **Avión:** un vuelo en una fecha determinada se hace en un tipo de avión en particular. El tipo de avión define la cantidad máxima de pasajeros que pueden viajar en ese vuelo para esa fecha.
- ▶ **ViajeroFrecuente:** el pasajero tiene la opción de acumular millas para un vuelo, si cuenta con una tarjeta de viajero frecuente de la aerolínea correspondiente.

6.5.5 Identificación de Módulos

El modelo del **dominio del problema**, puede hacerse bastante complejo en el caso de un sistema de gran tamaño, para lo cual es necesario separar las clases en **módulos**. De tal manera, el modelo completo se dividiría en una colección de módulos, donde cada módulo es una agrupación lógica de clases y sus asociaciones correspondientes. Para el sistema de reservaciones de vuelo, se pueden identificar dos módulos principales del dominio del problema, de acuerdo con la relación lógica entre las clases. Estos módulos son, *Registro*, que contiene las clases que guardan información sobre el usuario del sistema; y *Servicios*, que contiene las clases que guardan información sobre los vuelos, pasajeros y reservaciones. En otras palabras, las clases para el módulo de registro se relacionan con la utilización del sistema ligadas al actor *Base de Datos de Registro*, mientras que las clases para el módulo de servicios se relacionan con el propio sistema de reservaciones, ligadas al actor *Base de Datos de Reservaciones*. La razón de separar en dos módulos, va muy de la mano con la existencia de estos dos actores secundarios, ya que al corresponder cada actor secundario a una base de datos, los módulos afianzan esta correspondencia. Sin embargo, esto no tiene por qué ser realmente así, pudiendo existir un solo módulo para un sistema con múltiples actores secundarios, o incluso varios módulos por cada actor secundario. A continuación describimos los módulos para el sistema de reservaciones de vuelo.

SERVICIOS

En la figura 6.37 se muestran las clases pertenecientes al módulo de *Servicios* del sistema de reservaciones.

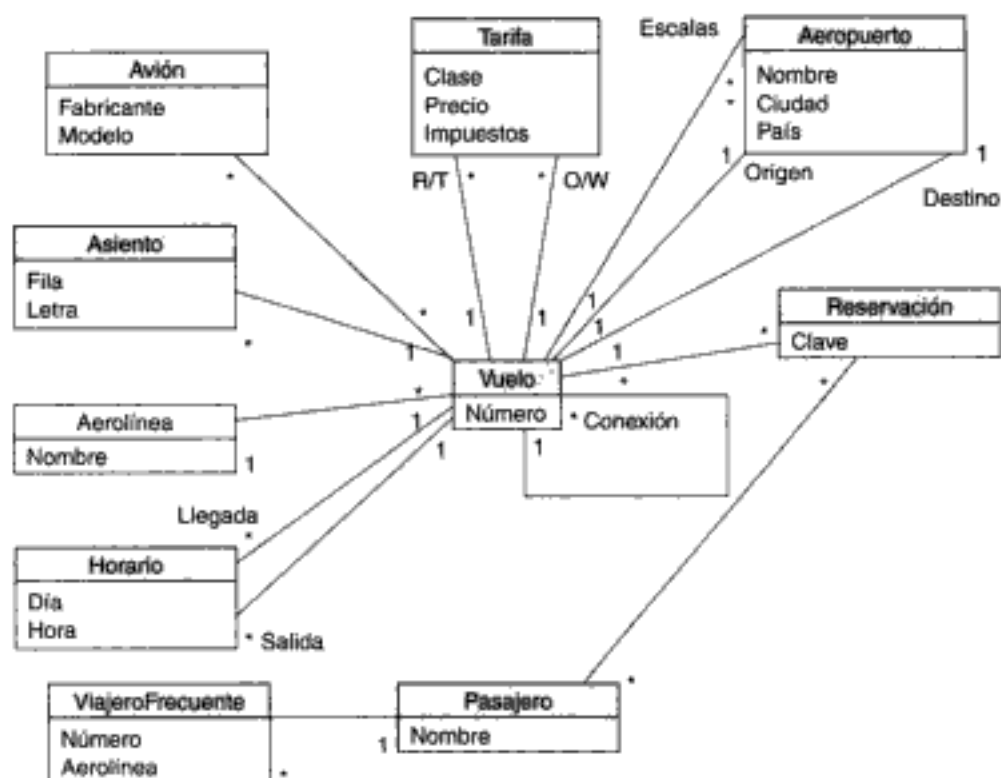


Figura 6.37 Diagrama de clases para el módulo de *Servicios* del sistema de reservas de vuelo.

REGISTRO

En la figura 6.38 se muestra las clases pertenecientes al módulo de *Registro* del sistema de reservaciones.



Figura 6.38 Diagrama de clases para el módulo de *Registro* del sistema de reservas de vuelo.

RESUMEN

En este capítulo se inicia la descripción de la metodología de desarrollo de software basada en casos de uso. En particular, se describe el modelo de requisitos. Se introduce el sistema de reservaciones de vuelos, el cual sirve de ejemplo de la aplicación de la metodología a lo largo del libro. Se describe el modelo de casos de uso, que consiste en actores y casos de uso, como base para la especificación funcional de un sistema. Se identifican y describen los actores y casos de uso más importantes para el sistema de reservaciones de vuelos. Se explican los modelos de interfaces y del dominio del problema de los cuales se obtienen especificaciones adicionales; entre ellos se destaca la importancia de un modelo de requisitos que separa los tres aspectos anteriores.

REFERENCIAS

1. Jacobson, I., Christensen, M., Jonsson, P., Oversand, G., "Object-Oriented Software Engineering: A Use Case Driven Approach, Addison-Wesley, 1992. <http://www.rational.com>.
2. <http://www.sabre.com>
3. <http://www.galileo.com>
4. <http://www.worldspan.com>
5. <http://www.amadeus.com>
6. <http://www.travelocity.com>
7. <http://www.expedia.co>

Modelo de análisis

Cuando ya se ha desarrollado y aceptado el modelo de requisitos, se inicia el desarrollo del modelo de análisis siguiendo el modelo de casos de uso,¹ cuyo objetivo es comprender y generar una arquitectura de objetos para el sistema con base en lo especificado en el modelo de requisitos. Durante esta etapa no se considera el ambiente de implementación, modelando el sistema bajo condiciones ideales. Tarde o temprano el sistema tendrá que adaptarse a las condiciones de implementación deseadas, lo que se hará durante el modelo de diseño.

Es importante enfatizar que el modelo de análisis es una representación conceptual, correspondiente al problema y modelo de requisitos, en término de clase de objetos. Cada una de estas clases contribuye de manera especial a lograr la arquitectura deseada, como se muestra en la figura 7.1.

Una cualidad importante de la metodología es la *rastreabilidad* (*traceability*) de los modelos, que permite evaluar el impacto de cambios durante el desarrollo. Dado que la mayoría de los sistemas serán modificados a lo largo de su vida, es necesario evaluar los efectos que estos cambios podrían tener sobre etapas posteriores, e incluso anteriores, del desarrollo.

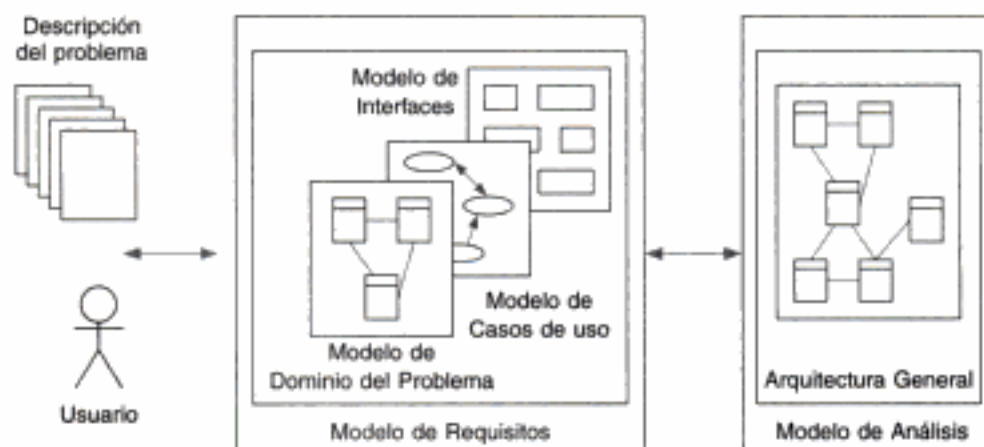


Figura 7.1 El diagrama muestra conceptualmente el modelo de análisis junto con la arquitectura general de objetos, en relación con el modelo de requisitos anteriormente desarrollado.

7.1 Arquitectura de clases

El modelo de análisis tiene como objetivo generar una arquitectura de objetos que sirva como base para el diseño del sistema. Como se discutió en el capítulo 3, dependiendo del tipo de aplicación existen diversas arquitecturas que se pueden utilizar, en este libro nos centraremos en aquellas arquitecturas especialmente diseñadas para el manejo de los *sistemas de información*, las cuales involucran la manipulación de la información guardada en bases de datos a partir de interfaces de usuario.

Las arquitecturas se distinguen según la organización de los objetos de acuerdo a su funcionalidad. Esto es también conocido como la *dimensión* de la arquitectura. Por ejemplo, si existe un grupo de objetos para el manejo de la funcionalidad de la aplicación y otro para interactuar con las entidades externas de la aplicación, como el usuario y las bases de datos, entonces se considera que la arquitectura es de *dos* dimensiones. Por el contrario, si existe un solo grupo de objetos que maneja de manera indistinta la funcionalidad junto con la interacción externa, entonces se considera que la arquitectura es de *una* sola dimensión.

En general, una arquitectura puede incluir cualquier número de dimensiones, algo que depende del tipo de aplicación que se desee desarrollar. En general, el planteamiento es: si se diseña un sistema con cierto número de dimensiones, ¿se obtendría un sistema más estable y fácil de extender que con un número menor o mayor? La respuesta depende de qué tan independiente sean los objetos de un eje de funcionalidad con los demás. Si se cuenta con ejes de funcionalidad completamente ortogonales, algo que es difícil de lograr, el efecto de cambios en una dimensión no debería afectar a las demás dimensiones. Sin embargo, si los grupos de objetos no son lo suficientemente independientes, aún se puede limitar el efecto de los posibles cambios.

En el caso de los sistemas de información, una de las arquitecturas más utilizadas es la de **Modelo, Vista, Control** (MVC - *Model, View, Control*),² popularizada por los ambientes de desarrollo para los lenguajes de programación de Smalltalk (ver capítulo 2). Esta arquitectura se basa en tres dimensiones principales: *Modelo* correspondiente a la *información*, *Vista* correspondiente a la *presentación* o interacción con el usuario y *Control* correspondiente al *comportamiento*, como se ilustra en la figura 7.2.

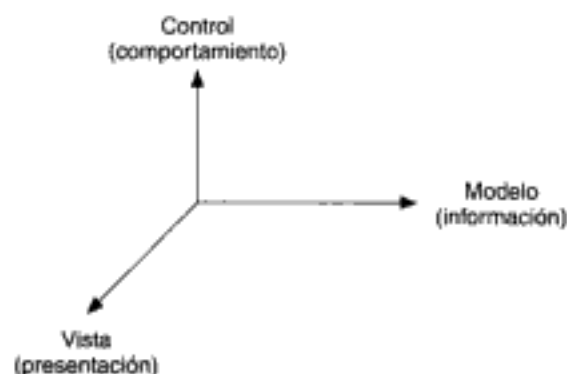


Figura 7.2 Diagrama de tres dimensiones correspondiente a la arquitectura *Modelo, Vista, Control* (MVC).

La vista o presentación de la información corresponde a las interfaces que se le presentan al usuario para el manejo de la información, donde por lo general pueden existir múltiples vistas sobre un mismo modelo. Típicamente la información representa el dominio del problema y se almacena en una base de datos. Por otro lado, el control corresponde a la manipulación de la información a través de sus diversas presentaciones. Aunque existe cierta dependencia entre estas tres dimensiones, se considera que la manera de presentar la información es independiente de la propia información y de cómo se controla ésta. Sin embargo, cada una de ellas probablemente experimente cambios a lo largo del ciclo de vida del sistema, donde el control es el más propenso a ser modificado, seguido de la vista y, finalmente, del modelo. En el modelo de análisis descrito aquí utilizaremos como base la arquitectura *MVC* para capturar estos tres aspectos de la funcionalidad, como se muestra en la figura 7.3.

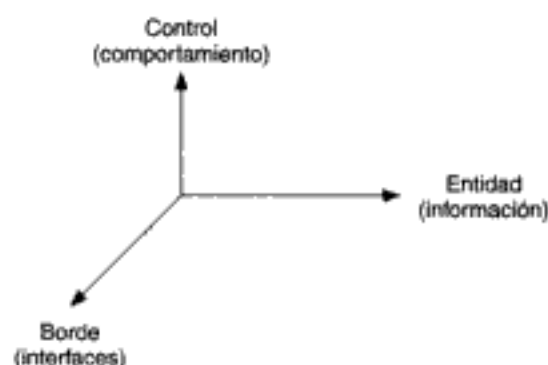


Figura 7.3 Diagrama de tres dimensiones correspondiente a la arquitectura del modelo de análisis, basado en el modelo de casos de uso.

En correspondencia con el modelo *MVC*, la arquitectura para el modelo de análisis se basará en tres tipos o *estereotipos* de objetos correspondientes a las tres dimensiones anteriores. Es importante notar la correspondencia con las tres dimensiones utilizadas durante el modelo de requisitos (ver figura 6.2).

7.1.1 Clases con estereotipos

El tipo de funcionalidad o “la razón de ser” de un objeto dentro de una arquitectura se conoce como su *estereotipo*. Siguiendo la metodología de casos de uso, la arquitectura del sistema para el modelo de análisis se basará en tres estereotipos:

- El estereotipo **entidad** (*entity*) para los objetos que guardan información sobre el estado interno del sistema a corto y largo plazo. Estos objetos corresponden al dominio del problema. Un ejemplo de objeto entidad es un registro de usuario con sus datos y comportamiento asociados.
- El estereotipo **borde** (*boundary*) para objetos que implementan las interfaces del sistema con el mundo externo, correspondientes a todos los actores, incluyendo a aquellos que no son humanos. Un ejemplo de objeto borde es una interface de usuario o pantalla para insertar o modificar información en el registro de usuario. Otro ejemplo es un objeto que se comunica con una base de datos externa al sistema.

- El estereotipo **control** (*control*) para objetos que implementan el comportamiento o control de la lógica de los casos de uso, especificando cuándo y cómo el sistema cambia de estado. Los objetos control modelan la funcionalidad que no se asocia naturalmente con un solo objeto. Un ejemplo típico de objeto control es validar un usuario existente o insertar uno nuevo. Este comportamiento requiere la interacción de múltiples objetos y actores.

La notación de UML para un estereotipo se muestra en la figura 7.4.

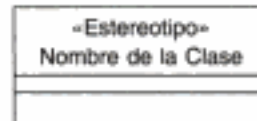


Figura 7.4 Diagrama de clase con estereotipo utilizando la notación de UML.

Los tres estereotipos, Entidad, Borde y Control, correspondientes a las tres dimensiones de la arquitectura de análisis, se ilustran en la figura 7.5.

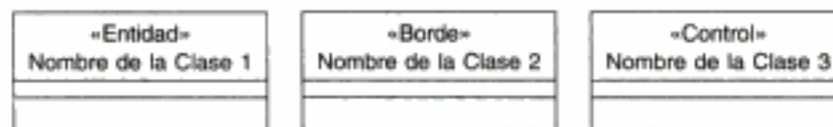


Figura 7.5 Diagrama de clase para los tres estereotipos.

Los tres estereotipos de la arquitectura de análisis se pueden describir a través de diagramas de *iconos* en lugar de rectángulos, se muestran como iconos en la figura 7.6.



Figura 7.6 Diagrama de clases mostrando los tres estereotipos como iconos: entidad, borde y control, respectivamente.

Es difícil lograr una ortogonalidad completa de estereotipos en los objetos. En general, es común que un objeto tenga cierta inclinación hacia una de las dimensiones, como se muestra en la figura 7.7.

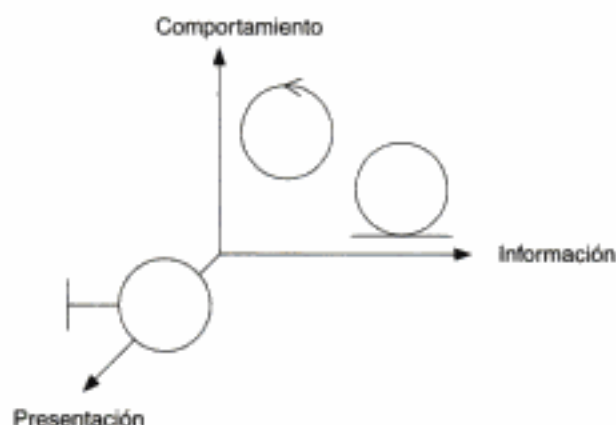


Figura 7.7 Diagrama que ilustra el traslape en las dimensiones de los objetos correspondientes a sus estereotipos.

7.1.2 Clases para casos de uso

Cuando se desarrolla el modelo de análisis, normalmente se trabaja con un caso de uso a la vez.

En cada caso de uso se identifican los objetos necesarios para su implementación. Los objetos se identifican según sus estereotipos de manera que correspondan con la funcionalidad ofrecida en cada uno. Se define explícitamente qué objeto es responsable de cuál comportamiento dentro del caso de uso. Se comienza identificando los objetos borde necesarios, continuando con los objetos entidad y, finalmente, los objetos control. Este proceso se continúa a los demás casos de uso. Dado que los mismos objetos pueden participar en varios casos de uso, durante este proceso es necesario identificar aquellos objetos comunes. Esto significa que cuando un conjunto de objetos ya existe, éstos pueden modificarse para ajustarlos al nuevo caso de uso. La meta es formar una arquitectura que reutilice el mayor número de objetos posible. De tal manera, la descripción original de los casos de uso se transforma en una descripción con base en los tres tipos de objetos, como se muestra en la figura 7.8.

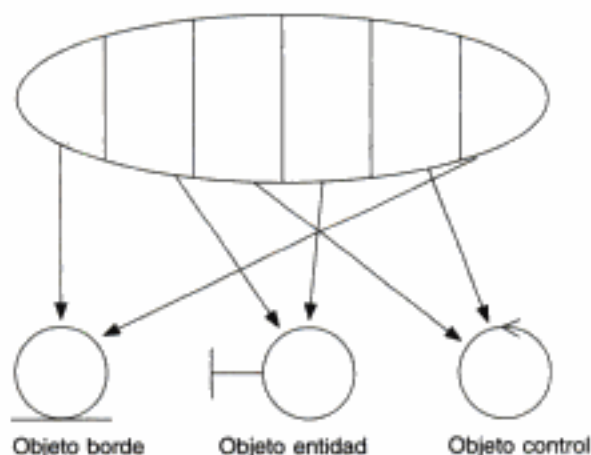


Figura 7.8 La funcionalidad de cada caso de uso se asigna a objetos distintos y de acuerdo con sus estereotipos.

La asignación de objetos a cada caso de uso se hace de acuerdo con los siguientes principios:

- ▶ La funcionalidad de los casos de uso que depende directamente de la interacción del sistema con el mundo externo se asigna a los objetos borde.
- ▶ La funcionalidad relacionada con el almacenamiento y manejo de información del dominio del problema se asigna a los objetos entidad.
- ▶ La funcionalidad específica a uno o varios casos de uso y que afecta a múltiples objetos a la vez, o que no se relaciona naturalmente con ningún objeto borde o entidad, se asigna a los objetos control. En general, se asigna un solo objeto control por caso de uso. Si el caso de uso es muy complejo, se pueden asignar objetos control adicionales.

7.2 Identificación de clases según estereotipos

Para llevar a cabo la transición del modelo de requisitos al modelo de análisis se deben identificar los objetos necesarios para implementar todos los casos de uso. La arquitectura de objetos debe considerar los tres tipos de estereotipos de objetos como se discutió anteriormente. Para ello, se deben identificar primero las clases borde, luego las clases entidad y, finalmente, las de control. En general, se desea asignar la funcionalidad general del caso de uso, correspondiente a la "política" de la aplicación, a los objetos control. Esta funcionalidad depende y afecta al resto de los objetos dentro del caso de uso. Por otro lado, los objetos entidad y borde deben contener funcionalidad local, limitando su efecto en los demás objetos. El trabajo del analista consiste en distribuir de la mejor manera posible el comportamiento especificado en el modelo de requisitos entre los diferentes tipos de objetos de la arquitectura de análisis.

La asignación de funcionalidad es bastante difícil en la práctica, y afecta en gran medida la calidad y mantenimiento del sistema. Los buenos analistas consideran desde un principio los posibles cambios futuros al sistema.

En general, los cambios más comunes a un sistema son los de funcionalidad y bordes. Los cambios de las interfaces deben afectar sólo a los objetos borde. Los cambios a la funcionalidad son más difíciles de administrar, ya que ésta puede abarcar todos los tipos de objetos.

En las siguientes secciones se describirá con mayor detalle el proceso de identificación de los tres tipos de objetos, identificando las clases para cada caso de uso según sus estereotipos. El desafío principal en dicho proceso, es decidir cuántas y cuáles clases deben asignarse por caso de uso.

7.2.1 Borde

Toda la funcionalidad especificada en las descripciones de los casos de uso que depende directamente de los aspectos externos del sistema, se ubica en los objetos borde, pues a través de ellos se comunican los actores con el sistema. La tarea de una clase borde es traducir los eventos generados por un actor en eventos comprendidos por el sistema, y traducir los eventos del sistema en una

presentación comprensible por el actor. Las clases borde, en otras palabras, describen la comunicación bidireccional entre el sistema y los actores.

Las clases borde son bastante fáciles de identificar, donde se cuenta con al menos tres estrategias:

1. Se pueden identificar con base en los actores.
2. Se pueden identificar con base en las descripciones de las interfaces del sistema que acompañan al modelo de requisitos.
3. Se pueden identificar con base en las descripciones de los casos de uso y extraer la funcionalidad específica a los objetos bordes.

Comenzaremos utilizando la primera estrategia correspondiente a los actores. Cada actor necesita su propia clase borde para comunicarse con el sistema. En muchos casos un actor puede necesitar de varios objetos borde. Es evidente que los objetos borde no son totalmente independientes de cada uno, ya que, en ciertos casos deben saber de la existencia de los demás. Por ejemplo, el usuario debe interactuar con las clases borde de la interface de usuario que a su vez se comunican con las clases borde de las pantallas. Debe estar muy bien definida la interacción de estos dos grupos de clase borde, las pantallas e interface de usuario.

Existen dos tipos de clase borde a modelar, dependiendo del tipo de actor: bordes a otros sistemas y bordes a usuarios humanos.

- En el caso de objetos borde que se comunican con otros sistemas, es muy común que la comunicación se describa mediante protocolos de comunicación. Los objetos bordes pueden traducir las salidas del sistema a un protocolo de comunicación estandarizado, o simplemente enviar eventos producidos internamente sin conversiones complejas. Una ventaja de esto, es que si se cambia el protocolo, estos cambios serán locales al objeto borde. Un mayor problema ocurre cuando existen señales continuas del mundo externo, como en los sistemas de medición o control. Entonces los objetos borde deben "muestrear" la señal de entrada, ya que internamente el sistema se procesa en término de eventos.
- En el caso de los objetos borde que se comunican con usuarios humanos se utilizan diversas técnicas para el modelado de la interacción. Es fundamental que las interfaces con el usuario sean lógicas y coherentes. En general, en las aplicaciones interactivas es común que las interfaces de usuario sean un aspecto fundamental y de mayor complejidad dentro de la aplicación completa.

Aunque cada tipo de objeto tiene un propósito distinto, es evidente que los objetos borde tienen como propósito principal el manejo de las presentaciones. Sin embargo, también pueden administrar información y tener comportamiento. Debe decidirse de manera individual la cantidad de información y comportamiento de un objeto borde. En un extremo, el objeto borde sólo debe enviar el evento que recibe del actor a otros objetos en el sistema, sin participar activamente en el curso de los eventos. En el otro extremo, el comportamiento del objeto borde puede ser muy complejo, de manera que la información se procese dentro del objeto borde, haciendo todo el procesamiento de eventos de manera local.

Para identificar qué parte del flujo de un caso de uso debe asignarse a los objetos borde, se deben analizar las interacciones entre los actores y los casos de uso. Esto significa buscar aspectos como, información que deba presentarse al actor y funcionalidad que dependa del comportamiento del actor.

En el ejemplo del sistema de reservaciones de vuelo, cada uno de los actores: *Usuario*, *Base de Datos de Registros* y *Base de Datos de Reservas*, requiere su propio objeto borde, como se muestra en la figura 7.9. El *Usuario* necesita las pantallas de presentación, mientras que la *Base de Datos de Registros* y *Base de Datos de Reservas* requiere sus propias clases bordes para intercambiar información con el sistema.

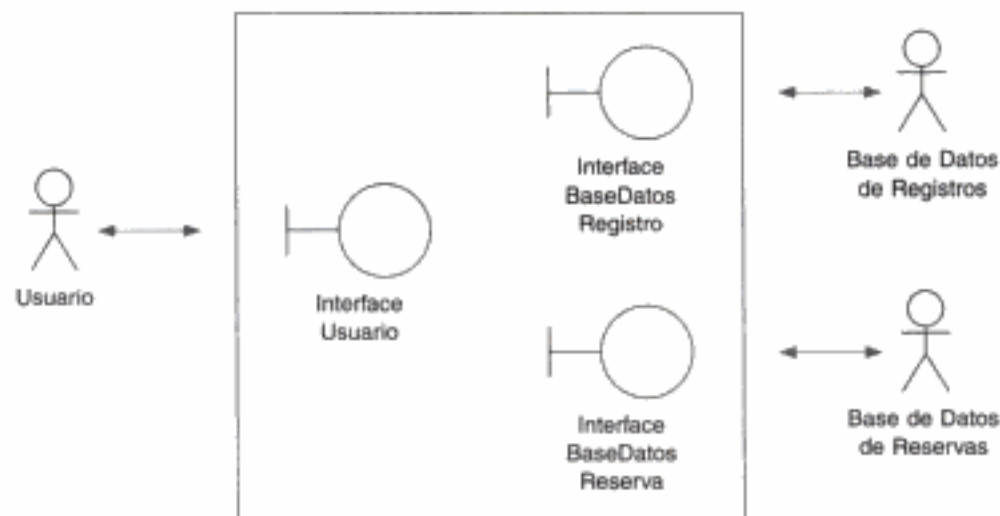


Figura 7.9 Clases bordes para el sistema de reservaciones de vuelo identificados directamente con los actores.

Aunque estas tres clases borde son suficientes para interactuar con los actores, se necesita incluir un número de clases borde adicionales correspondientes a cada pantalla que se le presenta al usuario, como se especificó inicialmente durante el modelo de requisitos. Por otro lado, pueden haber clases borde adicionales necesarias para manejos más especializados de las bases de datos, algo que postergaremos hasta el modelo de diseño. A continuación describimos las clases borde necesarias para cada caso de uso, de acuerdo con la documentación generada durante el modelo de requisitos del capítulo anterior. Se requiere incluir todas las pantallas que participan en los diversos casos de uso.

- *Validar usuario*: se interactúa con los actores *Usuario* y *Bases de Datos de Registros* a través de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosRegistro*, respectivamente. Se utiliza únicamente la pantalla principal del sistema (*P-1*) para la validación de usuario. (Nótese que los indicadores de tipo *P-1* corresponden a aquellos utilizados en el capítulo 6.) Por lo tanto, en término de pantallas, se incluye únicamente la clase borde *PantallaPrincipal*. En la figura 7.10 se muestran las clases borde identificadas en este caso de uso.



Figura 7.10 Clases borde identificadas del caso de uso *Validar Usuario*.

- *Ofrecer Servicios*: este caso de uso utiliza únicamente la pantalla de servicios del sistema (P-2). Por tanto, se incluye únicamente la clase borde *PantallaServicio*. Dado que se interactúa con el actor *Usuario* se incluye también la clase borde *InterfaceUsuario*. En la figura 7.11 se ilustran las clases borde identificadas en este caso de uso.



Figura 7.11 Clases bordes identificadas del caso de uso *Ofrecer Servicios*.

- *Registrar Usuario*: se interactúa con los actores *Usuario* y *Bases de Datos de Registros* a través de las clases bordes *InterfaceUsuario* e *InterfaceBaseDatosRegistro*, respectivamente. Adicionalmente se deben incluir clases bordes correspondientes a las pantallas propias de este caso de uso, que son las pantalla de *Registro de usuario* por primera vez (P-3) y *Obtener registro* (P-4). A las dos clases bordes correspondientes las llamaremos *PantallaCrearRegistroUsuario* y *PantallaObtenerRegUsuario*, respectivamente. Aunque la funcionalidad comienza en la pantalla principal del sistema (P-1) durante la validación de un usuario, esta validación se hace a través del caso de uso *Validar Usuario*, por lo cual esta funcionalidad no se incluye como parte de este caso de uso. En la figura 7.12 se muestran las clases bordes identificadas en este caso de uso.

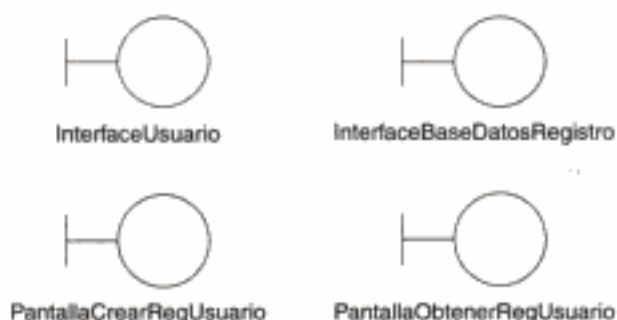


Figura 7.12 Clases bordes identificadas del caso de uso *Registrar Usuario*.

- **Registrar Tarjeta:** se interactúa con los actores *Usuario* y *Bases de Datos de Registros* a través de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosRegistro*, respectivamente. Se utilizan las pantallas *Registro de tarjeta por primera vez (P-5)* y *Registro de tarjeta (P-6)*. A las dos clases borde correspondientes las llamaremos *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta*, respectivamente. En la figura 7.13 se muestran las clases bordes identificadas en este caso de uso.

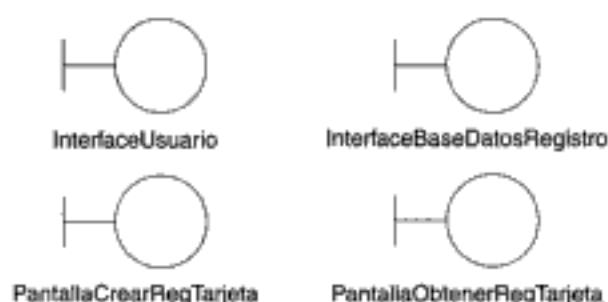


Figura 7.13 Clases bordes identificadas en el caso de uso *Registrar Tarjeta*.

- **Consultar Información:** se interactúa con los actores *Usuario* y *Base de Datos de Reservas* a través de las clases bordes *InterfaceUsuario* e *InterfaceBaseDatosReserva*, respectivamente. Adicionalmente se deben incluir clases borde correspondientes a las pantallas propias de este caso de uso, que son las pantallas de selección de tipo de consulta (P-7), consulta de horarios de vuelos (P-8), resultado de consulta de horarios de vuelos (P-9), consulta de tarifas de vuelos (P-10), resultado de consulta de tarifas de vuelos (P-11), consulta de estado de vuelo (P-12) y resultado de consulta de estado de vuelo (P-13). A las clases bordes correspondientes las llamaremos *PantallaConsultas*, *PantallaConsultaHorarios*, *PantallaResultadoHorarios*, *PantallaConsultaTarifas*, *PantallaResultadoTarifas*, *PantallaConsultaEstado* y *PantallaResultadoEstado*, respectivamente. En la figura 7.14 se ilustran las clases borde identificadas en este caso de uso.

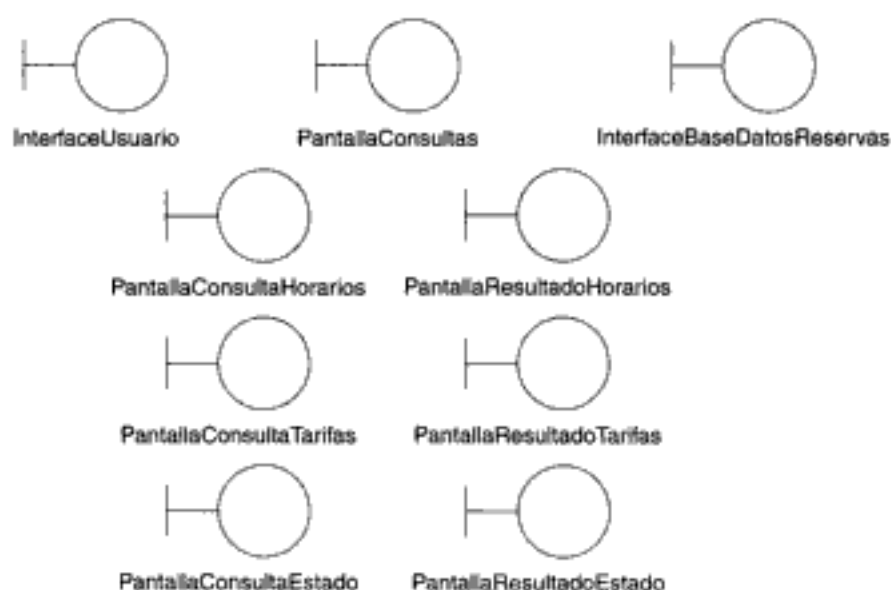


Figura 7.14 Clases bordes identificadas en el caso de uso *Consultar Información*.

- **Hacer Reservación:** se interactúa con los actores *Usuario* y *Base de Datos de Reservas*, a través de las clases bordes *InterfaceUsuario* e *InterfaceBaseDatosReservas*, respectivamente. Además, se deben incluir clases bordes correspondientes a las pantallas propias de este caso de uso, que son las pantallas de inserción de clave de reserva (*P-14*), solicitud de reserva de vuelos (*P-15*) y récord de reserva de vuelos (*P-16*). A las clases bordes correspondientes las llamaremos *PantallaClaveReservas*, *PantallaCrearReservaVuelos* y *PantallaRecordReservaVuelos*, respectivamente. En la figura 7.15 se ilustran las clases bordes identificadas en este caso de uso.

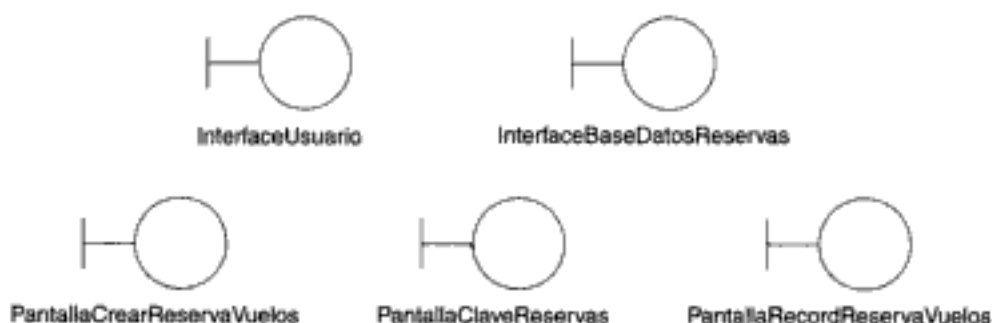


Figura 7.15 Clases bordes identificadas del caso de uso *Hacer Reservación*.

- **Pagar Reservación:** se interactúa con los actores *Usuario* y *Base de Datos de Reservas* a través de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosReservas*, respectivamente. Se utilizan las pantallas de pago de reserva de vuelos (*P-17*) y reembolso de reserva de vuelos (*P-18*). A las dos clases borde correspondientes las llamaremos *PantallaPagarRegTarjeta* y *PantallaReembolsarRegTarjeta*, respectivamente. En la figura 7.16 se muestran las clases bordes identificadas en este caso de uso.

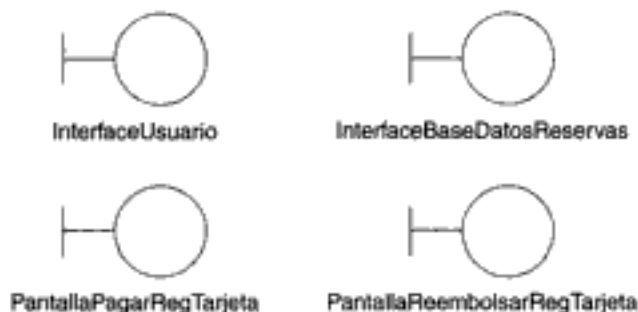


Figura 7.16 Clases bordes identificadas en el caso de uso *Pagar Reservación*.

En la tabla 7.1 se presenta el resumen de los casos de uso identificados durante el modelo de requisitos, junto con los actores y clases bordes correspondientes.

Tabla 7.1 Relación entre casos de uso y actores en el sistema de reservaciones de vuelo.

Caso de uso	Actores	Clases borde
Validar Usuario	Usuario, Base de Datos Registros	InterfaceUsuario, PantallaPrincipal, InterfaceBaseDatosRegistro
Ofrecer Servicios	Usuario	InterfaceUsuario, PantallaServicio
Registrar Usuario	Usuario, Base de Datos Registros	InterfaceUsuario, PantallaCrearRegistroUsuario, PantallaObtenerRegUsuario, InterfaceBaseDatosRegistro
Registrar Tarjeta	Usuario, Base de Datos Registros	InterfaceUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta, InterfaceBaseDatosRegistro
Consultar Información	Usuario, Base de Datos Registros	InterfaceUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta, InterfaceBaseDatosRegistroConsultarInformaciónUsuario, Base de Datos Reservas InterfaceUsuario, PantallaConsultas, PantallaConsultaHorarios, PantallaResultadoHorarios, PantallaConsultaTarifas, PantallaResultadoTarifas, PantallaConsultaEstado, PantallaResultadoEstado, InterfaceBaseDatosReserva
Hacer Reservación	Usuario, Base de Datos Reservas	InterfaceUsuario, PantallaClaveReservas, PantallaCrearReservaVuelos, PantallaRecordReservaVuelos, InterfaceBaseDatosReserva
Pagar Reservación	Usuario, Base de Datos Reservas	InterfaceUsuario, PantallaPagarRegTarjeta, PantallaReembolsarRegTarjeta, InterfaceBaseDatosReserva

7.2.2 Entidad

Se utilizan objetos entidad para modelar la información que el sistema debe manejar a corto y largo plazos. La información a corto plazo existe durante la ejecución de un caso de uso, mientras que la información a largo plazo trasciende los casos de uso, por lo que es necesario guardarla en alguna base de datos o archivo.

Los objetos entidad se identifican principalmente a partir del dominio del problema del modelo de requisitos. Dado que es posible identificar un gran número de objetos entidad, se deben considerar únicamente aquellos necesarios para la aplicación. Los casos de uso deben usarse como guías para esta identificación, y solamente aquellos objetos entidad que puedan justificarse de la descripción del caso de uso deben incluirse.

Adicionalmente, no es fácil decidir cuándo cierta información debe ser modelada como un objeto entidad o un atributo. Esto depende de cómo se usará la información. Si ésta cuenta con cierta estructura más allá de un simple valor numérico, entonces debe modelarse como un objeto entidad. Por otro lado, información que pueda describirse mediante un simple valor, debe modelarse

como un atributo de un objeto entidad. Esta decisión es algo arbitraria, ya que cierta información puede modelarse como objeto entidad en un sistema, mientras que en otro puede representarse mediante un atributo.

También es difícil identificar qué operaciones y cuáles atributos serán incluidos dentro de estos objetos.

Dado que la única forma para manipular un objeto entidad es por medio de sus operaciones, se deben identificar suficientes operaciones para manipular completamente el objeto entidad. La descripción detallada de los casos de uso es de nuevo un medio extremadamente valioso para identificar estas operaciones.

Las operaciones pueden ser sencillas, sirviendo de acceso a los valores de los atributos, o complejas, como en el caso de ciertos cálculos matemáticos basados en los valores de los atributos del objeto. Sea cual sea la complejidad, estas operaciones deben depender y afectar solamente a la información local.

Durante la identificación de objetos entidad, se encontrará que objetos similares aparecen en varios casos de uso.

A continuación se describen las clases entidad, correspondientes a los objetos entidad, identificados en los casos de uso del modelo de requisitos del capítulo anterior. Note que éstos pueden ser inicialmente obtenidos a partir del dominio del problema.

- *Validar Usuario*: este caso de uso requiere validar información exclusivamente guardada en el registro de usuario, lo que se hace en la clase entidad *RegistroUsuario*, utilizada también por el caso de uso *RegistrarUsuario*. En la figura 7.17 se muestra la clase entidad identificada en este caso de uso.



Figura 7.17 Clase entidad identificada en el caso de uso *Validar Usuario*.

- *Ofrecer Servicios*: este caso de uso administra las opciones de servicio y no requiere de ninguna clase entidad.
- *Registrar Usuario*: este caso de uso requiere guardar información exclusivamente acerca del usuario, lo que se hace en la clase entidad *Registro-*

Usuario. En la figura 7.18 se muestra la clase entidad identificada en este caso de uso.



Figura 7.18 Clase entidad identificada en el caso de uso *Registrar Usuario*.

- *Registrar Tarjeta*: este caso de uso requiere guardar información exclusiva acerca de la tarjeta del usuario, lo que se hace en la clase entidad *RegistroTarjeta*. En la figura 7.19 se presenta la clase entidad identificada en este caso de uso.



Figura 7.19 Clase entidad identificada en el caso de uso *Registrar Tarjeta*.

- *Consultar Información*: este caso de uso requiere de toda la información relacionada con consultas. Se pueden tomar las clases del dominio del problema y quitar aquellas relacionadas con registros y reservaciones. De tal manera tenemos las clases entidad *Asiento*, *Avión*, *Tarifa*, *Aeropuerto*, *Aerolínea*, *Vuelo* y *Horario*. En la figura 7.20 se muestran las clases entidad identificadas en este caso de uso.

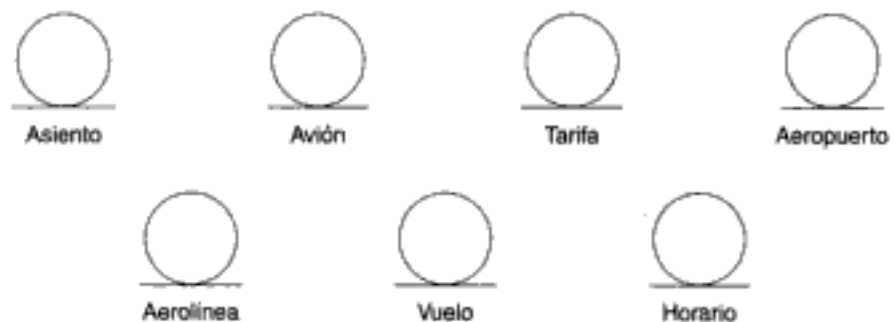


Figura 7.20 Clases entidad identificadas en el caso de uso *Consultar Información*.

- *Hacer Reservación*: este caso de uso requiere de la información relacionada con reservaciones. Se pueden tomar las clases del dominio del problema y quitar aquellas relacionadas con registros. De tal manera tenemos las clases entidad *Asiento*, *Avión*, *Tarifa*, *Aeropuerto*, *Aerolínea*, *Vuelo*, *Horario*, *ViajeroFrecuente*, *Pasajero* y *Reservación*. En la figura 7.21 se ilustran las clases entidad identificadas en este caso de uso.

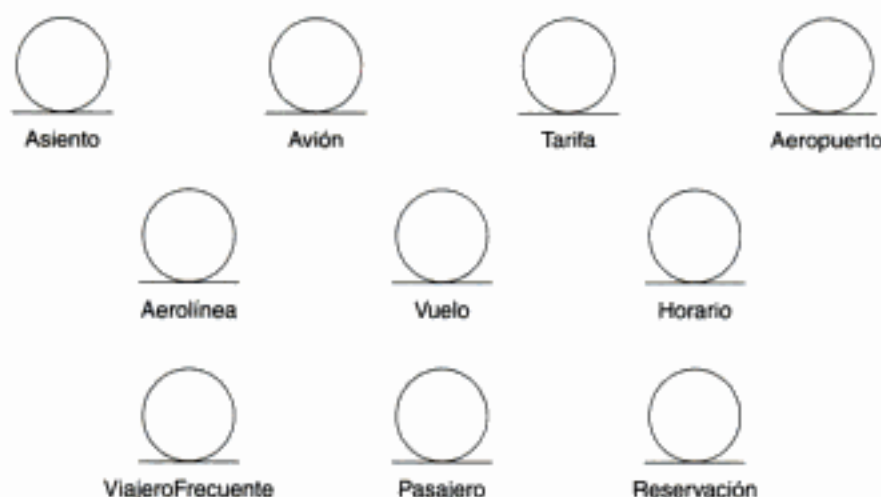


Figura 7.21 Clases entidad identificadas en el caso de uso *Hacer Reservación*.

- *Pagar Reservación*: este caso de uso requiere de la información relacionada con reservaciones. Dado que es una extensión al caso de uso *Hacer Reservación*, no es necesario volver a repetir todas las clases entidad, sino más bien especificar cualquier clase adicional. En particular, es necesario agregar el registro de tarjeta para completar el pago o el reembolso. De tal manera, tenemos una sola clase entidad *RegistroTarjeta*, la cual se muestra en la figura 7.22.



Figura 7.22 Clase entidad identificada en el caso de uso *Pagar Reservación*.

En la tabla 7.2 se muestra el resumen de los casos de uso identificados durante el modelo de requisitos junto con las clases entidad correspondientes.

Tabla 7.2 Relación entre casos de uso y clases entidad para el sistema de reservaciones de vuelo.

Caso de uso	Clases entidad
Validar Usuario	RegistroUsuario
Ofrecer Servicios	
Registrar Usuario	RegistroUsuario
Registrar Tarjeta	RegistroTarjeta
Consultar Información	Asiento, Avión, Tarifa, Aeropuerto, Aerolínea, Vuelo, Horario
Hacer Reservación	Asiento, Avión, Tarifa, Aeropuerto, Aerolínea, Vuelo, Horario, ViajeroFrecuente, Pasajero, Reservación
Pagar Reservación	Asiento, Avión, Tarifa, Aeropuerto, Aerolínea, Vuelo, Horario, ViajeroFrecuente, Pasajero, Reservación, RegistroTarjeta

7.2.3 Control

Hasta ahora se han identificado objetos borde y entidad a partir de cada caso de uso. En algunas situaciones, todo un caso de uso pudiera implementarse exclusivamente mediante estos dos tipos de objetos. Así no se necesitaría ningún objeto control para el respectivo caso de uso. Sin embargo, en la mayoría de los casos de uso, existe un comportamiento que no se puede asignar de forma natural a ninguno de los otros dos tipos de objetos. Una posibilidad es repartir el comportamiento entre los dos tipos de objetos, pero la solución no es buena si se considera el aspecto de extensibilidad. Un cambio en el comportamiento podría afectar varios objetos, dificultando su modificación. Por lo tanto, para evitar estos problemas, tal comportamiento se asigna a objetos control.

En general, es difícil lograr un buen balance en la distribución del comportamiento del caso de uso entre los objetos entidad, borde y control. Los objetos de control normalmente proveen la administración de los demás tipos de objetos, dependiendo de la existencia del propio caso de uso. Por lo tanto, los objetos control se especifica directamente de los casos de uso. Como primera aproximación, se especifica un objeto control para cada caso de uso. Dado que se asigna inicialmente el comportamiento a los objetos borde y entidad, el comportamiento restante se agrega a los objetos control. Una manera de asignar el comportamiento es modelar inicialmente el caso de uso sin ningún objeto control, en otras palabras, sólo utilizando objetos borde y objetos entidad. Cuando tal modelo se ha desarrollado, se verá que hay ciertos comportamientos que no se asignan de forma natural, a los diversos objetos, o incluso, se distribuye a varios objetos. Estos comportamientos deberían asignarse a los objetos control. Otra situación es que los comportamientos, después de distribuirse entre objetos borde y entidad, sea demasiado complicado. En tal caso, los comportamientos pueden ser distribuidos en varios objetos control.

En la mayoría de los sistemas, se promueve la distribución del manejo de un caso de uso en un solo objeto control. Sin embargo, la estrategia de asignación de control se debe decidir de acuerdo a cada aplicación.

En el sistema de reservaciones de vuelo especificaremos inicialmente un objeto control para cada caso de uso como se verá a continuación. A estas clases las llamaremos **manejadores** o **controladores** para distinguirlas de los demás estereotipos.

- *Validar Usuario*: este caso de uso requiere de un controlador para manejar la validación del registro de usuario. Dado que se basa en la misma información de registro, como enfoque inicial se puede utilizar la misma clase control que en el caso de uso anterior, *ManejadorRegistroUsuario*, como lo ilustra la figura 7.23.



Figura 7.23 Clase control para el caso de uso *Validar Usuario*.

- *Ofrecer Servicios*: este caso de uso requiere de un controlador para administrar los aspectos generales de los servicios. Como enfoque inicial se utiliza una clase general de control que llamaremos *ManejadorServicio*. En la figura 7.24 se muestra la clase control para este caso de uso.



Figura 7.24 Clase control para el caso de uso *Ofrecer Servicios*.

- *Registrar Usuario*: este caso de uso requiere de un controlador para manejar la información, lo que haremos mediante la clase control *ManejadorRegistroUsuario*. En la figura 7.25 se muestra la clase control para este caso de uso.



Figura 7.25 Clase control para el caso de uso *Registrar Usuario*.

- *Registrar Tarjeta*: este caso de uso requiere una clase controladora para administrar el registro de la tarjeta del usuario, lo que haremos mediante la clase control *ManejadorRegistroTarjeta*. En la figura 7.26 se muestra la clase control para este caso de uso.



Figura 7.26 Clase control para el caso de uso *Registrar Tarjeta*.

- *Consultar Información*: este caso de uso requiere de un controlador para manejar la información y las interfaces relacionadas con las consultas, lo que se hace en la clase control *ManejadorConsultas*. Dado que se tienen tres tipos de consultas distintas, podemos incluir tres controladores especializados: *ManejadorConsultaHorarios*, *ManejadorConsultaTarifas* y *ManejadorConsultaEstado*, para las consultas respectivas, lo cual se ilustra en la figura 7.27.



Figura 7.27 Clases control para el caso de uso *Consultar Información*.

- *Hacer Reservación*: este caso de uso requiere de un controlador para manejar lo relacionado con las reservas, lo que se hace mediante la clase control *ManejadorReservas* que se muestra en la figura 7.28.



ManejadorReservas

Figura 7.28 Clase control para el caso de uso *Hacer Reservación*.

- *Pagar Reservación*: este caso de uso requiere administrar lo relacionado con los pagos, lo que se hará mediante la clase control *ManejadorPagos*. En la figura 7.29 se muestran la clase control para este caso de uso.



ManejadorPagos

Figura 7.29 Clase control para el caso de uso *Pagar Reservación*.

- En general siempre es bueno incluir un controlador principal para administrar los aspectos generales del sistema, incluyendo la pantalla principal. A esta clase la llamaremos *ManejadorPrincipal*, la cual se ilustra en la figura 7.30.



ManejadorPrincipal

Figura 7.30 Clase control para el sistema completo.

En la tabla 7.3 se presenta el resumen de los casos de uso identificados durante el modelo de requisitos, junto con sus clases control correspondientes.

Tabla 7.3 Relación entre casos de uso y clases control para el sistema de reservaciones de vuelo.

Caso de uso	Clases control
Validar Usuario	<i>ManejadorRegistroUsuario</i>
Ofrecer Servicios	<i>ManejadorServicio</i>
Registrar Usuario	<i>ManejadorRegistroUsuario</i>
Registrar Tarjeta	<i>ManejadorRegistroTarjeta</i>
Consultar Información	<i>ManejadorConsultas</i> , <i>ManejadorConsultaHorarios</i> , <i>ManejadorConsultaTarifas</i> , <i>ManejadorConsultaEstado</i>
Hacer Reservación	<i>ManejadorReservas</i>
Pagar Reservación	<i>ManejadorPagos</i>
	<i>ManejadorPrincipal</i>

7.3 Clases según casos de uso

En esta sección mostramos un diagrama de clases por caso de uso de acuerdo con las clases identificadas en las secciones anteriores.

7.3.1 Validar Usuario

El caso de uso *Validar Usuario* involucra una clase control *ManejadorRegistroUsuario* que controla la información de *RegistroUsuario* y las clases borde *InterfaceUsuario* e *InterfaceBaseDatosRegistro*. Agregamos también la clase *PantallaPrincipal* por recibir la información de registro a ser validada y al *ManejadorPrincipal* por ser el controlador de la pantalla anterior, estas clases se ilustran en la figura 7.31.

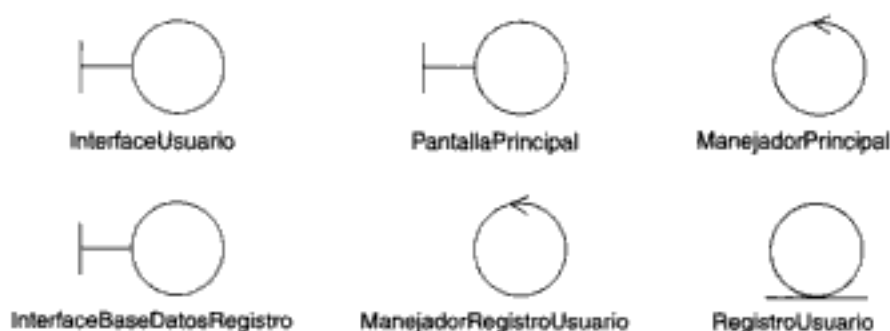


Figura 7.31 Clases identificadas para el caso de uso *Validar Usuario*.

7.3.2 Ofrecer Servicios

El caso de uso *Ofrecer Servicios* involucra una clase control *ManejadorServicio* que controla la pantalla *PantallaServicio*. Agregamos también la clase borde *InterfaceUsuario*. Estas clases se muestran en la figura 7.32.



Figura 7.32 Clases identificadas para el caso de uso *Ofrecer Servicios*.

7.3.3 Registrar Usuario

El caso de uso *Registrar Usuario* involucra una clase control *ManejadorRegistroUsuario* que controla la información de *RegistroUsuario* y las clases borde correspondiente a las pantallas *PantallaCrearRegistroUsuario* y *PantallaObtenerRegistroUsuario*, además de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosRegistro*. La figura 7.33 muestra estas clases.

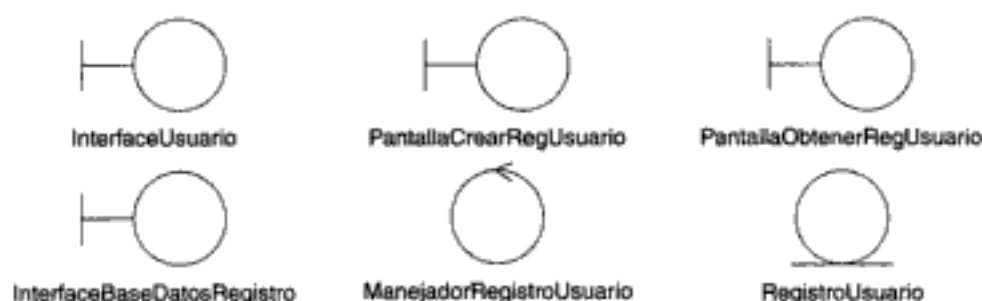


Figura 7.33 Clases identificadas para el caso de uso *Registrar Usuario*.

7.3.4 Registrar Tarjeta

El caso de uso *Registrar Tarjeta* involucra una clase control *ManejadorRegistroTarjeta* que controla la información de *RegistroTarjeta* y las clases borde correspondientes a las pantallas *PantallaCrearRegTarjeta* y *PantallaObtenerRegistroTarjeta*, además de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosRegistro*, como lo muestra la figura 7.34.

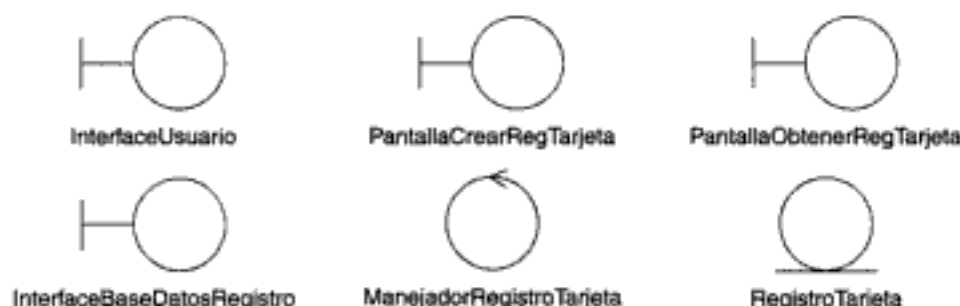


Figura 7.34 Clases identificadas para el caso de uso *Registrar Tarjeta*.

7.3.5 Consultar Información

El caso de uso *Consultar Información* involucra una clase control *ManejadorConsultas* que controla los diferentes tipos de consultas, junto con la clase borde, correspondiente a la pantalla *PantallaConsultas*, además de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosReservas*. Dado que este caso de uso tiene tres subflujos importantes, en lugar de describirlos en un solo diagrama, lo haremos en tres diagramas separados como veremos más adelante. En la figura 7.35 se muestran las clases principales identificadas en este caso de uso.

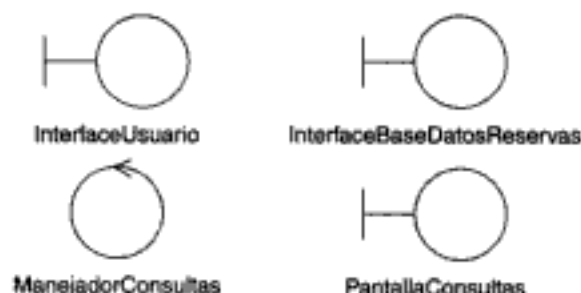


Figura 7.35 Clases identificadas del caso de uso *Consultar Información*.

CONSULTAR HORARIOS

El subflujo *Consultar Horarios* del caso de uso *Consultar Información* involucra las clases del diagrama de la figura 7.35, las cuales no incluyen en él. Se incluyen en el nuevo diagrama las clases borde correspondientes a las pantallas *PantallaConsultaHorarios* y *PantallaResultadoHorarios*, además de las clases entidad *Vuelo*, *Aeropuerto*, *Horario* y *Aerolínea*, junto con la clase control *ManejadorConsultaHorarios*. El resto de las clases entidad del dominio del problema no son necesarias para este subflujo, lo cual se ilustra en la figura 7.36.

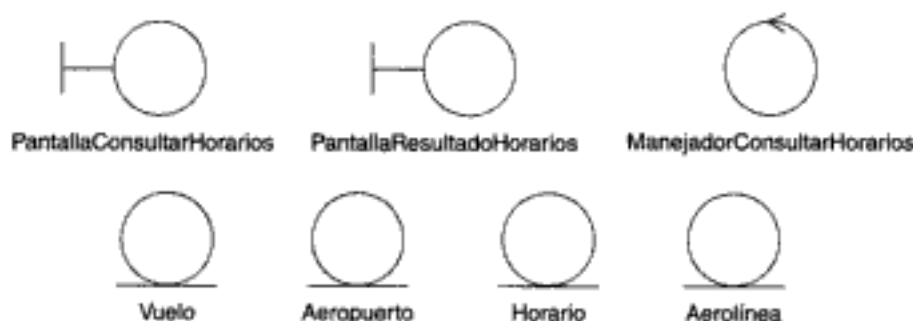


Figura 7.36 Clases identificadas para el subflujo *Consultar Horarios* del caso de uso *Consultar Información*.

CONSULTAR TARIFAS

El subflujo *Consultar Tarifas* del caso de uso *Consultar Información* involucra a todas las clases del diagrama de la figura 7.35, las cuales no se incluyen en él. Se incluyen en el nuevo diagrama las clases borde, correspondientes a las pantallas *PantallaConsultaTarifas* y *PantallaResultadoTarifas*, además de las clases entidad *Vuelo*, *Aeropuerto*, *Horario*, *Aerolínea* y *Tarifa*, junto con la clase control *ManejadorConsultaTarifas*. El resto de las clases entidad del dominio del problema no son necesarias en este subflujo. En la figura 7.37 se muestran las clases del caso de uso identificadas.



Figura 7.37 Clases identificadas para el subflujo *Consultar Tarifas* del caso de uso *Consultar Información*.

CONSULTAR ESTADO

El subflujo *Consultar Estado* del caso de uso *Consultar Información* involucra a todas las clases del diagrama de la figura 7.35, las cuales no se incluyen

en él. Se incluyen en el nuevo diagrama las clases borde correspondiente a las pantallas *PantallaConsultaEstado* y *PantallaResultadoEstado*, además de las clases entidad *Vuelo*, *Aeropuerto*, *Horario*, *Aerolínea* y *Avión*, junto con la clase control *ManejadorConsultaEstado*. El resto de las clases entidad del dominio del problema no son necesarias para este subflujo. En la figura 7.38 se muestran las clases identificadas en este caso de uso.



Figura 7.38 Clases identificadas para el subflujo *Consultar Estado* del caso de uso *Consultar Información*.

7.3.6 Hacer Reservación

El caso de uso *Hacer Reservación* involucra una clase control *ManejadorReservas*, que se encarga de controlar las reservaciones, junto con las clases bordes correspondientes a las pantallas *PantallaClaveReservas*, *PantallaCrearReservaVuelos* y *PantallaRecordReservaVuelos*, además de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosReserva*. Se incluyen en el diagrama todas las clases entidad necesarias: *Vuelo*, *Asiento*, *Avión*, *Tarifa*, *Horario*, *Aerolínea*, *Aeropuerto*, *Reservación*, *Pasajero* y *ViajeroFrecuente*. En la figura 7.39 se muestran las clases identificadas en este caso de uso.

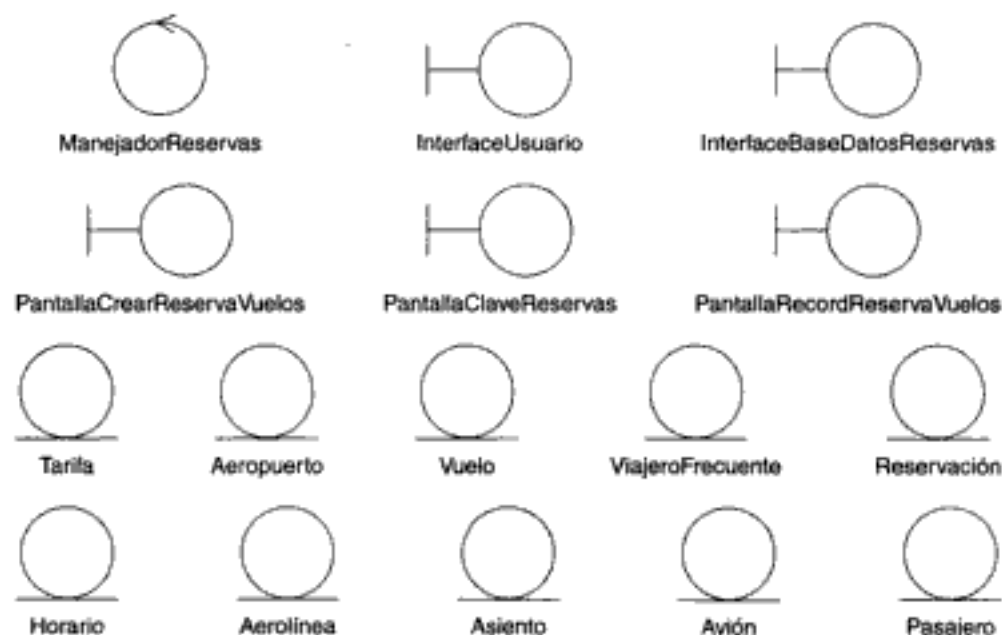


Figura 7.39 Clases identificadas para el caso de uso *Hacer Reservación*.

7.3.7 Pagar Reservación

El caso de uso *Pagar Reservación* involucra a la clase control *ManejadorPagos* que controla la información de pagos y las clases borde correspondientes a las pantallas *PantallaPagarRegTarjeta* y *PantallaReembolsarRegTarjeta*, además de las clases borde *InterfaceUsuario* e *InterfaceBaseDatosReserva*.

Se incluye también la clase *RegistroTarjeta* dado que es necesaria para obtener la información de la tarjeta de crédito. La figura 7.40 ilustra las clases identificadas en este caso de uso.



Figura 7.40 Clases identificadas para el caso de uso *Pagar Reservación*.

7.4 Diagramas de secuencias

Una vez identificadas las clases, se debe describir la interacción entre ellas para lograr la funcionalidad de los casos de uso. Éste es un paso muy importante, ya que con base en esta funcionalidad, se definirá la arquitectura del sistema, tanto estructural como funcional.

Para eso se introducen los **diagramas de secuencias**, también conocidos como de **interacción** o **eventos**, los cuales describen los diferentes casos de uso según la interacción o eventos enviados entre los objetos de la arquitectura del modelo de análisis. La descripción general de un diagrama de secuencia se muestra en la figura 7.41. El diagrama de secuencias describe aspectos dinámicos de un sistema, a diferencia de los diagramas de clases que muestran información estática. Por tal razón, los diagramas de secuencias utilizan objetos, a diferencia de los diagramas de clases que utilizan clases como elementos básicos. A diferencia de un diagrama de clases, el diagrama de secuencia puede representar múltiples objetos de manera independiente, incluyendo múltiples instancias de un mismo actor. Nótese el prefijo “:”, en la figura correspondiente a la notación para objetos, como se describió en el capítulo 4.

Cada objeto en el diagrama es representado con una línea vertical, correspondiente al eje del tiempo, donde el tiempo avanza hacia abajo. El diagrama de secuencia muestra los eventos que ocurren en el tiempo, los cuales son enviados de un objeto a otro. El orden de los objetos en el diagrama no es impor-

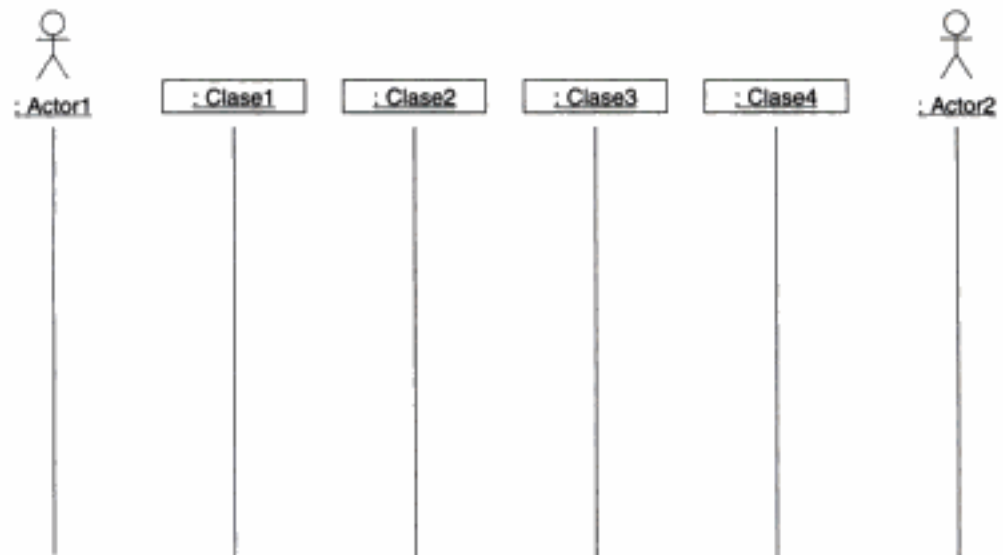


Figura 7.41 Descripción de un diagrama de secuencia.

tante, al igual que la distancia física entre los eventos. Lo importante en el diagrama es el orden en que los eventos ocurren y la dependencia entre ellos, en otras palabras, qué consecuencias tiene el envío de un evento.

El diagrama de secuencia de la figura 7.42 muestra un ejemplo de estos eventos.

En el diagrama de la figura 7.42, las barras gruesas corresponden a *actividades* del objeto, denominadas por *a*, mientras que las flechas corresponden a *eventos*, denominadas por *e*. Un evento se dibuja como una flecha horizontal que comienza en la barra del objeto que lo envía y termina en la barra del objeto que lo recibe. En este ejemplo los eventos son numerados sucesivamente utilizando el formato "*n*:*e*". Las actividades se inician por el arribo de eventos y el tiempo que duran es sólo relevante para resaltar eventos posteriores originados

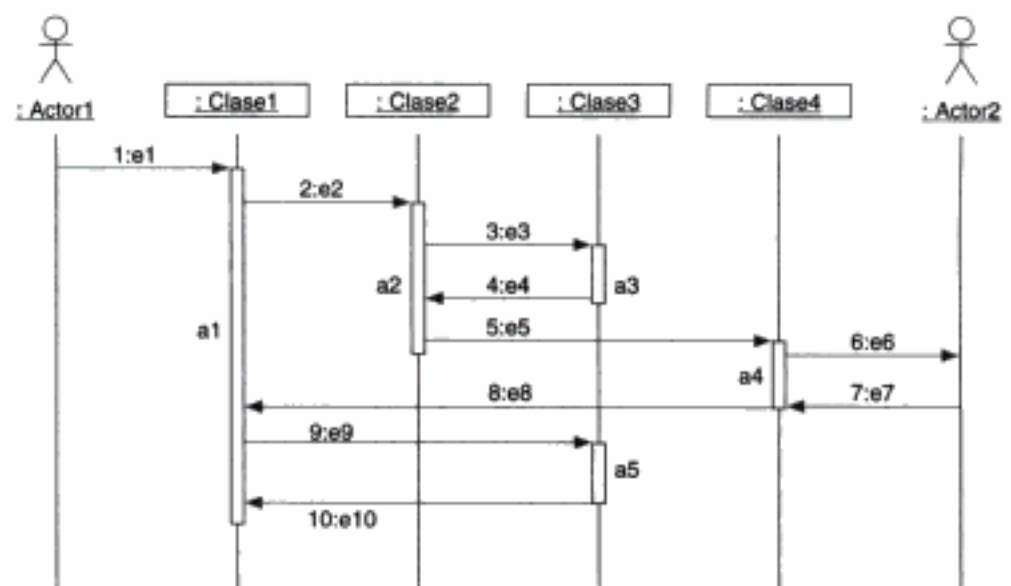


Figura 7.42 Diagrama de secuencias con eventos.

durante esa actividad. Tal es el caso del objeto de la *Clase1* que recibe un evento *e1*, el cual inicia una actividad que consiste primero en generar un evento *e2* y, así sucesivamente.

En general, un diagrama de secuencia permite apreciar la fluidez de los eventos en la arquitectura y la correspondencia de la funcionalidad con la del caso de uso. Es muy importante que no existan interrupciones entre eventos y que exista una continuidad con los eventos externos del sistema. Por ejemplo, consideremos la secuencia que comienza con el *Actor1* a través del evento *e1*. Este evento da lugar al evento *e2*, el cual a su vez da lugar al evento *e3* y así sucesivamente hasta llegar al evento *e7*. Sin embargo, se interrumpe la continuidad entre el evento *e7* y el evento *e8*. Esta situación es peligrosa, ya que al no haber una dependencia directa entre ambos eventos, se pierde la continuidad y el evento *e8* puede ocurrir incluso antes de *e7*, algo que pudiera ocasionar inconsistencias en la aplicación. Este tipo de situaciones se debe evitar, donde cada nuevo evento debe generarse a partir de uno anterior.

Como veremos en las siguientes secciones, se utilizarán los diagramas de secuencia para describir los *flujos principales* y *subflujos* de cada caso de uso. Esto ayudará a revisar si nuestra lógica es correcta y si existe consistencia entre los casos de uso del modelo de requisitos y la arquitectura de clases del modelo de análisis.

Dado que existen múltiples posibles flujos de secuencia, se describirán únicamente secuencias de funcionalidad completas que pudieran incluir múltiples flujos en múltiples casos de uso. A pesar que el inicio de muchos de los casos de uso se repiten, es importante que éstos muestren un flujo completo, de principio a fin de la funcionalidad deseada.

7.4.1 Registrar Usuario

En el caso de uso *Registrar Usuario* existen diversas secuencias que pueden ser instanciadas por un usuario. En esta sección mostramos las secuencias que pudieran desarrollarse entre los objetos para asegurar que la lógica que se utiliza sea correcta y que corresponda a los casos de uso especificados en el modelo de requisitos. En particular, mostraremos diagramas de secuencias para *Crear Registro Usuario*, *Actualizar Registro Usuario* y *Eliminar Registro Usuario*. Observe que estas secuencias incluirán a los subflujos *Crear Registro Usuario (S-1)*, *Actualizar Registro Usuario (S-4)* y *Eliminar Registro Usuario (S-5)*, junto con los casos de uso *Validar Usuario* y *Ofrecer Servicios*, además de los subflujos *Obtener Registro Usuario (S-2)* y *Administrar Registro Usuario (S-3)*.

CREAR REGISTRO USUARIO

El diagrama de secuencia para *Crear Registro Usuario* se muestra en la figura 7.43. Esta secuencia inicia en el flujo principal de *Registrar Usuario* que deberá incluir ciertas opciones definidas en *Validar Usuario* seguidos por el subflujo *Crear Registro Usuario (S-1)* y *Administrar Registro Usuario (S-3)*. Nótese que en los siguientes diagramas de secuencia, por razones de simplificación, se eliminaron los rectángulos verticales correspondientes a las actividades.

- *Validar Usuario*. Aunque la secuencia se inicia en el flujo principal de *Registrar Usuario*, se continúa inmediatamente con la inserción del caso de uso

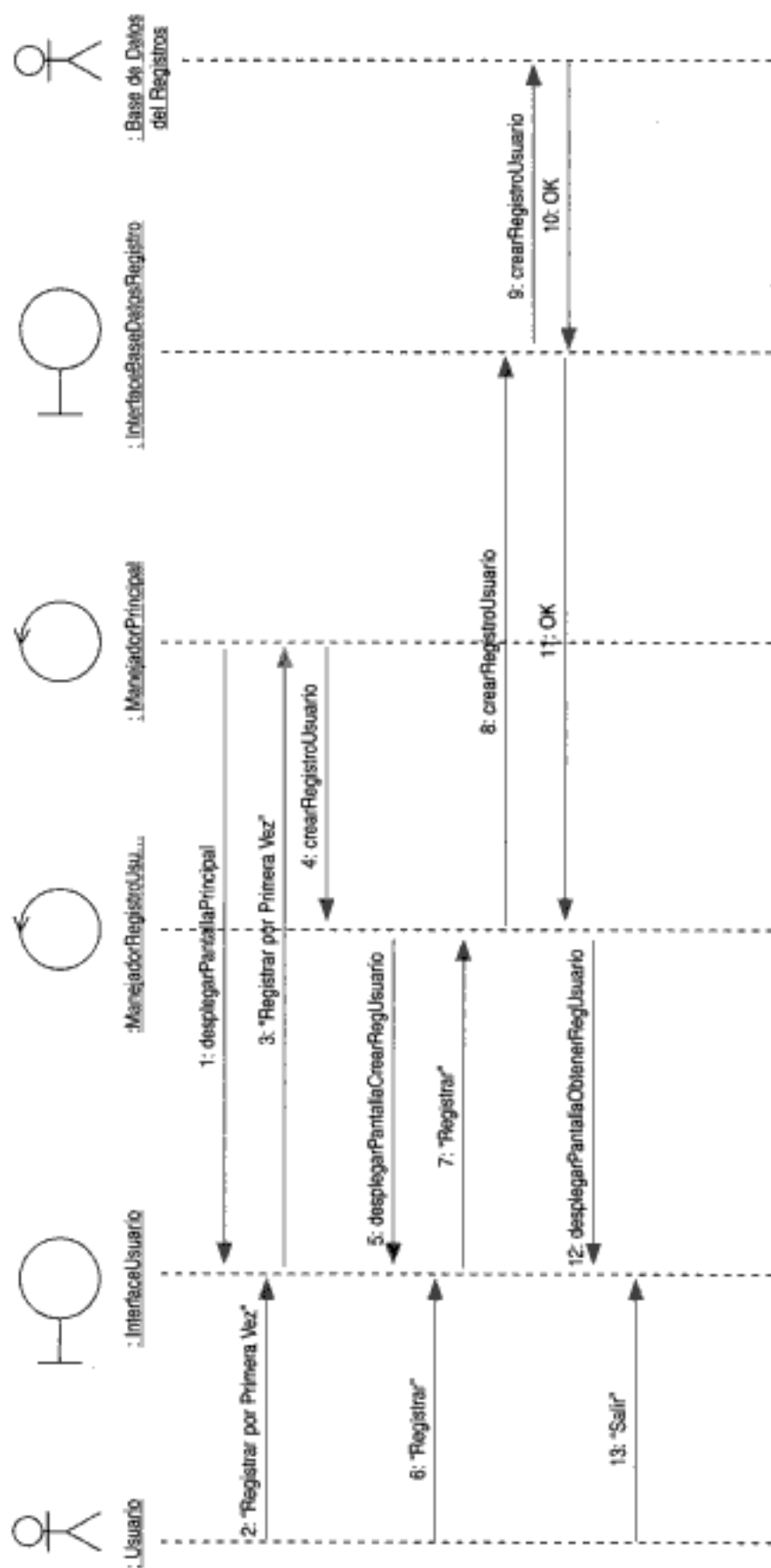


Figura 7.43 Diagrama de secuencia *Crear Registro Usuario* del caso de uso *Registrar Usuario*.

Validar Usuario, donde podemos iniciar con el *ManejadorPrincipal* solicitando el despliegado de la *PantallaPrincipal* mediante el evento *desplegarPantallaPrincipal*. Para continuar esta secuencia, el usuario deberá seleccionar la opción de "Registrar por Primera Vez" oprimiendo el botón correspondiente en la pantalla. La *InterfaceUsuario* por ser la controladora de las interfaces de usuario, recibe el evento y lo envía como un nuevo evento "Registrar por Primera Vez" al *ManejadorPrincipal*. El *ManejadorPrincipal*, que es el encargado de controlar la lógica general del sistema, reconoce que este evento corresponde a una actividad de registro y se lo envía como *crearRegUsuario* al *ManejadorRegistroUsuario*.

- *Registrar Usuario* subflujo *Crear Registro Usuario (S-1)*. En este momento el *ManejadorRegistroUsuario* reconoce el tipo de evento particular y solicita a la *InterfaceUsuario* el despliegado de la pantalla correspondiente mediante *desplegarPantallaCrearRegistroUsuario*. La *InterfaceUsuario* despliega esta pantalla, algo que no se muestra en el diagrama por ser un evento interno. Para continuar con la lógica principal de este subflujo, el usuario debe llenar sus datos, que no se muestran aquí, y oprime el botón "Registrar" para que esta información sea enviada a la clase *InterfaceUsuario*. Es importante resaltar que los datos como tales no generan eventos ni son de importancia en estos diagramas. Lo que genera eventos son los botones en las pantallas. Siguiendo con nuestra lógica, la *InterfaceUsuario* envía el evento "Registrar" al *ManejadorRegistroUsuario*. Este controlador es responsable de guardar la información de registro del usuario, por lo cual envía el evento *crearRegUsuario* a la *InterfaceBaseDatosRegistro*. Note que como en el caso de los datos, los objetos entidad como la clase *RegistroUsuario*, tampoco se muestran en el diagrama, dado que no agregan eventos interesantes para la lógica del sistema. Incluso se omiten del diagrama todas las clases correspondientes a pantallas, ya que sus eventos importantes son manejados por la *InterfaceUsuario*. Prosiguiendo con nuestra lógica, la *InterfaceBaseDatosRegistro* envía el evento *crearRegUsuario* al actor *Bases de Datos de Registros*. Este actor debe responder de alguna manera, y lo hace mediante un evento *OK*, el cual es luego enviado de manera sucesiva hasta llegar al *ManejadorRegistroUsuario*.
- *Registrar Usuario* subflujo *Administrar Registro Usuario (S-3)*. A continuación pasamos al subflujo *Administrar Registro Usuario (S-3)*, donde el *ManejadorRegistroUsuario* envía el evento *desplegarPantallaObtenerRegUsuario* a la *InterfaceUsuario*. En ese momento el *Usuario* presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario* seguido de los subflujos *Crear Registro Usuario (S-1)* y *Administrar Registro Usuario (S-3)* del caso de uso *Registrar Usuario*.

Vale la pena resaltar dos aspectos importantes en el diagrama. El primero es que en ningún momento se corta el flujo de los eventos. La única excepción en el diagrama son los eventos generados por el *Usuario*, que sólo se pueden generar a partir de que las pantallas hayan sido desplegadas, por lo cual realmente no hay ninguna interrupción lógica, sino más bien que no se muestran flujos de evento entre la *InterfaceUsuario* y el *Usuario*, ya que éstos son todos visuales (el usuario tendría que tener, por ejemplo, algún botón en su cuerpo para que realmente se muestre tal evento). El segundo aspecto es que los eventos entre objetos son como una cascada donde un objeto le envía un evento a

otro y éste le devuelve posteriormente un evento en respuesta, de manera directa o indirecta. Por ejemplo, el evento "7" es en respuesta al evento "5", mientras que el evento "11" es en respuesta al evento "8". Un ejemplo de respuesta indirecta es el caso del evento "3" respondido mediante los eventos "4" y "5". Una situación particular se da con el último evento correspondiente a "Salir", el cual interrumpe estos flujos de respuesta, ya que en algún lugar se debe terminar la secuencia.

ACTUALIZAR REGISTRO USUARIO

El diagrama de secuencia *Actualizar Registro Usuario* se muestra en la figura 7.44. Esta secuencia inicia en el flujo principal de *Registrar Usuario*, que incluye las opciones de validación definidas en *Validar Usuario*, seguidos por parte de *Ofrecer Servicios*, para luego proseguir con el subflujo *Obtener Registro Usuario (S-2)*, *Administrar Registro Usuario (S-3)* y obviamente *Actualizar Registro Usuario (S-4)*, para completar la secuencia de la actualización.

- *Validar Usuario*. Nuevamente, aunque la secuencia se inicia en el flujo principal de *Registrar Usuario*, se continúa inmediatamente con la inserción del caso de uso *Validar Usuario*, donde podemos iniciar con el *ManejadorPrincipal*, enviando el evento *desplegarPantallaPrincipal* a la *InterfaceUsuario*, el cual despliega la *PantallaPrincipal*. En este momento, el usuario debe validarse, insertando su login y contraseña. Al ser datos, éstos no generan ningún evento. Una vez que el usuario genere el evento "OK", al oprimir el botón correspondiente, se instancia la validación. La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual reconoce que este evento corresponde a una actividad de registro y envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Este controlador reconoce el tipo de evento particular y solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante un evento adicional con el mismo nombre. La *InterfaceBaseDatosRegistro* envía a su vez un evento similar al actor *Bases de Datos Registros*, el cual contesta con un evento "OK" si la validación es buena. Dado que sólo consideramos una secuencia de eventos, una validación incorrecta se mostraría en otro diagrama. El evento "OK" es sucesivamente enviado a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió el último "OK", solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- *Ofrecer Servicios*. A continuación se estudia el caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita entonces a la *InterfaceUsuario* el desplegado de la pantalla correspondiente a través de *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Obtener Registro". Este evento se envía de la *InterfaceUsuario* al *ManejadorServicio*, que envía el evento *obtenerRegistroUsuario* al *ManejadorRegistroUsuario*.
- *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*. A continuación regresamos al caso de uso *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatosRegistro* que obtenga el registro correspondiente mediante *obtenerRegistroUsuario*. Nuevamente, la *InterfaceBaseDatosRegistro* le pasa un evento similar al actor *Bases de Datos de Registros*, el cual contesta con

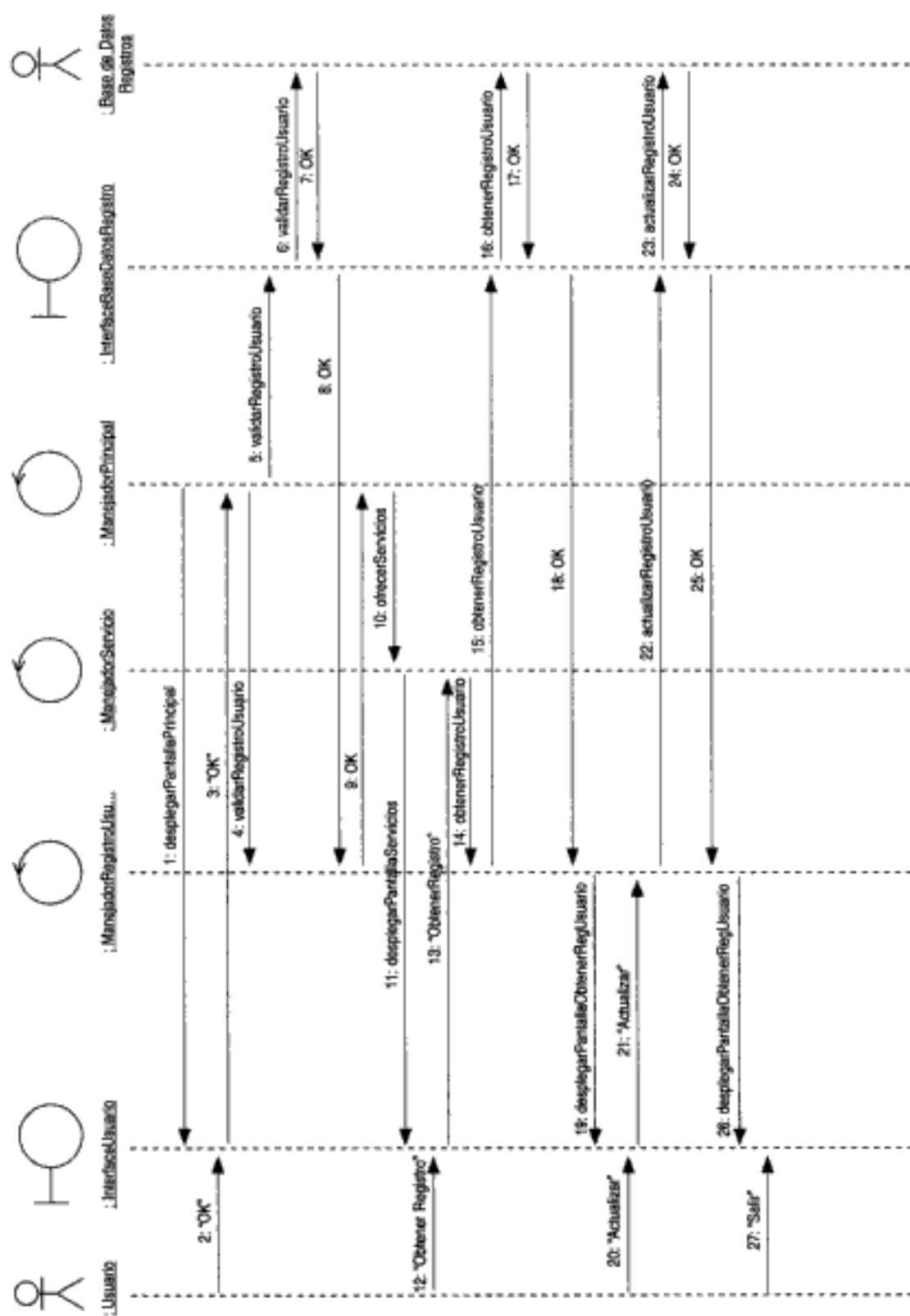


Figura 7.44 Diagrama de secuencia *Actualizar Registro Usuario* del caso de uso *Registrar Usuario*.

un OK. Junto a este OK deben enviarse los propios datos, los cuales no se muestran en el diagrama de secuencia por no generar eventos. El OK es luego enviado por la *InterfaceBaseDatosRegistro* al *ManejadorRegistroUsuario*.

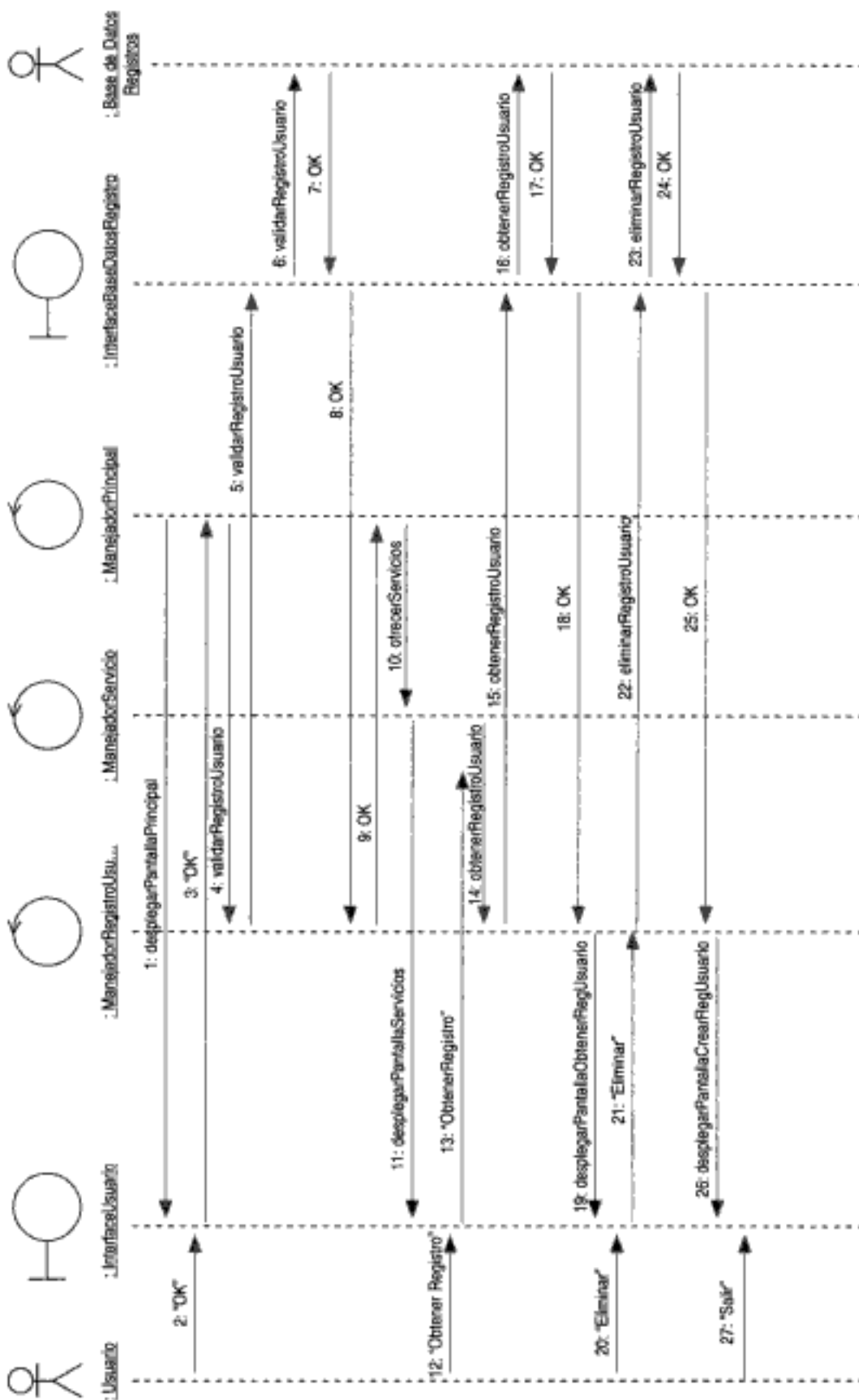
- *Registrar Usuario* subflujo *Administrar Registro Usuario* (S-3). A continuación podemos pasar al subflujo *Administrar Registro Usuario* (S-3), donde el *ManejadorRegistroUsuario* solicita a la *InterfaceUsuario* desplegar la pantalla de información de registro mediante el evento *desplegarPantallaObtenerRegUsuario*. En este momento el usuario actualiza sus datos, que no se muestran aquí y oprime el botón "Actualizar".
- *Registrar Usuario* subflujo *Actualizar Registro Usuario* (S-4). Siguiendo con la lógica, la *InterfaceUsuario* envía el mismo evento al *ManejadorRegistroUsuario*, el cual es responsable de actualizar el registro, por lo que envía el evento *actualizarRegistroUsuario* a la *InterfaceBaseDatosRegistro*, que a su vez envía el evento *actualizar RegistroUsuario* al actor *Bases de Datos de Registros*. Este actor responde con un OK, que luego envía de manera sucesiva hasta llegar al *ManejadorRegistroUsuario*.
- *Registrar Usuario* subflujo *Administrar Registro Usuario* (S-3). A continuación se pasa al subflujo *Administrar Registro Usuario* (S-3), donde el *ManejadorRegistroUsuario* envía el evento *desplegarPantallaObtenerRegUsuario* a la *InterfaceUsuario*. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario* seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Registrar Usuario* con los subflujos *Obtener Registro Usuario* (S-2), *Administrar Registro Usuario* (S-3) y *Actualizar Registro Usuario* (S-4).

ELIMINAR REGISTRO USUARIO

El diagrama de secuencia para el subflujo *Eliminar Registro Usuario* se muestra en la figura 7.45. Esta secuencia es bastante similar a la de *Actualizar Registro Usuario*, ya que inicia en el flujo principal de *Registrar Usuario*, que incluye las opciones de validación definidas en *Validar Usuario*, seguidos por ciertos aspectos de *Ofrecer Servicios*, para luego proseguir con el subflujo *Obtener Registro Usuario* (S-2), *Administrar Registro Usuario* (S-3) y *Eliminar Registro Usuario* (S-5), para completar la secuencia de la eliminación. Note que solamente cambia el último subflujo.

- *Validar Usuario*. Nuevamente, se comienza en *Validar Usuario* la secuencia con el evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* al *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*. El *ManejadorPrincipal* envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. El *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Bases de Datos Registros*, el cual contesta con un OK si la validación es buena. El OK se envía a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió este último OK, so-



licita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.

- ▶ *Ofrecer Servicios*. A continuación podemos pasar al caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita entonces a la *InterfaceUsuario* el desplegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Obtener Registro". Este evento se envía de la *InterfaceUsuario* al *ManejadorServicio*. El *ManejadorServicio* envía el evento *obtenerRegistroUsuario* al *ManejadorRegistroUsuario*.
- ▶ *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*. A continuación regresamos al caso de uso *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatosRegistro* que obtenga el registro correspondiente mediante *obtenerRegistroUsuario*. Nuevamente, la *InterfaceBaseDatosRegistro* le pasa un evento similar al actor *Bases de Datos de Registros*, el cual contesta con un OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorRegistroUsuario*.
- ▶ *Registrar Usuario* subflujo *Administrar Registro Usuario (S-3)*. A continuación se pasa al subflujo *Administrar Registro Usuario (S-3)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceUsuario* desplegar la pantalla de información de registro mediante el evento *desplegarPantallaObtenerRegUsuario*. Hasta aquí la secuencia es similar a *Actualizar Registro Usuario*. En este momento el usuario oprime el botón "Eliminar".
- ▶ *Registrar Usuario* subflujo *Eliminar Registro Usuario (S-3)*. Siguiendo con la lógica, la *InterfaceUsuario* envía el mismo evento al *ManejadorRegistroUsuario*, el cual es responsable de eliminar el registro, por lo cual envía el evento *eliminarRegistroUsuario* a la *InterfaceBaseDatosRegistro*. La *InterfaceBaseDatosRegistro* envía el evento *eliminarRegistroUsuario* al actor *Bases de Datos de Registros*. Este actor responde mediante un OK, el cual luego se envía de manera sucesiva hasta llegar al *ManejadorRegistroUsuario*.
- ▶ *Registrar Usuario* subflujo *Administrar Registro Usuario (S-3)*. A continuación se sigue al subflujo *Administrar Registro Usuario (S-3)*, donde el *ManejadorRegistroUsuario* envía el evento *desplegarPantallaCrearRegistroUsuario* a la *InterfaceUsuario*. En ese momento el usuario presiona "Saltar", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Registrar Usuario* con los subflujos *Obtener Registro Usuario (S-2)*, *Administrar Registro Usuario (S-3)* y *Eliminar Registro Usuario (S-4)*.

7.4.2 Registrar Tarjeta

En el caso de uso *Registrar Tarjeta* existen diversas secuencias que pueden ser instanciados por un usuario. En particular, se muestran diagramas de secuencias para *Crear Registro Tarjeta*, *Actualizar Registro Tarjeta* y *Eliminar Registro Tarjeta*. Note que estas secuencias incluirán obviamente a los subflujos *Crear Registro Tarjeta (S-1)*, *Actualizar Registro Tarjeta (S-4)* y *Eliminar Registro Tarjeta (S-5)*, de manera correspondiente, junto con los casos de uso *Validar Usuario* y *Ofrecer Servicios*, además de los subflujos *Obtener Registro Usuario (S-2)* y *Administrar Registro Usuario (S-3)* de *Registrar Usuario*, dado que *Registrar Tarjeta* es una extensión de éste.

CREAR REGISTRO TARJETA

El diagrama de secuencia para el subflujo *Crear Registro Tarjeta* se muestra en la figura 7.46. Esta secuencia inicia de manera similar a la secuencia *Actualizar Registro Usuario*, con la excepción de que en lugar de hacer una actualización, se crea un registro de tarjeta siguiendo el flujo principal de *Registrar Tarjeta*.

- ▶ *Validar Usuario*. La secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual responde con OK si la validación es buena. El OK se envía a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal*, como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió el último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- ▶ *Ofrecer Servicios*. A continuación podemos pasar al caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita entonces a la *InterfaceUsuario* el desplegado de la pantalla correspondiente mediante *desplegarPantalla-Servicio*. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Obtener Registro". Este evento se envía a la *Interface-Usuario* al *ManejadorServicio* mediante el mismo evento. El evento *obtener-RegistroUsuario* se envía luego por el *ManejadorServicio* al *Manejador-RegistroUsuario*.
- ▶ *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*. A continuación regresamos al caso de uso *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatos-Registro* que obtenga el registro correspondiente mediante el mismo evento. Nuevamente, la *InterfaceBaseDatosRegistro* pasa un evento similar al actor *Base de Datos de Registros*, el cual contesta con un OK. El OK es luego enviado por la *InterfaceBaseDatosRegistro* al *ManejadorRegistroUsuario*.
- ▶ *Registrar Usuario* subflujo *Administrar Registro Usuario (S-3)*. A continuación se sigue con el subflujo *Administrar Registro Usuario (S-3)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceUsuario* desplegar la pantalla de información de registro mediante el evento *desplegarPantallaObtener-RegUsuario*. Hasta aquí la secuencia es similar al subflujo *Actualizar Registro Usuario*. En este momento el usuario oprime el botón "Registrar Tarjeta".
- ▶ *Registrar Tarjeta* subflujo principal. Siguiendo con la lógica, la *Interface-Usuario* envía el mismo evento al *ManejadorRegistroUsuario*, el cual solicita *obtenerRegistroTarjeta* al *ManejadorRegistroTarjeta*.
- ▶ *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*. El *Manejador-RegistroTarjeta*, que es responsable de todo lo relacionado con el registro de tarjeta, solicita *obtenerRegistroTarjeta* a la *InterfaceBaseDatosRegistro*. La *InterfaceBaseDatosRegistro* envía el mismo evento al actor *Bases de Datos de Registros*. Este actor responde mediante un evento **NULO** correspondiente a un registro de tarjeta inexistente. Este evento se envía de regreso al *ManejadorRegistroTarjeta*.

- *Registrar Tarjeta* subflujo *Crear Registro Tarjeta (S-1)*. A continuación, el *ManejadorRegistroTarjeta* solicita *desplegarPantallaCrearRegistroTarjeta* a *InterfaceUsuario*. El usuario registra sus datos de tarjeta y presiona el botón de "Registrar", generando el evento *Registrar*. Como consecuencia de ello, *InterfaceUsuario* envía el mismo evento al *ManejadorRegistroTarjeta*, el cual envía el evento *crearRegistroTarjeta* a *InterfaceBaseDatosRegistro*. Este último envía el mismo evento al actor *Bases de Datos de Registros*, que contesta con un OK, que se envía sucesivamente de regreso a la *InterfaceBaseDatosRegistro*, para ser luego enviado al *ManejadorRegistroTarjeta*.
- *Registrar Tarjeta* subflujo *Administrar Registro Tarjeta (S-3)*. A continuación, el *ManejadorRegistroTarjeta* envía el evento *desplegarPantallaObtenerRegistroTarjeta* a la *InterfaceUsuario*. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Registrar Usuario* con los subflujos *Obtener Registro Usuario (S-2)*, *Administrar Registro Usuario (S-3)* y, finalmente, el flujo principal de *Registrar Tarjeta*, seguido por los subflujos *Crear Registro Tarjeta (S-1)*, *Obtener Registro Tarjeta (S-2)* y *Administrar Registro Tarjeta (S-3)*.

ACTUALIZAR REGISTRO TARJETA

El diagrama de secuencia *Actualizar Registro Tarjeta* se muestra en la figura 7.47. Esta secuencia es muy similar a *Crear Registro Tarjeta*, aunque en lugar de seguir al subflujo *Crear Registro Tarjeta (S-1)*, seguirán los subflujos *Obtener Registro Tarjeta (S-2)*, *Administrar Registro Tarjeta (S-3)* y obviamente *Actualizar Registro Tarjeta (S-4)*.

- *Validar Usuario*. La secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, que envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con OK si la validación es buena. El OK se envía a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió este último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- *Ofrecer Servicios*. A continuación el *ManejadorServicio* solicita entonces a la *InterfaceUsuario* el desplegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Obtener Registro". Este mismo evento se envía de la *InterfaceUsuario* al *ManejadorServicio*. El evento *obtenerRegistroUsuario* se manda luego por el *ManejadorServicio* al *ManejadorRegistroUsuario*.
- *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*. A continuación regresamos al caso de uso *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceBase-*

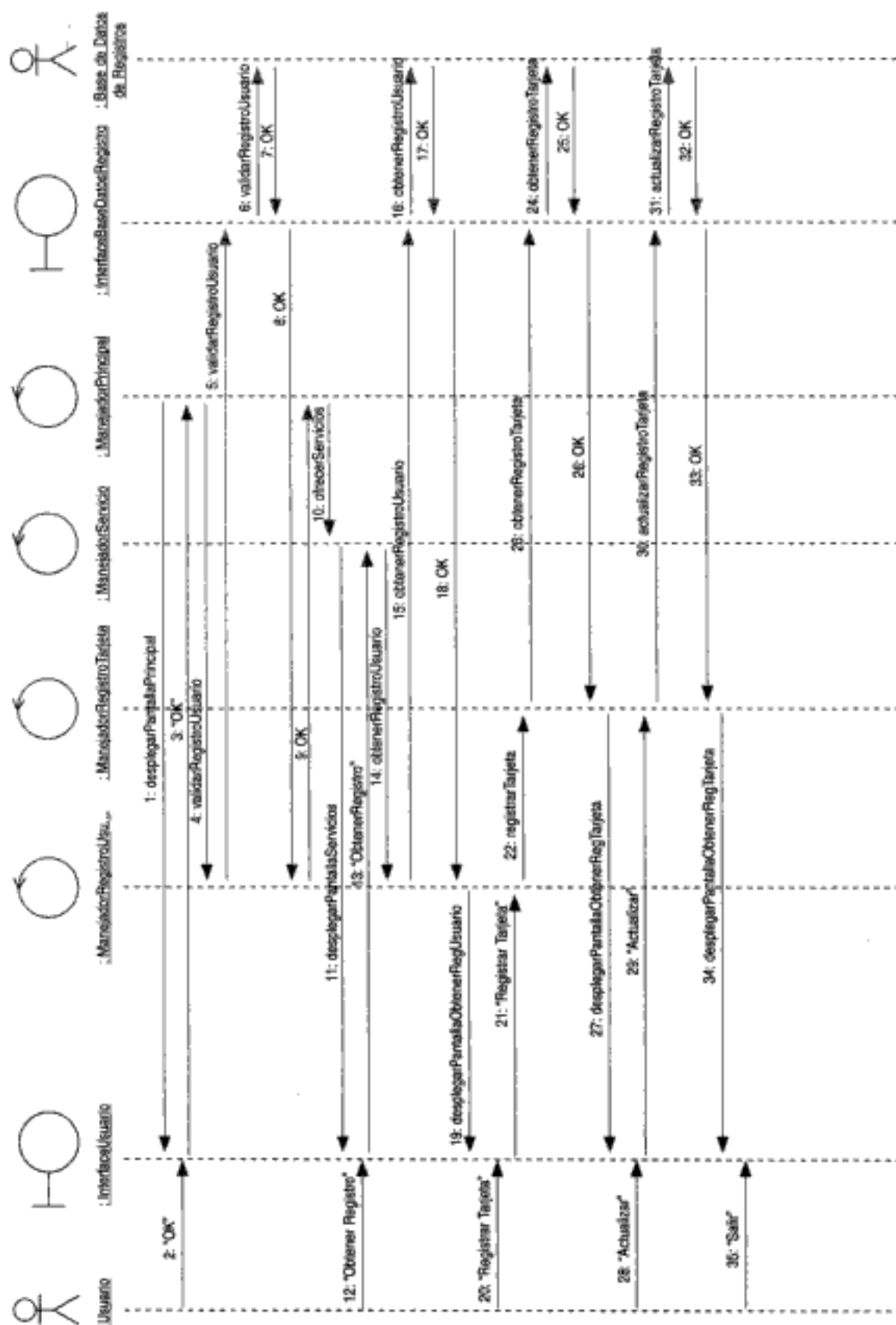


Figura 7.47 Diagrama de secuencia Actualizar Registro Tarjeta del caso de uso Registrar Tarjeta.

DatosRegistro que obtenga el registro correspondiente mediante el mismo evento. Nuevamente, la *InterfaceBaseDatosRegistro* le pasa un evento similar al actor *Base de Datos de Registros*, el cual contesta con un OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorRegistroUsuario*.

- ▶ *Registrar Usuario* subflujo *Administrar Registro Usuario (S-3)*. A continuación se pasa al subflujo *Administrar Registro Usuario (S-3)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceUsuario* desplegar la pantalla de información de registro mediante el evento *desplegarPantallaObtenerRegUsuario*. En este momento, el usuario oprime el botón "Registrar Tarjeta".
- ▶ *Registrar Tarjeta* subflujo principal. Siguiendo con la lógica, la *InterfaceUsuario* envía el mismo evento al *ManejadorRegistroUsuario*, el cual solicita *registrarTarjeta* al *ManejadorRegistroTarjeta*.
- ▶ *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*. El *ManejadorRegistroTarjeta* solicita *obtenerRegistroTarjeta* a la *InterfaceBaseDatosRegistro*, la cual envía el mismo evento al actor *Base de Datos de Registros*, que responde mediante un OK, correspondiente a un registro de tarjeta existente. Este evento es enviado de regreso al *ManejadorRegistroTarjeta*.
- ▶ *Registrar Tarjeta* subflujo *Administrar Registro Tarjeta (S-3)*. A continuación, el *ManejadorRegistroTarjeta* envía el evento *desplegarPantallaObtenerRegistroTarjeta* a *InterfaceUsuario*. El usuario actualiza sus datos de tarjeta y presiona el botón de "Actualizar", generando el evento con el mismo nombre.
- ▶ *Registrar Tarjeta* subflujo *Actualizar Registro Tarjeta (S-4)*. A continuación, *InterfaceUsuario* envía el mismo evento al *ManejadorRegistroTarjeta*, el cual solicita *actualizarRegistroTarjeta* a *InterfaceBaseDatosRegistro*. Este último envía el mismo evento al actor *Base de Datos de Registros*, el cual contesta con un OK que es sucesivamente enviado de regreso a la *InterfaceBaseDatosRegistro* y luego al *ManejadorRegistroTarjeta*.
- ▶ *Registrar Tarjeta* subflujo *Administrar Registro Tarjeta (S-3)*. A continuación, el *ManejadorRegistroTarjeta* envía el evento *desplegarPantallaObtenerRegTarjeta* a la *InterfaceUsuario*. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Registrar Usuario* con los subflujos *Obtener Registro Usuario (S-2)*, *Administrar Registro Usuario (S-3)* y, finalmente, el flujo principal de *Registrar Tarjeta*, seguido por los subflujos *Obtener Registro Tarjeta (S-2)*, *Administrar Registro Tarjeta (S-3)* y *Actualizar Registro Tarjeta (S-4)*.

ELIMINAR REGISTRO TARJETA

El diagrama de secuencia *Eliminar Registro Tarjeta* se muestra en la figura 7.48. Esta secuencia es muy similar a *Actualizar Registro Tarjeta*, siguiendo con los subflujos *Obtener Registro Tarjeta (S-2)*, *Administrar Registro Tarjeta (S-3)* y, obviamente, *Eliminar Registro Tarjeta (S-5)*.

- ▶ *Validar Usuario*. La secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* al *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*. El *ManejadorPrincipal* envía el evento *validarRegistroUsuario* al *ManejadorRegistro-*

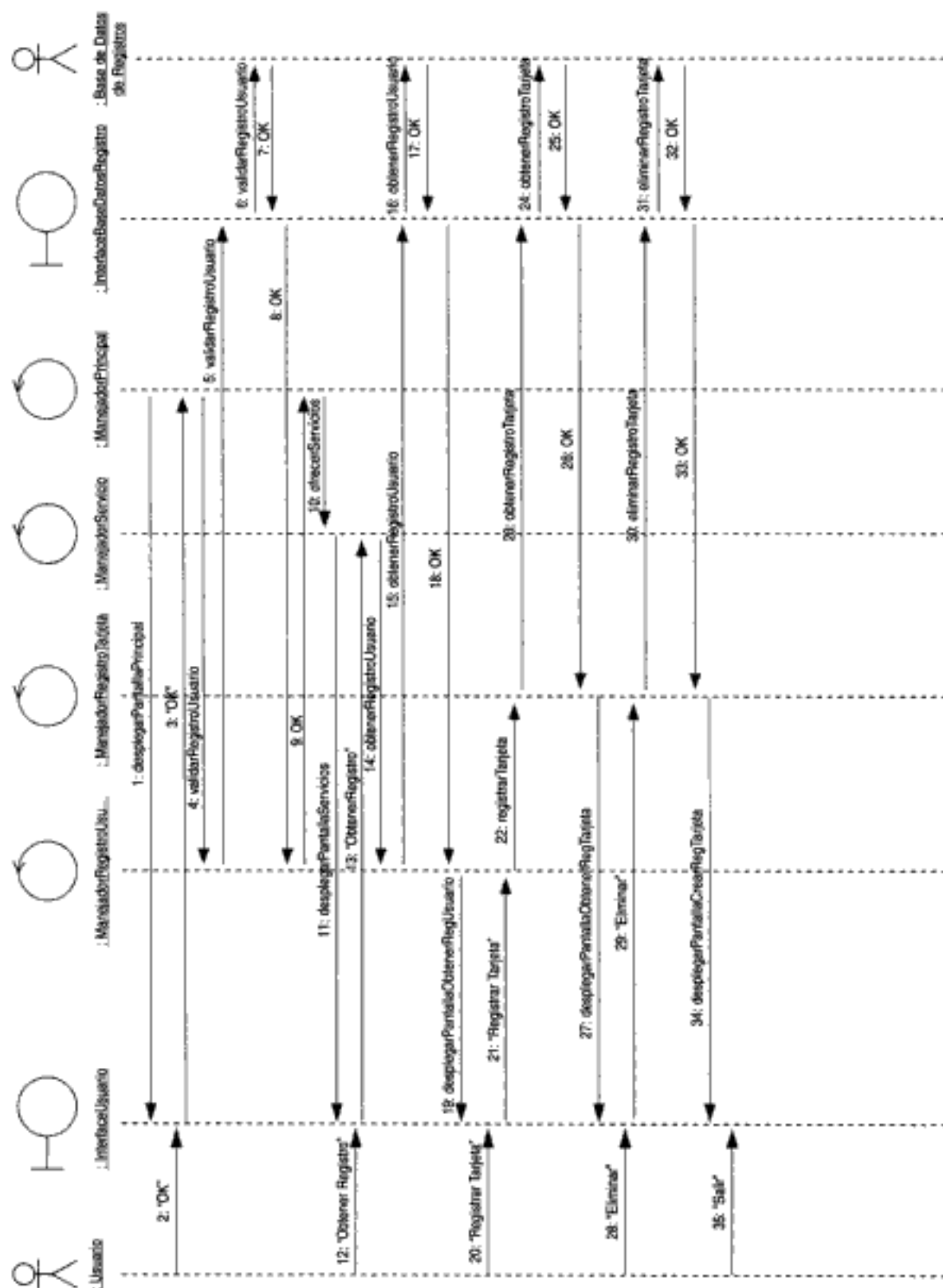


Figura 7.48 Diagrama de secuencia Eliminar Registro Tarjeta del caso de uso Registrar Tarjeta.

Usuario. El *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con un OK si la validación es buena. El OK es enviado a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió el último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.

- *Ofrecer Servicios*. A continuación, el *ManejadorServicio* solicita a la *InterfaceUsuario* el despliegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Obtener Registro". Este mismo evento se envía de la *InterfaceUsuario* al *ManejadorServicio*. Luego se envía el evento *obtenerRegistroUsuario* por el *ManejadorServicio* al *ManejadorRegistroUsuario*.
- *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*. A continuación regresamos al caso de uso *Registrar Usuario* subflujo *Obtener Registro Usuario (S-2)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatosRegistro* que obtenga el registro correspondiente mediante el mismo evento. Nuevamente, la *InterfaceBaseDatosRegistro* pasa un evento similar al actor *Base de Datos de Registros*, el cual responde un OK, que luego se manda por la *InterfaceBaseDatosRegistro* al *ManejadorRegistroUsuario*.
- *Registrar Usuario* subflujo *Administrar Registro Usuario (S-3)*. A continuación se analiza el subflujo *Administrar Registro Usuario (S-3)*, donde el *ManejadorRegistroUsuario* solicita a la *InterfaceUsuario* desplegar la pantalla de información de registro mediante el evento *desplegarPantallaObtenerRegistroUsuario*. En este momento, el usuario oprime el botón "Registrar Tarjeta".
- *Registrar Tarjeta* subflujo principal. Siguiendo con la lógica, la *InterfaceUsuario* envía el mismo evento al *ManejadorRegistroUsuario*, el cual envía el evento *registrarTarjeta* al *ManejadorRegistroTarjeta*.
- *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*. El *ManejadorRegistroTarjeta* solicita *obtenerRegistroTarjeta* a la *InterfaceBaseDatosRegistro*. Ésta envía el mismo evento al actor *Base de Datos de Registros*, que a su vez responde mediante un OK correspondiente a un registro de tarjeta existente. Este evento se envía de regreso al *ManejadorRegistroTarjeta*.
- *Registrar Tarjeta* subflujo *Administrar Registro Tarjeta (S-3)*. A continuación, el *ManejadorRegistroTarjeta* envía el evento *desplegarPantallaObtenerRegistroTarjeta* a *InterfaceUsuario*. Hasta aquí la secuencia es similar al subflujo *Actualizar Registro Tarjeta*, ya que el usuario presiona el botón de "Eliminar", generando un evento con el mismo nombre.
- *Registrar Tarjeta* subflujo *Eliminar Registro Tarjeta (S-5)*. Como consecuencia del evento "Eliminar", la *InterfaceUsuario* envía el evento con el mismo nombre al *ManejadorRegistroTarjeta*, el cual solicita *registrarTarjeta* al *ManejadorRegistroTarjeta*, que envía el evento *eliminarRegistroTarjeta* a *InterfaceBaseDatosRegistro*. Este último envía el mismo evento al actor *Base de Datos de Registros*, el cual contesta con un OK que se envía sucesivamente de regreso a la *InterfaceBaseDatosRegistro* y luego al *ManejadorRegistroTarjeta*.
- *Registrar Tarjeta* subflujo *Administrar Registro Tarjeta (S-3)*. A continuación, el *ManejadorRegistroTarjeta* envía el evento *desplegarPantallaCrearReg-*

Tarjeta a la *InterfaceUsuario*. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para continuar en el caso de uso *Registrar Usuario* con los subflujos *Obtener Registro Usuario (S-2)*, *Administrar Registro Usuario (S-3)* y finalmente el flujo principal de *Registrar Tarjeta*, seguido por los subflujos *Obtener Registro Tarjeta (S-2)*, *Administrar Registro Tarjeta (S-3)* y *Eliminar Registro Tarjeta (S-5)*.

7.4.3 Consultar Información

En el caso de uso *Consultar Información* existen diversos subflujos que pueden ser instanciados por un usuario. En particular, mostraremos diagramas de secuencias para *Consultar Horarios*, *Consultar Tarifas* y *Consultar Estado*. Observe que estas secuencias incluirán obviamente a los subflujos *Consultar Horarios (S-2)*, *Consultar Tarifas (S-3)* y *Consultar Estado (S-4)*, respectivamente. Además, se incluirán parte de los casos de uso de *Validar Usuario* y *Ofrecer Servicios*, además de subflujos adicionales dentro del caso de uso *Consultar Información*.

CONSULTAR HORARIOS

El diagrama de secuencia *Consultar Horarios* se muestra en la figura 7.49. Esta secuencia inicia en el flujo principal de *Consultar Información*, que incluye la validación del usuario en *Validar Usuario* seguidos por *Ofrecer Servicios* y los subflujos *Consultar (S-1)*, *Consultar Horarios (S-2)* y *Devolver Horarios (S-3)*.

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal*, de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. El *ManejadorRegistroUsuario* solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Bases de Datos Registros*, el cual contesta con OK si la validación es buena. El OK se envía a la *InterfaceBaseDatosRegistro*, de allí a *ManejadorRegistroUsuario* y luego a *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió el OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- *Ofrecer Servicios*. A continuación estudiamos el caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita a la *InterfaceUsuario* el despliegue de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Consultar Información". Se envía el mismo evento de la *InterfaceUsuario* al *ManejadorServicio*. El evento *consultarInformación* se envía por el *ManejadorServicio* al *ManejadorConsultas*.
- *Consultar Información* subflujo *Consultar (S-1)*. A continuación el *ManejadorConsultas* envía el evento *desplegarPantallaConsultas* a *InterfaceUsuario*. El usuario presiona "Horarios", lo cual genera que la *InterfaceUsuario* envíe

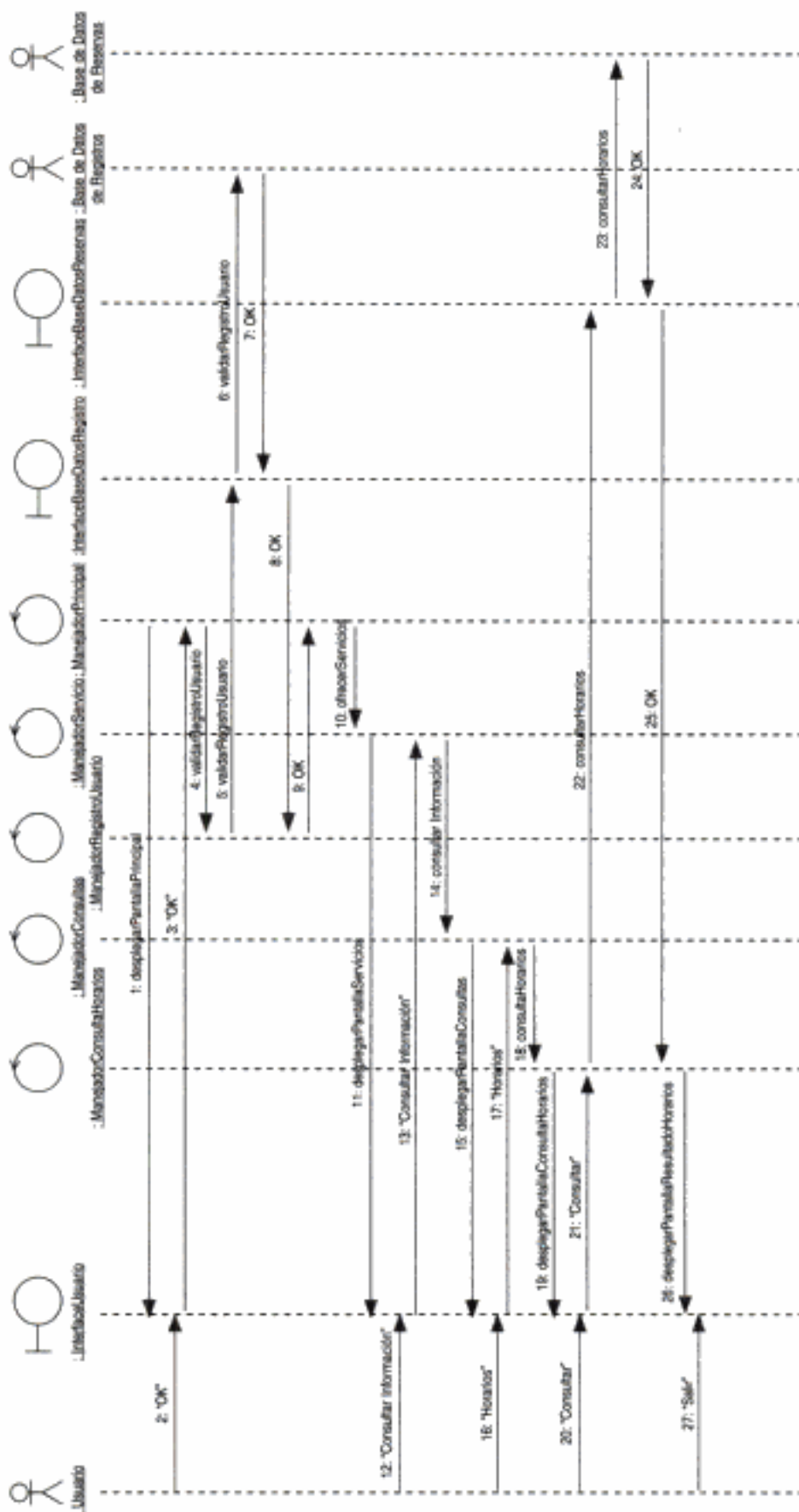


Figura 7.49 Diagrama de secuencia Consultar Horarios del caso de uso Consultar Información.

este evento de regreso a *ManejadorConsultas*, que a su vez envía el evento *consultarHorarios* al *ManejadorConsultaHorarios*.

- *Consultar Información* subflujo *Consultar Horarios (S-2)*. A continuación el *ManejadorConsultaHorarios* envía el evento *desplegarPantallaConsultaHorarios* a la *InterfaceUsuario*. El usuario llena la información de la consulta y oprime el botón "Consultar", el cual genera el mismo evento de *InterfaceUsuario* al *ManejadorConsultaHorarios*. Este último envía el evento *consultarHorarios* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, que contesta con OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorConsultaHorarios*.
- *Consultar Información* subflujo *Devolver Horarios (S-3)*. A continuación el *ManejadorConsultaHorarios* envía el evento *desplegarPantallaResultadoHorarios* a la *InterfaceUsuario* para desplegar los resultados de la búsqueda. En ese momento el *Usuario* presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar con el caso de uso *Consultar Información* con los subflujos *Consultar (S-1)*, *Consultar Horarios (S-2)* y *Devolver Horarios (S-3)*.

CONSULTAR TARIFAS

El diagrama de secuencia *Consultar Tarifas* se muestra en la figura 7.50. Esta secuencia inicia en el flujo principal de *Consultar Información*, que incluye la validación del usuario en *Validar Usuario* seguidos por *Ofrecer Servicios* y los subflujos *Consultar (S-1)*, *Consultar Tarifas (S-4)* y *Devolver Tarifas (S-5)*.

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, que envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con OK si la validación es buena. El OK se envía a la *InterfaceBaseDatosRegistro*, de allí a *ManejadorRegistroUsuario* y luego a *ManejadorPrincipal*, como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió el OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- *Ofrecer Servicios*. A continuación podemos pasar al caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita a la *InterfaceUsuario* que despliegue la pantalla correspondiente mediante *desplegarPantallaServicio*. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Consultar Información". Se envía el mismo evento de la *InterfaceUsuario* al *ManejadorServicio*. El evento *consultarInformación* se envía luego mediante el *ManejadorServicio* al *ManejadorConsultas*.
- *Consultar Información* subflujo *Consultar (S-1)*. A continuación el *ManejadorConsultas* envía el evento *desplegarPantallaConsultas* a la *InterfaceUsuario*. Hasta aquí la secuencia es similar a la secuencia *Consultar Horarios*. En esta nueva secuencia, el usuario presiona "Tarifas", lo cual genera que la

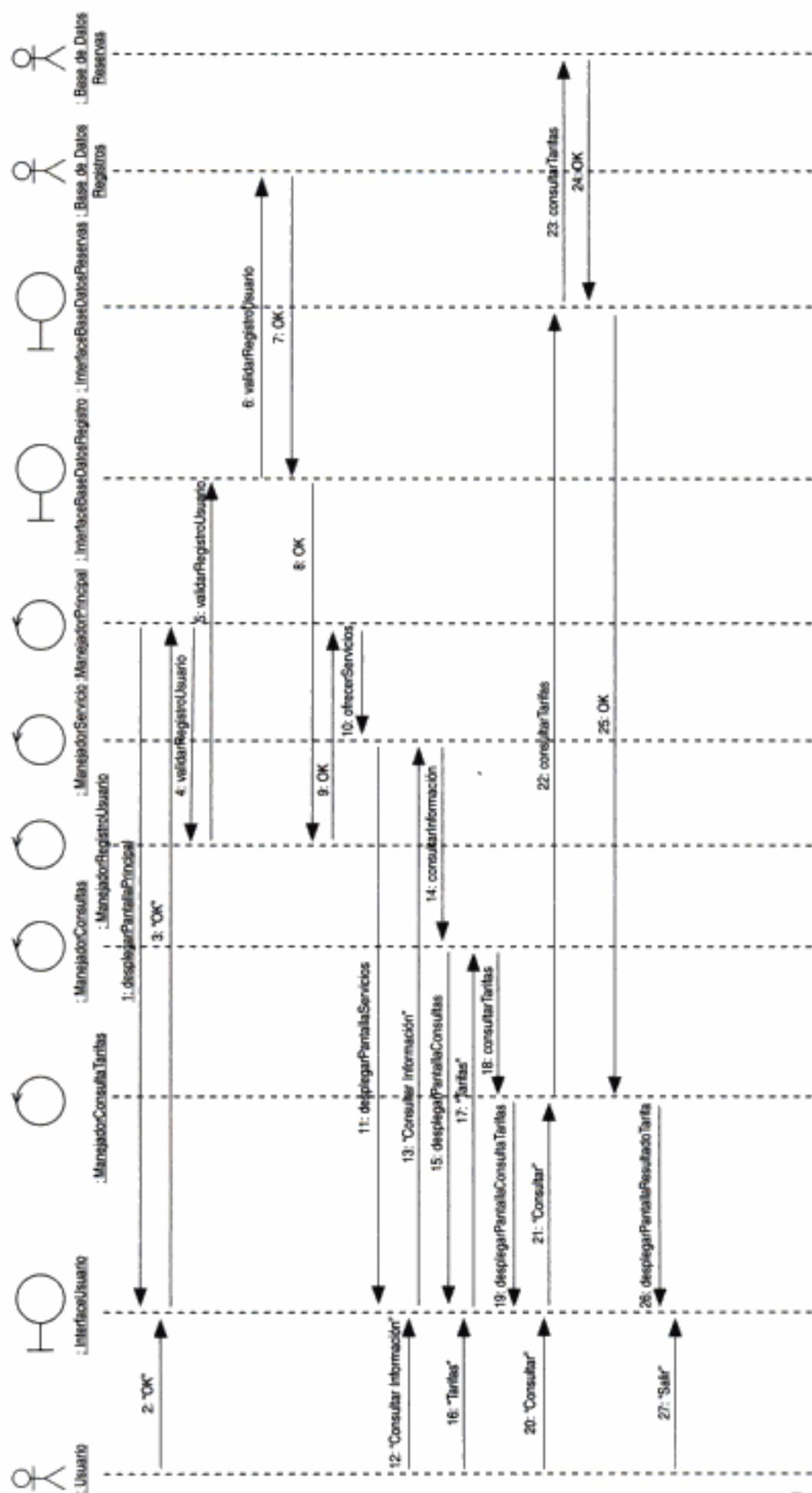


Figura 7.50 Diagrama de secuencia Consultar Tarifas del caso de uso Consultar Información.

InterfaceUsuario envíe este evento de regreso a *ManejadorConsultas*, que manda el evento *consultarTarifas* al *ManejadorConsultaTarifas*.

- *Consultar Información* subflujo *Consultar Tarifas* (S-4). A continuación el *ManejadorConsultaTarifas* envía el evento *desplegarPantallaConsultaTarifas* a *InterfaceUsuario*. El usuario llena la información de la consulta y oprime el botón "Consultar", el cual genera el mismo evento de *InterfaceUsuario* al *ManejadorConsultaTarifas*. Este último envía el evento *consultarTarifas* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, el cual contesta con un OK, que se manda por la *InterfaceBaseDatosRegistro* al *ManejadorConsultaTarifas*.
- *Consultar Información* subflujo *Devolver Tarifas* (S-5). A continuación el *ManejadorConsultaTarifas* envía el evento *desplegarPantallaResultadoTarifas* a la *InterfaceUsuario* para desplegar los resultados de la búsqueda.

En ese momento el usuario presiona "Saltar", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario* seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Consultar Información* con los subflujos *Consultar* (S-1), *Consultar Tarifas* (S-4) y *Devolver Tarifas* (S-5).

CONSULTAR ESTADO

El diagrama de secuencia *Consultar Estado*, de la figura 7.51, inicia en el flujo principal de *Consultar Información*, que incluye la validación del usuario en *Validar Usuario*, seguidos por *Ofrecer Servicios* y los subflujos *Consultar* (S-1), *Consultar Estado* (S-6) y *Devolver Estado* (S-7).

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal*, de la clase *ManejadorPrincipal*, a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual lo envía a *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros* el cual responde OK si la validación es buena. El OK es enviado a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal*, como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió este último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- *Ofrecer Servicios*. A continuación se pasa al caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita a la *InterfaceUsuario* el despliegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica de este subflujo, el usuario debe presionar "Consultar Información". Se envía el mismo evento de la *InterfaceUsuario* al *ManejadorServicio*. El evento *consultarInformación* se envía luego por el *ManejadorServicio* al *ManejadorConsultas*.
- *Consultar Información* subflujo *Consultar* (S-1). A continuación el *ManejadorConsultas* envía el evento *desplegarPantallaConsultas* a la *InterfaceUsuario*. Hasta aquí la secuencia es similar a la secuencia *Consultar Horarios* y

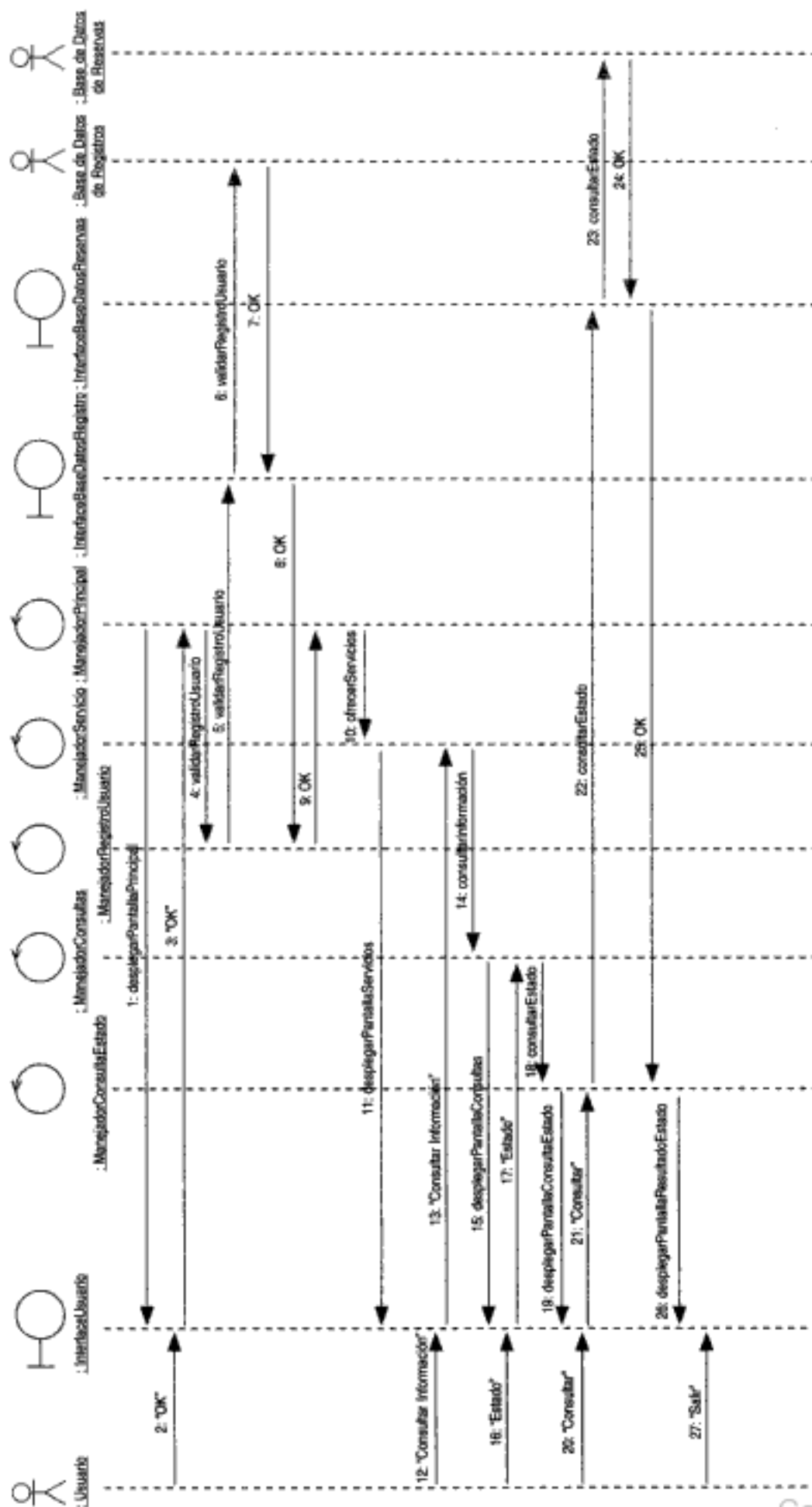


Figura 7.51 Diagrama de secuencia Consultar Estado del caso de uso Consultar Información.

Consultar Tarifas. En esta nueva secuencia, el usuario presiona "Estado", lo cual genera que la *InterfaceUsuario* envíe este evento de regreso a *ManejadorConsultas*, el cual envía el evento *consultarEstado* al *ManejadorConsultaEstado*.

- *Consultar Información* subflujo *Consultar Estado* (S-6). A continuación el *ManejadorConsultaEstado* envía el evento *desplegarPantallaConsultaEstado* a *InterfaceUsuario*. El usuario llena la información de la consulta y oprime el botón "Consultar", que genera el mismo evento de *InterfaceUsuario* al *ManejadorConsultaEstado*. Este último envía el evento *consultarEstado* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, el cual contesta con OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorConsultaEstado*.
- *Consultar Información* subflujo *Devolver Estado* (S-7). A continuación el *ManejadorConsultaEstado* envía el evento *desplegarPantallaResultadoEstado* a la *InterfaceUsuario* para desplegar los resultados de la búsqueda. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Consultar Información* con los subflujos *Consultar* (S-1), *Consultar Estado* (S-4) y *Devolver Estado* (S-7).

7.4.4 Hacer Reservación

En el caso de uso *Hacer Reservación* existen diversos subflujos que pueden ser instanciados por un usuario. En particular mostraremos diagramas de secuencias para *Crear Reservación*, *Actualizar Reservación* y *Eliminar Reservación*. Observe que estas secuencias incluirán obviamente a los subflujos *Crear Reservación* (S-2), *Actualizar Reservación* (S-5) y *Eliminar Reservación* (S-6), respectivamente. Además se incluirá parte de los casos de uso de *Validar Usuario* y *Ofrecer Servicios*, además de subflujos adicionales dentro del caso de uso *Hacer Reservación*.

CREAR RESERVACIÓN

El diagrama de secuencia *Crear Reservación* se muestra en la figura 7.52. Esta secuencia inicia en el flujo principal de *Hacer Reservación*, que incluye la validación del usuario en *Validar Usuario* seguidos por *Ofrecer Servicios* y los subflujos *Solicitar Clave Reservación* (S-1), *Crear Reservación* (S-2) y *Administrar Reservación* (S-4).

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con OK si la validación es buena, y a su vez es enviado a la *InterfaceBaseDatosRegistro*, de allí a *ManejadorRegistroUsuario* y luego a *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal*

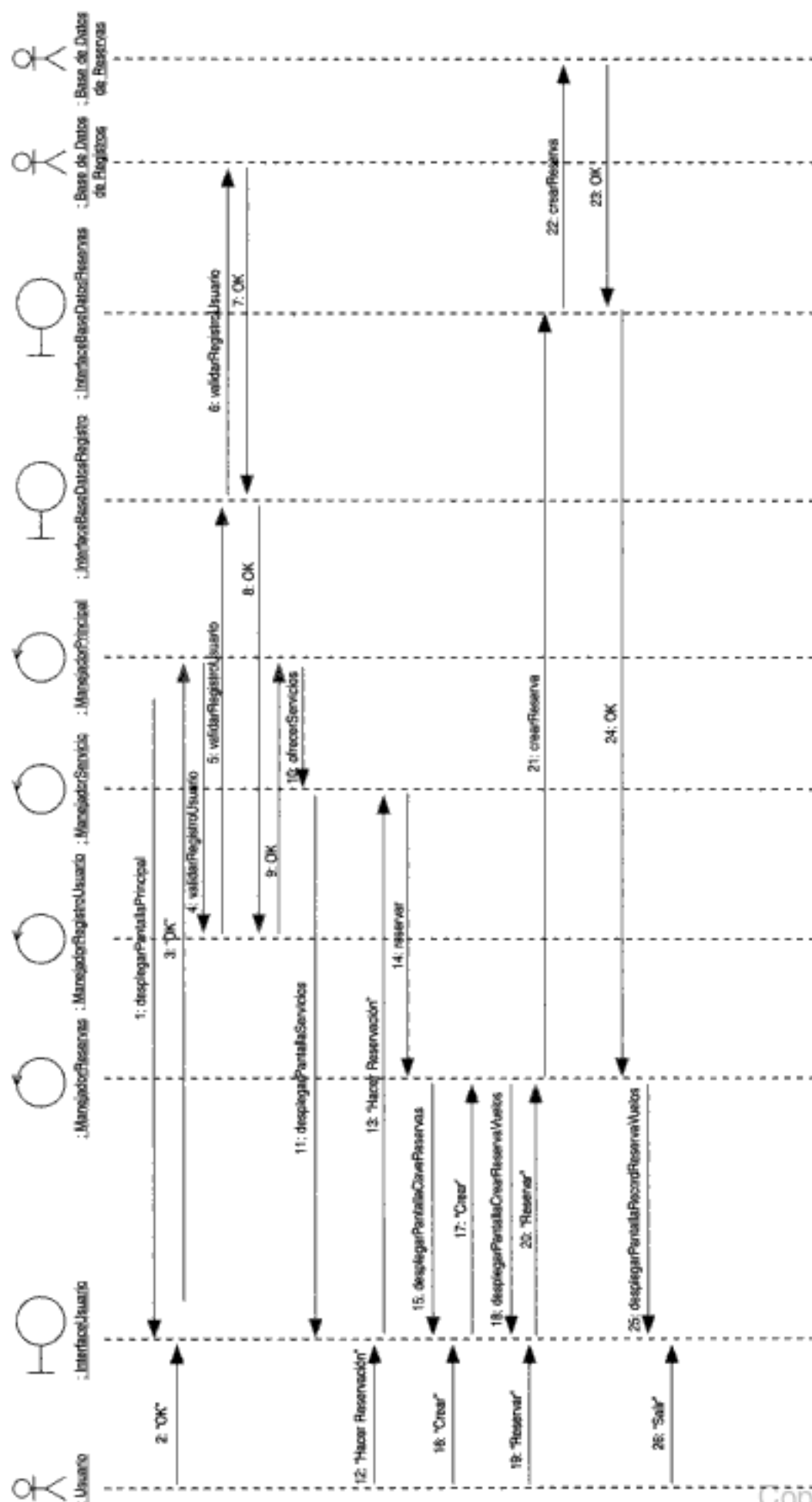


Figura 7.52 Diagrama de secuencia Crear Reservación del caso de uso Hacer Reservación.

recibió el último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.

- *Ofrecer Servicios*. A continuación se analiza el caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita entonces a la *InterfaceUsuario* el desplegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Hacer Reservación". Se envía el mismo evento de la *InterfaceUsuario* al *ManejadorServicio*. Posteriormente el *ManejadorServicio* envía el evento *Reservar* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Solicitar Clave Reservación (S-1)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaClaveReservas* a *InterfaceUsuario*. El usuario presiona el botón "Crear". La *InterfaceUsuario* envía el mismo evento al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Crear Reservación (S-2)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaCrearReservaVuelos* a *InterfaceUsuario*. El usuario presiona "Reservar". La *InterfaceUsuario* envía el mismo evento al *ManejadorReservas*, el cual envía el evento *crearReserva* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, que contesta con un OK. Éste se envía luego por la *InterfaceBaseDatosRegistro* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a la *InterfaceUsuario* para desplegar los resultados de la solicitud de reservación. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Hacer Reservación* con los subflujos *Solicitar Clave Reservación (S-1)*, *Crear Reservación (S-2)* y *Administrar Reservación (S-4)*.

ACTUALIZAR RESERVACIÓN

El diagrama de secuencia *Actualizar Reservación* se muestra en la figura 7.53. Esta secuencia inicia en el flujo principal de *Hacer Reservación*, que incluye la validación del usuario en *Validar Usuario*, seguido por *Ofrecer Servicios* y los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)*, *Administrar Reservación (S-4)* y *Actualizar Reservación (S-5)*.

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, que envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con OK si la validación es buena. El OK se envía a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Ya que el *ManejadorPrincipal* recibió el último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.

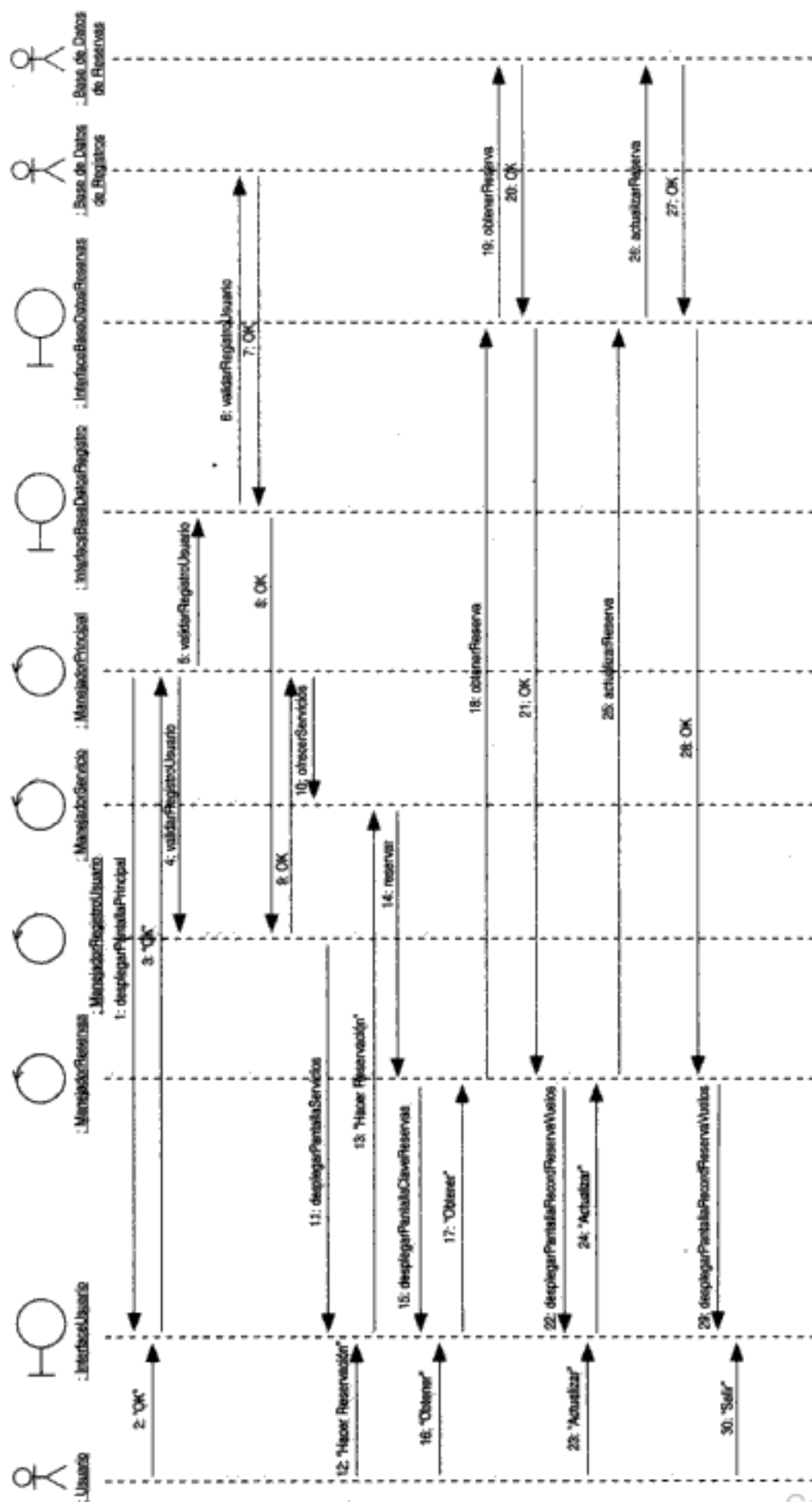


Figura 7.53 Diagrama de secuencia Actualizar Reservación del caso de uso Hacer Reservación.

- *Ofrecer Servicios*. A continuación se pasa al caso de uso *Ofrecer Servicios* donde el *ManejadorServicio* solicita a la *InterfaceUsuario* el despliegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "*Hacer Reservación*". Se genera un evento similar que se envía de la *InterfaceUsuario* al *ManejadorServicio*, el cual envía el evento *reservar* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Solicitar Clave Reservación (S-1)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaClaveReservas* a la *InterfaceUsuario*. El usuario presiona el botón "*Obtener*", junto con la inserción de la clave de récord de reservas. La *InterfaceUsuario* envía un evento similar de regreso al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Obtener Reservación (S-3)*. A continuación el *ManejadorReservas* envía el evento *obtenerReserva* a la *InterfaceBaseDatosReservas*, la cual a su vez envía este evento al actor *Base de Datos de Reservas*. El actor genera un OK si todo es correcto y se lo reenvía a *InterfaceBaseDatosReservas*, la cual lo manda al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a *InterfaceUsuario*. El usuario actualiza los datos de su reservación y presiona "*Actualizar*".
- *Hacer Reservación* subflujo *Actualizar Reservación (S-5)*. A continuación la *InterfaceUsuario* envía el evento "*Actualizar*" de regreso al *ManejadorReservas*. Éste solicita *actualizarReserva* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, el cual contesta con un OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a la *InterfaceUsuario* para desplegar los resultados de la actualización. En ese momento el usuario presiona "*Salir*", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Hacer Reservación* con los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)*, *Administrar Reservación (S-4)* y *Actualizar Reservación (S-5)*.

ELIMINAR RESERVACIÓN

El diagrama de secuencia *Eliminar Reservación* se muestra en la figura 7.54, la cual inicia en el flujo principal de *Hacer Reservación* que incluye la validación del usuario en *Validar Usuario*, seguidos por *Ofrecer Servicios* y los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)*, *Administrar Reservación (S-4)* y *Eliminar Reservación (S-6)*.

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. El *Manejador-*

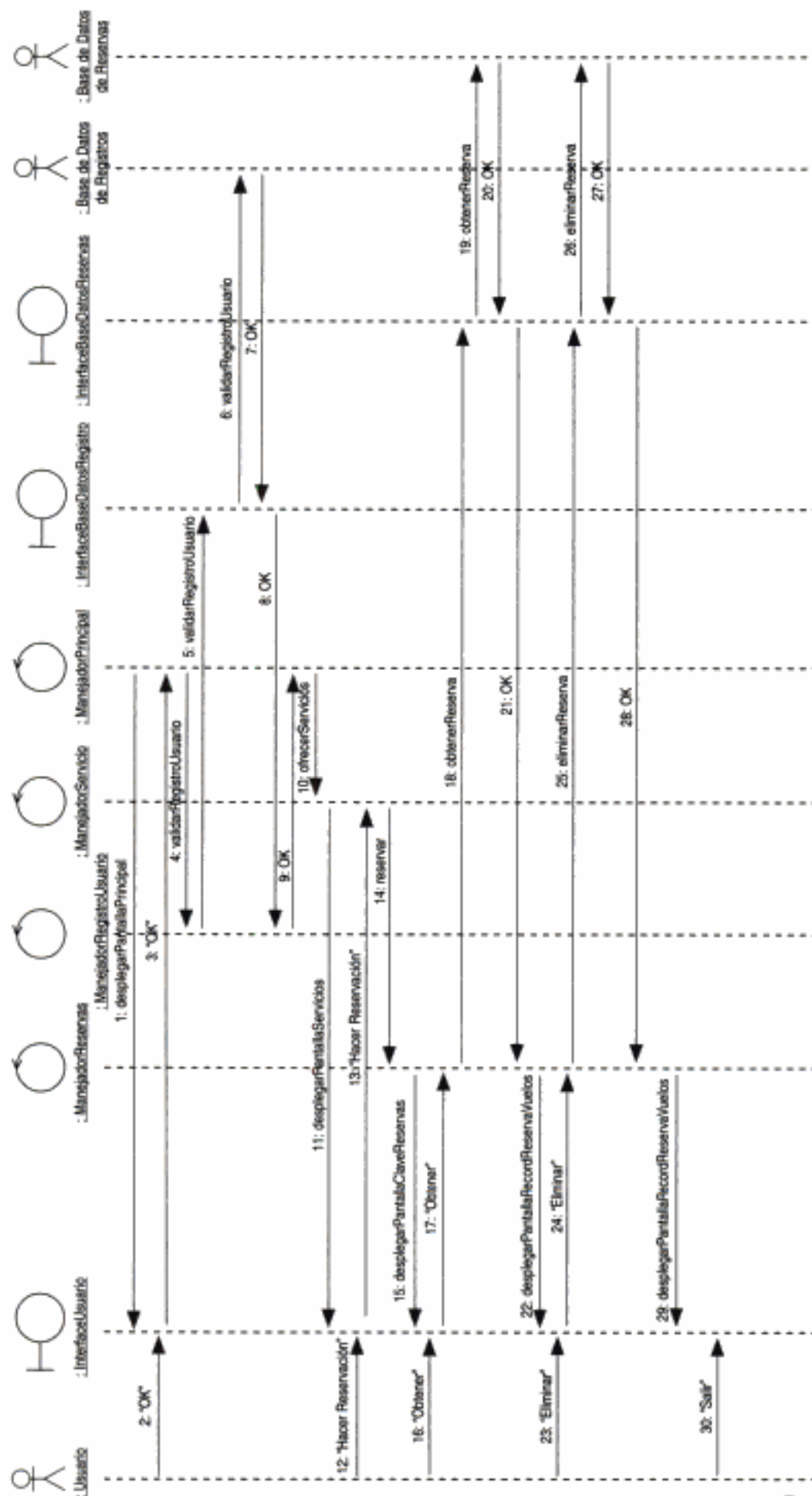


Figura 7.54 Diagrama de secuencia *Eliminar Reservación* del caso de uso *Hacer Reservación*.

RegistroUsuario solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con un OK si la validación es buena, que se envía a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal*, como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió el último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.

- *Ofrecer Servicios*. A continuación podemos pasar al caso de uso *Ofrecer Servicios* donde el *ManejadorServicio* solicita a la *InterfaceUsuario* el desplegado de la pantalla correspondiente a través de *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Hacer Reservación". Se envía un evento similar de la *InterfaceUsuario* al *ManejadorServicio*, que envía el evento *reservar* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Solicitar Clave Reservación (S-1)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaClaveReservas* a *InterfaceUsuario*. El usuario presiona el botón "Obtener", junto con la inserción de la clave de récord de reservas. La *InterfaceUsuario* envía un evento similar de regreso al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Obtener Reservación (S-3)*. A continuación el *ManejadorReservas* envía el evento *obtenerReserva* a la *InterfaceBaseDatosReservas*, la cual a su vez envía este evento al actor *Base de Datos de Reservas*. El actor genera un OK si todo es correcto y se lo envía de regreso a *InterfaceBaseDatosReservas*, la cual luego lo envía al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a *InterfaceUsuario*. Hasta aquí la secuencia es similar a *Actualizar Reservación*. En este momento el usuario presiona "Eliminar".
- *Hacer Reservación* subflujo *Eliminar Reservación (S-6)*. A continuación la *InterfaceUsuario* envía un evento similar de regreso al *ManejadorReservas*, el cual envía el evento *eliminarReserva* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, el cual contesta con un OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Crear Reservación (S-2)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaCrearReservaVuelos* a la *InterfaceUsuario* para desplegar una nueva pantalla de creación de reservas. En ese momento el usuario presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Hacer Reservación* con los subflujos *Solicitar Clave Reservación (S-1)*, *Crear Reservación (S-2)*, *Obtener Reservación (S-3)*, *Administrar Reservación (S-4)* y *Eliminar Reservación (S-6)*.

7.4.5 Pagar Reservación

En el caso de uso *Pagar Reservación* existen diversos subflujos que puede instanciar un usuario. En particular, mostraremos diagramas de secuencias para *Pagar Reservación* y *Reembolsar Pago*. Note que estas secuencias incluirán ob-

viamente a los subflujos *Pagar Reservación (S-1)* y *Reembolsar Pago (S-2)*, respectivamente. Además se incluirá parte de los casos de uso de *Validar Usuario* y *Ofrecer Servicios*, los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)* y *Administrar Reservación (S-4)*, *Hacer Reservación*, además de subflujos dentro del caso de uso *Pagar Reservación*.

PAGAR RESERVACIÓN

El diagrama de secuencia *Pagar Reservación* se muestra en la figura 7.55. Esta secuencia inicia en el flujo principal de *Pagar Reservación* que incluye, por ser una extensión, la validación del usuario en *Validar Usuario*, seguido por *Ofrecer Servicios*, los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)* y *Administrar Reservación (S-4)*, del caso de uso *Hacer Reservación*. Posteriormente, se ejecuta el flujo principal de *Pagar Reservación*, junto con el subflujo *Pagar Reservación (S-1)*. Además de esto, se ejecuta el subflujo *Obtener Registro Tarjeta (S-2)* del caso de uso *Registrar Tarjeta*.

- *Validar Usuario*. Esta secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, el cual envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el evento a la *Base de Datos de Registros*, el cual contesta con un OK si la validación es buena. El OK es enviado a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Ya que el *ManejadorPrincipal* recibió este último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.
- *Ofrecer Servicios*. A continuación se estudia el caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita a la *InterfaceUsuario* el desplegado de la pantalla correspondiente mediante *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "Hacer Reservación" y debe incluir un número correspondiente a la clave de reservación. Se envía un evento similar de la *InterfaceUsuario* al *ManejadorServicio*, que envía el evento *reservar* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Solicitar Clave Reservación (S-1)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaClaveReservas* a *InterfaceUsuario*. El usuario presiona el botón "Obtener", junto con la inserción de la clave de récord de reservas. La *InterfaceUsuario* envía el mismo evento de regreso al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Obtener Reservación (S-3)*. A continuación el *ManejadorReservas* envía el evento *obtenerReserva* a la *InterfaceBaseDatosReservas*, la cual a su vez envía este evento al actor *Base de Datos de Reservas*, quien genera un OK si todo es correcto y se lo envía de regreso a *InterfaceBaseDatosReservas*, la cual luego lo envía al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReserva-Vuelos* a *InterfaceUsuario*. Hasta aquí la secuencia es similar a *Actualizar Reservación* y *Eliminar Reservación* del caso de uso *Hacer Reservación*. En

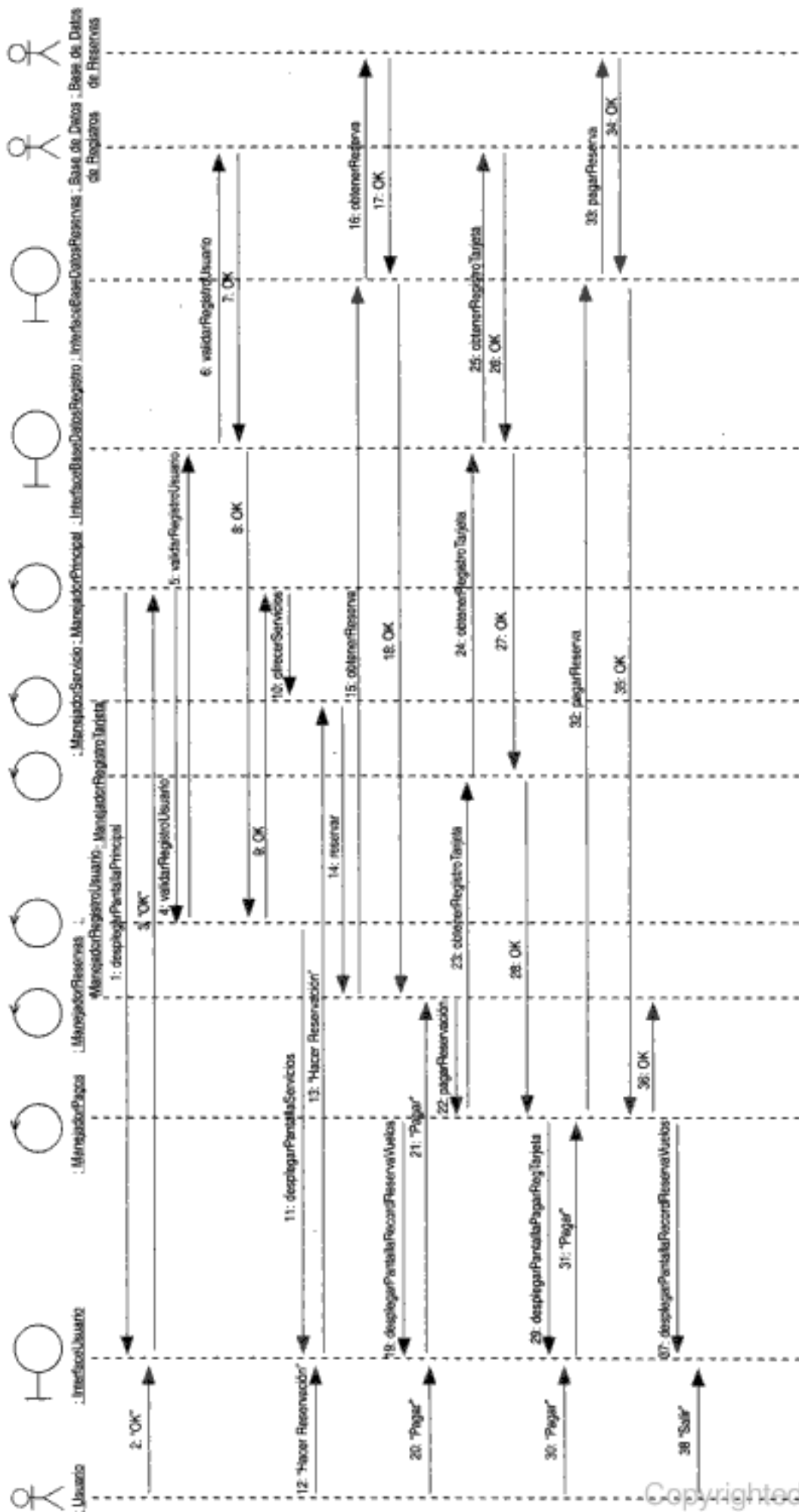


Figura 7.55 Diagrama de secuencia *Pagar Reservación* del caso de uso *Pagar Reservación*.

este momento, el usuario presiona "Pagar". La *InterfaceUsuario* envía el mismo evento de regreso al *ManejadorReservas*, que envía el evento *pagarReserva* al *ManejadorPagos*.

- ▶ *Pagar Reservación* flujo principal. A continuación el *ManejadorPagos* envía el evento *obtenerRegistroTarjeta* al *ManejadorRegistroTarjeta*.
- ▶ *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*. A continuación el *ManejadorRegistroTarjeta* envía el evento *obtenerRegistroTarjeta* a la *InterfaceBaseDatosRegistro*, la cual lo reenvía al actor *Bases de Datos Registros*. Este actor regresa un *OK* a la *InterfaceBaseDatosRegistro*, que a su vez lo envía de regreso a *ManejadorRegistroTarjeta*.
- ▶ *Pagar Reservación* flujo principal (continuación anterior). A continuación el *ManejadorRegistroTarjeta* responde *OK* al *ManejadorPagos*.
- ▶ *Pagar Reservación* subflujo *Pagar Reservación (S-1)*. A continuación el *ManejadorPagos* envía el evento *desplegarPantallaPagarRegTarjeta* a la *InterfaceUsuario*. El usuario genera el evento "Pagar". La *InterfaceUsuario* recibe la petición y envía el mismo evento al *ManejadorPagos*. Éste a su vez envía el evento *pagarReserva* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, el cual contesta con un *OK*, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorPagos*, el cual envía el *OK* al *ManejadorReservas*.
- ▶ *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a la *InterfaceUsuario* para desplegar los resultados del pago de reserva. En ese momento el *Usuario* presiona "Salir", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario*, seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Hacer Reservación* con los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)* y *Administrar Reservación (S-4)*, y finalizar en el caso de uso *Pagar Reservación* con el flujo principal y el subflujo *Pagar Reservación (S-1)*. Se incluye también el caso de uso *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*.

REEMBOLSAR PAGO

El diagrama de secuencia *Reembolsar Pago* se muestra en la figura 7.56. Esta secuencia es muy similar a *Pagar Reservación*, iniciando en el flujo principal de *Pagar Reservación* que incluye, por ser una extensión, la validación del usuario en *Validar Usuario*, seguido por *Ofrecer Servicios*, los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)* y *Administrar Reservación (S-4)*, del caso de uso *Hacer Reservación*. Posteriormente se ejecuta el flujo principal de *Pagar Reservación* junto con el subflujo *Reembolsar Pago (S-2)*. Además de esto, se ejecuta el subflujo *Obtener Registro Tarjeta (S-2)* del caso de uso *Registrar Tarjeta*.

- ▶ *Validar Usuario*. La secuencia comienza con la validación del usuario a partir del evento *desplegarPantallaPrincipal* de la clase *ManejadorPrincipal* a la *InterfaceUsuario*. El usuario se valida enviando el evento "OK". La *InterfaceUsuario* envía el mismo evento al *ManejadorPrincipal*, que envía el evento *validarRegistroUsuario* al *ManejadorRegistroUsuario*. Éste solicita a la *InterfaceBaseDatosRegistro* que haga una validación del usuario mediante el mismo evento. La *InterfaceBaseDatosRegistro* envía a su vez el

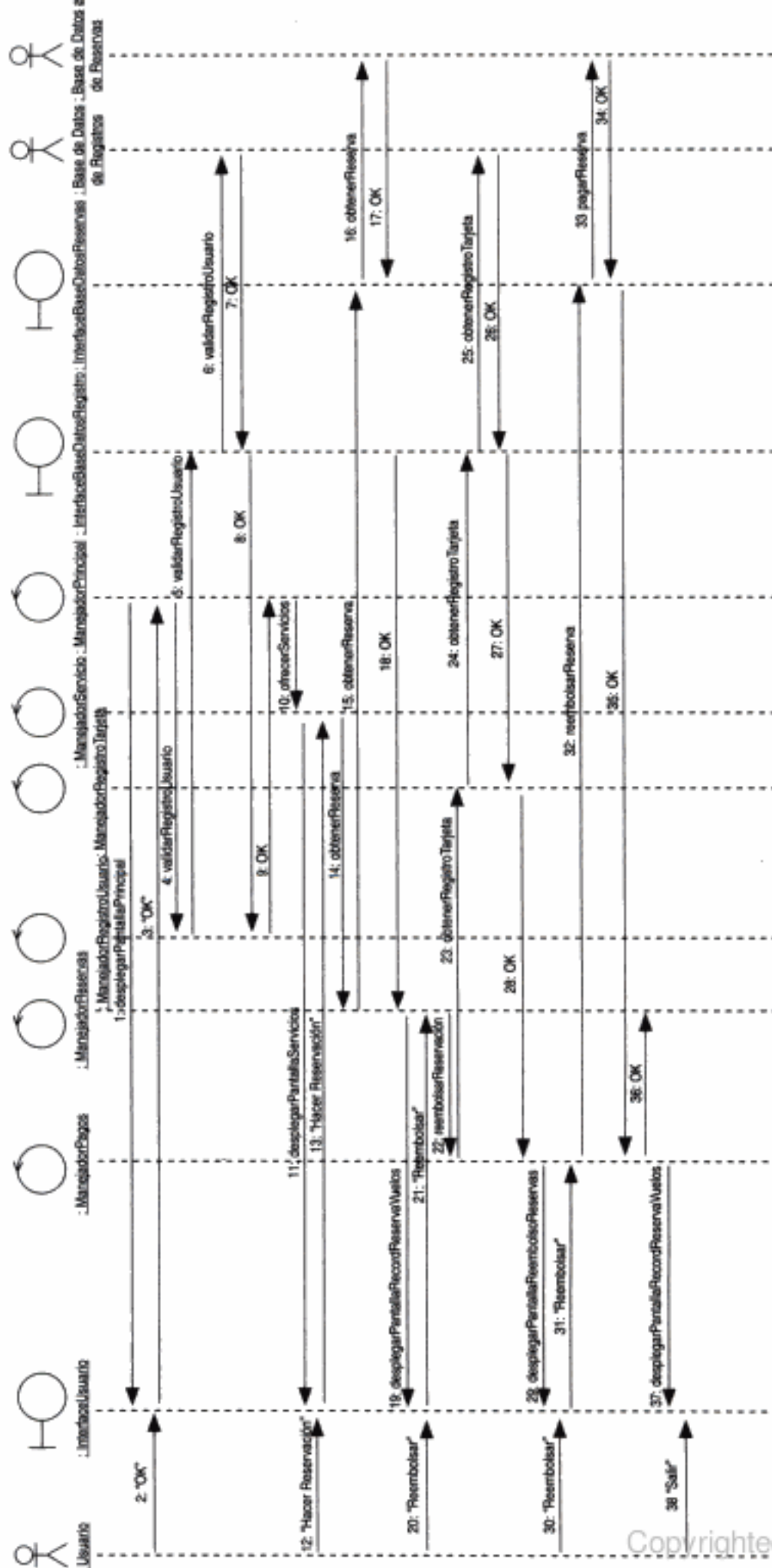


Figura 7.56 Diagrama de secuencia Reembolsar Pago del caso de uso de uso Pagar Reservación.

evento a la *Base de Datos de Registros*, la cual contesta con un OK si la validación es buena. El OK se manda a la *InterfaceBaseDatosRegistro*, de allí al *ManejadorRegistroUsuario* y luego al *ManejadorPrincipal* como respuesta a las secuencias de validación. Una vez que el *ManejadorPrincipal* recibió este último OK, solicita al *ManejadorServicio* que entre en acción mediante el evento *ofrecerServicios*.

- *Ofrecer Servicios*. A continuación podemos pasar al caso de uso *Ofrecer Servicios*, donde el *ManejadorServicio* solicita a la *InterfaceUsuario* el despliegado de la pantalla correspondiente por medio de *desplegarPantallaServicio*. La *InterfaceUsuario* despliega esta pantalla. Para continuar con la lógica principal de este subflujo, el usuario debe presionar "*Hacer Reservación*" e incluir un número correspondiente a la clave de reservación. Se envía el mismo evento de la *InterfaceUsuario* al *ManejadorServicio*, que envía el evento *reservar* al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Solicitar Clave Reservación (S-1)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaClaveReservas* a *InterfaceUsuario*. El usuario presiona el botón "*Obtener*", junto con la inserción de la clave de récord de reservas. La *InterfaceUsuario* envía el mismo evento de regreso al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Obtener Reservación (S-3)*. A continuación el *ManejadorReservas* envía el evento *obtenerReserva* a la *InterfaceBaseDatosReservas*, la cual a su vez lo reenvía al actor *Base de Datos de Reservas*. Éste genera un OK si todo es correcto y se lo envía de regreso a *InterfaceBaseDatosReservas*, la cual luego lo envía al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a *InterfaceUsuario*. Hasta aquí la secuencia es similar a la secuencia *Pagar Reservación*. En este momento el usuario presiona "*Reembolsar*". La *InterfaceUsuario* envía el mismo evento de regreso al *ManejadorReservas*, el cual envía el evento *reembolsarReservación* al *ManejadorPagos*.
- *Pagar Reservación* flujo principal. A continuación el *ManejadorPagos* envía el evento *obtenerRegistroTarjeta* al *ManejadorRegistroTarjeta*.
- *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*. A continuación el *ManejadorRegistroTarjeta* envía el evento *obtenerRegTarjeta* a la *InterfaceBaseDatosRegistro*, la cual envía el mismo evento al actor *Base de Datos de Registros*. Este actor regresa un OK a la *InterfaceBaseDatosRegistro*, la cual a su vez lo envía de regreso al *ManejadorRegistroTarjeta*.
- *Pagar Reservación* flujo principal (continuación anterior). A continuación el *ManejadorRegistroTarjeta* envía el evento OK al *ManejadorPagos*.
- *Pagar Reservación* subflujo *Reembolsar Pagos (S-2)*. A continuación el *ManejadorPagos* envía el evento *desplegarPantallaReembolsarRegTarjetas* a la *InterfaceUsuario*. El usuario genera el evento "*Reembolsar*". La *InterfaceUsuario* recibe la petición y envía el mismo evento al *ManejadorPagos*. Éste a su vez envía el evento *reembolsarReserva* a la *InterfaceBaseDatosReservas* para que haga la petición correspondiente al actor *Base de Datos de Reservas*, el cual contesta con un OK, que luego se envía por la *InterfaceBaseDatosRegistro* al *ManejadorPagos*, el cual lo reenvía al *ManejadorReservas*.
- *Hacer Reservación* subflujo *Administrar Reservación (S-4)*. A continuación el *ManejadorReservas* envía el evento *desplegarPantallaRecordReservaVuelos* a la *InterfaceUsuario* para desplegar los resultados del reembolso de reserva. En ese momento el *Usuario* presiona "*Salir*", dando por concluida la secuencia.

En resumen, la secuencia podrá iniciar con el caso de uso *Validar Usuario* seguido por el caso de uso *Ofrecer Servicios*, para luego continuar en el caso de uso *Hacer Reservación* con los subflujos *Solicitar Clave Reservación (S-1)*, *Obtener Reservación (S-3)* y *Administrar Reservación (S-4)*, y finalizar en el caso de uso *Pagar Reservación* con el flujo principal y el subflujo *Reembolsar Pago (S-2)*. Se incluye también el caso de uso *Registrar Tarjeta* subflujo *Obtener Registro Tarjeta (S-2)*.

7.5 Casos de uso para el sistema de reservación de vuelos

A partir de las secuencias analizadas anteriormente, se puede generar una descripción completa de los casos de uso del sistema de reservaciones de vuelo. Estas secuencias se insertan en los flujos o subflujos de los casos de uso respectivos. Dado que las secuencias no mencionan todos los posibles eventos, sino los principales, es necesario completarlos, en particular a clases borde y entidad que no aparecen en los diagramas de secuencias. Se debe revisar que no existan discontinuidades entre las secuencias, manteniendo siempre consistencia entre los casos de uso y las secuencias anteriores. Cualquier cambio en la lógica de las secuencias o casos de uso deberá reflejarse en todos los documentos correspondientes.

En las descripciones de los casos de uso, las clases se denotarán subrayadas, resaltando los nombres de los eventos entre clase en *cursivas*. Es muy importante que las frases sean claras y concisas, ya que esto facilitará posteriormente el modelo de diseño.

7.5.1 Validar Usuario

El flujo principal del caso de uso *Validar Usuario* se muestra a continuación.

Caso de uso	<i>Validar Usuario.</i>
Actores	<i>Usuario, Base de Datos de Registros.</i>
Tipo	<i>Inclusión.</i>
Propósito	Validar un usuario ya registrado para el uso del sistema de reservaciones de vuelo.
Resumen	Este caso de uso es iniciado por el <i>Usuario</i> . Valida al usuario mediante un login y password a ser validado con su respectivo registro de usuario para así poder utilizar el sistema de reservaciones.
Precondiciones	Si el <u>Usuario</u> aún no se ha registrado, requerirá ejecutar el caso de uso <i>Registrar Usuario</i> subflujo <i>Crear Registro Usuario</i> .
Flujo Principal	El <u>ManejadorPrincipal</u> solicita <i>desplegarPantallaPrincipal</i> a la <u>InterfaceUsuario</u> . La <u>InterfaceUsuario</u> <i>despliega</i> la <u>PantallaPrincipal</u> . La <u>PantallaPrincipal</u> <i>se despliega</i> . El Usuario puede seleccionar entre las siguientes opciones: "Registrarse por Primera Vez", "OK" y "Salir".

(continúa)

Flujo Principal (continuación)	Si la actividad seleccionada es "Registrarse Por Primera Vez", la <u>PantallaPrincipal</u> envía el evento "Registrarse Por Primera Vez" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Registrarse Por Primera Vez" al <u>ManejadorPrincipal</u>. El <u>ManejadorPrincipal</u> solicita <u>crearRegistroUsuario</u> al <u>ManejadorRegistroUsuario</u>. Se ejecuta el caso de uso <i>Registrar Usuario</i>, subflujo <i>Crear Registro Usuario (S-1)</i>. Si la actividad seleccionada es "OK", se valida el registro de usuario mediante un <i>login</i> y un <i>password</i> insertados por el <i>Usuario</i> en la <u>PantallaPrincipal</u>. La <u>PantallaPrincipal</u> envía el evento "OK" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "OK" al <u>ManejadorPrincipal</u>. El <u>ManejadorPrincipal</u> solicita <u>validarRegistroUsuario</u> al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>validarRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>validarRegistroUsuario</u> a la <u>Base de Datos de Registros</u>. La <u>Base de Datos de Registro</u> valida al usuario y devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> devuelve el OK al <u>ManejadorPrincipal</u>. Una vez validado el usuario (E-1), el <u>ManejadorPrincipal</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>. Se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaPrincipal</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorPrincipal</u>. El <u>ManejadorPrincipal</u> sale del sistema.
Subflujos	Ninguno.
Excepciones	E-1 no hubo validación: El <i>login/password</i> no se validó correctamente. Se le pide al usuario que vuelva a intentar hasta tres veces, después de lo cual se saldrá del sistema.

7.5.2 Ofrecer Servicios

El flujo principal del caso de uso *Ofrecer Servicios* se muestra a continuación.

Caso de uso	<i>Ofrecer Servicios.</i>
Actores	<i>Usuario.</i>
Tipo	<i>Inclusión.</i>
Propósito	Ofrecer los diversos servicios a un usuario ya registrado para el uso del sistema de reservaciones de vuelo.
Resumen	Este caso de uso es iniciado por el <i>Usuario</i> . Tiene opciones para utilizar los diversos servicios del sistema de reservaciones.
Precondiciones	Se requiere la validación correcta del usuario.
Flujo principal	El <u>ManejadorServicio</u> solicita <u>desplegarPantallaServicio</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaServicio</u>. La <u>PantallaServicio</u> se despliega. El <i>Usuario</i> puede seleccionar entre las siguientes actividades: "Consultar Información", "Hacer Reservación", "Obtener Registro" y "Salir". Si la actividad seleccionada es "Hacer Reservación", la <u>PantallaServicio</u> envía el evento "Hacer Reservación" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Hacer Reservación" al <u>ManejadorServicio</u>. El <u>ManejadorServicio</u> solicita <u>reservar</u> al <u>ManejadorReservas</u>. Se continúa con el caso de uso <i>Hacer Reservación</i>, subflujo <i>Solicitar Clave Reservación (S-1)</i>.

(continúa)

Flujo principal (continuación)	Si la actividad seleccionada es "Obtener Registro", la <u>PantallaServicio</u> envía el evento "Obtener Registro" a la <u>InterfaceUsuario</u> . La <u>InterfaceUsuario</u> envía el evento "Obtener Registro" al <u>ManejadorServicio</u> . El <u>ManejadorServicio</u> solicita registrar al <u>ManejadorRegistroUsuario</u> . Se continúa con el caso de uso <i>Registrar Usuario</i> , subflujo <i>Obtener Registro Usuario (S-2)</i> . Si la actividad seleccionada es "Salir", la <u>PantallaServicio</u> envía el evento "Salir" a la <u>InterfaceUsuario</u> . La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorServicio</u> . El <u>ManejadorServicio</u> sale del sistema.
Subflujos	Ninguno.
Excepciones	Ninguna.

7.5.3 Registrar Usuario

El flujo principal del caso de uso *Registrar Usuario* se muestra a continuación.

Caso de uso	<i>Registrar Usuario.</i>
Actores	<i>Usuario, Base de Datos de Registros.</i>
Tipo	<i>Básico.</i>
Propósito	Permitir a un usuario registrarse con el sistema de reservaciones de vuelo para su uso posterior.
Resumen	Este caso de uso es iniciado por el <i>Usuario</i> . Ofrece funcionalidad para crear, modificar y eliminar el registro de usuario con el sistema de reservaciones.
Precondiciones	Todos los subflujos, con excepción de <i>Registrarse Por Primera Vez</i> , requieren ejecutar inicialmente el caso de uso <i>Validar Usuario</i> .
Flujo principal	Se ejecuta el caso de uso <i>Validar usuario</i> . Dependiendo de las opciones seleccionadas por el <i>usuario</i> , se continuará con los diversos subflujos de este caso de uso.

El subflujo *Crear Registro Usuario (S-1)* se muestra a continuación.

Subflujos	<i>S-1 Crear Registro Usuario.</i> El <u>ManejadorRegistroUsuario</u> solicita <i>desplegarPantallaCrearRegistroUsuario</i> a la <u>InterfaceUsuario</u> . La <u>InterfaceUsuario</u> despliega la <u>PantallaCrearRegUsuario</u> . La <u>PantallaCrearRegUsuario</u> se despliega. Esta pantalla contiene información de registro que debe ser llenada por el <i>Usuario</i> , lo cual incluye nombre, apellido, calle, colonia, ciudad, país, código postal, teléfonos de la casa y oficina, número de fax, login, email, password y una entrada adicional de repetir password para asegurarse de su corrección. El login y el password serán utilizados por el sistema para validar al usuario. El <i>Usuario</i> puede seleccionar entre las siguientes actividades: "Registrar" y "Salir".
------------------	---

(continuá)

Subflujos (continuación)	Si el <u>Usuario</u> selecciona "Registrar", la <u>PantallaCrearRegUsuario</u> envía el evento "Registrar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Registrar" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>crearRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>crearRegistroUsuario</u> a la <u>Base de Datos Registro</u> (E-1, E-2, E-3, E-4). La <u>Base de Datos de Registros</u> devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <u>Administrar Registro Usuario</u> (S-3). Si la actividad seleccionada es "Salir", la <u>PantallaCrearRegistroUsuario</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> sale del sistema. (Si aún no se ha presionado "Registrar", la información se perderá.)
------------------------------------	--

El subflujo *Obtener Registro Usuario* (S-2) se muestra a continuación.

Subflujos (continuación)	<i>S-2 Obtener Registro Usuario.</i> El <u>ManejadorRegistroUsuario</u> solicita <u>obtenerRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>obtenerRegistroUsuario</u> a la <u>Base de Datos de Registros</u>. La <u>Base de Datos de Registros</u> devuelve el OK y el <u>RegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK y el <u>RegistroUsuario</u> al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <u>Administrar Registro Usuario</u> (S-3).
------------------------------------	---

El subflujo *Administrar Registro Usuario* (S-3) se muestra a continuación.

Subflujos (continuación)	<i>S-3 Administrar Registro Usuario.</i> El <u>ManejadorRegistroUsuario</u> solicita <u>desplegarPantallaObtenerRegUsuario</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaObtenerRegUsuario</u>. La <u>PantallaObtenerRegUsuario</u> se despliega. El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Eliminar", "Actualizar", "Registrar Tarjeta", "Servicios" y "Salir". Si el usuario presiona "Actualizar" se ejecuta el subflujo <u>Actualizar Registro Usuario</u> (S-4). Si el usuario selecciona "Eliminar" se ejecuta el subflujo <u>Eliminar Registro Usuario</u> (S-5). Si el usuario presiona "Registrar Tarjeta", la <u>PantallaObtenerRegUsuario</u> envía el evento "Registrar Tarjeta" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Registrar Tarjeta" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>registrarTarjeta</u> al <u>ManejadorRegistroTarjeta</u>. se continúa con el caso de uso <u>Registrar Tarjeta</u>. Si la actividad seleccionada es "Servicios", la <u>PantallaObtenerRegUsuario</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>. se continúa con el caso de uso <u>Ofrecer Servicios</u>. Si la actividad seleccionada es "Salir", la <u>PantallaObtenerRegUsuario</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> sale del sistema. (Si aún no se ha presionado "Actualizar", la nueva información se perderá.)
------------------------------------	---

El subflujo *Actualizar Registro Usuario (S-4)* se muestra a continuación.

Subflujos (continuación)	<i>S-4 Actualizar Registro Usuario.</i> La <u>PantallaObtenerRegUsuario</u> envía el evento "Actualizar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Actualizar" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>actualizarRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>actualizarRegistroUsuario</u> a la <u>Base de Datos Registro</u>. La <u>Base de Datos Registro</u> actualiza el <u>RegistroUsuario</u> (E-1, E-3, E-4) y devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <i>Administrar Registro Usuario (S-3)</i>.
------------------------------------	--

El subflujo *Eliminar Registro Usuario (S-5)* se muestra a continuación.

Subflujos (continuación)	<i>S-5 Eliminar Registro Usuario</i> La <u>PantallaObtenerRegUsuario</u> envía el evento "Eliminar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Eliminar" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>eliminarRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> envía el evento <u>eliminarRegistroUsuario</u> a la <u>Base de Datos de Registros</u>. La <u>Base de Datos de Registros</u> elimina el <u>RegistroUsuario</u> y devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <i>Crear Registro Usuario (S-1)</i>. <u>ManejadorRegistroUsuario</u> solicita <u>eliminarRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> envía el evento <u>eliminarRegistroUsuario</u> a la <u>Base de Datos de Registros</u>. La <u>Base de Datos de Registros</u> elimina el <u>RegistroUsuario</u> y devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <i>Crear Registro Usuario (S-1)</i>.
------------------------------------	---

Las excepciones del caso de uso se muestran a continuación.

Excepciones	<i>E-1 información incompleta:</i> falta llenar información en el registro de usuario. Se le vuelve a pedir al usuario que complete el registro. <i>E-2 registro ya existe:</i> si ya existe un registro bajo ese login, se le pedirá al usuario que lo cambie o que termine el caso de uso. <i>E-3 login incorrecto:</i> el login no es válido. Se vuelve a pedir al usuario que complete el registro. <i>E-4 contraseña incorrecta:</i> La contraseña escogida es muy sencilla o no se validó correctamente. Se vuelve a pedir al usuario que complete el registro.
--------------------	---

7.5.4 Registrar Tarjeta

El flujo principal del caso de uso *Registrar Tarjeta* se muestra a continuación.

Caso de uso	<i>Registrar Tarjeta.</i>
Actores	<i>Usuario, Base de Datos de Registros.</i>
Tipo	<i>Extensión.</i>
Propósito	Permitir a un usuario registrar una tarjeta de créditos con el sistema de reservaciones de vuelo para pagar boletos.
Resumen	Este caso de uso es iniciado por el <i>Usuario</i> . Ofrece funcionalidad para crear, modificar y eliminar el registro de tarjeta usuario para pagar las reservaciones directamente con el sistema de reservaciones.
Precondiciones	El usuario ya debe haberse registrado mediante la activación del caso de uso <i>Registrar Usuario</i> .
Flujo principal	Se continúa con el subflujo <i>Obtener Registro Tarjeta (S-2)</i> . Si no existe un <u>RegistroTarjeta</u> válido se continúa con el subflujo <i>Crear Registro Tarjeta (S-1)</i> . De lo contrario, si ya existe un <u>RegistroTarjeta</u> válido, se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i> .

El subflujo *Crear Registro Tarjeta (S-1)* se muestra a continuación.

Subflujos	<i>S-1 Crear Registro Tarjeta.</i> El <u>ManejadorRegistroTarjeta</u> solicita <i>desplegarPantallaCrearRegTarjeta</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaCrearRegTarjeta</u>. La <u>PantallaCrearRegTarjeta</u> se despliega. Esta pantalla contiene información que debe ser llenada por el <u>Usuario</u>, lo cual incluye el nombre como aparece en la tarjeta, número de tarjeta, el tipo de tarjeta, y la fecha de vencimiento. El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Registrar", "Servicios" y "Salir". Si el <u>Usuario</u> selecciona "Registrar", la <u>PantallaCrearRegTarjeta</u> envía el evento "Registrar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Registrar" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> solicita <i>crearRegistroTarjeta</i> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <i>crearRegistroTarjeta</i> a la <u>Base de Datos de Registros (E-1)</u>. La <u>Base de Datos de Registros</u> devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroTarjeta</u>. Se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaCrearRegTarjeta</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaCrearRegTarjeta</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> sale del sistema. (Si aún no se ha presionado "Registrar", la información se perderá.)
------------------	---

El subflujo *Obtener Registro Tarjeta (S-2)* se muestra a continuación.

Subflujos (continuación)	S-2 Obtener Registro Tarjeta. El <u>ManejadorRegistroTarjeta</u> solicita <u>obtenerRegistroTarjeta</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>obtenerRegistroTarjeta</u> a la <u>Base de Datos de Registros</u>. La <u>Base de Datos de Registros</u> devuelve el OK y el <u>RegistroTarjeta</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK y el <u>RegistroTarjeta</u> al <u>ManejadorRegistroTarjeta</u>. Se regresa al flujo anterior.
------------------------------------	---

El subflujo *Administrar Registro Tarjeta (S-3)* se muestra a continuación.

Subflujos (continuación)	S-3 Administrar Registro Tarjeta. El <u>ManejadorRegistroTarjeta</u> solicita <u>desplegarPantallaObtener-RegTarjeta</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaObtenerRegTarjeta</u>. La <u>PantallaObtenerRegTarjeta</u> se despliega. El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Actualizar", "Eliminar", "Servicios" y "Salir". Si el usuario presiona "Actualizar" se ejecuta el subflujo <i>Actualizar Registro Tarjeta (S-4)</i>. Si el usuario presiona "Eliminar" se ejecuta el subflujo <i>Eliminar Registro Tarjeta (S-5)</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaObtener-RegTarjeta</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaObtenerRegTarjeta</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> sale del sistema. (Si aún no se ha presionado "Actualizar", la nueva información se perderá.)
------------------------------------	--

El subflujo *Actualizar Registro Tarjeta (S-4)* se muestra a continuación.

Subflujos (continuación)	S-4 Actualizar Registro Tarjeta. La <u>PantallaObtenerRegTarjeta</u> envía el evento "Actualizar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Actualizar" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> solicita <u>actualizarRegistroTarjeta</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>actualizarRegistroTarjeta</u> a la <u>Base de Datos de Registros (E-1)</u>. La <u>Base de Datos de Registros</u> actualiza el <u>RegistroTarjeta</u> y devuelve el evento OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroTarjeta</u>. Se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i>.
------------------------------------	---

El subflujo *Eliminar Registro Tarjeta (S-5)* se muestra a continuación.

Subflujos (continuación)	<i>S-5 Eliminar Registro Tarjeta.</i> La <i>PantallaObtenerRegTarjeta</i> envía el evento "Eliminar" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Eliminar" al <i>ManejadorRegistroTarjeta</i>. El <i>ManejadorRegistroTarjeta</i> solicita <i>eliminarRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>. La <i>InterfaceBaseDatosRegistros</i> solicita <i>eliminarRegistroTarjeta</i> a la <i>Base de Datos de Registro</i>. La <i>Base de Datos de Registros</i> elimina el <i>RegistroTarjeta (E-1)</i> y devuelve el OK a la <i>InterfaceBaseDatosRegistro</i>. La <i>InterfaceBaseDatosRegistro</i> devuelve el OK al <i>ManejadorRegistroTarjeta</i>. Se continúa con el subflujo <i>Crear Registro Tarjeta (S-1)</i>.
------------------------------------	---

Las excepciones del caso de uso se muestran a continuación.

Excepciones	<i>E-1 información incompleta:</i> falta llenar información indispensable para completar el registro de tarjeta. Se vuelve a pedir al usuario que complete el registro de tarjeta.
--------------------	---

7.5.5 Consultar Información

El flujo principal del caso de uso *Consultar Información* se muestra a continuación.

Caso de uso	<i>Consultar Información.</i>
Actores	<i>Usuario, Base de Datos de Reservas.</i>
Tipo	<i>Básico.</i>
Propósito	Permitir a un usuario consultar información con el sistema de reservaciones de vuelo.
Resumen	Este caso de uso es iniciado por el <i>Usuario</i> . Ofrece funcionalidad para consultar información de horarios, tarifas y estado de vuelos con el sistema de reservaciones.
Precondiciones	Se requiere haber ejecutado anteriormente el caso de uso <i>Validar Usuario</i> .
Flujo principal	Se ejecuta el caso de uso <i>Validar Usuario</i> . Dependiendo de las opciones seleccionadas por el usuario, se continuará con los diversos subflujos de este caso de uso.

El subflujo *Consultar (S-1)* se muestra a continuación.

Subflujos	<i>S-1 Consultar.</i> El <i>ManejadorConsultas</i> solicita <i>desplegarPantallaConsultas</i> a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> despliega la <i>PantallaConsultas</i>. La <i>PantallaConsultas</i> se despliega. El <i>Usuario</i> puede seleccionar de las siguientes actividades: "Horarios", "Tarifas", "Estado", "Servicios" y "Salir".
------------------	--

(continúa)

Subflujos <i>(continuación)</i>	Si la actividad seleccionada es "Horarios", la <u>PantallaConsultas</u> envía el evento "Horarios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Horarios" al <u>ManejadorConsultas</u>. El <u>ManejadorConsultas</u> solicita <i>consultarHorarios</i> al <u>ManejadorConsultasHorarios</u>. Se continúa con el subflujo <i>Consultar Horarios (S-2)</i>. Si el usuario presiona "Tarifas", la <u>PantallaConsultas</u> envía el evento "Tarifas" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Tarifas" al <u>ManejadorConsultas</u>. El <u>ManejadorConsultas</u> solicita <i>consultarTarifas</i> al <u>ManejadorConsultasTarifas</u>. Se continúa con el subflujo <i>Consultar Tarifas (S-4)</i>. Si el usuario presiona "Estado", la <u>PantallaConsultas</u> envía el evento "Estado" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Estado" al <u>ManejadorConsultas</u>. El <u>ManejadorConsultas</u> solicita <i>consultarEstado</i> al <u>ManejadorConsultasEstado</u>. Se continúa con el subflujo <i>Consultar Estado (S-6)</i>. Si el usuario presiona "Servicios", la <u>PantallaConsultas</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultas</u>. El <u>ManejadorConsultas</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaConsultas</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultas</u>. El <u>ManejadorConsultas</u> sale del sistema.
---	---

El subflujo *Consultar Horarios (S-2)* se muestra a continuación.

Subflujos <i>(continuación)</i>	<i>S-2 Consultar Horarios.</i> El <u>ManejadorConsultaHorarios</u> solicita <i>desplegarPantallaConsultaHorarios</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaConsultaHorarios</u>. La <u>PantallaConsultaHorarios</u> se despliega. Esta pantalla debe ser llenada con información de ciudad de origen y destino, y preferencias opcionales de: aerolínea, horario y una opción de vuelo sólo directo. El Usuario puede seleccionar de las siguientes actividades: "Consultar", "Servicios" y "Salir". Si el usuario presiona "Consultar", la <u>PantallaConsultaHorarios</u> envía el evento "Consultar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Consultar" al <u>ManejadorConsultaHorarios</u>. El <u>ManejadorConsultaHorarios</u> solicita <i>consultarHorarios</i> a la <u>InterfaceBaseDatosReservas</u> (<i>E-1, E-2</i>). La <u>InterfaceBaseDatosReservas</u> solicita <i>consultarHorarios</i> a la <u>Base de Datos de Reservas</u>. La <u>Base de Datos de Reservas</u> devuelve los <u>Vuelos</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve los <u>Vuelos</u> al <u>ManejadorConsultaHorarios</u>. Se continúa con el subflujo <i>Devolver Horarios (S-3)</i>. Si el usuario presiona "Servicios", la <u>PantallaConsultaHorarios</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultaHorarios</u>. El <u>ManejadorConsultaHorarios</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaConsultaHorarios</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultaHorarios</u>. El <u>ManejadorConsultaHorarios</u> sale del sistema.
---	--

El subflujo *Devolver Horarios (S-3)* se muestra a continuación.

Subflujos (continuación)	S-3 Devolver Horarios. El <u>ManejadorConsultaHorarios</u> solicita <i>desplegarPantallaResultadoHorarios</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaResultadoHorarios</u>. La <u>PantallaResultadoHorarios</u> despliega información de <u>Vuelo</u>, <u>Aerolínea</u>, <u>Horarios</u> de Salida y Llegada y <u>Aeropuertos</u> de Origen, Destino y Escalas intermedias. El usuario puede seleccionar entre las siguientes opciones: "+", "-", "Nueva Consulta", "Servicios" y "Salir". Si el usuario presiona "+", la <u>PantallaResultadoHorarios</u> envía el evento "+" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "+" al <u>ManejadorConsultaHorarios</u>. Se continúa al inicio de este subflujo (presentando resultados adicionales). Si el usuario presiona "-", la <u>PantallaResultadoHorarios</u> envía el evento "-" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "-" al <u>ManejadorConsultaHorarios</u>. Se continúa al inicio de este subflujo (presentando resultados anteriores). Si el usuario presiona "Nueva Consulta", la <u>PantallaResultadoHorarios</u> envía el evento "Nueva Consulta" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Nueva Consulta" al <u>ManejadorConsultaHorarios</u>. Se continúa con el subflujo <i>Consultar Horarios (S-2)</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaResultadoHorarios</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultaHorarios</u>. El <u>ManejadorConsultaHorarios</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaResultadoHorarios</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultaHorarios</u>. El <u>ManejadorConsultaHorarios</u> sale del sistema.
------------------------------------	---

El subflujo *Consultar Tarifas (S-4)* se muestra a continuación.

Subflujos (continuación)	S-4 Consultar Tarifas. El <u>ManejadorConsultaTarifas</u> solicita <i>desplegarPantallaConsultaTarifas</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaConsultaTarifas</u>. La <u>PantallaConsultaTarifas</u> se despliega. Esta pantalla debe ser llenada con información de ciudad de origen y destino, y preferencias opcionales: fecha de salida, fecha de regreso, aerolínea, clase, y las opciones de organizar la información por menor tarifa, una opción de vuelo si sólo directo, y si la tarifa se basa en viaje redondo. El <u>Usuario</u> puede seleccionar de las siguientes actividades: "Consultar", "Servicios" y "Salir". Si el usuario presiona "Consultar", la <u>PantallaConsultaTarifas</u> envía el evento "Consultar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Consultar" al <u>ManejadorConsultaTarifas</u>. El <u>ManejadorConsultaTarifas</u> solicita <i>consultarTarifas</i> a la <u>InterfaceBaseDatosReservas (E-1,E-2)</u>. La <u>InterfaceBaseDatosReservas</u> solicita <i>consultarTarifas</i> a la <u>Base de Datos de Reservas</u>. La <u>Base de Datos de Reservas</u> devuelve los <u>Vuelos</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve los <u>Vuelos</u> al <u>Manejador-ConsultaTarifas</u>. Se continúa con el subflujo <i>Devolver Tarifas (S-5)</i>.
------------------------------------	---

(continúa)

Subflujos (continuación)	Si el usuario presiona "Servicios", la <u>PantallaConsultaTarifas</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultaTarifas</u>. El <u>ManejadorConsultaTarifas</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaConsultaTarifas</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultaTarifas</u>. El <u>ManejadorConsultaTarifas</u> sale del sistema.
------------------------------------	---

El subflujo *Devolver Tarifas (S-5)* se muestra a continuación.

Subflujos (continuación)	<i>S-5 Devolver Tarifas.</i> El <u>ManejadorConsultaTarifas</u> solicita <u>desplegarPantallaResultadoTarifas</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaResultadoTarifas</u>. La <u>PantallaResultadoTarifas</u> despliega información de <u>Vuelo</u>, <u>Aerolínea</u>, <u>Horarios</u> de <u>Salida</u> y <u>Llegada</u>, <u>Aeropuertos</u> de <u>Origen</u>, <u>Destino</u> y <u>Escalas</u> con posibles conexiones y <u>Tarifas</u> de ida y vuelta. El usuario puede seleccionar entre las siguientes opciones: "+", "-", "Nueva Consulta", "Servicios" y "Salir". Si el usuario presiona "+", la <u>PantallaResultadoTarifas</u> envía el evento "+" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "+" al <u>ManejadorConsultaTarifas</u>. Se continúa al inicio de este subflujo (presentando resultados adicionales). Si el usuario presiona "-", la <u>PantallaResultadoTarifas</u> envía el evento "-" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "-" al <u>ManejadorConsultaTarifas</u>. Se continúa al inicio de este subflujo (presentando resultados anteriores). Si el usuario presiona "Nueva Consulta", la <u>PantallaResultadoTarifas</u> envía el evento "Nueva Consulta" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Nueva Consulta" al <u>ManejadorConsultaTarifas</u>. Se continúa con el subflujo <i>Consultar Tarifas (S-4)</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaResultadoTarifas</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultaTarifas</u>. El <u>ManejadorConsultaTarifas</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaResultadoTarifas</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultaTarifas</u>. El <u>ManejadorConsultaTarifas</u> sale del sistema.
------------------------------------	--

El subflujo *Consultar Estado (S-6)* se muestra a continuación.

Subflujos (continuación)	<i>S-6 Consultar Estado.</i> El <u>ManejadorConsultaEstado</u> solicita <u>desplegarPantallaConsultaEstado</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaConsultaEstado</u>. La <u>PantallaConsultaEstado</u> se despliega. Esta pantalla debe ser llenada con información de ciudad de origen y destino, la aerolínea, el número de vuelo y la opción de vuelo de hoy.
------------------------------------	--

(continúa)

Subflujos (continuación)	El <u>Usuario</u> puede seleccionar de las siguientes actividades: "Consultar", "Regresar" y "Salir". Si el usuario presiona "Consultar", la <u>PantallaConsultaEstado</u> envía el evento "Consultar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Consultar" al <u>ManejadorConsultaEstado</u>. El <u>ManejadorConsultaEstado</u> solicita <i>consultarEstado</i> a la <u>InterfaceBaseDatosReservas</u> (E-1,E-2). La <u>InterfaceBaseDatosReservas</u> solicita <i>consultarEstado</i> a la <u>Base de Datos de Reservas</u>. La <u>Base de Datos de Reservas</u> devuelve el <u>Vuelo</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve el <u>Vuelo</u> al <u>ManejadorConsultaEstado</u>. Se continúa con el subflujo <i>Devolver Estado</i> (S-7). Si el usuario presiona "Servicios", la <u>PantallaConsultaEstado</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultaEstado</u>. El <u>ManejadorConsultaEstado</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaConsultaEstado</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultaEstado</u>. El <u>ManejadorConsultaEstado</u> <i>sale</i> del sistema.
------------------------------------	---

El subflujo *Devolver Estado* (S-7) se muestra a continuación.

Subflujos (continuación)	<i>S-7 Devolver Estado.</i> El <u>ManejadorConsultaEstado</u> solicita <i>desplegarPantallaResultadoEstado</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaResultadoEstado</u>. La <u>PantallaResultadoEstado</u> <i>despliega</i> la información de <u>Vuelo</u>, <u>Aerolínea</u>, <u>Horarios de Salida y Llegada</u>, <u>Aeropuertos</u> de Origen, Destino y Escalas intermedias, y <u>Avión</u> utilizado. El usuario puede seleccionar entre las siguientes opciones: "Nueva Consulta", "Servicios" y "Salir". Si el usuario presiona "Nueva Consulta", la <u>PantallaResultadoEstado</u> envía el evento "Nueva Consulta" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Nueva Consulta" al <u>ManejadorConsultaEstado</u>. Se continúa con el subflujo <i>Consultar Estado</i> (S-6). Si la actividad seleccionada es "Servicios", la <u>PantallaResultadoEstado</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorConsultaEstado</u>. El <u>ManejadorConsultaEstado</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <u>PantallaResultadoEstado</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorConsultaEstado</u>. El <u>ManejadorConsultaEstado</u> <i>sale</i> del sistema.
------------------------------------	--

Las excepciones del caso de uso se muestran a continuación.

Excepciones	<i>E-1 información incompleta:</i> falta llenar información indispensable, ciudad de origen o de destino. Se le vuelve a pedir al usuario la información. <i>E-2 información inválida:</i> una de las entradas de la solicitud es incorrecta.
--------------------	---

7.5.6 Hacer Reservación

El flujo principal del caso de uso *Hacer Reservación* se muestra a continuación.

Caso de uso	<i>Hacer Reservación.</i>
Actores	<i>Usuario, Base de Datos de Reservas.</i>
Tipo	<i>Básico.</i>
Propósito	Permitir a un usuario hacer reservaciones con el sistema de reservaciones de vuelo.
Resumen	Este caso de uso es iniciado por el <i>Usuario</i> . Ofrece funcionalidad para crear, obtener, modificar y eliminar reservaciones de vuelos con el sistema de reservaciones.
Precondiciones	Se debe haber ejecutado anteriormente el caso de uso <i>Validar Usuario</i> .
Flujo principal	Se ejecuta el caso de uso <i>Validar Usuario</i> . Dependiendo de las opciones seleccionadas por el <i>Usuario</i> , se continuará con los diversos subflujos de este caso de uso.

El subflujo *Solicitar Clave Reservación (S-1)* se muestra a continuación.

Subflujos (continuación)	<i>S-1 Solicitar Clave Reservación.</i> El <i>ManejadorReservas</i> solicita <i>desplegarPantallaClaveReservas</i> a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> despliega la <i>PantallaClaveReservas</i>. La <i>PantallaClaveReservas</i> se despliega. El <i>Usuario</i> puede seleccionar entre las siguientes opciones: "Crear", "Obtener", "Servicios" y "Salir". Si el <i>Usuario</i> selecciona "Crear", la <i>PantallaClaveReservas</i> envía el evento "Crear" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Crear" al <i>ManejadorReservas</i>. Se continúa con el subflujo <i>Crear Reservación (S-2)</i>. Si el <i>Usuario</i> selecciona "Obtener", la <i>PantallaClaveReservas</i> envía el evento "Obtener" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Obtener" al <i>ManejadorReservas</i>. Se continúa con el subflujo <i>Obtener Reservación (S-3) (E-1)</i>. Si el usuario presiona "Servicios", la <i>PantallaClaveReservas</i> envía el evento "Servicios" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Servicios" al <i>ManejadorReservas</i>. El <i>ManejadorReservas</i> solicita <i>ofrecerServicio</i> al <i>ManejadorServicio</i>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si el usuario presiona "Salir", la <i>PantallaClaveReservas</i> envía el evento "Salir" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Salir" al <i>ManejadorReservas</i>. El <i>ManejadorReservas</i> sale del sistema.
------------------------------------	--

El subflujo *Crear Reservación (S-2)* se muestra a continuación.

Subflujos (continuación)	<i>S-2 Crear Reservación.</i> El <i>ManejadorReservas</i> solicita <i>desplegarPantallaCrearReservaVuelos</i> a la <i>Interface-Usuario</i>. La <i>InterfaceUsuario</i> despliega la <i>PantallaCrearReservaVuelos</i>. La <i>PantallaCrearReservaVuelos</i> se despliega. Esta pantalla debe ser llenada con información de apellido y nombre del pasajero, un número de viajero frecuente opcional, aerolínea, número de vuelo, ciudad de origen y destino, fecha, clase, una opción de solicitar asiento y si desea ventana o pasillo y opcionalmente comida vegetal o carne.
------------------------------------	--

(continúa)

Subflujos (continuación)	El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Agregar", "+", "-", "Reservar", "Servicios" y "Salir". Si el usuario presiona "Agregar", la <u>PantallaCrearReservaVuelos</u> envía el evento "Agregar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Agregar" al <u>ManejadorReservas</u>. Se continúa al inicio de este subflujo (solicitando reservaciones adicionales). Si el usuario presiona "+", la <u>PantallaCrearReservaVuelos</u> envía el evento "+" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "+" al <u>ManejadorReservas</u>. Se continúa al inicio de este subflujo (presentando reservaciones adicionales). Si el usuario presiona "-", la <u>PantallaCrearReservaVuelos</u> envía el evento "-" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "-" al <u>ManejadorReservas</u>. Se continúa al inicio de este subflujo (presentando solicitudes anteriores). Si el usuario selecciona "Reservar", la <u>PantallaCrearReservaVuelos</u> envía el evento "Reservar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Reservar" al <u>ManejadorReservas</u> (E-2, E-3). El <u>ManejadorReservas</u> solicita <i>crearReserva</i> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> solicita <i>crearReserva</i> a la <u>Base de Datos de Reservas</u>. La <u>Base de Datos de Reservas</u> devuelve la <u>Reservación</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve la <u>Reservación</u> al <u>ManejadorReservas</u> (E-4). Se continúa con el subflujo <i>Administrar Reservación</i> (S-4). Si la actividad seleccionada es "Servicios", la <u>PantallaCrearReservaVuelos</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaCrearReservaVuelos</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> sale del sistema.
------------------------------------	---

El subflujo *Obtener Reservación* (S-3) se muestra a continuación.

Subflujos (continuación)	S-3 Obtener Reservación. El <u>ManejadorReservas</u> solicita <i>obtenerReserva</i> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> solicita <i>obtenerReserva</i> a la <u>Base de Datos de Reservas</u> (E-1). La <u>Base de Datos de Reservas</u> devuelve la <u>Reservación</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve la <u>Reservación</u> al <u>ManejadorReserva</u>. Se continúa con el subflujo <i>Administrar Reservación</i> (S-4).
------------------------------------	--

El subflujo *Administrar Reservación* (S-4) se muestra a continuación.

Subflujos (continuación)	S-4 Administrar Reservación. El <u>ManejadorReserva</u> solicita <i>desplegarPantallaRecordReservaVuelos</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> despliega la <u>PantallaRecordReservaVuelo</u>. La <u>PantallaRecord-ReservaVuelos</u> despliega información de <u>Reservación</u>, <u>Vuelo</u>, <u>Aerolínea</u>, <u>Horarios de Salida y Llegada</u>, <u>Aeropuertos de Origen</u>, <u>Destino</u> y <u>Escalas intermedias</u> y <u>Tarifas de ida y vuelta</u>.
------------------------------------	---

(continúa)

Subflujos (continuación)	El <u>Usuario</u> puede escoger las siguientes opciones: "Actualizar", "Eliminar", "+", "-", "Nueva Reserva", "Pagar", "Reembolsar", "Servicios" y "Salir". Si el usuario presiona "Actualizar" se ejecuta el subflujo <i>Actualizar Reservación</i> (S-5). Si el usuario presiona "Eliminar" se ejecuta el subflujo <i>Eliminar Reservación</i> (S-6). Si el usuario presiona "+", la <u>PantallaRecordReservaVuelos</u> envía el evento "+" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "+" al <u>ManejadorReservas</u>. Se continúa al inicio de este subflujo (presentando resultados adicionales). Si el usuario presiona "-", la <u>PantallaRecordReservaVuelos</u> envía el evento "-" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "-" al <u>ManejadorReservas</u>. Se continúa al inicio de este subflujo (presentando resultados anteriores). Si el usuario selecciona "Nueva Reserva", la <u>PantallaRecordReservaVuelos</u> envía el evento "Nueva Reserva" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Nueva Reserva" al <u>ManejadorReservas</u>. Se continúa con el subflujo <i>Crear Reservación</i> (S-2). Si la actividad seleccionada es "Pagar", la <u>PantallaRecordReservaVuelos</u> envía el evento "Pagar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Pagar" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> solicita <i>pagarReservación</i> al <u>ManejadorPagos</u>, se continúa con el caso de uso <i>Pagar Reservación</i>. Si la actividad seleccionada es "Reembolsar", la <u>PantallaRecordReservaVuelos</u> envía el evento "Reembolsar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Reembolsar" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> solicita <i>reembolsarReservación</i> al <u>ManejadorPagos</u>, se continúa con el caso de uso <i>Pagar Reservación</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaRecordReservaVuelos</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaRecordReservaVuelos</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> sale del sistema.
------------------------------------	--

El subflujo *Actualizar Reservación* (S-5) se muestra a continuación.

Subflujos (continuación)	S-5 <i>Actualizar Reservación</i>. La <u>PantallaRecordReservaVuelos</u> envía el evento "Actualizar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Actualizar" al <u>ManejadorReservas</u>. El <u>ManejadorReservas</u> solicita <i>actualizarReserva</i> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> solicita <i>actualizarReserva</i> a la <u>Base de Datos de Reservas</u> (E-2, E-3). La <u>Base de Datos de Reservas</u> devuelve la <i>Reservación</i> a la <u>InterfaceBaseDatos-Reservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve la <i>Reservación</i> al <u>ManejadorReserva</u> (E-4). Se continúa con el subflujo <i>Administrar Reservación</i> (S-4).
------------------------------------	---

El subflujo *Eliminar Reservación (S-6)* se muestra a continuación.

Subflujos (continuación)	<i>S-6 Eliminar Reservación.</i> La <i>PantallaRecordReservaVuelos</i> envía el evento "Eliminar" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Eliminar" al <i>ManejadorReservas</i>. El <i>ManejadorReservas</i> solicita <i>eliminarReserva</i> a la <i>InterfaceBaseDatosReservas</i>. La <i>InterfaceBaseDatosReservas</i> solicita <i>eliminarReserva</i> a la <i>Base de Datos de Reservas (E-5)</i>. La <i>Base de Datos de Reservas</i> devuelve el OK a la <i>InterfaceBaseDatosReservas</i>. La <i>InterfaceBaseDatosReservas</i> devuelve el OK al <i>ManejadorReserva</i>. Se continúa con el subflujo <i>Crear Reservación (S-2)</i>.
------------------------------------	---

Las excepciones del caso de uso se muestran a continuación.

Excepciones	Excepciones <i>E-1 récord inválido:</i> no existe el récord especificado. <i>E-2 información incompleta:</i> falta llenar información indispensable, ciudad de origen o de destino. Se le vuelve a pedir al usuario la información. <i>E-3 información inválida:</i> una de las entradas de la solicitud es incorrecta. <i>E-4 reserva sin éxito:</i> no se logró obtener una reserva. <i>E-5 eliminación reserva sin éxito:</i> no se logró eliminar la reserva.
--------------------	--

7.5.7 Pagar Reservación

El flujo principal del caso de uso *Pagar Reservación* se muestra a continuación.

Caso de uso	<i>Pagar Reservación.</i>
Actores	<i>Usuario, Base de Datos de Reservas.</i>
Tipo	<i>Extensión.</i>
Propósito	Permitir a un usuario pagar reservaciones con el sistema de reservaciones de vuelo.
Resumen	Este caso de uso se inicia con el <i>Usuario</i> . Ofrece funcionalidad para pagar y reembolsar pagos de reservaciones de vuelos con el sistema de reservaciones mediante tarjetas de crédito registradas con el sistema.
Precondiciones	Se requiere haber ejecutado anteriormente el caso de uso <i>Validar Usuario</i> y tener una reservación ya hecha mediante el caso de uso <i>Hacer Reservación</i> subflujo <i>Crear Reservación (S-2)</i> .
Flujo principal	El <i>ManejadorPagos</i> solicita <i>obtenerRegistroTarjeta</i> al <i>ManejadorRegistroTarjeta</i>. Se continúa con el caso de uso <i>Registrar Tarjeta</i>, subflujo <i>Obtener Registro Tarjeta (S-2)</i>. El <i>ManejadorRegistroTarjeta</i> devuelve el OK y el <i>RegistroTarjeta</i> al <i>ManejadorPagos</i>. Dependiendo si la solicitud original del <i>ManejadorReservas</i> al <i>ManejadorPagos</i> fue <i>pagarReservación</i>, se continúa el subflujo <i>Pagar Reservación (S-1)</i>; si fue <i>reembolsarReservación</i>, se continúa con el subflujo <i>Reembolsar Pago (S-2)</i>.

El subflujo *Pagar Reservación (S-1)* se muestra a continuación.

Subflujos	S-1 Pagar Reservación. El <u>ManejadorPagos</u> solicita <u>desplegarPantallaPagarRegistroTarjeta</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> <u>despliega</u> la <u>PantallaPagarRegTarjeta</u>. La <u>PantallaPagarRegTarjeta</u> <u>despliega</u> el <u>RegistroTarjeta</u>, la cual incluye información de nombre como aparece en la tarjeta, número de tarjeta, el tipo de tarjeta, la fecha de vencimiento y la cantidad a pagar (E-1). El <u>Usuario</u> podrá seleccionar entre las siguientes actividades: "Pagar", "Servicios" y "Salir". Si la actividad seleccionada es "Pagar", la <u>PantallaRecordReservaVuelos</u> envía el evento "Pagar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Pagar" al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> solicita <u>pagarReserva</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> solicita <u>pagarReserva</u> a la <u>Base de Datos de Reservas</u>. La <u>Base de Datos de Reservas</u> devuelve la <u>Reservación</u> a la <u>InterfaceBaseDatosReservas</u> (E-2). La <u>InterfaceBaseDatosReservas</u> devuelve la <u>Reservación</u> al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> devuelve la <u>Reservación</u> al <u>ManejadorReservas</u>. Se continúa con el caso de uso <u>Hacer Reservación</u>, subflujo <u>Solicitar Clave Reservación (S-1)</u>. Si la actividad seleccionada es "Servicios", la <u>PantallaPagarRegTarjeta</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>, se continúa con el caso de uso <u>Ofrecer Servicios</u>. Si la actividad seleccionada es "Salir", la <u>PantallaPagarRegTarjeta</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> sale del sistema.
------------------	--

El subflujo *Reembolsar Pago (S-2)* se muestra a continuación.

Subflujos (continuación)	S-2 Reembolsar Pago. El <u>ManejadorPagos</u> solicita <u>desplegarPantallaReembolsarRegistroTarjeta</u> a la <u>InterfaceUsuario</u>, la cual <u>despliega</u> la <u>PantallaReembolsarRegTarjeta</u>. a <u>PantallaPagarRegTarjeta</u> <u>despliega</u> el <u>RegistroTarjeta</u> la cual incluye información, de nombre como aparece en la tarjeta, número de tarjeta, el tipo de tarjeta, la fecha de vencimiento y la cantidad a reembolsar (E-f). El <u>Usuario</u> podrá seleccionar entre las siguientes actividades: "Reembolsar", "Servicios" y "Salir". Si la actividad seleccionada es "Reembolsar", la <u>PantallaRecordReservaVuelos</u> envía el evento "Reembolsar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Pagar" al <u>ManejadorPagos</u>, que solicita <u>reembolsarReserva</u> a la <u>InterfaceBaseDatosReservas</u>. El <u>ManejadorPagos</u> solicita <u>reembolsarReserva</u> a la <u>Base de Datos de Reservas</u> (E-3, E-4). La <u>Base de Datos de Reservas</u> devuelve la <u>Reservación</u> a la <u>InterfaceBaseDatosReservas</u>. La <u>InterfaceBaseDatosReservas</u> devuelve la <u>Reservación</u> al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> devuelve la <u>Reservación</u> al <u>ManejadorReservas</u>. Se continúa con el caso de uso <u>Hacer Reservación</u>, subflujo <u>Solicitar Clave Reservación (S-1)</u>.
------------------------------------	--

(continúa)

Subflujos (continuación)	Si la actividad seleccionada es "Servicios", la <u>PantallaReembolsarRegTarjeta</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaReembolsar-RegTarjeta</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorPagos</u>. El <u>ManejadorPagos</u> sale del sistema.
------------------------------------	--

Las excepciones del caso de uso se muestran a continuación.

Excepciones	<i>E-1 récord inválido:</i> no existe el récord especificado. El usuario deberá insertar los datos de la tarjeta. <i>E-2 pago inválido:</i> el pago no tuvo éxito o la información de pago está incompleta. <i>E-3 pago inexistente:</i> la reserva no ha sido pagada. <i>E-4 devolución inválida:</i> no se pudo hacer la devolución del pago.
--------------------	---

7.6 Diccionario de clases según Módulos

Como última etapa del modelo de análisis, se actualiza el diccionario de clases organizada según módulos, originalmente descrito en el dominio del problema, para incluir todas las clases identificadas durante el modelo de análisis.

Aunque no es obligatorio, aprovechamos para separar estas clases en diferentes módulos con el fin de lograr una mejor organización y correspondencia entre clases y casos de uso. Aquellas clases que participan en varios casos de uso se pueden asignar a módulos adicionales, como veremos a continuación para el sistema de reservaciones de vuelo.

Comenzamos con cuatro módulos o paquetes principales: *InterfaceUsuario*, *Principal*, *Registro* y *Servicios*, como se muestra en la figura 7.57.

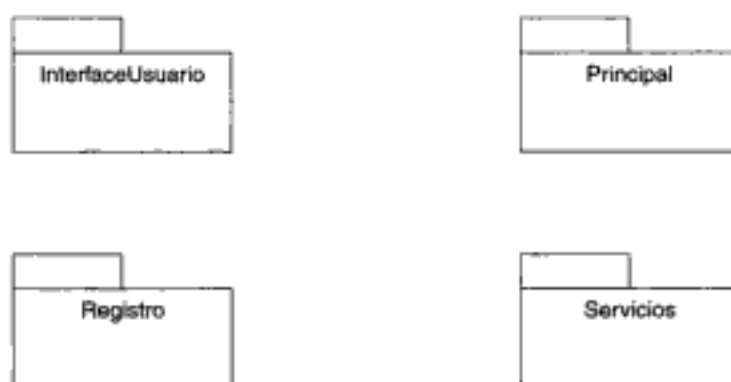


Figura 7.57 Módulos principales del sistema de reservaciones de vuelo.

7.6.1 InterfaceUsuario

El módulo *InterfaceUsuario* está compuesto por una clase utilizada para el manejo general de las interfaces de usuario:

- **InterfaceUsuario** - Clase Borde. Toda la interacción con el usuario se hace por medio del borde de usuario.

7.6.2 Principal

El módulo *Principal* está compuesto por clases comunes a la funcionalidad general del sistema:

- **PantallaPrincipal** - Clase Borde. *Pantalla* principal (P-1).
- **ManejadorPrincipal** - Clase Control. El manejador principal es el encargado de desplegar la pantalla principal de interacción con el usuario, y luego delegar las diferentes funciones a los manejadores especializados apropiados.

7.6.3 Registro

El módulo *Registro* se divide en los siguientes módulos: *Usuario*, *Tarjeta* e *InterfaceBD*, donde BD corresponde a la base de datos, como se muestra en la figura 7.58.

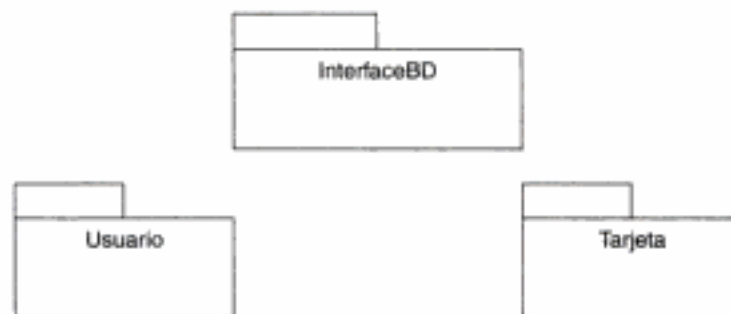


Figura 7.58 Módulos adicionales del módulo *Registro*.

USUARIO

El módulo *Usuario* está compuesto por las clases:

- **PantallaCrearRegistroUsuario** - Clase Borde. Pantalla de solicitud de registro de usuario (P-3).
- **PantallaObtenerRegUsuario** - Clase Borde. Pantalla de devolución con información de registro de usuario (P-4).
- **RegistroUsuario** - Clase Entidad. Para utilizar el sistema de reservaciones, el usuario debe estar registrado con el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa, teléfono de oficina, fax, email, login y password.

- **ManejadorRegistroUsuario** - Clase Control. El manejador de registro de usuario se encarga de todo lo relacionado con el registro del usuario para poder utilizar el sistema.

TARJETA

El módulo *Tarjeta* está compuesto por las clases:

- **PantallaCrearRegTarjeta** - Clase Borde. Pantalla de solicitud de registro de tarjeta (P-5).
- **PantallaObtenerRegTarjeta** - Clase Borde. Pantalla de devolución con información de registro de tarjeta (P-6).
- **RegistroTarjeta** - Clase Entidad. Para poder hacer un pago con una tarjeta de crédito, se debe tener un registro de tarjeta. El registro contiene información acerca de la tarjeta incluyendo nombre, número, expedidor y vencimiento. La tarjeta está ligada a un registro de usuario.
- **ManejadorRegistroTarjeta** - Clase Control. El manejador de registro de tarjeta se encarga de todo lo relacionado con registro de la tarjeta del usuario para poder pagar las reservaciones.

INTERFACEBD

El módulo *InterfaceBD*, correspondiente a la interface para la base de datos, está compuesto por la clase encargada de interactuar con la base de datos:

- **InterfaceBaseDatosRegistro** - Clase Borde. La información de cada usuario se almacena en la base de datos de registro, la cual se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información sobre la tarjeta de crédito para pagos en línea.

7.6.4 Servicios

El módulo *Servicio* se divide en los siguientes módulos: *Dominio*, *InterfaceBD*, *Consultas*, *Reservas* y *Pagos*, como se muestra en la figura 7.59.

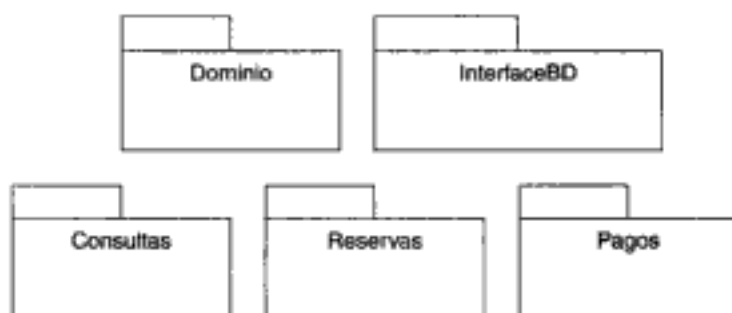


Figura 7.59 Módulos adicionales del módulo *Servicios*.

El módulo *Servicio* también incluye las siguientes clases:

- **PantallaServicio** - Clase Borde. Pantalla de servicios (P-2).

- **ManejadorServicio** - Clase Control. El manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.

DOMINIO

El módulo *Dominio* está compuesto por las clases:

- **Vuelo** - Clase Entidad. Se denomina por medio de un número. El vuelo tiene como origen un aeropuerto en una ciudad y tiene como destino un aeropuerto en otra ciudad. Un vuelo puede tener múltiples escalas y varios vuelos se relacionan por medio de conexiones. El vuelo pertenece a una aerolínea y puede operar varios días a la semana, teniendo un horario de salida y otro de llegada.
- **Reservación** - Clase Entidad. Para tomar un vuelo es necesario contar con una reservación previa, la cual debe pagarse antes de una fecha límite, que puede ser el propio día del vuelo. Una reservación puede hacerse para múltiples vuelos y múltiples pasajeros. La reservación cuenta con una clave que identifica un récord de reservación particular.
- **Horario** - Clase Entidad. El horario de un vuelo se determina por su hora de salida y su hora de llegada durante los días que opera.
- **Aerolínea** - Clase Entidad. La aerolínea provee servicio de múltiples vuelos entre diferentes ciudades bajo diferentes horarios. La aerolínea se identifica por un nombre.
- **Aeropuerto** - Clase Entidad. El aeropuerto sirve como origen, destino y escalas de un vuelo. El aeropuerto se encuentra en una ciudad de un país determinado.
- **Tarifa** - Clase Entidad. Los diferentes vuelos tienen múltiples tarifas para compra de boleto, que varían según la clase de boleto, si son de ida (viaje sencillo) o de ida y vuelta (viaje redondo), y dependiendo de las diversas restricciones y ofertas existentes.
- **Asiento** - Clase Entidad. Una reservación de vuelo puede incluir la asignación de asiento, especificada mediante una fila y un número. El número de asientos disponibles en un vuelo particular depende del tipo de avión que opere ese día.
- **Pasajero** - Clase Entidad. Para hacer una reservación se requiere dar el nombre del pasajero. Varios pasajeros pueden aparecer bajo una sola reservación.
- **Avión** - Clase Entidad. Un vuelo en una fecha determinada se hace en un tipo de avión particular. El tipo de avión define la cantidad máxima de pasajeros que pueden viajar en ese vuelo para esa fecha.
- **ViajeroFrecuente** - Clase Entidad. El pasajero tiene la opción de acumular millas para un vuelo particular si cuenta con una tarjeta de viajero frecuente para la aerolínea correspondiente.

INTERFACEBD

El módulo *InterfaceBD*, parte del módulo de servicios, incluye una clase para el acceso a la base de datos:

- **InterfaceBaseDatosReserva** - Clase Borde. La información del sistema de reservaciones de vuelo se almacena en la base de datos de reservación, la

cual se accesa mediante la interface de la base de datos de reservas. Esto permite generar consultas, reservas y pago de reservas de manera dinámica.

CONSULTAS

El módulo *Consultas* se divide en los siguientes módulos: *Horarios*, *Tarifas* y *Estado*, como se muestra en la figura 7.60.

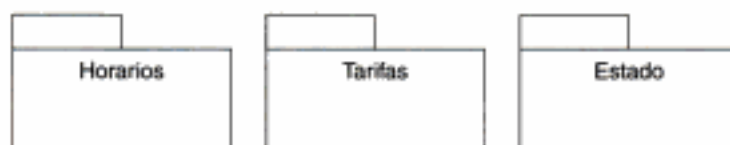


Figura 7.60 Módulos adicionales del módulo *Consultas*.

El módulo *Consultas* también incluye las siguientes clases:

- **PantallaConsultas** - Clase Borde. Pantalla de presentación de consultas (P-7).
- **ManejadorConsultas** - Clase Control. El manejador de consulta se encarga de enviar las peticiones de consulta particular a los manejadores de consulta especializados.

HORARIOS

El módulo *Horarios* está compuesto por las clases:

- **PantallaConsultaHorarios** - Clase Borde. Pantalla de presentación de consulta de horarios (P-8).
- **PantallaResultadoHorarios** - Clase Borde. Pantalla de devolución de consulta de horarios (P-9).
- **ManejadorConsultaHorarios** - Clase Control. El manejador de consulta de horarios se encarga de controlar las peticiones de consulta de horarios.

TARIFAS

El módulo *Tarifas* está compuesto por las clases:

- **PantallaConsultaTarifas** - Clase Borde. Pantalla de presentación de consulta de tarifas (P-10).
- **PantallaResultadoTarifas** - Clase Borde. Pantalla de devolución de consulta de tarifas (P-11).
- **ManejadorConsultaTarifas** - Clase Control. El manejador de consulta de tarifas se encarga de controlar las peticiones de consulta de tarifas.

ESTADO

El módulo *Estado* está compuesto por las clases:

- ▶ **PantallaConsultaEstado** - Clase Borde. Pantalla de presentación de consulta de estado (P-12).
- ▶ **PantallaResultadoEstado** - Clase Borde. Pantalla de devolución de consulta de estado (P-13).
- ▶ **ManejadorConsultaEstado** - Clase Control. El manejador de consulta de estado se encarga de controlar las peticiones de consulta de estado.

Reservas

El módulo *Reservas* está compuesto por las clases:

- ▶ **PantallaClaveReservas** - Clase Borde. Pantalla de solicitud de clave de reservas (P-14).
- ▶ **PantallaCrearReservaVuelos** - Clase Borde. Pantalla de solicitud de reservas (P-15).
- ▶ **PantallaRecordReservaVuelos** - Clase Borde. Pantalla de devolución de reservas (P-16).
- ▶ **ManejadorReservas** - Clase Control. El manejador de reserva se encarga de enviar las solicitudes de reserva a la base de datos del sistema de reservaciones.

PAGOS

El módulo *Pagos* está compuesto por las clases:

- ▶ **PantallaPagarRegTarjeta** - Clase Borde. Pantalla de solicitud de pago de reservas (P-17).
- ▶ **PantallaReembolsarRegTarjeta** - Clase Borde. Pantalla de solicitud de reembolso de pago (P-18).
- ▶ **ManejadorPagos** - Clase Control. El manejador de compra se encarga de enviar las solicitudes de compra de boleto a la base de datos del sistema de reservaciones.

La arquitectura de clases generada durante el modelo de análisis será posteriormente refinada y extendida mediante las consideraciones del ambiente de implementación durante el modelo de diseño. Esto incluye la especificación de los atributos y operaciones para todas las clases.

RESUMEN

En este capítulo se describe el modelo de análisis. Se le otorga especial relevancia a una arquitectura de tres dimensiones, correspondiente a los tres aspectos separados del modelo de requisitos. Se muestran los estereotipos básicos —borde, control y entidad— y se describe cómo identificar clases para cada caso de uso de acuerdo con estereotipos. Se describen los casos de uso en término de clases y se muestran los diagramas de secuencias correspondientes. El capítulo finaliza con la descripción del diccionario de clases del sistema de reservaciones de vuelos.

REFERENCIAS

1. Jacobson, I., Christénsen, M., Jonsson, P., Oversand, G., "Object-Oriented Software Engineering: A Use Case Driven Approach, Addison-Wesley, 1992. <http://www.national.com>.
2. <http://heim.ifi.uio.no/~trygver/themes/mvc/mvc-index.html>.

Modelo de diseño

El modelo de diseño es un refinamiento y formalización adicional del modelo de análisis, donde se toman en cuenta las consecuencias del ambiente de implementación. El resultado del modelo de diseño son especificaciones muy detalladas de todos los objetos, incluyendo sus operaciones y atributos. El modelo de diseño se basa en el diseño por responsabilidades.¹

Se requiere un modelo de diseño, ya que el modelo de análisis no es lo suficientemente formal para alcanzar el código fuente. Por tal motivo se refinan los objetos, incluyendo las operaciones y atributos. El sistema real también debe adaptarse al ambiente de implementación. En el análisis se considera un mundo ideal para el sistema, en la realidad se debe adaptar el sistema al ambiente de implementación, algo que cambia durante el ciclo de vida del sistema. Además, se deben considerar aspectos como, los requisitos de rendimiento, tiempo real, concurrencia, lenguaje de programación, manejo de base de datos, etc. Otro objetivo del diseño es validar los resultados de los modelos de requisitos y análisis. Durante el diseño, se ve si los resultados anteriores son apropiados para la implementación. Si se descubren aspectos que no están claros en alguno de los modelos anteriores, éstos se aclaran, posiblemente regresando a etapas anteriores.

Aunque esto se viera como deficiencias del resultado de las fases anteriores que se deben aclarar aquí, sería una visión incorrecta de las diferentes etapas del desarrollo, ya que el propósito de los modelos de requisitos y análisis es comprender el sistema y darle una buena estructura. También es importante comprender que las consideraciones tomadas en cuenta durante el diseño deben influir lo menos posible en la estructura del sistema. Es la propia aplicación la que controla la estructura, no las circunstancias de su implementación.

El modelo de diseño se considera como una formalización del espacio de análisis, extendiéndolo para incluir una dimensión adicional que corresponde al ambiente de implementación, como se ve en el diagrama de la figura 8.1.

Esta nueva dimensión, correspondiente al ambiente de implementación, se considera al mismo tiempo que se refina el modelo. La meta es refinarlo hasta que sea fácil escribir el código fuente. Como el modelo de análisis define la arquitectura general del sistema, se busca obtener una arquitectura detallada como resultado del modelo de diseño, de manera que haya una continuidad de refinamiento entre los dos modelos, como se ve en el diagrama de la figura 8.2.



Figura 8.1 El diseño añade el ambiente de implementación como un nuevo eje de desarrollo.

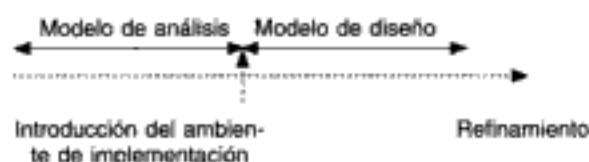


Figura 8.2 El modelo de diseño es una continuación del modelo de análisis.

En particular, se aprecia el cambio en los modelos a partir de la introducción del ambiente de implementación.

La transición de análisis a diseño debe decidirse por separado para cada aplicación particular. Aunque es posible continuar trabajando sobre el modelo de análisis, incluso durante la incorporación del ambiente de implementación, no es recomendable, ya que aumenta su complejidad. Por tanto, es conveniente tener un modelo de análisis ideal del sistema durante el ciclo de vida del sistema, dado que muchos de los cambios del sistema provienen de cambios en el ambiente de implementación. Tales cambios se incorporan fácilmente, ya que el mismo modelo de análisis sirve de base para el nuevo modelo de diseño. De esta manera, el modelo de diseño se ve como una especialización del modelo de análisis según el ambiente de implementación específico.

Si los cambios en el modelo de diseño provienen de un cambio en la lógica del sistema, entonces deben hacerse cambios en el modelo de análisis. Sin embargo, si el cambio es una consecuencia de la implementación, entonces los cambios no deben incorporarse en el modelo de análisis.

Las estructuras con las cuales se trabaja en el modelo de diseño son básicamente las mismas que en el modelo de análisis. Sin embargo, el punto de vista cambia, ya que se toma un paso hacia la implementación. El modelo de análisis debe verse como un modelo conceptual y lógico del sistema, en tanto que el modelo de diseño debe acercarse al código fuente. Esto significa que se cambia el punto de vista del modelo de diseño a una abstracción del código fuente final. Por tanto, el modelo de diseño debe ser una descripción de cómo debe estructurarse, administrarse y escribirse el código fuente.

De cierta manera, este enfoque es una extensión del concepto de la separación de la lógica de la aplicación y su implementación, donde la lógica se definió

durante el modelo de análisis, mientras que el diseño tiene la responsabilidad de mantener esta separación mediante métodos que sirven para tomar decisiones (control) y aquellos que no (borde y entidad).

En general, los cambios en la arquitectura del sistema para mejorar su rendimiento deben posponerse hasta que el sistema esté (parcialmente) construido. La experiencia muestra que en los sistemas grandes y complejos, con frecuencia se adivina incorrectamente cuáles son los cuellos de botella críticos del rendimiento. Para hacer una evaluación más adecuada es necesario evaluar parte del rendimiento del sistema construido, algo que también se puede adelantar a nivel de prototipos.

Para llevar a cabo estos objetivos se considera por separado los dos aspectos principales del modelo de diseño: el *diseño de objetos* y el *diseño de sistema*.

- **Diseño de objetos.** Se refina y formaliza el modelo para generar especificaciones muy detalladas de todos los objetos, incluyendo sus operaciones y atributos. Se describe cómo interactúan los objetos en cada caso de uso específico, especificando qué debe hacer cada operación en cada objeto. Este paso genera las interfaces de los objetos, las cuales después deben implementarse mediante métodos.
- **Diseño de sistema.** El modelo se adapta al ambiente de implementación. Este paso incluye identificar e investigar las consecuencias del ambiente de implementación sobre el diseño. Aquí deben tomarse las decisiones de implementación estratégicas: i) cómo se incorporará una base de datos en el sistema, ii) qué y cómo se usarán las bibliotecas de componentes, iii) qué lenguajes de programación se utilizarán, iv) cómo se manejarán los procesos, incluyendo comunicación y requisitos de rendimiento, v) cómo se diseñará el manejo de excepciones y recolección de basura, etc. Por ejemplo, como se mencionó en el capítulo 5, la mayor parte de los lenguajes de programación no tienen forma de implementar directamente una *asociación*. Durante el diseño se debe decidir cómo implementar mecanismos abstractos como la asociación. Similarmente, si el lenguaje de programación no ofrece ninguna técnica para apoyar la herencia, se debe especificar cómo implementarla. En resumen, se debe especificar cómo las circunstancias del ambiente de implementación deben manejarse en el sistema.

En general, si el ambiente de implementación tiene pocas consecuencias en el sistema, el diseño se basará casi exclusivamente en el diseño de objetos, o sea, en una extensión directa y detallada del modelo de análisis que describa los atributos y operaciones del sistema. Por el contrario, si el ambiente de implementación afecta de manera importante al sistema, el diseño se basará en una combinación de diseño de objetos y diseño de sistema, o sea, en un modelo de análisis que dirige el resultado final del sistema aunque éste se adaptará de manera importante al ambiente de implementación. En este caso, se minimizará el efecto del ambiente de implementación sobre el sistema, razón por la cual se comenzará con la descripción del diseño de objetos. Posteriormente, se describirán los aspectos particulares relacionados con el diseño de sistema.

A continuación se describen algunas estrategias generales de diseño antes de proseguir con los aspectos específicos del diseño.

8.1 Estrategias de diseño

Antes de resolver el diseño es necesario tomar decisiones generales sobre las estrategias de diseño a seguir. Algunas de las decisiones a tomar se presentan a continuación y se relacionan con aspectos que incluyen la *arquitectura*, *robustez*, *reuso* y *extensibilidad* del sistema.

8.1.1 Arquitectura

El término arquitectura se refiere, en este caso, a la organización de las clases dentro del sistema. Durante el modelo de análisis se generó una arquitectura de clases para el sistema y se definió la funcionalidad conceptual ofrecida por las distintas clases dentro de la arquitectura. Durante el diseño debe detallarse esta arquitectura, pudiéndose cambiar los aspectos considerados inicialmente, como fue la funcionalidad asignada a cada clase, e incluso las propias clases, como hemos mencionado al inicio del capítulo.

El conocimiento y funcionalidad asignada a cada clase se ve como la "inteligencia" de cada clase dentro del sistema. En otras palabras, algunas clases se ven más inteligentes que otras según el conocimiento y control que tengan sobre las demás clases. Por ejemplo, *colecciones* de objetos tales como *listas* o *arreglos* no se consideran inteligentes, ya que se emplean para obtener información acerca de las clases que almacenan, pero tienen relativamente poco impacto sobre éstas u otras clases dentro del sistema. Por otro lado, un manejador de interface de usuario requiere mayor inteligencia, pues debe administrar la interacción con el usuario, ya que maneja los eventos y las pantallas. Una clase aún más inteligente, es el controlador o manejador de la lógica completa de la aplicación, ya que es responsable de administrar los manejadores de borde y relacionar su funcionalidad con el resto del sistema. Como parte de la arquitectura de diseño se decide cómo distribuir la inteligencia entre las clases y qué aspectos de la inteligencia del sistema se debe asignar a cada una de ellas. Para ello existen tres alternativas principales:

- Un primer enfoque es minimizar el número de clases inteligentes. En el caso más extremo, sólo un objeto tendría conocimiento sobre todo el sistema. Los demás objetos tendrán un mínimo de inteligencia y el objeto inteligente servirá como controlador de los demás. Una ventaja de este enfoque es que sólo se requeriría comprender el flujo de control dentro del objeto principal para entender toda la aplicación. Sin embargo, se vuelve más compleja la extensibilidad del sistema, ya que cualquier cambio en el flujo de control se llevaría a cabo en un mismo objeto y afectaría potencialmente la lógica de toda la aplicación. De cierta manera, esto se considera como la "estructuración" del programa, en otras palabras, se transforma la orientación a objetos a programación estructurada, donde toda la aplicación consta de un solo "objeto".
- Un segundo enfoque es distribuir la inteligencia del sistema lo más homogéneo posible, diseñando todas las clases con inteligencia similar. Este enfoque se aproxima más con el espíritu de la orientación a objetos. Sin embargo, una distribución perfectamente homogénea es una tarea casi imposible, ya que los objetos varían en sus responsabilidades, su razón de ser depende de la aplicación. Por otro lado, la distribución de la inteligencia del siste-

ma de manera homogénea entre los objetos permite que cada objeto sepa menos cosas. Esto produce objetos más pequeños y más fáciles de comprender. La desventaja es que la inteligencia del sistema va de la mano con la especialización de las clases. Si todas las clases son "inteligentes", esto significará que éstas serán muy especializadas, dificultando la extensibilidad del sistema que requiere generalizar más las clases.

- El tercer enfoque es encontrar un balance entre los dos primeros. La idea es homogeneizar la inteligencia del sistema sólo entre ciertas clases, como las de control. El resto de las clases, entidad y borde, serán *tontas*, generalmente sobreviviendo a modificaciones en el sistema y manteniendo la lógica introducida durante el modelo de requisitos y el modelo de análisis posterior para lograr una mayor robustez del sistema.

8.1.2 Robustez

La robustez de un sistema debe ser uno de los objetivos principales del diseño. El sistema debe estar protegido contra errores y ofrecer diagnósticos que permitan identificar fallas, en particular aquellas que son fatales. Durante el desarrollo, a veces es bueno insertar instrucciones internas en el código para descubrir fallas, aunque luego se eliminen durante la producción. En general, se debe escoger lenguajes de programación que apoyen estos aspectos, como son el manejo de excepciones. Las principales consideraciones relacionadas con la robustez de un sistema son las siguientes:

- El sistema debe estar protegido contra parámetros incorrectos proporcionados por el usuario. Cualquier método que acepte parámetros del usuario debe validar la entrada para evitar problemas. El diseñador de métodos debe considerar dos tipos de condiciones de error: i) errores lógicos que se identifican durante el análisis y ii) errores de implementación, incluyendo errores del sistema operativo, como los errores de asignación de memoria, o errores de archivos de entrada y salida, etcétera.
- El sistema no debe optimizarse hasta que éste funcione de manera correcta. A menudo los programadores le dedican demasiado esfuerzo a mejorar partes del código de uso poco frecuente. Optimizar requiere medir primero el rendimiento del sistema. Se debe estudiar las alternativas, como aspectos de memoria, velocidad y simplicidad de implementación. No se debe optimizar más de lo necesario, ya que la optimización compromete la extensibilidad, reuso y comprensión del sistema.
- El sistema debe incluir estructuras de datos de tamaño variable, sin límites predefinidos. Durante el diseño es difícil predecir la capacidad máxima esperada para la estructura de datos en la aplicación. Por tanto, se deben escoger estructuras de datos como las listas, a diferencia de los arreglos.
- El sistema debe instrumentar un monitoreo de rendimiento y búsqueda de errores. El esfuerzo para llevarlo a cabo depende del ambiente de programación. Si el lenguaje de implementación no proporciona ningún apoyo, se añaden métodos de impresión para cada clase. También se añaden mensajes de entrada y salida a los métodos, imprimiendo selectivamente estos valores.
- El encapsulamiento es fundamental para la robustez del sistema. Ocultar la información interna, atributos e implementación de métodos de una clase, permite cambiarla sin afectar al resto del sistema. Únicamente la interface de los métodos afecta a las demás clases.

8.1.3 Reuso

El reuso es un aspecto fundamental del diseño. Cuanto más se pueda reutilizar el código será mejor la robustez del sistema. Las siguientes son algunas estrategias para mejorar las posibilidades de reuso del diseño:

- ▶ A través de la herencia se incrementa el reuso de código. Se toman los aspectos comunes a clases similares utilizando superclases comunes. Este enfoque es efectivo cuando las diferencias entre las clases son pequeñas y las similitudes son grandes. Es importante considerar la naturaleza de cada herencia para asegurar que no se llegue a extremos donde la aplicación de la herencia sea inadecuada.
- ▶ El uso impropio de la herencia hace que los programas sean difíciles de mantener y extender. Como alternativa, la delegación provee un mecanismo para el reuso de código, pero sin utilizar la herencia. Esto se basa en el uso de agregación a través de clases intermediarias que ocultan la funcionalidad de las clases a las cuales se delega.
- ▶ El encapsulamiento es muy efectivo para lograr el reuso, se aplica tanto al nivel de los objetos como de componentes desarrollados en otras aplicaciones. Estos componentes se reutilizan como se diseñaron, agregándolos a nuevas interfaces.

8.1.4 Extensibilidad

La mayor parte de los sistemas son extendidos de manera imprevista. Las siguientes son algunas de las perspectivas de extensibilidad:

- ▶ Se debe encapsular otra vez las clases, ocultando su estructura interna a las otras clases. Sólo los métodos de la clase deben acceder sus atributos.
- ▶ No se debe exportar estructuras de datos desde un método. Las estructuras de datos internas son específicas para el algoritmo del método. Si se exportan las estructuras se limita la flexibilidad para cambiar el algoritmo más adelante.
- ▶ Una clase particular debe tener un conocimiento limitado de la arquitectura de clases del sistema. Este conocimiento abarcará únicamente las asociaciones entre ésta y sus clases vecinas directas. Cualquier interacción con un vecino indirecto, se deberá hacer mediante llamadas a los vecinos directos.
- ▶ Se debe evitar expresiones que requieran un conocimiento explícito de los tipos de objetos. En su lugar, se debe aprovechar el *polimorfismo* a fin de seleccionar el comportamiento a ejecutarse, basado en el tipo implícito del objeto.
- ▶ Se debe distinguir entre operaciones privadas y públicas. Se vuelve costoso cambiar operaciones públicas, debiendo ser definidas con cuidado. Las operaciones privadas son internas en la clase y sirven únicamente de ayuda para implementar operaciones públicas. Las operaciones privadas pueden modificarse sin afectar a otras clases.

8.2 Diseño de objetos

El diseño de objetos es un proceso para añadir detalles al análisis y tomar decisiones junto con el diseño del sistema, o sea, al ambiente de implementación,

Hidden page

Tabla 8.1 Diagrama para la tarjeta de clase

Clase:	
Descripción:	
Módulo:	
Estereotipo:	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	

La tarjeta se divide en tres secciones:

- Encabezado, que consta del nombre de la *clase*, una *descripción* de la clase (similar a la descrita en el diccionario de clases de análisis), el *módulo* al que pertenece la clase, el *estereotipo* de la clase (entidad, borde o control), las *propiedades* de la clase (abstracta o concreta), una lista de *superclases*, una lista de *subclases* y una lista de *atributos*.
- Dos columnas debajo del encabezado, corresponden a las *responsabilidades* (a la izquierda) y *colaboraciones* (a la derecha) de la clase. En la columna izquierda, además de responsabilidades, se incluirá información sobre los *contratos*. El número de filas en estas dos columnas es extensible y no está limitado al número que aparece en el diagrama de la tabla 8.1.

Originalmente, detrás de cada tarjeta se agregaba una descripción corta del propósito de cada clase, algo que se hará a través del diccionario de clases. Además, se incluyen algunos datos adicionales al esquema original *CRC*.

Como ejemplo, se considera la clase *InterfaceUsuario*, una de las clases identificadas durante el modelo de análisis para el ejemplo del sistema de reservaciones de vuelo. La tarjeta de clase sería la que se muestra en la tabla 8.2.

Se debe elaborar una tarjeta para cada clase en el sistema, donde en principio las tarjetas incluirán únicamente entradas para el nombre de la clase, módulo al que pertenecen y estereotipo correspondiente. Dado que aún no se ha identificado herencia, no habrá entradas para propiedades, superclase o subclase. Como se aprecia, las tarjetas de clase tendrán información muy limitada, algo que durante el diseño se extenderá hasta alcanzar los métodos y atributos detallados. Una vez creadas estas tarjetas se procede a identificar las responsabilidades de cada clase como se verá a continuación.

Hidden page

Hidden page

Flujo Principal (continuación)	Si la actividad seleccionada es "OK", se valida el registro de usuario mediante un login y un password insertados por el <u>Usuario</u> en la <u>PantallaPrincipal</u>. La <u>PantallaPrincipal</u> envía el evento "OK" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "OK" al <u>ManejadorPrincipal</u>. El <u>ManejadorPrincipal</u> solicita <u>validarRegistroUsuario</u> al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>validarRegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <u>validarRegistroUsuario</u> a la <u>Base de Datos Registro</u>. La <u>Base de Datos Registro</u> valida al usuario y devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> devuelve el OK al <u>ManejadorPrincipal</u>. Una vez validado el usuario (E-1), el <u>ManejadorPrincipal</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>. Se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaPrincipal</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorPrincipal</u>. El <u>ManejadorPrincipal</u> sale del sistema.
--	---

Se toma cada una de las frases que describen el flujo y se analizan para decidir qué responsabilidad representa y a qué clase se le asigna:

1. El ManejadorPrincipal solicita *desplegarPantallaPrincipal* a la InterfaceUsuario. La primera pregunta es "cuál es la responsabilidad". La responsabilidad que se identifica aquí es "solicita *desplegarPantallaPrincipal*". La siguiente pregunta es "a quién se le asigna la responsabilidad". Existen dos opciones, asignar "solicita *desplegarPantallaPrincipal*" al ManejadorPrincipal o asignar "*desplegarPantallaPrincipal*" a la InterfaceUsuario. Para simplificar esta decisión y asegurarse, al menos en esta etapa de no perder información, se tomará una decisión "salomónica", asignando dos responsabilidades, una a cada clase. Esto es muy importante, ya que se asegura comenzar con un buen número de responsabilidades que luego se reorganizarán durante el transcurso del diseño. Es importante resaltar que, dado que muchas responsabilidades pueden darse de manera duplicada, se debe de evitar revisando las frases que generan duplicaciones. Por tanto, la responsabilidad inicial de "solicita *desplegarPantallaPrincipal*" se asignará al ManejadorPrincipal, definiendo la responsabilidad de manera completa como "solicita *desplegarPantallaPrincipal* a la InterfaceUsuario". Como se verá con la segunda frase, la responsabilidad de *desplegarPantallaPrincipal* se asignará de manera adecuada a la InterfaceUsuario, por lo que no será necesario hacerlo en este momento. Sin embargo, siempre se revisa que en la frase exista la responsabilidad complementaria para la segunda clase. El objetivo esencial es asegurarse de asignar responsabilidades a las diferentes clases, de manera que no se rompa con el flujo de colaboraciones, algo que debe tratarse con sumo cuidado para resolver cuanto antes el problema general de asignación de servicios. También se debe recordar que habrán varias etapas dentro de la actividad de diseño donde se podrá afinar la asignación de responsabilidades.
2. La InterfaceUsuario *despliega la PantallaPrincipal*. De manera análoga a la oración anterior, la responsabilidad que identificamos es "*despliega la PantallaPrincipal*" y la asignamos a InterfaceUsuario utilizando la misma lógica anterior. Observe que esta responsabilidad corresponde al servicio so-

licitado por ManejadorPrincipal en la oración anterior. Esto resalta el hecho de que las responsabilidades se están asignando de manera correcta y sin romper el flujo de colaboraciones.

3. **La PantallaPrincipal se despliega.** Esta responsabilidad se asigna a la única clase participante que es PantallaPrincipal. Note también, que esta responsabilidad, “despliega”, corresponde al servicio de despliegue de la PantallaPrincipal referido en la oración anterior.
4. **El Usuario selecciona entre las siguientes opciones: “Registrarse por Primera Vez”, “OK” y “Salir”.** En general, los actores no son parte de la arquitectura del sistema, por lo que no se les asigna ninguna responsabilidad. Además, ¡los actores son bastante *irresponsables* (en particular los usuarios)!
5. **Si la actividad seleccionada es “Registrarse por Primera Vez”, la PantallaPrincipal envía el evento “Registrarse por Primera Vez” a la InterfaceUsuario.** De manera análoga a las asignaciones anteriores, la responsabilidad “envía el evento “Registrarse por Primera Vez” a la InterfaceUsuario” se asigna a la PantallaPrincipal. Dado que en la siguiente frase la InterfaceUsuario envía a su vez el evento a otra clase, no se agrega ninguna responsabilidad adicional a esta última clase.
6. **La InterfaceUsuario envía el evento “Registrarse por Primera Vez” al ManejadorPrincipal.** De manera similar, la responsabilidad “envía el evento “Registrarse por Primera Vez” al ManejadorPrincipal” se asigna a la InterfaceUsuario. Regresando a la discusión original de la asignación de responsabilidades a ambas clases, se observa que la siguiente frase no asigna una responsabilidad de “recibir el evento” a la clase ManejadorPrincipal. Por tal motivo, se hará una segunda asignación de responsabilidad, que se llamará “maneja el evento “Registrarse por Primera Vez”” y se asignará al ManejadorPrincipal.
7. **El ManejadorPrincipal solicita crearRegistroUsuario al ManejadorRegistroUsuario.** La responsabilidad “solicita *crearRegistroUsuario* al ManejadorRegistroUsuario” se asigna al ManejadorPrincipal. Adicionalmente, se asigna la responsabilidad *crearRegistroUsuario* al ManejadorRegistroUsuario, ya que más adelante se verá que, en el caso de uso *Registrar Usuario*, subflujo *Crear Registro Usuario (S-1)*, no se continúa con una frase que defina la misma responsabilidad complementaria.
8. **Se ejecuta el caso de uso Registrar Usuario, subflujo Crear Registro Usuario (S-1).** Esta frase no genera ninguna responsabilidad, sólo una ramificación en el flujo del caso de uso.
9. **Si la actividad seleccionada es “OK”, se valida el registro de usuario mediante un login y un password insertados por el Usuario en la PantallaPrincipal.** No se incluyen oraciones que describen responsabilidades para actores, ya que no agregan responsabilidades.
10. **La PantallaPrincipal envía el evento “OK” a la InterfaceUsuario.** La responsabilidad es “envía el evento “OK” a la InterfaceUsuario” y se asigna a PantallaPrincipal.

11. **La InterfaceUsuario envía el evento "OK" al ManejadorPrincipal.** La responsabilidad es "envía el evento "OK" al ManejadorPrincipal" y se asigna a InterfaceUsuario. Adicionalmente, la responsabilidad "maneja el evento "OK" se asigna al ManejadorPrincipal.
12. **El ManejadorPrincipal solicita *validarRegistroUsuario* al ManejadorRegistroUsuario.** La responsabilidad es "solicita *validarRegistroUsuario* al ManejadorRegistroUsuario" y se asigna a ManejadorPrincipal.
13. **El ManejadorRegistroUsuario solicita *validarRegistroUsuario* a la InterfaceBaseDatosRegistro.** La responsabilidad "solicita *validarRegistroUsuario* a la InterfaceBaseDatosRegistro" se asigna a ManejadorRegistroUsuario.
14. **La InterfaceBaseDatosRegistro solicita *validarRegistroUsuario* a la Base de Datos Registro.** La responsabilidad es "solicita *validarRegistroUsuario* a la Base de Datos Registro" y se asigna a InterfaceBaseDatosRegistro.
15. **La Base de Datos Registro valida al usuario y devuelve el OK a la InterfaceBaseDatosRegistro.** Esta frase no agrega responsabilidades, ya que comprende un actor externo al sistema junto con un evento de devolución.
16. **La InterfaceBaseDatosRegistro devuelve el OK al ManejadorRegistroUsuario.** Esta frase no agrega responsabilidades, pues describe un evento de devolución.
17. **El ManejadorRegistroUsuario devuelve el OK al ManejadorPrincipal.** Nuevamente, esta frase no agrega responsabilidades, ya que describe un evento de devolución.
18. **Una vez validado el usuario (E-1), el ManejadorPrincipal solicita *ofrecerServicio* al ManejadorServicio.** La responsabilidad es "solicita *ofrecerServicio* al ManejadorServicio", la cual se asigna a ManejadorPrincipal. Adicionalmente, se asigna la responsabilidad "ofrecerServicio" a la clase ManejadorServicio para asegurarse que exista una responsabilidad complementaria en esta última clase.
19. **Se continúa con el caso de uso *Ofrecer Servicios*.** Ésta es una frase que describe continuación entre casos de uso y no agrega ninguna responsabilidad.
20. **Si la actividad seleccionada es "Salir", la PantallaPrincipal envía el evento "Salir" a la InterfaceUsuario.** La responsabilidad "envía el evento "Salir" se asigna a la InterfaceUsuario" a la PantallaPrincipal.
21. **La InterfaceUsuario envía el evento "Salir" al ManejadorPrincipal.** Se asigna la responsabilidad "envía el evento "Salir" al ManejadorPrincipal" a la InterfaceUsuario. Adicionalmente la responsabilidad "maneja el evento "Salir" se asigna al ManejadorPrincipal.
22. **El ManejadorPrincipal sale del sistema.** Se asigna la responsabilidad "sale del sistema" al ManejadorPrincipal.

A partir de estas frases, se obtienen las primeras responsabilidades y se insertan en las tarjetas de clase correspondientes para lograr una versión preliminar de las tarjetas de clase. En general, las tarjetas tienen la ventaja de forzar a la brevedad, de tal manera que las responsabilidades se listan lo más compacto posible. En las siguientes tablas se asignan las responsabilidades a las diferentes clases.

En la tabla 8.3 se muestran las responsabilidades ya identificadas para la clase *ManejadorPrincipal*. Observe, que se añade entre paréntesis el índice de la frase de la que provienen en la descripción del caso de uso.

Tabla 8.3 Tarjeta para la clase <i>ManejadorPrincipal</i> con responsabilidades ya identificadas.	
Clase: ManejadorPrincipal	
Descripción: el manejador principal es el encargado de desplegar la pantalla principal de interacción con el usuario, y luego delegar las diferentes funciones a los manejadores especializados apropiados.	
Módulo: Principal	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita <i>desplegarPantallaPrincipal</i> a la <i>InterfaceUsuario</i> (1)	
maneja el evento "Registrarse por Primera Vez" (6)	
solicita <i>crearRegistroUsuario</i> al <i>ManejadorRegistroUsuario</i> (7)	
maneja el evento "OK" (11)	
solicita <i>validarRegistroUsuario</i> al <i>ManejadorRegistroUsuario</i> (12)	
solicita <i>ofrecerServicio</i> al <i>ManejadorServicio</i> (18)	
maneja el evento "Salir" (21)	
sale del sistema (22)	

En la tabla 8.4 se muestran las responsabilidades ya identificadas para la clase *InterfaceUsuario*. Observe, que se añade entre paréntesis el índice de la frase de la que provienen en la descripción del caso de uso.

Tabla 8.4 Tarjeta para la clase <i>InterfaceUsuario</i> con responsabilidades ya identificadas.	
Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	

continúa

Tabla 8.4 Continuación

Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <u>PantallaPrincipal</u> (2)	
envía el evento "Registrarse por Primera Vez" al <u>ManejadorPrincipal</u> (6)	
envía el evento "OK" al <u>ManejadorPrincipal</u> (11)	
envía el evento "Salir" al <u>ManejadorPrincipal</u> (21)	

La tabla 8.5 muestra las responsabilidades ya identificadas para la clase *PantallaPrincipal*.

Tabla 8.5 Tarjeta para la clase *PantallaPrincipal* con responsabilidades ya identificadas.

Clase: PantallaPrincipal	
Descripción: pantalla principal (P-1).	
Módulo: Principal	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega (3)	
envía el evento "Registrarse por Primera Vez" a la <u>InterfaceUsuario</u> (5)	
envía el evento "OK" a la <u>InterfaceUsuario</u> (10)	
envía el evento "Salir" a la <u>InterfaceUsuario</u> (20)	

La tabla 8.6 muestra las responsabilidades ya identificadas para la clase *ManejadorRegistroUsuario*.

Tabla 8.6 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades ya identificadas.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de todo lo relacionado con el registro del usuario para utilizar el sistema.	

continúa

Tabla 8.6 Continuación

Módulo:	Registro.Usuario
Estereotipo:	Control
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
crearRegistroUsuario (7)	
solicita validarRegistroUsuario a la InterfaceBaseDatosRegistro (13)	

La tabla 8.7 muestra las responsabilidades ya identificadas para la clase *InterfaceBaseDatosRegistro*.

Tabla 8.7 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades ya identificadas.

Clase:	InterfaceBaseDatosRegistro
Descripción:	la información de cada usuario se almacena en la base de datos de registro, la cual se accesa mediante la interface de la base de datos de registro. Esto permite validar a los usuarios además de guardar información acerca de la tarjeta de crédito para pagos en línea.
Módulo:	Registro.InterfaceBD
Estereotipo:	Interface
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita validarRegistroUsuario a la BaseDatosRegistro (14)	

La tabla 8.8 muestra las responsabilidades identificadas hasta el momento para la clase *ManejadorServicio*.

Tabla 8.8 Tarjeta para la clase *ManejadorServicio* con responsabilidades ya identificadas.

Clase:	ManejadorServicio
Descripción:	el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.
Módulo:	Servicios
Estereotipo:	Control
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
ofrecerServicio (18)	

También es posible obtener responsabilidades a partir de las frases descritas en el manejo de excepciones, como se muestra a continuación.

Excepciones	<i>E-1 no hubo validación: el login/password no se validó correctamente. Se le pide al usuario que lo vuelva a intentar hasta tres veces, después de lo cual se saldrá del sistema.</i>
--------------------	---

Sin embargo, se hará la asignación de responsabilidades para los flujos básicos y no los alternos. En general, los flujos alternos, como los de excepción, se harán después de completar los flujos básicos.

OFRECER SERVICIOS

A continuación se muestra el flujo principal del caso de uso *OfrecerServicio* como se presentó al final del capítulo 7 correspondiente al modelo de análisis.

Flujo Principal	El <u>ManejadorServicio</u> solicita <i>desplegarPantallaServicio</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> <i>despliega</i> la <u>PantallaServicio</u>. La <u>PantallaServicio</u> se <i>despliega</i>. El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Consultar Información", "Hacer Reservación", "Obtener Registro" y "Salir". Si la actividad seleccionada es "Consultar Información", la <u>PantallaServicio</u> envía el evento "Consultar Información" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Consultar Información" al <u>ManejadorServicio</u>. El <u>ManejadorServicio</u> solicita <i>consultar</i> al <u>ManejadorConsultas</u>. Se continúa con el caso de uso <i>Consultar Información</i>, subflujo <i>Consultar</i> (S-1). Si la actividad seleccionada es "Hacer Reservación", la <u>PantallaServicio</u> envía el evento "Hacer Reservación" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Hacer Reservación" al <u>ManejadorServicio</u>. El <u>ManejadorServicio</u> solicita <i>reservar</i> al <u>ManejadorReservas</u>. Se continúa con el caso de uso <i>Hacer Reservación</i>, subflujo <i>Solicitar Clave Reservación</i> (S-1). Si la actividad seleccionada es "Obtener Registro", la <u>PantallaServicio</u> envía el evento "Obtener Registro" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Obtener Registro" al <u>ManejadorServicio</u>. El <u>ManejadorServicio</u> solicita <i>registrar</i> al <u>ManejadorRegistroUsuario</u>. Se continúa con el caso de uso <i>Registrar Usuario</i>, subflujo <i>Obtener Registro Usuario</i> (S-2). Si la actividad seleccionada es "Salir", la <u>PantallaServicio</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorServicio</u>. El <u>ManejadorServicio</u> <i>sale</i> del sistema.
------------------------	---

Nuevamente, se toma cada una de las frases que describen el flujo y se analizan para decidir qué responsabilidad representan y a qué clase se le asignan:

23. **El ManejadorServicio solicita *desplegarPantallaServicio* a la InterfaceUsuario.** La responsabilidad es "*solicita desplegarPantallaServicio* a la InterfaceUsuario" y se asigna a ManejadorServicio.
24. **La InterfaceUsuario *despliega* la PantallaServicio.** La responsabilidad es "*despliega* la PantallaServicio" y se asigna a InterfaceUsuario.
25. **La PantallaServicio se *despliega*.** La responsabilidad es "*despliega*" y se asigna a PantallaServicio.

Hidden page

jo correspondiente se observa que la responsabilidad *obtenerRegistroUsuario* se asigna más adelante.

38. **Se continúa con el caso de uso *Registrar Usuario*, en el subflujo *Obtener Registro Usuario (S-2)*.** Esta oración no involucra ninguna responsabilidad.
39. **Si la actividad seleccionada es "Salir", la *PantallaServicio* envía el evento "Salir" a la *InterfaceUsuario*.** Se asigna la responsabilidad "envía el evento "Salir" a la *InterfaceUsuario*" a la clase *PantallaServicio*.
40. **La *InterfaceUsuario* envía el evento "Salir" al *ManejadorServicio*.** Se asigna la responsabilidad "envía el evento "Salir" al *ManejadorServicio*" a la clase *InterfaceUsuario*. Adicionalmente, "maneja el evento "Salir" se asigna al *ManejadorServicio*.
41. **El *ManejadorServicio* sale del sistema.** Se asigna la responsabilidad "sale del sistema" al *ManejadorServicio*.

A partir de estas frases se obtienen nuevas responsabilidades y se insertan en las tarjetas de clase correspondientes. Dado que no todas las clases agregan nuevas responsabilidades, se mostrarán sólo aquellas que sí lo hacen. En particular, las responsabilidades identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no cambiaron y se mantienen iguales a las descritas en las tablas 8.3 y 8.5, respectivamente. De igual manera, las responsabilidades identificadas para las clases *ManejadorRegistroUsuario* e *InterfaceBaseDatosRegistro* no cambiaron y se mantienen iguales a las descritas en las tablas 8.6 y 8.7, respectivamente.

La tabla 8.9 muestra las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.4 junto con las nuevas responsabilidades.

Tabla 8.9 Tarjeta para la clase <i>InterfaceUsuario</i> con responsabilidades ya identificadas.	
Clase: <i>InterfaceUsuario</i>	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: <i>InterfaceUsuario</i>	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <i>PantallaPrincipal</i> (2)	
envía el evento "Registrarse por Primera Vez" al <i>ManejadorPrincipal</i> (6)	
envía el evento "OK" al <i>ManejadorPrincipal</i> (11)	

continúa

Hidden page

Hidden page

Nuevamente, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

42. El **ManejadorRegistroUsuario** solicita **desplegarPantallaCrearReg-Usuario a la InterfaceUsuario**. La responsabilidad es "solicita *desplegarPantallaCrearRegUsuario* a la *InterfaceUsuario*" y se asigna a *ManejadorRegistroUsuario*.
43. La **InterfaceUsuario** despliega la **PantallaCrearRegUsuario**. La responsabilidad es "*despliega la PantallaCrearRegUsuario*" y se asigna a *InterfaceUsuario*.
44. La **PantallaCrearRegUsuario** se **despliega**. La responsabilidad es "*despliega*" y se asigna a *PantallaCrearRegUsuario*.
45. Esta pantalla contiene información de registro que llena el **Usuario**, que incluye nombre, apellido, calle, colonia, ciudad, país, código postal, teléfonos de la casa y oficina, número de fax, *login*, email, *password* y una entrada adicional para repetir el *password* para asegurarse de su corrección. El *login* y *password* los utiliza el sistema para **validar al usuario**. Estas frases informan y no describen ninguna responsabilidad de interés en este momento.
46. El **Usuario** selecciona entre las siguientes actividades: "Registrar" y "Salir". Nuevamente esta es una frase que informa acerca de las opciones del Usuario y no agrega responsabilidades.
47. Si el **Usuario** selecciona "Registrar", la **PantallaCrearRegUsuario** envía el evento "Registrar" a la **InterfaceUsuario**. Se identifica la responsabilidad "envía el evento "Registrar" a la *InterfaceUsuario*" y se asigna a la *PantallaCrearRegUsuario*.
48. La **InterfaceUsuario** envía el evento "Registrar" al **ManejadorRegistro-Usuario**. Se identifica la responsabilidad "envía el evento "Registrar" al *ManejadorRegistroUsuario*" y se asigna a la *InterfaceUsuario*. Adicionalmente, se asigna la responsabilidad "maneja el evento "Registrar" al *ManejadorRegistroUsuario*.
49. El **ManejadorRegistroUsuario** solicita **crearRegistroUsuario a la InterfaceBaseDatosRegistro**. Se identifica la responsabilidad "solicita *crearRegistroUsuario* a la *InterfaceBaseDatosRegistro*" y se asigna al *ManejadorRegistroUsuario*.
50. La **InterfaceBaseDatosRegistro** solicita **crearRegistroUsuario a la Base de Datos Registro (E-1, E-2, E-3, E-4)**. Se identifica la responsabilidad "solicita *crearRegistroUsuario* a la *Base de Datos Registro*" y se asigna a la *InterfaceBaseDatosRegistro*. Obviamente, no tiene mucho sentido asignar responsabilidades a los actores, como en el caso de la *Base de Datos Registro*, ya que éstos son externos y normalmente ya han sido diseñados.
51. La **Base de Datos Registro** devuelve el OK a la **InterfaceBaseDatos-Registro**. Nuevamente, no se asigna ninguna responsabilidad a la *Base de Datos Registro* por ser externa al sistema. Por otro lado, y vale la pena resaltar, las frases de tipo "devolución de información" no originan asignación de responsabilidades, ya que únicamente son respuestas a solicitudes ante-

riores. Sólo se registran las propias solicitudes correspondientes a servicios del sistema.

52. **La InterfaceBaseDatosRegistro devuelve el OK al ManejadorRegistroUsuario.** No se asigna responsabilidades dado que la frase describe una devolución de información.
53. **Se continúa con el subflujo *Administrar Registro Usuario (S-3)*.** Nuevamente, esta frase informa de la continuación interna del caso de uso sin asignar responsabilidades.
54. **Si la actividad seleccionada es "Salir", la PantallaCrearRegUsuario envía el evento "Salir" a la InterfaceUsuario.** La responsabilidad "envía el evento "Salir" a la InterfaceUsuario" se asigna a la PantallaCrearRegUsuario.
55. **La InterfaceUsuario envía el evento "Salir" al ManejadorRegistroUsuario.** La responsabilidad "envía el evento "Salir" al ManejadorRegistroUsuario" se asigna a la InterfaceUsuario. Además, la responsabilidad "maneja el evento "Salir" se asigna al ManejadorRegistroUsuario.
56. **El ManejadorRegistroUsuario sale del sistema.** Se asigna la responsabilidad "sale del sistema" al ManejadorRegistroUsuario.
57. **(Si aún no se ha presionado "Registrar", se perderá la información.)**
De nuevo, ésta es una frase informativa, por lo que no se asignan nuevas responsabilidades.

A partir de estas frases se tienen nuevas responsabilidades y se insertan en las tarjetas de clase correspondientes. Dado que no todas las clases agregan nuevas responsabilidades, se mostrarán sólo aquellas que sí lo hacen. En particular, las responsabilidades identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no cambiaron y se mantienen iguales a las descritas en las tablas 8.3 y 8.5, respectivamente. De manera similar, las responsabilidades para las clases *ManejadorServicio* y *PantallaServicio* no cambiaron y se mantienen iguales a las descritas en las tablas 8.10 y 8.11, respectivamente.

En la tabla 8.12 aparecen las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.9 junto con las nuevas responsabilidades.

Tabla 8.12 Tarjeta para la clase <i>InterfaceUsuario</i> con responsabilidades identificadas hasta el momento.	
Clase:	InterfaceUsuario
Descripción:	toda la interacción con el usuario se hace por medio de la interface de usuario.
Módulo:	InterfaceUsuario
Estereotipo:	Borde
Propiedades:	
Superclases:	
Subclases:	
Atributos:	

continúa

Hidden page

Se agrega una nueva tarjeta de clase que describe las responsabilidades para la clase *PantallaCrearRegUsuario*, como se muestra en la tabla 8.14.

Tabla 8.14 Tarjeta para la clase <i>PantallaCrearRegUsuario</i> con responsabilidades identificadas hasta el momento.	
Clase: PantallaCrearRegUsuario	
Descripción: pantalla de solicitud de registro de usuario (P-3).	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega (44)	
envía el evento "Registrar" a la <i>InterfaceUsuario</i> (47)	
envía el evento "Salir" a la <i>InterfaceUsuario</i> (54)	

En la tabla 8.15 se muestran las responsabilidades para la clase *InterfaceBaseDatosRegistro* identificadas en la tabla 8.7 junto con las nuevas responsabilidades.

Tabla 8.15 Tarjeta para la clase <i>InterfaceBaseDatosRegistro</i> con responsabilidades identificadas hasta el momento.	
Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro a la que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Estereotipo: Interface	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita <i>validarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (14)	
solicita <i>crearRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (50)	

Continuamos con el subflujo *Obtener Registro Usuario (S-2)* del caso de uso *RegistrarUsuario* como se muestra a continuación,

Subflujos	S-2 Obtener Registro Usuario El <u>ManejadorRegistroUsuario</u> solicita <i>obtenerRegistroUsuario</i> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <i>obtenerRegistroUsuario</i> a la <u>Base de Datos Registro</u>. La <u>Base de Datos Registro</u> devuelve el OK y el <u>RegistroUsuario</u> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK y el <u>RegistroUsuario</u> al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <i>Administrar Registro Usuario</i> (S-3).
------------------	--

Nuevamente, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

58. El ManejadorRegistroUsuario solicita *obtenerRegistroUsuario* a la InterfaceBaseDatosRegistro. La responsabilidad es "solicita *obtenerRegistroUsuario* a la InterfaceBaseDatosRegistro" y se asigna a ManejadorRegistroUsuario.
59. La InterfaceBaseDatosRegistro solicita *obtenerRegistroUsuario* a la Base de Datos Registro. La responsabilidad es "solicita *obtenerRegistroUsuario* a la Base de Datos Registro" y se asigna a InterfaceBaseDatosRegistro.
60. La Base de Datos Registro devuelve el OK y el RegistroUsuario a la InterfaceBaseDatosRegistro. Nuevamente, los eventos de devolución de información no significan responsabilidades adicionales, por lo que esta frase no agrega ninguna.
61. La InterfaceBaseDatosRegistro devuelve el OK y el RegistroUsuario al ManejadorRegistroUsuario. Esta frase tampoco agrega ninguna responsabilidad.
62. Se continúa con el subflujo *Administrar Registro Usuario* (S-3). Aunque ésta es una frase exclusivamente informativa de la continuación interna del caso de uso, se aprovechará y asignará una nueva responsabilidad "solicita *administrarRegistroUsuario*" la que se asignará a ManejadorRegistroUsuario.

En total se agregaron dos nuevas responsabilidades. La primera responsabilidad (58) y la segunda (62) se añaden a la clase ManejadorRegistroUsuario como se muestra en la tabla 8.16 que incluye las responsabilidades identificadas en la tabla 8.13.

Tabla 8.16 Tarjeta para la clase ManejadorRegistroUsuario con responsabilidades identificadas hasta el momento.

Clase: ManejadorRegistroUsuario
Descripción: el manejador de registro de usuario se encarga de todo lo relacionado con registro del usuario para utilizar el sistema.
Módulo: Registro.Usuario
Estereotipo: Control

continúa

Tabla 8.16 Continuación

Propiedades:	
Superclases:	
Subclases:	
Atributos:	
crearRegistroUsuario (7)	
solicita validarRegistroUsuario a la InterfaceBaseDatosRegistro (13)	
solicita desplegarPantallaCrearRegUsuario a la InterfaceUsuario (42)	
maneja el evento "Registrar" (48)	
solicita crearRegistroUsuario a la InterfaceBaseDatosRegistro (49)	
maneja el evento "Salir" (55)	
sale del sistema (56)	
solicita obtenerRegistroUsuario a la InterfaceBaseDatosRegistro (58)	
solicita administrarRegistroUsuario (62)	

La segunda responsabilidad (59) se agrega a la clase *InterfaceBaseDatosRegistro* como se muestra en la tabla 8.17 que incluye las responsabilidades identificadas en la tabla 8.13.

Tabla 8.17 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades identificadas hasta el momento.

Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro, la que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Estereotipo: Interface	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita validarRegistroUsuario a la BaseDatosRegistro (14)	
solicita crearRegistroUsuario a la BaseDatosRegistro (50)	
solicita obtenerRegistroUsuario a la BaseDatosRegistro (59)	

Se continúa con el subflujo *Administrar Registro Usuario (S-3)* del caso de uso *Registrar Usuario* como se muestra a continuación,

Subflujos	S-3 Administrar Registro Usuario El <u>ManejadorRegistroUsuario</u> solicita <u>desplegarPantallaObtenerRegUsuario</u> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> <u>despliega</u> la <u>PantallaObtenerRegUsuario</u>. La <u>PantallaObtenerRegUsuario</u> se <u>despliega</u>. El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Eliminar", "Actualizar", "Registrar Tarjeta", "Servicios" y "Salir". Si el usuario presiona "Actualizar" se ejecuta el subflujo <i>Actualizar Registro Usuario (S-4)</i>. Si el usuario selecciona "Eliminar" se ejecuta el subflujo <i>Eliminar Registro Usuario (S-5)</i>. Si el usuario presiona "Registrar Tarjeta", la <u>PantallaObtenerRegUsuario</u> envía el evento "Registrar Tarjeta" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Registrar Tarjeta" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>registrarTarjeta</u> al <u>ManejadorRegistroTarjeta</u>, se continúa con el caso de uso <i>Registrar Tarjeta</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaObtenerRegUsuario</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <u>ofrecerServicio</u> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaObtenerRegUsuario</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> <u>sale</u> del sistema. (Si aún no se ha presionado "Actualizar", la nueva información se perderá.)
-----------	--

De nuevo, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

63. El ManejadorRegistroUsuario solicita desplegarPantallaObtenerRegUsuario a la InterfaceUsuario. La responsabilidad es "solicita desplegarPantallaObtenerRegUsuario a la InterfaceUsuario" y se asigna al ManejadorRegistroUsuario.
64. La InterfaceUsuario despliega la PantallaObtenerRegUsuario. La responsabilidad es "despliega la PantallaObtenerRegUsuario" y se asigna a la InterfaceUsuario.
65. La PantallaObtenerRegUsuario se despliega. La responsabilidad es "despliega" y se asigna a la PantallaObtenerRegUsuario.
66. El Usuario selecciona entre las siguientes actividades: "Eliminar", "Actualizar", "Registrar Tarjeta", "Servicios" y "Salir". Ésta es una frase informativa que describe las diversas opciones del Usuario y no agrega responsabilidades.
67. Si el usuario presiona "Actualizar" se ejecuta el subflujo *Actualizar Registro Usuario (S-4)*. Ésta es una frase informativa que describe la continuación interna del flujo del caso de uso y no agrega responsabilidades.
68. Si el usuario selecciona "Eliminar" se ejecuta el subflujo *Eliminar Registro Usuario (S-5)*. De manera similar a la frase anterior, ésta es una frase informativa que describe la continuación interna del flujo del caso de uso y no agrega responsabilidades.

69. Si el usuario presiona "Registrar Tarjeta", la PantallaObtenerReg-Usuario envía el evento "Registrar Tarjeta" a la InterfaceUsuario. La responsabilidad es "envía el evento "Registrar Tarjeta" a la InterfaceUsuario" y se asigna a la PantallaObtenerRegUsuario.
70. La InterfaceUsuario envía el evento "Registrar Tarjeta" al Manejador-RegistroUsuario. La responsabilidad es "envía el evento "Registrar Tarjeta" al ManejadorRegistroUsuario" y se asigna a la InterfaceUsuario. De manera adicional, se asigna la nueva responsabilidad "maneja el evento "Registrar Tarjeta" al ManejadorRegistroUsuario.
71. El ManejadorRegistroUsuario solicita *registrarTarjeta* al Manejador-RegistroTarjeta, se continúa con el caso de uso *Registrar Tarjeta*. La responsabilidad es "solicita *registrarTarjeta* al ManejadorRegistroTarjeta" y se asigna al ManejadorRegistroUsuario. Adicionalmente, se asigna la responsabilidad "*registrarTarjeta*" al ManejadorRegistroTarjeta. La última sección de la frase describe una continuación de lógica entre casos de uso y no agrega responsabilidades.
72. Si la actividad seleccionada es "Servicios", la PantallaObtenerReg-Usuario envía el evento "Servicios" a la InterfaceUsuario. La responsabilidad es "envía el evento "Servicios" a la InterfaceUsuario" y se asigna a PantallaObtenerRegUsuario.
73. La InterfaceUsuario envía el evento "Servicios" al ManejadorRegistro-Usuario. La responsabilidad es "envía el evento "Servicios" al Manejador-RegistroUsuario" y se asigna a InterfaceUsuario. Además, se agrega la responsabilidad "maneja el evento "Servicios" al ManejadorRegistroUsuario.
74. El ManejadorRegistroUsuario solicita *ofrecerServicio* al Manejador-Servicio, se continúa con el caso de uso *Ofrecer Servicios*. La responsabilidad es "solicita *ofrecerServicio* al ManejadorServicio" y se asigna al ManejadorRegistroUsuario. No es necesario volver a asignar la responsabilidad "*ofrecerServicio*" a ManejadorServicio, pues esto ya se hizo. La última sección de la frase describe la continuación del flujo interno del caso de uso y no agrega nuevas responsabilidades.
75. Si la actividad seleccionada es "Salir", la PantallaObtenerRegUsuario envía el evento "Salir" a la InterfaceUsuario. La responsabilidad es "envía el evento "Salir" a la InterfaceUsuario" y se asigna a PantallaObtener-RegUsuario.
76. La InterfaceUsuario envía el evento "Salir" al ManejadorRegistro-Usuario. La responsabilidad es "envía el evento "Salir" al ManejadorRegistro-Usuario" y se asigna a InterfaceUsuario. Adicionalmente se debe asignar la responsabilidad "maneja el evento "Salir" al ManejadorRegistroUsuario. Sin embargo, esta responsabilidad ya se asignó, por lo que no es necesario duplicarla.
77. El ManejadorRegistroUsuario *sale del sistema*. La responsabilidad es "*sale del sistema*" y se debe asignar a ManejadorRegistroUsuario. Sin embargo, ésta es una responsabilidad duplicada y no se vuelve a asignar.

78. (Si aún no se ha presionado "Actualizar", la nueva información se perderá.) Esta frase es informativa y no agrega responsabilidades adicionales.

A partir de estas frases se obtienen responsabilidades adicionales y se insertan en las tarjetas de clase correspondientes. Nuevamente, no todas las clases agregan nuevas responsabilidades. En particular, las responsabilidades identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no cambiaron y se mantienen igual a las descritas en las tablas 8.3 y 8.5, respectivamente. De manera similar, las responsabilidades identificadas para las clases *ManejadorServicio* y *PantallaServicio* no cambiaron y se mantienen iguales a las descritas en las tablas 8.10 y 8.11, respectivamente. Además, se mantienen iguales las clases *PantallaCrearRegUsuario* correspondiente a la tabla 8.14, junto con la *InterfaceBaseDatosRegistro* correspondiente a la tabla 8.17.

En la tabla 8.18 se muestran las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.12 junto con sus nuevas responsabilidades.

Tabla 8.18 Tarjeta para la clase *InterfaceUsuario* con responsabilidades identificadas hasta el momento.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <i>PantallaPrincipal</i> (2)	
envía el evento "Registrarse por Primera Vez" al <i>ManejadorPrincipal</i> (6)	
envía el evento "OK" al <i>ManejadorPrincipal</i> (11)	
envía el evento "Salir" al <i>ManejadorPrincipal</i> (21)	
despliega la <i>PantallaServicio</i> (24)	
envía el evento "Obtener Registro" al <i>ManejadorServicio</i> (36)	
envía el evento "Salir" al <i>ManejadorServicio</i> (40)	
despliega la <i>PantallaCrearRegUsuario</i> (43)	
envía el evento "Registrar" al <i>ManejadorRegistroUsuario</i> (48)	
envía el evento "Salir" al <i>ManejadorRegistroUsuario</i> (55)	
despliega la <i>PantallaObtenerRegUsuario</i> (64)	
envía el evento "Registrar Tarjeta" al <i>ManejadorRegistroUsuario</i> (70)	
envía el evento "Servicios" al <i>ManejadorRegistroUsuario</i> (73)	

Hidden page

Tabla 8.20 Continuación

Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega (65)	
envía el evento "Registrar Tarjeta" a la <u>InterfaceUsuario</u> (69)	
envía el evento "Servicios" a la <u>InterfaceUsuario</u> (72)	
envía el evento "Salir" a la <u>InterfaceUsuario</u> (75)	

La tabla 8.21 muestra las responsabilidades identificadas hasta el momento para la clase *ManejadorRegistroTarjeta*.

Tabla 8.21 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades identificadas hasta el momento.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario que permite pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
RegistrarTarjeta (71)	

Se continúa con el subflujo *Actualizar Registro Usuario (S-4)* del caso de uso *Registrar Usuario* como se muestra a continuación,

Subflujos	<i>S-4 Actualizar Registro Usuario</i> La <u>PantallaObtenerRegUsuario</u> envía el evento "Actualizar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Actualizar" al <u>ManejadorRegistroUsuario</u>. El <u>ManejadorRegistroUsuario</u> solicita <i>actualizarRegistroUsuario</i> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <i>actualizarRegistroUsuario</i> a la <u>Base de Datos Registro</u>. La <u>Base de Datos Registro</u> actualiza el <u>RegistroUsuario</u> (E-1, E-2, E-4) y devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroUsuario</u>. Se continúa con el subflujo <i>Administrar Registro Usuario (S-3)</i>.
------------------	---

Nuevamente, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

79. **La PantallaObtenerRegUsuario envía el evento “Actualizar” a la InterfaceUsuario.** La responsabilidad es “envía el evento “Actualizar” a la InterfaceUsuario” y se asigna a PantallaObtenerRegUsuario.
80. **La InterfaceUsuario envía el evento “Actualizar” al ManejadorRegistroUsuario.** La responsabilidad es “envía el evento “Actualizar” al ManejadorRegistroUsuario” y se asigna a InterfaceUsuario. Además, se agrega la responsabilidad “maneja el evento “Actualizar”” y se asigna al ManejadorRegistroUsuario.
81. **El ManejadorRegistroUsuario solicita *actualizarRegistroUsuario* a la InterfaceBaseDatosRegistro.** La responsabilidad es “solicita *actualizarRegistroUsuario* a la InterfaceBaseDatosRegistro” y se asigna al ManejadorRegistroUsuario.
82. **La InterfaceBaseDatosRegistro solicita *actualizarRegistroUsuario* a la Base de Datos Registro.** La responsabilidad es “solicita *actualizarRegistroUsuario* a la Base de Datos Registro” y se asigna a la InterfaceBaseDatosRegistro.
83. **La Base de Datos Registro actualiza el RegistroUsuario (E-1, E-2, E-4) y devuelve el OK a la InterfaceBaseDatosRegistro.** Esta frase se refiere a Base de Datos Registro, un actor externo al sistema, por lo que no se agregan nuevas responsabilidades. La segunda parte de la frase describe una devolución y tampoco se agrega ninguna responsabilidad.
84. **La InterfaceBaseDatosRegistro devuelve el OK al ManejadorRegistroUsuario.** Nuevamente, se describe una devolución por lo que no se agrega ninguna responsabilidad.
85. **Se continúa con el subflujo *Administrar Registro Usuario* (S-3).** Ésta es una frase que describe flujo interno de continuación del caso de uso, por lo que no agrega responsabilidades.

A partir de estas frases se obtienen nuevas responsabilidades y se insertan en las tarjetas de clase correspondientes. Dado que no todas las clases agregan nuevas responsabilidades, se mostrará sólo aquellas que sí lo hacen. En particular, las responsabilidades identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no se modificaron y se mantienen iguales a las descritas en la tabla 8.3 y 8.5, respectivamente. De manera similar, las responsabilidades identificadas para las clases *ManejadorServicio* y *PantallaServicio* no se modificaron y quedan igual a las descritas en la tabla 8.10 y 8.11, respectivamente. Además, se mantiene igual la clase *PantallaCrearRegUsuario* correspondiente a la tabla 8.14.

En la tabla 8.22 se muestran las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.18 junto con su nueva responsabilidad.

Tabla 8.22 Tarjeta para la clase *InterfaceUsuario* con responsabilidades identificadas hasta el momento.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <i>PantallaPrincipal</i> (2)	
envía el evento "Registrarse por Primera Vez" al <i>ManejadorPrincipal</i> (6)	
envía el evento "OK" al <i>ManejadorPrincipal</i> (11)	
envía el evento "Salir" al <i>ManejadorPrincipal</i> (21)	
despliega la <i>PantallaServicio</i> (24)	
envía el evento "Obtener Registro" al <i>ManejadorServicio</i> (36)	
envía el evento "Salir" al <i>ManejadorServicio</i> (40)	
despliega la <i>PantallaCrearRegUsuario</i> (43)	
envía el evento "Registrar" al <i>ManejadorRegistroUsuario</i> (48)	
envía el evento "Salir" al <i>ManejadorRegistroUsuario</i> (55)	
despliega la <i>PantallaObtenerRegUsuario</i> (64)	
envía el evento "Registrar Tarjeta" al <i>ManejadorRegistroUsuario</i> (70)	
envía el evento "Servicios" al <i>ManejadorRegistroUsuario</i> (73)	
envía el evento "Actualizar" al <i>ManejadorRegistroUsuario</i> (80)	

En la tabla 8.23 se muestran las responsabilidades para la clase *ManejadorRegistroUsuario* identificadas en la tabla 8.19 junto con las nuevas responsabilidades.

Tabla 8.23 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades ya identificadas.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de todo lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades:	
Superclases:	

continúa

Tabla 8.23 Continuación

Subclases:	
Atributos:	
<i>crearRegistroUsuario</i> (7)	
solicita <i>validarRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i> (13)	
solicita <i>desplegarPantallaCrearRegUsuario</i> a la <i>InterfaceUsuario</i> (42)	
maneja el evento "Registrar" (48)	
solicita <i>crearRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i> (49)	
maneja el evento "Salir" (55)	
<i>sale</i> del sistema (56)	
solicita <i>obtenerRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i> (58)	
solicita <i>administrarRegistroUsuario</i> (62)	
solicita <i>desplegarPantallaObtenerRegUsuario</i> a la <i>InterfaceUsuario</i> (63)	
maneja el evento "Registrar Tarjeta" (70)	
solicita <i>registrarTarjeta</i> al <i>ManejadorRegistroTarjeta</i> (71)	
maneja el evento "Servicios" (73)	
solicita <i>ofrecerServicios</i> al <i>ManejadorServicio</i> (74)	
maneja el evento "Actualizar" (80)	
solicita <i>actualizarRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i> (81)	

En la tabla 8.24 se muestran las responsabilidades para la clase *PantallaObtenerRegUsuario* identificadas en la tabla 8.20 junto con la nueva responsabilidad.

Tabla 8.24 Tarjeta para la clase *PantallaObtenerRegUsuario* con nuevas responsabilidades identificadas hasta el momento.

Clase: <i>PantallaObtenerRegUsuario</i>	
Descripción: pantalla de devolución con información de registro de usuario (P-4).	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i> (65)	
envía el evento "Registrar Tarjeta" a la <i>InterfaceUsuario</i> (69)	
envía el evento "Servicios" a la <i>InterfaceUsuario</i> (72)	
envía el evento "Salir" a la <i>InterfaceUsuario</i> (75)	
envía el evento "Actualizar" a la <i>InterfaceUsuario</i> (79)	

Hidden page

la responsabilidad “maneja el evento “Eliminar”” y se asigna al ManejadorRegistroUsuario.

88. **El ManejadorRegistroUsuario solicita *eliminarRegistroUsuario* a la InterfaceBaseDatosRegistro.** La responsabilidad es “solicita *eliminarRegistroUsuario* a la InterfaceBaseDatosRegistro” y se asigna al ManejadorRegistroUsuario.
89. **La InterfaceBaseDatosRegistro envía el evento *eliminarRegistroUsuario* a la Base de Datos Registro.** La responsabilidad es “solicita *eliminarRegistroUsuario* a la Base de Datos Registro” y se asigna a la InterfaceBaseDatosRegistro.
90. **La Base de Datos Registro elimina el RegistroUsuario y devuelve el OK a la InterfaceBaseDatosRegistro.** Esta frase se refiere a Base de Datos Registro, un actor externo al sistema, por tanto, no se agregan nuevas responsabilidades. La segunda parte de la frase describe una devolución y tampoco se agrega ninguna responsabilidad.
91. **La InterfaceBaseDatosRegistro devuelve el OK al ManejadorRegistroUsuario.** De nuevo se describe una devolución por lo que no se agrega ninguna responsabilidad.
92. **Se continúa con el subflujo *Crear Registro Usuario (S-1)*.** Ésta es una frase que describe flujo interno de continuación del caso de uso, y no agrega responsabilidades.

A partir de estas frases se obtienen nuevas responsabilidades y se insertan en las tarjetas de clase correspondientes. Dado que no todas las clases agregan nuevas responsabilidades, se mostrarán sólo aquellas que sí lo hacen. En particular, las responsabilidades identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no cambiaron y se mantienen igual a las descritas en la tabla 8.3 y 8.5, respectivamente. De manera similar, las responsabilidades identificadas para las clases *ManejadorServicio* y *PantallaServicio* no cambiaron y se mantienen igual a las descritas en las tablas 8.10 y 8.11, respectivamente. Además, se mantiene igual la clase *PantallaCrearRegUsuario* correspondiente a la tabla 8.14.

En la tabla 8.26 se muestran las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.22 junto con su nueva responsabilidad.

Tabla 8.26 Tarjeta para la clase <i>InterfaceUsuario</i> con responsabilidades identificadas hasta el momento.	
Clase:	InterfaceUsuario
Descripción:	toda la interacción con el usuario se hace por medio de la interface de usuario.
Módulo:	InterfaceUsuario
Estereotipo:	Borde
Propiedades:	
Superclases:	

continúa

Hidden page

Hidden page

En la tabla 8.29 se muestran las responsabilidades para la clase *InterfaceBaseDatosRegistro* identificadas en la tabla 8.25 junto con la nueva responsabilidad.

Tabla 8.29 Tarjeta para la clase <i>InterfaceBaseDatosRegistro</i> con responsabilidades identificadas hasta el momento.	
Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Estereotipo: Interface	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita <i>validarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (14)	
solicita <i>crearRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (50)	
solicita <i>obtenerRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (59)	
solicita <i>actualizarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (82)	
solicita <i>eliminarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (89)	

De manera general, también se obtienen responsabilidades a partir del manejo de excepciones, como se muestra a continuación,

Excepciones	<i>E-1 información incompleta:</i> falta información en el registro de usuario. Se le vuelve a pedir al usuario que complete el registro. <i>E-2 registro ya existe:</i> si ya existe un registro en ese <i>login</i>, se le pedirá al usuario que lo cambie o que termine el caso de uso. <i>E-3 login incorrecto:</i> el <i>login</i> no es válido. Se le vuelve a pedir al usuario que complete el registro. <i>E-4 contraseña incorrecta:</i> la contraseña escogida es muy sencilla o no se validó correctamente. Se le pide al usuario que complete el registro.
--------------------	--

Sin embargo, únicamente se concentrará en los flujos básicos y no en los alternos. En general, los alternos son los menos comunes, aunque importantes de diseñar pero no en una primera etapa.

REGISTRAR TARJETA

A continuación se muestra el flujo principal del caso de uso *Registrar Tarjeta* como se presentó al final del capítulo 7, correspondiente al modelo de análisis.

Flujo Principal	Se continúa con el subflujo <i>Obtener Registro Tarjeta (S-2)</i> . Si no existe un <u>RegistroTarjeta</u> válido se continúa con el subflujo <i>Crear Registro Tarjeta (S-1)</i> . De lo contrario, si ya existe un <u>RegistroTarjeta</u> válido, se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i> .
------------------------	---

Se toma cada una de las frases que describen el flujo y se analizarán para decidir qué responsabilidad representan y a qué clase se le asignan:

93. **Se continúa con el subflujo *Obtener Registro Tarjeta (S-2)*.** Esta frase describe el flujo interno del caso de uso y no agrega responsabilidades.
94. **Si no existe un RegistroTarjeta válido se continúa con el subflujo *Crear Registro Tarjeta (S-1)*.** Esta frase informa y no agrega responsabilidades a las clases.
95. **De lo contrario, si ya existe un RegistroTarjeta válido, se continúa con el subflujo *Administrar Registro Tarjeta (S-3)*.** Nuevamente, esta frase informa y no agrega responsabilidades a las clases.

Como se aprecia, el flujo anterior no agrega responsabilidades adicionales.

Se continúa con el subflujo *Crear Registro Tarjeta (S-1)* del caso de uso *Registrar-Tarjeta* como se muestra a continuación,

Subflujos	S-1 Crear Registro Tarjeta El <u>ManejadorRegistroTarjeta</u> solicita <i>desplegarPantallaCrearRegTarjeta</i> a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> <i>despliega</i> la <u>PantallaCrearRegTarjeta</u>. La <u>PantallaCrearRegTarjeta</u> <i>se despliega</i>. Esta pantalla contiene información que debe llenar el <u>Usuario</u>, que incluye el nombre como aparece en la tarjeta, número de tarjeta, el tipo de tarjeta y la fecha de vencimiento. El <u>Usuario</u> puede seleccionar entre las siguientes actividades: "Registrar", "Servicios" y "Salir". Si el <u>Usuario</u> selecciona "Registrar", la <u>PantallaCrearRegTarjeta</u> envía el evento "Registrar" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Registrar" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> solicita <i>crearRegistroTarjeta</i> a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> solicita <i>crearRegistroTarjeta</i> a la <u>Base de Datos Registro (E-1)</u>. La <u>Base de Datos Registro</u> devuelve el OK a la <u>InterfaceBaseDatosRegistro</u>. La <u>InterfaceBaseDatosRegistro</u> devuelve el OK al <u>ManejadorRegistroTarjeta</u>. Se continúa con el subflujo <i>Administrar Registro Tarjeta (S-3)</i>. Si la actividad seleccionada es "Servicios", la <u>PantallaCrearRegTarjeta</u> envía el evento "Servicios" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Servicios" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>, se continúa con el caso de uso <i>Ofrecer Servicios</i>. Si la actividad seleccionada es "Salir", la <u>PantallaCrearRegTarjeta</u> envía el evento "Salir" a la <u>InterfaceUsuario</u>. La <u>InterfaceUsuario</u> envía el evento "Salir" al <u>ManejadorRegistroTarjeta</u>. El <u>ManejadorRegistroTarjeta</u> <i>sale del sistema</i>. (Si aún no se ha presionado "Registrar", la información se perderá.)
------------------	---

De nuevo, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

Hidden page

107. **Se continúa con el subflujo *Administrar Registro Tarjeta (S-3)*.** Aunque ésta es una frase informativa de la continuación interna del caso de uso, se aprovechará y asignará una nueva responsabilidad “solicita *administrarRegistroTarjeta*” que se asignará a ManejadorRegistroTarjeta.
108. **Si la actividad seleccionada es “Servicios”, la PantallaCrearRegTarjeta envía el evento “Servicios” a la InterfaceUsuario.** La responsabilidad “envía el evento “Servicios” a la InterfaceUsuario” se asigna a la Pantalla-CrearRegTarjeta.
109. **La InterfaceUsuario envía el evento “Servicios” al ManejadorRegistro-Tarjeta.** Se asigna la responsabilidad “envía el evento “Servicios”” a la InterfaceUsuario. De manera adicional, se asigna la responsabilidad “maneja el evento “Servicios”” al ManejadorRegistroTarjeta.
110. **El ManejadorRegistroTarjeta solicita *ofrecerServicio* al Manejador-Servicio, se continúa con el caso de uso *Ofrecer Servicios*.** Se identifica la responsabilidad “solicita *ofrecerServicio* al ManejadorServicio” y se asigna al ManejadorRegistroTarjeta. La última sección no agrega responsabilidades por describir un flujo entre casos de uso.
111. **Si la actividad seleccionada es “Salir”, la PantallaCrearRegTarjeta envía el evento “Salir” a la InterfaceUsuario.** La responsabilidad “envía el evento “Salir” a la InterfaceUsuario” se asigna a la PantallaCrearReg-Tarjeta.
112. **La InterfaceUsuario envía el evento “Salir” al ManejadorRegistro-Tarjeta.** La responsabilidad “envía el evento “Salir” al ManejadorRegistro-Tarjeta” se asigna a la InterfaceUsuario. Adicionalmente, se asigna la responsabilidad “maneja el evento “Salir”” al ManejadorRegistroTarjeta.
113. **El ManejadorRegistroTarjeta sale del sistema.** Se asigna la responsabilidad “sale del sistema” al ManejadorRegistroTarjeta.
114. **(Si aún no se ha presionado “Registrar”, la información se perderá.)**
De nuevo, ésta es una frase que informa y no se asignan nuevas responsabilidades.

A partir de estas frases, se obtienen nuevas responsabilidades y se insertarán en las tarjetas de clase correspondientes. Dado que no todas las clases agregan nuevas responsabilidades, se mostrarán sólo aquellas que sí lo hacen. En particular, las responsabilidades identificadas para las clases *ManejadorPrincipal* y *Pantalla-Principal* no se modificaron y se mantienen igual a las descritas en las tablas 8.3 y 8.5, respectivamente. Las responsabilidades identificadas para las clases *ManejadorServicio* y *PantallaServicio* tampoco cambiaron y se mantienen igual a las descritas en la tabla 8.10 y 8.11, respectivamente. Además, las responsabilidades identificadas para las clases *ManejadorRegistroUsuario*, *PantallaCrearReg-Usuario* y *PantallaObtenerRegUsuario* tampoco se modificaron y se mantienen como las descritas en las tablas 8.27, tabla 8.14 y tabla 8.28, respectivamente.

En la tabla 8.30 se muestran las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.26 junto con sus nuevas responsabilidades.

Tabla 8.30 Tarjeta para la clase *InterfaceUsuario* con responsabilidades identificadas hasta el momento.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <u>PantallaPrincipal</u> (2)	
envía el evento "Registrarse por Primera Vez" al <u>ManejadorPrincipal</u> (6)	
envía el evento "OK" al <u>ManejadorPrincipal</u> (11)	
envía el evento "Salir" al <u>ManejadorPrincipal</u> (21)	
despliega la <u>PantallaServicio</u> (24)	
envía el evento "Obtener Registro" al <u>ManejadorServicio</u> (36)	
envía el evento "Salir" al <u>ManejadorServicio</u> (40)	
despliega la <u>PantallaCrearRegUsuario</u> (43)	
envía el evento "Registrar" al <u>ManejadorRegistroUsuario</u> (48)	
envía el evento "Salir" al <u>ManejadorRegistroUsuario</u> (55)	
despliega la <u>PantallaObtenerRegUsuario</u> (64)	
envía el evento "Registrar Tarjeta" al <u>ManejadorRegistroUsuario</u> (70)	
envía el evento "Servicios" al <u>ManejadorRegistroUsuario</u> (73)	
envía el evento "Actualizar" al <u>ManejadorRegistroUsuario</u> (80)	
envía el evento "Eliminar" al <u>ManejadorRegistroUsuario</u> (87)	
despliega la <u>PantallaCrearRegTarjeta</u> (97)	
envía el evento "Registrar" al <u>ManejadorRegistroTarjeta</u> (102)	
envía el evento "Servicios" al <u>ManejadorRegistroTarjeta</u> (109)	
envía el evento "Salir" al <u>ManejadorRegistroTarjeta</u> (112)	

En la tabla 8.31 se muestran las responsabilidades para la clase *ManejadorRegistroTarjeta* identificadas en la tabla 8.21 junto con las nuevas responsabilidades.

Tabla 8.31 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades identificadas hasta el momento.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con registro de la tarjeta del usuario que permite pagar las reservaciones.	
Módulo: Registro.Tarjeta	

continúa

Tabla 8.31 Continuación	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>registrarTarjeta</i> (71)	
solicita <i>desplegarPantallaCrearRegTarjeta</i> a la <i>InterfaceUsuario</i> (96)	
maneja el evento "Registrar" (102)	
solicita <i>crearRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i> (103)	
solicita <i>administrarRegistroTarjeta</i> (107)	
maneja el evento "Servicios" (109)	
solicita <i>ofrecerServicio</i> al <i>ManejadorServicio</i> (110)	
maneja el evento "Salir" (112)	
sale del sistema (113)	

Se agrega una nueva tarjeta de clase que describe las responsabilidades para la clase *PantallaCrearRegTarjeta*, como se muestra en la tabla 8.32.

Tabla 8.32 Tarjeta para la clase <i>PantallaCrearRegTarjeta</i> con las responsabilidades identificadas hasta el momento.	
Clase: <i>PantallaCrearRegTarjeta</i>	
Descripción: pantalla de solicitud de registro de tarjeta (P-5).	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i> (98)	
envía el evento "Registrar" a la <i>InterfaceUsuario</i> (101)	
envía el evento "Servicios" a la <i>InterfaceUsuario</i> (108)	
envía el evento "Salir" a la <i>InterfaceUsuario</i> (111)	

En la tabla 8.33 se muestran las responsabilidades para la clase *InterfaceBaseDatosRegistro* identificadas en la tabla 8.29 junto con la nueva responsabilidad.

Tabla 8.33 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades identificadas hasta el momento.

Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Estereotipo: Interface	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita <i>validarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (14)	
solicita <i>crearRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (50)	
solicita <i>obtenerRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (59)	
solicita <i>actualizarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (82)	
solicita <i>eliminarRegistroUsuario</i> a la <i>BaseDatosRegistro</i> (89)	
solicita <i>crearRegistroTarjeta</i> a la <i>BaseDatosRegistro</i> (104)	

Se continúa con el subflujo *Obtener Registro Tarjeta (S-2)* del caso de uso *Registrar Tarjeta* como se muestra a continuación,

Subflujos	S-2 Obtener Registro Tarjeta El <i>ManejadorRegistroTarjeta</i> solicita <i>obtenerRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>. La <i>InterfaceBaseDatosRegistro</i> solicita <i>obtenerRegistroTarjeta</i> a la <i>Base de Datos Registro</i>. La <i>Base de Datos Registro</i> devuelve el OK y el <i>RegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>. La <i>InterfaceBaseDatosRegistro</i> devuelve el OK y el <i>RegistroTarjeta</i> al <i>ManejadorRegistroTarjeta</i>. Se regresa al flujo anterior.
------------------	--

Nuevamente, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

115. El ***ManejadorRegistroTarjeta*** solicita ***obtenerRegistroTarjeta*** a la ***InterfaceBaseDatosRegistro***. La responsabilidad es "solicita *obtenerRegistroTarjeta* a la *InterfaceBaseDatosRegistro*" y se asigna a *ManejadorRegistroTarjeta*.
- * 116. La ***InterfaceBaseDatosRegistro*** solicita ***obtenerRegistroTarjeta*** a la ***Base de Datos Registro***. La responsabilidad es "solicita *obtenerRegistroTarjeta* a la *Base de Datos Registro*" y se asigna a *InterfaceBaseDatosRegistro*.

117. La **Base de Datos Registro** devuelve el **OK** y el **RegistroTarjeta** a la **InterfaceBaseDatosRegistro**. De nuevo, los eventos de devolución de información no significan responsabilidades adicionales, por lo que esta frase no agrega ninguna.
118. La **InterfaceBaseDatosRegistro** devuelve el **OK** y el **RegistroTarjeta** al **ManejadorRegistroTarjeta**. Esta frase tampoco agrega ninguna responsabilidad.
119. **Se regresa al flujo anterior**. Ésta es una frase exclusivamente informativa de la continuación del caso de uso y no agrega nuevas responsabilidades.

En total, se agregaron dos nuevas responsabilidades. La primera responsabilidad (115) se añade a la clase *ManejadorRegistroTarjeta* como se muestra en la tabla 8.34, que incluye las responsabilidades identificadas en la tabla 8.31.

Tabla 8.34 Tarjeta para la clase <i>ManejadorRegistroTarjeta</i> con responsabilidades identificadas hasta el momento.	
Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario que permite pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
registrarTarjeta (71)	
solicita desplegarPantallaCrearRegTarjeta a la InterfaceUsuario (96)	
maneja el evento "Registrar" (102)	
solicita crearRegistroTarjeta a la InterfaceBaseDatosRegistro (103)	
solicita administrarRegistroTarjeta (107)	
maneja el evento "Servicios" (109)	
solicita ofrecerServicio al ManejadorServicio (110)	
maneja el evento "Salir" (112)	
sale del sistema (113)	
solicita obtenerRegistroTarjeta a la InterfaceBaseDatosRegistro (115)	

La segunda responsabilidad se agrega a la clase *InterfaceBaseDatosRegistro* como se muestra en la tabla 8.35 que incluye las responsabilidades identificadas en la tabla 8.33.

Hidden page

120. **El ManejadorRegistroTarjeta solicita desplegarPantallaObtenerRegTarjeta a la InterfaceUsuario.** La responsabilidad es "*solicita desplegarPantallaObtenerRegTarjeta a la InterfaceUsuario*" y se asigna al ManejadorRegistroTarjeta.
121. **La InterfaceUsuario despliega la PantallaObtenerRegTarjeta.** La responsabilidad es "*despliega la PantallaObtenerRegTarjeta*" y se asigna a la InterfaceUsuario.
122. **La PantallaObtenerRegTarjeta se despliega.** La responsabilidad es "*despliega*" y se asigna a la PantallaObtenerRegTarjeta.
123. **El Usuario selecciona entre las siguientes actividades: "Actualizar", "Eliminar", "Servicios" y "Salir".** Ésta es una frase informativa que describe las diversas opciones del Usuario y no agrega responsabilidades.
124. **Si el usuario presiona "Actualizar" se ejecuta el subflujo Actualizar Registro Tarjeta (S-4).** Esta frase informa y describe una continuación interna del flujo del caso de uso y no agrega responsabilidades.
125. **Si el usuario presiona "Eliminar" se ejecuta el subflujo Eliminar Registro Tarjeta (S-5).** De manera similar a la frase anterior, ésta es una frase que informa y describe una continuación interna del flujo del caso de uso y no añade responsabilidades.
126. **Si la actividad seleccionada es "Servicios", la PantallaObtenerRegTarjeta envía el evento "Servicios" a la InterfaceUsuario.** La responsabilidad es "*envía el evento "Servicios" a la InterfaceUsuario*" y se asigna a PantallaObtenerRegTarjeta.
127. **La InterfaceUsuario envía el evento "Servicios" al ManejadorRegistroTarjeta.** La responsabilidad es "*envía el evento "Servicios" al ManejadorRegistroTarjeta*" y se debe asignar a InterfaceUsuario. Esta responsabilidad está duplicada y no es necesario volverla a agregar. Tampoco es necesario agregar la responsabilidad "maneja el evento "Servicios" al ManejadorRegistroTarjeta, pues también está duplicada.
128. **El ManejadorRegistroTarjeta solicita ofrecerServicio al ManejadorServicio, se continúa con el caso de uso Ofrecer Servicios.** La responsabilidad es "*solicita ofrecerServicio al ManejadorServicio*" y se debe asignar al ManejadorRegistroTarjeta. Nuevamente, ésta es una responsabilidad duplicada. La última sección de la frase describe la continuación del flujo interno del caso de uso y no añade nuevas responsabilidades.
129. **Si la actividad seleccionada es "Salir", la PantallaObtenerRegTarjeta envía el evento "Salir" a la InterfaceUsuario.** La responsabilidad es "*envía el evento "Salir" a la InterfaceUsuario*" y se asigna a PantallaObtenerRegTarjeta.
130. **La InterfaceUsuario envía el evento "Salir" al ManejadorRegistroTarjeta.** La responsabilidad es "*envía el evento "Salir" al ManejadorRegistroTarjeta*" y se asigna a InterfaceUsuario. Además, se debe asignar la respon-

sabilidad “maneja el evento “Salir”” a ManejadorRegistroTarjeta. Sin embargo, esta responsabilidad ya se asignó y no es necesario duplicarla.

131. **El ManejadorRegistroTarjeta sale del sistema.** La responsabilidad es “sale del sistema” y se debe asignar a ManejadorRegistroTarjeta. Ésta es una responsabilidad duplicada por lo que no se vuelve a asignar.
132. **(Si aún no se ha presionado “Actualizar”, la nueva información se perderá.)** Esta frase informa y no agrega responsabilidades.

A partir de estas frases se obtienen responsabilidades adicionales y se insertan en las tarjetas de clase correspondientes. No todas las clases agregan nuevas responsabilidades; en particular, las identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no cambiaron y permanecen igual a las descritas en las tablas 8.3 y 8.5, respectivamente. Las responsabilidades identificadas para las clases *ManejadorServicio* y *PantallaServicio* tampoco se modificaron y permanecen igual a las descritas en las tablas 8.10 y 8.11, respectivamente. Las responsabilidades identificadas para las clases *ManejadorRegistroUsuario*, *PantallaCrearRegUsuario* y *PantallaObtenerRegUsuario* tampoco cambiaron y se mantienen igual a las descritas en las tablas 8.27, 8.14 y tabla 8.28, respectivamente. Tampoco se modifican las responsabilidades de *PantallaCrearRegTarjeta* manteniéndose igual a las descritas en la tabla 8.32.

En la tabla 8.36 se muestran las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.30 junto con su nueva responsabilidad.

Tabla 8.36 Tarjeta para la clase *InterfaceUsuario* con responsabilidades identificadas hasta el momento.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <u>PantallaPrincipal</u> (2)	
envía el evento “Registrarse por Primera Vez” al <u>ManejadorPrincipal</u> (6)	
envía el evento “OK” al <u>ManejadorPrincipal</u> (11)	
envía el evento “Salir” al <u>ManejadorPrincipal</u> (21)	
despliega la <u>PantallaServicio</u> (24)	
envía el evento “Obtener Registro” al <u>ManejadorServicio</u> (36)	
envía el evento “Salir” al <u>ManejadorServicio</u> (40)	
despliega la <u>PantallaCrearRegUsuario</u> (43)	

continúa

Tabla 8.36 Continuación

envía el evento "Registrar" al <u>ManejadorRegistroUsuario</u> (48)	
envía el evento "Salir" al <u>ManejadorRegistroUsuario</u> (55)	
despliega la <u>PantallaObtenerRegUsuario</u> (64)	
envía el evento "Registrar Tarjeta" al <u>ManejadorRegistroUsuario</u> (70)	
envía el evento "Servicios" al <u>ManejadorRegistroUsuario</u> (73)	
envía el evento "Actualizar" al <u>ManejadorRegistroUsuario</u> (80)	
envía el evento "Eliminar" al <u>ManejadorRegistroUsuario</u> (87)	
despliega la <u>PantallaCrearRegTarjeta</u> (97)	
envía el evento "Registrar" al <u>ManejadorRegistroTarjeta</u> (102)	
envía el evento "Servicios" al <u>ManejadorRegistroTarjeta</u> (109)	
envía el evento "Salir" al <u>ManejadorRegistroTarjeta</u> (112)	
despliega la <u>PantallaObtenerRegTarjeta</u> (121)	

En la tabla 8.37 se muestran las responsabilidades para la clase *ManejadorRegistroTarjeta* identificadas en la tabla 8.34 junto con la nueva responsabilidad.

Tabla 8.37 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades identificadas hasta el momento.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario a fin de pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Esteriotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
registrarTarjeta (71)	
solicita desplegarPantallaCrearRegTarjeta a la <u>InterfaceUsuario</u> (96)	
maneja el evento "Registrar" (102)	
solicita crearRegistroTarjeta a la <u>InterfaceBaseDatosRegistro</u> (103)	
solicita administrarRegistroTarjeta (107)	
maneja el evento "Servicios" (109)	
solicita ofrecerServicio al <u>ManejadorServicio</u> (110)	
maneja el evento "Salir" (112)	
sale del sistema (113)	
solicita obtenerRegistroTarjeta a la <u>InterfaceBaseDatosRegistro</u> (115)	
solicita desplegarPantallaObtenerRegTarjeta a la <u>InterfaceUsuario</u> (120)	

Se agrega una nueva tarjeta de clase que describe las responsabilidades para la clase *PantallaObtenerRegTarjeta*, como se muestra en la tabla 8.38.

Tabla 8.38 Tarjeta para la clase <i>PantallaObtenerRegTarjeta</i> con las responsabilidades identificadas hasta el momento.	
Clase: PantallaObtenerRegTarjeta	
Descripción: pantalla de devolución con información de registro de tarjeta (P-6).	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i> (122)	
envía el evento "Servicios" a la <i>InterfaceUsuario</i> (126)	
envía el evento "Salir" a la <i>InterfaceUsuario</i> (129)	

Se continúa con el subflujo *Actualizar Registro Tarjeta* (S-4) del caso de uso *Registrar Tarjeta* como se muestra

Subflujos	S-4 Actualizar Registro Tarjeta La <i>PantallaObtenerRegTarjeta</i> envía el evento "Actualizar" a la <i>InterfaceUsuario</i>. La <i>InterfaceUsuario</i> envía el evento "Actualizar" al <i>ManejadorRegistroTarjeta</i>. El <i>ManejadorRegistroTarjeta</i> solicita <i>actualizarRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>. La <i>InterfaceBaseDatosRegistro</i> solicita <i>actualizarRegistroTarjeta</i> a la <i>Base de Datos Registro</i> (E-1). La <i>Base de Datos Registro</i> actualiza el <i>RegistroTarjeta</i> y devuelve el evento OK a la <i>InterfaceBaseDatosRegistro</i>. La <i>InterfaceBaseDatosRegistro</i> devuelve el OK al <i>ManejadorRegistroTarjeta</i>. Se continúa con el subflujo <i>Administrar Registro Tarjeta</i> (S-3).
------------------	--

De nuevo, se toma cada una de las frases y se analizan para identificar nuevas responsabilidades.

133. La ***PantallaObtenerRegTarjeta*** envía el evento "Actualizar" a la ***InterfaceUsuario***. La responsabilidad es "envía el evento "Actualizar" a la *InterfaceUsuario*" y se asigna a *PantallaObtenerRegTarjeta*.
134. La ***InterfaceUsuario*** envía el evento "Actualizar" al ***ManejadorRegistroTarjeta***. La responsabilidad es "envía el evento "Actualizar" al *ManejadorRegistroTarjeta*" y se asigna a *InterfaceUsuario*. Además, se agrega la responsabilidad "maneja el evento "Actualizar" y se asigna al *ManejadorRegistroTarjeta*.
135. El ***ManejadorRegistroTarjeta*** solicita *actualizarRegistroTarjeta* a la ***InterfaceBaseDatosRegistro***. La responsabilidad es "solicita *actualizar-*

RegistroTarjeta a la *InterfaceBaseDatosRegistro* y se asigna al *ManejadorRegistroTarjeta*.

136. **La *InterfaceBaseDatosRegistro* solicita *actualizarRegistroTarjeta* a la Base de Datos Registro (E-I).** La responsabilidad es "solicita *actualizarRegistroTarjeta* a la *Base de Datos Registro*" y se asigna a la *InterfaceBaseDatosRegistro*.
137. **La Base de Datos Registro actualiza el *RegistroTarjeta* y devuelve el evento OK a la *InterfaceBaseDatosRegistro*.** Esta frase se refiere a *Base de Datos Registro*, un actor externo al sistema, por lo que no se agregan nuevas responsabilidades. La segunda parte de la frase describe una devolución y tampoco se agrega ninguna responsabilidad.
138. **La *InterfaceBaseDatosRegistro* devuelve el OK al *ManejadorRegistroTarjeta*.** Nuevamente, se describe una devolución, por lo que no se agrega ninguna responsabilidad.
139. **Se continúa con el subflujo *Administrar Registro Tarjeta (S-3)*.** Ésta es una frase que describe el flujo interno de continuación del caso de uso que no agrega responsabilidades.

A partir de estas frases se obtienen responsabilidades adicionales y se insertan en las tarjetas de clase correspondientes. No todas las clases agregan nuevas responsabilidades; en particular, las identificadas para las clases *ManejadorPrincipal* y *PantallaPrincipal* no cambiaron y se mantienen igual a las descritas en las tablas 8.3 y 8.5, respectivamente. Las responsabilidades identificadas para las clases *ManejadorServicio* y *PantallaServicio* tampoco se modificaron y se mantienen igual a las descritas en las tablas 8.10 y 8.11, respectivamente. Además, las responsabilidades identificadas para las clases *ManejadorRegistroUsuario*, *PantallaCrearRegUsuario* y *PantallaObtenerRegUsuario* tampoco cambiaron y se mantienen igual a las descritas en las tablas 8.27, 8.14 y 8.28, respectivamente. Tampoco se modifican las responsabilidades de *PantallaCrearRegTarjeta* y se mantienen igual a las descritas en la tabla 8.32.

En la tabla 8.39 se presentan las responsabilidades para la clase *InterfaceUsuario* identificadas en la tabla 8.36 junto con su nueva responsabilidad.

Tabla 8.39 Tarjeta para la clase <i>InterfaceUsuario</i> con responsabilidades ya identificadas.	
Clase: <i>InterfaceUsuario</i>	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: <i>InterfaceUsuario</i>	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <i>PantallaPrincipal</i> (2)	

continúa

Tabla 8.39 Continuación

envía el evento "Registrarse por Primera Vez" al <i>ManejadorPrincipal</i> (6)	
envía el evento "OK" al <i>ManejadorPrincipal</i> (11)	
envía el evento "Salir" al <i>ManejadorPrincipal</i> (21)	
despliega la <i>PantallaServicio</i> (24)	
envía el evento "Obtener Registro" al <i>ManejadorServicio</i> (36)	
envía el evento "Salir" al <i>ManejadorServicio</i> (40)	
despliega la <i>PantallaCrearRegUsuario</i> (43)	
envía el evento "Registrar" al <i>ManejadorRegistroUsuario</i> (48)	
envía el evento "Salir" al <i>ManejadorRegistroUsuario</i> (55)	
despliega la <i>PantallaObtenerRegistroUsuario</i> (64)	
envía el evento "Registrar Tarjeta" al <i>ManejadorRegistroUsuario</i> (70)	
envía el evento "Servicios" al <i>ManejadorRegistroUsuario</i> (73)	
envía el evento "Actualizar" al <i>ManejadorRegistroUsuario</i> (80)	
envía el evento "Eliminar" al <i>ManejadorRegistroUsuario</i> (87)	
despliega la <i>PantallaCrearRegTarjeta</i> (97)	
envía el evento "Registrar" al <i>ManejadorRegistroTarjeta</i> (102)	
envía el evento "Servicios" al <i>ManejadorRegistroTarjeta</i> (109)	
envía el evento "Salir" al <i>ManejadorRegistroTarjeta</i> (112)	
despliega la <i>PantallaObtenerRegTarjeta</i> (121)	
envía el evento "Actualizar" al <i>ManejadorRegistroTarjeta</i> (134)	

En la tabla 8.40 se muestran las responsabilidades para la clase *ManejadorRegistroTarjeta* identificadas en la tabla 8.37 junto con las nuevas responsabilidades.

Tabla 8.40 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades identificadas hasta el momento.

Clase: <i>ManejadorRegistroTarjeta</i>	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario con el fin de pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
registrarTarjeta (71)	
solicita desplegarPantallaCrearRegTarjeta a la <i>InterfaceUsuario</i> (96)	

continúa

Hidden page

Hidden page

Hidden page

Hidden page

Tabla 8.44 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades identificadas hasta el momento.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario a fin de pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
registrarTarjeta (71)	
solicita desplegarPantallaCrearRegTarjeta a la InterfaceUsuario (96)	
maneja el evento "Registrar" (102)	
solicita crearRegistroTarjeta a la InterfaceBaseDatosRegistro (103)	
solicita administrarRegistroTarjeta (107)	
maneja el evento "Servicios" (109)	
solicita ofrecerServicio al ManejadorServicio (110)	
maneja el evento "Salir" (112)	
sale del sistema (113)	
solicita obtenerRegistroTarjeta a la InterfaceBaseDatosRegistro (115)	
solicita desplegarPantallaObtenerRegTarjeta a la InterfaceUsuario (120)	
maneja el evento "Actualizar" (134)	
solicita actualizarRegistroTarjeta a la InterfaceBaseDatosRegistro (135)	
maneja el evento "Eliminar" (141)	
solicita eliminarRegistroTarjeta a la InterfaceBaseDatosRegistro (142)	

En la tabla 8.45 se muestran las responsabilidades para la clase *PantallaObtenerRegTarjeta* identificadas en la tabla 8.41 junto con su nueva responsabilidad.

Tabla 8.45 Tarjeta para la clase *PantallaObtenerRegTarjeta* con las responsabilidades identificadas hasta el momento.

Clase: PantallaObtenerRegTarjeta	
Descripción: pantalla de devolución con información de registro de tarjeta (P-6).	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades:	
Superclases:	

continúa

Hidden page

Excepciones	<i>E-1 información incompleta:</i> falta información indispensable para completar el registro de tarjeta. Se le vuelve a pedir al usuario que complete el registro de tarjeta.
--------------------	--

Se deja lo anterior por el momento siguiendo las mismas consideraciones anteriores.

De manera similar a los casos de uso *ValidarUsuario*, *OfrecerServicio*, *RegistrarUsuario* y *RegistrarTarjeta*, se tendrá que definir las responsabilidades por clase para el sistema completo.

A continuación se hace una revisión final de las responsabilidades para las distintas clases descritas hasta el momento. Observe que se eliminan los números de referencia en las responsabilidades.

INTERFACEUSUARIO

La tarjeta para la clase *InterfaceUsuario* se muestra en la tabla 8.47. según las responsabilidades identificadas en la tabla 8.43.

Tabla 8.47 Tarjeta para la clase *InterfaceUsuario* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega la <u>PantallaPrincipal</u>	
envía el evento "Registrarse por Primera Vez" al <u>ManejadorPrincipal</u>	
envía el evento "OK" al <u>ManejadorPrincipal</u>	
envía el evento "Salir" al <u>ManejadorPrincipal</u>	
despliega la <u>PantallaServicio</u>	
envía el evento "Obtener Registro" al <u>ManejadorServicio</u>	
envía el evento "Salir" al <u>ManejadorServicio</u>	
despliega la <u>PantallaCrearRegUsuario</u>	
envía el evento "Registrar" al <u>ManejadorRegistroUsuario</u>	
envía el evento "Salir" al <u>ManejadorRegistroUsuario</u>	
despliega la <u>PantallaObtenerRegUsuario</u>	
envía el evento "Registrar Tarjeta" al <u>ManejadorRegistroUsuario</u>	

continúa

Tabla 8.47 Continuación

envía el evento "Servicios" al <u>ManejadorRegistroUsuario</u>	
envía el evento "Actualizar" al <u>ManejadorRegistroUsuario</u>	
envía el evento "Eliminar" al <u>ManejadorRegistroUsuario</u>	
despliega la <u>PantallaCrearRegTarjeta</u>	
envía el evento "Registrar" al <u>ManejadorRegistroTarjeta</u>	
envía el evento "Servicios" al <u>ManejadorRegistroTarjeta</u>	
envía el evento "Salir" al <u>ManejadorRegistroTarjeta</u>	
despliega la <u>PantallaObtenerRegTarjeta</u>	
envía el evento "Actualizar" al <u>ManejadorRegistroTarjeta</u>	
envía el evento "Eliminar" al <u>ManejadorRegistroTarjeta</u>	

PRINCIPAL

La tarjeta para la clase *ManejadorPrincipal* se muestra en la tabla 8.48 según las responsabilidades identificadas en la tabla 8.3.

Tabla 8.48 Tarjeta para la clase *ManejadorPrincipal* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorPrincipal	
Descripción: el manejador principal es el encargado de desplegar la pantalla principal de interacción con el usuario, y luego delegar las diferentes funciones a los manejadores especializados apropiados.	
Módulo: Principal	
Esteriotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita <i>desplegarPantallaPrincipal</i> a la <u>InterfaceUsuario</u>	
maneja el evento "Registrarse por Primera Vez"	
solicita <i>crearRegistroUsuario</i> al <u>ManejadorRegistroUsuario</u>	
maneja el evento "OK"	
solicita <i>validarRegistroUsuario</i> al <u>ManejadorRegistroUsuario</u>	
solicita <i>ofrecerServicio</i> al <u>ManejadorServicio</u>	
maneja el evento "Salir"	
sale del sistema	

La tarjeta para la clase *PantallaPrincipal* se muestra en la tabla 8.49 según las responsabilidades identificadas en la tabla 8.5.

Tabla 8.49 Tarjeta para la clase *PantallaPrincipal* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: PantallaPrincipal	
Descripción: pantalla principal (P-1).	
Módulo: Principal	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i>	
envía el evento "Registrarse por Primera Vez" a la <i>InterfaceUsuario</i>	
envía el evento "OK" a la <i>InterfaceUsuario</i>	
envía el evento "Salir" a la <i>InterfaceUsuario</i>	

REGISTRO

Este módulo se compone de los módulos de *Usuario*, *Tarjeta* e *InterfaceBD*.

USUARIO

En este módulo se describirán las clases de registro de usuario que son: *ManejadorRegistroUsuario*, *PantallaCrearRegUsuario*, *PantallaObtenerRegUsuario* y *RegistroUsuario*. La clase *ManejadorRegistroUsuario* administra todo lo relacionado con el registro de usuario, que incluye recibir y enviar eventos entre *ManejadorPrincipal*, *InterfaceUsuario*, e *InterfaceBaseDatosRegistro*, como se muestra en la tabla 8.50. Las responsabilidades se describen a partir de aquellas identificadas en la tabla 8.27.

Tabla 8.50 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de todo lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades:	
Superclases:	

continúa

Tabla 8.50 Continuación

Subclases:	
Atributos:	
<i>crearRegistroUsuario</i>	
solicita <i>validarRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i>	
solicita <i>desplegarPantallaCrearRegUsuario</i> a la <i>InterfaceUsuario</i>	
maneja el evento "Registrar"	
solicita <i>crearRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i>	
solicita <i>administrarRegistroUsuario</i>	
maneja el evento "Salir"	
<i>sale</i> del sistema	
solicita <i>obtenerRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i>	
solicita <i>desplegarPantallaObtenerRegUsuario</i> a la <i>InterfaceUsuario</i>	
maneja el evento "Registrar Tarjeta"	
solicita <i>registrarTarjeta</i> al <i>ManejadorRegistroTarjeta</i>	
maneja el evento "Servicios"	
solicita <i>ofrecerServicio</i> al <i>ManejadorServicio</i>	
maneja el evento "Actualizar"	
solicita <i>actualizarRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i>	
maneja el evento "Eliminar"	
solicita <i>eliminarRegistroUsuario</i> a la <i>InterfaceBaseDatosRegistro</i>	

La tarjeta para la clase *PantallaCrearRegUsuario* se muestra en la tabla 8.51 según las responsabilidades identificadas en la tabla 8.14.

Tabla 8.51 Tarjeta para la clase *PantallaCrearRegUsuario* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: <i>PantallaCrearRegUsuario</i>	
Descripción: pantalla de solicitud de registro de usuario (P-3).	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i>	
envía el evento "Registrar" a la <i>InterfaceUsuario</i>	
envía el evento "Salir" a la <i>InterfaceUsuario</i>	

La tarjeta para la clase *PantallaObtenerRegUsuario* se muestra en la tabla 8.52 según las responsabilidades identificadas en la tabla 8.28.

Tabla 8.52 Tarjeta para la clase <i>PantallaObtenerRegUsuario</i> con responsabilidades identificadas a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: PantallaObtenerRegUsuario	
Descripción: pantalla de devolución con información de registro de usuario (P-4).	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega	
envía el evento "Registrar Tarjeta" a la <i>InterfaceUsuario</i>	
envía el evento "Servicios" a la <i>InterfaceUsuario</i>	
envía el evento "Salir" a la <i>InterfaceUsuario</i>	
envía el evento "Actualizar" a la <i>InterfaceUsuario</i>	
envía el evento "Eliminar" a la <i>InterfaceUsuario</i>	

En este momento se agrega la clase *RegistroUsuario* que se encarga de guardar la información de registro de usuario, como se muestra en la tabla 8.53. Observe que hasta el momento no se han asignado responsabilidades a esta clase. Esto no debe preocuparnos aún, ya que por ser clase entidad sus responsabilidades son bastante limitadas y localizadas. En otras palabras, los cambios realizados no afectarán en mayor manera la lógica de la aplicación.

Tabla 8.53 Tarjeta para la clase <i>RegistroUsuario</i> a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: RegistroUsuario	
Descripción: para utilizar el sistema de reservaciones, el usuario debe estar registrado en el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa y oficina, fax, email, login y password.	
Módulo: Registro.Usuario	
Estereotipo: Entidad	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	

TARJETA

En este módulo se describen las clases de registro tarjeta que son *ManejadorRegistroTarjeta*, *PantallaCrearRegTarjeta*, *PantallaObtenerRegTarjeta* y *RegistroTarjeta*. La clase *ManejadorRegistroTarjeta* administra todo lo relacionado con el registro de tarjeta, que incluye recibir y enviar eventos entre *ManejadorRegistroUsuario*, *InterfaceUsuario* e *InterfaceBaseDatosRegistro*, como se muestra en la tabla 8.54. Las responsabilidades se describen a partir de aquellas identificadas en la tabla 8.44.

Tabla 8.54 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades identificadas a partir de los casos de uso: *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario a fin de pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>registrarTarjeta</i>	
solicita <i>desplegarPantallaCrearRegTarjeta</i> a la <i>InterfaceUsuario</i>	
manejar el evento "Registrar"	
solicita <i>crearRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>	
solicita <i>administrarRegistroTarjeta</i>	
manejar el evento "Servicios"	
solicita <i>ofrecerServicio</i> al <i>ManejadorServicio</i>	
manejar el evento "Salir"	
sale del sistema	
solicita <i>obtenerRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>	
solicita <i>desplegarPantallaObtenerRegTarjeta</i> a la <i>InterfaceUsuario</i>	
manejar el evento "Actualizar"	
solicita <i>actualizarRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>	
manejar el evento "Eliminar"	
solicita <i>eliminarRegistroTarjeta</i> a la <i>InterfaceBaseDatosRegistro</i>	

La tarjeta para la clase *PantallaCrearRegTarjeta* se muestra en la tabla 8.55 según las responsabilidades identificadas en la tabla 8.32.

Tabla 8.55 Tarjeta para la clase *PantallaCrearRegTarjeta* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: PantallaCrearRegTarjeta	
Descripción: pantalla de solicitud de registro de tarjeta (P-5).	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega	
envía el evento "Registrar" a la <i>InterfaceUsuario</i>	
envía el evento "Servicios" a la <i>InterfaceUsuario</i>	
envía el evento "Salir" a la <i>InterfaceUsuario</i>	

La tarjeta para la clase *PantallaObtenerRegTarjeta* se muestra en la tabla 8.56 según las responsabilidades identificadas en la tabla 8.45.

Tabla 8.56 Tarjeta para la clase *PantallaObtenerRegTarjeta* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: PantallaObtenerRegTarjeta	
Descripción: pantalla de devolución con información de registro de tarjeta (P-6).	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega	
envía el evento "Servicios" a la <i>InterfaceUsuario</i>	
envía el evento "Actualizar" a la <i>InterfaceUsuario</i>	
envía el evento "Eliminar" a la <i>InterfaceUsuario</i>	
envía el evento "Salir" a la <i>InterfaceUsuario</i>	

En este momento se añade la clase *RegistroTarjeta* que se encarga de guardar la información de registro de tarjeta, como se muestra en la tabla 8.57. Aún no se incluyen responsabilidades por ser una clase entidad.

Tabla 8.57 Tarjeta para la clase *RegistroTarjeta*
Servicios, Registrar Usuario y

Clase: RegistroTarjeta	
Descripción: para hacer un pago con una tarjeta de crédito, se debe tener un registro de tarjeta. El registro contiene información acerca de la tarjeta, incluyendo nombre, número, expedidor y vencimiento. La tarjeta está ligada a un registro de usuario.	
Módulo: Registro.Tarjeta	
Esteriotipo: Entidad	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	

INTERFACE BASE DE DATOS

La tarjeta para la clase *InterfaceBaseDatosRegistro* se muestra en la tabla 8.58 según las responsabilidades identificadas en la tabla 8.46. Esta clase es la encargada de interactuar con el actor *BaseDatosRegistro* para escribir y leer la información allí guardada, tanto de registro de usuario como de registro de tarjeta.

Tabla 8.58 Tarjeta para la clase *InterfaceBaseDatosRegistro*
de los casos de uso *Validar Usuario*, *Crear Registro*

Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Esteriotipo: Interface	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita <i>validarRegistroUsuario</i> a la <i>BaseDatosRegistro</i>	
solicita <i>crearRegistroUsuario</i> a la <i>BaseDatosRegistro</i>	
solicita <i>obtenerRegistroUsuario</i> a la <i>BaseDatosRegistro</i>	
solicita <i>actualizarRegistroUsuario</i> a la <i>BaseDatosRegistro</i>	
solicita <i>eliminarRegistroUsuario</i> a la <i>BaseDatosRegistro</i>	

continúa

Tabla 8.58 Continuación

solicita <i>crearRegistroTarjeta</i> a la <i>BaseDatosRegistro</i>	
solicita <i>obtenerRegistroTarjeta</i> a la <i>BaseDatosRegistro</i>	
solicita <i>actualizarRegistroTarjeta</i> a la <i>BaseDatosRegistro</i>	
solicita <i>eliminarRegistroTarjeta</i> a la <i>BaseDatosRegistro</i>	

SERVICIOS

La tarjeta para la clase *ManejadorServicio* se muestra en la tabla 8.59 según las responsabilidades identificadas en la tabla 8.10. La clase es la encargada de todo lo relacionado con consultas, reservaciones y pagos. Además de esto, es responsable de permitir al usuario acceder la información de registro.

Tabla 8.59 Tarjeta para la clase *ManejadorServicio* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: <i>ManejadorServicio</i>	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicios	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>ofrecerServicio</i>	
solicita <i>desplegarPantallaServicio</i> a la <i>InterfaceUsuario</i>	
maneja el evento "Obtener Registro"	
solicita <i>registrar</i> al <i>ManejadorRegistroUsuario</i>	
maneja el evento "Salir"	
<i>sale</i> del sistema	

La tarjeta para la clase *PantallaServicio* se muestra en la tabla 8.60 según las responsabilidades identificadas en la tabla 8.11. La clase es la encargada de presentar las opciones de servicio del sistema.

Tabla 8.60 Tarjeta para la clase *PantallaServicio* con responsabilidades identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: PantallaServicio	
Descripción: pantalla de servicios (P-2).	
Módulo: Servicios	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i>	
envía el evento "Obtener Registro" a la <i>InterfaceUsuario</i>	
envía el evento "Salir" a la <i>InterfaceUsuario</i>	

8.2.3 Colaboraciones

Es indispensable que los objetos dentro de un programa colaboren entre sí, o de lo contrario el programa constará de múltiples "mini-programas" independientes. Las colaboraciones entre objetos se dan con base en las relaciones entre las responsabilidades de las distintas clases. Por la infinidad de posibles relaciones entre clases, las colaboraciones son la fuente principal de complejidad en el diseño de objetos.

Las colaboraciones representan solicitudes de un objeto *cliente* a un objeto *servidor*. Los objetos pueden desempeñar el papel de clientes o servidores, dependiendo de su actividad en ese momento. Un objeto es un servidor cuando satisface una solicitud hecha por un objeto cliente. Desde el punto de vista del cliente, cada una de sus solicitudes de servicio o colaboración se asocia con una responsabilidad implementada por algún servidor. A su vez, la clase que ofrece el servicio solicita una colaboración con alguna otra clase servidor. Esta cadena de solicitudes y servicios se extiende según sea necesario.

El proceso de identificación de colaboraciones se basa en la funcionalidad de la aplicación y de la arquitectura de clases propuesta, algo que se hizo durante la etapa de análisis. Para identificar colaboraciones se analizan las responsabilidades de cada clase, se decide si cada una de éstas es capaz de satisfacer sus responsabilidades por sí misma o si requiere de otra clase para lograrlo. Se analiza qué tanto sabe cada clase y qué otras clases necesitan su conocimiento o funcionalidad. El patrón de colaboraciones revela el flujo de control e información dentro del sistema. Al analizar los patrones de comunicación entre objetos se reconoce cuando una responsabilidad se ha asignado de forma incorrecta, incluso puede revelar ausencia de responsabilidades. Se analizan las colaboraciones estudiando qué objetos dentro del sistema tienen el conocimiento que se necesita, qué clases colaboran y cuáles no. A través de las colaboraciones se analizan las dependencias entre clases.

- Si una clase no tiene colaboraciones con otras clases, en otras palabras, nadie colabora con ella y ella tampoco colabora, esta clase debe descartarse. Sin embargo, antes de hacerlo es conveniente revisar el diseño completo verificando todas las clases y colaboraciones. Es importante tener en cuenta que las colaboraciones son en una sola dirección, y que cierto tipo de clases son comúnmente clientes o servidores, ubicándose las clases en uno u otro extremo de la colaboración. En la mayor parte de los casos, las clases correspondientes a los estereotipos de *borde* y *entidad* no son clientes de otros objetos en el sistema, sino que existen para ser servidores, normalmente de las clases de *control*.

Durante el proceso de identificación de las colaboraciones se toma la tarjeta de clase, escribiendo en la columna derecha el nombre de la clase servidor, o sea, la clase que colabora para satisfacer la responsabilidad del cliente. Aunque una responsabilidad puede requerir involucrar múltiples colaboraciones, se escribe el nombre de la clase principal como servidor de la colaboración para evitar asociaciones de múltiples clases (ver capítulo 4). Por otro lado, si varias responsabilidades requieren de una misma clase para colaborar, se deben definir varias colaboraciones, una para cada responsabilidad. Debe asegurarse que exista una responsabilidad correspondiente en la clase servidor para cada colaboración que se especifique, incluyendo otras instancias de la misma clase. Se omiten llamadas dentro del mismo objeto.

En esta sección se describen las colaboraciones para el *Sistema de Reservaciones* con base en los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*. Observe que se omiten actores primarios, como *Usuario*, en las colaboraciones, aunque sí se incluyen los actores secundarios, como *BaseDatosRegistro*.

INTERFACEUSUARIO

A partir de la tabla 8.47 se generan las colaboraciones para la clase *InterfaceUsuario*, como se muestra en la tabla 8.61. Note que básicamente se está pasando la clase subrayada en la columna izquierda a la columna derecha. Este procedimiento se mantiene lo más mecánico posible durante esta etapa.

Tabla 8.61 Tarjeta para la clase *InterfaceUsuario* con responsabilidades y colaboraciones identificadas a partir de los casos de uso *ValidarUsuario*, *OfrecerServicios*, *RegistrarUsuario* y *RegistrarTarjeta*.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega	PantallaPrincipal

continúa

Hidden page

Hidden page

Hidden page

Tabla 8.65 Continuación	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega	
envía el evento "Registrar"	InterfaceUsuario
envía el evento "Salir"	InterfaceUsuario

A partir de la tabla 8.52 se generan las colaboraciones para la clase *Pantalla-ObtenerRegUsuario*, como se muestra en la tabla 8.66.

Tabla 8.66 Tarjeta para la clase <i>PantallaObtenerRegUsuario</i> con responsabilidades y colaboraciones identificadas a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: PantallaObtenerRegUsuario	
Descripción: pantalla de devolución con información de registro de usuario (P-4).	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
despliega	
envía el evento "Registrar Tarjeta"	InterfaceUsuario
envía el evento "Servicios"	InterfaceUsuario
envía el evento "Salir"	InterfaceUsuario
envía el evento "Actualizar"	InterfaceUsuario
envía el evento "Eliminar"	InterfaceUsuario

A partir de la tabla 8.53 se generan las colaboraciones para la clase *RegistroUsuario*. Dado que no se han asignado responsabilidades, tampoco habrán colaboraciones, como se muestra en la tabla 8.67.

Tabla 8.67 Tarjeta para la clase *RegistroUsuario* con responsabilidades y colaboraciones identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: RegistroUsuario	
Descripción: para utilizar el sistema de reservaciones, el usuario debe estar registrado en el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa y oficina, fax, email, login y password.	
Módulo: Registro.Usuario	
Estereotipo: Entidad	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	

TARJETA

En este módulo se describen las clases de registro de tarjeta que son *ManejadorRegistroTarjeta*, *PantallaCrearRegTarjeta*, *PantallaObtenerRegTarjeta* y *RegistroTarjeta*. A partir de la tabla 8.54 se generan las colaboraciones para la clase *ManejadorRegistroTarjeta*, como se muestra en la tabla 8.68.

Tabla 8.68 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades y colaboraciones identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de todo lo relacionado con el registro de la tarjeta del usuario para pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>registrarTarjeta</i>	
solicita <i>desplegarPantallaCrearRegTarjeta</i>	InterfaceUsuario
manejar el evento "Registrar"	
solicita <i>crearRegistroTarjeta</i>	InterfaceBaseDatosRegistro
solicita <i>administrarRegistroTarjeta</i>	

continúa

Tabla 8.68 Continuación	
manejar el evento "Servicios"	
solicita <i>ofrecerServicio</i>	ManejadorServicio
manejar el evento "Salir"	
sale del sistema	
solicita <i>obtenerRegistroTarjeta</i>	InterfaceBaseDatosRegistro
solicita <i>desplegarPantallaObtenerReg Tarjeta</i>	InterfaceUsuario
manejar el evento "Actualizar"	
solicita <i>actualizarRegistroTarjeta</i>	InterfaceBaseDatosRegistro
manejar el evento "Eliminar"	
solicita <i>eliminarRegistroTarjeta</i>	InterfaceBaseDatosRegistro

A partir de la tabla 8.55 se generan las colaboraciones para la clase *Pantalla-CrearRegTarjeta*, como se muestra en la tabla 8.69.

Tabla 8.69 Tarjeta para la clase <i>PantallaCrearRegTarjeta</i> con responsabilidades y colaboraciones identificadas a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: PantallaCrearRegTarjeta	
Descripción: pantalla de solicitud de registro de tarjeta (P-5).	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i>	
envía el evento "Registrar"	InterfaceUsuario
envía el evento "Servicios"	InterfaceUsuario
envía el evento "Salir"	InterfaceUsuario

A partir de la tabla 8.56 se generan las colaboraciones para la clase *Pantalla-ObtenerRegTarjeta*, como se muestra en la tabla 8.70.

Tabla 8.70 Tarjeta para la clase <i>PantallaObtenerRegTarjeta</i> con responsabilidades y colaboraciones identificadas a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: PantallaObtenerRegTarjeta	
Descripción: pantalla de devolución con información de registro de tarjeta (P-6).	

continúa

Hidden page

Tabla 8.72 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades y colaboraciones identificadas a partir de los casos de uso Validar Usuario, Ofrecer Servicios, Registrar Usuario y Registrar Tarjeta.

Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
solicita validarRegistroUsuario	BaseDatosRegistro
solicita crearRegistroUsuario	BaseDatosRegistro
solicita obtenerRegistroUsuario	BaseDatosRegistro
solicita actualizar RegistroUsuario	BaseDatosRegistro
solicita eliminarRegistroUsuario	BaseDatosRegistro
solicita crearRegistroTarjeta	BaseDatosRegistro
solicita obtenerRegistroTarjeta	BaseDatosRegistro
solicita actualizarRegistroTarjeta	BaseDatosRegistro
solicita eliminarRegistroTarjeta	BaseDatosRegistro

SERVICIOS

A partir de la tabla 8.59 se generan las colaboraciones para la clase *Manejador-Servicio*, como se muestra en la tabla 8.73.

Tabla 8.73 Tarjeta para la clase *ManejadorServicio* con responsabilidades y colaboraciones identificadas a partir de los casos de uso Validar Usuario, Ofrecer Servicios, Registrar Usuario y Registrar Tarjeta.

Clase: ManejadorServicio	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicio	
Estereotipo: Control	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	

continúa

Tabla 8.73 Continuación

<i>ofrecerServicio</i>	
solicita desplegarPantallaServicio	InterfaceUsuario
maneja el evento "Obtener Registro"	
solicita registrar	ManejadorRegistroUsuario
maneja el evento "Salir"	
sale del sistema	

A partir de la tabla 8.60 se generan las colaboraciones para la clase *Pantalla-Servicio*, como se muestra en la tabla 8.74.

Tabla 8.74 Tarjeta para la clase *PantallaServicio* con responsabilidades y colaboraciones identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: PantallaServicio	
Descripción: pantalla de servicios (P-2).	
Módulo: Servicio	
Estereotipo: Borde	
Propiedades:	
Superclases:	
Subclases:	
Atributos:	
<i>despliega</i>	
envía el evento "Obtener Registro"	InterfaceUsuario
envía el evento "Salir"	InterfaceUsuario

8.2.4 Jerarquías

El diseño de las jerarquías de herencia es uno de los aspectos de programación más importantes de la orientación a objetos. Mediante la herencia se logra una buena reutilización del código del sistema, obteniendo arquitecturas de clases más compactas, lo que reduce radicalmente el tamaño del sistema final.

La herencia se identifica a partir de las responsabilidades y colaboraciones obtenidas antes. De manera general, existen dos formas para aprovechar la herencia. La forma más común es la creación de superclases que guarden responsabilidades comunes a múltiples clases. La forma adicional y más avanzada está relacionada con el *polimorfismo* y busca aprovechar no sólo responsabilidades comunes sino también colaboraciones comunes entre clases. Estos dos enfoques también sirven de base para la extensibilidad de la arquitectura de clases. Por ejemplo, si varias clases definen una responsabilidad similar, se introduce una

superclase de la que estas clases hereden la responsabilidad común. Las nuevas superclases son clases abstractas. Cuanto mayor sea el número de clases concretas que se extienden a partir de una funcionalidad abstracta, mayor será la probabilidad de que la abstracción sobreviva las pruebas del tiempo y mejoras del software. Se necesita sólo una responsabilidad para definir una superclase abstracta, pero por lo menos se necesita dos subclases antes de diseñar una abstracción general útil. Si no se tiene o no se prevé, por lo menos dos casos, no se debe perder tiempo en construir la abstracción. Probablemente no se diseñe una funcionalidad apropiada general sin tener varios ejemplos concretos que sirvan de guía.

En esta etapa se debe revisar las tarjetas de clases y clasificar cada clase según sea concreta o abstracta. Además, se deben agregar tarjetas para las superclases (abstractas o concretas) según sea necesario, y reasignar responsabilidades para que correspondan al nuevo diseño. Mediante la reasignación de responsabilidades se busca producir jerarquías de clases que reutilicen y se extiendan más fácilmente. Aunque se llevará a cabo una sola etapa de diseño de jerarquías, la herencia debe aplicarse continuamente durante el diseño de objetos. En la tarjeta de clases se listan las responsabilidades propias no heredadas y aquellas responsabilidades que se sobrescriben. No es necesario volver a escribir las responsabilidades que se heredan de las superclases. Observe en las siguientes secciones que se agregarán nuevas superclases, para lo que se añadirán las correspondientes tarjetas de clases, así como información en las secciones de subclases y superclases, además la especificación respecto a si la clase es concreta o abstracta.

A continuación se describen las jerarquías de herencia para el *Sistema de Reservas* con base en los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

INTERFACEUSUARIO

Para comenzar, se analizan las diversas responsabilidades y colaboraciones asignadas a la clase *InterfaceUsuario* en la tabla 8.61 que se muestran nuevamente en la tabla 8.75.

Tabla 8.75 Responsabilidades y colaboraciones para la clase *InterfaceUsuario*.

despliega	PantallaPrincipal
envía el evento "Registrarse por Primera Vez"	ManejadorPrincipal
envía el evento "OK"	ManejadorPrincipal
envía el evento "Salir"	ManejadorPrincipal
despliega	PantallaServicio
envía el evento "Obtener Registro"	ManejadorServicio
envía el evento "Salir"	ManejadorServicio
despliega	PantallaCrearRegUsuario
envía el evento "Registrar"	ManejadorRegistroUsuario

continúa

Tabla 8.75 Continuación

envía el evento "Salir"	ManejadorRegistroUsuario
despliega	PantallaObtenerRegUsuario
envía el evento "Registrar Tarjeta"	ManejadorRegistroUsuario
envía el evento "Servicios"	ManejadorRegistroUsuario
envía el evento "Actualizar"	ManejadorRegistroUsuario
envía el evento "Eliminar"	ManejadorRegistroUsuario
despliega	PantallaCrearRegTarjeta
envía el evento "Registrar"	ManejadorRegistroTarjeta
envía el evento "Servicios"	ManejadorRegistroTarjeta
envía el evento "Salir"	ManejadorRegistroTarjeta
despliega	PantallaObtenerRegTarjeta
envía el evento "Actualizar"	ManejadorRegistroTarjeta
envía el evento "Eliminar"	ManejadorRegistroTarjeta

Al analizar estas responsabilidades y colaboraciones con más detalle, se aprecia que hay dos grupos de responsabilidades, aquellos correspondientes a "despliega" y los correspondientes a "envía el evento...", como se presenta de manera condensada en la tabla 8.76. Es importante apreciar que se *generalizan* las diversas responsabilidades "envía el evento ..." en una común, en donde el evento particular "OK", "Registrar", etc., se abstraen de la responsabilidad.

Tabla 8.76 Grupos de responsabilidades y colaboraciones para la clase *InterfaceUsuario*.

despliega	PantallaPrincipal, PantallaServicio, PantallaCrearRegUsuario, PantallaObtenerRegUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta
envía el evento ...	ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta

En el caso de la responsabilidad "despliega" ésta también se llama "despliega" para todas las clases colaboradoras *PantallaPrincipal*, *PantallaServicio*, *PantallaCrearRegUsuario*, *PantallaObtenerRegUsuario*, *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta*, como se muestra en la tabla 8.77. Esto es sumamente importante si se desea aprovechar el polimorfismo.

Tabla 8.77 Grupos de responsabilidades "despliega" para las distintas pantallas, correspondientes a las clases *PantallaPrincipal*, *PantallaServicio*, *PantallaCrearRegUsuario*, *PantallaObtenerRegUsuario*, *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta*.

despliega	
-----------	--

Hidden page

diferentes manejadores. Ésta es una situación característica de polimorfismo, donde una misma responsabilidad la aprovechan diversas clases colaboradoras que funcionalmente son similares. La solución es crear una nueva superclase *Pantalla* y otra *Manejador* de manera que las relaciones antes descritas en la tabla 8.76 se conviertan según se describen en la tabla 8.79. Note que aunque la relación de colaboración se simplifica, aún se incluye la lista de clases colaboradoras para no perder esta información a nivel descriptivo. Esto se denota utilizando la notación de “superclase” seguida por “:” y finalmente por la lista de “clases colaboradoras”. Sin embargo, el objetivo de diseño es que la *Interface-Usuario* deje de conocer explícitamente a todas las clases colaboradoras y que únicamente conozca a la clase general que luego será sobrecargada.

Tabla 8.79 Grupos de responsabilidades y colaboraciones para la clase *InterfaceUsuario* revisados según la creación de dos nuevas superclases: *Pantalla* y *Manejador*.

despliega	Pantalla: PantallaPrincipal, PantallaServicio, PantallaCrearRegUsuario, PantallaObtenerRegUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta
envía el evento ...	Manejador: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta

En la figura 8.4 se muestra la nueva jerarquía de herencia.

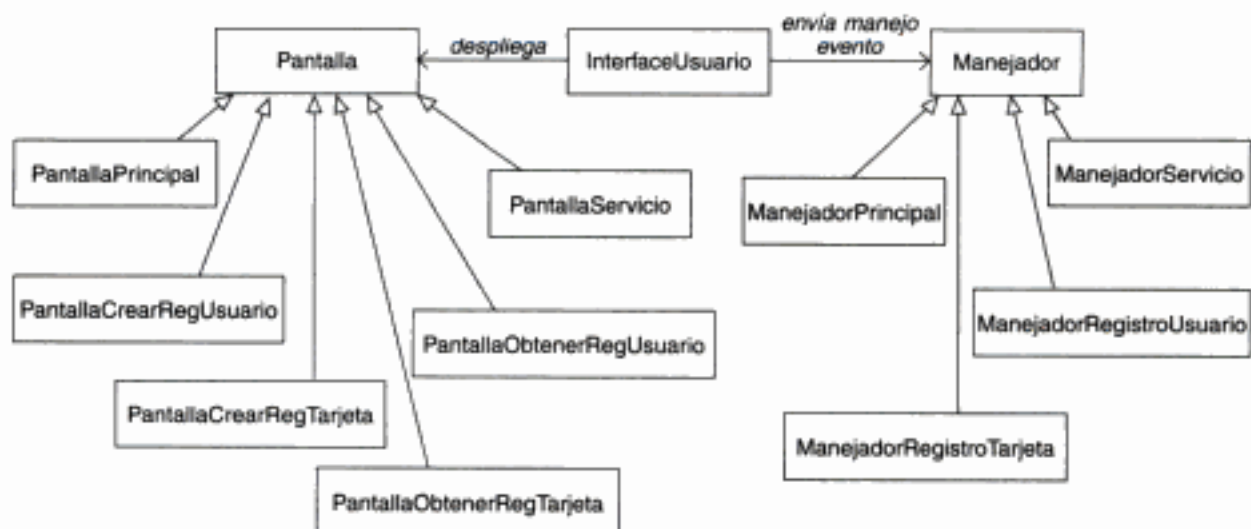


Figura 8.4 El diagrama muestra las colaboraciones descritas hasta el momento para la clase *InterfaceUsuario*, luego de la introducción de las superclases *Pantalla* y *Manejador*.

Es importante resaltar que se tendrá que agregar dos nuevas tarjetas de clases correspondientes a las clases *Pantalla* y *Manejador* recién introducidas. Dichas tarjetas deberán incluir responsabilidades correspondientes a “despliega” y “ma-

nejarEvento" que sobreescribirán las diversas pantallas y manejadores en la jerarquía de herencia.

La tarjeta de clase modificada para la clase *InterfaceUsuario* se muestra en la tabla 8.80. Como parte del proceso de afinación de nombres se cambia "despliega" por "desplegarPantalla", que es más descriptivo, y "envía el evento ..." por "enviarEvento ..." que es más compacto, así como "manejarEvento ..." en lugar de "maneja el evento ...". De esta manera, se reducen de forma radical el número de responsabilidades y colaboraciones de la clase *InterfaceUsuario*. Observe cómo los diversos "envía el evento ..." son abstraídos o generalizados por una sola responsabilidad más genérica llamada *enviarEvento*, al igual que "maneja el evento ..." que es abstraído mediante *manejarEvento*. Vale la pena resaltar la gran reducción en el número de responsabilidades y colaboraciones definidas para la clase *InterfaceUsuario*.

Tabla 8.80 Tarjeta para la clase *InterfaceUsuario* con responsabilidades, colaboraciones y jerarquías identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
desplegarPantalla	Pantalla: PantallaPrincipal, PantallaServicio, PantallaCrearRegUsuario, PantallaObtenerRegUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta
enviarEvento	Manejador: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta

Antes de especificar la tarjeta de clase para la nueva superclase *Pantalla* se verá otra fuente de generalización a partir de las diversas pantallas. En la tabla 8.81 se muestra de manera compacta las responsabilidades y colaboraciones para las diversas pantallas descritas en la sección de colaboraciones. Estas pantallas contienen una responsabilidad "despliega" y otra "envía el evento ...", que colabora con *InterfaceUsuario*. Nuevamente se abstraen las diversas responsabilidades "enviar el evento ..." en una sola.

Tabla 8.81 Grupos de responsabilidades y colaboraciones para las diversas pantallas.

despliega	
envía el evento ...	InterfaceUsuario

Estas responsabilidades con sus colaboraciones se muestran en la figura 8.5. Note que la dirección de la flecha va ahora en dirección de la *InterfazUsuario*. Para el nombre de la asociación se utiliza la responsabilidad reescrita "enviarEvento" definida en la clase *InterfazUsuario*, de la clase colaboradora.

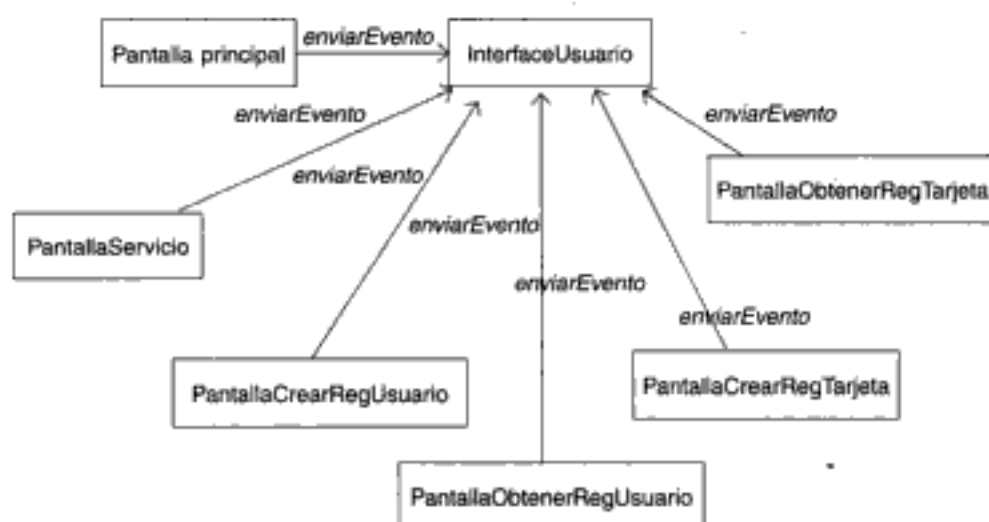


Figura 8.5 El diagrama muestra las colaboraciones descritas a partir de las diversas pantallas y en dirección a la *InterfazUsuario*.

Ésta es nuevamente una fuente de polimorfismo donde se aprovecha la clase *Pantalla* antes agregada. En la figura 8.6 se muestra el diagrama de herencia correspondiente a la figura 8.5 incluyendo la nueva superclase *Pantalla* en base al polimorfismo "enviarEvento" a partir de las diversas pantallas.

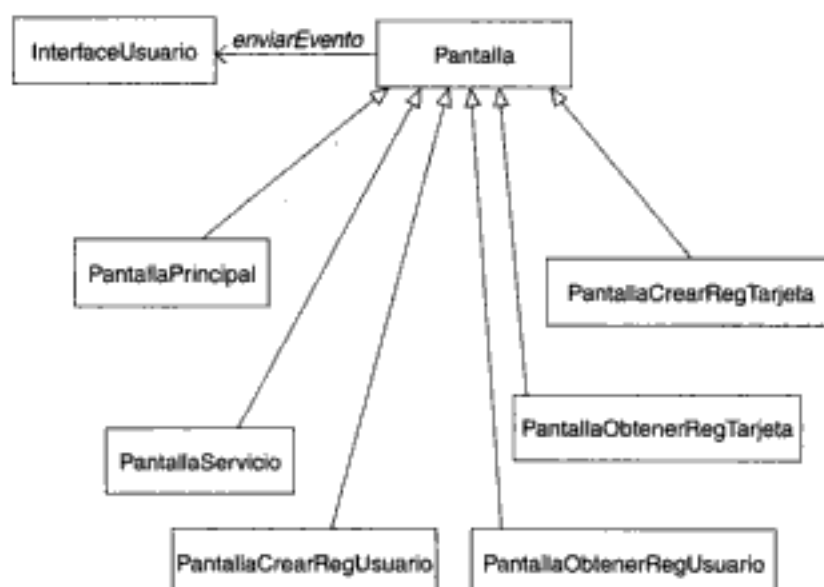


Figura 8.6 El diagrama muestra las colaboraciones descritas a partir de las diversas pantallas conteniendo la superclase *Pantalla* y en dirección a la *InterfaceUsuario*.

La responsabilidad “envía el evento ...” se reescribe como “enviarEvento” y junto con “desplegarPantalla” definen las responsabilidades para la clase *Pantalla*, como se muestra en la tabla 8.82. La clase *Pantalla* se define como *Abstracta*, ya que es una *superclase*. Además, en la sección de subclases se describen las diversas clases heredadas de ella.

Tabla 8.82 Tarjeta para la superclase clase *Pantalla* con responsabilidades, colaboraciones y jerarquías identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Pantalla	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases:	
Subclases: PantallaPrincipal, PantallaServicio, PantallaCrearRegUsuario, PantallaObtenerRegUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta	
Atributos:	
desplegarPantalla	
enviarEvento	InterfaceUsuario

Antes de continuar con la superclase *Manejador* se aprovecha para agregar dos nuevas superclases de tipo pantalla añadidas exclusivamente por razones de herencia y no polimorfismo. Estas clases son *PantallaRegUsuario* que define elementos comunes a las pantallas *PantallaCrearRegUsuario* y *PantallaObtenerRegUsuario*, y *PantallaRegTarjeta* que define elementos comunes a las pantallas *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta*. Los elementos comunes para estas pantallas son básicamente todos los campos de textos que se repiten entre ellas, difiriendo únicamente en los botones.

En la figura 8.7 se muestra la jerarquía de herencia a partir de la clase *Pantalla*, conteniendo las clases *PantallaRegUsuario* y *PantallaRegTarjeta*.

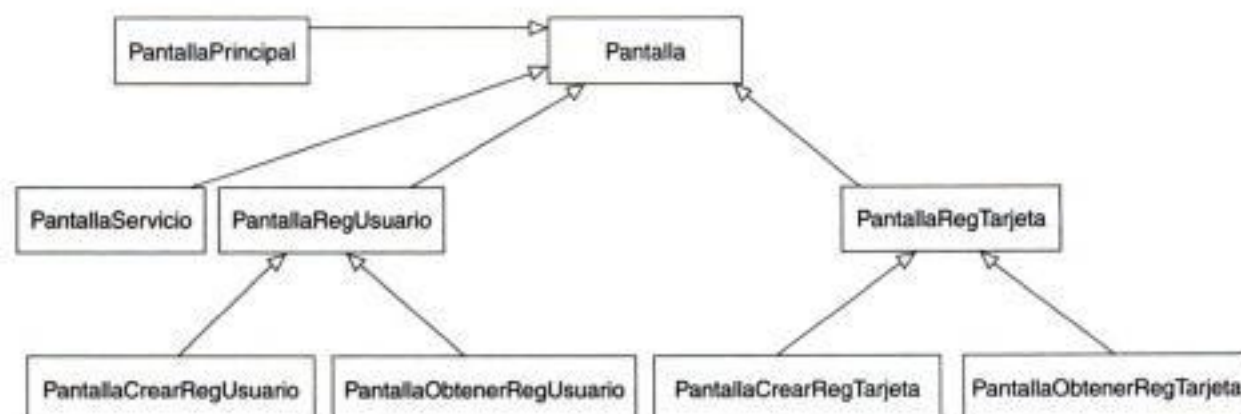


Figura 8.7 El diagrama muestra la jerarquía de clases para el módulo de registro a partir de la clase *Pantalla*, incluyendo las clases *PantallaRegUsuario* y *PantallaRegTarjeta*.

En la tabla 8.83 se describe la clase *Pantalla* redefinida según las modificaciones con las clases *PantallaRegUsuario* y *PantallaRegTarjeta* correspondiente a la figura 8.7. El cambio se da únicamente en la sección de subclases.

Tabla 8.83 Tarjeta para la superclase clase *Pantalla* con responsabilidades, colaboraciones y jerarquías revisadas.

Clase: Pantalla	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases:	
Subclases: PantallaPrincipal, PantallaServicio, PantallaRegUsuario, PantallaRegTarjeta	
Atributos:	
desplegarPantalla	
enviarEvento	InterfaceUsuario

PRINCIPAL

A continuación se llevará a cabo un proceso de generalización para la clase *Manejador* similar al proceso llevado a cabo para la clase *Pantalla*. En la tabla 8.84 se muestra de manera compacta las responsabilidades y colaboraciones comunes para los diversos manejadores descritos en la sección de colaboraciones. Los diferentes manejadores contienen varias responsabilidades de tipo "solicita desplegarPantalla..." en colaboración con la *InterfaceUsuario* y que se generalizan de manera similar mediante *desplegarPantalla*, otra responsabilidad "solicita ofrecerServicio" en colaboración con *ManejadorServicio* que es común a los diversos manejadores y que se reescribe como *ofrecerServicios*, y otra responsabilidad común "sale del sistema" que puede reescribirse mediante "salir". La responsabilidad de tipo "maneja el evento ..." que la sobreescriben cada uno de los manejadores también se generalizan mediante *manejarEvento*; sin embargo, debe existir un manejo particular según el evento que se haya recibido, por lo que además de la generalización agregaremos manejos particulares según los eventos. Además de los eventos anteriores se tienen otras responsabilidades como *crearRegistroUsuario* y *validarRegistroUsuario*, pero éstas ya no son comunes entre manejadores y no es posible generalizarlas.

Tabla 8.84 Grupos de responsabilidades y colaboraciones para el *ManejadorPrincipal*.

solicita desplegarPantalla ...	InterfaceUsuario
manejar el evento ...	
solicita crearRegistroUsuario	ManejadorRegistroUsuario
solicita validarRegistroUsuario	ManejadorRegistroUsuario
solicita ofrecerServicio	ManejadorServicio
sale del sistema	

Las responsabilidades y colaboraciones comunes entre manejadores, “desplegarPantalla” y “ofrecerServicios” (omitimos “manejarEvento” que ya se describió en la figura 8.4) se presentan en la figura 8.8.

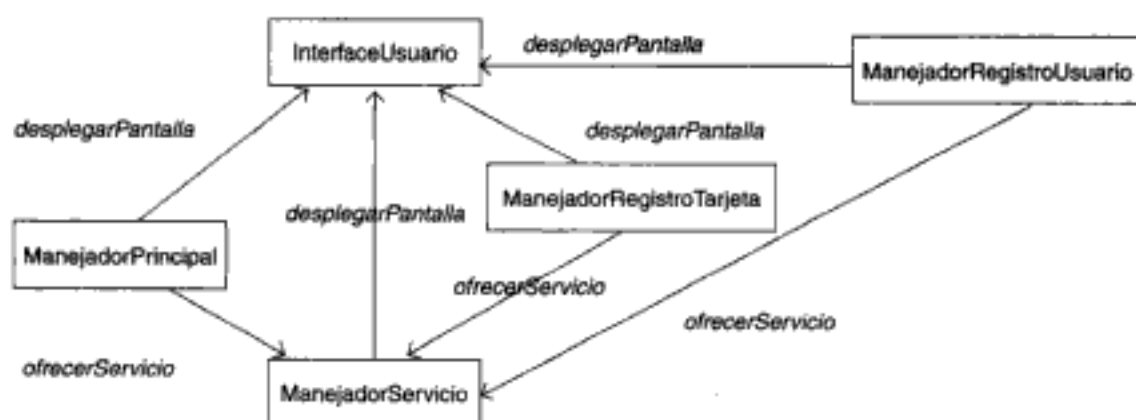


Figura 8.8 El diagrama muestra las colaboraciones descritas a partir de los diversos manejadores y en dirección a la *InterfaceUsuario* y *ManejadorServicio*.

Con la introducción de la clase *Manejador* se generalizan estas relaciones y aprovecha el polimorfismo correspondiente. En la figura 8.9 se muestra el diagrama de herencia correspondiente a la figura 8.8 con la inclusión de la nueva superclase *Manejador*.

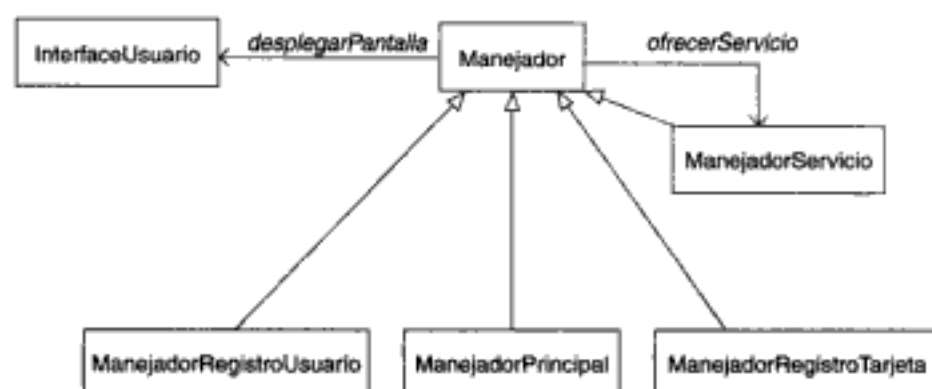


Figura 8.9 El diagrama presenta las colaboraciones descritas a partir de los diversos manejadores conteniendo la superclase *Manejador* y en dirección a la *InterfaceUsuario* y *ManejadorServicio*.

Las responsabilidades anteriores para los diversos manejadores se describen en la superclase *Manejador*, como se aprecia en la tabla 8.85. Observe que se modifican los nombres de las responsabilidades manejar el evento “Ofrecer Servicios” y manejar el evento “Salir” a “manejarEventoOfrecerServicios” y “manejarEventoSalir”, respectivamente. La clase *Manejador* se define como *Abstracta* ya que es una *superclase*. En la sección de subclases se agregan las diversas clases heredadas de ella.

Tabla 8.85 Tarjeta para la clase *Manejador* con responsabilidades, colaboraciones y jerarquías identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Manejador	
Descripción: superclase heredada por todos los manejadores del sistema.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Abstracta	
Superclases:	
Subclases: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta	
Atributos:	
manejarEvento	
desplegarPantalla	InterfaceUsuario
manejarEventoOfrecerServicio	
ofrecerServicio	ManejadorServicio
manejarEventoSalir	
salir	

En la figura 8.10 se muestra la jerarquía de herencia a partir de la superclase *Manejador*.



Figura 8.10 El diagrama presenta la jerarquía de clases a partir de la clase *Manejador*.

Ahora veamos cómo se describen las demás clases según estas modificaciones de herencia. A partir de la tabla 8.62 se generan las jerarquías para la clase *ManejadorPrincipal*, como se muestra en la tabla 8.86. La clase se define como concreta y se especifica su superclase. La única responsabilidad sobreescrita de las definidas en la superclase *Manejador* es "manejarEvento", ya que el polimorfismo va de la clase *InterfaceUsuario* a los diversos manejadores. Adicionalmente, y como se mencionó antes, se llamará a las responsabilidades particulares para el manejo de eventos, como, *manejarEventoRegistrar* y *manejarEvento-*

Validar, correspondiente a los eventos generados a partir de los botones, "Registrarse por Primera Vez" y "OK", respectivamente. Observe que los otros eventos generados por los botones "Ofrecer Servicios" y "Salir" se heredan a partir de las responsabilidades *manejarEventoOfrecerServicios* y *manejarEvento-Salir* definidos en la superclase *Manejador*. Adicionalmente, la clase *Manejador-Principal* define las responsabilidades "solicita *crearRegistroUsuario*" y "solicita *validarRegistroUsuario*" que se reescriben como *crearRegistroUsuario* y *validar-RegistroUsuario*, respectivamente.

Tabla 8.86 Tarjeta para la clase *ManejadorPrincipal* con responsabilidades, colaboraciones y jerarquías identificadas de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorPrincipal	
Descripción: el manejador principal es el encargado de desplegar la pantalla principal de interacción con el usuario, y luego delegar las diferentes funciones a los manejadores especializados apropiados.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
manejarEvento	
manejarEventoRegistrar	
crearRegistroUsuario	ManejadorRegistroUsuario
manejarEventoValidar	
validarRegistroUsuario	ManejadorRegistroUsuario

En el caso de las pantallas, todas las responsabilidades se especifican en la superclase *Pantalla*, por lo que no hay necesidad de incluir las responsabilidades descritas antes. Por tanto, la clase *PantallaPrincipal* ya descrita en la tabla 8.63 se detallará según las modificaciones en las jerarquías de herencia, como se muestra en la tabla 8.87. Se eliminan las responsabilidades y se especifica la clase como concreta y se agrega su superclase.

Tabla 8.87 Tarjeta para la clase *PantallaPrincipal* con responsabilidades, colaboraciones y jerarquías identificadas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: PantallaPrincipal	
Descripción: pantalla principal (P-1).	
Módulo: Principal	

continúa

Hidden page

Tabla 8.88 Continuación	
Propiedades: Abstracta	
Superclases:	
Subclases: RegistroUsuario, RegistroTarjeta	
Atributos:	

REGISTRO

Este módulo se compone de los módulos de *Usuario*, *Tarjeta* e *InterfaceBD*.

USUARIO

Esta sección abarca las clases de registro de usuario *ManejadorRegistroUsuario*, *PantallaCrearRegUsuario*, *PantallaObtenerRegUsuario* y *RegistroUsuario*.

En la tabla 8.64 se describe la clase *ManejadorRegistroUsuario*, como se muestra en la tabla 8.89. Con excepción de "manejarEvento" sobreescrita por todos los manejadores, las responsabilidades "desplegarPantalla", "ofrecerServicio" y "salir" son herencia de la superclase *Manejador*. Las demás responsabilidades se reescriben sin la palabra "solicita" para hacer más compactas las responsabilidades, y se reescriben las responsabilidades de tipo "maneja el evento "Actualizar"" como "manejarEventoActualizar".

Tabla 8.89 Tarjeta para la clase <i>ManejadorRegistroUsuario</i> con responsabilidades, colaboraciones y jerarquías identificadas a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de lo relacionado con registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
manejarEvento	
crearRegistroUsuario	
validarRegistroUsuario	InterfaceBaseDatosRegistro
manejarEventoRegistrar	
crearRegistroUsuario	InterfaceBaseDatosRegistro

continúa

Tabla 8.89 Continúa	
obtenerRegistroUsuario	InterfaceBaseDatosRegistro
administrarRegistroUsuario	
manejarEventoActualizar	
actualizarRegistroUsuario	InterfaceBaseDatosRegistro
manejarEventoEliminar	
eliminarRegistroUsuario	InterfaceBaseDatosRegistro
manejarEventoRegistrarTarjeta	
registrarTarjeta	ManejadorRegistroTarjeta

Se agrega la superclase *PantallaRegUsuario* antes mencionada, que se muestra en la tabla 8.90.

Tabla 8.90 Tarjeta para la clase <i>PantallaRegUsuario</i> con la jerarquía identificadas de las pantallas de registro de usuario a partir del caso de uso <i>Registrar Usuario</i> .	
Clase: PantallaRegUsuario	
Descripción: superclase con diseño gráfico común para las pantallas de registro de usuario.	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases: Pantalla	
Subclases: PantallaCrearRegUsuario, PantallaObtenerRegUsuario	
Atributos:	

A partir de la tabla 8.65 se generan las jerarquías para la clase *PantallaCrearRegUsuario*, como se muestra en la tabla 8.91. De manera similar a la clase *PantallaPrincipal*, se eliminaron las responsabilidades en esta clase al describirse en la superclase *Pantalla*.

Tabla 8.91 Tarjeta para la clase <i>PantallaCrearRegUsuario</i> con responsabilidades, colaboraciones y jerarquías identificadas a partir del caso de uso <i>Registrar Usuario</i> .	
Clase: PantallaCrearRegUsuario	
Descripción: pantalla de solicitud de registro de usuario (P-3).	
Módulo: Registro.Usuario	
Estereotipo: Borde	

continúa

Tabla 8.91 Continuación

Propiedades: Concreta	
Superclases: PantallaRegUsuario	
Subclases:	
Atributos:	

A partir de la tabla 8.66 se generan las jerarquías para la clase *PantallaObtenerRegUsuario*, como se muestra en la tabla 8.92.

Tabla 8.92 Tarjeta para la clase *PantallaObtenerRegUsuario* con responsabilidades, colaboraciones y jerarquías identificadas a partir del caso de uso *Registrar Usuario*.

Clase: PantallaObtenerRegUsuario	
Descripción: pantalla de devolución con información de registro de usuario (P-4).	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases: PantallaRegUsuario	
Subclases:	
Atributos:	

A partir de la tabla 8.67 se generan las jerarquías para la clase *RegistroUsuario*, como se muestra en la tabla 8.93. Observe que *RegistroUsuario* es una subclase de la recién introducida clase *Datos*.

Tabla 8.93 Tarjeta para la clase *RegistroUsuario* con responsabilidades, colaboraciones y jerarquías a partir de los casos de uso *Validar Usuario* y *Registrar Usuario*.

Clase: RegistroUsuario	
Descripción: para utilizar el sistema de reservaciones, el usuario debe estar registrado en el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa y oficina, fax, email, login y password.	
Módulo: Registro.Usuario	

continúa

Tabla 8.93 Continuación

Estereotipo: Entidad	
Propiedades: Concreta	
Superclases: Datos	
Subclases:	
Atributos:	

TARJETA

Esta sección abarca las clases de registro de tarjeta que son *ManejadoRegistroTarjeta*, *PantallaCrearRegTarjeta*, *PantallaObtenerRegTarjeta* y *RegistroTarjeta*. De manera similar a los manejadores anteriores, se describe a partir de la tabla 8.68 la clase *ManejadoRegistroTarjeta*, como se muestra en la tabla 8.94. Se sobreescribe la responsabilidad "manejarEvento" y se eliminan de esta tarjeta las responsabilidades descritas en la superclase *Pantalla*, o sea, "desplegarPantalla", "ofrecerServicio" y "Salir". Se mantienen las demás responsabilidades reescribiéndolas sin la palabra "solicita" y reescribiendo las responsabilidades de tipo "maneja el evento ...".

Tabla 8.94 Tarjeta para la clase *ManejadoRegistroTarjeta* con responsabilidades, colaboraciones y jerarquías identificadas del caso de uso *RegistrarTarjeta*.

Clase: <i>ManejadorRegistroTarjeta</i>	
Descripción: el manejador de registro de tarjeta se encarga de lo relacionado con el registro de la tarjeta del usuario para pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: <i>Manejador</i>	
Subclases:	
Atributos:	
manejarEvento	
registrarTarjeta	
manejarEventoRegistrar	
crearRegistroTarjeta	InterfaceBaseDatosRegistro
obtenerRegistroTarjeta	InterfaceBaseDatosRegistro
administrarRegistroTarjeta	
manejarEventoActualizar	
actualizarRegistroTarjeta	InterfaceBaseDatosRegistro
manejarEventoEliminar	
eliminarRegistroTarjeta	InterfaceBaseDatosRegistro

Hidden page

Hidden page

Tabla 8.99 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades, colaboraciones y jerarquías de escribir y leer información de registro de usuario y registro de tarjeta a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
validarRegistroUsuario	BaseDatosRegistro
crearRegistroUsuario	BaseDatosRegistro
obtenerRegistroUsuario	BaseDatosRegistro
actualizarRegistroUsuario	BaseDatosRegistro
eliminarRegistroUsuario	BaseDatosRegistro
crearRegistroTarjeta	BaseDatosRegistro
obtenerRegistroTarjeta	BaseDatosRegistro
actualizarRegistroTarjeta	BaseDatosRegistro
eliminarRegistroTarjeta	BaseDatosRegistro

SERVICIOS

A partir de la tabla 8.73 se generan las jerarquías para la clase *ManejadorServicio*, como se muestra en la tabla 8.100. Los cambios son similares a los demás manejadores.

Tabla 8.100 Tarjeta para la clase *ManejadorServicio* con responsabilidades, colaboraciones y jerarquías a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorServicio	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicios	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
manejarEvento	
ofrecerServicio	
manejarEventoRegistrar	
registrar	ManejadorRegistroUsuario

Hidden page

Hidden page

tar contratos heredados, e indicar la clase de la cual se heredan, aunque no hay necesidad de repetir los detalles del contrato. Para cada contrato se listan las responsabilidades de la clase en la que se basan, asignando cada responsabilidad al contrato correspondiente.

Si la responsabilidad requiere colaborar con una clase que define varios contratos, se debe indicar el número del contrato correspondiente, entre paréntesis, en la columna de la colaboración, para así simplificar el seguimiento de qué responsabilidad se relaciona con qué contrato.

En esta sección se describen los contratos para el *Sistema de Reservaciones*, con base en los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

INTERFACEUSUARIO

Considere las dos responsabilidades "desplegarPantalla" y "enviarEvento" asignadas a la clase *InterfaceUsuario* en la sección anterior de jerarquías las que se presentan en la tabla 8.102.

Tabla 8.102 Responsabilidades asignadas a la clase <i>InterfaceUsuario</i> a partir de la etapa de jerarquías.	
desplegarPantalla	Pantalla: PantallaPrincipal, PantallaServicio, PantallaCrearRegUsuario, PantallaObtenerRegUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta
enviarEvento	Manejador: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta

Si se investiga con mayor detalle estas responsabilidades se ve que "desplegarPantalla" la llaman los diversos manejadores, en tanto que "enviarEvento" la llaman las diversas pantallas, algo que se muestra en la figura 8.12.

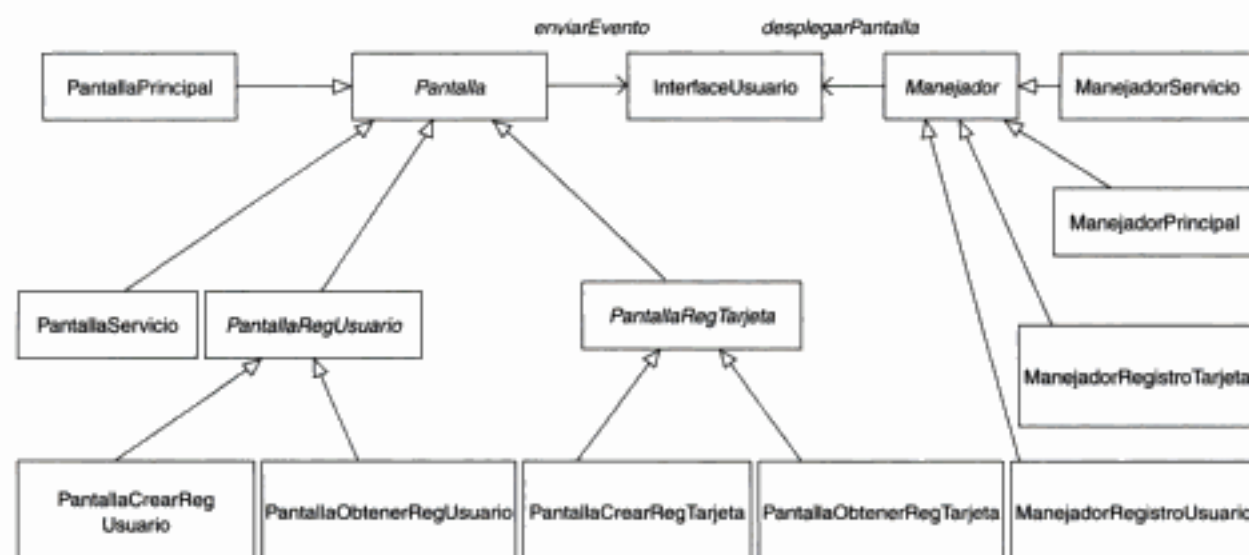


Figura 8.12 El diagrama muestra las diversas clases de manejadores y pantallas solicitando servicios de la clase *InterfaceUsuario* a través de "desplegarPantalla" y "enviarEvento", respectivamente.

Hidden page

Hidden page

Tabla 8.105 Tarjeta para la superclase clase *Pantalla* con responsabilidades, colaboraciones, jerarquías y contratos identificados a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Pantalla	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases:	
Subclases: PantallaPrincipal, PantallaServicio, PantallaRegUsuario, PantallaRegTarjeta	
Atributos:	
Contratos	
1. Desplegar Pantalla	
desplegarPantalla	
Responsabilidades Privadas	
enviarEvento	InterfaceUsuario (2)

Evento" a pesar de que ésta es privada. Además, se agregó un "2" entre paréntesis a la derecha de la clase colaboradora *InterfaceUsuario*. Este "2" corresponde al contrato número "2" definido en la clase *InterfaceUsuario*, en otras palabras el contrato "Enviar Evento". Agregando este número se tiene información adicional sobre la colaboración entre clases pero a nivel de contratos.

PRINCIPAL

De manera similar a *Pantalla* es posible identificar cuatro grupos de responsabilidades lógicamente separadas para los diversos manejadores, "manejarEvento", "desplegarPantalla", "manejarEventoOfrecerServicio", "ofrecerServicio", "manejarEventoSalir" y "salir", todos definidos en la superclase *Manejador*, como se muestra en la tabla 8.106, a partir de las responsabilidades definidas para la clase *Manejador* en la tabla 8.85.

Tabla 8.106 Responsabilidades para la superclase clase *Manejador* definidas en la tabla 8.85 en la sección de jerarquías.

manejarEvento	
desplegarPantalla	InterfaceUsuario
manejarEventoOfrecerServicio	
ofrecerServicio	ManejadorServicio
manejarEventoSalir	
salir	

Si se analizan estas responsabilidades, se observa que "manejarEvento" tiene como cliente la *InterfaceUsuario*. Sin embargo, las otras responsabilidades, "desplegarPantalla", "manejarEventoOfrecerServicio", "ofrecerServicio", "manejarEventoSalir" y "salir", no tiene ningún cliente externo a la clase, por lo que se convertirán en responsabilidades privadas. Observe nuevamente que dos de estas responsabilidades colaboran con otras clases a pesar de ser privadas. En la figura 8.14 se muestra la clase *InterfaceUsuario* que solicita servicio a las responsabilidades de los diversos manejadores a través de la responsabilidad "manejarEvento" descrita en la clase *Manejador*.

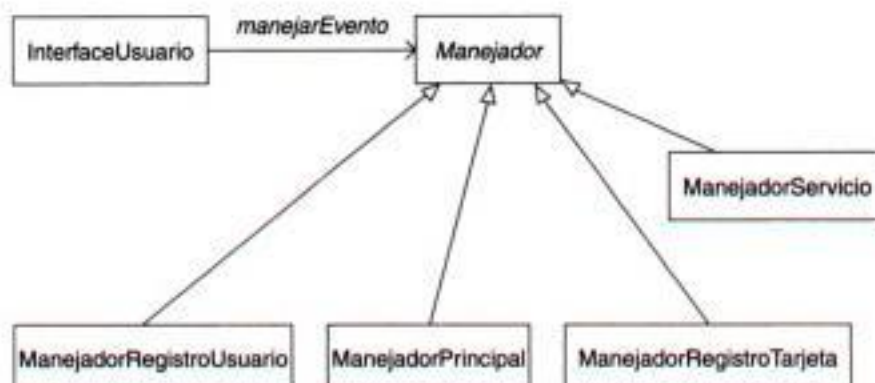


Figura 8.14 El diagrama muestra la clase *InterfaceUsuario* como cliente de los diversos manejadores a través de la responsabilidad "manejarEvento" de la clase *Manejador*.

Asignaremos como contrato número "1" a "manejarEvento", en tanto que "desplegarPantalla", "manejarEventoOfrecerServicio", "ofrecerServicio", "manejarEventoSalir" y "salir", serán responsabilidades privadas. Con base en estas consideraciones y a partir de la tabla 8.85 se genera la descripción para la clase *Manejador*, como se muestra en la tabla 8.107. Note en la columna derecha

Tabla 8.107 Tarjeta para la clase *Manejador* con responsabilidades, colaboraciones, jerarquías y contratos identificados de los diversos manejadores a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Manejador
Descripción: superclase heredada por todos los manejadores del sistema.
Módulo: Principal
Estereotipo: Control
Propiedades: Abstracta
Superclases:
Subclases: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta
Atributos:

continúa

Hidden page

Tabla 8.108 Continuación

1. Manejar Evento	
manejarEvento	
Responsabilidades Privadas	
desplegarPantalla	InterfaceUsuario (1)
manejarEventoOfrecerServicio	ManejadorServicio (2)
manejarEventoSalir	

Como ya se han explicado los números entre paréntesis agregados a las clases colaboradoras en la columna derecha, se volverá a definir la tarjeta para la clase *InterfaceUsuario* correspondiente a la tabla 8.103. Esto se muestra en la tabla 8.109 agregando la colaboración con el contrato "1", "Desplegar Pantalla", definido en la clase *Pantalla* y el contrato "2", "Manejar Evento", definido en la clase *Manejador*.

Tabla 8.109 Tarjeta para la clase *InterfaceUsuario* revisada con números de contratos para las colaboraciones.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Desplegar Pantalla	
desplegarPantalla	Pantalla (1): PantallaPrincipal (1), PantallaServicio (1), PantallaCrearRegUsuario (1), PantallaObtenerRegUsuario (1), PantallaCrearRegTarjeta (1), PantallaObtenerRegTarjeta (1)
2. Enviar Evento	
enviarEvento	Manejador (1): ManejadorPrincipal (1), ManejadorServicio (1), ManejadorRegistroUsuario (1), ManejadorRegistroTarjeta (1)

A continuación se consideran las responsabilidades definidas para la clase *ManejadorPrincipal* descritas en la tabla 8.86. La responsabilidad "manejar-Evento" sobreescribe la responsabilidad con el mismo nombre en la clase *Manejador*, por lo que se genera un contrato "1" que sobreescribe al contrato general definido en la superclase. Las responsabilidades "manejarEventoRegistrar",

"crearRegistroUsuario", "manejarEventoValidar" y "validarRegistroUsuario" son privadas, ya que los llama la propia clase, como consecuencia del contrato "Manejar Evento". La clase *ManejadorPrincipal* con los respectivos contratos se muestra en la tabla 8.110. Observe que se agrega el contrato "2" a la clase colaboradora *ManejadorRegistroUsuario*, algo que aún no se ha definido, pero se hará más adelante. Básicamente los contratos "1" corresponden en todos los manejadores al contrato "Manejar Evento", en tanto que los contratos a partir del número "2" representan los contratos adicionales de los manejadores, y se van dando según una numeración local incremental.

Tabla 8.110 Tarjeta para la clase *ManejadorPrincipal* con responsabilidades, colaboraciones, jerarquías y contratos identificados a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorPrincipal	
Descripción: el manejador principal es el encargado de desplegar la pantalla principal de interacción con el usuario, y luego delegar las diferentes funciones a los manejadores especializados apropiados.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento	
Responsabilidades Privadas	
manejarEventoRegistrar	
crearRegistroUsuario	ManejadorRegistroUsuario (2)
manejarEventoValidar	
validarRegistroUsuario	ManejadorRegistroUsuario (2)

De manera adicional y como se analizó hace un momento, es posible integrar las responsabilidades "manejarEventoRegistrar" y "crearRegistroUsuario" en una sola. Estas responsabilidades se identificaron inicialmente en la tabla 8.3 a partir de las frases (6) y (7), respectivamente; donde la segunda se identificó como continuación de la primera. A esta responsabilidad se le llamará "manejarEventoRegistrar" y se eliminará "crearRegistroUsuario" para enfatizar que fue generada a partir de un evento de botón. Se hará lo mismo con "manejarEventoValidar" y "validarRegistroUsuario", manteniendo "manejarEventoValidar". En la tabla 8.111 se mostrará la tarjeta de clase actualizada para *ManejadorPrincipal*.

Hidden page

Hidden page

Hidden page

el registro del usuario, sea a nivel de creación, validación u obtención. Existe siempre la alternativa de definir múltiples contratos; sin embargo, esto únicamente aumentaría la complejidad en este caso, ya que la funcionalidad se considera lógicamente similar.



Figura 8.16 El diagrama muestra las clases *ManejadorPrincipal* y *ManejadorServicio* como clientes del contrato "Registrar Usuario", contrato número "2", de la clase *ManejadorRegistroUsuario*.

En la tabla 8.115 se describe la tarjeta para la clase *ManejadorRegistroUsuario*. La responsabilidad "manejarEvento" se asigna al contrato número "1", "Manejar Evento", en tanto que las responsabilidades "crearRegistroUsuario", "validar-RegistroUsuario" y "obtenerRegistroUsuario", se asignan al contrato número "2", "Registrar Usuario". El resto de las responsabilidades se mantienen como responsabilidades privadas, ya que se llaman localmente dentro de la clase *ManejadorRegistroUsuario*. Observe que nuevamente se agregaron números de contratos a las diferentes clases colaboradoras. Estos contratos están aún por definirse; sin embargo, se mantiene la numeración lógica anterior.

Tabla 8.115 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades, colaboraciones, jerárquicas y contratos identificados a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroUsuario
Descripción: el manejador de registro de usuario se encarga de lo relacionado con el registro del usuario para utilizar el sistema.
Módulo: Registro.Usuario
Estereotipo: Control
Propiedades: Concreta
Superclases: Manejador
Subclases:
Atributos:

continúa

Tabla 8.115 Continuación

Contratos	
1. Manejar Evento	
manejarEvento	
2. Registrar Usuario	
crearRegistroUsuario	
validarRegistroUsuario	InterfaceBaseDatosRegistro (1)
obtenerRegistroUsuario	InterfaceBaseDatosRegistro (1)
Responsabilidades Privadas	
administrarRegistroUsuario	
manejarEventoRegistrar	
crearRegistroUsuario	InterfaceBaseDatosRegistro (1)
manejarEventoActualizar	
actualizarRegistroUsuario	InterfaceBaseDatosRegistro (1)
manejarEventoEliminar	
eliminarRegistroUsuario	InterfaceBaseDatosRegistro (1)
manejarEventoRegistrarTarjeta	
registrarTarjeta	ManejadorRegistroTarjeta (2)

De nuevo, se integran las responsabilidades generadas inicialmente a partir de un evento de botón, en particular, "manejarEventoRegistrar" y "crearRegistroUsuario" (note que existían dos eventos "crearRegistroUsuario" que lógicamente son distintos), "manejarEventoActualizar" y "actualizarRegistroUsuario", "manejarEventoEliminar" y "eliminarRegistroUsuario" y finalmente "manejarEventoRegistrarTarjeta" y "registrarTarjeta". La tarjeta de clase actualizada para la clase *ManejadorRegistroUsuario* se muestra en la tabla 8.116.

Tabla 8.116 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades, colaboraciones, jerarquías y contratos identificados a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	

continúa

Tabla 8.116 Continuación

1. Manejar Evento	
manejarEvento	
2. Registrar Usuario	
crearRegistroUsuario	
validarRegistroUsuario	InterfaceBaseDatosRegistro (1)
obtenerRegistroUsuario	InterfaceBaseDatosRegistro (1)
Responsabilidades Privadas	
administrarRegistroUsuario	
manejarEventoRegistrar	InterfaceBaseDatosRegistro (1)
manejarEventoActualizar	InterfaceBaseDatosRegistro (1)
manejarEventoEliminar	InterfaceBaseDatosRegistro (1)
manejarEventoRegistrarTarjeta	ManejadorRegistroTarjeta (2)

De manera similar a las demás pantallas, la clase *PantallaRegUsuario* se describe igual a como se describió originalmente en la tabla 8.90, como se presenta en la tabla 8.117.

Tabla 8.117 Tarjeta para la clase *PantallaRegUsuario* con responsabilidades, colaboraciones, jerarquías y contratos identificados del caso de uso *Registrar Usuario*.

Clase: PantallaRegUsuario	
Descripción: superclase con diseño gráfico común para las pantallas de registro de usuario.	
Módulo: Registro.Usuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases: Pantalla	
Subclases: PantallaCrearRegUsuario, PantallaObtenerRegUsuario	
Atributos:	

A partir de la tabla 8.91 se genera la clase *PantallaCrearRegUsuario*, como se muestra en la tabla 8.118.

Tabla 8.118 Tarjeta para la clase *PantallaCrearRegUsuario* con responsabilidades, colaboraciones, jerarquías y contratos identificados del caso de uso *Registrar Usuario*.

Clase: PantallaCrearRegUsuario	
Descripción: pantalla de solicitud de registro de usuario (P-3).	
Módulo: Registro.Usuario	

continúa

Tabla 8.118 Continuación

Estereotipo: Borde	
Propiedades: Concreta	
Superclases: Pantalla	
Subclases:	
Atributos:	

A partir de la tabla 8.92 se genera la clase *PantallaObtenerRegUsuario*, como se muestra en la tabla 8.119.

Tabla 8.119 Tarjeta para la clase *PantallaObtenerRegUsuario* con responsabilidades, colaboraciones, jerarquías y contratos identificados del caso de uso *Registrar Usuario*.

Clase: PantallaObtenerRegUsuario	
Descripción: pantalla de devolución con información de registro de usuario (P-4).	
Módulo: Registro,Usuario	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases: Pantalla	
Subclases:	
Atributos:	

La clase *RegistroUsuario* se mantiene igual como se describió en la tabla 8.93 y se muestra en la tabla 8.120.

Tabla 8.120 Tarjeta para la clase *RegistroUsuario* con responsabilidades, colaboraciones, jerarquías y contratos de actualizar y consultar información de registro para el caso de uso *Registrar Usuario*.

Clase: RegistroUsuario	
Descripción: para utilizar el sistema de reservaciones, el usuario debe estar registrado en el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa y oficina, fax, email, login y password.	
Módulo: Registro,Usuario	
Estereotipo: Entidad	
Propiedades: Concreta	
Superclases: Datos	
Subclases:	
Atributos:	

TARJETA

Esta sección abarca las clases de registro de tarjeta que son *ManejadorRegistroTarjeta*, *PantallaCrearRegTarjeta*, *PantallaObtenerRegTarjeta* y *RegistroTarjeta*.

Las responsabilidades de la clase *ManejadorRegistroTarjeta* se describieron en la tabla 8.94 y de nuevo en la tabla 8.121.

Tabla 8.121 Responsabilidades definidas para la clase <i>ManejadorRegistroTarjeta</i> según las tabla 8.94.	
manejarEvento	
registrarTarjeta	
manejarEventoRegistrar	
crearRegistroTarjeta	InterfaceBaseDatosRegistro
obtenerRegistroTarjeta	InterfaceBaseDatosRegistro
administrarRegistroTarjeta	
manejarEventoActualizar	
actualizarRegistroTarjeta	InterfaceBaseDatosRegistro
manejarEventoEliminar	
eliminarRegistroTarjeta	InterfaceBaseDatosRegistro

De manera similar a *ManejadorPrincipal* y *ManejadorRegistroUsuario*, la responsabilidad "manejarEvento" será asignada al contrato "1", "Manejar Evento", para sobreescribir el contrato con el mismo número en la superclase *Manejador*. La única responsabilidad accesada externamente es "registrarTarjeta", que la llama el *ManejadorRegistroUsuario*, como se muestra en la figura 8.17.



Figura 8.17 El diagrama muestra la clase *ManejadorRegistroUsuario* como cliente de la responsabilidad "registrarTarjeta" de la clase *ManejadorRegistroTarjeta*.

A partir de la tabla 8.94 se generan los contratos para la clase *ManejadorRegistroTarjeta*, como se muestra en la tabla 8.122. La responsabilidad "manejarEvento" se asigna al contrato número "1", "Manejar Evento", en tanto que la responsabilidad "registrarTarjeta" se asigna al contrato número "2", "Registrar Tarjeta". El resto de las responsabilidades se mantienen como privadas, ya que se llaman localmente dentro de la clase *ManejadorRegistroTarjeta*. Observe que nuevamente se agregaron números de contratos a las diferentes clases colaboradoras. Estos contratos están aún por definirse, sin embargo, se conserva la numeración lógica anterior.

Tabla 8.122 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades, colaboraciones, jerarquías y contratos identificados del caso de uso *Registrar Tarjeta*.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de lo relacionado con el registro de la tarjeta del usuario a fin de pagar las reservaciones.	
Propiedades: Concreta	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento	
2. Registrar Tarjeta	
registrarTarjeta	
Responsabilidades Privadas	
administrarRegistroTarjeta	
manejarEventoRegistrar	
crearRegistroTarjeta	InterfaceBaseDatosRegistro (2)
obtenerRegistroTarjeta	InterfaceBaseDatosRegistro (2)
manejarEventoActualizar	
actualizarRegistroTarjeta	InterfaceBaseDatosRegistro (2)
manejarEventoEliminar	
eliminarRegistroTarjeta	InterfaceBaseDatosRegistro (2)

Nuevamente, se integran las responsabilidades de "manejarEvento..." con las responsabilidades estrechamente relacionadas, como se muestra en la tabla 8.123. Además de esto, aprovechamos para agregar una nueva responsabilidad "crear-

Tabla 8.123 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades, colaboraciones, jerarquías y contratos identificados del caso de uso *Registrar Tarjeta*.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de lo relacionado con el registro de la tarjeta del usuario para pagar las reservaciones.	
Propiedades: Concreta	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Superclases: Manejador	
Subclases:	

continúa

Tabla 8.123 Continuación

Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento	
2. Registrar Tarjeta	
registrarTarjeta	
Responsabilidades Privadas	
crearRegistroTarjeta	
obtenerRegistroTarjeta	InterfaceBaseDatosRegistro (2)
administrarRegistroTarjeta	
manejarEventoRegistrar	InterfaceBaseDatosRegistro (2)
manejarEventoActualizar	InterfaceBaseDatosRegistro (2)
manejarEventoEliminar	InterfaceBaseDatosRegistro (2)

RegistroTarjeta" que es necesaria para dar servicio local al evento "registrar-Tarjeta", en caso de que no exista un registro de tarjeta anterior, de manera complementaria a "obtenerRegistroTarjeta". El método "administrarRegistro-Tarjeta" se encarga de desplegar la información de la tarjeta una vez obtenida o creada. Observe que este nuevo "crearRegistroTarjeta" es distinto a la responsabilidad con el mismo nombre, aunque en colaboración con *InterfaceBaseDatos-Registro*. La nueva responsabilidad "crearRegistroTarjeta" (sin colaboración) es análoga a "crearRegistroUsuario", que se encarga de desplegar la *PantallaCrear-RegTarjeta*, mientras que la segunda es responsable de crear el nuevo registro de tarjeta, una vez presionado el botón "registrar".

Las pantallas se describen nuevamente de manera similar a las anteriores. A partir de la tabla 8.95 se vuelve a describir la clase *PantallaRegTarjeta*, como se muestra en la tabla 8.124.

Tabla 8.124 Tarjeta para la clase *PantallaRegTarjeta* con responsabilidades, colaboraciones, jerarquías y contratos identificados del caso de uso *Registrar Tarjeta*.

Clase: PantallaRegTarjeta	
Descripción: superclase con diseño gráfico común para las pantallas de registro de tarjeta.	
Módulo: Registro.Tarjeta	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases: Pantalla	
Subclases: PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta	
Atributos:	

Hidden page

Hidden page

Hidden page

Tabla 8.129 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades, colaboraciones, jerarquías y contratos de escribir y leer información de registro de usuario y registro de tarjeta para los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorServicio	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicios	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento	
2. Ofrecer Servicio	
ofrecerServicio	
Responsabilidades Privadas	
manejarEventoRegistrar	manejadorRegistroUsuario (2)

propio a esta clase, "Ofrecer Servicio" que tiene como cliente a los demás manejadores, como se presenta en la figura 8.20.



Figura 8.20 El diagrama muestra a las diversas clases de manejadores como clientes de la clase *ManejadorServicio*.

A partir de la tabla 8.100 se describe la clase *ManejadorServicio*, como se muestra en la tabla 8.130. Además de los dos contratos antes mencionados se agrega una responsabilidad privada llamada "manejarEventoRegistrar" que ya está integrada con "registrar".

Hidden page

8.2.6 Subsistemas

Como se ha podido apreciar hasta el momento, la complejidad del sistema aumenta a medida que se incorporan nuevos detalles en el diseño, algo que por lo general es inevitable. Para lograr un mejor manejo de esta complejidad se introduce el concepto de *subsistemas*, que permite dividir el sistema completo en diversas partes, inspirado en la idea de "divide y vencerás". Los subsistemas permiten agrupar objetos relacionados para lograr cierta funcionalidad en "minisistemas". El sistema completo se compone de estos "minisistemas" o subsistemas, y cada subsistema puede subdividirse en subsistemas adicionales o definirse en términos de los objetos finales. Los subsistemas también permiten trabajar en diferentes partes del sistema en paralelo, mediante su asignación a múltiples diseñadores.

La organización en subsistemas se logra a partir de los contratos identificados antes entre los objetos. Externamente, los subsistemas son mecanismos de encapsulamiento, vistos como "cajas negras", donde sus objetos cooperan para proveer una unidad de funcionalidad claramente delimitada por el subsistema. Internamente, los subsistemas tienen estructuras complejas, con clases colaborando entre sí para satisfacer sus distintas responsabilidades contribuyendo al objetivo general del subsistema, o sea, satisfacer sus responsabilidades.

Se busca tener un fuerte acoplamiento funcional dentro de cada subsistema y un acoplamiento débil entre subsistemas, en otras palabras, se busca tener la mínima comunicación entre los diferentes subsistemas.

Un subsistema bien diseñado tiene pocas clases o subsistemas que apoyan directamente contratos, y un número mayor de colaboraciones entre clases y subsistemas internos.

Es importante distinguir entre el concepto de *módulo* y *subsistema*. Un módulo agrupa clases, correspondiente a *bibliotecas*, o sea, organizaciones estáticas definidas para almacenar y facilitar el acceso a las clases. Esto es similar al concepto de *directorios* en una computadora. Son estructuras puramente organizacionales sin ningún efecto sobre los propios procesos. Por otro lado, el subsistema agrupa objetos, siendo una abstracción dinámica correspondiente a la funcionalidad del proceso. En general, los objetos pertenecientes a un subsistema son instancias de clases que deben existir forzosamente en algún módulo. En tanto que las clases no deben duplicarse entre módulos, se permite instanciar objetos de la misma clase en múltiples subsistemas, siendo esto parte de la reutilización de código.

Los subsistemas deben incluirse completos o no incluirse, de manera que un sistema pueda ofrecerse con o sin ciertos subsistemas. Dada la dependencia entre subsistemas, si se incluye un subsistema, también se debe entregar el subsistema del cual depende.

Por lo común, los objetos de las clases entidad se reutilizan en los diversos subsistemas, en tanto que los objetos de control, y por lo general las de borde, son propios a cierto subsistema.

Hidden page

Hidden page

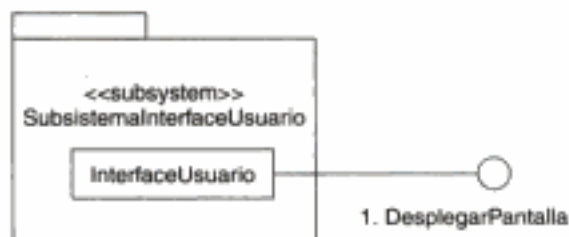


Figura 8.22 Diagrama de subsistema para el ejemplo del subsistema *InterfaceUsuario*.

respectivas clases servidores, utilizando una flecha del cliente al servidor, como se muestra en la figura 8.23 (la flecha de la derecha proveniente de *Clase Cliente*).

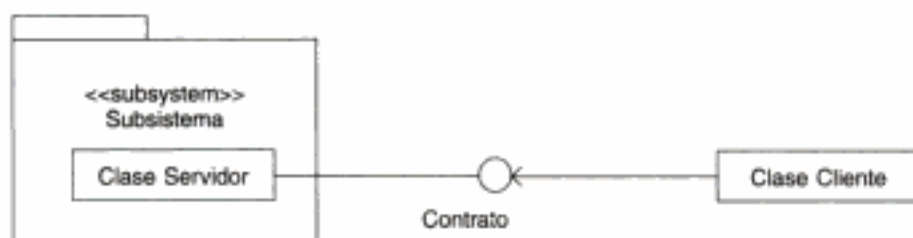


Figura 8.23 Diagrama de colaboración donde *Clase Cliente* llama al Contrato del Subsistema implementado por la *Clase Servidor*.

Si dos clases hacen solicitudes a un mismo contrato se dibujan múltiples flechas al mismo círculo. Por ejemplo, en la figura 8.24 se presentan dos manejadores llamando al contrato “Desplegar Pantalla” del subsistema *InterfaceUsuario*.

El problema de la complejidad es evidente cuando uno se fija en el diagrama de colaboración. El diagrama se vuelve un “espagueti”. No se entiende fácil-

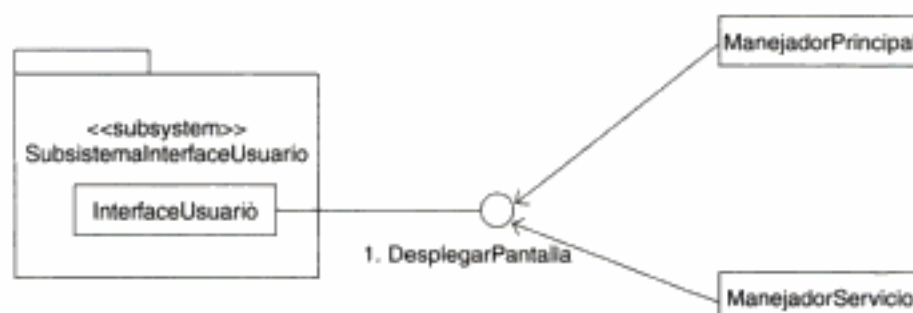


Figura 8.24 Diagrama de subsistema para el ejemplo del subsistema *InterfaceUsuario* con dos clientes para el contrato “Desplegar Pantalla”, *ManejadorPrincipal* y *ManejadorServicio*.

mente y la aplicación se vuelve imposible de mantener o modificar. Por tanto, la meta es simplificar los patrones de colaboración, algo que luego se traduce en una simplificación en los diagramas de colaboración.

A continuación se identificarán los subsistemas para el *Sistema de Reservaciones*. Dado que el enfoque del desarrollo del sistema ha sido a través de casos de uso, inicialmente será natural identificar subsistemas a partir de ellos. Sin embargo, el criterio principal para la asignación de clase a los subsistemas es minimizar la interacción entre clases en distintos subsistemas. Por otro lado, se debe mantener cierta relación con la lógica introducida en la arquitectura de clase. No hay que olvidar que se está diseñando. También se debe recordar que los subsistemas pueden definirse de manera jerárquica, a diferencia de los casos de uso.

Entonces, se debe analizar estas consideraciones en relación con nuestro sistema para identificar los subsistemas relevantes. Viendo el sistema desde un alto nivel podrían definirse dos grupos de clases generales: los relacionados con los registros y con consultas, reservaciones y pagos, o sea, los propios servicios del sistema. Por tanto, a un primer nivel se define un *SubsistemaRegistro* y otro *SubsistemaServicio*. Estos subsistemas se dividen en subsistemas adicionales si así se desea. Sin embargo, antes de definir subsistema más detallados veamos qué ocurre con los niveles superiores. Aunque quedaría bastante clara la asignación de la mayor parte de las clases a estos dos subsistemas, como en el caso del *ManejadorRegistroUsuario*, *ManejadorReservas*, etc., hay ciertas clases que requieren un análisis más detallado. Por ejemplo, ¿a qué subsistema se asigna la *InterfaceUsuario* y el *ManejadorPrincipal*?

Tanto la *InterfaceUsuario* como el *ManejadorPrincipal* son clases que no pertenecen de manera natural a ninguno de los dos subsistemas anteriores. Podría definirse un nuevo subsistema para ambos o, si lógicamente son independientes, definir dos nuevos subsistemas, uno para cada uno. Esto se hará dado que la *InterfaceUsuario* es genérica, en relación con la lógica interna, y bastante independiente de la aplicación en sí, a diferencia del *ManejadorPrincipal* que tiene cierto conocimiento sobre el registro y servicios de la aplicación.

Otra manera de conceptualizar la creación de estos dos nuevos subsistemas es que inicialmente se definió el sistema con base en su funcionalidad en términos de los casos de uso, o sea, sus servicios externos. Sin embargo, también se debe considerar el sistema en términos de sus servicios internos. Por ejemplo, la *InterfaceUsuario* ofrece servicios internos al sistema para que se desplieguen las diferentes pantallas y se obtengan eventos del *Usuario*. De igual manera, el *ManejadorPrincipal* ofrece funcionalidad para inicializar la aplicación, nuevamente un servicio interno que coordina entre funcionalidades externas. Por tanto, se definiría el sistema completo en términos de cuatro subsistemas, *SubsistemaInterfaceUsuario*, *SubsistemaPrincipal*, *SubsistemaRegistro* y *SubsistemaServicio*, como se muestra en la figura 8.25.

A continuación se describen estos subsistemas con mayor detalle, aunque limitándose a los subsistemas relevantes para la funcionalidad descrita en los casos de uso *ValidarUsuario*, *OfrecerServicio*, *RegistrarUsuario* y *RegistrarTarjeta*.

Hidden page

Tabla 8.134 Tarjeta de subsistema para *SubsistemaInterfaceUsuario* que muestra sus contratos y servidores a partir de los casos de uso *Registrar Usuario*, *Validar Usuario* y *Registrar Tarjeta*.

Subsistema: SubsistemaInterfaceUsuario	
Descripción: este subsistema agrupa los objetos que intervienen con el manejo general de las interfaces de usuario.	
Clases: InterfaceUsuario, PantallaPrincipal, PantallaServicio, PantallaCrearRegUsuario, PantallaObtenerRegUsuario, PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta	
Contratos	Servidor
1. Desplegar Pantalla	InterfaceUsuario (1)

El *SubsistemaInterfaceUsuario* se muestra gráficamente en la figura 8.26. Se presenta únicamente la clase *InterfaceUsuario* por ser el servidor del subsistema.

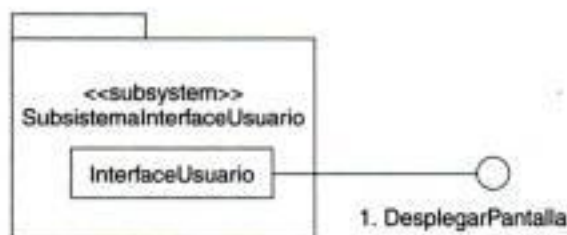


Figura 8.26 Diagrama de subsistemas para el *SubsistemaInterfaceUsuario* mostrando únicamente a *InterfaceUsuario*, la clase servidor.

SUBSISTEMA PRINCIPAL

El *SubsistemaPrincipal* consta principalmente de la clase *ManejadorPrincipal*. En la tabla 8.110 se puede apreciar que se definió un solo contrato, "Manejar Evento", que tiene como cliente la clase *InterfaceUsuario*. Dado que la clase *InterfaceUsuario* pertenece al *SubsistemaInterfaceUsuario*, el contrato "Manejar Evento" debe definirse como externo al subsistema. La tarjeta para el *SubsistemaPrincipal* se muestra en la tabla 8.135.

Tabla 8.135 Tarjeta de subsistema para *SubsistemaPrincipal* mostrando sus contratos y servidores a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Subsistema: SubsistemaPrincipal	
Descripción: este subsistema agrupa todos los objetos que intervienen con el manejo general del sistema.	
Clases: ManejadorPrincipal	
Contratos	Servidor
1. Manejar Evento	ManejadorPrincipal (1)

El *SubsistemaPrincipal* se muestra gráficamente en la figura 8.27. En el diagrama no se incluye la clase *PantallaPrincipal*, ya que ésta no ofrece servicios externos.

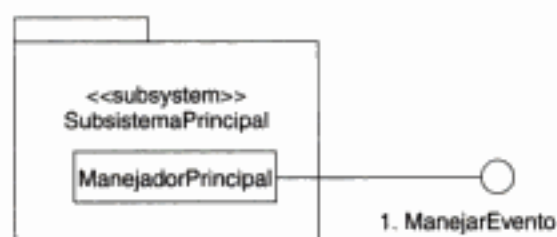


Figura 8.27 Diagrama de subsistemas para el *SubsistemaPrincipal*.

SUBSISTEMA REGISTRO

El *SubsistemaRegistro* consta de todas las clases relacionadas con el registro, con excepción de las pantallas que fueron asignadas al subsistema *SubsistemaInterfaceUsuario*. Las clases incluidas son *ManejadorRegistroUsuario*, *RegistroUsuario*, *ManejadorRegistroTarjeta*, *RegistroTarjeta* e *InterfaceBaseDatosRegistro*. Las clases entidad estarán contenidas junto con los manejadores y la clase de borde con la base de datos. En término de contratos, se observa que los dos contratos "Manejar Evento", pertenecientes a los dos manejadores, deben ser externos al subsistema, ya que los llama la *InterfaceUsuario*. Adicionalmente, existe otro contrato que se llama externamente, "Registrar Usuario" (el contrato "Registrar Tarjeta", aunque no se llama por clases pertenecientes a otros subsistemas para los casos de uso analizados hasta el momento, finalmente se agregará al subsistema como apoyo al caso de uso de "Pagar Reservación"). Por otro lado, las clases entidad, *RegistroUsuario* y *RegistroTarjeta*, aún no definen contratos, en tanto que la clase *InterfaceBaseDatosRegistro* contiene contratos únicamente para los manejadores internos de este subsistema. La tarjeta para el *SubsistemaRegistro* se muestra en la tabla 8.136. Observe que existen dos servidores para el primer

Tabla 8.136 Tarjeta de subsistema para *SubsistemaRegistro* mostrando sus contratos y servidores a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Subsistema: SubsistemaRegistro	
Descripción: este subsistema agrupa todos los objetos que intervienen con el manejo de registro de usuario, incluyendo registro de tarjeta.	
Clases: ManejadorRegistroUsuario, RegistroUsuario, ManejadorRegistroTarjeta, RegistroTarjeta, InterfaceBaseDatosRegistro	
Contratos	Servidor
1. Manejar Evento	ManejadorRegistroUsuario (1), ManejadorRegistroTarjeta (1)
2. Registrar Usuario	ManejadorRegistroUsuario (2)

Hidden page

El *SubsistemaServicio* se muestra gráficamente en la figura 8.29. Ambos contratos son ofrecidos por el *ManejadorServicio*.



Figura 8.29 Diagrama de subsistemas para el *SubsistemaServicios*.

SISTEMA

Una de las ventajas de definir subsistemas, es la posibilidad de visualizar el sistema completo a partir de éstos. En la figura 8.30 se integran los subsistemas anteriores mostrando los contratos entre subsistemas. Se aprecian seis contratos los que llaman las clases manejadoras en los subsistemas de *Registro*, *Servicios* y *Principal*, y la *InterfaceUsuario* en el subsistema correspondiente.

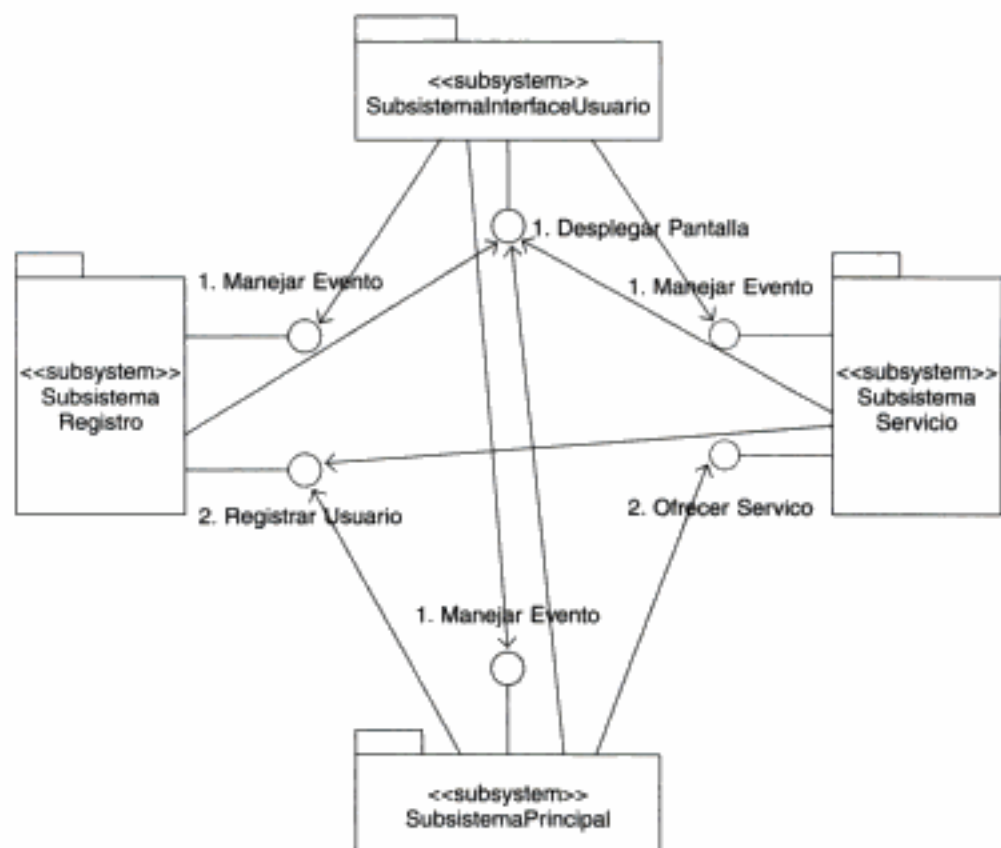


Figura 8.30 Subsistemas del sistema de reservaciones de vuelo.

Hidden page

Hidden page

Tabla 8.140 Continuación

Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento	
Responsabilidades Privadas	
manejarEventoRegistrar	SubsistemaRegistro (2)
manejarEventoValidar	SubsistemaRegistro (2)

La clase *PantallaPrincipal* se mantiene igual como se definió anteriormente en la tabla 8.112.

MÓDULO DOMINIO

La descripción de la clase *Datos* se mantiene igual como se mostró en la tabla 8.113.

MÓDULO REGISTRO

En el módulo de *Registro*, compuesto de los módulos *Usuario*, *Tarjeta* e *Interface-BD*, tampoco hay necesidad de modificar las clases.

MÓDULO SERVICIOS

La clase *ManejadorServicio*, como se muestra en la tabla 8.130, hace llamadas al contrato "2" de la clase *ManejadorRegistroUsuario* que pertenece al *Subsistema-Registro*. Por tanto, la colaboración debe modificarse de manera correspondiente. Los cambios para la clase *ManejadorServicio* se muestran en la tabla 8.141.

Tabla 8.141 Tarjeta para la clase *ManejadorServicio* con responsabilidades, colaboraciones, jerarquías, contratos y subsistemas a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorServicio
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.
Módulo: Servicios
Estereotipo: Control
Propiedades: Concreta
Superclases: Manejador

continúa

Hidden page

Hidden page

Hidden page

Hidden page

Tabla 8.148 Tarjeta para la superclase clase *Pantalla* con protocolo para la responsabilidad "enviarEvento".

Responsabilidades Privadas	
enviarEvento(Evento) devuelve void	InterfaceUsuario (2)

La tarjeta de clase modificada para la clase *Pantalla* se muestra en la tabla 8.149.

Tabla 8.149 Tarjeta para la superclase clase *Pantalla* con responsabilidades, colaboraciones, jerarquías, contratos, subsistemas y protocolos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Pantalla	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases:	
Subclases: PantallaPrincipal, PantallaServicio, PantallaRegUsuario, PantallaRegTarjeta	
Atributos:	
Contratos	
1. Desplegar Pantalla	
desplegarPantalla() devuelve void	
Responsabilidades Privadas	
enviarEvento(Evento) devuelve void	InterfaceUsuario (2)

PRINCIPAL

A partir de la tabla 8.139 se muestran las diversas responsabilidades para la clase *Manejador*, como se presenta en la tabla 8.150.

Tabla 8.150 Responsabilidades para la clase *Manejador* según se describió en la tabla 8.139.

1. Manejar Evento	
manejarEvento	
Responsabilidades Privadas	
desplegarPantalla	SubsistemaInterfaceUsuario (1)
manejarEventoOfrecerServicio	SubsistemaServicio (2)
manejarEventoSalir	

El contrato "Manejar Evento" lo llama la clase *InterfaceUsuario* que debe enviar el evento generado por el usuario y enviado luego por las diversas pantallas. Por tanto, debe incluirse un parámetro de tipo "Evento" y asignar nuevamente un tipo "void" de devolución. Las demás responsabilidades son privadas y se deja de asignar parámetros y definir un tipo "void" de devolución. Esto se muestra en la tabla 8.151.

Tabla 8.151 Tarjeta para la clase <i>Manejador</i> con responsabilidades, colaboraciones, jerarquías, contratos, subsistemas y protocolos identificados de los diversos manejadores para los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: Manejador	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Abstracta	
Superclases:	
Subclases: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
Responsabilidades Privadas	
desplegarPantalla() devuelve void	SubsistemaInterfaceUsuario (1)
manejarEventoOfrecerServicio() devuelve void	SubsistemaServicio (2)
manejarEventoSalir() devuelve void	

A partir de la tabla 8.140 se definen los protocolos para la clase *ManejadorPrincipal* a partir del contrato "Manejar Evento" y dos responsabilidades privadas. El contrato "Manejar Evento" contiene la responsabilidad "manejarEvento" que se debe definir de manera similar a la responsabilidad que se sobrescribe de la superclase *Manejador*. Las dos responsabilidades privadas se pueden dejar sin parámetros y devolver un tipo "void", como se muestra en la tabla 8.152.

Tabla 8.152 Tarjeta para la clase <i>ManejadorPrincipal</i> con responsabilidades, colaboraciones, jerarquías, contratos, subsistemas y protocolos identificados de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: ManejadorPrincipal	
Descripción: el manejador principal es el encargado de desplegar la pantalla principal de interacción con el usuario, y luego delegar las diferentes funciones a los manejadores especializados apropiados.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Concreta	

continúa

Hidden page

En el caso del contrato "Manejar Evento" se define un protocolo para la responsabilidad "manejarEvento" similar a la de la clase *Manejador*, como se hizo con el *ManejadorPrincipal*.

En relación con el contrato "Registrar Usuario" se tienen tres responsabilidades, "crearRegistroUsuario", "validarRegistroUsuario" y "obtenerRegistroUsuario". Las dos primeras las llama el *ManejadorPrincipal* para iniciar transacciones de registro. En el caso de "crearRegistroUsuario", el *ManejadorPrincipal* solicita al *ManejadorRegistroUsuario* que despliegue la *PantallaCrearRegUsuario*, algo que será controlado por este último. Dado que todo el control de la transacción está a cargo del *ManejadorRegistroUsuario*, no hay necesidad de agregar ningún parámetro y se devuelve un tipo "void".

En el caso de "validarRegistroUsuario", el *ManejadorPrincipal* solicita al *ManejadorRegistroUsuario* que valide los datos del usuario que se insertan en la *PantallaPrincipal*, la cual está a cargo del *ManejadorPrincipal*. El *ManejadorPrincipal* debe hacerle llegar los datos del usuario al *ManejadorRegistroUsuario*, algo para incluir como dos parámetros de tipo "String" ("cadenas de caracteres"), correspondientes al nombre del usuario (*log*) y su contraseña (*pass*). Como el *ManejadorPrincipal* decide con base en el resultado si proceden los servicios, también será necesario devolver algún tipo de valor que comunique si la solicitud fue aceptada, por tanto se agregará un tipo "boolean" como resultado.

En el caso de la responsabilidad "obtenerRegistroUsuario", el *ManejadorServicio* solicita al *ManejadorRegistroUsuario* que obtenga el registro de usuario y continúe el procesamiento. En tal caso, no se envía ningún parámetro y no es necesario esperar ninguna respuesta, por lo que la devolución queda como tipo "void".

En el caso de las responsabilidades privadas, se deja sin ningún parámetro y se devuelven tipos "void". Todos estos protocolos con la tarjeta de clase completa se muestran en la tabla 8.154.

Tabla 8.154 Tarjeta para la clase <i>ManejadorRegistroUsuario</i> con responsabilidades, colaboraciones, jerarquías, contratos, subsistemas y protocolos identificados de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de todo lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	

continúa

Tabla 8.154 Continuación

1. Manejar Evento	
manejarEvento(Evento) devuelve void	
2. Registrar Usuario	
crearRegistroUsuario() devuelve void	
validarRegistroUsuario(String log,String pass) devuelve boolean	InterfaceBaseDatosRegistro (1)
obtenerRegistroUsuario() devuelve void	InterfaceBaseDatosRegistro (1)
Responsabilidades Privadas	
administrarRegistroUsuario() devuelve void	
manejarEventoRegistrar() devuelve void	InterfaceBaseDatosRegistro (1)
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (1)
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (1)
manejarEventoRegistrarTarjeta() devuelve void	ManejadorRegistroTarjeta (2)

La descripción de las clases *PantallaRegUsuario*, *PantallaCrearRegUsuario*, *PantallaObtenerRegUsuario* y *RegistroUsuario* quedan igual, ya que no incluyen responsabilidades.

TARJETA

En la tabla 8.123 se definieron las diferentes responsabilidades para la clase *ManejadorRegistroTarjeta*, como se muestra en la tabla 8.155.

En el caso del contrato "Manejar Evento" se debe definir un protocolo para la responsabilidad "manejarEvento" similar a la de la clase *Manejador*, como se hizo con el *ManejadorPrincipal* y también con el *ManejadorRegistroUsuario*.

Tabla 8.155 Tarjeta para la clase *ManejadorRegistroTarjeta* como se definió en la tabla 8.123.

1. Manejar Evento	
manejarEvento	
2. Registrar Tarjeta	
registrarTarjeta	
Responsabilidades Privadas	
crearRegistroTarjeta	
obtenerRegistroTarjeta	InterfaceBaseDatosRegistro (2)
administrarRegistroTarjeta	
manejarEventoRegistrar	InterfaceBaseDatosRegistro (2)
manejarEventoActualizar	InterfaceBaseDatosRegistro (2)
manejarEventoEliminar	InterfaceBaseDatosRegistro (2)

En relación con el contrato "Registrar Tarjeta", la responsabilidad "registrar-Tarjeta" la llama el *ManejadorRegistroUsuario*. Dado que debe haber alguna manera de relacionar el registro de usuario con el registro de tarjeta, se envía como parámetro algún identificador de usuario (log) definido por ejemplo de tipo "String". El control del contrato continúa en el *ManejadorRegistroTarjeta*, por lo que el tipo a devolver es "void".

Las responsabilidades privadas nuevamente se definen sin argumentos y con devolución de tipo "void".

Todos estos protocolos con la tarjeta de clase completa se muestran en la tabla 8.156.

Tabla 8.156 Tarjeta para la clase <i>ManejadorRegistroTarjeta</i> con responsabilidades, colaboraciones, jerarquías, contratos, subsistemas y protocolos identificados del caso de uso <i>Registrar Tarjeta</i> .		
Clase: ManejadorRegistroTarjeta		
Descripción: el manejador de registro de tarjeta se encarga de lo relacionado con el registro de la tarjeta del usuario para pagar las reservaciones.		
Módulo: Registro.Tarjeta		
Estereotipo: Control		
Propiedades: Concreta		
Superclases: Manejador		
Subclases:		
Atributos:		
Contratos		
1. Manejar Evento		
manejarEvento(Evento) devuelve void		
2. Registrar Tarjeta		
registrarTarjeta(String log) devuelve void		
Responsabilidades Privadas		
crearRegistroTarjeta() devuelve void		
obtenerRegistroTarjeta() devuelve void		InterfaceBaseDatosRegistro (2)
administrarRegistroUsuario() devuelve void		
manejarEventoRegistrar() devuelve void		InterfaceBaseDatosRegistro (2)
manejarEventoActualizar() devuelve void		InterfaceBaseDatosRegistro (2)
manejarEventoEliminar() devuelve void		InterfaceBaseDatosRegistro (2)

La descripción de las clases *PantallaRegTarjeta*, *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta* se actualizan de manera similar a la *Pantalla-Principal*. Por otro lado, *RegistroTarjeta* se actualiza de igual manera a *Registro-Usuario*.

INTERFACE BASE DATOS REGISTRO

En la tabla 8.129 se definieron las diferentes responsabilidades para la clase *InterfaceBaseDatosRegistro*, como se muestra en la tabla 8.157.

Tabla 8.157 Responsabilidades para la clase *InterfaceBaseDatosRegistro* como se definió en la tabla 8.129.

1. Registrar Usuario	
validarRegistroUsuario	BaseDatosRegistro
crearRegistroUsuario	BaseDatosRegistro
obtenerRegistroUsuario	BaseDatosRegistro
actualizarRegistroUsuario	BaseDatosRegistro
eliminarRegistroUsuario	BaseDatosRegistro
2. Registrar Tarjeta	
crearRegistroTarjeta	BaseDatosRegistro
obtenerRegistroTarjeta	BaseDatosRegistro
actualizarRegistroTarjeta	BaseDatosRegistro
eliminarRegistroTarjeta	BaseDatosRegistro

Se definen dos contratos, "Registrar Usuario" y "Registrar Tarjeta", los que se encargan de interactuar con la base de datos para el almacenamiento de los registros de usuario y de tarjeta, respectivamente. Todas las responsabilidades deben enviar parámetros correspondientes a los registros para ser guardados en la base de datos, o incluso para devolverse, ya que se hace esto último a través de los parámetros.

En el caso del contrato "Registrar Usuario" se tendría como parámetro a la clase *RegistroUsuario* para las diversas responsabilidades, excepto la validación. En el caso de "validarRegistroUsuario", se tiene que enviar, además del *RegistroUsuario*, dos parámetros de tipo "String" correspondientes al nombre del usuario (log) y su contraseña (pass). Vale la pena resaltar, que en el caso de *ObtenerRegistroUsuario* y *ObtenerRegistroTarjeta* es necesario enviar un identificador de usuario (log) ya que no existe aún un registro en memoria del que se pueda obtener el identificador del usuario.

En el caso del contrato "Registrar Tarjeta" se haría algo similar con las responsabilidades, excepto que el parámetro sería de tipo *RegistroTarjeta*.

En general es bueno devolver un "boolean" para revisar si tuvo éxito la transacción.

Todos estos protocolos con la tarjeta de clase completa se muestran en la tabla 8.158.

Hidden page

Hidden page

hará a continuación es detallar un poco más el diseño especificando *atributos* correspondientes a estructuras de datos y *algoritmos* que especifiquen la manera de implementar las diversas responsabilidades correspondientes a los múltiples contratos de la aplicación.

En general, los *atributos* corresponden a los aspectos estructurales de las clases, como son los valores (números y textos), junto con las referencias a otros objetos. Estos conceptos varían en gran manera entre lenguajes. Por ejemplo, en el caso de Java y C++ la distinción entre valores y referencias es clara. En el caso de lenguajes como Smalltalk, no hay distinción alguna dado que todos los atributos corresponden a referencias entre objetos. Los atributos correspondientes a referencias deben agregarse según las colaboraciones establecidas durante el diseño. Los atributos que guardan valores son lo último que se especifica en el diseño de objetos.

A continuación se describen los atributos esenciales para las clases del *Sistema de Reservaciones*, con base en los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

INTERFACEUSUARIO

Como se observó anteriormente, la clase *InterfaceUsuario* se relaciona primordialmente y a través del polimorfismo con las diversas pantallas y manejadores. Dado que sólo se desplegará una pantalla a la vez y se comunicará con un solo manejador en un momento específico, es suficiente definir dos tipos de referencias, *Manejador* para comunicarse con los manejadores y *Pantalla* para comunicarse con las pantallas. Por lo tanto, se modifican la tarjeta de clase descrita anteriormente en la tabla 8.145 correspondiente a la *InterfaceUsuario* como se muestra en la tabla 8.161. Observe que los atributos se describen con base en un tipo, un tipo y un nombre o, incluso, solamente el nombre del atributo.

Tabla 8.161 Tarjeta para la clase *InterfaceUsuario* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceUsuario	
Descripción: la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos: Manejador, Pantalla	
Contratos	
1. Desplegar Pantalla	

continúa

Tabla 8.161 Continuación

desplegarPantalla(Pantalla) devuelve void	Pantalla (1) : PantallaPrincipal (1), PantallaServicio (1), PantallaCrearRegUsuario (1), PantallaObtenerRegUsuario (1), PantallaCrearRegTarjeta (1), PantallaObtenerRegTarjeta (1)
2. Enviar Evento	
enviarEvento(Evento, Manejador) devuelve void	Manejador (1) : SubsistemaPrincipal (1), SubsistemaServicio (1), SubsistemaRegistro (1)

De manera similar a la *InterfaceUsuario*, las pantallas deben tener conocimiento de la *InterfaceUsuario* y de los manejadores que las controlan. Aunque es posible especificar el tipo particular de manejador que administra una pantalla en particular, se aprovecha y define una referencia de tipo genérica a *Manejador*, ya que cada pantalla la administra un solo manejador. Este conocimiento se implementa mediante atributos correspondientes a las clases anteriores. La tarjeta de clases para la *Pantalla* descrita en la tabla 8.149 se muestra en la tabla 8.162.

Tabla 8.162 Tarjeta para la superclase *Pantalla* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Pantalla	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclases:	
Subclases: PantallaPrincipal, PantallaServicio, PantallaRegUsuario, PantallaRegTarjeta	
Atributos: InterfaceUsuario, Manejador	
Contratos	
1. Desplegar Pantalla	
desplegarPantalla() devuelve void	
Responsabilidades Privadas	
enviarEvento(Evento) devuelve void	InterfaceUsuario (2)

PRINCIPAL

De manera similar a la *InterfaceUsuario* y las pantallas, los manejadores deben acceder a la propia *InterfaceUsuario* y también a sus diferentes pantallas. Por tal motivo, se definen dos atributos, uno de tipo *InterfaceUsuario* y otro de tipo *Pantalla* correspondiente a la pantalla actualmente desplegada. Al revisar un poco más, se aprecia que la mayor parte de los manejadores requieren comu-

Hidden page

Hidden page

Tabla 8.165 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de todo lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades: Concreta	
Superclase: Manejador	
Subclases:	
Atributos: PantallaCrearRegUsuario, PantallaObtenerRegUsuario, RegistroUsuario, ManejadorRegistro-Tarjeta, InterfaceBaseDatosRegistro	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
2. Registrar Usuario	
crearRegistroUsuario() devuelve void	
validarRegistroUsuario(String log,String pass) devuelve boolean	InterfaceBaseDatosRegistro (1)
obtenerRegistroUsuario() devuelve void	InterfaceBaseDatosRegistro (1)
Responsabilidades Privadas	
administrarRegistroUsuario() devuelve void	
manejarEventoRegistrar() devuelve void	InterfaceBaseDatosRegistro (1)
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (1)
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (1)
manejarEventoRegistrarTarjeta() devuelve void	ManejadorRegistroTarjeta (2)

La clase *RegistroUsuario* incluye los atributos asignados durante la identificación del dominio del problema y descrita en los casos de uso, como se muestra en la tabla 8.166.

Tabla 8.166 Tarjeta para la clase *RegistroUsuario* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos para los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: RegistroUsuario	
Descripción: para utilizar el sistema de reservaciones, el usuario debe estar registrado en el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa y oficina, fax, email, login y password.	
Módulo: Registro.Usuario	
Estereotipo: Entidad	

continúa

Tabla 8.166 Continuación

Propiedades: Concreta	
Superclases: Datos	
Subclases:	
Atributos: login, password, nombre, apellido, dirección, colonia, ciudad, país, CP, telCasa, telOficina, fax, email, rpassword	

TARJETA

Los atributos para la clase *ManejadorRegistroTarjeta* se considerarán de manera similar al *ManejadorRegistroUsuario*. Se incluirán referencias a sus dos pantallas, *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta*, al *RegistroTarjeta* donde se guardará la información que utiliza el manejador, y a la *InterfaceBaseDatosRegistro* encargada de acceder la base de datos. La tarjeta especificada en la tabla 8.156 se describe con atributos en la tabla 8.167.

Tabla 8.167 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos identificados del caso de uso *Registrar Tarjeta*.

Clase: ManejadorRegistroTarjeta	
Descripción: el manejador de registro de tarjeta se encarga de lo relacionado con el registro de la tarjeta del usuario para pagar las reservaciones.	
Módulo: Registro.Tarjeta	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos: PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta, RegistroTarjeta, InterfaceBaseDatosRegistro	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
2. Registrar Tarjeta	
registrarTarjeta(String log) devuelve void	
Responsabilidades Privadas	
crearRegistroTarjeta() devuelve void	
obtenerRegistroTarjeta() devuelve void	InterfaceBaseDatosRegistro (2)
administrarRegistroTarjeta() devuelve void	
manejarEventoRegistrar() devuelve void	InterfaceBaseDatosRegistro (2)
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (2)
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (2)

La descripción de las clases *PantallaRegTarjeta*, *PantallaCrearRegTarjeta* y *PantallaObtenerRegTarjeta* son similares a las anteriores.

La clase *RegistroTarjeta* incluye atributos identificados durante la especificación del dominio del problema y descritos nuevamente en los casos de uso. Estos atributos se muestran en la tabla 8.168.

Tabla 8.168: Tarjeta para la clase <i>RegistroTarjeta</i> con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos para el caso de uso <i>RegistrarTarjeta</i> .	
Clase: RegistroTarjeta	
Descripción: para hacer un pago con una tarjeta de crédito, se debe tener un registro de tarjeta. El registro contiene información acerca de la tarjeta, incluyendo nombre, número, expedidor y vencimiento. La tarjeta está ligada a un registro de usuario.	
Módulo: Registro.Tarjeta	
Estereotipo: Entidad	
Propiedades: Concreta	
Superclases: Datos	
Subclases:	
Atributos: login, nombre, número, tipo, fecha	

INTERFACE BASE DATOS REGISTRO

La clase *InterfaceBaseDatosRegistro* no requiere atributos por el momento.

SERVICIOS

La clase *ManejadorServicio* mostrada en la tabla 8.160 requiere únicamente una referencia a la clase *ManejadorRegistroUsuario* para referirle solicitudes relacionadas con registro, como se muestra en la tabla 8.169.

Tabla 8.169: Tarjeta para la clase <i>ManejadorServicio</i> con responsabilidades, colaboraciones, jerarquías, contratos, protocolos y atributos a partir de los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: ManejadorServicio	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicios	
Estereotipo: Control	

continúa

Tabla 8.169 Continuación

Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
2. Ofrecer Servicio	
ofrecerServicio() devuelve void	
Responsabilidades Privadas	
manejarEventoRegistrar() devuelve void	SubsistemaRegistro (2)

La descripción de la clase *PantallaServicio* se mantiene igual a su descripción en la tabla 8.131, como se hizo con las demás pantallas.

8.2.9 Algoritmos

Los *algoritmos* definen la lógica utilizada por cada operación para resolver la responsabilidad a la que corresponden. En general, muchas responsabilidades son suficientemente simples y no requieren de algoritmos, como son las responsabilidades de delegar entre los objetos para consultar o modificar los valores de los atributos. Éste es el caso más común en este ejemplo del sistema de reservaciones de vuelos. Sin embargo, es especialmente importante especificar los algoritmos para implementar funciones de lógica más compleja, como ordenar un conjunto de números o cadenas, o hacer cálculos de intereses sobre cuentas bancarias. Los algoritmos se especifican de manera declarativa o de procedimiento, dependiendo de su complejidad. Cuando se especifica el algoritmo como de procedimiento, esto se hace mediante un diagrama de flujo. Por el contrario, un algoritmo especificado de manera declarativa, se hace mediante una especificación textual, en cuyo caso se usa algún conocimiento adicional para implementar el algoritmo. Vale la pena resaltar que los aspectos algorítmicos pueden llegar a ser una de las fuentes de mayor complejidad en un sistema.

A continuación se especifican estos algoritmos de manera declarativa, principalmente con un comentario sencillo.

INTERFACEUSUARIO

A partir de la tabla 8.161 se describe la clase *InterfaceUsuario* con algoritmos como se muestra en la tabla 8.170.

Hidden page

Tabla 8.171 Continuación

desplegarPantalla() devuelve void	
<i>Método encargado de desplegar la pantalla actual</i>	
Responsabilidades Privadas	
enviarEvento(Evento) devuelve void	InterfaceUsuario (2)
<i>Método encargado de recibir eventos del sistema de ventanas. Se envía el evento recibido a la InterfaceUsuario</i>	

PRINCIPAL

A partir de la tabla 8.163 se describe la clase *Manejador* con algoritmos como se muestra en la tabla 8.172.

Tabla 8.172 Tarjeta para la clase *Manejador* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados de los diversos manejadores para los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: Manejador	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Abstracta	
Superclases:	
Subclases: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta	
Atributos: InterfaceUsuario, Pantalla, ManejadorServicio, Manejador	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
Responsabilidades Privadas	
desplegarPantalla() devuelve void	SubsistemaInterfaceUsuario (1)
<i>Método encargado de desplegar las pantallas administradas por los manejadores. Se solicita al SubsistemaInterfaceUsuario que las despliegue</i>	
manejarEventoOfrecerServicio() devuelve void	SubsistemaServicio (2)
<i>Método encargado de solicitar al SubsistemaServicio que ofrezca los servicios correspondientes</i>	
manejarEventoSalir() devuelve void	
<i>Método encargado de salir del sistema</i>	

Hidden page

diagramas de flujos, se aprovecha el siguiente capítulo para describir el diseño hecho directamente con base en código de Java.

REGISTRO

En el módulo de *Registro*, compuesto de los módulos *Usuario*, *Tarjeta* e *InterfaceBD*, sólo se agregan especificaciones algorítmicas a las clases de control, pero no las de interface o entidad, como se verá a continuación.

USUARIO

A partir de la tabla 8.165 se describe la clase *ManejadorRegistroUsuario* con algoritmos como se muestra en la tabla 8.174.

Tabla 8.174 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos: PantallaCrearRegUsuario, PantallaObtenerRegUsuario, RegistroUsuario, ManejadorRegistroTarjeta, InterfaceBaseDatosRegistro	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método sobrescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
2. Registrar Usuario	
crearRegistroUsuario() devuelve void	
<i>Método encargado de desplegar una pantalla de creación de registro de usuario a través del contrato de "Registrar Usuario"</i>	
validarRegistroUsuario(String log,String pass) devuelve boolean	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la validación de un usuario a través del contrato de "Registrar Usuario"</i>	
obtenerRegistroUsuario() devuelve void	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la obtención de un RegistroUsuario a través del contrato de "Registrar Usuario"</i>	

continúa

Tabla 8.174 Continuación

Responsabilidades Privadas	
administrarRegistroUsuario() devuelve void	
<i>Método encargado de desplegar una pantalla de obtención de registro de usuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoRegistrar() devuelve void	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la creación de un nuevo RegistroUsuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la actualización de un RegistroUsuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la eliminación de un RegistroUsuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoRegistrarTarjeta() devuelve void	ManejadorRegistroTarjeta (2)
<i>Método encargado de solicitar al ManejadorRegistroTarjeta que procese el contrato "Registrar Tarjeta"</i>	

La descripción de las clases *PantallaRegUsuario*, *PantallaCrearRegUsuario*, *PantallaObtenerRegUsuario* y *RegistroUsuario* se mantienen igual.

TARJETA

A partir de la tabla 8.167 se describe la clase *ManejadorRegistroTarjeta* con algoritmos como se muestra en la tabla 8.175.

Tabla 8.175 Tarjeta para la clase *ManejadorRegistroTarjeta* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados del caso de uso *Registrar Tarjeta*.

Clase: ManejadorRegistroTarjeta
Descripción: el manejador de registro de tarjeta se encarga de lo relacionado con el registro de la tarjeta del usuario para pagar las reservaciones.
Módulo: Registro.Tarjeta
Esteriotipo: Control
Propiedades: Concreta
Superclases: Manejador
Subclases:

continúa

Tabla 8.175 Continuación

Atributos: PantallaCrearRegTarjeta, PantallaObtenerRegTarjeta, RegistroTarjeta, InterfaceRegistro

Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método sobrescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
2. Registrar Tarjeta	
RegistrarTarjeta(String log) devuelve void	
<i>Método encargado de crear u obtener un RegistroTarjeta</i>	
Responsabilidades Privadas	
crearRegistroTarjeta() devuelve void	
<i>Método encargado de desplegar la pantalla de creación de registro de tarjeta a través del contrato de "Registrar Tarjeta"</i>	
obtenerRegistroTarjeta() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la obtención de un RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
administrarRegistroTarjeta() devuelve void	
<i>Método encargado de desplegar una pantalla de obtención de registro de tarjeta a través del contrato de "Registrar Tarjeta"</i>	
manejarEventoRegistrar() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la creación de un nuevo RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la actualización de un RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la eliminación de un RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	

La descripción de las clases *PantallaRegTarjeta*, *PantallaCrearRegTarjeta*, *PantallaObtenerRegTarjeta* y *RegistroTarjeta* se mantienen igual.

INTERFACE BASE DATOS REGISTRO

A partir de la tabla 8.158 se describe la clase *InterfaceBaseDatosRegistro* con algoritmos como se muestra en la tabla 8.176.

Tabla 8.176 Tarjeta para la clase <i>InterfaceBaseDatosRegistro</i> con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados para los casos de uso <i>Validar Usuario</i> , <i>Ofrecer Servicios</i> , <i>Registrar Usuario</i> y <i>Registrar Tarjeta</i> .	
Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Registrar Usuario	
validarRegistro(RegistroUsuario, String log, String pass) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	
crearRegistro(RegistroUsuario) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario</i>	
obtenerRegistro(RegistroUsuario, String log) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario</i>	
actualizarRegistro(RegistroUsuario) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario</i>	
eliminarRegistro(RegistroUsuario) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario</i>	
2. Registrar Tarjeta	
crearRegistro(RegistroTarjeta) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo Registro Tarjeta</i>	

continúa

Tabla 8.176 Continúa

obtenerRegistro(RegistroTarjeta, String log) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroTarjeta</i>	
actualizarRegistro(RegistroTarjeta) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroTarjeta</i>	
eliminarRegistro(RegistroTarjeta) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroTarjeta</i>	

SERVICIOS

A partir de la tabla 8.169 se describe la clase *ManejadorServicio* con algoritmos como se muestra en la tabla 8.177.

Tabla 8.177 Tarjeta para la clase *ManejadorServicio* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorServicio	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicios	
Esteriotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos:	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método sobreescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
2. Ofrecer Servicio	
ofrecerServicio() devuelve void	
<i>Método encargado de hacer solicitudes de consultar, reservar y administración de registros</i>	
Responsabilidades Privadas	
manejarEventoRegistrar() devuelve void	SubsistemaRegistro (2)
<i>Método encargado de hacer la solicitud de administración de registros al SubsistemaRegistro</i>	

Hidden page

Aunque esta decisión se ha simplificado hoy en día gracias a lenguajes como Java, no todos los lenguajes de programación orientados a objetos implementan de la misma forma los diferentes conceptos de la orientación a objetos. Existen aspectos, como el manejo del encapsulamiento, referencias, herencia múltiple y otros aspectos que varían de manera importante entre lenguajes.

Como parte de la implementación de las clases en Java se definirán los siguientes aspectos:

- ▶ Encapsulamiento o *visibilidad* de los métodos tanto como los atributos mediante modificadores de tipo **public** en el caso de contratos y responsabilidades *públicas*, **private** en el caso de responsabilidades y atributos *privados*, y **protected** en el caso de responsabilidades y atributos *privados* definidos a nivel de una superclase, y que se necesiten acceder a nivel de sus subclases.
- ▶ Protocolos correspondientes a tipos primitivos existentes en Java, como *int*, *float*, *boolean*, y objetos con tipos predefinidos en Java, como *String*, *Vector*.

De manera general, la implementación se basará en definiciones y manejos estándares de Java.

8.3.2 Interfaces gráficas

Las interfaces gráficas tienen como objetivo esencial administrar la interacción con el usuario mediante elementos gráficos, como lo son los botones, menús y textos. En general, las aplicaciones interactivas donde el control del ratón y el teclado desempeñan un papel primordial de control, se conocen como sistemas controlados o dirigidos por *eventos*. Estos eventos corresponden al movimiento del ratón, como oprimir o soltar uno de sus botones, oprimir una tecla, junto con eventos que no son directamente iniciados por el usuario, como los de desplegar una pantalla o interrumpir un programa. Desarrollar un sistema dirigido por eventos significa que la aplicación desde un inicio debe considerar un diseño adecuado. Por ejemplo, en el caso de Java, se escoge inicialmente una de sus bibliotecas gráficas, como AWT o Swing, para luego utilizar el manejo apropiado a través de clases como *Frame*, *Canvas*, *Panel*, *Button*, *Event*, etc. Más allá de los elementos o clase particulares de estas bibliotecas, también se afecta la lógica de diseño, ya que se debe contemplar, en qué momento es apropiado procesar nuevos eventos y cómo se inicializará el sistema.

Para evitar mayor complejidad, nos limitaremos a ventanas relativamente sencillas, tanto a nivel de diseño gráfico como del tipo de elementos internos que se incorporarán. El diseño se basará en el prototipo gráfico descrito anteriormente en el capítulo 5, donde se creó un solo marco de ventana (*frame*), el cual muestra las diferentes pantallas en diferentes momentos, todas dentro del mismo marco. Tomada esta decisión, que obviamente no es la única alternativa, básicamente se tendrá una clase controladora única de la ventana, la clase *InterfaceUsuario* definida antes. Si hubieran múltiples ventanas independientes (en el "desktop") entonces se debería definir a cada marco como una clase controladora, correspondiente a cada pantalla, para luego tener una clase controladora para los múltiples marcos. Dada nuestra decisión, las pantallas se vuelven elementos internos de la ventana única y bajo el control de la *InterfaceUsuario*.

¿Cómo afecta esto al diseño de objetos? Se afecta de la siguiente manera. En lugar de que cada pantalla reciba un evento por parte del usuario (administrado a través del sistema de ventanas del sistema operativo), se hace que todos los eventos sean recibidos directamente por la *InterfaceUsuario*, volviendo a las pantallas más pasivas, con menor conocimiento de su entorno y más sencillas de manipular. En otras palabras, se convierten las pantallas en elementos exclusivamente visuales quitándole todo el conocimiento necesario para manejar eventos generados externamente.

Esto afecta directamente las responsabilidades de la clase *InterfaceUsuario* y de las pantallas. Los cambios principales serán en relación con el cliente del contrato "2" el cual será el manejador de ventanas del sistema en lugar de las pantallas. Esto significa que el protocolo de la responsabilidad *enviarEvento* correspondiente al contrato "2" de la clase *InterfaceUsuario* deberá modificarse, tomado de su estado definido en la tabla 8.170 y mostrado nuevamente en la tabla 8.178.

Tabla 8.178 Contrato *enviarEvento* para la clase *InterfaceUsuario* definido en la tabla 8.170.

2. Enviar Evento	
enviarEvento(Evento, Manejador) devuelve void	Manejador (1) : SubsistemaPrincipal (1), SubsistemaServicio (1), SubsistemaRegistro (1)

Dado que la llamada al contrato "2" es externa a nuestro sistema, no habrá conocimiento sobre el manejador que controla la pantalla actual. Por tanto, se omitirá el parámetro *Manejador* en el protocolo, como se muestra en la tabla 8.179.

Tabla 8.179 Contrato *enviarEvento* con protocolo revisado para la clase *InterfaceUsuario* definido en la tabla 8.168.

2. Enviar Evento	
enviarEvento(Evento) devuelve void	Manejador (1) : SubsistemaPrincipal (1), SubsistemaServicio (1), SubsistemaRegistro (1)

La clase *InterfaceUsuario* modificada se muestra en la tabla 8.180.

Tabla 8.180 Tarjeta para la clase *InterfaceUsuario* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y algoritmos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceUsuario
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.
Módulo: InterfaceUsuario
Estereotipo: Borde

continúa

Hidden page

Hidden page

da de una clase debe incluir un identificador para la **llave primaria**, y uno o más identificadores para la llave primaria de la tabla derivada de las asociaciones. La estrategia es compatible con la orientación a objetos, donde los objetos tienen una identidad aparte de sus propiedades. Existen beneficios en el uso de identificadores, ya que éstos son inmutables y completamente independientes a cambios en los valores de los datos y su lugar físico. Aunque cada clase puede corresponder a una o más tablas, por lo general se diseñan tablas que correspondan a un objeto completo. La tarea de diseño se simplifica, creando una tabla correspondiente a cada clase entidad del dominio del problema y manteniendo las asociaciones entre clases. Obviamente, el diseño de tablas puede optimizarse para un acceso más eficiente, algo que no se tratará aquí.

Dado que nuestra descripción se ha concentrado hasta el momento en lo referente a registro de usuario y tarjeta, nos limitaremos a mostrar el diseño de dichas tablas. Por ejemplo, en la tabla 8.182, se muestra el diseño para el *RegistroUsuario*, donde se define *login* como la llave primaria de la tabla. La primera fila representa los nombres de los campos, mientras que la segunda muestra un ejemplo de datos.

Tabla 8.182 Diseño de la base de datos para la tabla correspondiente a la clase *RegistroUsuario*.

login	password	nombre	apellido	dirección	Ciudad	país	CP	telCasa	telOf	fax	email
alfredo	awr	Alfredo	Weitzenfeld	Río Hondo	México DF	México	01000	1234	5678	9012	alfredo@itam.mx

En la tabla 8.183, se muestra el diseño de la tabla para el *RegistroTarjeta*, donde se incluye *login* como referencia a la llave primaria de la tabla de *RegistroUsuario*.

Tabla 8.183 Diseño de la base de datos para la tabla correspondiente a la clase *RegistroTarjeta*.

login	nombre	número	tipo	fecha
alfredo	Alfredo Weitzenfeld	1234-5678-9012	Visa	120205

En la figura 8.31 se muestra de manera esquemática la relación entre las tablas *RegistroUsuario* y *RegistroTarjeta*.

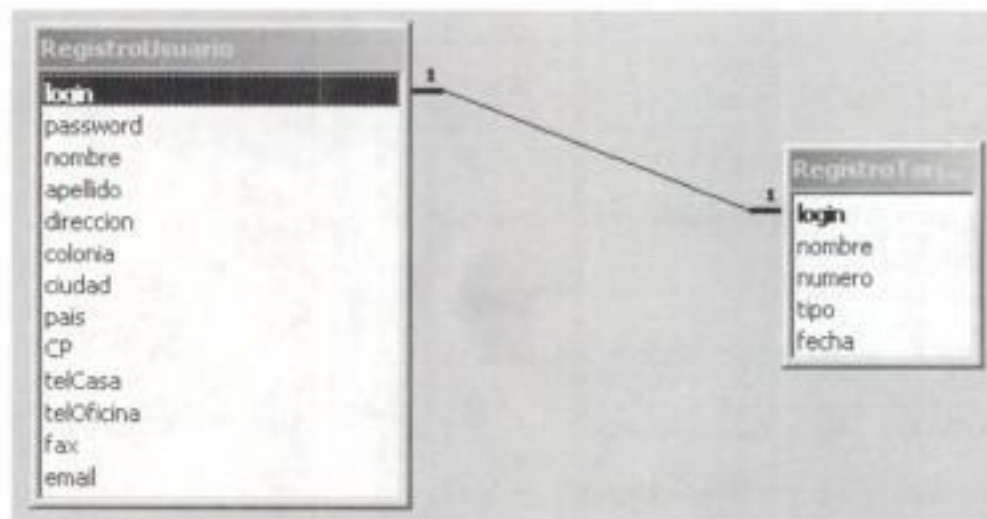


Figura 8.31 Diagrama que muestra la relación esquemática entre las tablas *RegistroUsuario* y *RegistroTarjeta*.

8.3.4 Archivos

Aunque es más efectivo trabajar con bases de datos, es posible utilizar archivos, sobre todo cuando la especificación del sistema así lo requiera. De tal forma, se aprovechará para extender el manejo de base de datos y mostrar cómo se logra un objetivo similar mediante archivos. Este ejemplo también servirá para mostrar el poder de la extensibilidad, definiendo una jerarquía de herencia a nivel de las clases borde para acceder tanto bases de datos como archivos de manera casi transparente. Para ello, se creará una nueva superclase *InterfaceRegistro*, de la cual heredará la clase *InterfaceBaseDatosRegistro* y la nueva clase *InterfaceArchivoRegistro*. La superclase se definirá como abstracta y deberá tener contratos y responsabilidades públicas similares a las ya definidas en la clase *InterfaceBaseDatosRegistro*. Esta clase se muestra en la tabla 8.184.

Tabla 8.184 Tarjeta para la clase *InterfaceRegistro* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y algoritmos para los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceRegistro
Descripción: superclase para las interfaces a base de datos de registro y archivos.
Módulo: Registro.InterfaceDB
Esteriotipo: Borde
Propiedades: Abstracta

continúa

Tabla 8.184 Continúa

Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Registrar Usuario	
validarRegistro(RegistroUsuario, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	BaseDatosRegistro
crearRegistro(RegistroUsuario) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario</i>	BaseDatosRegistro
obtenerRegistro(RegistroUsuario, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario</i>	BaseDatosRegistro
actualizarRegistro(RegistroUsuario) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario</i>	BaseDatosRegistro
eliminarRegistro(RegistroUsuario) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario</i>	BaseDatosRegistro
2. Registrar Tarjeta	
crearRegistro(RegistroTarjeta) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroTarjeta</i>	BaseDatosRegistro
obtenerRegistro(RegistroTarjeta, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroTarjeta</i>	BaseDatosRegistro
actualizarRegistro(RegistroTarjeta) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroTarjeta</i>	BaseDatosRegistro
eliminarRegistro(RegistroTarjeta) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroTarjeta</i>	BaseDatosRegistro

En el caso de la base de datos, la clase *InterfaceBaseDatosRegistro* se comunica con el DBMS (manejador del sistema de la base de datos) para hacer solicitudes a cualquier tabla. Sin embargo, en el caso de los archivos, tal manejador no existe y será necesario hacerlo de manera manual. Por tal motivo, además de la clase *InterfaceArchivoRegistro*, se creará otra nueva clase *ArchivoRegistro* que se encargará de administrar los diferentes archivos, instanciando una por archivo manipulado. De tal manera, se modificarán las colaboraciones en la *InterfaceArchivoRegistro* para hacerlas con la clase *ArchivoRegistro* en lugar de *BaseDatosRegistro*, como se muestra en la tabla 8.185. Observe que todos los contratos deben ser idénticos a los de la superclase *InterfaceRegistro* al igual que *InterfaceBaseDatosRegistro* para lograr la sobreescritura y aprovechar el polimorfismo.

Tabla 8.185 Tarjeta para la clase *InterfaceArchivoRegistro* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y algoritmos para los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceArchivoRegistro	
Descripción: la información de cada usuario se almacena en los archivos de registro que se accesa mediante la interface de archivo de registro. Esto permite validar a los usuarios además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Registrar Usuario	
validarRegistro(RegistroUsuario, String log, String pass) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	
crearRegistro(RegistroUsuario) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario</i>	
obtenerRegistro(RegistroUsuario, String log) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario</i>	
actualizarRegistro(RegistroUsuario) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario</i>	

continúa

Tabla 8.185 Continuación

eliminarRegistro(RegistroUsuario) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario</i>	
2. Registrar Tarjeta	
crearRegistro(RegistroTarjeta) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroTarjeta</i>	
obtenerRegistro(RegistroTarjeta, String log) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroTarjeta</i>	
actualizarRegistro(RegistroTarjeta) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroTarjeta</i>	
eliminarRegistro(RegistroTarjeta) devuelve boolean	ArchivoRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroTarjeta</i>	

La clase *ArchivoRegistro* de se muestra en la tabla 8.186.

Tabla 8.186 Tarjeta para la clase *InterfaceArchivoRegistro* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y algoritmos para los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ArchivoRegistro	
Descripción: la formación de cada registro de usuario se almacena en un archivo que es leído por el sistema. Esto permite a la clase <i>InterfaceArchivoRegistro</i> administrar todos los registros correspondientes.	
Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Registrar Usuario	
validarRegistro(RegistroUsuario, String log, String pass) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	

continúa

Hidden page

De manera similar, el archivo correspondiente al *RegistroTarjeta* se muestra a continuación,

```
2
alfredo|Alfredo Weitzenfeld|123456789|MasterCard|01/05|
reservas|Alfredo Weitzenfeld|987654321|Visa|02/4|
```

Observe que la asociación entre ambos archivos es mediante el campo de "login", el primer campo en ambos archivos.

8.4 Revisión del diseño

Como parte de la revisión final, se decidió hacer ciertas optimizaciones en el diseño. Una de estas optimizaciones es a nivel de acceso a la base de datos de registro. Se observa que el *ManejadorRegistroUsuario* tiene a la *InterfaceBaseDatosRegistro* como colaborador durante la validación de un usuario, al igual que durante la obtención de un registro. Sin embargo, en lugar de hacer dos accesos, primero una validación de usuario y luego obtener el registro de la base de datos, se incorporan ambas responsabilidades en una sola. Por tanto, al momento de validar un registro, se obtiene el registro de usuario de manera inmediata. En la tabla 8.187 se muestra la clase *ManejadorRegistroUsuario* descrita en la tabla 8.174. Note que se eliminó la colaboración de *obtenerRegistroUsuario* con la *InterfaceBaseDatosRegistro*, además de modificar la descripción del método.

Tabla 8.187 Tarjeta para la clase *ManejadorRegistroUsuario* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ManejadorRegistroUsuario	
Descripción: el manejador de registro de usuario se encarga de lo relacionado con el registro del usuario para utilizar el sistema.	
Módulo: Registro.Usuario	
Estereotipo: Control	
Propiedades: Concreta	
Superclases: Manejador	
Subclases:	
Atributos: PantallaCrearRegUsuario, PantallaObtenerRegUsuario, RegistroUsuario, ManejadorRegistroTarjeta, InterfaceBaseDatosRegistro	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método sobrescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	

continúa

Tabla 8.187 Continuación

2. Registrar Usuario	
crearRegistroUsuario() devuelve void	
<i>Método encargado de desplegar una pantalla de creación de registro de usuario a través del contrato de "Registrar Usuario"</i>	
validarRegistroUsuario(String log,String pass) devuelve boolean	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la validación de un usuario a través del contrato de "Registrar Usuario"</i>	
obtenerRegistroUsuario() devuelve void	
<i>Método encargado de obtener un RegistroUsuario ya validado a través del contrato de "Registrar Usuario"</i>	
Responsabilidades privadas	
administrarRegistroUsuario() devuelve void	
<i>Método encargado de desplegar una pantalla de obtención de registro de usuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoRegistrar() devuelve void	InterfaceceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la creación de un nuevo RegistroUsuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la actualización de un RegistroUsuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (1)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la eliminación de un RegistroUsuario a través del contrato de "Registrar Usuario"</i>	
manejarEventoRegistrarTarjeta() devuelve void	ManejadorRegistroTarjeta (2)
<i>Método encargado de solicitar al ManejadorRegistroTarjeta que procese el contrato "Registrar Tarjeta"</i>	

De manera adicional, se aprovechará para hacer una revisión en el manejo de las clases entidad en relación con las clases borde, en particular en relación con las interfaces de bases de datos y archivo. El manejo consiste en generalizar a la clase entidad mediante su superclase para reducir el número de contratos a uno solo. En otras palabras, en lugar de dos contratos, "Registrar Usuario" y "Registrar Tarjeta", se tiene uno solo como se verá a continuación. En la tabla 8.188 se muestra la tarjeta de clase para InterfaceRegistro, mostrada en la tabla 8.184, unificando bajo un solo contrato el acceso a las bases de datos o archivos mediante un parámetro generalizado de tipo *Datos*.

Hidden page

Tabla 8.189 Continuación

Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Concreta	
Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	ArchivoRegistro
crearRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro
obtenerRegistro(Datos, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro
actualizarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro
eliminarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro

De manera similar, se toma la tarjeta para la clase *ArchivoRegistro*, mostrada en la tabla 8.186, en la cual se integran ambos contratos en uno solo, como aparece en la tabla 8.190.

Tabla 8.190 Tarjeta para la clase *ArchivoRegistro* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y algoritmos para los casos de uso *Validar Usuario*, *Ofrecer Servicio*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: ArchivoRegistro
Descripción: la información de cada registro de usuario se almacena en un archivo que es leído por el sistema. Esto permite a la clase <i>InterfaceArchivoRegistro</i> administrar todos los registros correspondientes.
Módulo: Registro.InterfaceDB
Estereotipo: Borde
Propiedades: Concreta

continúa

Tabla 8.190 Continuación

Superclases:	
Subclases:	
Atributos:	
Contratos	
1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	
crearRegistro(Datos) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	
obtenerRegistro(Datos, String log) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	
actualizarRegistro(Datos) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	
eliminarRegistro(Datos) devuelve boolean	BaseDatosRegistro
<i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	

A partir de la tabla 8.176 se describe la clase *InterfaceBaseDatosRegistro* actualizada como se muestra en la tabla 8.191.

Tabla 8.191 Tarjeta para la clase *InterfaceBaseDatosRegistro* con responsabilidades, colaboraciones, jerarquías, contratos, protocolos, atributos y especificación de algoritmos identificados para los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*.

Clase: InterfaceBaseDatosRegistro
Descripción: la información de cada usuario se almacena en la base de datos de registro que se accesa mediante la interface de la base de datos de registro. Esto permite validar a los usuarios, además de guardar información acerca de la tarjeta de crédito para pagos en línea.
Módulo: Registro.InterfaceDB
Estereotipo: Borde
Propiedades: Concreta
Superclases:

continúa

Hidden page

Hidden page

8.5.2 Registrar Usuario: Actualizar Registro Usuario

El diagrama de secuencia *Actualizar Registro Usuario* para los casos de uso *Validar Usuario*, *Ofrecer Servicios* y *Registrar Usuario*, subflujo *Actualizar Registro Usuario* se muestra en la figura 8.33.

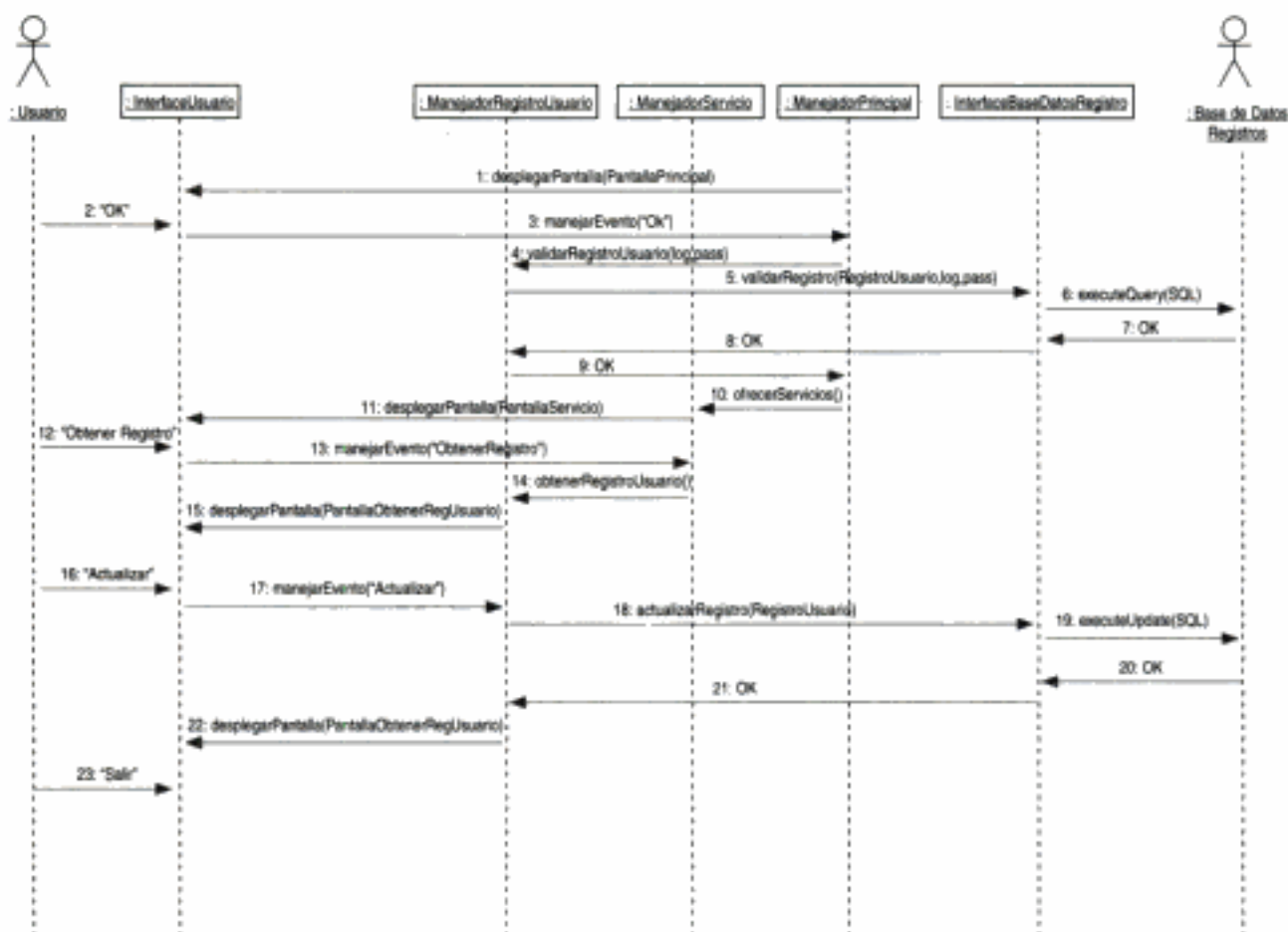


Figura 8.33 Diagrama de secuencias *Actualizar Registro Usuario* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios* y *Registrar Usuario*, subflujo *Actualizar Registro Usuario*.

8.5.3 Registrar Usuario: Eliminar Registro Usuario

El diagrama de secuencia *Eliminar Registro Usuario* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios* y *Registrar Usuario*, subflujo *Eliminar Registro Usuario* se muestra en la figura 8.34.

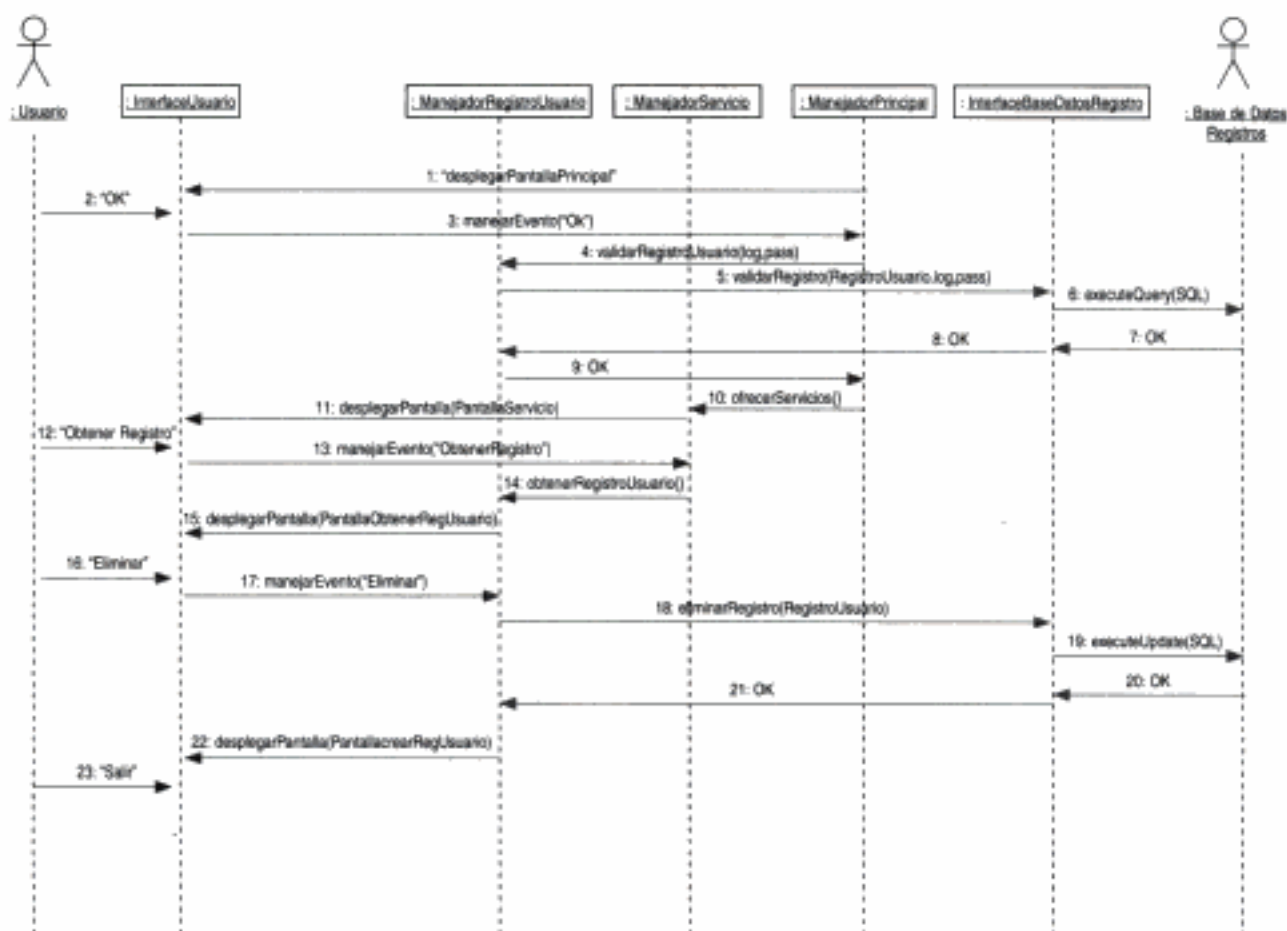


Figura 8.34 Diagrama de secuencias *Eliminar Registro Usuario* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios* y *Registrar Usuario*, subflujo *Eliminar Registro Usuario*.

8.5.4 Registrar Tarjeta: Crear Registro Tarjeta

El diagrama de secuencia *Crear Registro Tarjeta* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*, subflujo *Crear Registro Tarjeta* se muestra en la figura 8.35.

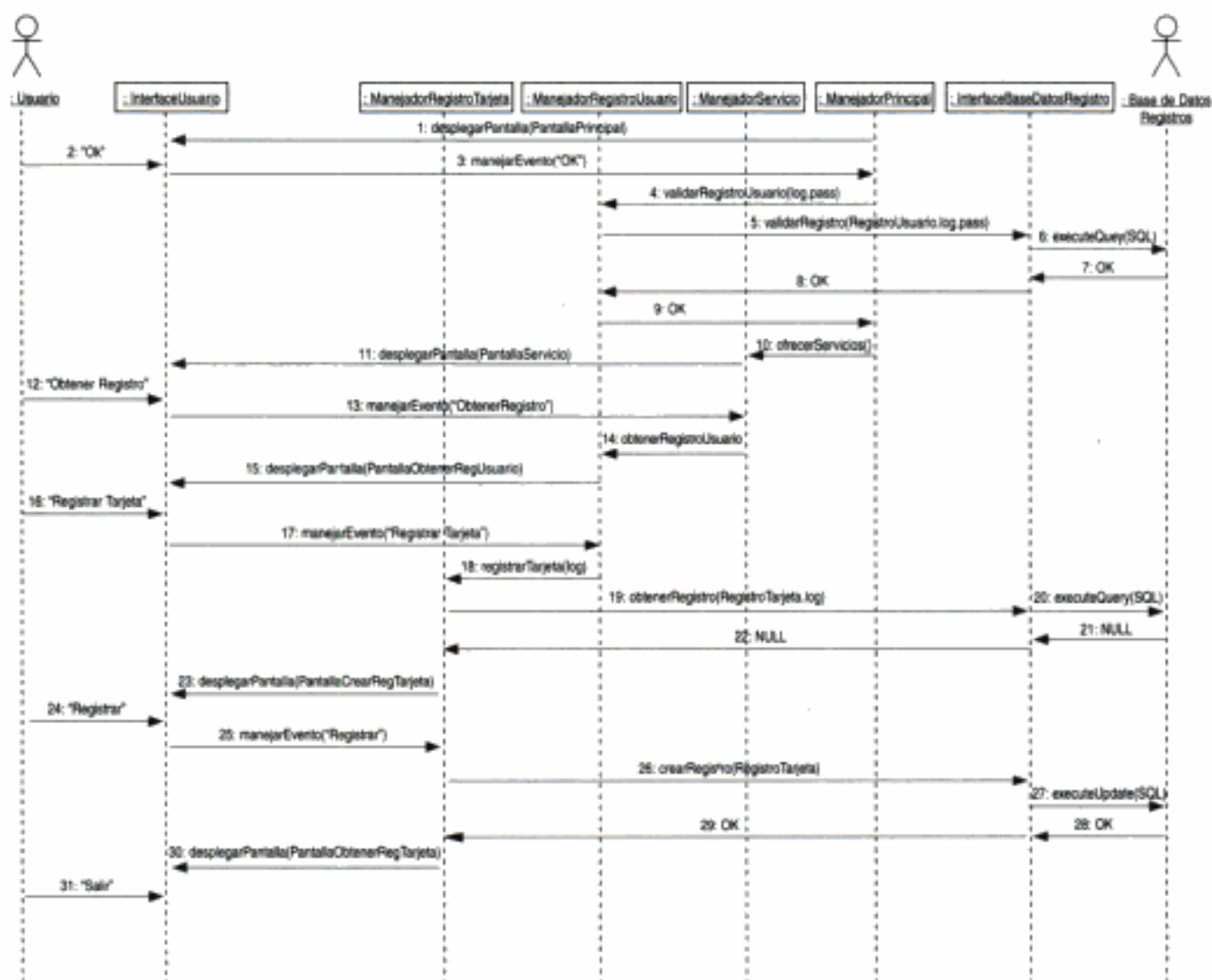


Figura 8.35 Diagrama de secuencias *Crear Registro Tarjeta* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*, subflujo *Crear Registro Tarjeta*.

Hidden page

8.5.6 Registrar Tarjeta: Eliminar Registro Tarjeta

El diagrama de secuencia *Eliminar Registro Tarjeta* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*, subflujo *Eliminar Registro Tarjeta* se muestra en la figura 8.37.

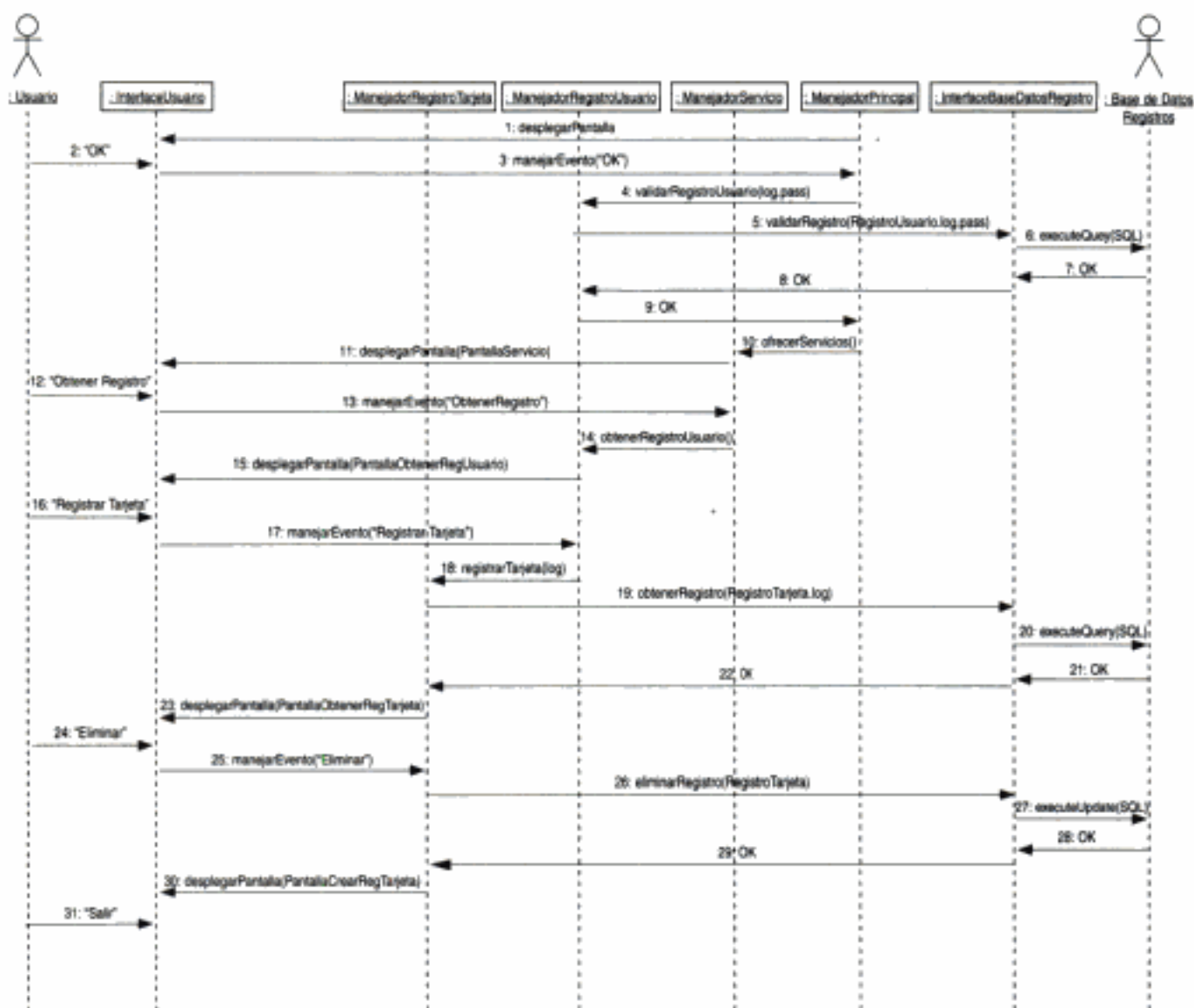


Figura 8.37 Diagrama de secuencias *Eliminar Registro Tarjeta* a partir de los casos de uso *Validar Usuario*, *Ofrecer Servicios*, *Registrar Usuario* y *Registrar Tarjeta*, subflujo *Eliminar Registro Tarjeta*.

RESUMEN

En este capítulo se describe el modelo de diseño y se explican diversas estrategias para elaborarlos. Se comienza con el diseño de objetos en el cual se detalla la arquitectura de clases especificada en el modelo de análisis. Luego, se introduce el concepto de tarjetas de clases utilizadas para describir los aspectos más relevantes, correspondientes a sus responsabilidades, colaboraciones, contratos, jerarquías, subsistemas, protocolos, atributos y algoritmos. Se describe el diseño de sistema que identifica el ambiente de implementación deseado. Se integra con el diseño de objetos para terminar con una revisión del diseño completo. Se muestran los diagramas de secuencias del modelo de diseño.

REFERENCIAS

1. Wirfs-Brock, R., Wilkerson, B., Wiener, L., 1990, *Designing Object-Oriented Software*, Prentice-Hall.

Modelo de implementación

El modelo de implementación toma el resultado del modelo de diseño para generar el código final. Se extiende el diseño por responsabilidades del capítulo anterior.¹ Esta traducción debe ser relativamente sencilla y directa, ya que las decisiones mayores han sido tomadas durante las etapas previas. Durante el modelo de implementación se adapta al lenguaje de programación y/o la base de datos, según la especificación del diseño y las propiedades del lenguaje de implementación y base de datos. Aunque el diseño de objetos es bastante independiente del lenguaje actual, todos los lenguajes tienen sus particularidades, las cuales deben adecuarse durante la implementación final. La elección del lenguaje influye en el diseño, pero éste no debe depender de los detalles de aquél. Si se cambia de lenguaje de programación, no debe requerirse el rediseño del sistema.

En general, no se debe comenzar a programar de manera prematura. Primero es importante completar el proceso de planeación del sistema final desarrollado durante el diseño. Se deben usar guías de programación existentes en la organización. Si no se cuenta con ellas, el equipo de software deben crear sus propias guías para decidir aspectos tales como formatos para la asignación de nombres a las variables, estilo de programación, métodos de documentación y documentación en línea. Vale la pena resaltar que aunque existe cierta automatización del proceso de generación del código final, en su gran mayoría los programadores hacen de manera "manual" la transición final a código fuente.

9.1 Programación en Java

En esta sección tomamos la especificación del diseño presentado en el capítulo 8 y generamos la programación, en este caso, en Java.

9.1.1 *InterfaceUsuario*

Comenzamos la implementación de la clase *InterfaceUsuario*, a partir de la tarjeta de clase que se muestra nuevamente en la tabla 9.1.

Tabla 9.1 Tarjeta para la clase *InterfaceUsuario*.

Clase: InterfaceUsuario	
Descripción: toda la interacción con el usuario se hace por medio de la interface de usuario.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Concreta	
Superclase:	
Subclase:	
Atributos: Manejador, Pantalla	
Contratos	
1. Desplegar Pantalla	
desplegarPantalla(Pantalla) devuelve void <i>Método encargado de desplegar las pantallas enviadas como parámetros. Se delega el despliegue particular a cada pantalla</i>	Pantalla (1) : PantallaPrincipal (1), PantallaServicio (1), PantallaCrearRegUsuario (1), PantallaObtenerRegUsuario (1), PantallaCrearRegTarjeta (1), PantallaObtenerRegTarjeta (1)
2. Enviar Evento	
enviarEvento(Evento) devuelve void <i>Método encargado de recibir eventos del sistema de ventanas. Se envía el evento recibido a los distintos manejadores</i>	Manejador (1) : SubsistemaPrincipal (1), SubsistemaServicio (1), SubsistemaRegistro (1)

La implementación de la clase *InterfaceUsuario*, que se hará a partir de la biblioteca AWT ("java.awt") de Java, se basa en la descripción de manejo de ventanas realizada en el capítulo 5. En nuestro manejo de ventanas heredamos la clase *InterfaceUsuario* de la clase *Frame*, de Java, para generar una sola ventana, la cual mostrará diferentes pantallas dentro de su marco pero en diferentes momentos. Esta clase implementa los manejadores de eventos de ventana y acciones, como ya se describió y se describirá a continuación:

```
public class InterfaceUsuario extends Frame
    implements WindowListener, ActionListener
```

Los atributos de la clase son de tipo *Manejador* y *Pantalla*, como se dijo anteriormente. Asignamos como nombre de las variables los mismos tipos pero en minúscula y privados por ser atributos.

```
private Manejador manejador;
private Pantalla pantalla;
```

Los métodos que se sobrescribirán de estos manejadores de eventos fueron también descritos anteriormente. En el caso de eventos de ventanas se sobres-

criben ("implements") los métodos mencionados en la *interface* llamada *WindowListener*.

```
public void windowClosed(WindowEvent event) {}
public void windowDeiconified(WindowEvent event) {}

public void windowIconified(WindowEvent event) {}
public void windowActivated(WindowEvent event) {}
public void windowDeactivated(WindowEvent event) {}
public void windowOpened(WindowEvent event) {}
public void windowClosing(WindowEvent event) {
    System.exit(0);
}
```

En el caso de eventos relacionados con acciones (botones) se debe sobrescribir un solo método de la *interface* *actionListener*, el método *actionPerformed*, el cual es el punto de entrada de los eventos provenientes del *Usuario* que son administrados por el manejador de ventanas del sistema. Recordemos que la clase *InterfaceUsuario* define un contrato "2" llamado *enviarEvento*, el cual debe recibir los eventos relacionados con los botones para luego enviárselos a los distintos manejadores. Este método *enviarEvento* corresponde a *actionPerformed* y debe reescribirse con dicho nombre, como se muestra a continuación. El parámetro, definido originalmente como *Evento*, corresponde ahora a *ActionEvent*, de Java.

```
public void actionPerformed(ActionEvent event)
```

Aprovechando que estamos en la etapa de definición del método correspondiente al contrato "2", veamos cómo debe implementarse. El método, como se especificó anteriormente, tiene como responsabilidad reenviar el evento a los diversos manejadores. Aunque podríamos reenviar el mismo tipo de evento *ActionEvent* a los manejadores, de alguna manera limitaríamos la extensibilidad del sistema, ya que este tipo podría cambiar en el futuro, lo cual dependerá de las bibliotecas particulares utilizadas. En su lugar, podríamos definir nuestro propio tipo de evento para comunicación interna del sistema o, en el caso de eventos sencillos, como los que manejaremos aquí, simplemente enviar un *String* correspondiente al nombre del botón presionado, el cual podemos obtener mediante la llamada *event.getActionCommand()*. La llamada a los manejadores se debe hacer a través del contrato "1", "Manejar Evento", específicamente la responsabilidad *manejarEvento*, la cual fue definida para recibir un parámetro de tipo *String*. Este método es sobrescrito por todos los manejadores, pues la llamada se hace a partir de la referencia genérica de *manejador*. La implementación del método se muestra a continuación. La llamada se hace dentro de un "if" para asegurar que no exista una referencia nula.

```
if (manejador != null)
    manejador.manejarEvento(event.getActionCommand());
```

Ya definido el manejo del evento, tanto de ventana como de acciones (contrato "2"), continuamos con el despliegue de pantallas correspondiente al contrato "1", "Desplegar Pantallas", el cual tiene un solo método definido, *desplegarPantalla*, que tiene como parámetro el tipo general *Pantalla*, como se muestra a continuación:

```
public void desplegarPantalla(Pantalla p)
```

De manera similar al contrato "1" de "Manejar Evento", éste tiene como responsabilidad solicitar a las diversas pantallas que se desplieguen. Sin embargo, antes de desplegar la siguiente pantalla, debemos borrar la actual, lo cual se logra mediante un nuevo método que definiremos en la clase *Pantalla*, la que se encarga de borrar los elementos particulares de la pantalla. Aunque podríamos hacerlo a nivel general de la *InterfaceUsuario*, existen elementos conocidos por cada pantalla, como los botones, que en el caso de la biblioteca AWT es importante deshabilitar antes de proseguir con el agregado de nuevos botones a una misma ventana. Al nuevo método lo denominaremos *borrarPantalla* y lo llamaremos a partir de la referencia genérica de *pantalla*, pero es necesario verificar que la referencia no sea nula, como se muestra a continuación:

```
if (pantalla != null)
    pantalla.borrarPantalla();
```

En este punto ya estamos listos para desplegar nuestra nueva pantalla. Como paso preliminar asignamos el valor de la referencia de "p" enviada como parámetro a la referencia local *pantalla*.

```
if (p != null)
    pantalla = p;
```

Luego de ello, estamos en condiciones de desplegar las nuevas pantallas mediante la solicitud al contrato "1" de las diversas pantallas, correspondiente a la responsabilidad *desplegarPantalla*. Como hicimos anteriormente, revisamos que el valor de *pantalla* no sea nulo y hacemos la llamada de despliegue.

```
if (pantalla != null)
    pantalla.desplegarPantalla();
```

Finalmente, debemos pedir al manejador de ventanas que muestre la nueva pantalla, algo que se hace mediante la llamada de *show* definida en la clase *Frame*, superclase de *InterfaceUsuario*.

```
show();
```

Por último, es necesario definir el constructor de la clase. La definición es bastante similar a la descrita en el capítulo 5. Sin embargo, la diferencia principal radica en que el constructor es llamado por la clase *ManejadorPrincipal* como parte del proceso de inicialización, por lo cual se debe agregar el parámetro correspondiente en el constructor, como se muestra a continuación:

```
public InterfaceUsuario(Manejador m);
```

El cuerpo del constructor es similar al que se describió en el capítulo 5, donde simplemente asignamos el parámetro "m" a la referencia local *manejador*.

```
setSize(800,600);
setBackground(Color.lightGray);
addWindowListener(this);
manejador = m;
```

Como podemos observar, la definición de *InterfaceUsuario* se basa en lo descrito en el capítulo 5.

La tarjeta para la clase *Pantalla* se muestra nuevamente en la tabla 9.2.

Tabla 9.2 Tarjeta para la superclase *Pantalla*.

Clase: Pantalla	
Descripción: Pantalla heredada por las demás clases de tipo pantalla.	
Módulo: InterfaceUsuario	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclase:	
Subclase: PantallaPrincipal, PantallaServicio, PantallaRegUsuario, PantallaRegTarjeta	
Atributos: InterfaceUsuario, Manejador	
Contratos	
1. Desplegar Pantalla	
desplegarPantalla() devuelve void	
<i>Método encargado de desplegar la pantalla actual</i>	

La clase *Pantalla* se define como una clase abstracta. Dado que la pantalla existe dentro de un marco (*InterfaceUsuario* hereda de *Frame*), no es necesario agregar ninguna herencia a esta clase.

```
public abstract class Pantalla
```

Los atributos de la clase son de tipo *InterfaceUsuario* y *Manejador*, como se definió anteriormente. Asignamos como nombre de las variables los mismos tipos, pero en minúscula y privados, ya que minimizaremos su efecto sobre la superclase *Pantalla* y no sobre cada una de ellas por separado. Si este procedimiento no funciona, deberemos cambiar la visibilidad a *protected*.

```
private InterfaceUsuario interfaceUsuario;
private Manejador manejador;
```

Como parte de la implementación, es necesario incorporar el ejemplo de pantallas e *Interface Usuario* desarrollado en el capítulo 5. Definimos cuatro vectores correspondientes a todos los elementos que queremos administrar dentro de cada pantalla: paneles, botones, textos y etiquetas. Para cada uno de ellos definimos una variable temporal, las cuales utilizaremos como referencias temporales de los objetos instanciados.

```
protected Vector paneles,botones,textos,etiquetas;
protected Panel panel;
protected Button boton;
protected TextField texto;
protected Label etiqueta;
```

El constructor de la pantalla debe recibir las referencias de la *InterfaceUsuario* y del manejador que acaba de instanciar la pantalla, así como permitir iniciali-

zar y crear los elementos internos de la ventana, lo cual llevamos a cabo a través de los métodos `inicializarPantalla` y `crearPantalla`, respectivamente.

```
public Pantalla(InterfaceUsuario ui, Manejador m) {
    interfaceUsuario = ui;
    manejador = m;
    inicializarPantalla();
    crearPantalla();
}
```

El método `inicializarPantalla` se encarga de inicializar los vectores como se muestra a continuación. Definimos tanto los métodos locales a las pantallas como aquellos que son llamados por la clase *InterfaceUsuario* como protegidos, como se mostró en el capítulo 5.

```
protected void inicializarPantalla() {
    paneles = new Vector();
    botones = new Vector();
    textos = new Vector();
    etiquetas = new Vector();
}
```

El método `crearPantalla` se define como abstracto y como protegido.

```
protected abstract void crearPantalla();
```

Otro método que ya ha sido mencionado y que se define también como protegido es `borrarPantalla`, el cual está encargado de borrar los elementos de una pantalla en el momento en que se despliega una nueva.

```
protected void borrarPantalla() {
    interfaceUsuario.removeAll();
    int bs = botones.size();
    for (int i = 0; i < bs; i++)
        if ((boton = (Button)botones.elementAt(i)) != null)
            boton.removeActionListener(interfaceUsuario);
}
```

Aprovecharemos para describir otros dos métodos que muchas clases utilizan para agregar botones de "Salir" y de "Servicios". El método que agrega el botón de "Salir" es el siguiente:

```
protected void agregarBotonesSalir(Panel panel){
    boton = new Button ("Salir");
    panel.add(boton);
    botones.addElement(boton);
    paneles.addElement(panel);
}
```

El que agrega tanto el botón de "Servicios" como el de "Salir" es el siguiente:

```
protected void agregarBotonesServiciosSalir(Panel panel){
    boton = new Button ("Servicios");
    botones.addElement(boton);
    panel.add(boton);
    agregarBotonesSalir(panel);
}
```

El contrato "1", "Desplegar Pantalla", consta de la responsabilidad `desplegarPantalla`, la cual también fue definida y explicada en el capítulo 5. La mostramos nuevamente, esta vez como parte del contrato. También la definimos como protegida, debido a que es llamada por la `InterfaceUsuario`, la cual está definida como parte del mismo paquete.

```
protected void desplegarPantalla() {
    int ps = paneles.size();
    interfaceUsuario.setLayout(new GridLayout(ps,1));
    for (int i = 0; i < ps; i++)
        interfaceUsuario.add((Panel)paneles.elementAt(i));
    int bs = botones.size();
    for (int i = 0; i < bs; i++)
        if ((boton = (Button)botones.elementAt(i)) != null)
            boton.addActionListener(interfaceUsuario);
}
```

9.1.2 Principal

A continuación se describe la clase *Manejador*, como se muestra nuevamente en la tabla 9.3.

Tabla 9.3 Tarjeta para la clase <i>Manejador</i> .	
Clase: Manejador	
Descripción: pantalla heredada por las demás clases de tipo pantalla.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Abstracta	
Superclase:	
Subclase: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta	
Atributos: InterfaceUsuario, Pantalla, ManejadorServicio, Manejador	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
Responsabilidades Privadas	
desplegarPantalla() devuelve void	SubsistemaInterfaceUsuario (1)
<i>Método encargado de desplegar las pantallas administradas por los manejadores. Se solicita al SubsistemaInterfaceUsuario que las despliegue</i>	
manejarEventoOfrecerServicio() devuelve void	SubsistemaServicio (2)
<i>Método encargado de solicitar al SubsistemaServicio que ofrezca los servicios correspondientes</i>	
manejarEventoSalir() devuelve void	
<i>Método encargado de salir del sistema</i>	

Hidden page

Hidden page

Tabla 9.4 Continuación

manejarEvento(Evento) devuelve void <i>Método sobrescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
Responsabilidades Privadas	
manejarEventoRegistrar() devuelve void <i>Método encargado de solicitar al SubsistemaRegistro que dé servicio al contrato de "Registrar Usuario"</i>	SubsistemaRegistro (2)
manejarEventoValidar() devuelve void <i>Método encargado de solicitar al SubsistemaServicio que dé servicio al contrato de "Ofrecer Servicio"</i>	SubsistemaRegistro (2)

La clase `ManejadorPrincipal` es una subclase de `Manejador`, por lo cual debe definir la extensión correspondiente.

```
public class ManejadorPrincipal extends Manejador
```

Los atributos de la clase `ManejadorPrincipal` son `PantallaPrincipal`, `ManejadorServicio` y `ManejadorRegistroUsuario`. Como veremos más adelante, los diversos manejadores hacen referencia a estos últimos dos manejadores, por lo cual aprovechamos para definirlos en la superclase `Manejador` en lugar de estar redefiniéndolos en cada subclase especializada. De tal manera aprovechamos el reuso de código a través de la herencia.

```
private Pantalla pantallaPrincipal;  
private ManejadorServicio ms;  
private ManejadorRegistroUsuario mru;
```

Dado que el `ManejadorPrincipal` es el encargado de inicializar la aplicación, definimos el método estático `main` como parte de la definición de esta clase.

```
public static void main(String[] args) {  
    ManejadorPrincipal m = new ManejadorPrincipal();  
}
```

El constructor de la clase `ManejadorPrincipal` instancia a la `InterfaceUsuario` y también a los manejadores con los cuales se comunicará, el `ManejadorServicio` y el `ManejadorRegistroUsuario`. Al objeto referido por `interfaceUsuario`, definido en la clase `Manejador`, mediante un "this" le pasamos una referencia local del `ManejadorPrincipal`. Al objeto referido por `ms`, correspondiente a un objeto de tipo `ManejadorServicio` y también definido en la superclase `Manejador`, se le pasa la referencia local "this" y también una referencia `interfaceUsuario`, para que luego pueda comunicarse en el momento de desplegar pantallas. Hacemos algo similar con el `ManejadorRegistroUsuario` a través de la referencia `mru`. Esta referencia, que aún no ha sido declarada, puede agregarse a esta clase o directamente a la superclase `Manejador`. Dado que también el `ManejadorServicio` necesitará de una referencia al `ManejadorRegistroUsuario`, aprovechamos para

Hidden page

Hidden page

La clase *PantallaPrincipal* hereda de la superclase *Pantalla*.

```
public class PantallaPrincipal extends Pantalla
```

El constructor de la clase pasa los parámetros de instanciación, *ui* y *m*, a la superclase, mediante el método *super*. Los parámetros corresponden a las referencias que debe mantener toda pantalla, una a la *InterfaceUsuario* y otra al *Manejador* que administra la pantalla.

```
public PantallaPrincipal (InterfaceUsuario ui, Manejador m) {  
    super(ui,m);  
}
```

Aunque podríamos haber creado los elementos internos de la pantalla dentro del constructor, preferimos hacerlo a través de un método adicional, *crearPantalla*, lo cual le otorga más flexibilidad al código, ya que podemos generar los eventos en el momento que deseemos y no necesariamente durante la instanciación del objeto. Sólo mostramos parte del código interno del método, ya que el detalle fue explicado en el capítulo 5.

```
protected void crearPantalla() {  
    panel = new Panel();  
    panel.setLayout(new GridLayout(3,1));  
    panel.add(new Label("SISTEMA DE RESERVACIONES DE VUELO",  
        Label.CENTER));  
    panel.add(new Label("Pantalla Principal (P-1)", Label.CENTER));  
    paneles.addElement(panel);  
    ...  
    panel = new Panel();  
    panel.add(new Label("Login:", Label.LEFT));  
    texto = new TextField(20);  
    texto.setName("login");  
    textos.addElement(texto);  
    panel.add(texto);  
    paneles.addElement(panel);  
  
    panel = new Panel();  
    panel.add(new Label("Password:"));  
    texto = new TextField(20);  
    texto.setName("password");  
    texto.setEchoChar('#');  
    textos.addElement(texto);  
    panel.add(texto);  
    paneles.addElement(panel);  
    ...  
}
```

Vale la pena resaltar ciertos cambios en el manejo de los elementos gráficos que aún no han sido comentados, los cuales están relacionados principalmente con el manejo de los campos de texto, ya que de allí obtendremos información insertada por el usuario. Para lograr un manejo generalizado de esta información, debemos agregar un nombre particular a cada campo de texto. En el caso de "login", sería:

```
texto.setName("login");
```

Una vez creado el texto, lo agregamos a la lista de textos, de manera similar a los paneles y botones:

```
textos.addElement(texto);
```

En el caso de "password", agregamos la opción de cambiar el carácter de despliegue a un "#", mediante:

```
texto.setEchoChar('#');
```

De los cambios anteriores, lo más importante para nuestro diseño es que se agregó un nombre para cada campo de texto, los cuales luego utilizaremos para identificar campos en las pantallas y así obtener de manera generalizada su información o escribir en ella, aspecto que veremos más adelante.

9.1.3 Dominio

Aunque podríamos definir los diversos atributos de cada clase entidad en las clases correspondientes, haremos un pequeño rediseño generalizando un poco la especificación de estos atributos a partir de la clase *Datos*. Para ello definiremos una colección de objetos que guarde internamente tanto el nombre del atributo como su valor correspondiente. En el caso de los nombres, la colección debe guardar tipos *String*, mientras que en el caso de los valores pueden corresponder a cualquier tipo de objeto. Para nuestro ejemplo también utilizaremos valores de tipo cadenas, aunque en general este recurso restringe, en alguna medida, el manejo de los datos. En todo caso, es sencillo extender este diseño a los demás tipos, como *Integer*, etc. Antes de describir la clase *Datos*, aplicaremos una nueva a la cual llamaremos *Atributo*, analizada en la tabla 9.6.

Tabla 9.6 Superclase entidad *Atributo*.

Clase: Atributo	
Descripción: clase entidad que contiene los atributos de cualquier clase entidad que herede de <i>Datos</i> .	
Módulo: Dominio	
Estereotipo: Entidad	
Propiedades: Concreta	
Superclase:	
Subclase:	
Atributos: nombre, valor	

Esta clase, que nos permite administrar los atributos de cualquier clase entidad que herede de *Datos* de manera homogénea, se describe de la siguiente manera:

```
public class Atributo
```

Definimos dos atributos, uno correspondiente al nombre del atributo, y el otro a su valor, en este caso también de tipo cadena:

```
private String nombre;
private String valor;
```

En el momento de instanciar el atributo, se le debe asignar un nombre y un valor iniciales:

```
public Atributo(String str, String v) {
    nombre = str;
    valor = v;
}
```

El nombre del atributo lo leemos a través del siguiente método:

```
public String leerNombre() {
    return nombre;
}
```

El valor del atributo lo leemos a través del siguiente método:

```
public String leerValor() {
    return valor;
}
```

Es importante poder modificar los valores del atributo, para lo cual definimos el siguiente método:

```
public void escribirValor(String v) {
    valor = v;
}
```

Finalmente, podemos agregar un método que imprima tanto el nombre como el valor del atributo, tal como:

```
public void print() {
    System.out.print(nombre+": ");
    System.out.println(valor);
}
```

La descripción de la clase *Datos* se muestra nuevamente en la tabla 9.7.

Tabla 9.7 Superclase entidad <i>Datos</i> .	
Clase: Datos	
Descripción: superclase para todas las clases entidad.	
Módulo: Dominio	
Estereotipo: Entidad	
Propiedades: Abstracta	
Superclase:	
Subclase: RegistroUsuario, RegistroTarjeta	
Atributos:	

La definición de la clase es la siguiente:

```
public class Datos
```

Definimos una colección con base en la clase *Vector* de Java, lo cual es bastante sencillo y flexible. Junto con la lista, a la cual llamaremos *listaAtributos*, definiremos un *numAtributosBD* correspondiente a los atributos que deberán guardarse en la base de datos, ya que no es necesario guardarlos a todos, *atributo* que será utilizado como variable temporal. Esto se declara de la siguiente manera:

```
protected Vector listaAtributos;  
protected int numAtributosBD;  
protected Atributo atributo;
```

El constructor de la clase *Datos* debe inicializar el vector.

```
public Datos() {  
    listaAtributos = new Vector;  
}
```

Luego necesitamos agregar un grupo de métodos para manipular la información. El primero será *agregarAtributo*, el cual será llamado por las subclases cuando deseen incluir atributos en su definición. Para ello, se pasa el nombre del atributo como primer parámetro, segundo parámetro correspondiente a su valor, y un tercero de tipo *boolean* si se desea que ese valor se guarde o no en la base de datos. Nótese que únicamente en el caso de que la bandera *fg* sea verdadera, se incrementará *numAtributosBD*.

```
protected void agregarAtributo(String nombre,  
    String valor, boolean fg) {  
    atributo = new Atributo(nombre,valor);  
    listaAtributos.addElement(atributo);  
    if (fg == true)  
        numAtributosBD++;  
}
```

Para leer los nombres o valores guardados en los vectores, definimos los siguientes dos métodos, *leerNombre* y *leerValor*. Dado que los vectores se accesan como arreglos, el parámetro que se pasa es el índice del atributo. Nótese que primero es necesario obtener el atributo para luego lograr su nombre. El método *leerNombre* se describe a continuación,

```
public String leerNombre(int i) {  
    String str = null;  
    atributo = (Atributo) listaAtributos.elementAt(i);  
    if (atributo != null)  
        str = atributo.leerNombre();  
    return str;  
}
```

El método *leerValor* se describe a continuación,

```
public String leerValor(int i) {  
    String str = null;  
    atributo = (Atributo) listaAtributos.elementAt(i);
```

Hidden page

Hidden page

La clase *ManejadorRegistroUsuario* hereda de la superclase *Manejador* y se define de la siguiente manera:

```
public class ManejadorRegistroUsuario extends Manejador
```

Los atributos que definiremos para la clase son los siguientes:

```
private Pantalla pantallaCrearRegUsuario;  
private Pantalla pantallaObtenerRegUsuario;  
private RegistroUsuario registroUsuario;  
private ManejadorRegistroTarjeta mrt;  
private InterfaceBaseDatosRegistro interfaceRegistro;
```

El constructor se encarga de instanciar los diversos objetos. Las dos pantallas se instanciarán de manera dinámica, igual que el *ManejadorRegistroTarjeta*.

```
public ManejadorRegistroUsuario(Manejador m, InterfaceUsuario ui) {  
    super(m, ui);  
    registroUsuario = new RegistroUsuario();  
    interfaceRegistro = new InterfaceBaseDatosRegistro();  
}
```

El contrato "1", "Manejar Evento", debe contemplar los diversos botones que aparecen en las pantallas administradas por este manejador. El caso de "Registrar" llama al método *manejarEventoRegistrar*, "Actualizar" llama al método *manejarEventoActualizar*, "Eliminar" llama al método *manejarEventoEliminar*, "Registrar Tarjeta" llama al método *manejarEventoRegistrarTarjeta*, mientras que los botones "Servicio" y "Salir" son administrados por la superclase *Manejador* a través del método *manejarEventosAdicionales*. Estos métodos los describiremos con mayores detalles en breve.

```
public void manejarEvento(String str) {  
    if (str.equals("Registrar"))  
        manejarEventoRegistrar();  
    else if (str.equals("Actualizar"))  
        manejarEventoActualizar();  
    else if (str.equals("Eliminar"))  
        manejarEventoEliminar();  
    else if (str.equals("Registrar Tarjeta"))  
        manejarEventoRegistrarTarjeta();  
    else  
        manejarEventosAdicionales(str);  
}
```

El contrato "2", "Registrar Usuario", consta de tres responsabilidades: *crearRegistroUsuario*, *validarRegistroUsuario* y *obtenerRegistroUsuario*. El método *crearRegistroUsuario* se encarga de desplegar la pantalla *pantallaCrearRegUsuario*. Antes de solicitar su despliegue se debe verificar que la pantalla existe.

```
public void crearRegistroUsuario() {  
    if (pantallaCrearRegUsuario == null)  
        pantallaCrearRegUsuario =  
            new PantallaCrearRegUsuario(interfaceUsuario, this);  
    desplegarPantalla(pantallaCrearRegUsuario);  
}
```

El método `validarRegistroUsuario`, que recibe dos parámetros del usuario, "login" y "password", se encarga de solicitar a la clase *InterfaceBaseDatos-Registro*, a través de la referencia `interfaceRegistro`, que valide al usuario. El resultado debe ser un "verdadero" o "falso" ("true" o "false"). Nótese que estamos enviando un objeto de tipo `RegistroUsuario` como parte de la llamada a la clase *InterfaceBaseDatosRegistro*, lo cual realmente no es necesario, pero lo hacemos para aprovechar la llamada de validación a la base de datos y obtener, junto con la validación, la información del usuario. De esta manera evitamos tener que hacer una segunda llamada para obtener los datos del usuario.

```
public boolean validarRegistroUsuario(String log, String pass) {
    if (registroUsuario == null)
        registroUsuario = new RegistroUsuario();
    return
        interfaceRegistro.validarRegistro(registroUsuario, log, pass);
}
```

El método `obtenerRegistroUsuario` se encarga de revisar que exista ya un registro de usuario para luego llamar al método `administrarRegistroUsuario`. Nótese que este método originalmente accedía a la base de datos, pero aprovechando la obtención del registro del usuario durante la validación evitamos este doble acceso.

```
public void obtenerRegistroUsuario() {
    if (registroUsuario != null)
        administrarRegistroUsuario();
}
```

El método `administrarRegistroUsuario` se encarga de desplegar la pantalla `pantallaObtenerRegUsuario`. Se escriben los datos del *RegistroUsuario* en la pantalla mediante el método `escribirElementos` definido en la superclase *Manejador*. Continuaremos con el resto de los métodos de la clase *ManejadorRegistro-Usuario* antes de explicar cómo escribir y leer datos de las pantallas.

```
private void administrarRegistroUsuario() {
    if (pantallaObtenerRegUsuario == null)
        pantallaObtenerRegUsuario =
            new PantallaObtenerRegUsuario(interfaceUsuario, this);
    escribirElementos(pantallaObtenerRegUsuario, registroUsuario);
    desplegarPantalla(pantallaObtenerRegUsuario);
}
```

El método `escribirElementos`, que definimos en la clase *Manejador* (aunque lo mostramos recién aquí), se encarga de solicitar a la pantalla que actualice sus campos de textos según los datos enviados. Más adelante explicaremos los detalles de esta escritura.

```
protected void escribirElementos(Pantalla p, Datos datos) {
    p.escribirElementos(datos);
}
```

El método `manejarEventoRegistrar` verifica que se haya instanciado un *Registro-Usuario* para luego guardar allí los datos insertados por el usuario en la pantalla mediante el método `leerElementos`, definido en la superclase *Manejador* (aná-

logo a *escribirElementos*). Una vez obtenidos los datos y guardados en el *RegistroUsuario*, son enviados a la *InterfaceBaseDatosRegistro*, a través del método *crearRegistro*, para ser guardados en la base de datos. Finalmente, aprovechamos el método *administrarRegistroUsuario* para desplegar la *Pantalla-ObtenerRegUsuario* con los nuevos datos.

```
private void manejarEventoRegistrar() {
    if (registroUsuario == null)
        registroUsuario = new RegistroUsuario();
    leerElementos(pantalla, registroUsuario);
    interfaceRegistro.crearRegistro(registroUsuario);
    administrarRegistroUsuario();
}
```

El método *leerElementos*, definido en la clase *Manejador* se encarga de solicitar a la pantalla copiar los campos en el objeto de tipo *Datos*. Más adelante explicaremos los detalles de esta lectura.

```
protected void leerElementos(Pantalla p, Datos datos) {
    p.leerElementos(datos);
}
```

El método *manejarEventoActualizar* se comporta de manera similar a *registrar-Usuario* en el sentido que se leen los elementos de la pantalla para luego actualizar la base de datos mediante el método *actualizarRegistro* (antes se creaba el registro). Se podría agregar la llamada *obtenerRegistroUsuario* al final, pero dado que ya está en la pantalla *PantallaObtenerRegUsuario* no hay necesidad de hacerlo.

```
private void manejarEventoActualizar() {
    if (registroUsuario == null)
        registroUsuario = new RegistroUsuario();
    leerElementos(pantalla, registroUsuario);
    interfaceRegistro.actualizarRegistro(registroUsuario);
}
```

El método *manejarEventoEliminar* toma un *RegistroUsuario* ya existente, actualmente desplegado en la pantalla, y solicita a la *InterfaceBaseDatosRegistro* su eliminación mediante el método *eliminarRegistro*. Una vez eliminado el registro se regresa al flujo correspondiente a la creación de un nuevo registro mediante la llamada *crearRegistroUsuario*.

```
private void manejarEventoEliminar() {
    if (registroUsuario != null) {
        interfaceRegistro.eliminarRegistro(registroUsuario);
        crearRegistroUsuario();
    }
}
```

Finalmente, el método *manejarEventoRegistrarTarjeta* se encarga de solicitar al *ManejadorRegistroTarjeta* que procese el contrato "Registrar Tarjeta", correspondiente al método *registrarTarjeta* de este último. Como parte de este método, primero se instancia el nuevo manejador para luego obtener un identificador correspondiente al usuario actual, tarea que se lleva a cabo mediante la llamada *leerValor(0)* del *RegistroUsuario*. El "0" corresponde al primer atributo del registro, en otras palabras, el "login".

```
private void manejarEventoRegistrarTarjeta() {
    if (registroUsuario != null) {
        if (mrt == null)
            mrt = new ManejadorRegistroTarjeta(this, interfaceUsuario);
        String id = registroUsuario.leerValor(0);
        mrt.registrarTarjeta(id);
    }
}
```

La descripción de las clases *PantallaRegUsuario*, *PantallaCrearRegUsuario* y *PantallaObtenerRegUsuario* es muy similar a la *PantallaPrincipal* anteriormente descrita.

Antes de proseguir, es importante añadir los métodos faltantes de la clase *Pantalla* (descrita antes de manera parcial). Los métodos aún pendientes de describir son *leerTexto*, *leerElementos* y *escribirElementos*.

El método *leerTexto* se muestra a continuación:

```
public String leerTexto(String name0) {
    String name = null, str = null;
    for (int j = 0; name0.equals(name) == false && j < textos.size();
        j++) {
        name = ((Component) textos.elementAt(j)).getName();
        if (name0.equals(name) == true)
            str = ((TextField) textos.elementAt(j)).getText();
    }
    return str;
}
```

El método recibe el nombre de *name0*, correspondiente al campo que se desea leer de la pantalla. Recordemos que cada campo de texto en la pantalla fue asignado con un nombre. El ciclo "for" compara este nombre con la variable *name* mediante la llamada *name0.equals(name) == false*. Cuando estas dos variables son iguales el ciclo termina. También puede terminar cuando se han revisado todos los campos de texto en la pantalla, especificado mediante *textos.size()*. La variable *name* lee los diferentes nombres asignados a los campos de la pantalla, algo que se hace mediante la llamada

```
name = ((Component) textos.elementAt(j)).getName();
```

Se hace nuevamente comparación de nombres:

```
if (name0.equals(name) == true)
```

Si los nombres son iguales, se prosigue leyendo el dato insertado por el usuario:

```
str = ((TextField) textos.elementAt(j)).getText();
```

Dado que los campos coinciden, el ciclo "for" termina sale del método y regresa el valor de *str* correspondiente al dato insertado por el usuario en el campo correspondiente.

El método anterior, diseñado para leer el dato correspondiente a un solo campo de la pantalla, se extiende en el método *leerElementos* para leer todos los cam-

Hidden page

```

        name = null;
        for (int j = 0; name0.equals(name) == false && j <
            textos.size(); j++) {
            name = ((Component) textos.elementAt(j)).getName();
            if (name0.equals(name) == true)
                ((TextField) textos.elementAt(j)).setText(str);
        }
    }
}

```

Ambos ciclos son similares, salvo dos modificaciones en la lógica. La primera es que el valor de `str` se obtiene del atributo en lugar del campo de la pantalla:

```
str = (String)datos.leerValor(i);
```

El segundo cambio es que se escribe del atributo a la pantalla en lugar de la forma en que se hacía antes:

```
((TextField) textos.elementAt(j)).setText(str);
```

Recuérdese que estos tres métodos son parte de la definición de la clase *Pantalla*. Ahora continuamos con la clase de registro de usuario.

La clase *RegistroUsuario* se muestra en la tabla 9.9.

Tabla 9.9 Tarjeta para la clase *RegistroUsuario*.

Clase: RegistroUsuario	
Descripción: para utilizar el sistema de reservaciones, el usuario debe estar registrado en el sistema. El registro contiene información acerca del usuario que incluye nombre, dirección, colonia, ciudad, país, código postal, teléfono de casa y oficina, fax, email, login y password.	
Módulo: Registro.Usuario	
Estereotipo: Entidad	
Propiedades: Concreta	
Superclase: Datos	
Subclase:	
Atributos: login, password, nombre, apellido, dirección, colonia, ciudad, país, CP, telCasa, telOficina, fax, email, rpassword	

La clase *RegistroUsuario* hereda de la superclase *Datos*.

```
public class RegistroUsuario extends Datos
```

Dentro del constructor de la clase inicializamos los atributos mediante llamadas a `agregarAtributo` definidos en la superclase *Datos*. El primer parámetro es el nombre del atributo, en el segundo inicializamos su valor, en este caso con cadenas vacías, mientras que el tercero corresponde a la bandera que especifica si el atributo deberá ("true") o no ("false") ser guardado en la base de datos durante el procesamiento correspondiente.

Hidden page

Tabla 9.10 Continuación

manejarEvento(Evento) devuelve void	
<i>Método sobrescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
2. Registrar Tarjeta	
registrarTarjeta(String log) devuelve void	
<i>Método encargado de crear u obtener un RegistroTarjeta</i>	
Responsabilidades Privadas	
crearRegistroTarjeta() devuelve void	
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la creación de un nuevo RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
obtenerRegistroTarjeta() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la obtención de un RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
administrarRegistroTarjeta() devuelve void	
<i>Método encargado de desplegar una pantalla de obtención de registro de tarjeta a través del contrato de "Registrar Tarjeta"</i>	
manejarEventoRegistrar() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la creación de un nuevo RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
manejarEventoActualizar() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatosRegistro la actualización de un RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	
manejarEventoEliminar() devuelve void	InterfaceBaseDatosRegistro (2)
<i>Método encargado de solicitar a la InterfaceBaseDatos-Registro la eliminación de un RegistroTarjeta a través del contrato de "Registrar Tarjeta"</i>	

La clase ManejadorRegistroTarjeta hereda de la superclase Manejador.

```
public class ManejadorRegistroTarjeta extends Manejador
```

Se definen los atributos especificados anteriormente, y agregamos uno nuevo, `idRegistro`, de tipo *String*, para guardar la referencia al "login" del usuario dueño del registro de tarjeta manipulado. Este nuevo atributo facilitará el manejo local de la información.

Hidden page

Hidden page

Hidden page

Tabla 9.12 Tarjeta de la clase *InterfaceRegistro*.

Clase: InterfaceRegistro	
Descripción: superclase para las interfaces con base en datos de registro y archivos.	
Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Abstracta	
Superclase:	
Subclase:	
Atributos:	
Contratos	
1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	BaseDatosRegistro
crearRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
obtenerRegistro(Datos, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
actualizarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
eliminarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro

Definimos la superclase *InterfaceRegistro* de la siguiente manera

```
public abstract class InterfaceRegistro
```

Definimos un solo grupo de métodos para ambos contratos, pero además generalizamos el parámetro a *Datos* y especificamos todos los métodos para que devuelvan un tipo booleano,

```
public abstract boolean crearRegistro(Datos reg);
public abstract boolean actualizarRegistro(Datos reg);
public abstract boolean eliminarRegistro(Datos reg);
public abstract boolean validarRegistro(Datos reg,String log,String
    pass);
public abstract boolean obtenerRegistro(Datos reg,String log);
```

Agregamos un método que necesitaremos para lograr un manejo genérico de los diferentes objetos entidad. Este método se encargará de recibir como parámetro un objeto especializado y devolver el nombre como `String` de su clase. Luego utilizaremos este nombre para instanciar nuevos objetos de esta clase de manera anónima, tarea que en Java es muy fácil.

```
public String getClassName(Datos reg){
    String regname;
    Class regclass = reg.getClass();
    String fullname = regclass.getName();
    int index = fullname.lastIndexOf('.');
    if (index > 0)
        regname = fullname.substring(index+1);
    else
        regname = fullname;
    return regname;
}
```

Se obtiene la clase de un objeto mediante la siguiente llamada:

```
Class regclass = reg.getClass();
```

Luego obtenemos el nombre como cadena de esta clase:

```
String fullname = regclass.getName();
```

Sin embargo, este nombre incluye el prefijo de todos los paquetes. Si sólo deseamos obtener el nombre propio de la clase sin información sobre sus paquetes, lo que debemos hacer es obtener la posición del último "." en la cadena

```
int index = fullname.lastIndexOf('.');
```

Finalmente buscamos el nombre a partir del último ".":

```
regname = fullname.substring(index+1);
```

En el caso de que no incluya el nombre ningún paquete, debemos devolverlo completo,

```
regname = fullname;
```

La clase *InterfaceBaseDatosRegistro* se muestra en la tabla 9.13.

Tabla 9.13 Tarjeta para la clase *InterfaceBaseDatosRegistro*.

Clase: <code>InterfaceBaseDatosRegistro</code>
Descripción: la información de cada usuario se almacena en la base de datos de registro, la cual se accesa mediante la interface de la base de datos de registro. Esto permite validar a los distintos usuarios además de guardar información sobre la tarjeta de crédito para pagos en línea.
Módulo: <code>Registro.InterfaceDB</code>
Estereotipo: Borde

continúa

Tabla 9.13 Continuación

Propiedades: Concreta	
Superclase:	
Subclase:	
Atributos:	
Contratos	
1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	BaseDatosRegistro
crearRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
obtenerRegistro(Datos, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
actualizarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
eliminarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro

La clase *InterfaceBaseDatosRegistro* se define a continuación:

```
public class InterfaceBaseDatosRegistro extends InterfaceRegistro
```

Se declaran tres atributos, como se explicó en el capítulo 5. La variable de tipo *Connection* se utiliza para hacer la conexión a la base de datos, la variable de tipo *Statement* para enviar la llamada de SQL a la base de datos, y la variable de tipo *ResultSet* para obtener los resultados de la llamada a la base de datos.

```
private Connection con;  
private Statement stmt;  
private ResultSet rs;
```

El constructor de la clase revisa mediante *revisarDriverSun* que exista la biblioteca con los "drivers" necesarios. Si es así, se solicita abrir la conexión mediante la llamada *abrirConexion*. Los parámetros enviados a esta última llamada, en nuestro caso, son el nombre de la base de datos para poder identificarla en el sistema, el "login" y "password" si es que éstos han sido habilitados.

Para una aplicación de un solo usuario se puede abrir la conexión a la base de datos al inicio de la aplicación, como lo hacemos en nuestro ejemplo, para luego cerrarla al finalizarla. En el caso de aplicaciones con múltiples usuarios donde pueden existir accesos concurrentes a la base de datos, estas conexiones deben hacerse de manera dinámica, mediante la apertura y cierre de las co-

Hidden page

Hidden page

```

private boolean actualizarRecordSetRegistro(String query){
    try {
        int n = stmt.executeUpdate (query);
        return true;
    }
    catch (SQLException ex) {
        ...
    }
    return false;
}

```

Este método consiste principalmente en la llamada *stmt.executeUpdate*, la cual recibe el "query" en forma de *String* y devuelve un entero correspondiente al número de récords que fueron insertados o actualizados correctamente.

El método para actualizar información *actualizarRegistro* de cierta tabla es muy similar al anterior, con la diferencia de que se basa en un dato existente.

```

public boolean actualizarRegistro(Datos reg){
    String log = reg.leerValor(0);
    String regname = getClass().getName(reg);
    String textsql = reg.serializarSQL();
    String str = "UPDATE " + regname + " SET " + textsql +
        " WHERE Login = '" + log + "'";
    return actualizarRecordSetRegistro(str);
}

```

El método recibe un registro, *reg*, del cual obtiene el nombre de su clase, el que nuevamente debe corresponder con el nombre de la tabla correspondiente en la base de datos. Así se generará nuevamente una llamada en SQL, sólo que algo diferente de la anterior. De nueva cuenta aprovechamos que todas las clases entidad heredan de *Datos* para especificar un método *serializarSQL* a nivel de la superclase, que se encargará de generar el formato de texto deseado por SQL. La llamada es la siguiente:

```
String textsql = reg.serializarSQL();
```

Por ejemplo, el texto devuelto por *serializarSQL* a partir de un nuevo registro de usuario sería similar al siguiente:

```

login = 'alfredo', password = 'awr', nombre = 'Alfredo', apellido =
'Weitzenfeld', direccion = 'Rio Hondo #1', colonia = 'San Angel
Tizapán', ciudad = 'Mexico DF', país = 'Mexico', CP = '01000', telCasa =
'56284000', telOficina = '56284000 x3614', fax = '56162211', email =
'alfredo@itam.mx'

```

Una vez obtenido el texto con los datos de la llamada, se prosigue hasta completar la llamada

```

String str = "UPDATE " + regname + " SET " + textsql +
    " WHERE Login = '" + log + "'";

```

En este caso el nombre de la tabla será *RegistroUsuario*, similar al de la clase, y se formaría la siguiente llamada de SQL completa:

```

UPDATE RegistroUsuario SET login = 'alfredo', password = 'awr', nombre =
'Alfredo', apellido = 'Weitzenfeld', direccion = 'Rio Hondo #1', colonia

```

Hidden page

Hidden page

Al finalizar la lectura, se actualizan los campos que no se hayan leído, es decir:

```
datos.actualizarAtributos();
```

En el caso de que en la consulta se obtuvieran como resultado múltiples objetos, es necesario extender el manejo de las llamadas anteriores de manera adecuada.

El método para obtener información `obtenerRegistro` de cierta tabla es similar al anterior, aunque más sencillo, ya que no incluye la validación de la contraseña pero sí la información sobre el usuario, como se muestra a continuación:

```
public boolean obtenerRegistro(Datos reg,String log){
    String regname = getClassName(reg);
    String querysql = "SELECT * FROM " + regname +
        " WHERE (login = '" + log + "')";
    return leerRecordSetRegistro(querysql,reg);
}
```

La clase *InterfaceArchivoRegistro* se muestra en la tabla 9.14.

Tabla 9.14 Tarjeta para la clase *InterfaceArchivoRegistro*.

Clase: InterfaceArchivoRegistro	
Descripción: la información de cada usuario se almacena en la archivos de registro que se accesa mediante la interface de archivo de registro. Esto permite validar a los usuarios además de guardar información acerca de la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceDB	
Estereotipo: Borde	
Propiedades: Concreta	
Superclase:	
Subclase:	
Atributos:	
Contratos	
1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	ArchivoRegistro
crearRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro
obtenerRegistro(Datos, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro

continúa

Tabla 9.14 Continuación

actualizarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro
eliminarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	ArchivoRegistro

La clase *InterfaceArchivoRegistro* hereda de la superclase *InterfaceRegistro*.

```
public class InterfaceArchivoRegistro extends InterfaceRegistro
```

Definimos un número de atributos privados, como son la dirección de la ubicación de los archivos, junto con variables temporales que serán descritas más adelante.

```
private String path = "reservaciones/baseDatos";
private Datos reg;
private String regname;
private File freg, ftar;
private Vector archivoRegistro;
private ArchivoRegistro ar;
```

El constructor se encarga de inicializar la lectura de los archivos. Aquí haremos algo que no es muy eficiente, pero en el caso de archivos pequeños no significa gran costo. Nos referimos a leer todos los archivos completos a memoria para administrar la información más fácilmente. Recuérdese que esta tarea sería prohibitiva en el caso de grandes archivos, los cuales habría que leer en secciones limitadas según las necesidades particulares del momento.

En este caso, dado que se trata de dos archivos relacionados con registro, *RegistroUsuario.dat* y *RegistroTarjeta.dat*, correspondientes a las dos tablas anteriormente descritas, lo que haremos será leerlos inicialmente a memoria, como se describe continuación:

```
public InterfaceArchivoRegistro() {
    archivoRegistro = new Vector();
    reg = new RegistroUsuario();
    regname = getClassName(reg);
    ar = new ArchivoRegistro(path, reg, regname);
    archivoRegistro.addElement(ar);
    reg = new RegistroTarjeta();
    regname = getClassName(reg);
    ar = new ArchivoRegistro(path, reg, regname);
    archivoRegistro.addElement(ar);
}
```

Al principio, se instancia un objeto de tipo *Vector*, llamado *archivoRegistro*, donde guardaremos objetos de tipo *ArchivoRegistro*, cada uno de ellos correspondiente a un tipo de datos de registro. A continuación se crea un objeto de tipo *RegistroUsuario*. Luego, se crea un objeto *ArchivoRegistro*, como se muestra a continuación:


```
ar = new ArchivoRegistro("reservaciones/baseDatos", reg,
    "RegistroUsuario");
```

Este método lee del archivo y carga sus contenidos a memoria, como veremos más adelante cuando describamos esta clase. Por cada archivo que se lee, agregamos una referencia al vector *archivoRegistro*.

Hacemos lo mismo con el archivo que guarda información de *RegistroTarjeta*.

El método *obtenerRegistro* es el encargado de obtener datos sobre un usuario, tarea que realiza, en primer lugar, mediante la obtención del nombre de la clase, como se hizo anteriormente.

```
public boolean obtenerRegistro(Datos reg, String log){
    String regname = getClassName(reg);
    for (int i = 0; i < archivoRegistro.size(); i++) {
        ar = (ArchivoRegistro) archivoRegistro.elementAt(i);
        if (ar != null && regname.equals(ar.getName()) == true)
            return ar.leerRegistro(reg, log);
    }
    return false;
}
```

Se realiza una búsqueda de los diferentes archivos leídos en memoria, lo cual depende del tipo de información que se busca. El ciclo "for" pasa por todos estos objetos, dos en nuestro caso (*RegistroUsuario* y *RegistroTarjeta*) y compara su nombre con el tipo deseado:

```
if (ar != null && regname.equals(ar.getName()) == true)
```

Una vez que concuerdan el nombre de la clase con el del tipo de archivo en memoria, se copian los datos de éste al registro mediante la llamada al método *leerRegistro* de la siguiente manera:

```
return ar.leerRegistro(reg, log);
```

El método *crearRegistro* es similar al anterior y se encarga, en primer término, de buscar el archivo correcto para luego hacer la llamada al *archivoRegistro* correspondiente.

```
public void crearRegistro(Datos reg){
    String regname = getClassName(reg);
    for (int i = 0; i < archivoRegistro.size(); i++) {
        ar = (ArchivoRegistro) archivoRegistro.elementAt(i);
        if (ar != null && regname.equals(ar.getName()) == true)
            ar.crearRegistro(reg);
    }
}
```

El método *crearRegistro* hace una llamada interna al método *crearRegistro* de la clase *ArchivoRegistro*.

De manera similar, el método *actualizarRegistro* hace una llamada al método *actualizarRegistro* del *ArchivoRegistro* correspondiente.

Hidden page

Tabla 9.15 Continuación

1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	BaseDatosRegistro
crearRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
obtenerRegistro(Datos, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
actualizarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
eliminarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro

A continuación se muestra la clase *ArchivoRegistro*.

```
public class ArchivoRegistro
```

Definimos un número de atributos privados, entre los cuales la *listaRegistro* guarda la lista de todos los récords del archivo leído, y una referencia al archivo, incluyendo su nombre, junto con otras variables temporales.

```
private Vector listaRegistro;  
private File freg;  
private Class creg;  
private String name;  
private String filename;
```

Dado que cada *ArchivoRegistro* se encarga de un solo archivo, aprovechamos esta clase para hacer la inicialización correspondiente. Leemos el nombre de la clase *classname* para obtener el nombre del archivo, al cual agregaremos la terminación ".dat". Se inicializa la *listaRegistro*, la referencia al archivo mediante *freg*, el cual se obtiene a través del *dirname* y *filename*. Finalmente hacemos la lectura de los archivos mediante la llamada *inicializarRegistrosArchivo*.

```
public ArchivoRegistro(String dirname, Datos reg, String classname){  
    creg = reg.getClass();  
    name = classname;  
    filename = classname + ".dat";  
    listaRegistro = new Vector();  
    freg = new File(dirname,filename);  
    inicializarRegistrosArchivo();  
}
```

Hidden page

```
2
alfredo|Alfredo Weitzenfeld|123456789|MasterCard|01/05|
reservas|Alfredo Weitzenfeld|987654321|Visa|02/4|
```

En este caso, cada registro de usuario cuenta con un registro de tarjeta correspondiente.

Ambos tipos de archivos son procesados de la siguiente forma:

```
String s = is.readLine();
Integer ns = Integer.valueOf(s);
int n = ns.intValue();
```

Las últimas dos líneas convierten la cadena "2" referida por "s" en un número entero.

Posteriormente se hace un ciclo "for" que ejecutará por el número de datos que hayan, y en cada ciclo instanciará un nuevo dato donde guardará los atributos:

```
Datos = (Datos) creg.newInstance();
```

Luego de ello, se continuará con la lectura de cada registro mediante:

```
s = is.readLine();
```

Como se puede notar, en el archivo se separan los diversos campos mediante "|" (líneas verticales). Para ello definimos un "string tokenizer" en Java de la siguiente forma:

```
StringTokenizer t = new StringTokenizer(s, "|");
```

Dentro del ciclo "for" se van leyendo las diversas cadenas separadas mediante líneas verticales

```
String val = t.nextToken();
```

El valor obtenido, *val*, es luego copiado al dato mediante:

```
datos.escribirValor(j,val);
```

Al finalizar la lectura, se actualizan los campos que no se leyeron:

```
datos.actualizarAtributos();
```

Se prosigue guardando en el vector el dato obtenido,

```
vectorDatos.addElement(datos);
```

A continuación escribimos los métodos que dan servicio a los contratos con nombre similar en la clase *ArchivoRegistro*. Comenzamos con *LeerRegistro*, encargado de leer a un registro temporal *reg* los datos guardados en memoria correspondiente al usuario especificado por *log*, como se muestra a continuación:

Hidden page

El método `actualizarArchivoRegistro` es el opuesto a `leerArchivoRegistro`. En lugar de leer un archivo, el archivo se sobrescribe, tarea que se debe llevar a cabo cada vez que exista una modificación en algún registro en memoria. Este método llama al método `escribirDatos`.

```
private void actualizarArchivoRegistro(){
    try{
        BufferedWriter os = new BufferedWriter(new FileWriter(freg));
        escribirDatos(listaRegistro, os);
        os.close();
    }
    catch(IOException e){
        ...
    }
}
```

El método `escribirDatos` es opuesto al de `leerDatos` anteriormente descrito.

```
private void escribirDatos(Vector vectorDatos, BufferedWriter os)
    throws IOException{
    int num = vectorDatos.size();
    String numStr = String.valueOf(num);
    os.write(numStr);
    os.newLine();
    for (int i = 0; i < num; i++) {
        Datos datos = (Datos) vectorDatos.elementAt(i);
        for (int j = 0; j < datos.numeroAtributos(); j++) {
            String str = datos.leerValor(j);
            os.write(str+"|");
        }
        os.newLine();
    }
}
```

Se lee el tamaño del vector de registros:

```
int num = vectorDatos.size();
```

Este número se convierte en una cadena:

```
String numStr = String.valueOf(num);
```

La cadena se escribe como una línea completa en el archivo

```
os.write(numStr);
os.newLine();
```

A continuación se obtienen los datos del registro:

```
Datos datos = (Datos) vectorDatos.elementAt(i);
```

Estos datos se copian uno por uno al archivo separándolos con una línea vertical, de la siguiente manera:

```
String str = datos.leerValor(j);
os.write(str+"|");
```

Finalmente se pasa a la siguiente línea para continuar con otro registro como parte del ciclo "for":

```
os.newLine();
```


El método `eliminarRegistro` es muy similar a `actualizarRegistro`, con la diferencia de que primero se debe eliminar el registro de la lista para luego actualizar el archivo correspondiente, como se muestra a continuación:

```
public void eliminarRegistro(Datos reg){
    int indice = leerIndiceRegistro(reg.leerValor(0));
    if (indice != -1) {
        listaRegistro.removeElementAt(indice);
        actualizarArchivoRegistro();
    }
}
```

El método `validarRegistro` se encarga de leer el registro de la memoria según el log especificado, mediante la llamada `leerRegistro`, para luego comparar si el pass corresponde, como se muestra a continuación:

```
public boolean validarRegistro(Datos reg, String log, String pass){
    if (leerRegistro(reg,log) != false && reg != null) {
        String preg = reg.leerValor(1);
        if (pass.equals(preg))
            return true;
    }
    return false;
}
```

9.1.5 Servicios

La clase *ManejadorServicio* se muestra en la tabla 9.16.

Tabla 9.16 Tarjeta de la clase <i>ManejadorServicio</i> .	
Clase: ManejadorServicio	
Descripción: el manejador de servicios se encarga de enviar las peticiones particulares de servicios a los manejadores especializados para consulta, reserva y compra.	
Módulo: Servicios	
Estereotipo: Control	
Propiedades: Concreta	
Superclase: Manejador	
Subclase:	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void	
<i>Método sobrescrito de la clase Manejador, encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
2. Ofrecer Servicio	
ofrecerServicio() devuelve void	
<i>Método encargado de hacer solicitudes para consultar, reservar y administrar registros</i>	

continúa

Hidden page

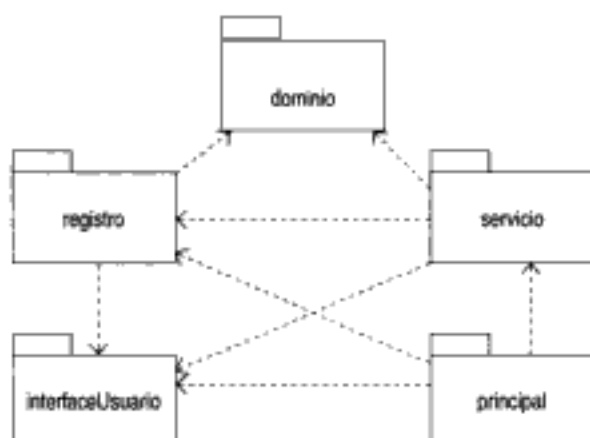


Figura 9.1 Diagrama de módulos para el sistema completo.

9.2.1 InterfaceUsuario

El diagrama de clases del módulo *InterfaceUsuario* se muestra en la figura 9.2.

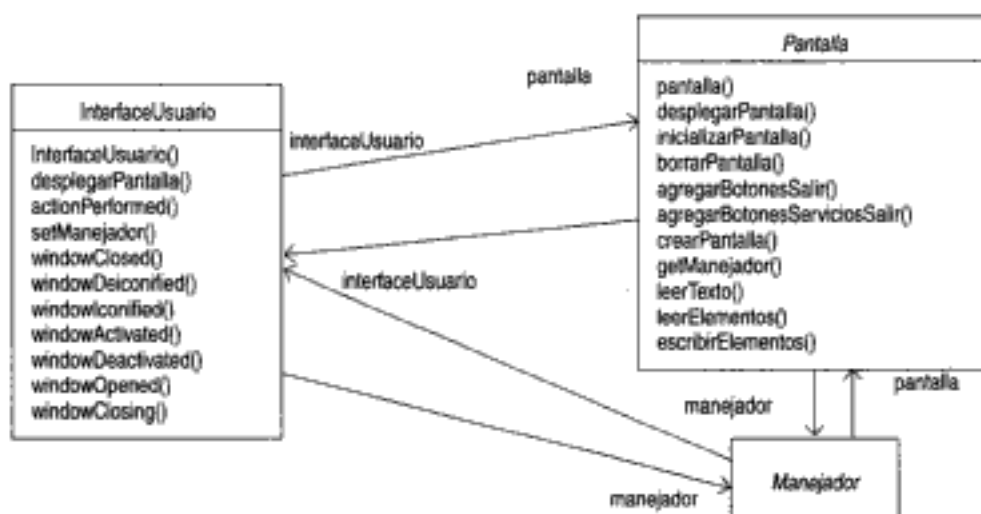


Figura 9.2 Diagrama de clases del módulo *InterfaceUsuario*.

9.2.2 Principal

El diagrama de clases del módulo *Principal* se muestra en la figura 9.3.

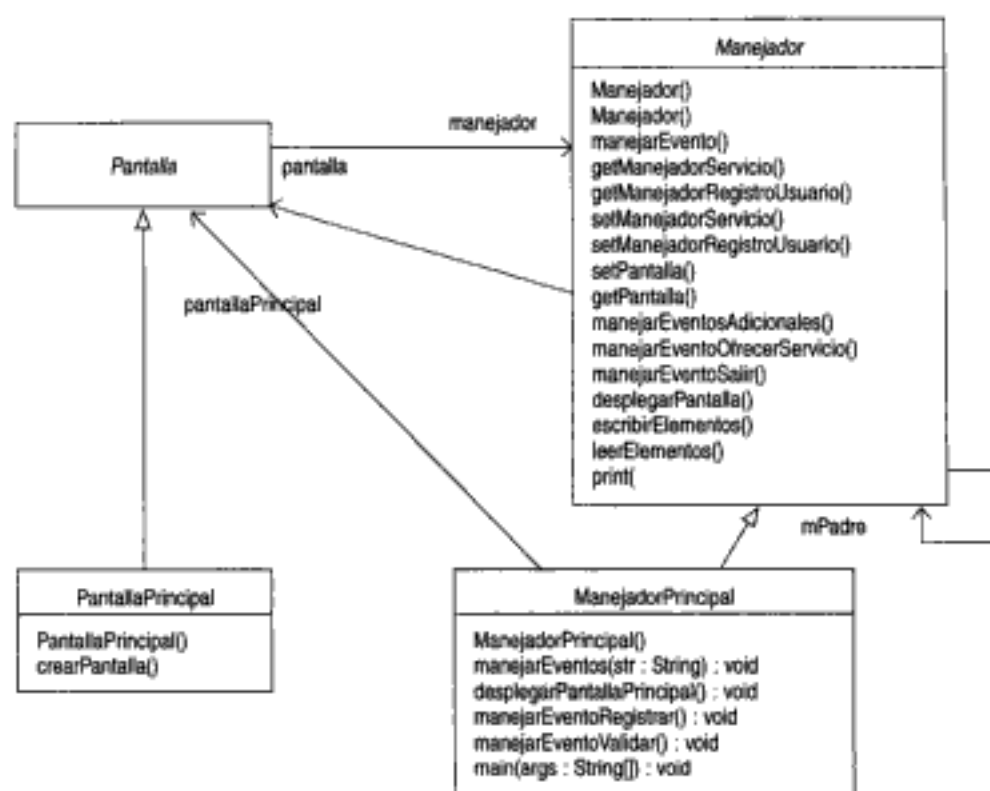


Figura 9.3 Diagrama de clases del módulo *Principal*.

9.2.3 Dominio

El diagrama de clases del módulo *Dominio* se muestra en la figura 9.4.

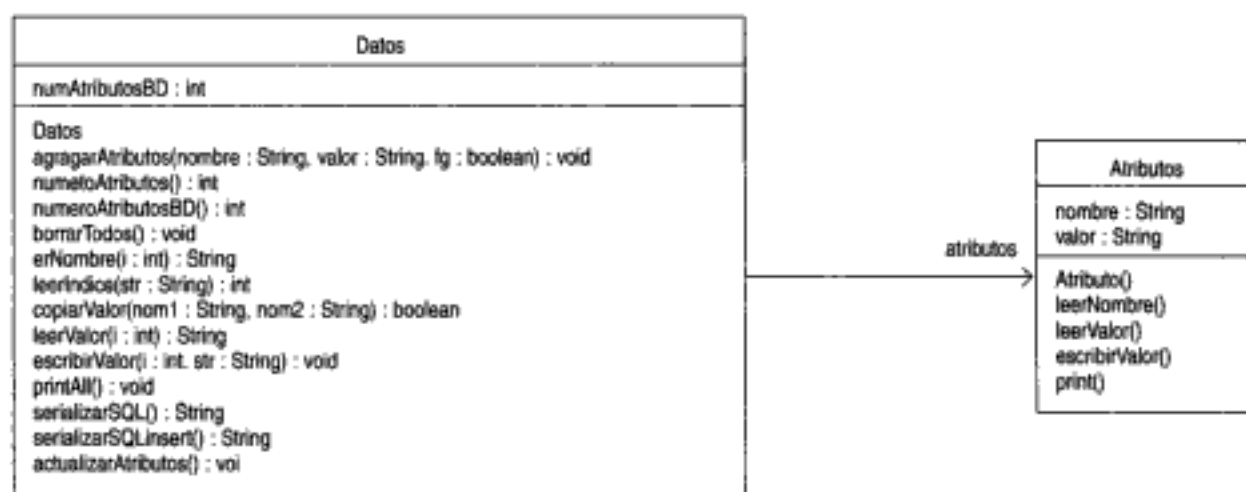


Figura 9.4 Diagrama de clases del módulo *Dominio*.

9.2.4 Registro

El módulo de *Registro* está compuesto por los módulos de *Usuario*, *Tarjeta* y *InterfaceBD*, como se muestra en la figura 9.5 y con mayor detalle en las siguientes secciones.

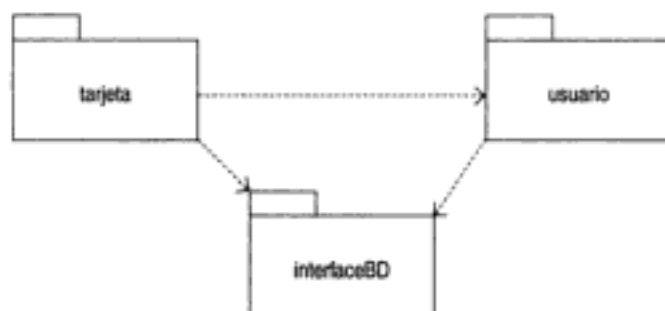


Figura 9.5 Diagrama de módulos del sistema de registro.

USUARIO

El diagrama de clases del módulo *Usuario* se muestra en la figura 9.6.

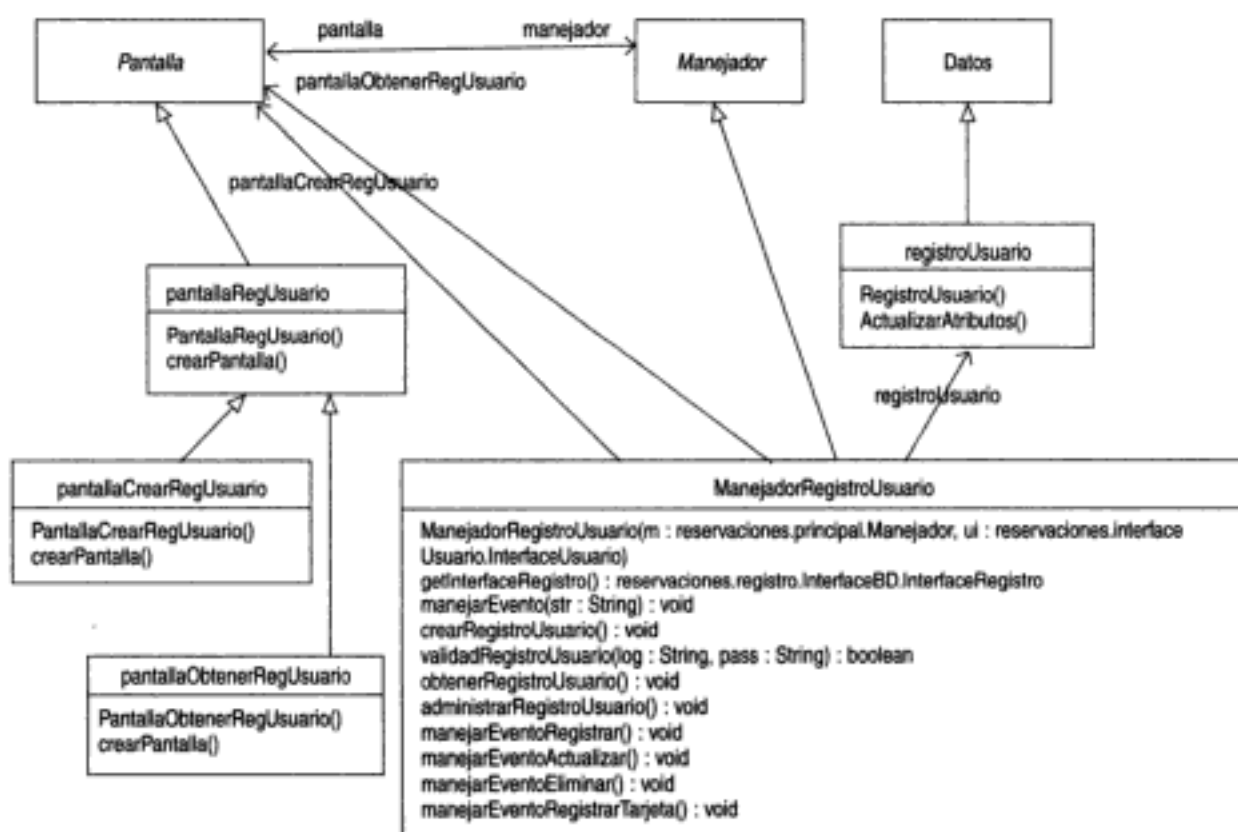


Figura 9.6 Diagrama de clases del módulo *Usuario*.

TARJETA

El diagrama de clases del módulo *Tarjeta* se muestra en la figura 9.7.

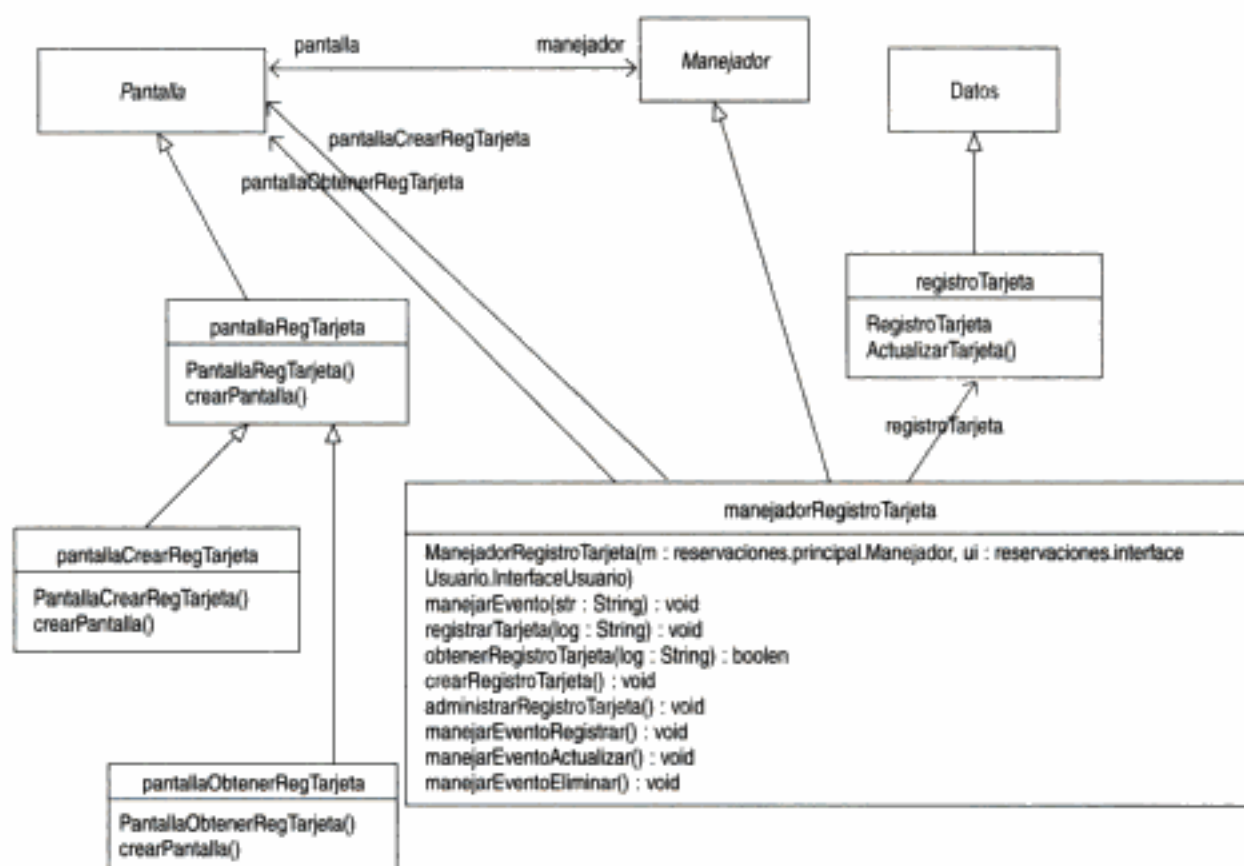


Figura 9.7 Diagrama de clases del módulo *Tarjeta*.

INTERFACEBD

El diagrama de clases del módulo *InterfaceBD* se muestra en la figura 9.8.

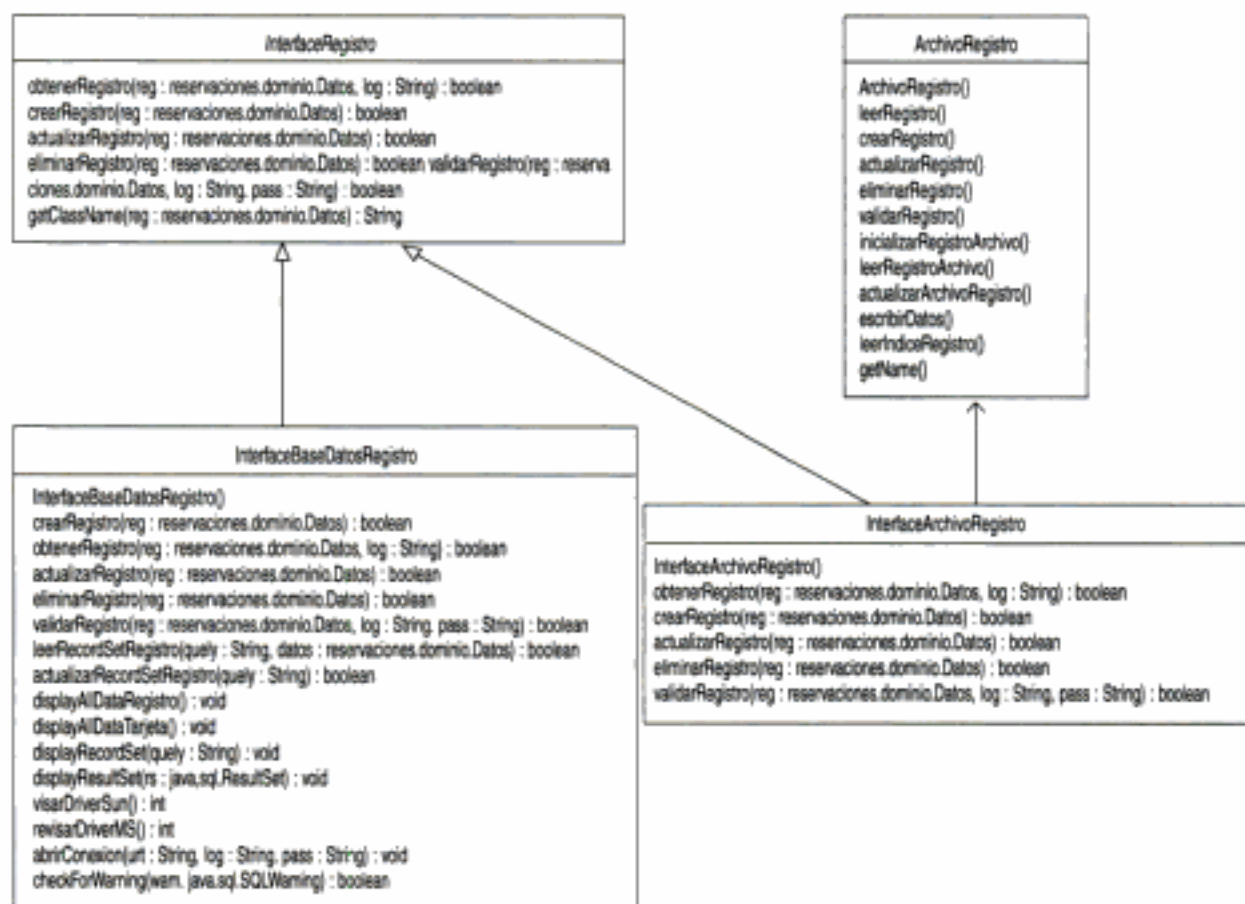


Figura 9.8 Diagrama de clases del módulo *InterfaceBD*.

9.2.5 Servicios

Las clases del módulo *Servicios* se muestran en la figura 9.9.

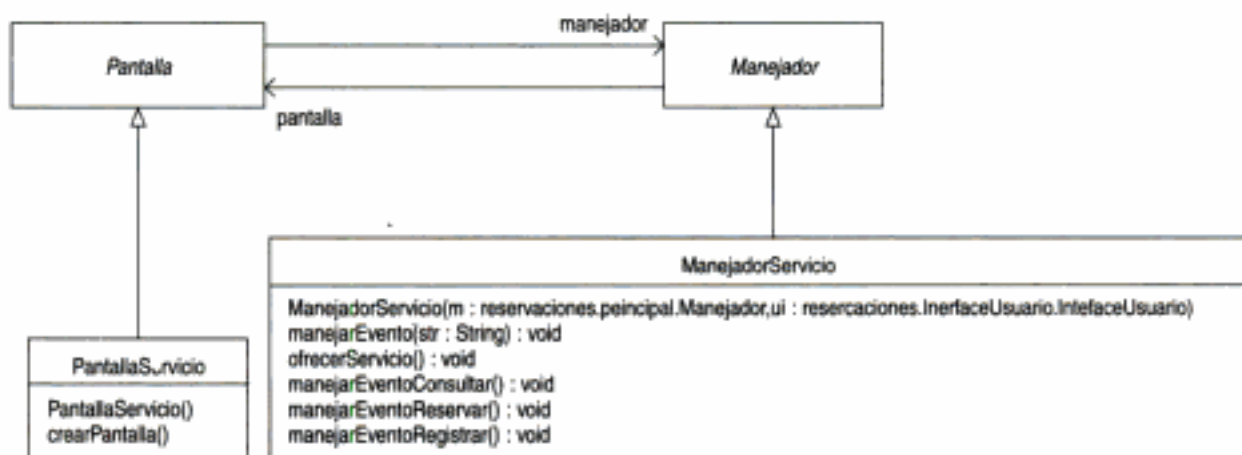


Figura 9.9 Diagrama de clases del módulo *Servicios*.

RESUMEN

En este capítulo se describe el modelo de implementación. Se toma la especificación de clases elaboradas durante el modelo de diseño y se describe, de acuerdo con el diseño de sistema, las clases de forma detallada y en el lenguaje de programación deseado, en este caso, Java. Se muestra la liga a las bases de datos. Además, se explican los diagramas de clases finales.

REFERENCIAS

1. Wirfs-Brock, R., Wilkerson, B., Wiener, L., 1990, Designing Object-Oriented Software, Prentice-Hall.

Modelo de pruebas

Probar un producto es relativamente independiente de la metodología de desarrollo utilizada para construirlo.

Existen diversos tipos de pruebas aplicados durante las diferentes actividades del proceso de desarrollo, las cuales requieren de tiempo y presupuesto adicionales, que pueden llegar a significar un alto porcentaje del costo total. Por tal motivo, el modelo de pruebas debe ser planificado con anticipación y de manera integral junto con el desarrollo del sistema. Es un error pensar que las pruebas son la última actividad del desarrollo, ya que no se puede lograr software de alta calidad sólo mediante pruebas finales y depuraciones. Las mismas deben hacerse simultáneamente con el desarrollo del sistema. Además, las pruebas finales deben tener como objetivo la certificación final de la calidad del producto y no la búsqueda de errores. Detectarlos al final del desarrollo es bastante problemático, dado que ello requiere regresar a etapas anteriores para resolverlos. Se considera que “evitar defectos” es más importante que “removerlos”.

10.1 Definición de conceptos

Las siguientes definiciones pueden ser utilizadas para precisar ciertos conceptos conocidos de manera informal como “bugs”:¹

- Una *falla* (*failure*) ocurre cuando un programa no se comporta de manera adecuada. La *falla* es una propiedad (estadística) de un sistema en ejecución.
- Una *falta* (*fault*) tiene lugar en el código del programa. La existencia de una falta en el programa puede ocasionar una *falla* (*failure*) en el sistema. No puede haber una *falta* si el programa no puede *fallar* (*fail*).
- Un *error* es una acción humana que provoca que un software contenga una falta. Un *error* puede significar la existencia de una *falta* en el programa, lo cual hace que el sistema *falla*.

Un aspecto importante relacionado con los conceptos anteriores es que no se puede garantizar ni probar que un sistema jamás *falla*, si no que sólo se puede demostrar que contiene *faltas*. En otras palabras, no encontrar faltas no significa que la prueba haya sido exitosa. Sólo lo es si ha encontrado faltas. Sin embargo, pruebas exitosas significan que no se ha desarrollado un buen sistema.

Dada la dificultad de probarlo y de encontrar faltas, el encargado de detectarlas en el código es generalmente una persona distinta al desarrollador del sistema. Esto también significa un costo adicional en el desarrollo de éste, por lo cual a veces sólo se prueban las partes principales del sistema.

Es muy común que cuando se corrigen las faltas detectadas se introduzcan otras nuevas en el sistema, lo cual requiere someterlo nuevamente a prueba esperando que el número de faltas introducidas sea menor que la existente con anterioridad. Generalmente se genera una nueva falta por cada tercera falta corregida.²

10.2 Tipos de pruebas

Los tipos de pruebas se dividen de manera general en pruebas de *verificación* y *validación*. En el primer caso se revisa si el resultado corresponde a la especificación del sistema, es decir, si se está construyendo el sistema de manera correcta, algo que por sí sólo no garantiza la satisfacción de los clientes. En el segundo caso, se revisa si el resultado es realmente lo que el cliente quería, en otras palabras, si se está construyendo el sistema correcto, de manera que tanto la especificación como el resultado lo sean. En este capítulo nos concentraremos principalmente en la verificación del sistema. Por otra parte, su validación debe llevarse a cabo durante la especificación inicial a través de prototipos que deben ser aprobados por el cliente, y que correspondan a la funcionalidad deseada. El sistema debe validarse continuamente durante su proceso de desarrollo de manera que siempre corresponda con lo especificado. La validación se basa en el modelo de casos de uso.

Existen también diferentes técnicas y niveles de pruebas que pueden aplicarse, las cuales se describen en las siguientes secciones.

10.2.1 Técnicas de pruebas

Las técnicas utilizadas para realizar las pruebas son muy variadas, pero se pueden destacar las siguientes:

- **Prueba de regresión.** Tiene como propósito verificar el sistema luego, de haberle introducido cambios, por ejemplo después de corregir una falta, de manera que se mantenga la funcionalidad especificada originalmente.
- **Prueba de operación.** Su objetivo es verificar el sistema en operación por un largo periodo bajo condiciones normales de uso. Este tipo de prueba mide la *confiabilidad* (*reliability*) del sistema.
- **Prueba de escala completa.** Trata de verificar el sistema en su carga máxima mediante la asignación de los parámetros a su valor límite y la interconexión del sistema con un máximo de equipos y usuarios simultáneos. Su máxima expresión es la *prueba de estrés* (*stressing*), que significa que se prueba el sistema en los límites extremos para determinar su nivel de tolerancia y si ocurre algún tipo de falla.
- **Prueba de rendimiento** (*performance*) o **de capacidad.** Tiene como propósito medir la capacidad de procesamiento del sistema bajo diferentes cargas, incluyendo espacio de almacenamiento y utilización de la unidad de

procesamiento. Los valores medidos se comparan con los valores requeridos.

- ▶ **Prueba de sobrecarga.** Pretende observar cómo se comporta el sistema cuando se le aplica una sobrecarga, más allá de las pruebas de escala completa y rendimiento. Aunque no se puede esperar que el sistema tolere estas pruebas, debería funcionar correctamente, es decir, sobrevivir a picos de carga y evitar que ocurra una catástrofe. Es siempre importante saber en qué momento y de qué manera cae el rendimiento del sistema.
- ▶ **Prueba negativa.** Tiene como propósito medir el estrés del sistema en situaciones inesperadas, como casos de uso que normalmente no serían utilizados de manera simultánea. El sistema se usa intencional y sistemáticamente de forma incorrecta. Este maltrato debe ser planeado de forma cuidadosa para probar aspectos especialmente críticos.
- ▶ **Prueba basada en requisitos o prueba de casos de uso.** Intenta llevar a cabo pruebas basadas directamente en la especificación de requisitos. Pueden utilizarse los mismos casos de uso originales como casos de prueba. También pueden ser utilizadas para verificar las especificaciones de rendimiento o de escala completa. Se trata de verificar que el sistema final cumple con las especificaciones funcionales descritas por los casos de uso originales.
- ▶ **Pruebas ergonómicas.** Tienen como propósito probar los aspectos ergonómicos del sistema, en otras palabras, las interfaces hombre-máquina en el caso de que éstas existan. Por ejemplo, se prueba si las interfaces son congruentes con los casos de uso a los cuales corresponden, o entre diferentes casos de uso, si los menús son lógicos y legibles, si los mensajes del sistema son visibles, si se puede entender los mensajes de falla, etcétera.
- ▶ **Prueba de documentación de usuario.** Tiene como propósito probar la documentación de usuario, incluyendo el manual de éste y la documentación de mantenimiento y servicio. Se prueba que los manuales y el comportamiento del sistema sean congruentes entre sí, que sean legibles, con una buena redacción y, en general, que sean comprensibles.
- ▶ **Prueba de aceptación o de validación.** Pretende lograr una revisión final por parte de la organización que solicitó el sistema, lo cual, a menudo, significa validación del sistema. El sistema se prueba en su ambiente real por un periodo extenso. Cuando se termina la prueba, se toma la decisión de aceptar o no el producto. Este tipo de prueba es a veces conocida como *prueba alfa*. Si no existe un cliente particular que haya solicitado el sistema, por ejemplo en el caso de un producto de software de venta al público, a menudo se hace una *prueba beta*, lo cual implica que antes de enviarlo al público en general, el producto es probado por clientes seleccionados que utilizan el sistema y reportan las fallas experimentadas.

10.2.2 Nivel de pruebas

Existen tres niveles principales para aplicar las diversas técnicas de pruebas:

- ▶ **Prueba de unidad.** Mediante esta prueba sólo una unidad es probada como tal, como una clase, un paquete de servicio o un subsistema.
- ▶ **Prueba de integración.** En ella se verifica que las unidades trabajen juntas correctamente. Ambas pueden ser realizadas mediante casos de uso de pruebas, los cuales pueden ser aplicados a clases, paquetes de servicio, subsistemas y el sistema completo.

- **Prueba de sistema.** Verifica el sistema completo o su aplicación como tal. Se toma el punto de vista del usuario final y los casos de uso de pruebas ejecutan acciones típicas del usuario.

PRUEBA DE UNIDAD

La prueba de unidad es la de más bajo nivel. En un sistema tradicional son, a menudo, pruebas de procedimientos o subrutinas. En un sistema orientado a objetos se aplican en un nivel más alto, a partir de las clases. Por lo tanto, una prueba de unidad en un sistema orientado a objetos es más complejo que en sistemas estructurados, dado que los objetos involucran estados encapsulados que puede afectar el comportamiento y corrección de la unidad. Además, aspectos como herencia y polimorfismo agregan complejidad adicional a las pruebas.

Tradicionalmente, una prueba de unidad consiste en una prueba estructural (o de caja blanca), lo cual requiere conocer el diseño interno de la unidad, y una prueba de especificación (de caja negra), basada sólo en la especificación del comportamiento externamente visible de la unidad. Normalmente ambas pruebas son necesarias y se complementan entre sí. Los sistemas orientados a objetos pueden utilizar estas pruebas y además requerir otras basadas en estado, correspondiente al estado encapsulado del objeto y la interacción de las operaciones.

Como las pruebas estructurales dependen de la estructura del sistema, mientras que las basadas en estado y de especificación pueden afectar la estructura del sistema, es preferible hacerlas al final. Si se encuentra una falta en cualquiera de las pruebas anteriores, es necesario modificar de manera correspondiente el sistema, lo cual afectará a la prueba estructural. Las pruebas, descritas con mayor detalle a continuación, pueden seguirse en el orden descrito:

- La **prueba de especificación, o de caja negra**, tiene como propósito verificar las relaciones de entrada y salida de una unidad. Su objetivo es verificar "qué" hace la unidad, pero sin averiguar "cómo" lo hace. Se envían estímulos con diferentes parámetros como entrada y se comparan con las salidas esperadas. Se revisa que éstas sean correctas, como en el caso de las operaciones matemáticas. Dado que las unidades se comunican mediante interfaces bien definidas, la prueba de especificación es bastante directa.
- La **prueba basada en estado** verifica las interacciones entre operaciones de una clase según cambios en los atributos de un objeto. No se puede ver a las operaciones de un objeto como aisladas e independientes de los valores de los atributos. Es necesario hacer pruebas del objeto de acuerdo con su "ciclo de vida", lo cual es especialmente importante cuando se trata de objetos controlados por estado, descritos mediante diagramas de transición de estados. En la realidad es imposible revisar todas las posibles combinaciones de valores de atributos en combinación con todos los posibles estímulos. Es suficiente revisar los estados identificados en los diagramas de transición de estados de los objetos. Además, algunas combinaciones particulares de atributos pueden ser de mayor interés que otras. Otras combinaciones se pueden considerar *conjuntos de equivalencia* (*equivalence*

set) de comportamiento, lo cual evita revisiones adicionales. Por lo tanto, es deseable revisar valores particulares significativos para luego asignar un conjunto de equivalencias a cada grupo de valores relacionados. Por otro lado, algunas operaciones no afectan el estado, como son las operaciones puramente de lectura, por lo cual no necesitan ser consideradas en la prueba basada en estados. Las demás operaciones deberán ejecutarse para todos los posibles estados iniciales.

- La **prueba estructural**, que tiene como propósito verificar que la estructura interna de la unidad sea correcta, se conoce también como prueba basada en programa o de caja blanca, dado que debe conocerse cómo está implementada internamente la unidad. Es deseable cubrir todas las posibles combinaciones de parámetros, valores de variables y flujos del código, de manera que todas las instrucciones se ejecuten. Para examinar la eficacia de los casos de prueba se debe medir la *cobertura de prueba* (*coverage test*), donde cada ruta del código sea cubierta o probada al menos una vez. La mayoría de los problemas provienen de combinaciones de rutas inusuales como aquellas que involucran ciclos. Los casos de prueba se ilustran mediante diagramas de flujo (*flowcharts*).

PRUEBA DE INTEGRACIÓN

Después de haber probado todas las unidades, éstas deben ser integradas en unidades más grandes hasta generar el sistema completo. El propósito de la prueba de integración es probar que las diferentes unidades trabajen juntas de manera apropiada. En esta prueba se incluye la de paquetes de servicio, casos de uso, subsistemas y el sistema completo. Durante las pruebas de combinación de unidades, algo que se incrementa exponencialmente, se pueden detectar fallas imposibles de encontrar durante las pruebas de una sola unidad. No se debe comenzar la prueba de integración hasta que las pruebas de unidad estén listas.

La prueba de integración se basa principalmente en la *prueba de casos de uso*, que se lleva a cabo a partir de los diagramas de secuencias, mediante la cual se pueden identificar los estímulos enviados entre el usuario y el sistema, y entre los objetos del sistema. La prueba de casos de uso se divide en pruebas de curso básico, cursos alternos y documentación de usuario. Se prueba primero los flujos básicos, los esenciales del sistema, para luego concentrar la atención en los flujos alternos, correspondientes a flujos que ocurren de manera inusual, como en el caso de manejo de excepciones. Los casos de uso que extienden o son incluidos en otros casos de uso se prueban después de verificar los casos de uso básicos donde éstos se insertan.

PRUEBA DE SISTEMA

Al principio, las pruebas de casos de uso se hacen de manera independiente, basadas en el modelo de requisitos. Después de probar todos los casos de uso aislados, se prueba el sistema entero como uno solo. Se ejecutan varios casos de uso en paralelo y se somete el sistema a diferentes cargas. Las pruebas de sistema pueden involucrar *pruebas de operación*, *de escala completa*, *negativas*, *basadas en requisitos* o *casos de uso* y *pruebas de documentación de usuario*.

10.3 Proceso de pruebas

El proceso de pruebas involucra consideraciones similares al del proceso de desarrollo de software, incluyendo estrategias, actividades y métodos, los cuales deben ser aplicados de manera concurrente con el proceso de desarrollo de software. En particular, las actividades de pruebas abarcan los siguientes aspectos: planeación, construcción y ejecución. Por lo general, se mantiene una *bitácora de prueba* (*test log*) durante todo el proceso de pruebas.

10.3.1 Estrategia de prueba

Existen diversas estrategias para el proceso de pruebas, entre las que se destacan el orden en que se van a llevar a cabo, la partición de equivalencias de pruebas que se van a aplicar y la posibilidad de automatizarlas.

- **Orden de pruebas.** Tiene como propósito definir en qué momento y en qué orden se aplicarán las pruebas. Aunque éstas deben ser planeadas con anterioridad, las verificaciones típicamente se aplican durante el diseño, implementación y operación del sistema. Existen dos enfoques generales para el orden en que se efectuarán las pruebas: de *arriba hacia abajo* y de *abajo hacia arriba*. Este orden depende en gran medida de la estrategia de diseño ya que debe lograr una buena correspondencia con el proceso de desarrollo utilizado. Si se aplican pruebas correspondientes a un diseño de arriba hacia abajo, entonces se deben desarrollar inicialmente las interfaces entre subsistemas, para tratar de probar los protocolos a alto nivel antes de ir a los niveles inferiores. Si se hace un diseño de abajo hacia arriba se puede certificar primero las unidades de bajo nivel y luego las interfaces entre ellas. Esta técnica aprovecha que una vez probadas las unidades, se elimina la necesidad de crear servidores especializados para pruebas. En el de pruebas de arriba hacia abajo se necesitan servidores de pruebas especiales que simulen el ambiente alrededor de la unidad que se verifica. En el caso extremo se debe construir una estructura completa para simular todos los objetos del sistema que aún no existen. Normalmente, es suficiente tener una base de pruebas que sea de una magnitud de orden menor que lo que se está probando, como una clase de prueba por cada paquete de servicio (contratos), un contrato para cada subsistema, etcétera.
- **Alcance de pruebas.** Tiene como propósito identificar el tipo, número y casos de pruebas que se aplicarán para revisar los diferentes aspectos del sistema. Si se considera que los tipos de pruebas son bastante amplios y extensos, el objetivo es seleccionar un número pequeño pero razonable de casos de prueba dentro de un gran número de posibles pruebas donde la probabilidad de encontrar faltas sea alto. Se define la *partición de equivalencias* según *conjuntos equivalentes* (*equivalent set*) de pruebas definiendo un grupo de condiciones donde el sistema o algún componente se comportará de manera similar ante diferentes pruebas. La idea es escribir casos de prueba que cubran todos los conjuntos de equivalencia, pero tener un caso de prueba para cada conjunto de equivalencia.
- **Automatización de pruebas.** Tiene como propósito reducir el esfuerzo y costo de las pruebas mediante la *automatización* del proceso o aspectos de él. Este objetivo puede lograrse mediante programas de pruebas especiales asociados a un conjunto de datos de pruebas. El programa de prueba debe

Hidden page

corregir de manera adecuada para luego aplicar nuevamente las pruebas anteriores. Por ejemplo, se puede tener como objetivo general detectar el máximo posible de faltas presentes en el sistema, por lo cual se puede tener como objetivo particular del diseño de pruebas minimizar el número de faltas por cada 1 000 líneas de código, o algo similar.

10.3.4 Ejecución de la prueba

Durante esta etapa se utiliza la especificación del diseño de prueba y los reportes de ésta. La estrategia es aplicar de manera paralela el mayor caso de pruebas posible. Se ejecutan las pruebas automáticas y manuales de manera correspondiente y se indican los resultados esperados. Si alguna prueba particular falla, se interrumpe su aplicación y se anota el resultado para luego analizar el defecto y corregirlo, si ello es posible. Una vez corregido, la prueba se ejecuta nuevamente. Todas las pruebas son ponderadas según su importancia, y se puede determinar si la prueba completa ha sido aprobada o no según su resultado. Se analizan los resultados de la prueba completa y se anota en los reportes un resumen de ella, los recursos utilizados, los resultados individuales y si los resultados fueron aprobatorios o no. El reporte de la prueba debe también mostrar el resultado de cada una de ellas, recursos utilizados y acciones tomadas.

Si al ejecutar las pruebas se detectan fallas, se deben analizar sus resultados y la causa de la falta identificada. Ésta no necesariamente tiene que ser responsabilidad del sistema, sino que puede haber sido provocada por otras circunstancias tal como si la prueba se aplicó de manera incorrecta, si existen errores en los datos utilizados o en el programa de prueba, si existe una falla causada por la base de prueba, o si el software del sistema se comporta de forma incorrecta. Si la falla no fue provocada por el objeto en prueba, se debe corregir y hacerla nuevamente. Si fue por causa del sistema, será necesario identificar qué clases o paquetes de servicios deficientes deben ser devueltos al diseñador. El proceso de detección de faltas se simplifica si las unidades probadas ofrecen facilidades apropiadas, por ejemplo, contadores o bitácoras de faltas.

10.4 Pruebas del Sistema de Reservaciones de Vuelos

En lo que respecta al *Sistema de Reservaciones de Vuelos* desarrollado a lo largo del libro, nos limitaremos a verificarlo de acuerdo con la prueba de requisitos o casos de uso. Como objetivo de la prueba revisaremos que la funcionalidad implementada corresponda a los casos de uso especificados durante el modelo de requisitos. A continuación revisaremos los casos de uso principales, básicos y de extensión, los cuales fueron descritos durante el diseño *Registrar Usuario* y *Registrar Tarjeta*. Nótese que los casos de uso *Validar Usuario* y *Ofrecer Servicios* no los describiremos de manera independiente, ya que son inclusiones de los demás.

10.4.1 Registrar Usuario

Se prueba la secuencia más importante de los casos de uso *Registrar Usuario*: *Crear Registro Usuario*, *Actualizar Registro Usuario* y *Eliminar Registro Usuario*.

Hidden page

Hidden page

Hidden page

Hidden page

Hidden page

Hidden page

Hidden page

Para finalizar el caso de uso, el usuario debe presionar el botón "Salir" en la PantallaCrearRegistroUsuario (P-3), como se muestra en la figura 10.15.

SYSTEMA DE RESERVACIONES DE VUELO
Pantalla Crear Registro Usuario (P-3)

Nombre Apellido

Correo Celular

Ciudad Pais Código Postal

Tel. Casa Tel. Cel. Fax

Login Email

Password Repetir Password

Figura 10.15 PantallaObtenerRegUsuario (P-3) con el botón "Salir" presionado.

10.4.2 Registrar tarjeta

Se prueban las secuencia más importante del caso de uso *Registrar Tarjeta*: *Crear Registro Tarjeta*, *Actualizar Registro Tarjeta* y *Eliminar Registro Tarjeta*.

CREAR REGISTRO TARJETA

El caso de uso *Registrar Tarjeta* es una extensión del caso de uso *Registrar Usuario*. La secuencia *Crear Registro Tarjeta* depende de que no exista un registro anterior de tarjeta para el usuario actual. El diagrama de secuencia de alto nivel se muestra en la figura 10.16.

La secuencia comienza de manera similar a *Actualizar Registro Usuario*. En lugar de hacer una actualización de registro de usuario, se presiona el botón "Registrar Tarjeta" en la PantallaObtenerRegUsuario (P-4), como se muestra en la figura 10.17.

El caso de uso continúa cuando el sistema presenta la PantallaCrearRegTarjeta (P-5), la cual debe ser llenada con los datos de tarjeta de crédito solicitados. Nótese que en este subflujo aún no existe una tarjeta de registro para el usuario. Éste debe presionar el botón "Registrar", como se muestra en la figura 10.18.

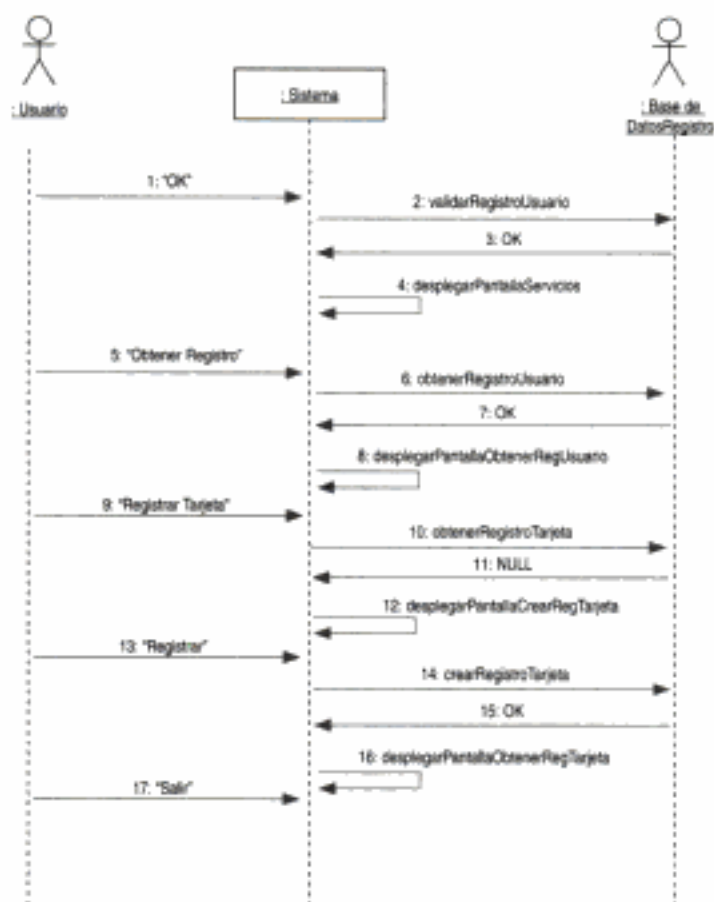


Figura 10.16 Diagrama de secuencias *Crear Registro Tarjeta* del caso de uso *Registrar Tarjeta*.

SISTEMA DE RESERVACIONES DE VUELO
Pantalla de Obtener Registro Usuario (P-4)

Nombre: Apellido:

Dirección: Colonia:

Ciudad: País: Código Postal:

Tel. Casa: Tel. Cel.: Fax:

Login: Email:

Contraseña: Repetir Contraseña:

Figura 10.17 Pantalla *ObtenerRegUsuario (P-4)* con el botón "Registrar Tarjeta" presionado.

Hidden page

Hidden page

ma presentando la PantallaObtenerRegTarjeta (P-6) en la cual, el usuario podrá modificar los datos deseados de la tarjeta de crédito, tal como la fecha de vencimiento. Para ello debe presionar el botón "Actualizar", como se muestra en la figura 10.22.

Figura 10.22 PantallaObtenerRegTarjeta (P-6) con los datos de registro de la tarjeta, tales como la fecha de vencimiento modificada y con el botón "Actualizar" presionado.

Se puede revisar en la *Base de Datos Registro* que los valores han sido actualizados correctamente en la base de datos, como se observa en la figura 10.23.

RegistroTarjeta : Table					
	login	nombre	numero	tipo	fecha
	+ weitzenfeld	Alfredo Weitzenfeld	123456789	Visa	01/05
Record: 1 2 of 2					

Figura 10.23 Imagen de la *Base de Datos de Registro* que presenta el registro de tarjeta recién actualizada.

Para finalizar el caso de uso, el usuario debe presionar el botón "Salir" en la PantallaObtenerRegTarjeta (P-6), como se mostró en la figura 10.20.

ELIMINAR REGISTRO TARJETA

La secuencia *Eliminar Registro Tarjeta* se muestra a nivel funcional en la figura 10.24.

La secuencia comienza de manera similar a *Actualizar Registro Tarjeta*, es decir, se debe presionar el botón "Registrar Tarjeta" en la PantallaObtenerRegUsuario (P-4), como se mostró en la figura 10.17. A continuación el sistema presenta la pantalla PantallaObtenerRegTarjeta (P-6). Para eliminar el registro de tarjeta, el usuario debe presionar el botón "Eliminar", como se muestra en la figura 10.25.

Hidden page

Hidden page

IV

P A R T E

Programación y desarrollo de software para Internet

En esta parte del libro, la última, se hace una introducción a la programación en Internet con tecnología de Java incluyendo HTML, Servlets, JSP y JavaBeans (capítulo 11). A continuación, se incluye una extensión para Internet del sistema que se desarrolló en la parte precedente (capítulo 12).

Programación con HTML, Servlets y JSP

En este capítulo se hará una breve introducción a la programación en HTML, Servlets y Java Server Pages (JSP), para luego, en el capítulo 12, extender el ejemplo del sistema de reservaciones de vuelos a un ambiente cliente-servidor que pueda ejecutarse en Internet.

11.1 Arquitectura cliente-servidor

Para desarrollar sistemas que se procesen en Internet es necesario comprender los conceptos básicos de una arquitectura conocida como *cliente-servidor*. En la figura 11.1 se muestra un esquema general donde existen múltiples *clientes* que se comunican a un *servidor*.

Aunque las arquitecturas *cliente-servidor* no son exclusivas del World Wide Web, o incluso de Internet, en este capítulo nos concentraremos en las tecnologías que directamente servirán de apoyo para extender el sistema de reservaciones

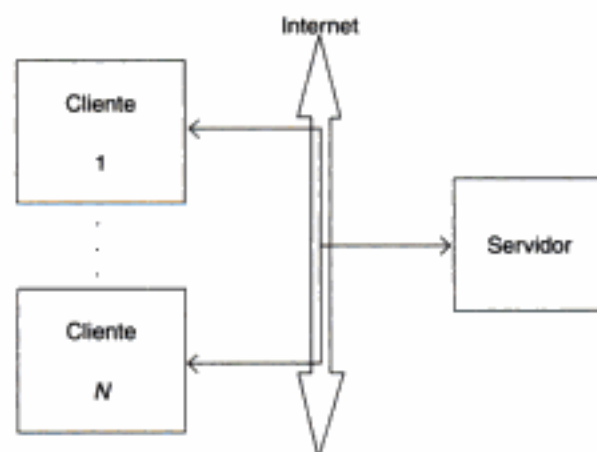


Figura 11.1 Esquema de una arquitectura *cliente-servidor*.

de vuelo a Internet utilizando tecnologías en Java, en otras palabras HTML (que es general a todos los navegadores de Internet), Servlets y JSP.

11.1.1 Cliente

Si se considera que el usuario, a través de una computadora local correspondiente al cliente, es el interesado en interactuar con los programas que existen en Internet, el cliente tiene como función primordial facilitar la interacción del usuario. En otras palabras, el objetivo básico del cliente en la arquitectura cliente-servidor es facilitar la presentación y control de la información administrada por la aplicación, algo similar al rol de las clases borde en relación con un actor de tipo usuario. Por tal motivo, la mayoría de las tecnologías que se procesan en el cliente están dirigidas a facilitar la visualización y control de la información, como en el caso de HTML,¹ Flash,² Javascript,³ VBScript⁴ y JScript⁵. En la sección 11.2 se describirá brevemente HTML, la tecnología del lado del cliente que se utiliza en el proyecto del sistema de reservaciones de vuelo.

11.1.2 Servidor

El servidor es el responsable de prestar los servicios requeridos por los clientes. Es común que la mayor parte de una aplicación en una arquitectura *cliente-servidor* se encuentre del lado del servidor. Esto se hace por razones de costo, eficiencia y facilidad para dar servicio a múltiples clientes de manera concurrente. En general, muchos de los lenguajes tradicionales de programación han sido utilizados para programar los servidores, pero existen ciertas tecnologías diseñadas específicamente para arquitecturas *cliente-servidor* en Internet. Históricamente, el primer modelo de programación para arquitecturas cliente-servidor en Internet fueron los CGI⁶ (Common Gateway Interface), que en la actualidad se han extendido con estándares más modernos como ASP⁷ (Active Server Pages), ColdFusion,⁸ PHP⁹ (Php Hypertext Preprocessor), Servlets¹⁰ y JSP¹¹ (Java Server Pages). En las secciones 11.3 y 11.4 se describirán los Servlets y JSP, respectivamente, ambas tecnologías basadas en la programación en Java.

11.2 HTML

El lenguaje HTML (HyperText Markup Language) fue desarrollado por Tim Berners-Lee,¹² simultáneamente a la creación de la Web. HTML es un lenguaje de hipertexto que permite el despliegue de documentos que incluyan ligas con otros documentos. En la figura 11.2 se muestra un esquema del procesamiento



Figura 11.2 Esquema de una arquitectura *cliente-servidor* para Internet con solicitudes y envío de documentos en HTML.

Hidden page

Hidden page

y el texto que aparecerá como nombre de la referencia. El documento puede estar almacenado en el mismo servidor o en otro, en cuyo caso se debe utilizar una referencia de tipo "url" al estilo "http://servidor/.../Ejemplo2.html". El código completo se muestra a continuación:

Ejemplo 11.2 Ejemplo de documento HTML con liga de página.

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservas de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<A HREF="ResultadoEjemplo11.2.html">ir a resultado Ejemplo 2</A>
</BODY>
</HTML>
```

En la figura 11.4 se muestra el archivo una vez desplegado.



Figura 11.4 Despliegue en el navegador del ejemplo 11.2 de documento HTML que muestra una referencia a otro documento, la cual puede ser seleccionada por el usuario.

Si el usuario presiona sobre la liga ("ir a resultado Ejemplo 2") se despliega el archivo correspondiente, en este caso "ResultadoEjemplo11.2.html", como se muestra en la figura 11.5.



Figura 11.5 Despliegue en el navegador del ejemplo 11.2 de documento HTML llamado a través de la referencia que se presenta en la figura 11.4.

11.2.4 Formas

Otro aspecto importante de HTML son las formas para insertar datos y presionar botones dentro de una página. Esta tarea se hace mediante dos tipos de etiquetas, la primera llamada "FORM", para la administración de la información, con formato `<FORM ACTION="url">` y que termina con la etiqueta `</FORM>`. El *url* indica el archivo donde se encuentra el programa encargado de manipular la información de la forma; por ejemplo, la dirección de un servlet o JSP, algo que describiremos más adelante en este capítulo.

La segunda etiqueta es "INPUT", donde se especifica el tipo de entrada de datos, con formato `<INPUT TYPE="tipo" opciones>` (sin terminación). Los tipos más importantes que se destacarán son "TEXT", correspondiente a un campo de texto, "PASSWORD", que corresponde a un campo de texto donde el texto insertado se despliega en la pantalla mediante caracteres codificados, por ejemplo "*", "SUBMIT" correspondiente a un botón y "CHECKBOX", que corresponde a un campo seleccionable de valor binario. En relación con las opciones, éstas son principalmente el nombre del elemento, `NAME="nombre"` y su valor, si cabe, `VALUE="valor"`. Otro formato de entrada similar a "INPUT" y que vale la pena resaltar es "SELECT", el cual permite generar múltiples opciones o "combos". A continuación se muestra un ejemplo sencillo del uso de una forma.

Ejemplo 11.3 Ejemplo de documento HTML con forma de entrada.

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservas de Vuelo
</TITLE>
</HEAD>
<BODY>
```

```

<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
Forma de entrada
<FORM ACTION="archivo.html"></P>
  <P><INPUT TYPE="SUBMIT" VALUE="Registrarse por Primera Vez"
NAME="boton1"></P>
  <P>Login: <INPUT TYPE="TEXT" NAME="login" SIZE="20"></p>
  <P>Password: <INPUT TYPE="PASSWORD" NAME="password" SIZE="20"></P>
  <P><INPUT type="SUBMIT" VALUE="OK" NAME="boton2">
  <INPUT TYPE="SUBMIT" VALUE="Salir" NAME="boton3"></P>
</FORM>
</BODY>
</HTML>

```

Nótese los siguientes aspectos:

- ▶ Se incluye como acción el nombre de *archivo.html*, lo cual significa que al presionar cualquier botón se desplegará el documento mencionado.
- ▶ Se definieron tres botones utilizando el tipo "SUBMIT". Nótese que a cada uno se le asignó un nombre y un valor distintos. El valor corresponde a la etiqueta del botón mientras que el nombre es interno y se utiliza eventualmente para acceder al valor asociado.
- ▶ Se definieron dos campos de texto utilizando el tipo "TEXT". Nuevamente se emplean nombres distintos y se asigna de manera adicional el tamaño del campo.

En la figura 11.6 se muestra el ejemplo 11.3 una vez desplegado.

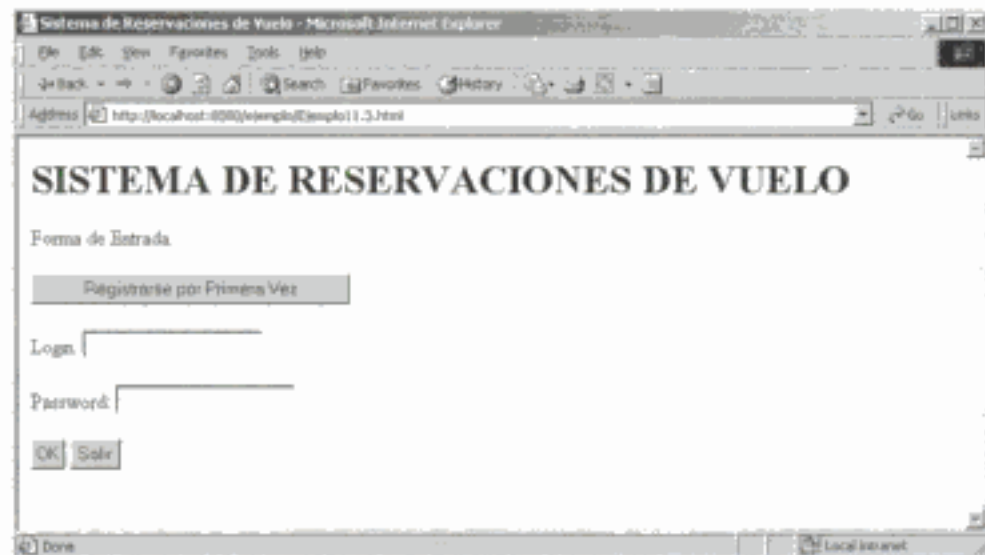


Figura 11.6 Despliegue en el navegador del ejemplo 11.3 de documento HTML con formas de entrada, campos de texto y botones.

En la figura 11.7 se muestra el despliegue luego de oprimir el botón "OK" y la introducción de los datos "alfredo" para el "login" y "weizenfeld" para el "password". Nótese que los valores de estos campos se incluyen como parámetros al siguiente documento a desplegar, que en este caso es el mismo.

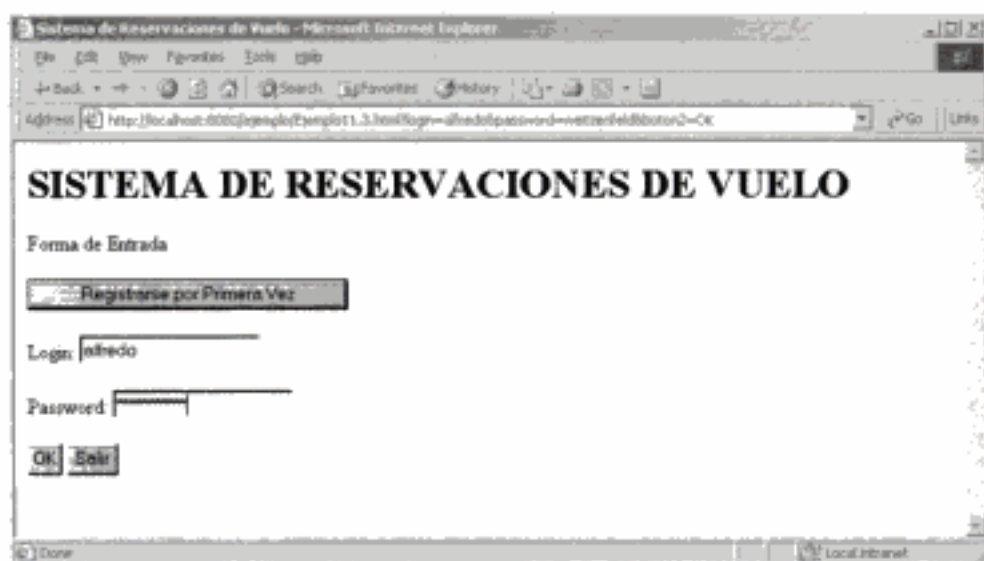


Figura 11.7 Despliegue en el navegador del ejemplo 11.3 de documento HTML con formas de entrada después de oprimir el botón de "OK".

11.3 Servlets

Como se ha visto hasta el momento, los documentos HTML sirven para desplegar información en un navegador, algo que en la actualidad está muy estandarizado. A pesar de incluir formas para campos de texto y botones, estos campos deben ser procesados por herramientas del lado del servidor, tarea que los servidores de tipo HTTP no pueden llevar a cabo. Para ello, se requiere de un *servidor de aplicación* que pueda procesar las acciones generadas en la página HTML del navegador. Otro aspecto fundamental de un servidor de aplicación es que debe ser capaz de administrar múltiples *sesiones* de usuarios o clientes a la vez. El servlet es la tecnología básica de Java para dar servicio de aplicación en Internet. En la figura 11.8 se muestra un esquema del funcionamiento

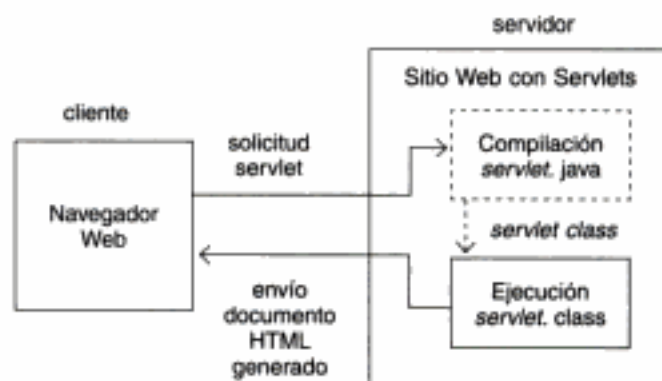


Figura 11.8 Esquema de una arquitectura *cliente-servidor* para Internet con solicitudes de servlets y envío de documentos en HTML generados dinámicamente por el servlet.

Hidden page

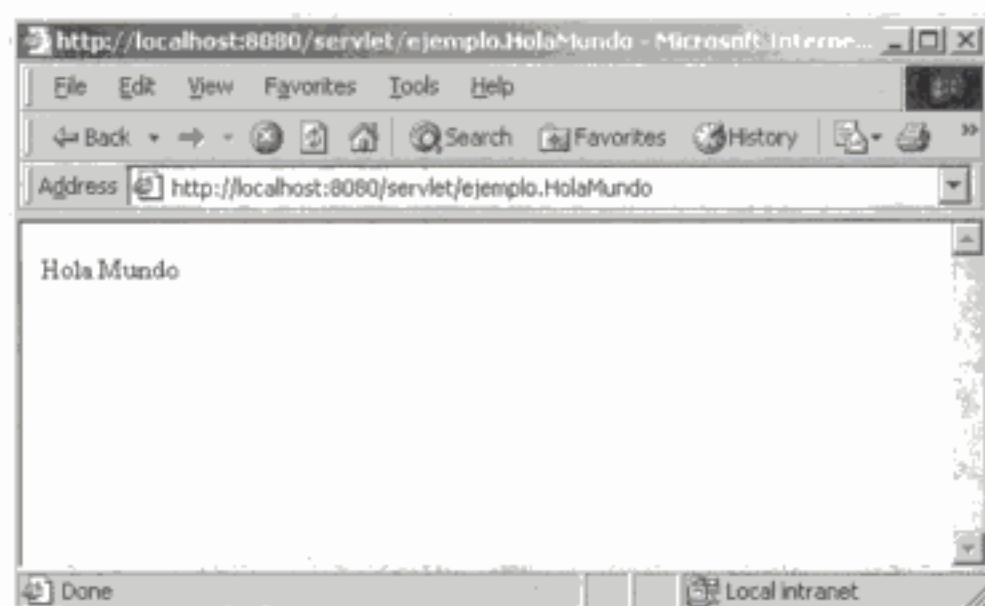


Figura 11.9 Despliegue en el navegador del ejemplo 11.4 de llamada al servlet *HolaMundo*.

guiente ejemplo, *HolaMundo2*, imprime con formato de tipo encabezado la frase "Hola Mundo". El servlet se muestra en el ejemplo 11.5.

Ejemplo 11.5 Ejemplo de modificación del formato del texto *HolaMundo* en servlet.

```
public class HolaMundo2 extends HttpServlet {
    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<HTML>\n" +
            "<HEAD><TITLE>Hola Mundo</TITLE></HEAD>\n" +
            "<BODY>\n" +
            "<H1>Hola Mundo</H1>\n" +
            "</BODY></HTML>");
    }
}
```

El formato básico es similar a la versión anterior, aunque deben considerarse los siguientes aspectos:

- El texto se escribe en formato HTML (en lugar de texto simple en el caso de omisión), lo cual se especifica mediante el tipo de contenido "text/html".

- En lugar de enviar el texto sin formato por el canal de respuesta, se envía la especificación de la pantalla completa comenzando con el encabezado `<HTML>` al principio de la cadena.

En la figura 11.10 se muestra el navegador con la nueva llamada y el despliegue del resultado. Nótese el cambio en el tipo de letra.

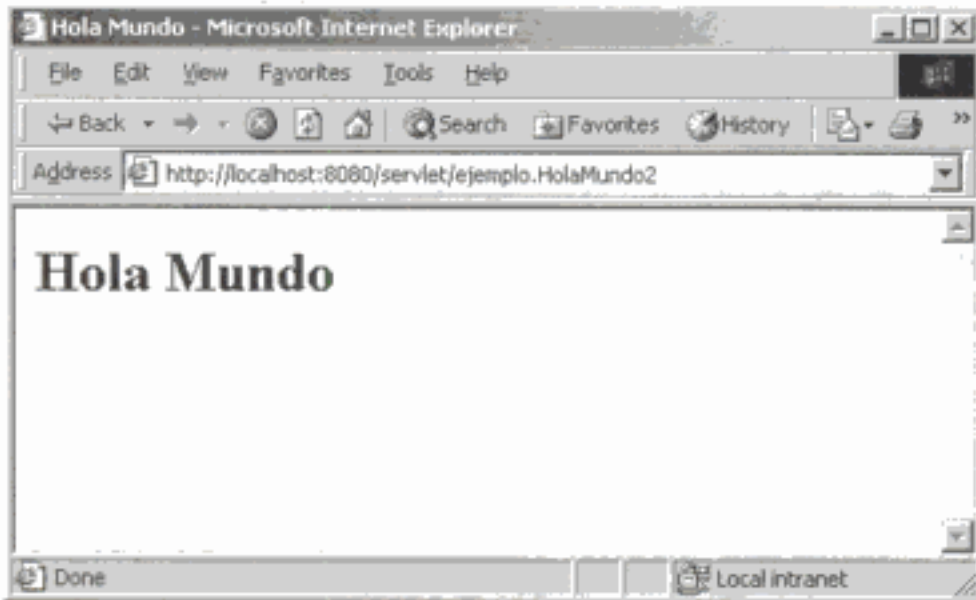


Figura 11.10 Despliegue en el navegador del ejemplo 11.5 de llamada al servlet *HolaMundo2*.

11.3.2 Formas

Consideremos las formas que fueron creadas en HTML, descritas en la sección 11.2.4, pero esta vez agregaremos la referencia al servlet como parte de la acción, tal cual se muestra en el ejemplo 11.6.

Ejemplo 11.6 Ejemplo de forma de entrada en el documento en HTML con referencia al servlet *ManejoForma*.

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservaciones de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
Forma de Entrada
<FORM ACTION="http://localhost:8080/servlet/ejemplo.ManejoForma"
METHOD="GET"></P>
  <P><INPUT TYPE="SUBMIT" VALUE="Registrarse por Primera Vez"
NAME="boton1"></P>
```

Hidden page

El formato general es similar a la versión anterior. En la figura 11.11 se muestra el resultado del ejemplo 11.6, la cual consiste en desplegar los valores insertados por el usuario, además de informar del botón presionado.

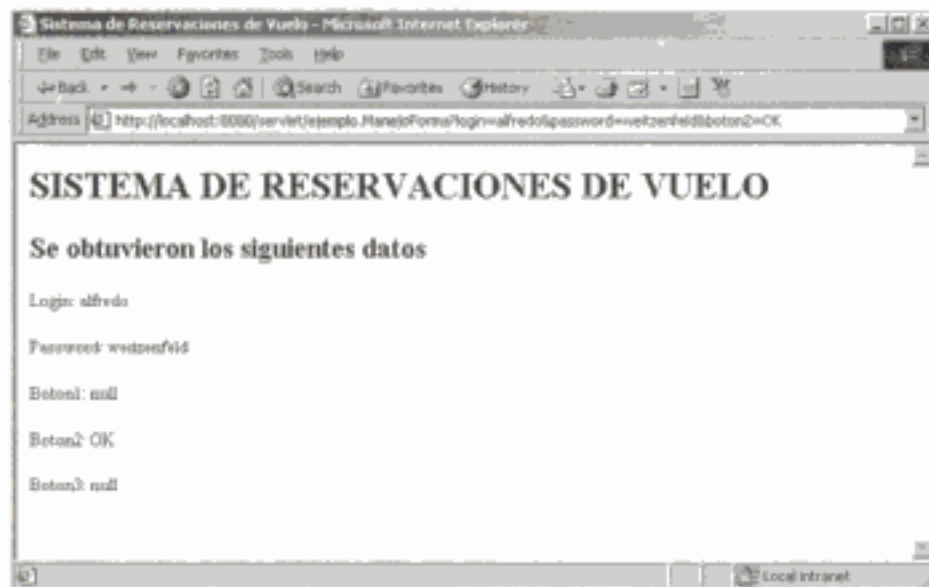


Figura 11.11 Despliegue en el navegador del resultado del ejemplo 11.6 de servlet que imprime los datos y el botón presionado por el usuario en una forma de entrada en HTML.

Nótese los siguientes aspectos del resultado:

- Los valores obtenidos por el servlet corresponden a los valores insertados en los campos de texto y al nombre del botón presionado. Los botones que no fueron presionados despliegan el valor "null".

Vale la pena observar el código generado dinámicamente como resultado de la ejecución del servlet y enviado al navegador, como se muestra en el ejemplo 11.6b. Nótese que este código en HTML es el resultado de insertar los valores de entrada en el servlet programado originalmente.

Ejemplo 11.6 (continuación) Resultado de la ejecución del servlet *ManejoForma*.

```
<HTML>
<HEAD>
<TITLE>Sistema de Reservas de Vuelo</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<P><H2>Se obtuvieron los siguientes datos</H2></P>
<P>Login: alfredo</P>
<P>Password: weizenfeld</P>
<P>Boton1: null</P>
<P>Boton2: OK</P>
<P>Boton3: null</P>
</BODY>
</HTML>
```

11.4 JSP

Una de las limitaciones de los servlets radica en que la generación de pantallas HTML se hace en su interior. Aunque el documento base en HTML sea generado externamente y guardado como archivo para posteriormente ser leído por el servlet, en algún momento el diseñador de pantallas deberá interactuar con el servlet y la programación en Java. En general, la tarea de diseño de pantallas en HTML recae en el diseñador gráfico más que en el desarrollador de software; por lo tanto, es una desventaja para el diseñador gráfico, tener que programar en un lenguaje como Java. Por otro lado, el desarrollador de software es normalmente el responsable de la programación. Para simplificar la tarea del diseño de pantallas en el cliente, bajo la tecnología de Java, se creó la tecnología de Java Server Pages (JSP), la cual agrega un nivel adicional de procesamiento sobre los servlets y simplifica su programación. En la figura 11.12 se muestra un esquema del funcionamiento de los archivos JSP. El navegador solicita un archivo JSP (en lugar de un documento HTML o servlet, como se hacía anteriormente). El archivo JSP, con terminación ".jsp" (*servlet-jsp.jsp*), debe pro-

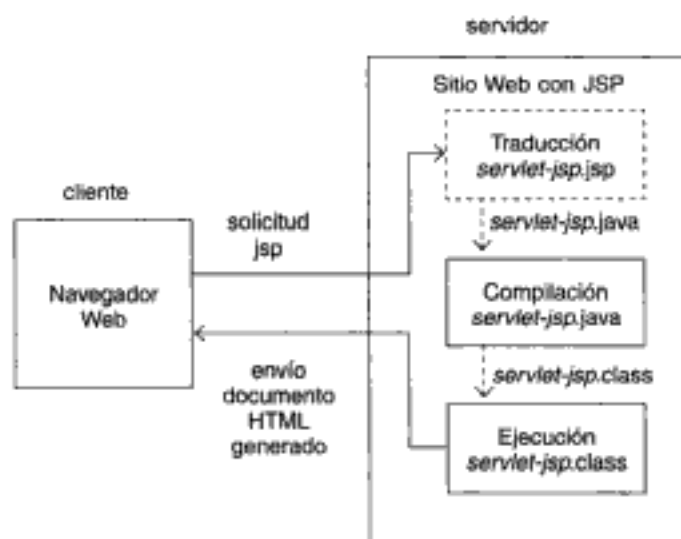


Figura 11.12 Esquema de una arquitectura *cliente-servidor* para Internet con solicitudes de servlets y envío de documentos en HTML generados dinámicamente por el servlet.

cesarse para generar un servlet (*servlet-jsp.java*). El servlet generado es nuevamente un programa en Java (*servlet-jsp.java*), el cual debe compilarse. Una vez compilado, el programa (*servlet.class*) se ejecuta para obtener como resultado un documento HTML generado dinámicamente y enviado al navegador. Nótese que en el momento de generar el servlet, se genera un segundo archivo de tipo HTML, el cual es posteriormente leído para generar el archivo HTML resultante. De tal manera se cumple el objetivo anterior, y el diseñador gráfico hace su diseño mediante un lenguaje de alto nivel que no involucra la programación en Java.

Hidden page

Hidden page

te de los valores asignados por el programador durante las declaraciones de las variables, como se muestra a continuación:

Ejemplo 11.8 Ejemplo de una tabla de tamaño variable con datos de salida.

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservas de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<%! int filas = 5; %>
<%! int columnas = 3; %>
<P>Tabla de Salida (Filas: <%=filas%>, Columnas: <%=columnas%>)</P>
<TABLE BORDER=10>
<% for (int i=0; i<filas; i++) { %>
    <TR>
    <% for (int j=0; j<columnas; j++) { %>
    <TD>Reserva <%=i+1%>.<%=j+1%></TD>
    <% } %>
    </TR>
<% } %>
</TABLE>
<FORM>
<P><INPUT type="SUBMIT" VALUE="OK" NAME="boton2">
<INPUT TYPE="SUBMIT" VALUE="Salir" NAME="boton3"></P>
</FORM>
</BODY>
</HTML>
```

Nótese los siguientes aspectos:

- ▶ Se agregaron dos declaraciones de enteros correspondientes a los valores de las filas y columnas de las tablas que serán generadas en el despliegue.
- ▶ Se imprime el valor de estas dos variables mediante expresiones de asignación.
- ▶ Se insertan dos scriptlets correspondientes a las instrucciones "for", las cuales controlan el número de veces que se van a desplegar los elementos de la tabla. Nótese que se debe agregar un scriptlet "}" al final de la impresión de la tabla para finalizar la instrucción del "for".
- ▶ Dentro de la tabla se asigna de manera dinámica la numeración de la fila y columna correspondientes.

En la figura 11.14 se muestra el resultado del ejemplo 11.8, el cual consiste en llenar los campos de texto con los valores insertados dentro del documento JSP.

Importar. Para facilitar el uso de clases ya definidas, es posible importarlas al archivo de manera análoga a la importación de clases realizada durante la definición de nuevas clases en Java. Las importaciones tienen la forma `<%@ page import importación %>`, donde *importación* sigue el formato tradicional de Java, aunque definido entre comillas, como se muestra a continuación:

```
<HTML>
<HEAD>
```

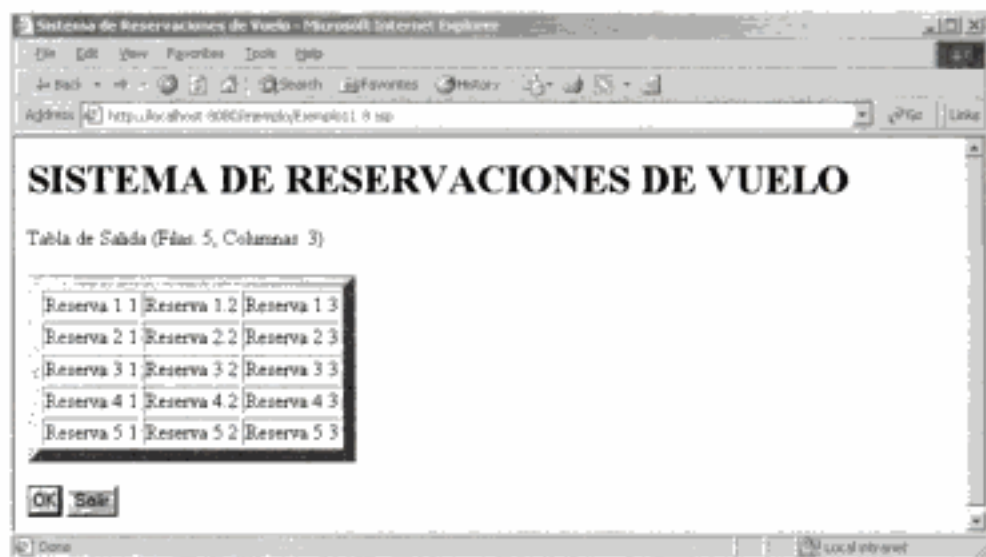



Figura 11.14 Despliegue en el navegador del ejemplo 11.8 de documento JSP con la tabla generada de tamaño variable.

```
<TITLE>
Sistema de Reservas de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<%@ page import="java.util.*" %>
</BODY>
</HTML>
```

Incluir. En un archivo de JSP es posible incluir otros archivos. El formato para llevar a cabo esta tarea es `<%@ include file=archivo %>` o `<jsp:include page=archivo />`, donde *archivo* puede estar en cualquier formato y terminación, incluyendo HTML y JSP, como se muestra a continuación:

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservas de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<%@ include file="archivo1.html" %>
<jsp:include page="archivo2.html" />
</BODY>
</HTML>
```

Vale la pena observar que la diferencia entre los dos formatos es que el primero se hace durante la solicitud, mientras que el segundo se hace durante la traducción del archivo.

Continuar. En un archivo de JSP es posible especificar el siguiente archivo que se desea desplegar. El formato para continuar es `<jsp:forward page=archivo`

Hidden page

Hidden page

Hidden page

plo, en la siguiente línea se instancia un bean llamado `ejemploBean` de la clase `ejemplos.EjemploBean` con alcance a lo largo de toda la sesión del usuario.

```
<jsp:useBean id="ejemploBean" class="ejemplos.EjemploBean"
scope="session" />
```

El formato del bean es una clase simple de Java, la cual debe incluir métodos que permitan leer y escribir parámetros. Por ejemplo, a continuación se muestra la clase `ejemplos.EjemploBean`, la que cuenta con métodos para leer y escribir dos valores: filas y columnas, las dos, en este caso, de tipo entero.

```
package ejemplos;
public class EjemploBean {
    private int filas = 1;
    private int columnas = 1;
    public int getFilas() { return filas; }
    public int getColumnas() { return columnas; }
    public void setFilas(int n) { filas = n; }
    public void setColumnas(int n) { columnas = n; }
}
```

Para leer los parámetros guardados en el bean se utiliza la siguiente llamada:

```
<jsp:getProperty name="nombre" property="propiedad" value="valor" />
```

El *nombre* corresponde al identificador para referirse al bean ya instanciado. La *propiedad* especifica el método que será llamado. Para generar el nombre completo del método se debe agregar un *get* y pasarse a mayúscula la primera letra del nombre de la propiedad. Por ejemplo, en las siguientes líneas se hacen llamadas a los métodos `getFilas` y `getColumnas` de la clase `ejemplos.EjemploBean`, respectivamente.

```
<jsp:getProperty name="ejemploBean" property="filas" />
<jsp:getProperty name="ejemploBean" property="columnas" />
```

Estas dos llamadas son equivalentes a las dos expresiones siguientes:

```
<%=ejemploBean.getFilas(); %>
<%=ejemploBean.getColumnas(); %>
```

Para escribir parámetros en el bean se utiliza la siguiente llamada:

```
<jsp:setProperty name="nombre" property="propiedad" />
```

El *nombre* corresponde al identificador para referirse al bean ya instanciado. La *propiedad* especifica el método que será llamado. Para generar el nombre completo del método se debe agregar un *set* y pasarse a mayúscula la primera letra del nombre de la propiedad. El *valor* especifica el valor que será pasado como parámetro en la llamada. Por ejemplo, en las siguientes líneas se hacen llamadas a los métodos `setFilas` y `setColumnas`, con valores "5" y "3", de la clase `ejemplos.EjemploBean`.

```
<jsp:setProperty name="ejemploBean" property="filas" value="5" />
<jsp:setProperty name="ejemploBean" property="columnas" value="3" />
```

Estas dos llamadas son equivalentes a las siguientes expresiones, respectivamente.

```
<% ejemploBean.setFilas(5); %>
<% ejemploBean.setColumns(3); %>
```

En el ejemplo 11.12 se muestra el uso de los JavaBeans, donde se utilizará el bean para guardar los valores de las filas y columnas del ejemplo 11.10.

Ejemplo 11.10 Ejemplo de una tabla de tamaño variable con datos de salida.

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservas de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<% int filas; %>
<% int columnas; %>
<jsp:useBean id="ejemploBean" class="ejemplos.EjemploBean" scope="session" />
<jsp:setProperty name="ejemploBean" property="filas" value="5" />
<jsp:setProperty name="ejemploBean" property="columnas" value="3" />
<P>Tabla de Salida (Filas:
<jsp:getProperty name="ejemploBean" property="filas" />
, Columnas:
<jsp:getProperty name="ejemploBean" property="columnas" />
)</P>
<% filas = ejemploBean.getFilas(); %>
<% columnas = ejemploBean.getColumnas(); %>
<TABLE BORDER=10>
<% for (int i=0; i<filas; i++) { %>
    <TR>
    <% for (int j=0; j<columnas; j++) { %>
        <TD>Reserva <%=i+1%>.<%=j+1%></TD>
    <% } %>
    </TR>
<% } %>
</TABLE>
<FORM>
    <P><INPUT type="SUBMIT" VALUE="OK" NAME="boton2">
    <INPUT type="SUBMIT" VALUE="Salir" NAME="boton3"></P>
</FORM>
</BODY>
</HTML>
```

Nótese los siguientes aspectos:

- ▶ Se agregaron dos declaraciones de enteros correspondientes a los valores de las filas y columnas de las tablas, aunque sin valor inicial.
- ▶ Se instancia un bean donde se guardarán los valores de las filas y columnas.
- ▶ Se escriben los valores de las filas y columnas en el bean mediante la llamada *setProperty*.

- Se imprime el valor de estas dos variables mediante llamadas *getProperty* al bean.
- Se asignan los valores de estas dos variables en las variables locales *filas* y *columnas* a través de scriptlets.
- Se insertan los scriptlets correspondientes a las instrucciones "for", las cuales controlan el número de veces que se van a desplegar los elementos de la tabla. Nótese que se debe agregar un scriptlet "}" al final de la impresión de la tabla para finalizar la instrucción del "for".
- Dentro de la tabla se asigna de manera dinámica la numeración de la fila y columna correspondiente.

El resultado del ejemplo 11.10 es similar al del ejemplo 11.8 que se muestra en la figura 11.14. La gran ventaja de este esquema es que los datos pueden compartirse más allá de la página actual, algo que no era posible con la declaración de variables locales.

A continuación, en el ejemplo 11.11 se muestra una extensión al ejemplo 11.9, pero esta vez se agrega un *bean* para guardar los datos insertados por el usuario.

Ejemplo 11.11 Ejemplo de continuación de un documento JSP a otro utilizando un bean.

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservaciones de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<jsp:useBean id="usuarioBean" class="ejemplos.UsuarioBean"
scope="session" />
<FORM ACTION="/ejemplo/UsuarioJSPBean.jsp" METHOD="POST"></P>
  <P><INPUT TYPE="SUBMIT" VALUE="Registrarse por Primera Vez"
NAME="boton"></P>
  <P>Login: <INPUT TYPE="TEXT" NAME="login" SIZE="20"></P>
  <P>Password: <INPUT TYPE="PASSWORD" NAME="password" SIZE="20"></P>
  <P>  <INPUT type="SUBMIT" VALUE="OK" NAME="boton">
    <INPUT TYPE="SUBMIT" VALUE="Salir" NAME="boton"></P>
</FORM>
</BODY>
</HTML>
```

Nótese los siguientes aspectos:

- Se instancia un *bean* llamado *UsuarioBean*, el cual se mostrará más adelante.
- Se especifica como dirección donde se procesará la acción al archivo */ejemplo/UsuarioJSPBean.jsp*.
- El resto de la especificación es similar a la del ejemplo 11.9.

El archivo *UsuarioBean.java* se muestra a continuación:

Ejemplo 11.11 (continuación) Validación del login y password (archivo UsuarioBean.jsp).

```
public class UsuarioBean {
    private String loginBD = "alfredo";
    private String passwordBD = "weitzenfeld";
    private String login = "";
    private String password = "";
    public String getLogin() { return login; }
    public String getPassword() { return password; }
    public void setLogin(String str) { login = str; }
    public void setPassword(String str) { password = str; }
    public boolean validar(String log,String pass) {
        login = log;
        password = pass;
        if (login.equals(loginBD) == true && password.equals(passwordBD) ==
true)
            return true;
        return false;
    }
}
```

El archivo /ejemplo/UsuarioJSPBean.jsp es el encargado de leer los parámetros insertados por el usuario y decidir cuál es el siguiente archivo de continuación, como se muestra en seguida:

Ejemplo 11.11 (continuación) Presentación del resultado (archivo UsuarioJSPBean.jsp).

```
<HTML>
<HEAD>
<TITLE>
Sistema de Reservaciones de Vuelo
</TITLE>
</HEAD>
<BODY>
<H1>SISTEMA DE RESERVACIONES DE VUELO</H1>
<jsp:useBean id="usuarioBean" class="ejemplos.UsuarioBean"
scope="session" />
<%! String login = ""; %>
<%! String password = ""; %>
<%! String showPage = ""; %>
<%! String action = ""; %>
<%! String mensaje = ""; %>
<%
    action = request.getParameter("boton");
    if (action.equals("Salir") == true)
        mensaje="Salida del sistema";
    else if (action.equals("OK") == true) {
        if (usuarioBean != null){
            login = request.getParameter("login");
            password = request.getParameter("password");
            if (usuarioBean.validar(login,password) == true)
                mensaje="Bienvenido, " + login;
            else
                mensaje="Usuario desconocido";
        }
        else
            mensaje="Bean desconocido";
    }
}
```

Hidden page

Hidden page

```

        if (usuarioBean.validar(login, password) == true)
            mensaje="Bienvenido, " + login;
        else
            mensaje="Usuario desconocido";
        }
    }
    else
        mensaje="Bean desconocido";
    }
    else
        mensaje=action;
    showPage = "/ejemplo/mensajePage.jsp?mensaje=" + mensaje;
    gotoPage(showPage,request, response);
}
private void gotoPage(String address,HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
    ServletContext sc = getServletContext();
    RequestDispatcher dispatcher = sc.getRequestDispatcher(address);
    dispatcher.forward(request, response);
}
}
}

```

El resultado del ejemplo es idéntico al obtenido en los ejemplos 11.9 y 11.11. Su organización cambia, pues la lógica que antes se encontraba en el JSP, en el archivo `UsuarioJSPBean.jsp`, ahora se encuentra en el servlet. Esta modificación es muy importante, ya que mediante este cambio el archivo JSP está limitado a contener funcionalidad únicamente de clase borde. En este caso, se elimina el segundo archivo JSP por completo, aunque podrían combinarse múltiples archivos JSP en relación con uno o varios servlets, como veremos con el ejemplo del sistema de reservaciones de vuelos en el capítulo 12.

RESUMEN

En este capítulo se introduce al lector en la programación para Internet, otorgándole especial importancia a la Web, mediante la utilización de tecnología de Java. Se presenta el concepto de arquitectura cliente servidor, en contraste con la arquitectura stand-alone para un solo usuario concurrente. Se describe y dan ejemplos de la programación en HTML, Servlets y JSP. Se introduce la arquitectura basada en la integración de servlets y JSP, la cual es utilizada para desarrollar el sistema de reservaciones de vuelos en la Web.

REFERENCIAS

1. <http://www.w3.org/MarkUp>
2. <http://www.macromedia.com/software/flash>
3. <http://javascript.internet.com>
4. <http://msdn.microsoft.com/library/en-us/script56/html/vtorVBScript.asp>
5. [http://msdn.microsoft.com/library/en-us/script56/html/js56\(sor\)JScrip.asp](http://msdn.microsoft.com/library/en-us/script56/html/js56(sor)JScrip.asp)
6. <http://www.cgi-resources.com>
7. <http://msdn.microsoft.com/asp>
8. <http://www.macromedia.com/software/coldfusion>
9. <http://www.php.net>
10. <http://java.sun.com/products/servlet>
11. <http://java.sun.com/products/jsp>
12. Tim Berners-Lee, 1999, *Weaving the Web, The Original Design and Ultimate Destiny of the World Wide Web*, Harper-Collins.
13. <http://www.xml.com>

Desarrollo de software para Internet

En este capítulo se extiende el desarrollo de software descrito en la parte III del libro, de aplicaciones *stand-alone* a aplicaciones *cliente-servidor* en Internet. Se aplica nuevamente la metodología de diseño por responsabilidades.¹ Un objetivo que se ha mencionado a lo largo del libro y que se destacará nuevamente en este capítulo es el de lograr *extensibilidad* a partir del sistema existente. A menos que se hagan cambios en la especificación del sistema, tanto los requisitos como el análisis no tendrían por qué modificarse. De tal forma, se busca minimizar el número de cambios y tratar de aprovechar al máximo la arquitectura y modelos de diseño, implementación y pruebas existentes. En este capítulo se presentarán estos cambios y extensiones.

12.1 Modelo de diseño

Como se vio en el capítulo 8, el modelo de diseño consiste principalmente de los diseños de objetos y sistema. Comenzaremos con el diseño de sistema para evaluar el efecto del nuevo ambiente de implementación.

12.1.1 Diseño de sistema

Como parte del diseño de una arquitectura *cliente-servidor*, es necesario especificar un control distribuido de tareas entre el cliente y el servidor. Si recordamos que en la versión aplicación, el sistema completo se ejecuta como un solo proceso, es necesario especificar a quién le corresponde ahora llevar a cabo las diversas tareas, entre ellas el despliegue gráfico y el acceso a la base de datos. En general, la idea es maximizar el procesamiento en el servidor de manera que se limiten las necesidades de procesamiento en el cliente y se eviten situaciones que pudieran afectar la seguridad del sistema; por ejemplo, asegurar que todo acceso a la base de datos se haga desde el servidor y no desde un cliente. El enfoque que se aplicará será asignar únicamente el despliegue gráfico e interacción con el usuario en el cliente, mientras que todo el resto de la aplicación se hará en el servidor. Para llevar a cabo esta distribución de tareas se harán modificaciones en el sistema en relación con: 1. despliegue gráfico, 2.

control de despliegue gráfico, 3. control de aplicación y 4. control de sesión. En la figura 12.1 se muestra el diseño conceptual de la arquitectura distribuida que se utilizará para el sistema de reservaciones de vuelos en Internet.

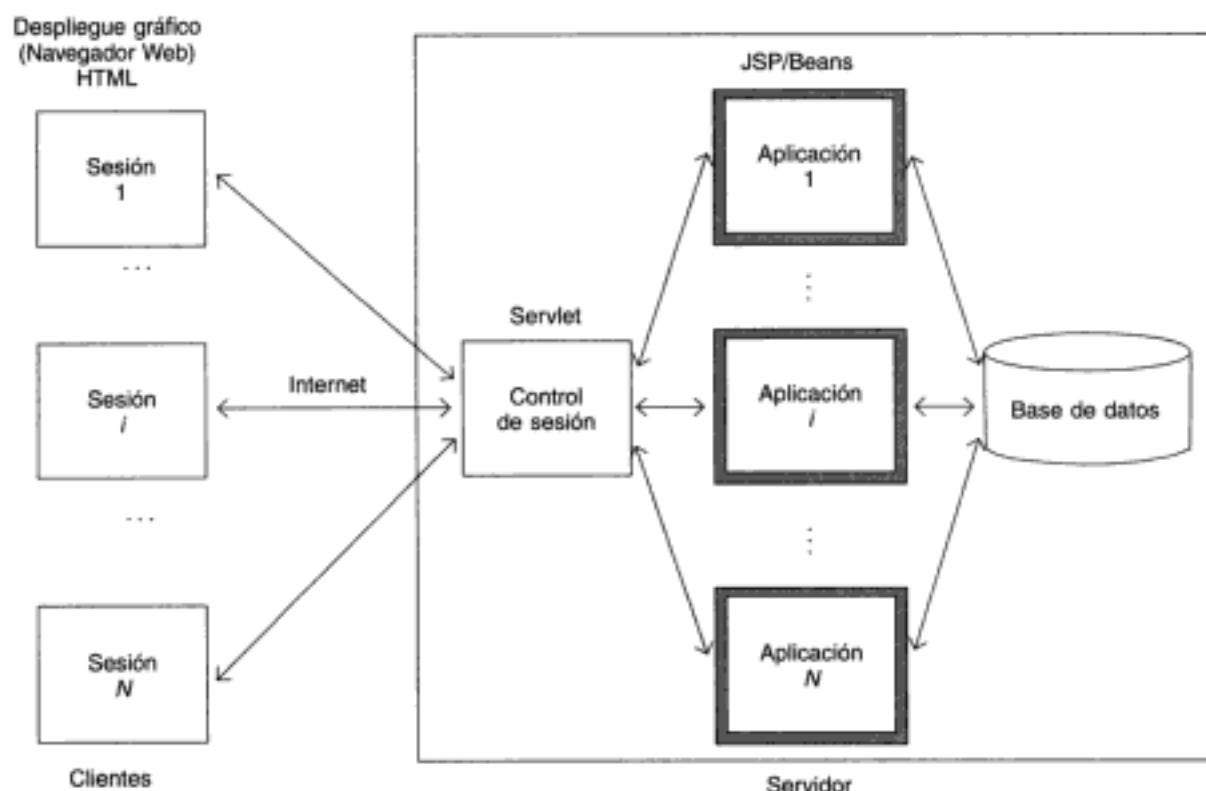


Figura 12.1 Arquitectura cliente-servidor para el sistema de reservaciones de vuelos.

- **HTML: despliegue gráfico.** Como consecuencia de la distribución del sistema, es necesario modificar el sistema gráfico ya que las pantallas deberán desplegarse en un navegador de Internet. Aunque existen diversas opciones tecnológicas de diseño de pantallas, como son los applets, flash, javascript, etc., el diseño lo haremos con tecnología de HTML. Este enfoque, que limita la complejidad de las interfaces gráficas, es suficiente para duplicar el diseño del prototipo original del sistema de reservaciones de vuelo. Por lo tanto, será necesario traducir todas las pantallas existentes, basadas en la biblioteca AWT de Java, a HTML. Será necesario hacer ciertos ajustes adicionales en la aplicación, en aquellos lugares donde se hace mención a las pantallas originales, como durante su creación y despliegue.
- **JSP: control de despliegue gráfico.** Se utilizará un control de despliegue gráfico mediante scripts de JSP que permitan fácilmente insertar datos en los archivos HTML resultantes. Estas extensiones serán limitadas para no agregar *control* en las pantallas de despliegue gráfico.
- **JavaBeans: control de aplicación.** Se utilizarán JavaBeans para mantener una referencia entre la sesión del usuario, incluyendo despliegues gráficos, y la aplicación.
- **Servlet: control de sesión.** Aunque podría administrarse el sistema mediante JSP y JavaBeans, se agrega un servlet que se encargará de identificar explícitamente las diferentes sesiones de usuario y asegurar que cada

una accese de manera correcta y segura su instancia de aplicación correspondiente. En lugar de responsabilizar a los archivos JSP y JavaBeans, de llevar a cabo la continuación a las siguientes pantallas, esta tarea será llevada a cabo por el servlet. Este enfoque permite reducir nuevamente el control en los despliegues gráficos y concentrar toda la seguridad en el manejo de los despliegues en un solo lugar, el servlet.

12.1.2 Diseño de objetos

Como parte del diseño de objetos se modificarán ciertas clases, lo que permitirá mantener bajo una misma arquitectura general tanto la aplicación *stand-alone* como la de *cliente-servidor*. En la figura 12.2 se muestra la organización

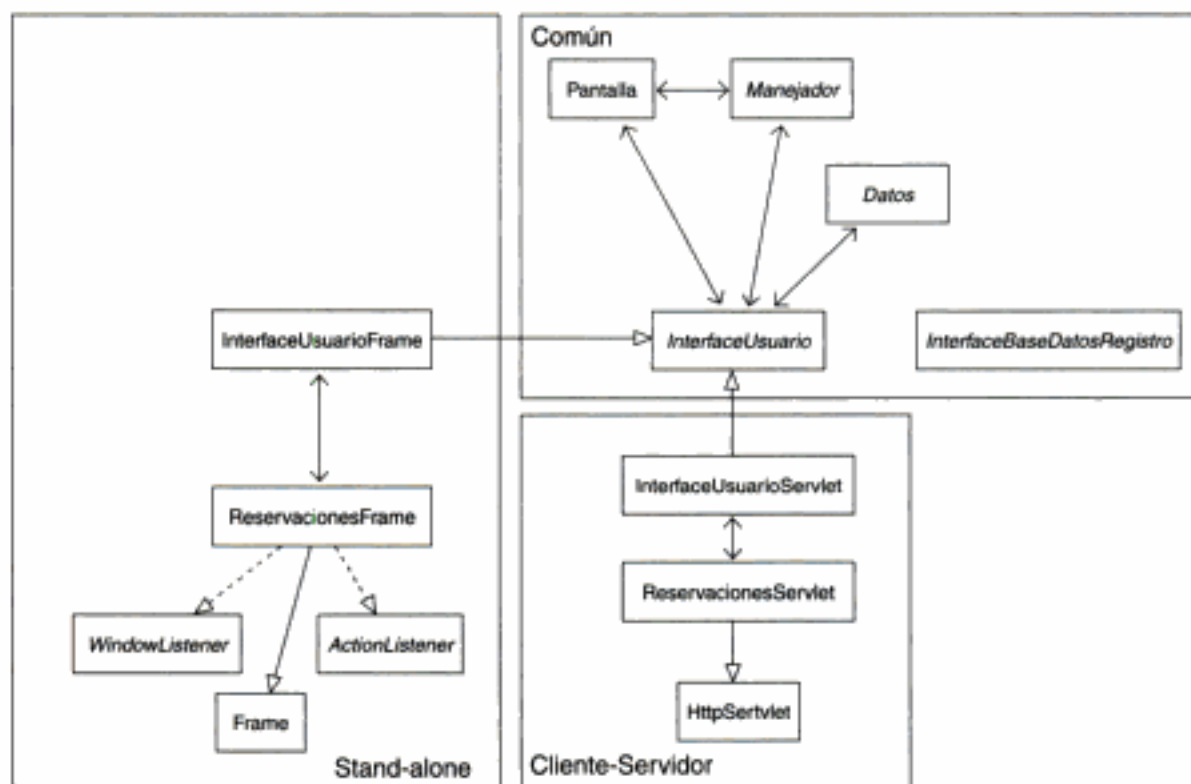


Figura 12.2 Organización de clases comunes, particulares a la arquitectura *stand-alone* y *cliente-servidor* en el sistema de reservaciones de vuelos.

de clases de acuerdo a clases comunes para ambas arquitecturas y aquellas que son propias de cada una. Observe que los dos tipos de aplicaciones son integrados con el resto de la arquitectura común a ambas mediante la extensión de herencia a partir de la clase *InterfaceUsuario*. En las siguientes secciones se verán los cambios que deberán hacerse a estas clases, además de la creación de nuevas, como en el caso de *InterfaceUsuarioFrame* y *ReservacionesFrame*, para la arquitectura *stand-alone*, del lado izquierdo en la figura, e *InterfaceUsuarioServlet* y *ReservacionesServlet*, para la arquitectura *cliente-servidor* en Internet, abajo del lado derecho en la figura.

CLASES COMUNES

La primera parte de esta extensión, en lo referente a clases comunes, consiste en generalizar la clase *InterfaceUsuario* y eliminar su herencia de *Frame*. Esta etapa es necesaria para que la clase *InterfaceUsuario* pueda ser utilizada bajo ambas arquitecturas. De manera similar, se aprovecha la clase *Pantalla* ya existente, modificándola para que apoye a ambas arquitecturas. Será necesario también modificar los manejadores de manera que, en lugar de instanciar directamente la pantalla, como se hacía originalmente, por medio de un "new", ahora se haga mediante un método que indirecte esta llamada. Éstos y otros cambios son descritos a continuación con más detalle.

INTERFACEUSUARIO

Se deben hacer los siguientes cambios a la clase *InterfaceUsuario*:

- ▶ La clase se define como abstracta para dar lugar a especializaciones de esta clase para las dos versiones del sistema.
- ▶ Se elimina la herencia de *Frame* y la implementación de los "listeners" (*WindowListener* y *ActionListener*), los cuales son implementados en la clase *ReservacionesFrame*, aspecto que ya no se aplica en la versión *cliente-servidor*. Nótese que deben eliminarse de la clase *InterfaceUsuario* todos los métodos sobrescritos de los "listeners".
- ▶ El método *desplegarPantalla*, que anteriormente desplegaba pantallas utilizando la biblioteca AWT, se convierte en abstracto para dar lugar al polimorfismo y su sobrescritura en las clases que heredan de *InterfaceUsuario*.
- ▶ Se agrega al contrato "Desplegar Pantalla" un nuevo método abstracto *crearPantalla* que permite instanciar la pantalla de diferente manera, de acuerdo con la arquitectura utilizada.
- ▶ Se agregan al contrato "Desplegar Pantalla" métodos para la lectura y escritura de datos en las pantallas. Estos métodos son *leerElementos*, *escribirElementos* y *leerTexto*, que, nuevamente, podrán ser sobrescritos en las clases que heredan de *InterfaceUsuario*.
- ▶ Se mantiene el método *enviarEvento*, originalmente implementado como *actionPerformed*, para ser sobrescrito por las subclases de *InterfaceUsuario*.

La tarjeta de clase *InterfaceUsuario* se muestra en la tabla 12.1. Observe que se elimina la lista de subclases colaboradoras de *desplegarPantalla*, y se mantiene sólo la clase *Pantalla*, ya que ésta es ahora concreta y aparece como parámetro en el método.

PANTALLA

Se modifica la clase *Pantalla* en apoyo de ambas arquitecturas en lo relacionado con el despliegue gráfico. Los cambios que se le introducen a dicha clase son los siguientes:

- ▶ La clase se hace concreta para permitir su utilización en la arquitectura *cliente-servidor* sin necesidad de contar con subclases, dado que éstas serán creadas directamente con base en HTML.

Hidden page

Hidden page

Tabla 12.3 Tarjeta para la clase *Manejador*.

Clase: Manejador	
Descripción: Superclase heredada por todos los manejadores del sistema.	
Módulo: Principal	
Estereotipo: Control	
Propiedades: Abstracta	
Superclase:	
Subclase: ManejadorPrincipal, ManejadorServicio, ManejadorRegistroUsuario, ManejadorRegistroTarjeta	
Atributos: InterfaceUsuario, Pantalla, ManejadorServicio, Manejador	
Contratos	
1. Manejar Evento	
manejarEvento(Evento) devuelve void <i>Método encargado de recibir eventos del sistema de ventanas a través de la InterfaceUsuario</i>	
Responsabilidades Privadas	
desplegarPantalla() devuelve void <i>Método encargado de desplegar las pantallas administradas por los manejadores. Se solicita al SubsistemaInterfaceUsuario que las despliegue</i>	SubsistemaInterfaceUsuario (1)
crearPantalla(String) devuelve void <i>Método para la creación de pantallas</i>	SubsistemaInterfaceUsuario (1)
leerElementos(Pantalla, Datos) devuelve void <i>Método abstracto para leer datos de una pantalla</i>	SubsistemaInterfaceUsuario (1)
escribirElementos(Pantalla, Datos) devuelve void <i>Método abstracto para escribir datos en una pantalla</i>	SubsistemaInterfaceUsuario (1)
manejarEventoOfrecerServicio() devuelve void <i>Método encargado de solicitar al SubsistemaServicio que ofrezca los servicios correspondientes</i>	SubsistemaServicio (2)
manejarEventoSalir() devuelve void <i>Método encargado de salir del sistema</i>	

MANEJADORES ESPECIALIZADOS

Los cambios que presentan los diferentes manejadores son los siguientes:

- Todos los manejadores llaman al método *crearPantalla*, definido en la superclase *Manejador*, en lugar de instanciar directamente la pantalla que iban a desplegar. Este procedimiento permite sobrescribir la llamada en el apoyo de ambas arquitecturas.

En particular, en la clase *ManejadorPrincipal* se hacen los siguientes cambios:

- Las llamadas especializadas para obtener información del login y contraseña se hacen a partir de la clase *InterfaceUsuario* en lugar de *Pantalla*.
- Se elimina el método *main*, el cual es reubicado en la clase *InterfaceUsuarioFrame* por ser instanciada únicamente en el caso de una aplicación *stand-alone*.

INTERFACEBASEDATOSREGISTRO

Se modifica la clase *InterfaceBaseDatosRegistro* en apoyo de ambas arquitecturas en lo relacionado con el acceso a la base de datos. Los cambios que se introducen a la clase *InterfaceBaseDatosRegistro* son los siguientes:

- Se agregan los métodos *abrir* y *cerrar* para la apertura y cierre respectivos de las conexiones a la base de datos.
- Se modifican las llamadas a la base de datos, de manera que cada una incluya inicialmente la apertura de la conexión con ella. La apertura es seguida por la llamada correspondiente y terminada por el cierre de la conexión. Esta modificación es necesaria para permitir múltiples accesos a la base de datos de manera simultánea.

La tarjeta de clase *InterfaceBaseDatosRegistro* se muestra en la tabla 12.4.

Tabla 12.4 Tarjeta para la clase *InterfaceBaseDatosRegistro*.

Clase: InterfaceBaseDatosRegistro	
Descripción: la información de cada usuario se almacena en la base de datos de registro, la cual se accesa mediante la interfaz de la base de datos de registro. Esto permite validar a los distintos usuarios, además de guardar información sobre la tarjeta de crédito para pagos en línea.	
Módulo: Registro.InterfaceBD	
Estereotipo: Borde	
Propiedades: Concreta	
Superclase:	
Subclase:	
Atributos:	
Contratos	
1. Registrar Usuario / 2. Registrar Tarjeta	
validarRegistro(Datos, String log, String pass) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la validación de un usuario</i>	BaseDatosRegistro
crearRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la creación de un nuevo RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro

continúa

Tabla 12.4 Continuación

obtenerRegistro(Datos, String log) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la obtención de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
actualizarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la actualización de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
eliminarRegistro(Datos) devuelve boolean <i>Método encargado de solicitar a la BaseDatosRegistro la eliminación de un RegistroUsuario o RegistroTarjeta</i>	BaseDatosRegistro
Responsabilidades privadas	
abrir() devuelve void <i>Método encargado de abrir una nueva conexión a la BaseDatosRegistro</i>	BaseDatosRegistro
cerrar() devuelve void <i>Método encargado de cerrar la conexión a la BaseDatosRegistro</i>	BaseDatosRegistro

INTERFACEARCHIVOREGISTRO

En el caso de la arquitectura *cliente-servidor*, es deseable restringir el acceso externo únicamente a bases de datos y no a archivos. Esta limitación se debe a que el manejo de archivos fue hecho de manera limitada, y tendría que extenderse para apoyar acceso simultáneo de usuarios, algo que no se hará para el ejemplo del sistemas de reservaciones de vuelo.

CLASES STAND-ALONE

Como parte de la arquitectura *stand-alone* se agregan dos nuevas clases, *InterfaceUsuarioFrame* y *ReservacionesFrame*, la última de la cuales hereda de *Frame*. A pesar de que las clases especializadas a partir de la clase *Pantalla* son particulares para la arquitectura *stand-alone*, no es necesario modificar las originales ya existentes. Estas clases son descritas a continuación.

INTERFACEUSUARIOFRAME

Se agrega una nueva clase *InterfaceUsuarioFrame* basada en la clase original de *InterfaceUsuario*, la cual hereda de la nueva superclase con el mismo nombre. El método *enviarEvento* se hereda de la superclase *InterfaceUsuario*, por lo cual no es necesario sobrescribirlo. Las especializaciones de la clase *InterfaceUsuarioFrame* se describen a continuación:

- Se sobrescribe el método *desplegarPantalla* de manera que permita desplegar pantallas implementadas en AWT, de acuerdo con los cambios en el manejo de pantallas para la arquitectura *stand-alone*.

Hidden page

Hidden page

Hidden page

Hidden page

PANTALLAS JSP

En el caso de la arquitectura *cliente-servidor*, es necesario hacer un diseño de pantallas directamente en HTML, utilizando el control de JSP y JavaBeans. Se debe crear un nuevo archivo por cada pantalla existente, dándole una terminación `“.jsp”`. El control de JSP se limitará a administrar el bean del cual se obtendrán los datos de regreso para el usuario. Los datos insertados por el usuario en la pantalla son administrados por el servlet.

12.1.3 Diagramas de secuencias

Es interesante apreciar que los diagramas de secuencia originalmente diseñados son reutilizables casi en su totalidad, dado que describen la funcionalidad de la aplicación más que el manejo de los despliegues gráficos. Si no se agregan nuevas clases a los diagramas de secuencia ya existentes, y se mantiene la superclase *InterfaceUsuario*, a pesar de ser abstracta, los diagramas serían idénticos a los ya diseñados con excepción de los cambios hechos en el acceso a la base de datos, los cuales requieren de una apertura de la conexión previo a cada llamada de la clase *InterfaceBaseDatosRegistro* a la base de datos y su cierre posterior inmediato, como se muestra en la figura 12.3.

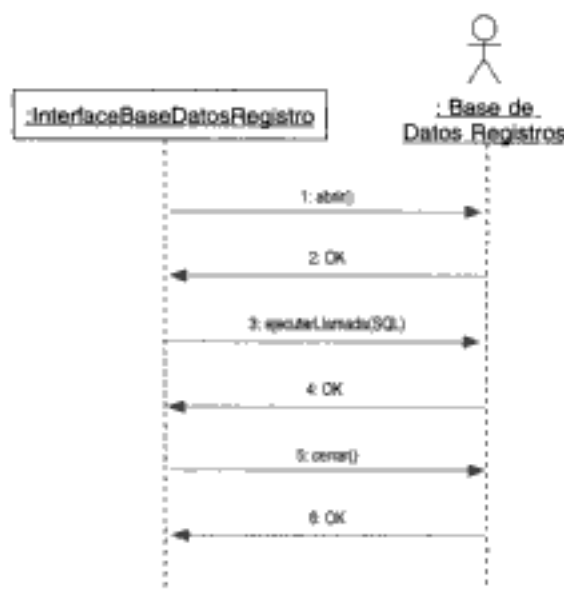


Figura 12.3 Diagrama de secuencias para el acceso a la *Base de Datos de Registros* a partir de una llamada de la clase *InterfaceBaseDatosRegistro*. Inicialmente se abre la conexión, luego se hace la llamada y de inmediato se cierra la conexión. Este ciclo se repite durante cada acceso a la base de datos.

En el caso de la arquitectura *cliente-servidor*, la figura 12.4 muestra la secuencia de envío de un evento del usuario al servlet y viceversa. El ejemplo muestra el caso para el despliegue de la *PantallaPrincipal*, con el usuario presionando el botón “Registrar Por Primera Vez”. La llamada entre el *ManejadorPrincipal*, la clase *InterfaceUsuarioServlet* y la clase *ReservacionesServlet* es similar al caso original. Sin embargo, existe cierta diferencia en la implementación de las lla-

Hidden page

Hidden page

Hidden page

Hidden page

El método `abrirConexion` se mantiene igual que el anterior.

```
private void abrirConexion(String url,String log,String pass) {
    try {
        con = DriverManager.getConnection (url, log, pass);
        ...
    }
    ...
}
```

Se agrega un segundo método, `cerrar`, encargado de cerrar la conexión a la base de datos una vez terminado el acceso. Nótese que la variable “con” guarda la referencia a la conexión abierta, como se muestra a continuación:

```
public void cerrar() {
    try {
        if (con != null)
            con.close();
    }
    catch (SQLException ex) {
        ...
    }
}
```

Los cambios requeridos para apoyar las modificaciones anteriores se hacen principalmente mediante dos métodos. El primero es el `actualizarRecordSetRegistro` el cual debe abrir la conexión, actualizar la base de datos y finalmente cerrar la conexión.

```
private boolean actualizarRecordSetRegistro(String query) {
    ...
    try {
        abrir();
        int n = stmt.executeUpdate (query);
        ...
    }
    ...
    cerrar();
    return fg;
}
```

El segundo método es `leerRecordSetRegistro`, el cual debe, de manera similar, abrir la conexión, leer de la base de datos y luego cerrarla

```
private boolean leerRecordSetRegistro(String query,Datos datos) {
    ...
    try {
        abrir();
        rs = stmt.executeQuery (query);
        ...
    }
    ...
    cerrar();
    return fg;
}
```

UTILERÍAS

Para facilitar el uso de ciertos métodos comunes usados por diversas clases se crea una nueva clase llamada "Utilerias". El primer método es `instanciarClase` el cual instancia un objeto a partir del nombre de la clase de manera dinámica.

```
public Object instanciarClase(String classname){
    Object object = null;
    try {
        Class c = Class.forName(classname);
    }
    catch (Exception ex){
        ex.printStackTrace();
    }
    return object;
}
```

El método `getClassPath` devuelve el nombre del paquete donde se encuentra la clase que define cierto objeto.

```
public static String getClassPath(Object object){
    String name;
    Class objclass = object.getClass();
    String fullname = objclass.getName();
    int index = fullname.lastIndexOf('.');
    if (index > 0)
        name = fullname.substring(0,index);
    else
        name = fullname;
    return name;
}
```

El método `getClassName` devuelve el nombre de la clase a partir de cierto objeto.

```
public static String getClassName(Object object){
    String name;
    Class objclass = object.getClass();
    String fullname = objclass.getName();
    int index = fullname.lastIndexOf('.');
    if (index > 0)
        name = fullname.substring(index+1);
    else
        name = fullname;
    return name;
}
```

12.2.2 Clases stand-alone

A continuación se describen las clases de la arquitectura *stand-alone*.

INTERFACEUSUARIOFRAME

La clase `InterfaceUsuarioFrame`, descrita en la tarjeta de clase de la tabla 12.5, hereda de `InterfaceUsuario` y tiene como atributo una referencia a la clase `ReservacionesFrame`.

```
public class InterfaceUsuarioFrame extends InterfaceUsuario {
    private ReservacionesFrame frame;
}
```

El constructor se encarga de instanciar objetos de *ReservacionesFrame* y *ManejadorPrincipal*. Dado que ésta es la versión de tipo aplicación *stand-alone*, se envía un argumento de instanciación a *ManejadorPrincipal* de tipo "true".

```
public InterfaceUsuarioFrame() {
    frame = new ReservacionesFrame(this);
    manejador = new ManejadorPrincipal(this,true); // true = app
}
```

El despliegado de las pantallas es similar a la versión anterior. La única diferencia es la partición de la funcionalidad original entre esta clase y la de *ReservacionesFrame*. En esta última se hace la llamada final de "show", la cual se hereda de *Frame*.

```
public void desplegarPantalla(Pantalla p) {
    if (p != null) {
        pantalla = p;
        if (frame != null)
            frame.desplegarPantalla(p);
    }
}
```

Uno de los cambios más importantes, es la posibilidad de instanciar pantallas de manera dinámica a partir de sus nombres, tanto para la aplicación *stand-alone* como para *cliente-servidor*. Este objetivo lo logramos mediante el método *crearPantalla*, el cual recibe como parámetros la ubicación o paquete al cual pertenece la pantalla y su nombre correspondiente. Nótese que se utiliza la clase "Utilerias" donde se definen métodos estáticos genéricos. Los dos últimos métodos actualizan las referencias dentro de la clase *Pantalla*.

```
public Pantalla crearPantalla(String classpath,String classname) {
    pantalla = (Pantalla) Utilerias.instanciarClase(
        classpath + "." + classname);
    pantalla.setInterfaceUsuario(this);
    pantalla.inicializarFrame();
    return pantalla;
}
```

Se sobrescriben tres métodos: *escribirElementos*, *leerElementos* y *leerTexto* para escribir y leer datos de una pantalla a través de la clase *InterfaceUsuarioFrame*.

```
public void escribirElementos(Pantalla p,Datos datos) {
    if (p != null)
        p.escribirElementos(datos);
}
public void leerElementos(Pantalla p, Datos datos) {
    if (p != null)
        p.leerElementos(datos);
}
public String leerTexto(Pantalla p, String str) {
    return p.leerTexto(str);
}
```

Hidden page

Hidden page

Hidden page

Hidden page

Hidden page

Hidden page

```
String password = interfaceUsuario.leerDatos("password");
String rpassword = interfaceUsuario.leerDatos("rpassword");
%>
```

Posteriormente se define la acción de continuación, correspondiente al servlet *ReservacionesServlet*,

```
<form action="/servlet/reservaciones.interfaceUsuario.ReservacionesServlet" method="POST">
```

La inserción de los datos en el despliegue provienen del bean y utilizan scriptlets, por ejemplo, `<%=nombre%>`, como se muestra a continuación:

```
Nombre: <input type="text" name="nombre" size="20"
value="<%=nombre%>">
Apellido: <input type="text" name="apellido" size="20"
value="<%=apellido%>">
...
Password: <input type="password" name="password" size="25"
value="<%=password%>">
Repetir Password: <input type="password" name="rpassword" size="29"
value="<%=rpassword%>">
```

Las siguientes líneas definen el resto de los elementos gráficos en HTML,

```
<input type="submit" value="Eliminar" name="Button">
<input type="submit" value="Actualizar" name="Button">
<input type="submit" value="Registrar Tarjeta" name="Button">
<input type="submit" value="Servicios" name="Button">
<input type="submit" value="Salir" name="Button">
```

El resto del documento contiene elementos ya descritos anteriormente.

12.2.4 Diagrama de clases

El diagrama de clases que se muestra es una extensión del diagrama de la figura 12.2, en la figura 12.5.

12.3 Modelo de pruebas

En esta sección se verifica la extensión del sistema para Internet de acuerdo con la prueba de requisitos especificada en el capítulo 10. A continuación se revisan los casos de uso principales: *Registrar Usuario* y *Registrar Tarjeta*. Se omiten los diagramas de secuencia y las referencias a la base de datos, ya que son similares a lo presentado en el capítulo 10.

12.3.1 Registrar Usuario

Se prueban las secuencias más importantes del caso de uso *Registrar Usuario*: *Crear Registro Usuario*, *Actualizar Registro Usuario* y *Eliminar Registro Usuario*.

CREAR REGISTRO USUARIO

La secuencia comienza cuando el usuario presiona el botón "Registrarse por Primera Vez" en la *PantallaPrincipal (P-I)*, como se muestra en la figura 12.6.

Hidden page



Figura 12.6 PantallaPrincipal (P-1) con el botón "Registrarse por Primera Vez" oprimido.

A continuación el sistema presenta la *PantallaCrearRegistroUsuario (P-3)*, como se muestra en la figura 12.7.

El usuario debe llenar la *PantallaCrearRegistroUsuario (P-3)* con datos de registro. Al finalizar la inserción de los datos, el usuario debe presionar el botón "Registrar", como se muestra en la figura 12.8.

Figura 12.7 PantallaCrearRegUsuario (P-3) sin datos.



Figura 12.8 PantallaCrearRegUsuario (P-3) una vez que se han llenado los campos correspondientes y con el botón "Registrar" oprimido.

Se puede revisar en la *Base de Datos Registro* que los valores han sido creados e insertados correctamente en la base de datos.

A continuación, el sistema despliega la *PantallaObtenerRegUsuario (P-4)*, como se muestra en la figura 12.9. El usuario puede finalizar el caso de uso presionando el botón "Salir" en la *PantallaObtenerRegUsuario (P-4)*.



Figura 12.9 PantallaObtenerRegUsuario (P-4) con el botón "Salir" presionado.

Hidden page

Hidden page



Figura 12.13 PantallaObtenerRegUsuario (P-4) con el botón "Eliminar" presionado.

El sistema despliega la *PantallaCrearRegistroUsuario (P-3)*, lo cual da la posibilidad de crear un nuevo registro de usuario.

A su vez, éste puede finalizar el caso de uso presionando el botón "Salir" en la *PantallaCrearRegistroUsuario (P-3)*, como se muestra en la figura 12.14.

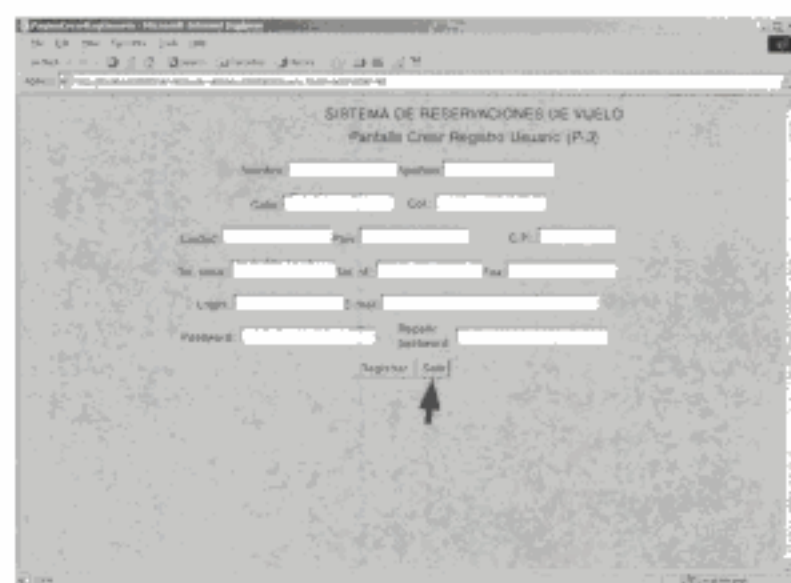


Figura 12.14 PantallaCrearRegUsuario (P-3) con el botón "Salir" presionado.

12.3.2 Registrar Tarjeta

Se prueban las secuencias más importantes del caso de uso *Registrar Tarjeta*: *Crear Registro Tarjeta*, *Actualizar Registro Tarjeta* y *Eliminar Registro Tarjeta*.

CREAR REGISTRO TARJETA

El caso de uso *Registrar Tarjeta* es una extensión del caso de uso *Registrar Usuario*. La secuencia *Crear Registro Tarjeta* depende de que no exista un registro anterior de tarjeta del usuario actual.

La secuencia comienza de manera similar a "Actualizar Registro Usuario". En lugar de hacer una actualización de registro de usuario se presiona el botón "Registrar Tarjeta" en la *PantallaObtenerRegUsuario (P-4)*, como se muestra en la figura 12.15.



Figura 12.15 *PantallaObtenerRegUsuario (P-4)* con el botón "Registrar Tarjeta" presionado.

El caso de uso continúa cuando el sistema presenta la *PantallaCrearRegTarjeta (P-5)*, la cual debe ser llenada con los datos de tarjeta de crédito solicitados. Nótese que en este subflujo aún no existe una tarjeta de registro del usuario. Éste deberá presionar el botón "Registrar", como se muestra en la figura 12.16.



Figura 12.16 Pantalla Crear Reg Tarjeta (P-5) con los datos de tarjeta de crédito solicitados y con el botón “Registrar” presionado.

Se puede revisar en la *Base de Datos Registro* que los valores han sido creados e insertados correctamente en la base de datos.

A continuación el sistema despliega la *Pantalla Obtener Reg Tarjeta (P-6)*, como se muestra en la figura 12.17.



Figura 12.17 Pantalla Obtener Reg Tarjeta (P-6) con el botón “Salir” presionado.

El usuario puede finalizar el caso de uso si presiona el botón "Salir" en la *PantallaObtenerRegTarjeta (P-6)*.

ACTUALIZAR REGISTRO TARJETA

La secuencia *Actualizar Registro Tarjeta* depende de que ya exista un registro anterior de tarjeta para el usuario actual.

La secuencia se inicia de manera similar a *Actualizar Registro Usuario*, donde en la *PantallaObtenerRegUsuario (P-4)* se presiona el botón "Registrar Tarjeta", como se mostró en la figura 12.15. El caso de uso continúa cuando el sistema presenta la *PantallaObtenerRegTarjeta (P-6)*, donde el usuario puede modificar los datos deseados de la tarjeta de crédito, tal como la fecha de vencimiento. Para ello, el usuario debe presionar el botón "Actualizar", como se muestra en la figura 12.18.



Figura 12.18 *PantallaObtenerRegTarjeta (P-6)* con los datos de registro de la tarjeta, tales como la fecha de vencimiento modificado y con el botón "Actualizar" presionado.

Se puede revisar en la *Base de Datos Registro* que los valores han sido actualizados correctamente en la base de datos.

El usuario puede finalizar el caso de uso presionando el botón "Salir" en la *PantallaObtenerRegTarjeta (P-6)*, como se mostró en la figura 12.17.

ELIMINAR REGISTRO TARJETA

La secuencia del flujo *Eliminar Registro Tarjeta* comienza de manera similar a *Actualizar Registro Tarjeta*, es decir, cuando se ejerce presión sobre el botón "Registrar Tarjeta" en la *PantallaObtenerRegUsuario (P-4)*, como se mostró en la figura 12.15. A continuación el sistema presenta la pantalla *PantallaObtenerRegTarjeta (P-6)*. Para eliminar el registro de tarjeta, el usuario debe presionar el botón "Eliminar", como se muestra en la figura 12.19.



Figura 12.19 PantallaObtenerRegTarjeta (P-6) con el botón "Eliminar" presionado.

El sistema presenta la *PantallaCrearRegTarjeta (P-5)*, lo cual da la posibilidad de crear un nuevo registro de tarjeta.

Se puede revisar en la *Base de Datos Registro* que los valores han sido eliminados correctamente de la base de datos.

El usuario puede finalizar el caso de uso presionando el botón "Salir", como se muestra en la figura 12.20.



Figura 12.20 PantallaCrearRegTarjeta (P-5) con el botón "Salir" presionado.

RESUMEN

En este capítulo se presenta una introducción al desarrollo de software para Internet mediante el empleo de la metodología de casos de uso. Se muestra cómo extender el sistema de reservaciones de vuelos desarrollado a partir de modificaciones en el modelo de diseño, de manera que se apoye tanto un sistema stand-alone como cliente-servidor con las mínimas modificaciones posibles. Se describe el modelo de implementación de los sistemas stand-alone y cliente-servidor. Se llevan a cabo las validaciones en el modelo de pruebas, similares a las realizadas en el capítulo 10, pero aplicadas en este capítulo al sistema cliente-servidor.

REFERENCIAS

1. Wirfs-Brock, R., Wilkerson, B., Wiener, L., 1990. Designing Object-Oriented Software, Prentice-Hall.

Hidden page

GLOSARIO

Siglas	Inglés	Español
ACM	Association for Computer Machinery	Asociación para Maquinaria de Computación
API	Application Program Interface	Interface de Programa de Aplicación
ASCII	American Standard Code for Information Interchange	Código Estándar Americano para el Intercambio de Información
ASP	Active Server Pages	Páginas de Servidor Activas
AWT	Abstract Window Toolkit	Herramienta de Ventanas Abstracta
BD	Database	Base de Datos
CASE	Computer Assisted Software Engineering	Ingeniería de Software Asistida por Computadora
CGI	Common Gateway Interface	Interface "Gateway" Común
CMM	Capability Maturity Model	Modelo de Madurez de Capabilidad
CMMI	Capability Maturity Model Integrated	Modelo de Madurez de Capabilidad Integrado
CORBA	Common Object Request Broker Architecture	Arquitectura de Intermediario de Solicitud de Objetos Comunes
CPSR	Computer Professionals for Social Responsibility	Profesionales de la Computación para la Responsabilidad Social
CRC	Class-Responsibility-Collaboration	Clase-Responsabilidad-Colaboración
DB	Database	Base de Datos
DBMS	Database Management System	Sistema de Administración de Base de Datos
EJB	Enterprise Java Beans	"Beans" de Java Empresariales
ERP	Enterprise Resource Planning	Planeación de Recursos de la Empresa
FTP	File Transfer Protocol	Protocolo de Transferencia de Archivos
GUI	Graphic User Interface	Interfaz Gráfica de Usuario
HCI	Human-Computer Interface	Interfaz Humano-Computadora
HTML	HyperText Markup Language	Lenguaje de Marcas de Hipertexto
HTTP	HyperText Transfer Protocol	Protocolo de Transferencia de Hipertexto
IEEE	Institute of Electrical and Electronic Engineers	Instituto de Ingenieros Eléctricos y Electrónicos
IO	Input-Output	Entrada-Salida
IP	Internet Protocol	Protocolo de Internet
ISO	International Organization for Standardization	Organización Internacional para la Normalización
IU	User Interface	Interfaz de Usuario
J2EE	Java 2 Enterprise Edition	Edición Empresarial de Java 2
J2ME	Java 2 Micro Edition	Edición "Micro" de Java 2

Siglas	Inglés	Español
J2SE	Java 2 Standard Edition	Edición Estándar de Java 2
JDBC	Java Database Connector	Conector de Base de Datos de Java
JDK	Java Development Kit	Paquete de Desarrollo de Java
JFC	Java Foundation Classes	Clases de Fundación de Java
JIT	Just In Time	Justo A Tiempo
JMS	Java Message Service	Servicio de Mensajes de Java
JScript	Java Script	"Script" de Java
JSP	Java Server Pages	Páginas de Servidor de Java
JVM	Java Virtual Machine	Máquina Virtual de Java
KPA	Key Process Area	Area de Proceso Clave
MINT	Integrated Methodology	Metodología Integrada
MTBF	Mean Time Between Failures	Tiempo Promedio Entre Fallas
MTTR	Mean Time To Repair	Tiempo Promedio Para Reparar
MVC	Model-View-Controller	Modelo-Vista-Controlador
O/W	One Way	Sólo Ida
ODBC	Object Database Connector	Conector de Base de Datos de Objetos
OODBMS	Object Oriented Database Management System	Sistema de Administración de Base de Datos Orientada a Objetos
PEMM	Performance Engineering Maturity Model	Modelo de Madurez de Ingeniería de Rendimiento
PHP	Php Hypertext Preprocessor	Preprocesador de Hipertexto Php
PSP	Personal Software Process	Proceso de Software Personal
R/T	Round Trip	Viaje Redondo
RDD	Responsibility Driven Design	Diseño Dirigido por Responsabilidades
RUP	Rational Unified Process	Proceso Unificado de Rational
SDK	Standard Development Kit	Paquete de Desarrollo Estándar
SEI	Software Engineering Institute	Instituto de Ingeniería de Software
SPICE	Software Process Improvement and Capability Determination	Proceso de Software de Mejoría y Determinación de Capabilidad
SQL	Standard Query Language	Lenguaje de Consultas Estándar
TCP	Transmission Control Protocol	Protocolo de Control de Transmisión
TSP	Team Software Process	Proceso de Software en Equipo
UI	User Interface	Interfaz de Usuario
UIMS	User Interface Management System	Sistema de Administración de Interfaz de Usuario
UML	Unified Modeling Language	Lenguaje de Modelado Unificado
UP	Unified Process	Proceso Unificado
URL	Uniform Resource Locator	Localizador de Recursos Unificado
VBScript	Visual Basic Script	"Script" de Visual Basic
WWW	World Wide Web	Red Mundial
XML	Extended Markup Language	Lenguaje de Marcas Extendido
XP	Extreme Programming	Programación Extrema
Y2K	Year 2000	Año 2000

BIBLIOGRAFÍA

Experiencia de software

- Brooks, F., 1995, *The Mythical Man-Month*, edición de Aniversario, Addison-Wesley.
- Glass, R., 1998, *Software Runaways*, 1a. ed., Prentice-Hall.
- Goldberg A., Rubin. K, 1995, *Succeeding with Objects: Decision Framework for Project Management*, 1a. ed., Addison-Wesley.
- Yourdon, E., 1993, *Decline & Fall of the American Programmer*, 1a. ed., Yourdon Press.
- Yourdon, E., 1996, *Rise & Resurrection of the American Programmer*, 1a. ed., Yourdon Press.

Lenguajes de programación

- Ada (United States Department of Defense), 1983, *Reference Manual for the Ada Programming Language*, ANSI/MIL-std 1815-a.
- Algol (Group Algol de l'AFCET), 1975, *Manuel du Langage Algorithmique ALGOL 68*, Hermann, París.
- Birtwistle, G., Dahl, O.J., Myhrhaug, B., Nygaard, K., 1973, *SIMULA Begin*, Petrocelli Charter, Nueva York.
- Campione, Walrath, 1999, *The Java Tutorial: Object Oriented Programming for the Internet*, 2a. ed., Addison-Wesley.
- Cox, B.J., 1983, *Object Oriented Programming in C*, *UNIX Review*, pp. 67-70, octubre/noviembre.
- Dahl, O.J., Nygaard, K., 1966, *SIMULA - An ALGOL-Based Simulation Language*, *Communications of the ACM*, 9(9):671-678.
- Goldberg, A., Robson, D., 1983, *SMALLTALK-80, The Language and its Implementation*, Addison-Wesley, MA.
- Hall, M., Brown, L., 2003, *Core Servlets and JavaServer Pages*, vol. 1: *Core Technologies*, 2a. ed., Prentice-Hall.
- Ingalls, D.H.H., 1978, *The SMALLTALK-76 Programming System Design and Implementation*, In *Proc. of the 5th POPL*, Tucson, AZ, pp. 9-17.
- Kay, A., 1968, *FLEX: A Flexible Extendible Language*, TR 4-7, Comp. Science Dept., University of Utah, Salt Lake City.
- Kay, A., 1969, *The Reactive Engine*, PhD Thesis, University of Utah, Salt Lake City.
- Kay, A., Goldberg, A., 1976, *SMALLTALK-72, Instruction Manual*, Technical Report SSL-76-6, Xerox PARC, Palo Alto, CA.
- Keene, S.E., 1989, *Object-Oriented Programming in Commol Lisp, A Programmer's Guide to CLOS*, Addison-Wesley, MA.

- Kristensen, B.B., Madsen, O.L., Møller-Pedersen, B., Nygaard, K., 1987, The BETA Programming Language, In B. Shriver and P. Wegner, editores, *Research Directions in Object Oriented Programming*, pp. 7-48, MIT Press, MA.
- Liskov, B., Guttag, J., 1986, *Abstraction and Specification in Program Development*, MIT Press/McGraw-Hill.
- Meyer, B., 1987, Eiffel: Programming for Reusability and Extendibility, *ACM SIGPLAN Notices*, 22(2):85-94.
- Minsky, M., 1975, A Framework for Representing Knowledge, In P. Winston, editor, *The Psychology of Computer Vision*, pp. 211-281, McGraw-Hill, Nueva York.
- Moon, D., 1986, Object-Oriented Programming with Flavors, In *Proc. of the 1st OOPSLA*, Portland, OR, pp. 1-8.
- Naur, P., 1963, Revised Report on the Algorithmic Language ALGOL 60, *Communications of the ACM*, 6(1):1-17.
- Smith, D.C., Irby, C., Kimball, R., Verplank, B., Harslem, E., 1983, Designing the Star User Interface, In P. Degano and E. Sandwell, editors, *Integrated Interactive Computing Systems*, pp. 297-313, North-Holland, Amsterdam, 1983.
- Stroustrup, B., 1986, *The C++ Programming Language*, 1a. ed., Addison-Wesley, MA.
- Stroustrup, B., 1991, *The C++ Programming Language*, 2a. ed., Addison-Wesley, MA.
- Wirth, N., 1971, The Programming Language PASCAL, *Acta Informatica*, 1(1): 25-68.

Procesos y metodologías

- Beck, K., Cunningham, W., 1989, A Laboratory for Teaching Object-Oriented Thinking, *Proceedings OOPSLA '89*, pp 1-6, Nueva Orleans, Louisiana, octubre. ACM. Published as ACM SIGPLAN Notices, vol. 24, núm. 10.
- Booch, G., 1994, *Object-Oriented Analysis and Design with Application*, Benjamin/Cummings.
- Booch, G., Rumbaugh, J., Jacobson, I., 1998, *The Unified Modeling Language User Guide*, Addison-Wesley.
- Coad, P., Yourdon, E., 1991, *Object-Oriented Analysis*, 2a. ed., Yourdon Press/Prentice-Hall, NJ.
- Coleman, D., Arnold, P., Bodoff, S., Dollin, C., Gilchrist, H., Hayes, F., Jeremaes, P., 1994, *Object Oriented Development: The FUSION Method*, Prentice-Hall, NJ.
- de Champeaux, D., Lea, D., Faure, P., 1993, *Object-Oriented Systems Development*, Addison-Wesley, MA.
- DeMarco, T., 1979, *Structured Analysis and System Specification*, Prentice-Hall.
- Embley, D.W., Kurtz, B.D., Woodfield, S.N., 1992, *Object-Oriented Systems Analysis: A Model Driven Approach*, Yourdon Press/Prentice-Hall.
- Firesmith, D.G., 1993, *Object-Oriented Requirements Analysis and Design: A Software Engineering Approach*, Wiley.
- Henderson-Sellers, B., Edwards, J.M., 1994, *Book Two of Object Oriented Knowledge: The Working Object*, Prentice Hall, Sydney, Australia.
- Jacobson, C., Christensen, M., Jonsson, P., Overgaard, G., 1992, *Object-Oriented Software Engineering: A Use-Case Driven Approach*, Addison-Wesley.
- Jacobson, C., Booch, G., Rumbaugh, J., 1999, *The Unified Software Development Process*, Addison-Wesley.
- Martin, J., Odell, J.J., 1992, *Object-Oriented Analysis and Design*, Prentice-Hall.
- Page-Jones, M., 1980, *Practical Guide to Structured System Design*, Prentice-Hall.

Hidden page

- Fritchman, B.L., Gluck, R.L., Jagannathan, D., Thompson, J.P., Tolbert, D.K., 1990, SIM: Design and Implementation of a Semantic Database System, in A.F. Cardenas and D. McLeod, Editors, *Research Foundations in Object Oriented and Semantic Database*, Prentice-Hall, NJ.
- Hornick, M., Zdonik, S.B., 1987, A Shared, Segmented Memory System for an Object Oriented Database, *ACM Transactions on Office Information Systems*, 5(1), enero.
- Hudson, S., King, R., 1989, CACTIS: A Self Adaptive Concurrent Implementation of an Object Oriented Database System, *ACM Transactions on Database Systems*, 14(3), septiembre.
- Ingres, 1989, INGRES 6.3 Reference Manual, Ingres Corporation, Alameda, CA.
- Innovative Systems, 1988, VISION: A Technical Overview, Innovative Systems.
- Itasca Systems, Inc, 1990, ITASCA System Overview, Unisys, Minneapolis, Minnesota.
- Kachler, T., Krasner, G., 1983, LOOM: Large Object Oriented Memory for Smalltalk-80 Systems, in G. Krsner Editor, *Smalltalk-80: Gits of History, Words of Advice*, Addison-Wesley, MA.
- Kim, W., *et al.*, 1988, Features of the ORION Object Oriented Database System, in W. Kim and F.H. Lochovsky, Editors, *Object Oriented Concepts, Databases, and Applications*, Addison Wesley, MA.
- Moss, J.E.B., Sinofsky, S., 1988, Managing Persistent Data with Mnome: Designing a Reliable Shared Object Interface, in K.R. Dittrich, Editor, *Advances in Object Oriented Database Systems*, 2nd Intl Workshop on Object Oriented Database Systems, Bad Münster, Germany, septiembre, Springer Verlag, Nueva York.
- Object Databases, 1990, Gbase Reference Manual, Object Databases Corp., Cambridge, MA.
- Object Design, 1990, An Introduction to ObjectStore, Object Design Inc., Burlington, MA.
- Objectivity, 1990, Objectivity Database System Overview, Objectivity Inc., Menlo Park, CA.
- Ontologic, 1986, Vbase Functional Specification, Ontologic, Inc., Billerica MA.
- Ontos, 1989, ONTOS Reference Manual, Ontologic Inc., Billerica MA.
- Schaffert, C., O'Brien, P., Bullis, B., 1986, Persistent and Shared Objects in Trellis/Owl, in K.R. Dittrich and U. Dayal, Editors, *Proc. of the Intl. Workshop on Object Oriented Database Systems*, Asilomar, CA.
- Schwartz, P., *et al.*, 1986, Extensibility in the Starbust Database System, in K.R. Dittrich and U. Dayal, editors, *Proc. of the Intl. Workshop on Object Oriented Database Systems*, Asilomar, CA.
- Smith, J.M., *et al.*, 1983, ADAPLEX Rationale and Reference Manual, TR CCA-83-8, Computer Corp. of America, Cambridge, MA, mayo.
- Spector, A.Z., Bloch, J., Daniels, D., Draves, R., Duchamp, D., Eppinger, J., Menees, S., Thompson, D., 1986, The Camelot Project, TR CMU-CS-86-166, Computer Science, Carnegie-Mellon University, Pittsburg, Pennsylvania.
- Stonebraker, M., Rowe, L.A., 1986, The Design of POSTGRES, in *Proc. of the ACM SIGMOD Conf*, Washington D.C.
- Sybase, 1989, SYBASE Reference Manual, Sybase Corp., Emeryville, CA.
- Versant, 1990, VERSANT Technical Overview, Versant Object Technologies, Inc., Menlo Park, CA.
- Weinreb, D., 1988, An Object Oriented Database System to Support an Integrated Programming Environment, *IEEE Data Engineering*, 11(2), junio.

ÍNDICE

A

abstracción, [25](#)
accesos, [81](#), [96](#)
ACTION, [606](#)
actividad (es), [38](#), [276](#)
 análisis, [38](#)
 diseño, [38](#)
 documentación, [39](#)
 implementación, [38](#)
 integración, [38](#)
 mantenimiento, [39](#)
 pruebas, [39](#)
 requisitos, [38](#)
actor (es), [199](#), [200](#), [210](#)
 concretos, [202](#)
 primarios, [201](#)
 secundarios, [201](#)
Ada, [31](#)
agregación, [101](#), [109](#)
Airborne Self-Protection Jammer, [12](#)
alcance, [152](#)
 de pruebas, [582](#)
ALGOL, [30](#)
algoritmos, [339](#)
Allstate Insurance, [11](#)
análisis, [195](#)
ancestros, [107](#)
API, [30](#)
aplicaciones, [180](#), [181](#)
applet (s), [132](#), [180](#), [181](#)
archivos, [145](#)
argumentos, [79](#)
arquitectura, [36](#), [254](#), [336](#)
 von Neumann, [22](#)
arreglo (s), [139](#), [140](#)
asignación, [142](#)
asociación (es), [159](#), [162](#)
 como clases, [100](#)
 derivadas, [94](#)
 reflexivas, [87](#), [162](#)
 ternarias, [99](#)
ASP, [602](#)
AT&T, [6](#)

atributos, [74](#), [157](#), [170](#), [178](#), [339](#), [340](#)
 básicos, [78](#)
 de liga y asociación, [96](#)
 derivados, [78](#)
automatización de pruebas, [582](#)
avión de carga c-17, [12](#)
AWT, [524](#)

B

bala de plata, [16](#)
Bank of America (1988), [11](#)
Bank of New York, [5](#)
bases de datos, [41](#), [145](#), [503](#)
BASIC, [30](#)
beta, [32](#)
bibliotecas, [131](#), [133](#)
bombardero B-1, [11](#)
borde, [197](#), [255](#), [258](#)
bytecode, [132](#)

C

C, [31](#)
C#, [32](#)
C++, [32](#)
cadenas, [141](#)
calidad, [15](#)
 de software, [56](#)
cancelaciones en los sistemas de software, [11](#)
cardinalidad, [88](#)
CASE, [49](#)
casos de uso, [195](#), [196](#), [202](#), [211](#)
 abstractos, [206](#)
 concretos, [206](#)
CGI, [602](#)
ciclo de vida, [18](#), [35](#), [43](#)
Cisco, [9](#)
clase-responsabilidad-colaboración (CRC),
 [339](#)
clases, [72](#), [255](#)
 abstractas, [116](#), [173](#)
 concretas, [116](#)

- contenedoras, [163](#)
- disjuntas, [120](#)
- no disjuntas, [120](#)
- clasificación, [28](#)
- cliente, [402](#), [601](#)
- cliente-servidor, [639](#), [650](#)
- CLU, [31](#)
- CMM, [57](#)
- COBOL, [30](#)
- código nativo, [148](#)
- cohetes Ariane, [7](#)
- colaboraciones, [339](#), [340](#), [402](#)
- ColdFusion, [602](#)
- comentarios, [621](#)
- compilación, [131](#), [132](#)
- complejidad del software, [13](#)
- comportamiento, [196](#), [254](#)
- composición, [101](#), [109](#), [159](#), [163](#)
- confiabilidad, [14](#)
- constantes, [139](#)
- constructores, [155](#)
- consultas, [81](#)
- continuar, [619](#)
- contratos, [339](#), [340](#)
- control, [144](#), [254](#), [256](#), [268](#)
- costo y complejidad, del software, [3](#)

D

- declaraciones, [138](#)
- delimitación del sistema, [200](#)
- descendientes, [107](#)
- descripción del problema, [197](#)
- diagrama (s)
 - de instancias, [71](#)
 - de secuencias, [275](#), [515](#)
 - eventos, [275](#)
 - interacción, [275](#)
- diccionario, [162](#), [340](#)
- de clases, [235](#), [247](#), [327](#)
- dimensión, [254](#)
- discriminador, [115](#)
- diseño, [195](#), [333](#)
 - de objetos, [338](#)
 - por responsabilidades (RDD), [339](#)
- DMV, [11](#)
- documentación, [195](#)
- dominio del problema, [197](#)
- Dylan, [32](#)

E

- eBay, [9](#)
- Eiffel, [32](#)
- ejecución, [132](#)
- encapsulación, [27](#)

- encapsulamiento, [153](#)
- ensamblados, [101](#)
- entidad, [255](#)
- error, [577](#)
 - del milenio, [10](#)
- escenarios, [202](#)
- estación nuclear Bruce, [6](#)
- estereotipo (s), [255](#), [258](#), [340](#)
- estrategia, [46](#)
- estructuras estáticas, [178](#)
- etiquetas, [603](#)
- eventos, [276](#)
- excepciones, [147](#)
- expresiones, [158](#)
- extensibilidad, [26](#), [336](#), [338](#)
- extensión, [111](#), [205](#), [207](#)
 - de clase, [112](#)

F

- F-18, [5](#)
- falla (s), [577](#)
 - en sistemas de software, [4](#)
- falta, [577](#)
- finalizador, [148](#)
- flash, [602](#)
- flujos principales, [205](#), [277](#)
- formas, [606](#)
- FORTAN, [29](#)
- fracaso del Mariner [1](#), [4](#)

G

- ganar-ganar, [54](#)
- generalización, [28](#), [205](#), [206](#)
 - y herencia, [106](#)
- GET, [612](#)
- glosario, [235](#)
- grado de la asociación, [85](#)

H

- Haskell, [32](#)
- herencia múltiple, [119](#), [176](#)
- herramientas, [49](#)
- HREF=, [604](#)
- HTML, [601](#), [602](#), [630](#)

I

- identidad, [71](#)
- identificación
 - de asociaciones, [242](#)

- de atributos, [246](#)
- de clases, [235](#)
- de módulos, [249](#)
- identificadores, [77](#), [135](#)
- implementación, [85](#), [195](#), [523](#)
- importación, [136](#)
- incluir, [618](#)
- inclusión, [205](#), [206](#), [207](#)
- información, [197](#), [254](#)
- inicializador, [179](#)
- input, [606](#)
- instanciación, [73](#)
- interfaces, [174](#), [197](#)
 - gráficas del usuario, [182](#)
- Internet, [629](#)
- ISO, [59](#)

J

- Java, [32](#), [129](#), [148](#)
- JavaBeans, [621](#), [630](#)
- Java development kit, [132](#)
- JavaScript, [602](#)
- Java server pages (JSP), [601](#), [602](#), [614](#), [630](#)
- Java virtual machine, [130](#)
- jerarquías, [339](#), [412](#)
- just-in-time compiler, [130](#)

L

- legacy, [36](#)
- lenguajes de programación, [41](#)
- ley
 - de Brooks, [17](#)
 - de Lehman, [13](#)
- ligado dinámico, [129](#)
- ligas, [159](#)
 - y asociación, [83](#)
- LISP, [29](#)
- lista, [163](#), [164](#)
- London Stock Exchange, [11](#)

M

- MCI, [8](#)
- metaclasses, [180](#)
- metadato, [146](#)
- METHOD, [612](#)
- metodologías, [42](#)
 - estructuradas, [44](#)
- métodos, [42](#), [81](#), [157](#), [170](#), [179](#)
 - abstractos, [173](#)
- misiles Patriot, [6](#)
- misión crítica, [4](#)

- ML, [32](#)
- modelo, [254](#)
 - de análisis, [39](#)
 - de cascada, [50](#)
 - de casos de uso, [199](#)
 - de diseño, [40](#)
 - de espiral, [53](#)
 - de implementación, [40](#)
 - de integración, [41](#)
 - de pruebas, [41](#)
 - de requisitos, [39](#)
 - del dominio del problema, [235](#)
 - evolucionario, [52](#)
 - incremental, [51](#)
- modula, [31](#)
- modularidad, [25](#)
- módulos, [121](#), [327](#)
- multiplicidad, [88](#), [161](#), [162](#), [244](#)
- MVC, [254](#)

N

- Nasdaq, [10](#)
- nodo, [164](#)

O

- objective-C, [31](#)
- objectory, [195](#), [196](#)
- objetos, [69](#)
- operaciones, [70](#), [150](#)
 - de asociación, [99](#)
- operadores, [142](#)
- orden de pruebas, [582](#)
- orientadas a objetos, [44](#), [129](#)

P

- paneles, [184](#), [186](#)
- paquete (s), [121](#), [131](#), [133](#), [136](#), [154](#)
- parametrización, [82](#)
- pascal, [30](#)
- PEMM, [60](#)
- pentium de Intel, [7](#)
- PHP, [602](#)
- pila, [163](#)
- PL/I, [30](#)
- polimorfismo, [28](#), [82](#), [170](#), [412](#)
- post, [612](#)
- presentación, [197](#), [254](#)
- proceso
 - de software, [35](#)
 - unificado, [56](#)

- programación
 - en Java, 523
 - estructurada, 21
 - extrema, 55
 - orientada a objetos, 22
 - Prolog, 31
 - propagación de operaciones, 105
 - protocolos, 339, 470
 - prototipo, 47
 - prueba (s), 195, 577
 - basada en estado, 580
 - basada en requisitos o de casos de uso, 579
 - de aceptación o de validación, 579
 - de documentación de usuario, 579
 - de escala completa, 578
 - de especificación, 580
 - de integración, 579
 - de operación, 578
 - de regresión, 578
 - de rendimiento, 578
 - de sistema, 580
 - de sobrecarga, 579
 - de unidad, 579
 - ergonómicas, 579
 - estructural, 581
 - negativa, 579
 - PSP, 63
 - puntos de función, 14
- R**
- RAD, 52
 - rendimiento, 333
 - requisitos, 195
 - responsabilidades, 339, 340, 341
 - restricción (es), 111
 - de atributos, 78
 - de clase, 112
 - de ligas y asociaciones, 93
 - retrasos en los sistemas de software, 11
 - reuso, 336, 338
 - reutilización, 26, 48
 - riqueza funcional, 15
 - robustez, 14, 336
 - rol, 91, 159
 - RUP, 195
- S**
- self, 32
 - serialización, 148
 - servidor, 402, 601
 - Servlets, 601, 602, 608, 630
 - sesiones, 608
 - SIMULA, 30
 - sincronización, 148
 - sistema (s)
 - de facturación para PG&E, 12
 - de información, 254
 - de reservaciones CONFIRM, 12
 - Smalltalk, 31
 - sobrecarga, 151
 - sobrecostos en los sistemas de software, 11
 - sobreescritura, 170
 - de operaciones, 113
 - software suficientemente bueno, 15
 - SQL, 558
 - stand-alone, 648
 - subflujos, 204, 205, 277
 - subsistemas, 339, 457
 - superclase, 169
- T**
- tarjetas de clases, 339
 - telescopio espacial Hubble, 6
 - Therac-25, 5
 - Tickit, 61
 - tiempo/costo, 15
 - tipos
 - no-primitivos, 138
 - primitivos, 137
 - Toshiba, 9
 - TSP, 64
- U**
- UML, 69, 148, 256
 - USS Vincennes, 5
- V**
- validación, 41
 - variables, 138
 - VBScript, 602
 - verificación, 41
 - vista, 254

Hidden page

Hidden page

Hidden page

INGENIERÍA DE SOFTWARE

ORIENTADA
A OBJETOS
CON UML, JAVA
E INTERNET

Este libro es el resultado de casi una década de investigación y docencia a nivel licenciatura, maestría y cursos especiales, en el área de la tecnología orientada a objetos. Su enfoque integral de la teoría y la práctica facilita la comprensión y el desarrollo de sistemas de calidad mediante la creación de una arquitectura de software y el seguimiento de una metodología bien definida. Esta valiosa característica

se refuerza con el uso de un mismo sistema de software en todo el libro, en lugar de pequeños ejemplos aislados.

Está dirigido tanto a estudiantes universitarios como a profesionistas en el área del desarrollo de software orientado a objetos. Como libro de texto, su objetivo es satisfacer las necesidades de los cursos de licenciatura o maestría de introducción a la ingeniería de software con tecnología orientada a objetos, semestrales o trimestrales.

El título particular de este libro, *Ingeniería de Software Orientada a Objetos con UML, Java e Internet*, refleja el interés de resaltar ciertos aspectos que en la actualidad tienen un significado muy especial. El Lenguaje de Modelado Unificado (UML - Unified Modeling Language) es el estándar más importante a nivel mundial para el modelado de sistemas orientados a objetos. El lenguaje de programación de Java es uno de los más utilizados en el desarrollo de aplicaciones orientadas a objetos. Por último, Internet es en la actualidad uno de los ambientes más importantes para la creación de software por su fácil acceso desde cualquier lugar del planeta.

THOMSON

MÉXICO Y AMÉRICA CENTRAL
Tel. 52(55)1500-6000
Fax 52(55)5281-2656
editor@thomsonlearning.com.mx
México, D.F., MÉXICO

AMÉRICA DEL SUR
thomson@thomsonlearning.com.ar
Buenos Aires, ARGENTINA

EL CARIBE
Tel. (787) 641-1112
Fax (787) 641-1119
San Juan, PUERTO RICO

PACTO ANDINO
Tel. (571) 340-9470
Fax (571) 340-9475
cliente@thomsonlearning.com.co
Bogotá, COLOMBIA

ESPAÑA
Tel. (3491) 445-3350
Fax (3491) 445-6218
clientes@paraninfo.es
Madrid, España

ISSN 170-686-190-4



9 789706 861900